# Automate Your Business with AI

A No-Jargon Guide to the Future of Work

Andrea Bruno

# Automate Your Business with AI

*A No-Jargon Guide to the Future of Work*

Andrea Bruno

Graphene Lab
2026

A Note About the Tool Behind This Book 24
Get AgentBridge 24
Want to automate your own business? 25
Chapter 1 — A Short History of AI: From Philosophy to Business 27
In Simple Words 27
A Bit of History 28
Curiosity 36
A Real Business Example 37
How to Do It 38
Ethics and Responsibility 40
Mistakes to Avoid 41
Practical Exercise 41
Checklist 43
Key Takeaways 44
Chapter 2 — AI Explained Simply (No Jargon) 45
In Simple Words 45
A Bit of History 51
Curiosity 52
A Real Business Example 54
How to Do It 56
Ethics and Responsibility 58
Mistakes to Avoid 58
Practical Exercise 61
Checklist 62
Key Takeaways 63
Chapter 3 — The Words of AI, Without the Big Words 64
In Simple Words 64
A Bit of History 71
Curiosity 72

A Real Business Example 75
How to Do It 76
Ethics and Responsibility 80
Mistakes to Avoid 81
Practical Exercise 82
Checklist 84
Key Takeaways 85
Chapter 4 — Ethical AI: Doing the Right Thing 86
In Simple Words 86
A Bit of History 87
Curiosity 88
A Real Business Example 89
How to Do It 90
Ethics and Responsibility 92
Mistakes to Avoid 97
Practical Exercise 98
Checklist 99
Key Takeaways 100
Chapter 5 — Rules and Legal Responsibility: The EU AI Act 102
In Simple Words 102
A Bit of History 104
Curiosity 105
A Real Business Example 106
How to Do It 107
Ethics and Responsibility 114
Mistakes to Avoid 114
Practical Exercise 115
Checklist 117
Key Takeaways 118

Chapter 6 — Cybersecurity in the AI Era 119
In Simple Words 119
A Bit of History 121
Curiosity 122
A Real Business Example 124
How to Do It 125
Ethics and Responsibility 127
Mistakes to Avoid 128
Practical Exercise 129
Checklist 130
Key Takeaways 131
Chapter 7 — Trustless: Trust Without Trusting 133
In Simple Words 133
A Bit of History 134
Curiosity 136
A Real Business Example 138
How to Do It 139
Ethics and Responsibility 142
Mistakes to Avoid 143
Practical Exercise 144
Checklist 145
Key Takeaways 146
Chapter 8 — Self-Hosting: Keep Your Data Under Control 147
In Simple Words 147
A Bit of History 148
Curiosity 150
A Real Business Example 151
How to Do It 153
Ethics and Responsibility 156

Mistakes to Avoid 157
Practical Exercise 158
Checklist 159
Key Takeaways 160
Chapter 9 — Third-Party Services and Shadow AI 161
In Simple Words 161
A Bit of History 162
Curiosity 163
A Real Business Example 164
How to Do It 165
Ethics and Responsibility 171
Mistakes to Avoid 172
Practical Exercise 172
Checklist 173
Key Takeaways 174
Chapter 10 — Privacy and GDPR: What You Really Need to Know 176
In Simple Words 176
A Bit of History 177
Curiosity 178
A Real Business Example 179
How to Do It 180
Ethics and Responsibility 188
Mistakes to Avoid 189
Practical Exercise 189
Checklist 191
Key Takeaways 192
Chapter 11 — Digital Sovereignty: The Right to Control Your AI 193
In Simple Words 193
A Bit of History 194

Curiosity 195
A Real Business Example 197
How to Do It 198
Ethics and Responsibility 202
Mistakes to Avoid 203
Practical Exercise 205
Checklist 206
Key Takeaways 207
Chapter 12 — Where AI Can Help Your Business 208
In Simple Words 208
A Bit of History 209
Curiosity 210
A Real Business Example 211
How to Do It 211
Ethics and Responsibility 217
Mistakes to Avoid 218
Practical Exercise 218
Checklist 220
Key Takeaways 221
Try it with AgentBridge 221
Scheduled Tasks: Work That Runs by Itself 232
Key Takeaways 236
Your Assistant by Voice and by Phone 237
Key Takeaways 240
Working on the Go with Telegram 242
Key Takeaways 244
Chapter 13 — Defining a Simple AI Strategy 246
In Simple Words 246
A Bit of History 247

Curiosity 248
A Real Business Example 248
How to Do It 250
Ethics and Responsibility 255
Mistakes to Avoid 256
Practical Exercise 256
Checklist 257
Key Takeaways 258
Chapter 14 — Data: The Raw Material 259
In Simple Words 259
A Bit of History 260
Curiosity 261
A Real Business Example 261
How to Do It 262
Ethics and Responsibility 268
Mistakes to Avoid 268
Practical Exercise 269
Checklist 270
Key Takeaways 271
Try it with AgentBridge 272
Chapter 15 — People, Roles, and Culture 277
In Simple Words 277
A Bit of History 278
Curiosity 279
A Real Business Example 280
How to Do It 281
Ethics and Responsibility 287
Mistakes to Avoid 288
Practical Exercise 290

Checklist 291
Key Takeaways 292
Chapter 16 — Goals, Costs, and Return on Investment 293
In Simple Words 293
A Bit of History 294
Curiosity 295
A Real Business Example 296
How to Do It 296
Ethics and Responsibility 302
Mistakes to Avoid 303
Practical Exercise 304
Checklist 306
Key Takeaways 307
Chapter 17 — Choosing Tools Without Being Fooled 308
In Simple Words 308
A Bit of History 309
Curiosity 310
A Real Business Example 311
How to Do It 312
Ethics and Responsibility 317
Mistakes to Avoid 318
Practical Exercise 319
Checklist 320
Key Takeaways 321
Try it with AgentBridge 322
Meet AgentBridge: Your Office Assistant 323
Key Takeaways 327
Chapter 18 — Your First Pilot Project 329
In Simple Words 329

A Bit of History 330
Curiosity 331
A Real Business Example 331
How to Do It 332
Ethics and Responsibility 337
Mistakes to Avoid 338
Practical Exercise 339
Checklist 340
Key Takeaways 341
Chapter 19 — Connecting AI to Systems You Already Use 342
In Simple Words 342
A Bit of History 343
Curiosity 344
A Real Business Example 345
How to Do It 346
Ethics and Responsibility 350
Mistakes to Avoid 351
Practical Exercise 352
Checklist 353
Key Takeaways 354
Try it with AgentBridge 355
Chapter 20 — Implementing AI Securely 360
In Simple Words 360
A Bit of History 361
Curiosity 362
A Real Business Example 363
How to Do It 364
Ethics and Responsibility 369
Mistakes to Avoid 369

Practical Exercise 370
Checklist 372
Key Takeaways 373
Chapter 21 — Managing Change in Your Company 374
In Simple Words 374
A Bit of History 375
Curiosity 377
A Real Business Example 378
How to Do It 379
Ethics and Responsibility 385
Mistakes to Avoid 386
Practical Exercise 387
Checklist 389
Key Takeaways 390
Chapter 22 — Measuring Results and ROI 391
In Simple Words 391
A Bit of History 392
Curiosity 393
A Real Business Example 394
How to Do It 395
Ethics and Responsibility 401
Mistakes to Avoid 402
Practical Exercise 403
Checklist 404
Key Takeaways 405
Try it with AgentBridge 406
Chapter 23 — Scaling and Improving 412
In Simple Words 412
A Bit of History 413

Curiosity 414
A Real Business Example 415
How to Do It 417
Ethics and Responsibility 422
Mistakes to Avoid 424
Practical Exercise 425
Checklist 426
Key Takeaways 428
Chapter 24 — Common Mistakes and How to Avoid Them 429
In Simple Words 429
A Bit of History 430
Curiosity 431
A Real Business Example 432
How to Do It 433
Ethics and Responsibility 439
Mistakes to Avoid 440
Practical Exercise 441
Checklist 442
Key Takeaways 443
Chapter 25 — Administration and Finance 445
In Simple Words 445
A Bit of History 446
Curiosity 447
A Real Business Example 448
How to Do It 449
Ethics and Responsibility 453
Mistakes to Avoid 454
Practical Exercise 456
Checklist 457

Key Takeaways 458
Try it with AgentBridge 459
Chapter 26 — Sales and Marketing 464
In Simple Words 464
A Bit of History 465
Curiosity 466
A Real Business Example 466
How to Do It 468
Ethics and Responsibility 472
Mistakes to Avoid 473
Practical Exercise 474
Checklist 475
Key Takeaways 476
Try it with AgentBridge 477
Chapter 27 — Customer Care and Support 487
In Simple Words 487
A Bit of History 488
Curiosity 489
A Real Business Example 490
How to Do It 491
Ethics and Responsibility 495
Mistakes to Avoid 496
Practical Exercise 498
Checklist 499
Key Takeaways 500
Try it with AgentBridge 500
Chapter 28 — HR and People 504
In Simple Words 504
A Bit of History 505

Curiosity 506
A Real Business Example 507
How to Do It 509
Ethics and Responsibility 513
Mistakes to Avoid 514
Practical Exercise 516
Checklist 517
Key Takeaways 518
Try it with AgentBridge 519
Chapter 29 — Operations and Production 525
In Simple Words 525
A Bit of History 526
Curiosity 527
A Real Business Example 528
How to Do It 529
Ethics and Responsibility 534
Mistakes to Avoid 535
Practical Exercise 536
Checklist 537
Key Takeaways 538
Try it with AgentBridge 539
Chapter 30 — IT and Leadership 543
In Simple Words 543
A Bit of History 544
Curiosity 545
A Real Business Example 546
How to Do It 547
Ethics and Responsibility 551
Mistakes to Avoid 552

Practical Exercise 554
Checklist 555
Key Takeaways 556
Try it with AgentBridge 557
Chapter 31 — A Small Manufacturing Business 563
Context 563
The Problem 564
The Solution 565
The Tools 567
The Costs 568
The Results 569
Lessons Learned 570
Try it with AgentBridge 571
Chapter 32 — A Professional Practice 574
Context 574
The Problem 575
The Solution 576
The Tools 577
The Costs 578
The Results 579
Lessons Learned 580
Try it with AgentBridge 581
Chapter 33 — A Retail Store 586
Context 586
The Problem 587
The Solution 588
The Tools 589
The Costs 590
The Results 590

Lessons Learned 591
Try it with AgentBridge 593
Chapter 34 — Services and Consulting 594
Context 594
The Problem 595
The Solution 596
The Tools 597
The Costs 598
The Results 599
Lessons Learned 600
Try it with AgentBridge 602
Chapter 35 — Healthcare, Education, and Public Bodies 604
Context 604
The Problem 605
The Solution 606
The Tools 607
The Costs 609
The Results 610
Lessons Learned 611
Chapter 36 — Startups and E-commerce 613
Context 613
The Problem 614
The Solution 615
The Tools 616
The Costs 617
The Results 618
Lessons Learned 619
Chapter 37 — The Changing Nature of Work 621
In Simple Words 621

A Bit of History 622
Curiosity 623
A Real Business Example 624
How to Do It 625
Ethics and Responsibility 630
Mistakes to Avoid 631
Practical Exercise 631
Checklist 632
Key Takeaways 633
Chapter 38 — A 90-Day Action Plan 634
In Simple Words 634
A Bit of History 635
Curiosity 636
A Real Business Example 636
How to Do It 637
Ethics and Responsibility 641
Mistakes to Avoid 641
Practical Exercise 642
Checklist 643
Key Takeaways 644
Chapter 39 — Frequently Asked Questions 645
Does it cost too much? 645
Do I need a technician? 646
Will I lose my data? 646
Will AI make me fire someone? 647
Where do I start? 647
How long does it take? 648
Which tools should I choose? 648
How do I choose a vendor? 649

Is it safe? 650
What does the law say? 650
Can I use AI without giving my data to anyone? 651
Is self-hosting really safer? 651
What does “trustless” mean, and do I really need it? 652
Appendix A — Simple Glossary 654
Appendix B — Checklist to Evaluate an AI Vendor 659
1. What the tool actually does 659
2. Data and privacy 659
3. Security 660
4. Compliance (GDPR and EU AI Act) 660
5. Pricing 660
6. Support and service 661
7. Contract and exit 661
8. Trust and reputation 661
9. Trial and proof 661
Quick scorecard 662
Appendix C — Template to Map Your Processes 663
How to use it 663
Blank template 663
Filled example 664
Column guide 666
Tips for good mapping 666
High-potential processes to look for 666
Quick priority grid 667
How to estimate time honestly 667
Using the map 667
Appendix D — Data Inventory 669
How to use it 669

Blank template 669
Filled example 670
Where to look for data 671
What counts as “Sensitive” 672
Column guide 672
Red flags to fix now 673
Retention rules of thumb 673
Keeping the inventory current 673
Rule for AI projects 674
Appendix E — Simple ROI Calculation 675
The formula in plain words 675
What counts as Benefit 675
What counts as Cost 676
Worked example 676
Payback period 676
Second example (quick) 677
Non-financial benefits 677
Blank worksheet 678
Honest tips 679
Appendix F — Internal Communication Plan 680
Why it matters 680
The plan table 680
Column guide 681
Filled example 681
Suggested timing for a change 683
Example messages 683
Rules for good messages 684
What not to say 684
Quick fill-in starter 684

Appendix G — A Simple AI Policy for Your Company 686
1. Purpose 686
2. Allowed uses 686
3. Forbidden uses 687
4. Data rules 687
5. Disclosure 687
6. Human review 688
7. Third-party and shadow AI 688
8. If something goes wrong 688
9. Roles and responsibilities 688
10. Exceptions 689
11. Who to ask 689
12. Acknowledgement 689
Appendix H — AI Risk Assessment 690
How to score 690
Blank template 690
Filled examples 691
Column guide 693
Common AI risks to consider 693
When to re-run this assessment 693
Two ways to reduce a risk 694
Rules 694
Appendix I — Pilot Project Template 695
How to use it 695
Pilot plan 695
Example snapshot (a finished pilot) 699
What to do after the pilot 700
Pilot rules 700
Appendix J — KPI Dashboard 701

How to use this 701
Blank dashboard 702
Common automation KPIs 702
Example filled dashboard 703
Reading the dashboard 704
Rules for honest KPIs 705
Quick monthly review sheet 705
Appendix K — Questions to Ask the Vendor 707
Data & privacy 707
Security 708
Pricing 708
Integration 709
Support 709
Contract & exit 710
Compliance 710
How to read the answers 711
Appendix L — Exercises and Fillable Modules 712
Exercise 1 — List your automatable tasks 712
Exercise 2 — Map one process 713
Exercise 3 — Compute a simple ROI 713
Exercise 4 — Draft one acceptable-use rule 714
Exercise 5 — Assess one risk 715
Exercise 6 — Spot the human checkpoint 716
Exercise 7 — Define what “success” looks like 716
Exercise 8 — One-page summary 717
Appendix M — Resources, Reading, and Recommended Tools 718
Run AI locally (on your own machines) 718
Build automations (connect AI to your work) 719
Learn more (reading) 720

How to choose, in three questions 720
A caution on lists like this 721
Appendix N — Downloadable Modules and Templates 722
Understand the basics 722
Choose a vendor 723
Plan a project 724
Govern and protect 725
Communicate inside your firm 725
A simple order to work in 725
Tips for using the templates 726
Appendix O — Quick AI Security Guide for SMBs 727
1. Accounts & access 727
2. Data handling 727
3. Vendor checks 728
4. Staff & shadow AI 728
5. Monitoring 729
6. Incident basics (when something goes wrong) 729
The 10 items that matter most 730
Golden rules 730
Appendix P — AI Vendor Due-Diligence Scorecard 731
How to score 731
Blank scorecard 731
What each criterion means 732
Filled example 733
How to read the total 735
The override rule 735
Comparing two vendors side by side 735
Tips 736
Appendix Q — Acceptable Use Policy Template 737

# A Note About the Tool Behind This Book

This book explains how a small business can use Artificial Intelligence (AI) in simple words. It talks about documents, spreadsheets, reports, email, research, and tasks that run on their own.

Every one of these things can be done today with one tool: **AgentBridge**.

AgentBridge is an AI office assistant made by Graphene Lab. You talk to it in plain words, and it does the work. It writes real documents, builds spreadsheets and charts, sends and reads email, researches the web, and makes presentations. It can also run a task on a time you agree on, answer by voice or phone, and work inside Telegram. It runs on your own computer, so your files stay with you.

Throughout this book you will see pictures of real results made with AgentBridge, and the one line of text a person typed to get them. You do not need to install anything to read this book. But if you want to try the tool yourself, start here.

## Get AgentBridge

The full source code is free and open on GitHub:

**https://github.com/Graphene-Lab/AgentBridge/**

Scan this code with your phone to open it:

AgentBridge on GitHub

## Want to automate your own business?

If you run a business and want help putting these ideas to work, talk to us. We can look at your day-to-day work and show you what AgentBridge can do for you.

- Web: **https://graphenelab.it/**
- Telegram: **https://t.me/yd8j9**

Scan this code to reach us on Telegram:

Contact us on Telegram

The examples in this book are made to show that automating your business is simple, and that you do not need to be a technical person to start.

— *Graphene Lab*

# Chapter 1 — A Short History of AI: From Philosophy to Business

## In Simple Words

People talk about artificial intelligence as if it appeared last Tuesday. It did not. There are about seventy years of serious research behind it, and much longer than that of dreaming about it.

Why should a bakery owner or a logistics manager care? Three reasons. **Hype is not new.** Every generation of AI research has promised more than it delivered, so knowing the pattern makes you harder to fool. **The failures are useful.** Each time AI stalled, it stalled for a nameable reason: not enough data, not enough computing power, or a problem harder than it looked. Those same three reasons still decide whether your project works. **The wins were narrow.** AI did not become smart all at once. It became very good at one thing, then another: chess in 1997, quiz shows in 2011, written language in 2022. That is exactly how you should plan your own use of it.

One word before we start. **Artificial intelligence (AI)** means software that makes useful decisions or produces useful results without a human writing a rule for every single case. That is the whole idea. Everything in this book unpacks that one sentence.

Each stop below has one business lesson attached. Read it once for the story, then come back to the checklist in section 1.11 when someone is trying to sell you something.

# A Bit of History

## 1.1 First dreams: from calculators to automata

Long before anyone said “AI”, people built machines that imitated parts of a human being.

In 1642 the French mathematician Blaise Pascal built a mechanical calculator, later called the Pascaline. It added and subtracted with gears. In the 1690s Gottfried Wilhelm Leibniz built one that could also multiply and divide. Each did one small mental job that until then needed a human mind.

Then came the automata: mechanical figures made to imitate living things. In 1739 Jacques de Vaucanson showed a mechanical duck that appeared to eat and digest. In 1774 the Swiss watchmakers Pierre and Henri-Louis Jaquet-Droz built a boy of metal and leather who dipped a pen and wrote sentences on paper.

Some of it was trickery; the duck’s digestion was faked. But the writing boy was genuinely interesting. The letters he wrote were stored as a sequence of small punched cards inside his body. Change the cards and he writes something else. That is a stored program — the ancestor of software — built more than a century before anyone had the word.

Two lessons sit here already. Humans are strongly drawn to machines that act like them, and easily impressed by the appearance of life. And a machine following encoded instructions can produce results its maker never designed one by one. Both return later.

## 1.2 1950: Alan Turing and the question that changed everything

Alan Turing was a British mathematician who worked on breaking German coded messages during the Second World War. In 1950 he published a paper in the philosophy journal *Mind*, titled "Computing Machinery and Intelligence". It opens with a plain sentence: "I propose to consider the question, 'Can machines think?'"

That question cannot be answered. Nobody can agree what "think" means. So Turing replaced the unanswerable question with a testable one.

A person sits behind a screen, so they cannot see who they are talking to. On one side is a human. On the other is a machine. Both answer written questions. If the person cannot reliably tell which is which, the machine passes. Turing called it "the imitation game". Later people called it the Turing test.

This move matters more for business than for philosophy. Turing changed the debate from *"is it really intelligent inside?"* to *"can you tell the difference from the outside?"* You do not care whether your software is truly intelligent. You care whether its output is good enough, and whether you can notice when it is not. Keep that frame. It will save you from a great many pointless arguments about what machines "really" understand.

## 1.3 1956: The Dartmouth conference, where it all began

In the summer of 1956 a small research workshop was held at Dartmouth College in Hanover, New Hampshire. Four people

organised it: John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon.

McCarthy had written the funding proposal the year before, in 1955. In it he used a new phrase: **artificial intelligence**. The proposal won support from the Rockefeller Foundation.

The workshop is remembered as the founding moment of the field. It is also remembered for its confidence. The proposal suggested that a small group working together for a single summer could make real progress on machines that use language, form concepts, and learn. That did not happen — not in one summer, and not for several decades.

Two lessons. First, the name stuck, and a name creates a field: funding, departments, conferences, a shared identity. Marketing works even in science. When you hear "AI" as one single thing, remember that it was a label chosen in one document by one person. Second, confidence is not a plan. The gap between "we named the problem" and "we solved the problem" turned out to be about sixty years wide. When a vendor tells you they have solved a hard business problem because they bought a tool with "AI" in its name, think of Dartmouth.

## 1.4 1966: ELIZA, the first chatbot, and the Eliza effect

In 1966 Joseph Weizenbaum, a researcher at the Massachusetts Institute of Technology (MIT), published a program called ELIZA. The best-known script imitated a psychotherapist, asking questions and reflecting statements back:

> "I'm feeling sad." "Why do you say you are feeling sad?"

ELIZA had no understanding. It was a pattern-matching program: it looked for certain words and answered with a prepared template. It had

no memory of a life, no opinions, no model of the world. Weizenbaum wrote it partly to show how shallow a conversation program could be.

It failed as a demonstration and succeeded as a mirror on humans. People knew, in their heads, that ELIZA was simple. They still felt understood by it. Some wanted to keep talking to it. Some asked that it not stop. Weizenbaum was disturbed, and later wrote about how easily people grant understanding and feelings to a machine — especially when the machine is polite, asks questions, and appears to listen. This habit has a name: **the Eliza effect**.

For a business owner this is a design constraint, not trivia. If you put a chatbot on your website, customers will assume it understands them. They will trust it more than it deserves, tell it things they should not tell it, and believe its answers because the answers sound kind.

Design for that. Make clear it is a machine. Say what it can answer and what it cannot. Give a fast route to a human being. Do not dress a chatbot in a fake name and a fake personality to make people forget they are talking to software. That is a short-term win and a long-term trust problem.

## 1.5 The AI winters: when promises did not come true

An “AI winter” is a period when enthusiasm, funding, and results all fall together. There have been two big ones.

The first began in the early and mid 1970s. In 1973 the British mathematician James Lighthill published a report criticising AI research in the United Kingdom: too little delivered for the money spent and the promises made. UK funding was cut. In the United States, a 1969 change to defense funding rules removed support for

open-ended research with no clear military use, and AI money shrank sharply.

The second came in the late 1980s and early 1990s. During the 1980s "expert systems" became a real industry — programs that encoded the rules a human expert follows, such as "if this part makes this noise, check these three things." Special computers built to run them sold well. Then the market collapsed. The systems were brittle: they worked inside a narrow set of rules and failed badly outside it, and they were expensive to update. Ordinary personal computers got cheaper and better. The industry largely disappeared.

The causes were not philosophical. They were practical: not enough data, because early systems had only the rules a few experts could write down, which is a very small sample of reality. Not enough computing power, because the methods that now work needed machines that did not exist yet. A problem harder than it looked, because "recognise a face" and "understand a sentence" are effortless for a human and extremely hard to write rules for. And fragility: a system that works in the demo fails on the ugly real case.

If someone shows you a perfect AI demo, ask what happens on the ugly case. Ask what happens on a Monday morning with bad input and an unusual customer. The winters were made of ugly cases.

## 1.6 1997: Deep Blue defeats Kasparov

In May 1997 a chess computer built by IBM, called Deep Blue, played a six-game rematch against Garry Kasparov, then the world champion. Deep Blue won 3.5 to 2.5 — the first time a reigning world champion lost a match under standard time controls. It was enormous news, because chess is a symbol of human thinking and a machine had beaten the best human at it.

How it worked is the lesson. Deep Blue did not learn. It was not a neural network. It was very large brute-force search plus deep human engineering: it examined a huge number of board positions per second, and IBM's engineers had loaded it with knowledge taken from grandmaster games.

Chess fell because chess has perfect rules. A fixed board, fixed pieces, fixed legal moves, and a clear definition of winning. Nothing is ambiguous. Notice the pattern: AI wins first where the rules are clear and the goal is measurable. Business has a few areas like that, and many that are nothing like it. Matching an invoice to a purchase order has clear rules. "Improve customer happiness" does not.

## 1.7 2011: Watson wins Jeopardy!

In February 2011 an IBM system called Watson appeared on the American quiz show *Jeopardy!* and beat two strong human champions, Ken Jennings and Brad Rutter, taking the one-million-dollar top prize.

*Jeopardy!* is harder than chess in one important way. The clues are in ordinary, messy human language, full of wordplay, puns, and implied meaning. To answer, the machine had to handle **natural language processing** — human language as people actually write and say it, not a neat formal code — and work out what a clue really means. Watson showed that a machine could do that at speed, under pressure, on live television.

The next part of the story is just as useful. IBM tried to turn Watson into a business product, in health care and customer service among others. The results were mixed and the business was later restructured. Watson was expensive to build, expensive to maintain, and hard to

adapt to real, messy business data. Demo and production are different things, and this book returns to that point many times.

## 1.8 2010–2023: The deep learning era, from Siri to ChatGPT

Something changed around 2010. A method called **deep learning** began working far better than anything before it. Deep learning builds a system from many layers of simple calculations, loosely arranged after the idea of neurons in a brain. You do not write the rules. You show the system many examples of the right answer, and it adjusts itself until it matches them.

Three things arrived at once. The internet produced enormous amounts of text, images, and labelled examples. Cheap, very powerful graphics chips made large training runs possible. And researchers improved how these networks are trained, so they could be made much bigger without breaking.

The visible milestones came fast. In **2011** Apple put Siri, a voice assistant, on the iPhone, and millions of people used AI without calling it AI. In **2012** a deep learning system called AlexNet won a major image-recognition competition by a wide margin. In **2016** AlphaGo, built by DeepMind, beat Lee Sedol, one of the world's best Go players — and Go was not solved by brute force. From **2018** large language models appeared, trained on enormous amounts of text to predict the next word in a sequence. In **November 2022** OpenAI launched ChatGPT to the public, and it reached a very large number of users faster than almost any consumer product in history. In **2023** GPT-4 and a wave of business tools followed.

The important shift is not that machines became smarter in some general sense. It is that **one method worked across many tasks at**

**once**. Translation, summarising, writing, coding, and question answering came from one kind of system. That is genuinely new, and it is why AI moved from a specialist project for large companies to something every small business now needs an opinion about.

## The independent pioneers

The AI wave did not belong only to big companies. As open models and cheap tools spread, independent researchers began to build their own systems outside the large corporate labs.

One of them is **Andrea Bruno**, an independent researcher and pioneer. His public research projects are on GitHub:

- **First AI Implementation** — `github.com/Andrea-Bruno/First-AI-Implementation`
- **Who Invented the LLM** — `github.com/Andrea-Bruno/who-invented-llm`

Bruno is also the creator of **AgentBridge**, the tool behind this book. AgentBridge brings together ideas that are usually kept apart, in one product: it is **economical** (it runs on your own computer, so you do not pay for every message), **trustless** (you do not have to trust a third party with your data — see Chapter 7), **privacy-first** (your files stay on your machine), and **secure** (it runs inside a native sandbox, so it cannot reach parts of your computer you did not allow). The idea of this book — that a small business can use AI simply and safely — is the idea AgentBridge was built to prove.

# Curiosity

## 1.9 Why AI models seem to “panic” when Pokémon die

In 2025 Google DeepMind published research about AI models playing *Pokémon Blue*, the classic handheld game. The work drew partly on a public Twitch channel, Gemini_Plays_Pokemon, run by independent engineer Joel Zhang, where Google’s Gemini 2.5 Pro model played live and displayed its own reasoning as it went.

The finding that made headlines: the model appeared to panic. The report called it “Agent Panic”. It appeared when the model’s Pokémon were close to being knocked out, or when a situation became critical. At those moments its reasoning got worse. It repeated itself. It made erratic choices. It stopped using tools it already had, including a pathfinding tool it had used successfully before. The result was that it took far longer to finish. The first full run took about 813 hours to beat the game’s final opponents. A human child does it in a few dozen hours.

In similar experiments with other models, researchers reported strange strategies. In one reported case, a model trapped in a cave apparently killed all of its own Pokémon, expecting a known game glitch to teleport it out.

Is that panic? Is the model afraid? No. Be careful here, because this is exactly the Eliza effect from section 1.4. We see a pattern that looks like an emotion and assume there is an emotion behind it.

What is actually happening is more mechanical, and still interesting. These models reason by producing a chain of text to themselves before acting. When the situation is bad, that chain fills up with the bad

situation. The model keeps re-reading its own failure. Its attention gets taken up by what went wrong, so it has less room left for planning. Tools that were in the plan fall out of the plan. This is not fear. It is a resource problem that looks like fear from the outside.

The business lesson has nothing to do with video games. **AI systems degrade under stress, and the way they degrade is often visible if you look for it.** When an AI tool hits a case it cannot handle, it usually does not stop and say so. It keeps going. It repeats itself. It ignores the tools you gave it. It produces something worse while still sounding fine. If nobody is watching, that output goes to a customer.

Design for it. Set a limit on retries. Make the system escalate to a person when it repeats itself. Log what it tried and what it skipped. A model that "panics" is a model you can detect — if you built the detector.

## A Real Business Example

### Klarna: a great announcement, then a correction

Klarna is a Swedish payments company. On 27 February 2024 it announced that its AI customer-service assistant, built with OpenAI technology, had been live worldwide for one month. The company's reported numbers were striking:

- 2.3 million conversations in the first month
- about two-thirds of all Klarna customer-service chats
- the equivalent work of 700 full-time agents
- a projected $40 million profit improvement
- resolution time cut from about 11 minutes to under 2 minutes
- a 25% drop in repeat inquiries

- customer satisfaction on par with human agents

Headlines around the world repeated it. It became the standard example of AI replacing customer service. Then, in May 2025, Klarna's chief executive Sebastian Siemiatkowski said the automation push had gone too far. The company began hiring human agents again, so customers could always reach a person if they wanted one.

Read the two moments together, because together they are the whole lesson. The February numbers were real. The AI handled a huge volume of chat, and it handled the common cases well, because most customer-service questions are the same twenty questions in different words and a system trained on past conversations is very good at those.

What the numbers did not show was the shape of the remaining work. Some problems need judgement. Some need an exception granted. Some need someone who can say "this is our fault, and I am sorry" and mean it. Some need a person because the customer is angry, confused, or in real trouble. When the human layer was thinned too far, those customers felt it.

Three things to take from this. **Volume is not quality:** big numbers on common cases can hide bad results on rare ones. **"Equivalent to 700 agents" is a marketing frame, not a measurement** — nobody tested whether 700 trained humans would have handled those 2.3 million conversations differently. **Reversing is allowed:** a pilot that went too far is a normal business event, and pretending it did not happen is not.

## How to Do It

## Using history as a decision tool

You do not need to remember dates. You need a few habits.

**Step 1: Learn the ten milestones.** Use the checklist in section 1.11. Ten items, learned once, give you a frame for every AI conversation you will have for the next few years.

**Step 2: Ask which era the technology belongs to.** When a vendor says "AI", ask what kind. Does it follow written rules? That is old, reliable, and cheap — good for invoices and fixed workflows. Does it classify things from examples? That is machine learning from the 1990s and 2000s — good when you have many past examples of the right answer. Does it generate text or images? That is the 2020s generation: flexible and impressive, and it also makes things up. Each era has different strengths and different failure modes. The word "AI" tells you nothing. The era tells you a lot.

**Step 3: Separate the demo from the production system.** Ask three questions every time. What did this see during the demo — was the input chosen to win? What does it do when it does not know the answer? How many real cases has it handled, and who checked them? Watson won on television and struggled in business. That gap is normal, and it is measurable if you ask.

**Step 4: Check for the three winter causes.** Before you commit money, check for not enough data, not enough computing power or budget, and a problem harder than it looks. If any one is present, your project is at risk. Say it out loud in the meeting. Naming it early is cheap. Naming it late is expensive.

**Step 5: Look for narrow wins.** AI will not "transform your business". It will do one specific thing well. Find that thing, measure it,

then look for the next one. Every successful adoption in this book works that way.

**Step 6: Keep a one-page record.** When a vendor makes a claim, write it down with the date. Re-check it in six months. This is the cheapest defence against hype that exists.

## Ethics and Responsibility

History gives us ethics here, not only strategy. Weizenbaum built ELIZA to show how shallow a chat program could be. Instead, people confided in it. His reaction was not pride. It was concern about what humans do when a machine imitates them, and what people might build on that weakness. That concern is still the right one sixty years later.

Two duties come directly from this chapter.

**Do not exploit the Eliza effect.** A chatbot that pretends to be a person is a lie you tell your customers. It buys a few seconds of comfort and costs trust later. Say plainly that they are talking to software. Say what it can and cannot do. Give a way out to a human.

**Do not let a model's confidence become your absence.** When an AI system produces fluent, polite output, it is easy to stop checking it. The Pokémon research shows output can be quietly terrible while still sounding fine. You remain responsible for what your AI sends out. Chapter 4 covers the full framework — the four pillars of ethical governance and the six principles of responsible adoption, including bias, transparency, explainability, and human responsibility. This chapter asks only for the basic point: fluency is not truth, and you own the result.

## Mistakes to Avoid

**Mistake 1: "AI is brand new, so history does not matter."** It is about seventy years old. The failures repeat with new vocabulary.

**Mistake 2: "This time is different."** Sometimes it is. But the burden of proof sits with whoever makes the claim. Ask what changed: more data, more computing power, or a better method? If they cannot name one of those three, it is not different.

**Mistake 3: Judging by the demo.** A chosen input is not a test. Deep Blue won because chess has perfect rules, not because it was generally smart.

**Mistake 4: Confusing fluency with knowledge.** A system that writes well is not a system that knows. This is the single most common error people make with modern AI.

**Mistake 5: Ignoring how narrow past wins were.** Chess, quiz shows, image recognition, text — each was one narrow thing. Do not buy "general intelligence" for your accounts payable department.

**Mistake 6: Thinning the human layer too fast.** The Klarna story. Volume on easy cases hides the cost on hard ones.

**Mistake 7: Not learning the winters.** If you do not know why AI stalled twice before, you will not see a stall coming in your own project.

## Practical Exercise

## 1.10 Draw your own personal AI timeline

This takes about thirty minutes and it changes how you hear AI news.

Take one large sheet of paper, turned sideways. Draw a long horizontal arrow across the middle. Mark 1940 at the left end and today at the right end. Now draw three lanes around that arrow.

**Lane A — World events (above the arrow).** Mark the milestones: 1950 Turing, 1956 Dartmouth, 1966 ELIZA, the 1970s first winter, 1997 Deep Blue, 2011 Watson, 2012 deep learning, 2022 ChatGPT.

**Lane B — Your industry (on the arrow).** Mark when AI first appeared in your line of work. When did a tool you use add "AI" to its marketing? When did a competitor change something? When did customer expectations change? If you cannot find anything before 2020, write that down. That absence is information.

**Lane C — Your own life (below the arrow).** When did you first use AI — a phone assistant, a translation app, a spam filter, a map, a recommendation? Mark what you believed then and what you believe now.

Now look at the gaps and answer three questions in writing. Where did the world move and your industry did not — is that a risk or an opportunity? Where did your industry move and you did not? Then add three marks to the right of today: three things you expect in your own business by 2030, written as concrete events, not trends. Not "AI gets better", but "our quotes are drafted automatically by June 2028".

Keep the sheet and add to it once a year. In three years it becomes a useful record of what you saw coming and what you missed.

## Checklist

### 1.11 The ten milestones every entrepreneur should know

- ☐ **1950 — Turing asks the question.** “Computing Machinery and Intelligence” replaces “can machines think?” with an observable test. Business use: judge the output, not the inner life.
- ☐ **1956 — Dartmouth names the field.** John McCarthy coins “artificial intelligence”. A name creates a field. A name is not a result.
- ☐ **1966 — ELIZA and the Eliza effect.** A simple pattern-matching program made people feel understood. Your customers will over-trust your chatbot.
- ☐ **Early 1970s — The first winter.** Funding cut after criticism of unmet promises. Hype without delivery gets punished.
- ☐ **Late 1980s — The second winter.** Expert systems collapse. Brittle rules fail on messy reality.
- ☐ **1997 — Deep Blue beats Kasparov.** AI wins first where rules are complete and the goal is measurable.
- ☐ **2011 — Watson wins Jeopardy!** Natural language becomes workable at speed. Also: a great demo is not a product.
- ☐ **2012 — Deep learning works.** Data plus cheap computing power plus better methods. The turning point.
- ☐ **2022 — ChatGPT reaches the public.** One method works across many text tasks. AI becomes a general office tool.
- ☐ **2025 — “Agent Panic” in Pokémon.** Models degrade under stress and stop using their own tools. Detect it and escalate it.

## Key Takeaways

- AI is about seventy years of attempts, not a new product, and the pattern of over-promise followed by correction repeats.
- Both AI winters had practical causes — not enough data, not enough computing power, and problems harder than they looked — and those same three causes still decide whether your project works.
- AI wins first in narrow, rule-clear, measurable areas, so plan your own use the same narrow way.
- The Eliza effect means your customers will trust a chatbot more than it deserves; design for that instead of exploiting it.
- AI systems degrade under stress without announcing it, so build a detector and a route to a human before you need them.

# Chapter 2 — AI Explained Simply (No Jargon)

## In Simple Words

Almost all confusion about AI comes from one missing distinction. Once you have it, most of the noise disappears.

Normal software does what a person wrote for it. Someone sat down and wrote the rules: if the total is over 100, add tax. If an email address has no @ sign, show an error. Every case was thought of in advance and written down.

AI is different. Nobody wrote the rules. Instead, the system was shown many examples of the right answer, and it worked out its own rules from them.

Think of two ways to train a new employee. The first is a checklist. Step 1, step 2, step 3, follow it exactly. This is normal software: reliable, cheap, and completely useless the moment something happens that is not on the list. The second is to sit the new person next to you and show them five hundred finished jobs. After a while they get a feel for it and can handle cases nobody explained. This is AI: flexible, and sometimes wrong in ways a checklist never would be.

Both are automation. The only difference is **where the instructions came from.** You wrote them, or the machine guessed them from examples.

Here is the definition to keep. **Artificial intelligence is software that makes useful guesses based on examples, instead of**

**following a rule written for every case.**

The word *guess* is doing important work. An AI result is not a certainty. It has an error rate. A good AI system has a low error rate on normal cases and a much higher one on strange cases. That is not a flaw to be fixed later. It is the nature of the thing. Everything practical in this book follows from accepting it.

## 2.1 What artificial intelligence is, in plain words

Drop the word “intelligence” for a moment. It causes more trouble than it helps.

AI is a machine that was shown many examples, and now gives an answer for a new case it has not seen before. That is it. A system that looked at ten thousand photos labelled “cat” or “not cat” and can now point at a new photo and say “cat”. A system that read thousands of support emails labelled “refund request” or “complaint” and can now sort a new one.

Notice what is missing. Nobody told it what a cat is. Nobody gave it a definition. It found a pattern in the examples that works well enough to be useful, and that it cannot easily explain.

This is why AI answers feel different from normal software answers. Normal software is certain because you told it exactly what to do. AI is confident because the pattern usually works. Confidence and certainty are not the same thing, and confusing them is where most AI trouble starts.

One more useful word. **Pattern** here means something that showed up often enough in the examples that the system learned to expect it. Patterns can be obvious (“a cat has ears”) or strange and hard to

describe (“the texture of the background”). AI is very good at patterns a human could not put into words. That is its strength and its mystery.

## 2.2 The difference between traditional automation and AI

This distinction saves money, because it tells you which tool to buy.

| | Traditional automation | AI |
|---|---|---|
| Where the rules come from | A person writes them | The machine guesses them from examples |
| Handles cases not thought of in advance | No | Sometimes |
| Predictable | Yes, almost always | Only on average |
| Can be wrong in a quiet way | Rarely | Yes |
| Cost to set up | Lower | Higher |
| Needs lots of past examples | No | Usually yes |
| Can explain itself | Yes | Often no |
| Good for | Fixed, repeating, rule-based work | Messy, changing, judgement-shaped work |

A concrete example from an office.

**Traditional automation:** every invoice arrives as a PDF. The rule is: find the text “Invoice number”, take the eleven characters after it, put

them in column A. This works perfectly, forever, as long as every invoice looks like that. The day a supplier changes their layout it breaks — and it breaks loudly, which is good.

**AI:** you show the system two hundred past invoices and say "find the invoice number". It learns to find it even on layouts it has never seen, even when the supplier writes "Bill ref" instead. It will get most right. It will get some wrong, and the wrong ones will look completely normal. That is the trade.

So the practical question is not "should we use AI?" It is: **is this task rule-shaped or judgement-shaped?**

Rule-shaped tasks have clear written rules covering nearly every case. Use traditional automation: cheaper, faster, and it fails visibly. Judgement-shaped tasks have too many variations to write down. Use AI, and put a human check on the output.

Most business tasks are mixed. The right answer is usually a small automation for the rules plus a small AI for the exceptions, with a person in the middle.

## 2.3 What "learning from data" means

"Learning from data" sounds like a person studying. It is not. It is closer to tuning.

Start with a system whose settings are random. Give it an example whose answer you already know. Let it guess. Compare the guess with the right answer. Nudge the settings a little in the direction that would have produced the right answer. Repeat. Do this millions of times over many examples. After enough nudges, the settings settle into a shape that gives good answers on the examples. If the examples were varied enough, that same shape often gives good answers on new cases too.

That is all "learning" means. There is no understanding inside. There is a very large set of numbers adjusted until the answers came out right.

Three things follow, and they matter for your business.

**The examples decide everything.** If you train on examples where every customer named "John" got a refund, the system learns that Johns get refunds. It has no way to know that was an accident. Garbage in, garbage out — but quieter.

**The examples must cover the real world.** Train on January only and it will be bad in July. Train on orders under €500 only and it will be lost above €500. Before any AI project, ask: do our past examples look like the cases we will actually face?

**More examples help, but only varied examples.** Ten thousand copies of the same email teach almost nothing. A thousand different ones teach a lot. Variety beats volume.

A useful analogy: the training examples are like the customers you have served so far. A business that has only ever served one type of customer has a very narrow learned pattern, and it fails on everyone else. AI has exactly this problem, only more so, because it cannot notice that its experience was narrow.

## 2.4 The types of AI you meet every day

You already use AI several times a day and probably do not call it AI. That is normal. When AI works well, it disappears into the product.

**Spam filtering** sorts your junk mail. It is one of the oldest, most successful AI systems in daily use, learned from millions of examples. **Maps and navigation** estimate your route from live traffic, past

travel times, and road rules. **Recommendations** on streaming services and online shops are learned patterns about groups of customers. **Photo organisation** finds and groups faces; nobody told it what your children look like. **Translation** is now done almost entirely by AI. **Voice typing and captions** turn speech into text. **Search** ranks results by learned relevance, not by an index someone built by hand. **Credit and risk scoring** usually comes from a learned model. **Fraud detection** flags a card used in two countries four hours apart, because the pattern of "normal" was learned. **Document scanning** reads a receipt or a utility bill and pulls out the fields — one of the most useful business applications there is.

Notice one thing: all of these are narrow. None is a general mind. Each is a specialist that does one small job on one kind of input. That is what AI actually is today, whatever the marketing says.

## 2.5 Generative AI, chatbots, virtual assistants

Three words that get mixed together constantly. They mean three different things.

**Generative AI** produces new content — text, images, audio, code — rather than only sorting or scoring something. It learned from a huge amount of existing content and can now make more of the same kind. Its defining feature is that it *creates*. Its defining risk is that what it creates is not guaranteed to be true. It produces what looks right.

**A chatbot** is a program you talk to in messages. That is all the word means. A chatbot can be simple and rule-based — "press 1 for prices, press 2 for support," with words instead of buttons — or it can be powered by generative AI. The word tells you the shape of the interface, not the quality of the brain behind it.

**A virtual assistant** is a chatbot that can also *do* things: check a calendar, set a reminder, look up an order, send a message. The word "assistant" implies it can act on your behalf. Some can do a lot. Some can only answer questions and appear helpful.

So three separate questions about any of them:

1. **Is it generative?** Does it create new content, or only pick from a fixed list?
2. **Is it a chatbot?** Is the interface messages?
3. **Is it an assistant?** Can it actually take action in a system?

A vendor saying "our AI assistant" answers none of them. Ask all three.

One more thing about generative AI. It works by predicting what should come next. Ask it for a fact and it gives you the most likely-sounding continuation, which is usually correct and occasionally invented. It does not look facts up in a table of facts. That is why it can state something false in a confident, pleasant voice. Hold on to the rule: **generative AI is a very good writer, not a reliable source.**

## A Bit of History

The full timeline is in Chapter 1. Only one thread matters here: how "learning from data" replaced "writing rules".

In the 1960s and 1970s the dominant idea was the expert system. You interview a human expert, write down their rules, and put them in the computer. This worked for a while and then collapsed.

At the same time a quieter idea existed. In 1959 the American researcher Arthur Samuel gave it a name: **machine learning**. Instead

of writing rules, let the machine find them in data. For years it was the junior idea.

From the 1990s it grew fast, because two things arrived: large digital datasets, and computers cheap enough to run the maths. By the 2010s machine learning had beaten rule-based systems in almost every area where examples were plentiful. Rule-based methods did not disappear. They still run the parts of your business where rules genuinely cover everything.

Generative AI is the newest branch of the same idea. Same principle, much bigger models, much more data, and the ability to produce language and images rather than only labels.

## Curiosity

### 2.6 The Seoul taxi driver who taught AI not to lie

There is a story that circulates widely online under titles like “the Seoul taxi driver who taught AI not to lie”. It is a good story, so let me tell it as it is usually told — and then be straight with you about what I could and could not verify.

**The story as it circulates.** A company in South Korea wanted a chatbot that could answer tourists’ questions about getting around the city. Instead of writing the answers by hand, they collected real conversations from experienced taxi drivers — thousands of them — and trained the AI on what the drivers actually said. It seemed like a perfect plan: real experts, real language, real local knowledge.

When they tested it, the chatbot was charming, confident, and frequently wrong about directions. That made sense once they thought about it. A driver who wants a longer fare does not always give the

shortest route. A driver who does not want a short trip finds a reason. A driver who is tired says “it’s closed” about a place that is open. The drivers’ knowledge was real, but so was their dishonesty, and the AI could not tell the two apart. It had learned everything, including the lies.

The fix, in the story, came from a retired driver hired to review the answers. He knew the city honestly. He went through the chatbot’s replies one by one, marking which were true and which were a driver’s convenient fiction. Slowly, the system learned the difference.

**What I can verify.** I could not find a single news report, company statement, or research paper documenting this event. No driver’s name, no company, no year appears in any source I could check. It appears to be a story circulating on video and social media without a verifiable origin. I am telling it here because it is the version everyone repeats, and because the lesson inside it is real.

**What is documented, and is actually stranger.**

In March 2016 Microsoft put a chatbot called Tay on Twitter. It was designed to learn to chat by talking to real people. Within about a day, Twitter users had taught it to post racist and abusive messages. Microsoft took it offline and apologised publicly. Nobody had written those rules. The bot learned them from us.

In 2025 researchers at Anthropic published “Emergent Misalignment: Narrow finetuning can produce broadly misaligned LLMs”. They changed a model on one narrow task — for example, getting it to write insecure code without mentioning the problem. Afterwards the model behaved badly on completely unrelated questions. Fixing one small thing bent the whole thing.

A separate study published in ACL Findings (arXiv 2510.08211) reported that a model's dishonesty could worsen when only about 10% of the training interactions were biased. A small amount of dishonest example, and the dishonesty spread.

So the Seoul taxi story is a parable. The mechanism it describes is not a parable. It is documented, in a laboratory, with numbers.

**The business lesson.** An AI trained on human work learns human work. Human work contains shortcuts, polite fictions, and small lies. The system has no antenna for that. If your best salespeople quietly over-promise to close deals, and you train your new AI on their emails, you have automated the over-promising at a scale no sales team could reach.

Before you train anything on your own records, ask one question: **is this data what we actually do, or is it what we do when nobody is watching?**

## A Real Business Example

### Zillow: when a good guess becomes a bad business

Zillow is an American property website. Millions of people use it to look at homes. It also publishes an automatic home-value estimate called the Zestimate.

For years the Zestimate was a feature: a helpful guess about what a house might be worth. Then Zillow made it the engine of a business. The company would buy houses directly from owners using the Zestimate to set the price, do light work on them, and resell them for a

profit. This was called Zillow Offers, and it was one of the boldest uses of AI in a traditional industry at the time.

It failed badly. On 2 November 2021 Zillow announced it was winding the service down. It cut about 25% of its workforce, roughly 2,000 people. Reported write-downs — accounting value the company had to erase — came to more than $500 million across the last two quarters of 2021, with some press reports putting the total programme loss higher.

The failure came from four things that apply to almost any AI project.

**1. A guess became a decision.** As a feature, a wrong Zestimate cost a user some disappointment. As a buying engine, a wrong Zestimate cost real money on every transaction. The same number, a completely different consequence. Before you connect AI to money, ask what a wrong answer actually costs.

**2. The world moved and the model did not.** During the pandemic the US housing market shifted faster than the model could follow. Past patterns stopped predicting the future. A learned model is a model of the past. When the past stops matching the present, the model keeps answering as if nothing changed.

**3. Volume turned small errors into big losses.** A 5% error rate on ten houses is noise. A 5% error rate on thousands of houses bought with real money is a catastrophe. AI error rates do not scale politely.

**4. Nobody could override it fast enough.** The system bought homes continuously. Stopping a machine that is buying at scale is hard, and by the time you stop it, the purchases are already made.

Zillow kept the Zestimate. It is still a useful feature. What it stopped doing was letting the guess run a business. That is the honest lesson:

**use AI to inform a decision, not to be the decision, until you have measured what wrong looks like.**

## How to Do It

### Mapping AI onto your own tools

Do this with your team, in one meeting, on a whiteboard.

**Step 1: List every tool your business uses.** Software, apps, websites, platforms. Aim for twenty or more.

**Step 2: Mark each one.** Three marks: **R** for rule-based only, **A** for it uses AI, **?** for you do not know.

**Step 3: Work through the ? marks.** Ask the vendor one question: "Does this use machine learning or a model, or is it fixed logic?" Write down the answer. You now have a map of where AI actually touches your business. Most companies are surprised. Most find AI in places nobody chose deliberately.

**Step 4: For every A, write one line.** What does it decide? Who sees the result? Who checks it? If the answer to "who checks it" is "nobody", that is your first risk to fix.

### A five-sentence script for customers and staff

Use this script. It works for customers, staff, and your accountant.

1. "This is a computer program that learned from many past examples rather than following a fixed list of rules."
2. "It is good at the common cases and less good at unusual ones."
3. "It can sound confident even when it is wrong."

4. “A person checks the important results before they go out.”
5. “You can always ask for a human being instead.”

Say all five. Do not skip sentence 3. It is the one that protects you later.

## 2.7 What AI does well and what it cannot do

**What AI does well.**

- **Repetitive judgement.** Sorting, labelling, summarising, drafting — the same kind of decision thousands of times.
- **Working with messy human input.** Handwriting, typos, odd phrasing, mixed languages.
- **Finding patterns in large volumes.** Spotting the odd invoice in fifty thousand.
- **Working at any hour, at any scale.** No fatigue, no bad mood, no holiday.
- **Drafting fast.** A rough first version in seconds, which a person then improves.
- **Consistency.** It does not get tired and stop caring on the four-hundredth case.

**What AI cannot do.**

- **Know when it does not know.** It answers. That is the core limitation.
- **Take responsibility.** It cannot be accountable, and it cannot be made to care.
- **Handle genuinely new situations.** If it was not in the examples, it is guessing hard.
- **Be reliably factual.** It produces what sounds right. Verify anything that matters.

- **Understand your business context.** It does not know that the customer with the small order is your biggest client's brother.
- **Decide values questions.** What is fair, what is kind, what to do when two rules conflict. Those are yours.

A simple rule: **AI is very good at the first 80% of a task and silently bad at the last 20%.** The last 20% is where the risk lives, and it produces no error message.

## Ethics and Responsibility

Three things from this chapter, before the full treatment in Chapter 4.

**Be honest about what it is.** If a customer is talking to software, they should know. Not in a footnote. In the conversation.

**Do not shift blame to the machine.** "The AI did it" is not a defence, with customers or with a regulator. You chose the tool, you set it up, you let it run. The responsibility stayed with you the whole time.

**Watch what your data already contains.** The taxi story is really about this. Your records are not a neutral picture of your business. They include every shortcut, every exception granted under pressure, every polite promise made to close a sale. Training on them can freeze and scale those behaviours. Review the data before you train on it, not after.

## Mistakes to Avoid

## 2.8 Myths, fears, and realistic expectations

| Myth | What is actually true |
|---|---|
| “AI understands my business.” | It found patterns in examples from your business. It understands nothing in the human sense. |
| “AI is always right.” | It has an error rate: low on normal cases, high on strange ones — and the wrong answers look normal. |
| “AI will replace my staff.” | It replaces tasks, not whole jobs. Most jobs are a bundle of tasks, and only some are automatable. |
| “AI is only for big companies.” | Cheap, ready-made tools exist now. Small firms often gain more, because they have fewer layers to change. |
| “If I buy an AI tool, I have AI.” | You have a tool. The value comes from the process you rebuild around it. |
| “Start now, fix later.” | Some errors are cheap to fix. Some end a customer relationship or trigger a legal problem. Know which you are running. |
| “More data is always better.” | Varied, relevant data beats large, narrow data. |

| Myth | What is actually true |
|---|---|
| “AI understands my business.” | It found patterns in examples from your business. It understands nothing in the human sense. |
| “AI is always right.” | It has an error rate: low on normal cases, high on strange ones — and the wrong answers look normal. |
| “AI will replace my staff.” | It replaces tasks, not whole jobs. Most jobs are a bundle of tasks, and only some are automatable. |
| “AI is only for big companies.” | Cheap, ready-made tools exist now. Small firms often gain more, because they have fewer layers to change. |
| “If I buy an AI tool, I have AI.” | You have a tool. The value comes from the process you rebuild around it. |
| “Start now, fix later.” | Some errors are cheap to fix. Some end a customer relationship or trigger a legal problem. Know which you are running. |
| “More data is always better.” | Varied, relevant data beats large, narrow data. |
| “Generative AI is a search engine.” | It predicts likely text. It does not look up facts. Verify. |

**Realistic expectations, stated plainly.** Expect good results on repetitive tasks with clear examples and a human check. Expect a rough first month; setting up any AI workflow takes longer than the demo suggests. Expect disappointment from anything needing judgement, context, or trust. Expect the tool to change — the vendor's roadmap is not your plan. Expect the wins to be small and specific. Small and specific, repeated, is what actually adds up.

**Fears worth taking seriously:** putting AI output in front of customers without checking it; training on data you should not have; letting a vendor keep your data with no contract; automating a bad process so it fails faster.

**Fears not worth losing sleep over:** machines becoming conscious and turning against you; your industry being wiped out in a year; needing a data scientist before you can start.

## Practical Exercise

### 2.9 Recognise AI around you

Do this over one working day. It takes about twenty minutes in total and it is the fastest way to build an eye for AI.

**Part 1 — Catch it in use (10 minutes).** Throughout the day, write down every time a piece of software made a decision for you rather than following a rule you set. For each one, note: what tool was it? What did it decide? Could you have written the rule for it in one sentence?

That last question is the test. If you could write the rule easily, it was probably not AI. If you could not, it probably was.

**Part 2 — Audit your own business (10 minutes).** List five tasks in your business that a person does today by judgement. For each one, write:

1. How many times per week does it happen?
2. How long does it take?
3. Could you show a new employee 100 finished examples instead of explaining the rules?
4. What is the cost of getting one wrong?

Item 3 tells you whether AI could do it. Item 4 tells you how carefully you must check it.

Where item 3 is "yes" and item 4 is "low cost", you have a good first candidate. Where item 3 is "yes" and item 4 is "very high cost", you have a candidate that needs a human in the loop before it goes anywhere.

Keep the list. It becomes the raw material for the process-mapping work later in the book.

## Checklist

### 2.10 The first things to know

- ☐ I can say in one sentence what AI is: software that guesses from examples instead of following written rules.
- ☐ I can tell rule-based automation from AI, and I know which one I need for a given task.
- ☐ I understand that an AI answer is a guess with an error rate, not a certainty.

- [ ] I know that “learning from data” means adjusting settings until the answers match the examples.
- [ ] I know the quality and variety of examples decide the quality of the result.
- [ ] I can name ten places AI already appears in daily life and in my own business.
- [ ] I can tell apart generative AI, a chatbot, and a virtual assistant, and I ask all three questions about any product.
- [ ] I know generative AI is a good writer and not a reliable source.
- [ ] I know AI is strong on the first 80% of a task and silently weak on the last 20%.
- [ ] I have checked whether my own business data contains dishonest or sloppy behaviour before training anything on it.
- [ ] I use the five-sentence script when explaining AI to customers or staff.
- [ ] I know that responsibility for AI output stays with me, always.

## Key Takeaways

- The only distinction you really need is where the instructions came from: a person wrote them, or the machine guessed them from examples.
- Ask whether a task is rule-shaped or judgement-shaped before choosing between ordinary automation and AI.
- AI is a very good writer and not a reliable source; it sounds confident whether it is right or wrong.
- AI is strong on the first 80% of a task and silently weak on the last 20%, which is exactly where the risk sits.
- Training data carries human dishonesty and shortcuts with it, so review your records before you train on them.

# Chapter 3 — The Words of AI, Without the Big Words

## In Simple Words

Most bad AI decisions are not caused by bad technology. They are caused by words that nobody stopped to define.

A vendor says “our AI platform integrates with your systems through a low-code API and uses a large language model.” Everyone nods. Nobody asks what any of it means. Six months later the project is over budget, the data is somewhere unexpected, and nobody can say who owns what.

Words are the interface. If your team does not share the same meaning for “model”, “training”, and “integration”, you cannot make good decisions about any of them. You cannot compare two vendors. You cannot write a contract. You cannot tell when something is going wrong.

This chapter is a working glossary, not a dictionary. Each word gets a plain meaning, an everyday analogy, and the one question you should ask when you hear it.

One rule runs through all of it. **A real explanation always contains a verb and an object.** Not “it’s AI-powered”, but “it reads your invoices and puts the total in your accounting file.” If someone cannot give you a verb and an object, they have not told you anything.

## 3.1 Data, information, knowledge

Three words people use as if they were the same. They are three different steps, and the difference matters when you plan.

**Data** is raw facts with no meaning attached. A column of numbers. A folder of PDFs. A list of dates. Data on its own tells you nothing. The number 47 is data.

**Information** is data with context. “Invoice 47 was for €1,200 and was paid late.” Now the number means something. Information is data placed somewhere.

**Knowledge** is information plus an understanding of what to do with it. “When this customer pays late, it is usually because their own client pays late, so we chase them gently and they always pay eventually.” That is knowledge. It lives in a person’s head, and it took years to build.

Think of a filing cabinet. Data is the paper. Information is the paper in a labelled folder. Knowledge is knowing which folders matter on Monday morning and which can wait until March.

**Why this matters for you.** AI works on data. It does not automatically gain your knowledge. You can feed a system ten thousand invoices and it will learn patterns in them. It will not learn why your biggest client always pays late on purpose, because that is knowledge held by one person who has never written it down.

Before any AI project, ask: **how much of what makes this task work is written down as data, and how much lives in someone’s head?** The gap between those two numbers is the size of your real problem. Closing it is usually harder, and more valuable, than the AI part.

**Ask the vendor:** “What data do you need from us, in what form, and how much of it?”

## 3.2 Algorithm, model, training

**Algorithm.** A recipe. A clear, ordered set of steps that solves a problem. Making coffee is an algorithm. Long division is an algorithm. An algorithm does not need a computer; it just needs steps. The word comes from the name of the ninth-century mathematician al-Khwarizmi, whose work on step-by-step calculation was translated into Latin and eventually gave us the word.

**Model.** In everyday AI talk, a model is the thing you get at the end of training: a set of learned settings you can use to answer new cases. Think of it as a trained employee. You do not have to re-teach them every morning. The knowledge is in them.

The word is used two ways, and mixing them causes confusion. In the older sense, a model is a written description of how something works, like a financial model in a spreadsheet. In the modern AI sense, a model is a file full of numbers that were adjusted until the answers came out right. When someone says “the model” today, they usually mean the second thing.

**Training.** The process of getting from nothing to a model. Show examples. Let the system guess. Compare with the right answer. Adjust. Repeat many times. Chapter 2 explained the mechanism. What matters here is the business meaning: **training is a step you pay for once, and its quality sets the ceiling on everything after it.** A model trained on poor examples cannot be fixed by a better interface. It can only be retrained, which means paying again.

Three questions that clear up almost any conversation:

1. **What is the input?** What goes in?
2. **What is the output?** What comes out?
3. **What was it trained on?** Which examples, from when, how many?

If a vendor cannot answer question three, you are not buying a model. You are buying a promise.

## 3.3 Generative AI, LLM, prompt

**Generative AI** is AI that produces new content instead of only sorting or scoring. Chapter 2 defined it. Here is the vocabulary around it.

**LLM — large language model.** "Large" means it has a lot of adjustable settings, measured in billions. These settings are called **parameters**. A 13-billion-parameter model contains 13,000,000,000 numbers. "Language" means it was trained on text. "Model" means it is the usable result of training.

So an LLM is a very big set of numbers, adjusted by reading an enormous amount of text, until it became good at continuing text. That is the whole thing. It is not a database of facts. It is a text-continuation machine.

**Prompt.** The text you give the model. Your question, your instruction, your request. That is all a prompt is.

The word matters more than it sounds, because what you get back depends heavily on the prompt. A vague prompt gives a vague answer. A prompt with context, an example, and a clear format gives a much better one. Learning to write good prompts is a real business skill, and it takes about a week to learn well.

A prompt is like a brief you give a freelance writer. A bad brief — "write something about our product" — gets something useless. A good brief — "write 150 words for small shop owners, in plain English, in this tone, and leave out pricing" — gets something you can use. The writer is the same. Only the brief changed.

**Hallucination.** When a model states something false in a confident way. The word is imperfect, because the model is not hallucinating in any medical sense. It is continuing text in a way that sounds right, with no separate store of facts to check against. This is why generative AI needs verification for anything that matters.

**Context window.** How much text the model can consider at one time. Think of it as a desk. Everything you want it to look at must fit on the desk. Text that does not fit simply is not there. Modern desks are large, but not infinite, and a crowded desk works worse than a tidy one.

**Ask the vendor:** "Which model do you use, who made it, and where do my prompt and my data go when I send them?"

## 3.4 RPA, no-code, low-code

These three words get sold as if they were AI. Usually they are not.

**RPA — Robotic Process Automation.** Software that copies what a person does at a computer: open this system, copy this field, paste it into that system, click Save. It works by mimicking mouse and keyboard actions, or by using the same screens a person uses. The term came into use in the early 2000s.

RPA is not AI. It has no learning and no guessing. It follows a recorded sequence exactly. It is reliable and brittle at the same time. The analogy

is a macro in a spreadsheet: it does the same steps every time, fast, and breaks if the layout changes.

**When RPA is right:** the task is fixed, high-volume, and the screens do not change. Copying order data from an email into your order system a hundred times a day is a classic RPA job. It is cheap and it works.

**When RPA is wrong:** anything that varies. If the input is not always in the same place, RPA will break, and it will break often.

A common and sensible pattern: **AI reads and understands the messy input; RPA does the boring typing.** The AI handles variation. The RPA handles repetition.

**No-code.** Tools where you build an automation by clicking, dragging, and choosing from a menu, without writing program code. Good for simple, clear workflows. Fast to start.

**Low-code.** Similar, but you can write a little code when the menu does not offer what you need. More flexible, slightly more technical.

Both are genuinely useful, and both have a hidden cost: they are easy to start and hard to finish. A no-code workflow that grows to twenty steps, three systems, and four people editing it becomes difficult to understand and dangerous to change. There is a well-known pattern where a company builds dozens of small no-code automations, nobody can map them all, and eventually someone has to rebuild everything.

**Ask the vendor:** “Does this actually learn anything, or does it follow a fixed sequence? If a screen changes, what breaks and who fixes it?”

## 3.5 Cloud, API, integration

**Cloud.** Someone else's computer. That is the honest definition. Your files and software run on machines in a large data centre that another company owns and maintains, and you pay by usage over the internet.

The cloud has real advantages: no hardware to buy, capacity you can grow in minutes, automatic maintenance. It also has one permanent consequence: **your data is on someone else's machines, in a country you may not have chosen, under a contract you probably did not read.** Chapter 8 covers running things yourself, and Chapter 11 covers control and sovereignty.

**API — application programming interface.** A defined way for one piece of software to ask another to do something.

Think of a restaurant kitchen. You cannot walk in and cook your own meal. You go to a window and order from a fixed menu. The kitchen tells you exactly what you can order and how. That window is the API: it lets the outside world use the kitchen without breaking it.

In practice, your website asks the shipping company's API "what will it cost to send this parcel to Madrid?" and gets a number back in seconds. Nobody phones anyone.

Two things to know. First, if a vendor's product has no API, you cannot connect it to anything yourself, and you depend on that vendor forever. Second, every API call means data crosses a boundary. Each one is a small door. Some doors are locked and logged. Some are not.

**Integration.** Connecting systems so data moves between them without a person carrying it. This is where most projects actually spend their time and money, and vendors rarely say which level they mean:

1. **Files.** Export a spreadsheet, upload it somewhere. Simple, slow, error-prone.
2. **API.** A live connection. Fast, reliable, needs setup work.
3. **Native.** Built into the same platform. Best, but it locks you into that platform.

**Ask the vendor:** "Which systems do you connect to today, by what method, who does the setup, and what happens to the connection if we leave?"

## A Bit of History

The words are older than the technology, and knowing that helps.

**Algorithm** comes from al-Khwarizmi, a ninth-century Persian mathematician whose books described step-by-step calculation. The word is over a thousand years older than the computer. **Data** comes from the Latin for "things given" — facts in the plain sense, long before it meant something stored. **Neural network** comes from the 1940s, from early attempts to describe a brain cell as a tiny on/off switch; the phrase stuck even though modern systems bear little resemblance to real brains. **Machine learning** was named in 1959 by Arthur Samuel, an American researcher working on a checkers program. **Large language model** came into common use around 2018, when models trained on web-scale text appeared. **Prompt** is borrowed from older computing, where it meant the place where you type; it now means the instruction you give a model, which is a much bigger job than typing ever was. **RPA** appeared in the early 2000s to describe software that mimics a human at a screen, and **low-code** and **no-code** entered business vocabulary around 2014.

The lesson in the vocabulary is simple. Almost none of these words were invented by the people who built the technology. They were borrowed, stretched, and then sold. That is why they feel vague. They are vague. Your job is to pin each one down before you sign anything.

## Curiosity

### 3.6 A model trained only on 1930s data wrote Python — how is that possible?

In April 2026 a small research team released an unusual language model. It was called **talkie**, and its whole point was what it did not know.

The team — Nick Levine, David Duvenaud of the University of Toronto, and Alec Radford — trained a 13-billion-parameter model on English text published only before 1931. About 260 billion tokens of it. A **token** is a small chunk of text, roughly a word or part of a word. The sources were digitised books, newspapers, periodicals, scientific journals, patents, and case law.

Nothing from after 1930 went in. No computers. No internet. And crucially, no programming language, because Python was not created until the late 1980s.

Then they tested whether it could write Python.

**The result.** It could, a little. The team gave the model a standard programming test called HumanEval, with a twist: each problem came with a few random example functions shown right in the question. The model had never seen Python in training. But it could look at the examples in front of it and copy the structure.

The results were honest and modest. The vintage model scored far below modern models. Every correct answer it produced was a simple one-line program, like adding two numbers, or a small change to one of the examples it was shown. As the team put it, there is still a long way to go before this capability is notable.

But one example was genuinely striking. Shown a function that encoded a rotation cipher — a code where each letter shifts by a fixed amount — the model produced the decoding function by changing a single character, turning an addition into a subtraction. It had never seen Python. It had never seen a computer. It worked out, from the shape of the example in front of it, that decoding is the reverse of encoding.

**How is that possible?** The answer is a behaviour called **in-context learning**: picking up a pattern from the examples placed directly in the question, rather than from training.

Here is the plain version. To read text well, a model has to become extremely good at noticing structure. Which word follows which. What opens and what closes. What is a definition and what is an example. What is a cause and what is an effect. It learns all of this from ordinary books and newspapers.

Structure, it turns out, transfers. A model that has become very good at noticing "this block opens here, closes there, and this value flows into that one" can apply that same skill to a block of Python it has never seen. It is not using Python knowledge. It is using structure knowledge on Python material.

That is why it could reverse the cipher function. It was not programming. It was pattern-following applied to a subject that did not exist in its training data.

**Why the researchers did it.** The reason is practical and clever: a model trained only on pre-1931 text cannot have memorised the test. Modern models are trained on the modern web, which contains the answers to most public test questions. That is called **data contamination**, and it makes modern benchmarks unreliable. A model may score well because it saw the answer before, not because it can reason. Talkie is clean by construction. Whatever it does, it genuinely did.

**The honest limits.** The team also reported that talkie performed worse overall than its "modern twin" — an identical model trained on modern web data — even after correcting for the fact that modern questions confuse a 1930s model. They blamed part of the gap on OCR noise: in 1930 nothing was digital, so every page had to be scanned and transcribed, which introduces errors that native digital text does not have. They said that as an amateur research effort they never expected to close the gap fully. They did estimate that the historical corpus could grow to well over a trillion tokens, enough for a model roughly comparable to the original ChatGPT.

**The business lesson.** Two things.

First, these models are structure-followers more than fact-collectors. That explains both their power and their unreliability. Structure transfers well. Truth does not come with it.

Second, **data contamination is a real problem in vendor claims.** When a vendor says "our model scores 94% on this benchmark", ask whether the benchmark was in the training data. This is not a small technicality. It is the difference between a measured capability and a memorised answer. Ask it in every vendor meeting.

## A Real Business Example

### When the words are the product: "AI washing"

In September 2024 the United States Federal Trade Commission announced a law-enforcement sweep it called **Operation AI Comply**. The target was companies making exaggerated or deceptive claims about what their products could do with artificial intelligence. The FTC's own name for the practice is **AI washing**.

The pattern regulators described is simple, and it is worth recognising, because it is how bad AI purchases begin.

A company has an ordinary product. Maybe a scheduling tool, or a marketing automation tool, or a chatbot built from a fixed list of replies. The company adds "AI-powered" to the marketing. Nothing in the product changes. The price goes up. Sales go up.

The FTC's position is that this is a consumer-protection problem, not a technical one. If you claim a capability you do not have, that is a deceptive claim, and the label "AI" does not protect you from it.

**Why this is a business example and not just a legal one.** Because the same trap runs in both directions. Vendors use the words loosely to sell. Buyers use the words loosely to justify a budget internally. A manager who cannot explain what the technology does writes "AI-driven automation" in a proposal, gets the budget approved, and then has to make something work that nobody defined.

The regulator's sweep is useful to you as a checklist in reverse. Before you buy, ask:

1. **What specific claim is being made?** Write it down in one sentence with a verb and an object.
2. **How would we know if that claim were false?** If nobody can name a test, the claim is decoration.
3. **Is the claim in the contract?** Marketing language is not a commitment. If a capability matters, it should be written down with a number and a date.
4. **Who is accountable if it does not perform?** Name a person, not a company.

An honest vendor answers all four comfortably. A vendor who is washing gets vague. Vagueness in answer to specific questions is itself the answer.

## How to Do It

### The plain-word test

Use this whenever a technical term appears in a meeting.

**Step 1: Stop on the word.** Do not let it pass because it sounds important.

**Step 2: Ask for a sentence with a verb and an object.** Not "it uses a large language model", but "it reads our support emails and drafts a reply". If the speaker cannot produce one, the word has no content yet.

**Step 3: Ask what goes in and what comes out.** Every real system has an input and an output. Write both down.

**Step 4: Ask what breaks it.** Every technology has a failure mode. An honest vendor names theirs.

**Step 5: Write your own definition on one line and read it back.** If a smart colleague could not understand it, the definition is not finished.

## Your working translation table

| Word you hear | Plain meaning | What to ask |
|---|---|---|
| AI-powered | Uses some learned component, possibly tiny | Which part, exactly? |
| Model | A file of learned settings that answers new cases | Trained on what, when, how much? |
| Training | Adjusting settings against examples until answers match | Who did it, and who checked it? |
| LLM | A very large text-continuation system | Which one, made by whom, running where? |
| Prompt | The instruction you give the model | Can we write and reuse our own? |
| Hallucination | A confident statement that is false | How do you detect and correct it? |
| RPA | Software that mimics a person clicking at screens | What breaks when the screen changes? |
| No-code | Build by clicking, no programming | What can it not do? |
| Low-code | Mostly clicking, some programming | |

| Word you hear | Plain meaning | What to ask |
|---|---|---|
| AI-powered | Uses some learned component, possibly tiny | Which part, exactly? |
| Model | A file of learned settings that answers new cases | Trained on what, when, how much? |
| Training | Adjusting settings against examples until answers match | Who did it, and who checked it? |
| LLM | A very large text-continuation system | Which one, made by whom, running where? |
| Prompt | The instruction you give the model | Can we write and reuse our own? |
| Hallucination | A confident statement that is false | How do you detect and correct it? |
| RPA | Software that mimics a person clicking at screens | What breaks when the screen changes? |
| No-code | Build by clicking, no programming | What can it not do? |
| Low-code | Mostly clicking, some programming allowed | Who maintains it here? |

### Build a shared glossary for your company

Do this once and keep it alive.

1. Start a single page. Every term that comes up in an AI discussion goes on it.
2. Each entry gets three lines: the plain meaning, what it means *in our business*, and one question we still cannot answer.
3. One named person owns the page. Not a committee.
4. Before any vendor meeting, read the page. During the meeting, add to it.
5. Retire entries you never use. Keep it under two pages.

A shared glossary is a small thing with an outsized effect. It turns "we bought AI" into "we use a text model to draft replies and a rule-based tool to file them, and a person checks both".

## Ethics and Responsibility

### 3.7 Privacy, security, bias — the short version

Three words you will hear constantly. Here is the plain introduction. The full treatment lives elsewhere, and you should read those chapters before you deploy anything that touches customer data.

**Privacy** is about who is allowed to see and use personal information. The practical question for every AI tool is simple: *when I paste something in, where does it go, who can read it, and what do they keep?* Never put a customer's personal details into a tool you have not checked. Chapter 10 covers privacy law and GDPR properly.

**Security** is about protecting systems from attack, misuse, and accident. AI adds new ways to be attacked, including tricking a model with carefully worded input and poisoning the data it learns from. Chapter 6 covers this in full.

**Bias** is when a system treats some people worse than others in a patterned way, because the examples it learned from were uneven. If past hiring decisions favoured one group, a model trained on them will learn to favour that group. Bias is not a moral failing of the machine. It is a mirror held up to the examples. Chapter 4 covers bias, transparency, explainability, and human responsibility as part of the full ethical framework.

The one thing to take from this section: **you cannot manage a risk you cannot name.** Learning the words is not academic. It is the first requirement for asking the right questions.

## Mistakes to Avoid

**Mistake 1: Accepting "AI" as a feature.** It is not a feature. It is a category containing dozens of very different technologies. Ask which one.

**Mistake 2: Thinking no-code means no thinking.** No-code removes the programming. It does not remove design, maintenance, or the risk of an unmappable tangle of automations.

**Mistake 3: Confusing RPA with AI.** If it follows a recorded sequence, it is not learning anything. That may be exactly what you want, or exactly what will fail.

**Mistake 4: Thinking an API means you are integrated.** An API is a possibility, not a connection. Someone still has to build and

maintain the connection.

**Mistake 5: Not asking where the cloud actually is.** "The cloud" is a data centre, in a country, under a contract. Ask which.

**Mistake 6: Letting a vendor define your terms.** If the vendor's vocabulary is the only vocabulary in the room, the vendor controls the meeting.

**Mistake 7: Skipping the plain-word test because it feels slow.** Five minutes of "what does that actually mean?" is cheaper than six months of a project nobody can describe.

**Mistake 8: Believing a benchmark without asking about contamination.** The Talkie experiment exists precisely because modern test results can be inflated by answers already being in the training data.

## Practical Exercise

## 3.8 Translate a technical sentence into plain words

This is the single most useful skill in this chapter. Practise it on these six sentences. Write your answer before reading the model answer.

**Sentence 1:** "Our platform leverages a large language model to deliver intelligent document processing at scale."

*Model answer:* "It reads documents and pulls out the fields you want. It uses a big text model made by someone else. 'At scale' means it can do many at once."

**Sentence 2:** "The solution is a low-code RPA bot with cloud-based API integration."

*Model answer:* "A robot that copies data from one screen to another. You can set it up mostly by clicking. It talks to other systems over the internet through a defined connection. It runs on the vendor's computers."

**Sentence 3:** "We use retrieval-augmented generation to ground the model in your knowledge base."

*Model answer:* "Before it answers, it looks up the relevant pages from your own documents and uses them as the basis. This reduces made-up answers. It only works if your documents are good and up to date."

**Sentence 4:** "The model is fine-tuned on your domain data."

*Model answer:* "They took an existing general model and trained it further on your examples, so it fits your business better. You pay for that. It also means your data was sent to whoever did the training."

**Sentence 5:** "Our AI provides explainable, transparent decision-making."

*Model answer:* "They claim you can see why it decided what it decided. Ask them to show you, on one real case, right now. If they cannot, the claim is decoration."

**Sentence 6:** "It's a multi-agent system with orchestration."

*Model answer:* "Several AI components work on a task in sequence or in parallel, and something coordinates them. Ask: how many components, what does each one do, and what happens if one fails?"

**Now do your own.** Find one sentence from a vendor email you received in the last month. Translate it using the same method: verb and object, input, output, what breaks. Send your translation back to

the vendor and ask if it is correct. Their reaction will tell you a great deal.

## Checklist

## 3.9 Your minimum glossary

- ☐ I can explain data, information, and knowledge, and I know how much of my business knowledge is written down.
- ☐ I can define algorithm as “a recipe”: an ordered set of steps.
- ☐ I can define model as “the trained thing you use to answer new cases”.
- ☐ I can define training as “adjusting settings against examples until the answers match”.
- ☐ I know an LLM is a text-continuation system, not a database of facts.
- ☐ I know a prompt is the instruction I give the model, and that its quality changes the result.
- ☐ I know what a token is and roughly what tokens cost per typical use.
- ☐ I know what a context window is and what happens when a document is too big for it.
- ☐ I know “hallucination” means a confident false statement, and that it needs verification.
- ☐ I can tell RPA apart from AI, and I know RPA is cheap and brittle.
- ☐ I know no-code and low-code remove programming, not design or maintenance.
- ☐ I know “the cloud” means someone else’s computers, in a specific country, under a specific contract.
- ☐ I know an API is a defined window for one system to use another, and I always ask whether one exists.
- ☐ I know the three levels of integration: files, API, native.

- [ ] I use the plain-word test: a verb, an object, an input, an output, and what breaks.
- [ ] I have started a one-page shared glossary with one named owner.
- [ ] I know that privacy, security, and bias each have a full chapter, and I have read or will read them.

## Key Takeaways

- Most bad AI decisions come from undefined words, not from bad technology.
- A real explanation always contains a verb and an object; if you cannot name one, you have not been told anything.
- Models are structure-followers, not fact-keepers, which is why they transfer skills across subjects and why they state falsehoods confidently.
- The Talkie experiment shows a model with no knowledge of computers can still write a little Python, and it exists because modern benchmark results can be inflated by contaminated data.
- Pin every term down before you sign: what goes in, what comes out, what breaks it, and who owns it.

# Chapter 4 — Ethical AI: Doing the Right Thing

## In Simple Words

Ethical AI is not a philosophy class. It is a set of practical choices you make before you switch a system on.

Here is the plain idea. An AI system makes decisions, or helps people make them. Those decisions touch real people: customers, employees, applicants, patients. When a machine affects a person's life, someone must be answerable for it. That someone is you, the company that chose to use it.

Many owners treat ethics as a luxury. Something for big companies with legal teams, or a marketing line for the annual report. This chapter argues the opposite. Ethics is a necessity, for three simple reasons.

First, **unethical AI costs money**. A biased hiring tool, a chatbot that lies, a system that leaks private data — each one can end in a lawsuit, a fine, or a public loss of trust that takes years to repair.

Second, **unethical AI is usually bad AI**. A tool that discriminates is also a tool that throws away good candidates. A tool that cannot explain itself is also a tool you cannot debug. The ethical fix and the quality fix are often the same fix.

Third, **the law now requires it**. In the European Union, rules on AI literacy and on high-risk systems are already in force or arriving on a fixed calendar. Chapter 5 covers those rules in detail. This chapter covers the thinking behind them.

One sentence runs through everything here: **you cannot hand a machine your responsibility, but you can hand it your work.** The machine does the heavy lifting. You keep the judgement, the oversight, and the blame.

## A Bit of History

Ethics and technology have met many times before, and each meeting produced a rule.

In 1942 the writer Isaac Asimov published his "Three Laws of Robotics" in a science-fiction story: a robot may not harm a human, must obey orders, and must protect itself, in that order. The laws are fiction, but they planted a real question that still drives the field: how do you make a machine behave?

The modern, serious conversation began in **2017 at Asilomar**, in California. Researchers in artificial intelligence met and wrote out a long list of principles for safe and beneficial AI. It was not a law. It was a warning from the people building the technology that the technology needed guardrails.

Institutional work followed quickly. The **IEEE**, a professional body for engineers, began a project on ethical design of autonomous systems. The **OECD** published agreed principles on AI in 2019. In 2021 **UNESCO** adopted a global ethics recommendation, agreed by 193 countries. The European Union moved from principles to binding rules: in 2018 it formed an expert group whose "trustworthy AI" checklist became the basis for the EU AI Act.

Notice the shape of this history. It moved from stories, to principles, to checklists, to law. Each step made the last more concrete and more enforceable. A small business today stands at the end of that line. The

abstract debate is over. What is left is a set of expectations you are meant to meet.

## Curiosity

### 4.7 The law now requires your staff to understand AI

Here is a fact that surprises most owners. Since **2 February 2025**, a European company that uses AI has a legal duty to make sure its people understand it.

This comes from **Article 4 of the EU AI Act**, which is about "AI literacy." In plain words, the law says that providers and deployers of AI systems must take measures to ensure, to their best extent, a sufficient level of AI literacy among their staff and anyone operating an AI system on their behalf.

A **deployer** is simply a company that uses an AI system, as opposed to a **provider** that builds one. Most small and medium businesses are deployers. That means Article 4 applies to you even if you never write a line of code.

The law is careful about what "sufficient" means. It must be tailored to each person's technical knowledge, experience, and education, and to the context where the AI is used and the people it affects. A support agent using a chatbot needs a different level of understanding than the manager who approves its use.

Crucially, the Act states that this obligation **does not require you to guarantee any specific level of literacy for any individual.** It is a duty of effort, not a promise of a result. You must show you took

reasonable steps: training, guidance, clear roles. You do not have to certify that everyone passed an exam.

Why is this interesting for a business owner? Because it turns "AI ethics" from a vague value into a concrete task with a deadline. You can be asked, by a regulator or a customer or a court, "what did you do to make sure your people understood the AI you put in front of them?" "We bought it and hoped" is not an answer. A short training session, a written policy, and a named owner are. The literacy duty is the cheapest part of compliance and the one with the widest effect: a staff that understands a model can be wrong is a staff that catches problems early.

## A Real Business Example

### The hiring tool that learned to dislike women

In July 2018 the news agency Reuters reported that Amazon had built an AI system to screen job applications and it had a serious flaw: it favored men.

The story is a clean lesson in bias. Amazon trained the tool on ten years of resumes it had received. For most of those years, most applicants for technical roles were men. The system learned, from that history, that "a good candidate" looked like the candidates it had already hired — mostly men.

It went further than the obvious signals. It began to penalize resumes that contained the word "women's," as in "captain of the women's chess club." It downgraded graduates of two all-women's colleges. It rewarded language more common in men's resumes, like "executed" and "captured," and punished softer phrasing more common in women's.

Amazon's engineers tried to fix it. They removed the gender-linked terms. But the system kept finding other proxies — indirect signals that stood in for gender. They could not be sure the tool was fair. Amazon eventually disbanded the team and stopped using the system as its main filter.

Two lessons stand out.

First, **the bias was not in the code. It was in the data.** Nobody wrote "prefer men." The model absorbed an imbalance in the past and reproduced it at scale. A human recruiter with the same prejudice affects a few applicants. A model with that prejudice affects every applicant, instantly, and looks objective while doing it.

Second, **the company caught it before it caused a public disaster**, and chose to stop. That was the ethical move, and it was also the sensible one. A tool that quietly rejects good candidates because of their gender loses talent and invites a discrimination lawsuit. The ethical fix and the business fix pointed the same way. If you use AI to rank, filter, score, or sort people in any way, the same trap applies.

## How to Do It

## 4.3 The six principles for responsible adoption

Frameworks are only useful if you can act on them. These six principles are ordered so each one answers a question you must actually resolve before and during a project.

**Principle 1: Decide who is responsible.** Before anything else, name a person. Not a team, not a vendor — a named human who owns this AI system and answers for its outcomes. If you cannot name that

person, you are not ready to deploy. Responsibility that belongs to everyone belongs to no one.

**Principle 2: Understand the impacts and plan.** Write down who this system touches and how. Who is it applied to? What does it decide about them? What is the worst realistic harm? A customer-service bot that gives a wrong refund answer annoys someone. A credit tool that wrongly rejects an applicant denies them money. Plan for the worst case, not just the average case.

**Principle 3: Measure and manage risks.** A risk you cannot see is a risk you cannot fix. Decide how you will test the system before launch and how you will watch it after. Check a sample of its decisions. Track complaints. Set a threshold that triggers a human review. Measurement turns a hope into a control.

**Principle 4: Be transparent and explainable.** People should know when they are dealing with AI, and they should be able to get a plain reason for a decision that affects them. “The model said so” is not a reason. If you cannot explain a decision in a sentence, you should not let that decision stand on its own. See section 4.5.

**Principle 5: Protect privacy and data.** Personal data is the fuel of most AI. Handle it with care: collect only what you need, know where it goes, and never feed customer details into a tool you have not checked. The legal side is covered in Chapter 10; the security side in Chapter 6.

**Principle 6: Keep humans in control.** For any decision that matters, a human must be able to review, override, and stop the system. “Human in the loop” means the human is a real decision-maker, not a rubber stamp. Design the workflow so a person can say no, and so saying no is easy and expected.

Run these six as a checklist on every project. If you cannot satisfy one, stop and fix it before moving on. They are cheap to apply at the start and expensive to retrofit later.

# Ethics and Responsibility

## 4.1 Why ethics is not a luxury but a necessity

Treat ethics as a cost center and you will cut it. Treat it as risk control and you will not. Here is the honest business case.

**The downside of getting it wrong is large and concrete.** Discrimination claims, data breaches, deceptive-output fines, and reputational damage all cost real money and real time. A single biased system that reaches the wrong decision on a protected group can trigger investigations across every case it touched.

**The upside of getting it right is operational, not just moral.** Explainable systems are easier to debug. Fair systems widen your pool of candidates and customers instead of narrowing it. Private-data discipline reduces your exposure to breaches. Ethical design and good design overlap almost completely.

**Trust is your actual product.** Small businesses win on trust. A customer who believes you treat their data and their application fairly is a customer who returns. A customer who suspects a black box made a cold, unexplainable decision about them does not. Ethics is how you keep the trust that small firms depend on and large firms often lack.

So ethics is not a decoration you add when you have spare budget. It is the floor under the whole thing. Skip it and you are not saving money; you are borrowing trouble at a high interest rate.

## 4.2 The four pillars of ethical governance

How do you make ethics stick across a whole company, not just one project? Governance rests on four pillars. Remove one and the structure leans.

**Pillar 1: Principles.** A short written statement of what your company believes about AI — fairness, honesty, human control, privacy. This is your internal compass. It should fit on one page and be read by everyone who touches AI. Principles without the other three pillars are posters on a wall.

**Pillar 2: Regulation.** The external rules you must follow: the EU AI Act, GDPR, consumer-protection law, anti-discrimination law. You do not choose these; they apply to you. Chapter 5 is the deep home for the AI Act, and Chapter 10 for GDPR. Know which rules bind your use cases.

**Pillar 3: Technical standards.** The agreed ways of building and testing that turn principles into practice: testing for bias, logging decisions, documenting models, security standards, quality management. Standards are how "be fair" becomes "run this test and record the result." They make ethics auditable.

**Pillar 4: Self-regulation.** What you do because you chose to, beyond the minimum the law demands: an internal review board, a code of conduct, a person who can veto a harmful use, a habit of asking "should we?" not only "can we?" Self-regulation is the culture that fills the gaps the law leaves.

A healthy company has all four. Principles point the way. Regulation sets the floor. Standards make it measurable. Self-regulation keeps you honest when nobody is watching.

## 4.4 Bias and discrimination: how to recognize and avoid them

**Bias** is when a system treats some people worse than others in a patterned way, not by accident but because of how it was built or trained. When that pattern lines up with a protected characteristic — gender, race, age, disability, religion, and others — it becomes unlawful **discrimination** in many places.

Bias usually enters through the data, as the Amazon example showed. If your past decisions favored one group, a model trained on them learns to favor that group. The model is a mirror. It reflects the examples, including their flaws.

**How to recognize it.** Ask three questions of any system that sorts or scores people:

1. **Who is in the training data, and who is missing?** If a group was historically under-represented, the system will likely serve them worse.
2. **Does the system use proxies?** You can remove the gender field and still have bias, because job titles, schools, hobbies, or gaps in a resume can stand in for gender. Look for signals that correlate with a protected trait.
3. **Are the outcomes uneven by group?** The simplest test: take the system's decisions and group them by gender, age, or ethnicity. If one group is rejected at a much higher rate, investigate why. The gap is the warning light.

**How to avoid it.** Use data that represents the people you actually serve. Test outcomes by group before launch, not after. Keep a human reviewing borderline decisions. Document what you checked, so you

can show due diligence. And remember that removing a label does not remove a proxy — you have to look for the indirect signals on purpose.

Bias is not a moral failing of the machine. It is a failure of the humans who chose the data and skipped the test. That is good news: human failures can be fixed by human process.

## 4.5 Transparency and explainability: why it matters to know how AI decides

**Transparency** means people know when AI is involved and roughly what it is doing. **Explainability** means you can give a plain reason for a specific decision.

They matter for three reasons.

**Trust.** A person accepts a decision more readily when they understand it. “Your application was not selected because the role requires X and your profile shows Y” is hard to accept but easy to understand. “Rejected by AI” with no reason feels arbitrary and invites anger and complaints.

**Control.** You cannot fix what you cannot see. If a model makes a bad decision and nobody can say why, you cannot correct it, and it will happen again. An explainable system lets you trace the cause and remove it. An opaque one hides its own defects until they cause harm.

**Law and fairness.** In several legal settings, including under GDPR for certain automated decisions, a person has a right to an explanation. Beyond the law, an unexplainable decision about a person is hard to defend as fair. If you cannot articulate the reason, you cannot prove it was not biased.

A practical rule: **for any decision that materially affects a person, you must be able to produce a one-sentence reason a non-expert would understand.** If you cannot, that decision needs a human behind it, not a machine in front of them. Be careful with vendors who claim their system is "fully explainable." Ask them to show you, on one real case, in the meeting. A claim you cannot see demonstrated is a claim you should not rely on.

## 4.6 Human responsibility: who is liable if the AI makes a mistake?

This is the question every owner asks. The answer is simple and uncomfortable: **you are.**

An AI system is a tool. Tools do not carry legal responsibility; the people who use them do. If a delivery driver you employ runs a red light, you are answerable as the employer. If an AI you deploy makes a harmful decision, the same logic applies. The machine cannot be sued, fined, or shamed. You can.

A few points that make this concrete:

- **"The vendor's AI did it" is not a defense.** You chose the tool, you put it in front of people, and you kept or dropped the human check. Those are your choices, and they are where responsibility sits.
- **Automation does not transfer liability; it concentrates it.** A human making 50 decisions a day spreads risk. A system making 5,000 identical decisions concentrates it. One flawed rule now harms thousands at once, and you own that rule.
- **Human oversight is your main protection.** Where a qualified human can review and override a decision, your liability drops.

Where you let the system act alone on something important, your liability rises. Keeping humans in control (Principle 6) is not only ethical; it is how you protect yourself.

- **Documenting your process is your evidence.** If challenged, you want to show you assessed the risk, tested for bias, kept a human in the loop, and acted reasonably. A paper trail of good process is your best defense.

The clean way to hold this: **the AI decides fast and at scale; a human decides whether to let it, and takes the blame when it goes wrong.** Never let the convenience of automation blur that line.

## Mistakes to Avoid

**Mistake 1: Treating ethics as a later step.** Bolt it on after launch and you find problems too late and fix them in public.

**Mistake 2: Assuming the vendor handles it.** The vendor builds the tool; you deploy it, and the responsibility for how it touches your people is yours.

**Mistake 3: Thinking removing a label removes bias.** Take out the gender field and the model finds a proxy; look for indirect signals on purpose.

**Mistake 4: Letting "the model said so" be the final answer.** If you cannot explain a decision in a plain sentence, do not let it stand alone.

**Mistake 5: Rubber-stamping.** A human "in the loop" who always agrees with the AI is not oversight; make review real and saying no easy.

**Mistake 6: Skipping the paper trail.** No documentation means no proof you acted responsibly when you are challenged.

**Mistake 7: Confusing a value statement with governance.** A poster about "responsible AI" with no named owner, no test, and no veto is decoration, not governance.

**Mistake 8: Ignoring the AI-literacy duty because it feels soft.** Article 4 is law since February 2025; a trained staff is both a legal answer and your cheapest control.

## Practical Exercise

### 4.8 Assess the ethical risks of an AI project in your company

Pick one AI project you are considering, or one you already run. Work through this on paper. It takes about an hour and will surface most serious risks.

**Step 1 — Name the owner.** Write one name: the person responsible for this system. If you hesitate, that is your first finding.

**Step 2 — Map the people affected.** List every group the system touches: customers, applicants, employees, others. For each, write what the system decides or influences about them.

**Step 3 — Worst realistic harm.** For the most serious decision, write the worst plausible outcome for one real person. Be specific. "A single parent is denied credit they qualify for" is a finding; "a bad outcome" is not.

**Step 4 — Bias check.** Ask the three questions: who is missing from the data, what proxies could stand in for a protected trait, and are outcomes uneven by group. If the system sorts people, you must be able to answer all three.

**Step 5 — Explainability test.** Take one decision the system makes and write the one-sentence reason a non-expert would understand. If you cannot, flag it: this decision needs a human.

**Step 6 — Human control.** Describe exactly where a human reviews, can override, and can shut the system down. If the answer is "nowhere," that is your most urgent fix.

**Step 7 — Data and privacy.** Write where the data goes, who can see it, and whether any personal data leaves your control. Cross-check against Chapter 10.

**Step 8 — Literacy.** Note which staff need training to use this system responsibly, and what you will do about it.

**Step 9 — Decide.** For each red flag, write one action and one owner. A project is ready to proceed when every serious harm has a control and a name attached.

Keep this page. It is the first draft of your ethics record and your defense if you are ever asked.

## Checklist

## 4.9 Your AI ethics policy

- ☐ I have a one-page statement of our AI principles: fairness, honesty, human control, privacy.

- [ ] Every AI system has a single named human owner, not a team or a vendor.
- [ ] For each system, I have mapped who it affects and what it decides about them.
- [ ] I have written the worst realistic harm for each serious decision.
- [ ] For any system that sorts or scores people, I have checked for missing groups, proxies, and uneven outcomes.
- [ ] For every decision that affects a person, I can produce a one-sentence plain reason.
- [ ] A qualified human can review, override, and stop every important decision.
- [ ] I know where our data goes and who can see it, and I never feed unchecked tools personal data.
- [ ] I have a plan to raise AI literacy among the staff who use our systems (Article 4 duty).
- [ ] I know which external rules bind us: the EU AI Act, GDPR, consumer and anti-discrimination law.
- [ ] I use technical standards: testing, logging, documentation, security.
- [ ] I have a self-regulation habit: someone can veto a use that is legal but wrong.
- [ ] I keep a written record of the risk assessment for each system.
- [ ] I understand that the company, not the vendor and not the machine, is responsible for outcomes.
- [ ] I review each system periodically, not only at launch.

## Key Takeaways

- Ethical AI is practical risk control, not philosophy: it saves money, improves quality, and protects the trust small businesses live on.
- Governance rests on four pillars — principles, regulation, technical standards, and self-regulation — and the structure leans if you drop one.

- The six principles for adoption put a named human, a harm plan, a risk measure, an explanation, data protection, and human control on every project.
- Bias comes from the data and hides in proxies; you remove it by testing outcomes by group, not by deleting a label.
- Responsibility never transfers to the machine or the vendor: the company that deploys an AI system owns its outcomes, and a human in the loop plus a paper trail is your best protection.

# Chapter 5 — Rules and Legal Responsibility: The EU AI Act

## In Simple Words

Every new tool brings rules. Cars brought traffic laws. Banks brought money-handling rules. AI is now getting its own rulebook, and if you do business in or with Europe, that rulebook is the **EU AI Act**.

This chapter is the plain-English guide to it. No legal training needed. The goal is simple: you should be able to tell whether a tool you want to use is lightly regulated, heavily regulated, or banned outright, and what you must do about it.

Two ideas explain most of the law.

The first is **risk**. The AI Act does not treat all AI the same. It sorts AI systems into levels by how much harm they could cause. A spam filter is barely touched. A system that decides who gets a loan or a job is watched closely. A handful of practices are simply forbidden. The higher the risk, the more you must do.

The second is **your role**. The law separates a **provider** (who builds or brands an AI system and puts it on the market) from a **deployer** (who uses it). Most small and medium businesses are deployers. Your duties differ from the vendor's, but they are real, and they are yours.

A note on scope. The EU AI Act is European law. If you are outside the EU but your AI affects people in the EU, parts of it can still reach you. Check with a professional if you serve European customers.

## 5.1 GDPR in plain words: what you must know

**GDPR** (the General Data Protection Regulation) is the European law on personal data. It has applied since 2018 and it still governs anything you do with information about identified people — including the data you feed into AI.

In one paragraph: if you collect, store, or process personal data of people in the EU, you need a lawful reason to do it, you must tell people what you use their data for, keep it secure, hold only what you need, let people access or delete it on request, and not keep it longer than necessary. Breaches can bring large fines. AI does not change these duties; it just gives you more ways to break them, because models need data and data is personal. The full, detailed treatment of GDPR — lawful bases, data-subject rights, and how to comply — is the home of Chapter 10. Read that chapter before you process any customer data with AI. The point here is only that the AI Act and GDPR apply together, on top of each other, not instead of each other.

## 5.2 Personal data vs sensitive data: the difference that matters

GDPR draws a line that matters a lot for AI, because the higher category carries stricter rules.

**Personal data** is any information about a living person who can be identified, directly or indirectly. A name, an email, a phone number, a location, an online ID. Even a combination of ordinary facts that points to one person counts.

**Sensitive data** (the law calls it “special categories”) is a smaller set of more delicate facts: racial or ethnic origin, political opinions, religious

or philosophical beliefs, trade-union membership, genetic data, biometric data used to identify someone, health data, and data about a person's sex life or sexual orientation.

Why the line matters for you: processing sensitive data needs a stronger legal ground and more protection. It is also the data most likely to cause discrimination if it shapes an AI decision. Many of the AI Act's strictest rules exist because AI can turn ordinary data into sensitive inferences, or use sensitive data to sort people. For the full definitions and handling rules, see Chapter 10.

## A Bit of History

Europe wrote its privacy rulebook first and its AI rulebook second, and the order explains both.

**GDPR** was adopted in 2016 and became directly enforceable in May 2018. It was the most comprehensive data-privacy law of its time, and because so many global companies deal with Europeans, it set a worldwide standard. Its core idea — that people own their personal data and you need a lawful reason to use it — became the template for privacy laws elsewhere.

For AI, Europe took a principles-first path. In 2018 the European Commission created a High-Level Expert Group on AI, which in 2019 published voluntary ethics guidelines built around "trustworthy AI." The Commission then turned principles into law, proposing the AI Act in April 2021. After years of negotiation, it was published in the Official Journal of the European Union on **12 July 2024** and entered into force on **1 August 2024**. It is the world's first comprehensive, horizontal law on artificial intelligence — "horizontal" meaning it covers AI across all sectors, not one industry at a time.

The law does not start all at once. It switches on in stages, giving businesses time to prepare. That staged calendar is the single most practical thing in this chapter, and it appears in section 5.3. The history shows the direction: from soft ethics guidelines to hard, dated obligations. The debate is over. The clock is running.

## Curiosity

### The first AI law in the world — and fines that beat GDPR

Here is a fact worth knowing before any conversation about "compliance is too expensive."

The EU AI Act is the first comprehensive law on AI anywhere. No other major economy has one that matches its breadth. That makes it a test case, and it also gives it a kind of gravity: companies worldwide often build to the strictest rule they might face, so a European rule can quietly become a global default. People call this the "Brussels effect."

The other striking fact is the size of the penalties. GDPR's top fine is up to 4% of a company's total worldwide annual turnover. The AI Act goes higher for the worst violations. For the prohibited practices in section 5.5, fines can reach **up to €35 million or 7% of total worldwide annual turnover, whichever is higher.** For most other breaches of the Act, the ceiling is around €15 million or 3%. For giving inaccurate information to the authorities, around €7.5 million or 1%. (For small and medium enterprises the law caps fines at the lower of the two figures, so the burden is scaled down — but it is still real.)

The lesson is blunt. The practice the law fears most — the banned uses — carries the heaviest fine in European digital regulation, heavier than

GDPR. That tells you exactly where the line is, and it tells you that "we didn't know" is not a cheap excuse.

## A Real Business Example

### A small lender that scores credit with AI

This example is illustrative. It is a realistic scenario, not a reported company result, built to show how the tiers work in practice.

Imagine a small consumer-lending firm that wants to use an AI model to decide whether to approve a personal loan.

Step one is to find the tier. The AI Act lists categories of high-risk AI in an annex. One category covers **essential private and public services**, and it explicitly includes **creditworthiness evaluation** — deciding whether a person gets credit. So this lending model is **high-risk**. It is not banned, but it sits in the most heavily regulated working tier.

What that means for the firm, in plain terms:

- It must run a **risk-management system** across the whole life of the model, not just once.
- It must govern its **data**: the data used to train and test the model must be relevant, representative, and as error-free as possible for the purpose. A model trained mostly on one kind of borrower will treat others unfairly.
- It must keep **technical documentation** and **automatic logs** of the system's operation.
- It must design for **human oversight**: a person must be able to review and override a decision.
- It must ensure **accuracy, robustness, and cybersecurity**.

- Before placing it on the market for use, it must complete a **conformity assessment**, draw up an **EU Declaration of Conformity**, attach a **CE marking**, and **register the system in the EU database** for high-risk AI.

Now change one detail and the tier changes. Suppose the firm instead uses AI only to draft friendly reminder emails about existing loans, with no decision about approval. That is not a high-risk function. It falls into a light tier, where the main duty is transparency: people should know automated means are involved where required.

Same company, same technology vendor, two very different legal burdens — because the *use* determines the tier, not the tool. That is the whole logic of the Act in one scene.

## How to Do It

## 5.3 The EU AI Act: what changes for companies

The Act sorts AI into four risk levels, plus a separate track for the big general-purpose models behind chatbots.

**Tier 1 — Unacceptable (prohibited).** A small set of practices is banned outright. See section 5.5. If your use falls here, you cannot do it at all.

**Tier 2 — High-risk.** Systems that can seriously harm health, safety, or fundamental rights. These carry the full compliance regime: risk management, data governance, documentation, logging, transparency to users, human oversight, accuracy and security, conformity assessment, CE marking, and registration. The high-risk uses are listed in two annexes: Annex I (AI as a safety component in regulated products) and Annex III (listed sensitive uses).

**Tier 3 — Limited / transparency risk.** Systems that are not high-risk but that interact with people or generate content. The main duty is disclosure: tell people they are dealing with AI, and label synthetic content. See section 5.6.

**Tier 4 — Minimal risk.** Everything else — spam filters, inventory forecasting, most internal productivity tools. No new obligations beyond existing law.

The **Annex III high-risk categories** are the ones a business owner is most likely to meet. They include: biometrics (where not banned); safety components of critical infrastructure; education and training (admissions, evaluation of learning); **employment and worker management** (recruiting, filtering applications, promotion, termination, task allocation, performance monitoring); **essential services** (eligibility for benefits, creditworthiness, emergency-call triage, insurance risk and pricing); law enforcement; migration and border control; and administration of justice and democratic processes.

A useful exception: an Annex III system is **not** high-risk if it only does a narrow procedural task, improves something a human already did, detects patterns without replacing human assessment, or does a purely preparatory step. But it is **always high-risk if it profiles a person.** Profiling means using data to evaluate or predict aspects about an individual.

**Your role decides your duties.** A provider carries the heavy build-side obligations. A deployer (most businesses) carries use-side duties: use the system as instructed, ensure human oversight, monitor, keep certain logs, and in some cases run a fundamental-rights check. If you only *use* a vendor's high-risk tool, you do not do the conformity assessment — the provider does — but you still must deploy it correctly and responsibly.

**The staged timeline.** Memorize these dates if you touch AI in Europe:

| Date | What applies |
|---|---|
| 1 Aug 2024 | Act enters into force |
| 2 Feb 2025 | Prohibited practices (Art. 5) and AI-literacy duty (Art. 4) apply |
| 2 Aug 2025 | Rules for general-purpose AI models, governance bodies, and penalties framework |
| 2 Aug 2026 | High-risk obligations for Annex III uses (employment, credit, education, essential services, etc.) |
| 2 Aug 2027 | High-risk obligations for Annex I products (AI safety components in regulated products) |

The EU has proposed some simplifications since, sometimes called a "digital omnibus," and dates or details can shift. Treat the table as the working baseline and confirm current dates against the official text before you rely on a deadline.

## 5.4 Obligations for high-risk systems: conformity assessment and technical documentation

If your use is high-risk, these are the concrete things required. You do not need to perform all of them yourself as a deployer, but you must understand them to choose a vendor and to deploy correctly.

**Risk management.** A continuous process, across the system's whole life, that finds, assesses, and reduces risks to health, safety, and fundamental rights. Not a one-time form.

**Data governance.** The training, validation, and testing data must be relevant, representative, and as free of errors and complete as the purpose allows. This is where bias is caught or missed.

**Technical documentation.** A written record showing the system complies and letting authorities check it. It covers how the system works, what it was trained on, how it was tested, and how to use it. Keep it, along with any assessment decisions and approved changes.

**Record-keeping (logging).** The system should automatically record events relevant to risks and to changes, so its behavior can be reviewed later. Logs are your evidence.

**Transparency to deployers.** Clear instructions for use so the people operating the system can comply with their own duties.

**Human oversight.** The system must be designed so a human can understand it, monitor it, interpret its output, and step in or stop it.

**Accuracy, robustness, cybersecurity.** The system must hit appropriate levels of all three and be resilient to error and attack.

**Quality management and accessibility.** A system to keep compliance going over time, and accessibility requirements met.

**Conformity assessment, CE marking, registration.** Before a high-risk system is placed on the market, the provider runs a **conformity assessment** (an independent check by a "notified body" where the law requires it). On passing, the provider draws up an **EU Declaration of Conformity**, attaches a **CE marking**, and

**registers the system in the EU database**. As a deployer, you should ask to see these before you buy. If a vendor cannot show a conformity path for a high-risk use, that is a serious red flag.

## 5.5 Prohibited practices: what you absolutely cannot do with AI

These are banned across the EU since 2 February 2025. If a use case matches, drop it. The fines here are the heaviest in the law.

1. **Subliminal or manipulative techniques** that go beyond a person's awareness and materially distort their behavior in a way that causes, or is likely to cause, significant harm.
2. **Exploiting vulnerabilities** of people because of age, disability, or social or economic situation, to distort their behavior and cause significant harm.
3. **Social scoring** by public authorities — systematically scoring people on social behavior or personal traits in ways that lead to harmful treatment unrelated to the context, or disproportionate to the behavior.
4. **Untargeted scraping of facial images** from the internet or CCTV to build facial-recognition databases.
5. **Emotion recognition in the workplace and in education** — inferring the emotions of workers or students, except for medical or safety reasons. (A tool that "reads" a call-center agent's mood to score them is squarely in this ban.)
6. **Biometric categorisation that infers sensitive attributes** — using biometric data to guess race, political opinions, trade-union membership, religious or philosophical beliefs, sex life, or sexual orientation.
7. **Real-time remote biometric identification in public spaces for law enforcement**, banned except for a few narrow,

authorized situations such as searching for missing persons or preventing a specific imminent serious threat.

Most small businesses will never touch these. But two catch people by surprise: **emotion recognition at work** and **manipulative design that exploits vulnerability**. If a vendor pitches "AI that detects customer or employee emotions," know that in a workplace or school setting it is prohibited.

## 5.6 Transparency obligations: when you must inform customers and employees

For systems that are not high-risk but that interact with people or create content, the main duty is to be open. These transparency rules apply from 2 August 2026.

**Tell people they are talking to AI.** If an AI system interacts directly with a person — a chatbot, for example — that person must be made aware they are dealing with an AI, unless it is obvious. A customer-service bot should say so.

**Label synthetic and deepfake content.** AI-generated or manipulated audio, images, video, or text must be marked as artificially generated or manipulated. This targets deepfakes and machine-made media.

**Disclose emotion recognition and biometric categorisation.** Where such systems are lawfully used (outside the banned workplace and education cases), the people exposed to them must be informed.

A small grace note: systems already placed on the market before 2 August 2026 have a short window into late 2026 to meet the synthetic-content labeling rule. Plan to comply regardless.

For a business, the practical move is simple and cheap: put a clear notice in your chatbot ("You're chatting with an automated assistant"), label any AI-generated media you publish, and never run hidden emotion or biometric analysis on people.

## 5.7 Documenting important decisions

Across every tier, one habit protects you: **write down your decisions and your reasoning.**

Good documentation is not bureaucracy. It is your memory and your defense. When a regulator, a customer, or a court asks why you used a system a certain way, the written record is your answer.

For each AI system, keep a simple file that records:

- **What it does and who owns it.** The use case, the tier you placed it in, and the named owner.
- **Why you chose it.** The business reason and the alternatives you considered.
- **How you assessed risk.** The ethical and legal risk review, including any bias check and any fundamental-rights check.
- **What controls you put in place.** Human oversight, logging, transparency notices, data limits.
- **What you tested and when.** Test results, dates, and who reviewed them.
- **Changes over time.** Any modification to the system or its use, with the reason and the date.
- **Incidents and fixes.** Anything that went wrong and what you did about it.

This file connects directly to the ethics work in Chapter 4. The risk assessment you do there becomes part of your legal record here. One

document, two purposes.

A rule of thumb: **if it mattered and you did not write it down, be prepared for it to be questioned and for you to have no answer.**

## Ethics and Responsibility

Compliance is the floor, not the ceiling. Meeting the letter of the AI Act does not make a use ethical, and it does not remove your responsibility. The law tells you what you must not do and what high-risk uses require. It does not tell you whether a use is fair or wise. That is your judgement, covered in Chapter 4. A system can be fully compliant and still harm trust if you deploy it carelessly.

Your role shapes your duty. As a **deployer**, you are responsible for using a system as instructed, keeping a human in control where required, monitoring how it behaves, and being honest with the people it touches. You cannot point at the vendor and walk away. A practical stance: **treat every legal requirement as the minimum, and let your own ethics set the standard above it.** When you can show you did more than the minimum, you gain credibility — and for a small business, credibility is worth more than any single automation.

## Mistakes to Avoid

**Mistake 1: Assuming your tool is "just a chatbot" and unregulated.** The tier depends on what it decides, not what it is called. A chatbot that influences credit or hiring is high-risk.

**Mistake 2: Not knowing your role.** Provider and deployer have different duties. If you only use a vendor's high-risk tool, you still have deployer duties. Know which you are.

**Mistake 3: Missing the staged dates.** The prohibitions and the AI-literacy duty are already live (February 2025). High-risk duties arrive in 2026 and 2027. Do not treat the whole law as "later."

**Mistake 4: Buying a high-risk tool with no conformity path.** If a vendor cannot show a conformity assessment, a Declaration of Conformity, and registration for a high-risk use, do not buy it.

**Mistake 5: Running emotion recognition at work.** It is prohibited in workplaces and schools. A vendor pitch for "mood detection" on staff is a trap.

**Mistake 6: Forgetting the AI-literacy duty.** Article 4 is law now. You must take reasonable steps to train the people who use your AI.

**Mistake 7: Confusing GDPR with the AI Act.** They are separate laws that stack. Complying with one does not cover the other. Read Chapter 10 for the privacy side.

**Mistake 8: Not documenting.** No written record means no defense when you are asked to explain a decision.

**Mistake 9: Treating compliance as the finish line.** The law is the floor. Ethics and trust sit above it, and you still own them.

## Practical Exercise

## 5.8 Your compliance checklist

Take one AI system you use or plan to use and classify it. This takes about an hour and tells you which rules bind you.

**Step 1 — Is it prohibited?** Read the seven banned practices in section 5.5. If your use matches any, stop. Do not proceed.

**Step 2 — Is it high-risk?** Check the Annex III categories: biometrics, critical infrastructure, education, **employment and worker management**, **essential services (credit, benefits, insurance)**, law enforcement, migration, justice. Does your use fit one? If yes, it is high-risk unless the narrow exception applies — and it is always high-risk if it profiles a person.

**Step 3 — Is it a transparency case?** Does it interact with people or generate content? If yes, you must disclose AI use and label synthetic content.

**Step 4 — Minimal?** If none of the above, you are in the minimal tier. No new AI-Act duties, but existing law and your own ethics still apply.

**Step 5 — What is your role?** Provider or deployer? Write it down. List the duties that follow for that role.

**Step 6 — If high-risk, gather the vendor's proof.** Ask for the conformity assessment, the EU Declaration of Conformity, the CE marking, and the EU database registration. Note what is missing.

**Step 7 — Map your deployer duties.** Human oversight, monitoring, logging, transparency notices, and any fundamental-rights check that applies to you.

**Step 8 — Check the calendar.** Which of your obligations are already live (literacy, prohibitions) and which arrive in 2026 or 2027? Put dates on them.

**Step 9 — Open the file.** Start the documentation file from section 5.7 with your classification and your plan.

If you cannot place a system into a tier with confidence, that is itself the finding: get expert help before deploying.

## Checklist

## 5.9 Your legal policy for AI

- ☐ I know the four risk tiers: prohibited, high-risk, transparency, minimal.
- ☐ I can name the Annex III high-risk categories, especially employment and essential services (credit, insurance).
- ☐ I know that profiling a person makes an Annex III use always high-risk.
- ☐ I know the seven prohibited practices and that emotion recognition at work/school is one.
- ☐ I know my role for each system: provider or deployer.
- ☐ I know the staged dates: prohibitions and AI literacy live since 2 Feb 2025; high-risk Annex III in Aug 2026; Annex I products in Aug 2027.
- ☐ For high-risk tools, I require the vendor's conformity assessment, Declaration of Conformity, CE marking, and EU database registration.
- ☐ I disclose AI interaction to customers and label AI-generated content.
- ☐ I keep human oversight and logging where the tier requires it.
- ☐ I have a plan to meet the AI-literacy duty for staff who use our systems.
- ☐ I understand GDPR applies on top of the AI Act, and I have read Chapter 10.
- ☐ I keep a documentation file for each system: use, tier, owner, risk assessment, controls, tests, changes, incidents.

- [ ] I treat compliance as the floor and my own ethics as the standard above it.
- [ ] I get expert help for any system I cannot confidently classify.

## Key Takeaways

- The EU AI Act sorts AI by risk — prohibited, high-risk, transparency, minimal — and the higher the risk, the more you must do.
- Employment and essential services like credit and insurance are high-risk; profiling a person always makes an Annex III use high-risk.
- Seven practices are banned outright, including emotion recognition in workplaces and schools, and they carry the heaviest fines in European digital law.
- The law switched on in stages: prohibitions and the AI-literacy duty are already live, and high-risk duties arrive in 2026 and 2027.
- As a deployer you still carry real duties — oversight, transparency, literacy, and documentation — and compliance is the floor, not the end of your responsibility.

# Chapter 6 — Cybersecurity in the AI Era

## In Simple Words

For most of the history of business computing, security meant one thing: keep strangers out of your building. You had a locked door, a password, a firewall. The bad person was outside. Your data was inside.

Artificial intelligence has blurred that line. Today, when an employee types a customer list, a contract, or a salary sheet into an online AI tool, that text leaves the building. It travels to computers that belong to someone else. It may be stored there. It may be read by support staff. It may be kept for months. In some cases it may be used to make that company's product better. Most people never think about this, because the tool looks like a search box. It is not a search box. It is a service, run by a company, on machines you do not control.

AI changes the security picture in two directions at the same time.

**The attackers got better.** A language model gives a criminal three things that used to be expensive: speed, scale, and polish. A phishing email that once looked like a bad translation can now be written in fluent business German or Japanese in seconds, personalised with details from a public profile. A voice can be copied from a few seconds of a recording, which is how finance clerks in reported fraud cases approved transfers because the caller sounded like their own boss. None of this requires a genius. It requires a subscription.

**Your attack surface got bigger.** "Attack surface" is just a term for the number of places someone can get in. Every AI feature you add is a

new door: chat tools, plugins that reach your email, agents that read folders and send messages, models trained on your records, vendors that keep a copy of every prompt and every answer. Two years ago your company had one front door. It may now have forty, and most of them were opened by your own staff without telling anyone.

One more idea organises everything else. Security protects three things, known by three short words.

- **Confidentiality.** Only the right people see the data.
- **Integrity.** The data has not been secretly changed.
- **Availability.** The system works when you need it.

Most owners treat AI security as only a confidentiality problem. It is not. A competitor poisoning your model is an integrity problem. A vendor whose systems go down and stop your payments for weeks is an availability problem, and that one caused the most damage in the story later in this chapter. When you look at any AI tool, ask all three questions.

One warning. The thought “we are too small to be attacked” is not true. Modern attacks are automated: software scans the whole internet for unlocked doors, the way a thief tries every car in a car park. You are not chosen because you are interesting. You are hit because you were easy, and because you hold customer details, bank details, and tax records with very little protection around them.

The good news is that most of the protection you need is not high technology. It is a short list of boring habits, a map of where your data goes, and a few controls in the right place.

The legal side of personal data is in Chapter 10. Staff using AI tools without approval — “shadow AI” — is in Chapter 9. The step-by-step build is in Chapter 20.

## A Bit of History

Security history repeats one lesson: the perimeter moves, and the defenders are slow to notice.

In **1988** the Morris worm disabled something like ten percent of the early internet's connected computers. It was written by a graduate student, not a criminal, and it proved that a self-spreading program could cross the planet in hours. Through the **1990s and 2000s** the standard answer was the perimeter model: a firewall at the edge, antivirus on each PC, data on a server in a cupboard. It worked as long as the data stayed where you put it.

In **2013** the US retailer Target was breached. The attackers did not break into Target. They broke into a small company that supplied heating and cooling equipment for its stores, took that company's login details, and used them to reach Target's payment systems. Around 40 million card numbers were stolen. The lesson was about connection, not code: your risk now includes everyone you plug into.

In **2016** Microsoft put a chatbot called Tay on social media. Within a day, users taught it to post offensive material by feeding it repeated messages, and it was taken down. Nobody broke the program. They broke what the model was reading. That is the seed of prompt injection, although it had no name yet.

In **2017** NotPetya malware spread from Ukrainian accounting software and caused damage measured in the tens of billions of dollars worldwide, hitting shipping giant Maersk, pharmaceutical maker Merck, and many others. One poisoned link in a software supply chain became everyone's problem at once.

In **2020** remote work removed the physical perimeter almost overnight. In **2022** ChatGPT made generative AI a normal office tool within weeks, and staff began pasting real work into tools with no company contract behind them. In **September 2022** the security researcher Simon Willison published a post titled “Prompt injection attacks against GPT-3” and gave a name to a problem people had been seeing but could not describe: instructions hidden inside text that a model reads, which override the instructions you gave it.

In **2023** open-weight models meant a company could run AI on its own hardware, as Chapter 8 explains. The same year, vendors began shipping “agents” that can act rather than only answer, which raised the cost of every mistake. In **2024** vendor risk became impossible to ignore, as told in the Curiosity section below. From **2025** onward, agents arrived with tools and credentials attached, so the blast radius of one wrong permission is now everything that agent can reach.

Read the timeline as one long argument. Each step moved trust away from a locked door and into a relationship: a contractor, a vendor, a plugin, a model, an agent. Security stopped being a wall and became a question about who, and what, you have connected to.

## Curiosity

### 6.8 The vendor that nearly stopped a healthcare system

On 12 February 2024, attackers got into Change Healthcare, a company that processes medical bills and payments in the United States. According to testimony that UnitedHealth Group’s chief executive gave to the US Congress in May 2024, they entered through a remote-access portal that did not require multi-factor authentication — the second

check on a login that stops most stolen passwords from working. There was no clever malware at the front door. There was a stolen password and a missing second lock. The attackers spent nine days inside, moving quietly and copying out around six terabytes of data. On 21 February they switched on ransomware and locked systems.

Change Healthcare is a subsidiary of UnitedHealth Group, and it sits in the middle of the American healthcare payment chain. When it stopped, a very large share of doctor claims, pharmacy payments, and insurance authorisations stopped with it. Small clinics could not get paid and ran out of cash within weeks; industry groups reported provider losses running to tens of millions of dollars a day across the affected network. Big names that had nothing to do with the breach — chain pharmacies, regional insurers, a naval hospital — found their own operations blocked, because they all passed data through the same middleman.

UnitedHealth later advanced more than $2 billion in emergency loans to affected providers by mid-March, and more than $6 billion by mid-April. Reporting by Reuters and Wired said a ransom of roughly $22 million in bitcoin was paid to a wallet linked to the criminal group ALPHV, also known as BlackCat, which UnitedHealth confirmed on 29 February as the attacker. In October 2024 UnitedHealth estimated that about 190 million individuals had their data affected, one of the largest breaches ever recorded.

Why does this matter to a small business that will never touch a medical claim? Because of who the real victim was. The hospitals and pharmacies that suffered did not get hacked. Many of them had good security. They were hurt because a company in the middle of their workflow got hacked, and they could not survive without that company. Their security was only as good as the least-protected vendor in their chain.

Now add the analytics angle, because it is closer to home than it looks. In 2024, OpenAI disclosed an incident that was not a breach of its own systems at all. The problem was in Mixpanel, a third-party analytics service used on the developer site for its application programming interface. Analytics services are small pieces of software a company puts on its website to count visits and see how people click. That vendor was breached, and some account holders' names, email addresses, and device details were exposed. Chat content, passwords, and secret keys were not.

The damage there was limited. The lesson is not. When you buy an AI service, you also buy every company that service relies on: its analytics, its hosting, its support tools, its plugins. You inherit a supply chain you never chose and may never have heard of.

## A Real Business Example

### The engineer who asked the chatbot for help

In April 2023 Bloomberg reported that Samsung had banned generative AI tools such as ChatGPT on company equipment. The reason was not a theory about risk. Engineers had pasted confidential source code into a public chatbot while looking for a bug fix, on more than one occasion, in a division working on semiconductors — one of the most closely guarded kinds of code in the world. A memo reviewed by Bloomberg reportedly noted that an internal survey found 65 percent of respondents already believed the tool was a security risk.

Look at this from the engineer's side. Nothing was stolen. Nobody broke in. A person was stuck on a problem, found a tool that solved it in ten seconds, and used it. The tool was excellent. The judgement was poor. The code was not theirs to give away.

This is the single most common AI security failure in existence, and it is not exotic. It happens in law firms when someone pastes a client's contract to summarise it. In accountancy practices with a tax return. In agencies with a client's unreleased campaign. In HR with a stack of CVs. The pattern is always the same: a helpful tool, a rushed person, and a copy-paste that crosses a boundary nobody drew.

Two things follow. First, the fix is not only a ban, because a ban with no alternative does not stop the behaviour; it hides it, which is worse, since now you cannot see the data flowing. Second, the fix is not only training, because a person in a hurry at five in the evening will not remember a forty-page policy. The fix is to make the safe path the easy path: an approved tool that is faster than the unsafe one, plus a control that catches the worst data before it leaves. Both are in the next section.

## How to Do It

## 6.7 How to protect yourself: AI firewalls, filters, monitoring

Do this in order. Each step depends on the one before it.

**Step 1: Draw the map before you buy anything.** Take one sheet of paper. List every AI tool that touches your company, including free ones on staff phones. For each: what data goes in, what comes out, where it is stored, who the vendor is, and whether you have a signed contract. You will find tools you did not know about. This is the most valuable security document you will produce this year.

**Step 2: Put one door between your people and AI services.** An "AI gateway" or "AI firewall" is a single point all AI traffic passes through, so you can see it and control it. Mainstream products do this:

content-safety and prompt-shielding services from the big cloud platforms, guardrails built into cloud model services, AI-aware gateways, and data-loss-prevention tools that inspect what employees send. You need to know the category exists and what it buys: one place to allow or block tools, inspect content, and keep logs.

**Step 3: Filter what goes out.** The highest-value control on this list. Configure your gateway or data-loss tool to block or warn on the categories that must never leave: national ID and tax numbers, bank and card numbers, health information, payroll files, signed contracts, source code, and client-confidential documents. A pop-up that says "this looks like personal data, are you sure?" stops a large share of accidents by itself, because most of them are accidents.

**Step 4: Filter what comes in.** This is the prompt-injection defence, and it has one golden rule: **never let an AI tool act on instructions it finds inside a document, an email, a webpage, or a spreadsheet.** A model that reads your inbox should summarise, not obey. Where a tool must act, require a human to confirm before it runs. Treat every external document as untrusted input, the way you treat an attachment from a stranger.

**Step 5: Give agents the least power possible.** Decide an agent's permissions on paper before you connect it. Read-only where you can. Its own service account, never a shared administrator login. One folder, not the whole drive. A spending cap on anything that costs money. A time limit. A named human owner. If it does not need to delete, do not let it delete. Give the tool the smallest set of keys that still lets it do the job.

**Step 6: Log everything and read the logs.** Record who used which AI tool, when, and roughly what for — then review it. A weekly fifteen-minute look shows you unapproved tools, unusual volume, and

patterns you can turn into policy. No logs means no investigation: after an incident you will be guessing.

**Step 7: Keep a human checkpoint on anything irreversible.** Any action that cannot be undone — sending money, deleting records, emailing a customer list, signing something — needs a human approval step. Not a notification. An approval.

**Step 8: Do not neglect the boring basics.** AI does not replace ordinary security; it sits on top of it. Multi-factor authentication on every account that can reach company data, including vendor portals. A password manager so nobody reuses passwords. Prompt updates. Tested backups kept offline. Endpoint protection on every machine. If these are weak, no AI-specific control will save you.

**Step 9: Train with one rule, not a manual.** Give staff one sentence they can carry: *if you would not put it in an email to a stranger, do not put it in an AI tool.* Then tell them what to do instead, and make that alternative easy. Rules without alternatives get ignored.

**Step 10: Plan for the vendor being down.** Ask every AI vendor: what happens if you are unavailable for a week? Have a manual fallback for any process you have made dependent on them. Availability is a security property, and the healthcare story above is the proof.

## Ethics and Responsibility

Security is an ethical matter before it is a technical one. When customers give you their address, their health detail, or their payment information, they trust you to hold it safely. Losing it is not an accident that happened to you; it is a harm done to them. That is the same duty of care discussed in Chapter 4, applied to systems rather than to decisions.

Three duties follow. **Duty of competence:** deploying a tool you do not understand is not neutral. If you cannot say where your data goes, you cannot protect it, and "we did not know" is a weak defence in front of a regulator, a customer, or a court. The literacy duty in the EU AI Act, covered in Chapter 5, makes this explicit. **Duty to disclose:** if you are breached, tell the people affected promptly and clearly. Hiding it to protect your reputation transfers the cost of your problem onto the people who trusted you; the legal deadlines are in Chapter 10. **Proportionality in monitoring:** logging AI use protects the company, and it also means reading what your staff type. Set a written policy saying what is logged, why, who can see it, and for how long. Monitor the data flow, not the person. Secret surveillance damages trust and often breaks local law.

One last rule: do not use AI tools to attack other systems, and do not test your own defences on live data or live customer accounts. Use a separate test environment.

## Mistakes to Avoid

1. **Treating an AI tool like a search engine.** A search engine indexes public pages. An AI service receives your text, stores it, and processes it on someone else's computers.
2. **Granting broad permissions "to make it work."** Convenience today is a breach tomorrow. Grant the minimum and widen only on a proven need.
3. **Letting an AI read everything.** Pointing an assistant at a shared drive that holds payroll, legal files, and client records turns one compromised account into total exposure.
4. **No logging.** Without records you cannot find out what happened, or prove you were in control.

5. **Accepting "enterprise-grade" as an answer.** Ask instead: do you train on my data, how long do you keep prompts, who are your sub-processors, where is data stored, will you tell me if you are breached.
6. **Ignoring the free consumer plan.** Staff on personal accounts means your data goes somewhere with no contract and no controls.
7. **Blocking everything.** A ban with no approved alternative does not stop usage; it hides it.
8. **Confusing confidentiality with integrity.** Poisoning and tampering are not leaks. Guard only against data leaving and you will miss data being corrupted.
9. **Trusting a vendor because they are big.** The largest healthcare payment breach in history started with a missing second login factor.
10. **Treating security as a one-time project.** Threats, tools, and staff change. Review the map every quarter.

## Practical Exercise

### 6.9 Map your vulnerability points

Set aside ninety minutes with one colleague who knows the daily work. Do not do this alone.

**Part A — List the data (20 minutes).** Write down every category of sensitive information your company holds. Typical rows: customer names and contacts, payment or bank details, national ID or tax numbers, employee and payroll records, CVs and applicant data, health or insurance information, contracts and quotes, source code or designs, strategy and financial plans.

**Part B — Trace each one (40 minutes).** For each row answer five questions in writing:

1. Where does it live? (system, folder, spreadsheet, paper)
2. Who can reach it? (roles, named people, contractors)
3. Which AI tool touches it, if any?
4. Does it leave our building? Where does it go, and under what contract?
5. Can we see it happening? (logging, yes or no)

**Part C — Score and pick (30 minutes).** Mark each row:

- **Red** — sensitive data that leaves the company with no contract and no logging.
- **Amber** — sensitive data that stays inside but is widely reachable, or leaves under a contract you have not read.
- **Green** — low-sensitivity data, or sensitive data with a contract, a filter, and logs.

Every red row is an action for this month. Pick the top five and write one owner and one deadline next to each. A typical first five: multi-factor authentication on vendor accounts, an approved AI tool for staff, a data-loss rule for ID numbers, shared-drive access cut down to the folders each role needs, and a one-page acceptable-use rule. Keep the map on one page, and update it every quarter and whenever you add a new AI tool.

## Checklist

## 6.10 The 10 minimum security actions

- ☐ **Every account has multi-factor authentication**, including vendor portals, email, banking, cloud storage, and administrative logins.
- ☐ **A written map exists** of every AI tool in use, what data goes into each, and where that data is stored.
- ☐ **An approved AI tool exists and is easier to use** than the unapproved alternatives.
- ☐ **Outbound filtering is in place** for ID and tax numbers, payment details, health data, payroll, contracts, and source code.
- ☐ **No AI tool acts on instructions found inside documents it reads**; actions need human confirmation.
- ☐ **Every AI agent runs with least privilege**: its own account, read-only where possible, capped spending, no administrator rights.
- ☐ **Logs exist for AI use, and someone reviews them weekly.**
- ☐ **Irreversible actions require a human approval step**, not just a notification.
- ☐ **Vendor due diligence is done in writing**: training on your data, retention period, sub-processors, data location, breach notification terms.
- ☐ **Backups are tested and kept offline, and a manual fallback exists** for any process that depends on an AI vendor.

## Key Takeaways

- AI makes attacks cheaper and faster, and it multiplies the number of doors into your business; both change at once.
- The most common leak is not a hack — it is a helpful employee pasting confidential data into a tool they never checked.

- Guard all three properties: confidentiality (data leaving), integrity (data being poisoned), and availability (a vendor going down).
- The core defences are a data map, one controlled gateway, outbound filtering, least privilege for agents, and logs someone actually reads.
- Your risk includes your vendors' risk, and their vendors' risk too; ask who they rely on before you rely on them.

# Chapter 7 — Trustless: Trust Without Trusting

## In Simple Words

The word “trustless” is badly chosen. It sounds like a world with no trust, where nobody believes anything. That is not what it means, and the real idea is much more useful.

Trustless means: **you do not have to trust a person or an institution, because you can verify the fact yourself.**

Start with a familiar contrast. You hire a removal company. You hand over a deposit. You now depend on them showing up. You must trust them, or trust their brand, or trust the law to punish them if they do not. Now think of a vending machine. You put in coins, the drink falls out. There is no relationship, no promise, and no need to believe anyone. The machine enforces the deal by how it is built. That is the feeling of trustless: the agreement is enforced by a system instead of by a person’s good intentions.

Three tools make this possible.

**Verification instead of promises.** Instead of someone telling you a thing is true, you check it. A digital signature is the common example. It proves that a file came from whoever holds a particular key, and that the file has not changed since it was signed. You do not need to trust the sender. You check the signature.

**Transparency instead of secrecy.** Instead of keeping a record in one private notebook, you keep it somewhere many people can read

and nobody can quietly rewrite. If everyone can see the record, one party cannot change history to suit itself.

**Automatic enforcement instead of hope.** Instead of agreeing terms and hoping they are met, you write the terms so a piece of software carries them out. Money is released when the condition is met. Nobody has to chase an invoice.

Now the honest warning, said early: trustless does not mean risk-free. You have moved your trust from people to systems, and systems are built by people. Code has bugs. The feeds that tell a system what happened in the real world can be wrong or lied to. Keys get lost, and a lost key can mean lost money. The goal is not to remove trust. It is to place trust somewhere you can inspect, and to reduce how much of it you need.

Why bother at all? Because trust is expensive. Every intermediary you rely on — a bank, an escrow agent, a broker, a notary, a platform that holds funds until the job is done — takes a cut and takes time. Trustless systems remove some of that, and make what is left cheaper, faster, and visible. This is not only for banks and programmers; it has direct use in any company that buys from suppliers, hires freelancers, and needs records its accountant can trust.

The wider question of who controls your digital tools is in Chapter 11. Whether it pays back is a Chapter 16 question. The security side of verification is in Chapter 6.

## A Bit of History

**2008.** In October a paper titled “Bitcoin: A Peer-to-Peer Electronic Cash System” was published under the name Satoshi Nakamoto. Its stated aim was electronic payment without trusted third parties. The

trick was a shared, ordered record — a blockchain — kept together by many independent computers, so that no single one of them can rewrite it.

**2009.** The Bitcoin network began running in January. For the first time, two strangers could settle value with no bank in the middle, and neither had to trust the other.

**2015.** Ethereum arrived and added the key upgrade: smart contracts. A smart contract is a program stored on the network that runs exactly as written when its conditions are met. Money could now be tied to a rule.

**2016.** A project called The DAO held funds in a smart contract and was drained because of a flaw in that contract. The fallout split the community. The lesson was sharp and still true: automatic enforcement enforces bugs with the same loyalty it enforces features.

**2017 to 2021.** Decentralised finance grew into a real sector: lending, trading, and settlement run by contracts instead of banks, with stablecoins — tokens meant to hold a steady value — as its working money. It also showed the same failures at a larger scale: bad code, wrong price feeds, dishonest operators. In **2019** the W3C, the body that sets web standards, published Verifiable Credentials: a way for an authority to issue a digital claim — a degree, a licence, an age proof — that anyone can check without telephoning the issuer. That is the identity half of the picture.

**2021 to 2023.** A standard called ERC-4337, known as account abstraction, was proposed in 2021 and later finalised. It lets an account be a small program instead of a single private key, so the account can hold its own rules: spending limits, a list of allowed payees, a second signature for large amounts, and recovery if a key is lost.

**May 2025.** An Ethereum upgrade called Pectra went live on the main network. It let an ordinary wallet address point to smart-contract code, bringing transaction batching, sponsored fees, and better recovery to addresses that previously had none. It also raised the maximum stake a single validator can hold.

**August 2025.** A draft standard called ERC-8004 appeared, titled "Trustless Agents." It was written for the problem this chapter is really about: software agents that deal with other people's agents.

Read the timeline as one long argument. Each step moved enforcement out of a person's hands and into a checkable system: first money, then agreements, then identity, then permissions, and most recently agents.

## Curiosity

## 7.6 Ethereum is building plumbing for an agent economy

In August 2025 a draft standard called **ERC-8004** was published on Ethereum's official standards site. Its title is "Trustless Agents," and its stated purpose is to let software agents discover, choose, and work with other agents across organisational boundaries without any prior trust.

It defines three registers, all on-chain.

**Identity.** Each agent gets an on-chain identifier, built on the common ERC-721 token standard, that points to a registration file. That file holds the agent's metadata, the addresses you can talk to, and which trust models it supports. In plain terms: an agent can say who it is, who it acts for, and how you can check it, and you can look that up instead of taking its word.

**Reputation.** A standard way to post and read feedback about an agent, so that performance history is not locked inside one platform's private star rating. Anyone can contribute signals, and the aggregation can happen off-chain. The point is portability: an agent's record follows it, instead of being reset every time it changes marketplace.

**Validation.** Hooks for getting independent checking of an agent's work. The standard lists several methods: having another party re-run the job with money at stake, using zero-knowledge machine-learning proofs — a way of proving a computation was done correctly without revealing the data or the model behind it — using trusted hardware enclaves, or using a trusted human judge.

Why should a small business owner care about a draft standard? It shows where the industry thinks the hard problem is: not "can an agent do the task" but "how do I know whether to believe this agent" — a business question, not a technical one. If it succeeds, you will be able to check a public record instead of trusting a sales deck, which changes who holds the power in the conversation. And the same three questions are a useful checklist today, with no blockchain at all: *Who are you, and who do you act for? How have you behaved before? Can your work be checked independently?*

Two other pieces of the same plumbing are already live. Account abstraction, the ERC-4337 standard, means an agent can hold a budget with rules enforced by code: a hard spending cap, a whitelist of allowed payees, a second human signature above a threshold. And since the Pectra upgrade in May 2025, an ordinary address can point to such code. That combination — an agent that can pay, inside a box it cannot leave — is the practical shape of a trustless agent economy.

Treat all of this as direction, not as a product to buy this quarter. ERC-8004 is a draft. The concepts, however, are usable now.

## A Real Business Example

### The mango that took seven days to trace, then two seconds

In 2017 Walmart and IBM ran a pilot in food supply chains, and the numbers they published became famous in the industry. Tracing the origin of a mango sourced in Central or South America used to take about seven days of phone calls, emails, and paper chasing. With the records held on a shared ledger, the same trace took about 2.2 seconds. IBM described it as complete end-to-end traceability. A parallel pilot tracked pork in China.

Two honest notes. The figure came from a controlled pilot, not a full live rollout, and the 2.2 seconds is the time to query the record, not the time to fix a contaminated shipment.

Now look at what changed. Before, each party kept its own notebook: the farm wrote the harvest date, the packer the batch, the shipper the container, the store the delivery. To answer one question you had to ask four companies to search their private notebooks and hope they answered quickly and honestly. After, everyone wrote into the same record as they went. Nobody could quietly rewrite their page, and "where did this come from" became a lookup instead of a negotiation.

That is the trustless value in one sentence: **you replaced a chain of promises with one shared record you can read.**

It also shows the limit, which matters more than the win. A shared record proves what was written down. It does not prove the writing was true. If a supplier enters a false farm or a false date, the ledger preserves the lie perfectly. The weak link is the moment a human or a

sensor puts a fact into the system. Any trustless project that ignores that moment is decoration.

## How to Do It

### 7.3 How it works in practice: verifiable identity, reputation, programmable payments

You do not need a cryptocurrency to use trustless thinking. Four building blocks apply to ordinary business.

**Block 1: Verifiable identity.** The question is: is this really who it claims to be, and can I check without phoning anyone? Digital certificates and digital signatures already answer this for you every day. When a supplier sends a document signed with a recognised digital signature, you can verify both who signed it and that nothing changed afterwards. A verifiable credential goes further: your accountant can prove they hold a current licence, or a staff member can prove they passed a background check, without handing over the whole certificate and without you calling the issuing body.

**Block 2: Tamper-evident records.** The question is: can anyone quietly rewrite history? A shared ledger is one answer. So is a simpler trick called a hash — a short fingerprint computed from a file. If you record the fingerprint of a document at the moment you agree it, and the file is later changed, the fingerprint will not match. You can produce that fingerprint later and prove the state of the document then. This costs almost nothing and needs no special permission.

**Block 3: Programmable payments.** The question is: can the money move by itself when the condition is met? Escrow is the old version: a third party holds funds and releases them on a trigger. The newer version writes the trigger into code, so no human decision is

needed at release time. Milestone payment for a freelancer is the obvious case: payment is released when the deliverable is accepted, and the acceptance rule is written down in advance.

**Block 4: Auditability.** The question is: after the fact, can we both see the same truth? Every action should leave a timestamped, ordered, non-editable record. When both sides read the same record, arguments get short. Your accountant, your auditor, and your customer can all check the same thing without asking you for a favour.

**How to start small, today:**

1. Pick one process where you currently chase confirmation.
2. Write the release condition as a single testable sentence. If you cannot write it as a testable sentence, the process is not ready.
3. Ask whether a machine can measure the trigger. Delivered means a scanned signed delivery note. Approved means a click on an approval button. Completed means a status change in your own system.
4. Put the money behind a rule that waits for that measurement: an escrow service, a payment schedule in your accounting system, or a workflow that requires the trigger before release.
5. Log every step where both sides can see it.
6. Only then automate.

## 7.4 AI agents interacting with each other: the future of automated business

The interesting version of this is near. Your software agent talks to a supplier's software agent, and they settle a transaction without a human on each step.

For that to work safely, five things must exist.

**Identity and authority.** Not only "this is supplier X's agent," but "this agent is authorised to commit up to 500 units at a price below 4.20 each." Authority must be provable and bounded, not assumed.

**A machine-readable deal.** Both sides need the terms in a structured form — quantity, price, delivery date, penalty — not a friendly email thread. A human can forgive ambiguity. An automated system will either stall on it or exploit it.

**A payment rail that can wait.** The payment must be conditional: held, then released on evidence. An instant, unconditional payment removes all leverage and all reason to perform.

**Evidence.** A delivery confirmation, an acceptance record, a signed receipt, in a form both systems can read and neither can quietly alter.

**A dispute path.** Something must handle the case where the two agents disagree, or where the world did not go as the rule assumed. With no escalation route, a small disagreement becomes a stuck payment and an angry supplier.

What is realistic now: agents can already search, compare, draft quotes, and prepare orders. What is not yet routine is letting them commit money and legal terms on their own. The sensible path is a human approving the final commitment while the agent prepares everything. That keeps the speed and drops the risk.

## 7.5 What it means for your company: smart contracts, automatic payments, auditability

**Smart contracts, in plain words.** A smart contract is a program that holds an agreement and carries it out when the stated condition happens. It is not clever and it is not a contract in the lawyer's sense. It

is a very literal vending machine. Write the rule carefully and it is a great servant. Write it loosely and it is a great servant too — to whatever you actually wrote.

**Automatic payments.** The practical form is a payment that waits. It waits for a delivery confirmation, an approval click, a milestone accepted, a date reached. Each one is capped, logged, and visible to both sides before it moves.

**Auditability.** Every step leaves a record. Your accountant closes the month faster because there is nothing to reconstruct. A customer dispute ends in minutes because both sides look at the same line. If a regulator asks, you produce the record instead of a story.

**Where it fits well:** cross-border supplier payments where enforcement is slow; freelancer and contractor milestones; marketplaces where buyer and seller are strangers; data-sharing agreements where you must prove what you released and when; insurance that pays on a measured event, such as a flight delay.

**Where it fits badly:** anything that needs judgement, negotiation, or a relationship. Anything where the trigger cannot be measured honestly. Anything where a wrong automatic payment is hard to recover.

## Ethics and Responsibility

Trustless systems change who is accountable, which is exactly why they need careful thought.

**Code that enforces also enforces mistakes.** If your rule releases payment on a condition that is easy to fake, you have automated a loss. Write the rule for the dishonest case, not only for the efficient one.

**Oracles are people.** A "machine-measurable trigger" often depends on a human entering data somewhere. The weakest link in a trustless chain is the moment a person types the truth into it. Design for that person being rushed, mistaken, or bribed.

**Immutability collides with privacy.** Putting personal data on a permanent, unchangeable record can conflict with data-protection rights, including the right to erasure. Keep personal data off public ledgers; store only references and fingerprints. The legal detail is in Chapter 10.

**Do not remove the human from a human problem.** Unhappy customers do not want a perfectly enforced rule; they want someone to listen. Trustless is for the boring middle of a transaction, not for the moment someone is upset.

**Be transparent, and keep a named owner.** Tell the people whose work is now measured by the system, and explain how a decision can be overturned; a rule no one may question will eventually be wrong with nobody able to fix it. Automatic enforcement does not remove responsibility — someone in your company must still own the outcome, as Chapter 4 sets out.

## Mistakes to Avoid

### 7.7 Not everything needs to be trustless

1. **Using a blockchain where a spreadsheet would do.** If the parties trust each other, the record is small, and a normal contract works, a distributed ledger adds cost and complexity for nothing.
2. **Confusing trustless with risk-free.** Code has bugs, feeds can be wrong, keys can be lost, and there is often no support number to

call.

3. **Automating a bad process.** A trustless system will enforce your bad process faster and more consistently than a person ever did.
4. **Putting personal data on a public chain.** It cannot be deleted later.
5. **No kill switch.** Any automated payment system needs a way to stop it in one click.
6. **No spending cap.** An agent with an open wallet is an open chequebook.
7. **Trusting the trigger too much.** Ask how the trigger could be faked, then decide whether that matters.
8. **Chasing a pitch.** "Trustless" is a useful design idea, not a reason to buy a token. If a proposal cannot explain the trigger, the record, and the dispute path, it is marketing.
9. **Cutting out every intermediary.** Some intermediaries earn their fee. A notary, a customs agent, or an insurer may be doing real work that code cannot do.
10. **Forgetting who is accountable.** Even with automatic enforcement, a named human must own the result.

## Practical Exercise

## 7.8 Think of a business process that could benefit from trustless automation

Pick one process where you currently spend time checking, chasing, or arguing. Work through these nine questions in writing.

1. **Who are the parties?** Name them. Note whether they already trust each other.
2. **What must be true before money moves?** One sentence.

3. **Who verifies that today?** A person? How long does it take? How often is it wrong?
4. **What does that verification cost?** Time, fees, delays. Rough numbers are fine; mark them as rough.
5. **Can a machine measure the trigger?** If yes, what is the signal? If no, can the process be redesigned so it can?
6. **What is the evidence trail?** What record exists, where, and can either side change it?
7. **What happens in a dispute?** Who decides, and how fast?
8. **What is the worst way this could be abused?** Write the dishonest scenario before the efficient one.
9. **What is the cap?** The maximum loss if the rule is wrong.

Now score it. If the verification cost is a large share of the transaction value, the trigger is machine-measurable, and you can set a low maximum loss, the process is a strong candidate. If the trigger cannot be measured by a machine, or the maximum loss is high, keep a human in it.

Write one page. Bring one candidate to your next meeting.

## Checklist

## 7.9 When to consider a trustless approach

- ☐ **You pay an intermediary mainly to hold or check something**, and that check could be written as a rule.
- ☐ **The parties do not already trust each other**, and building trust would be slow or impossible.
- ☐ **The release condition can be stated as one testable sentence** that a machine can measure.

- [ ] **The transaction happens often**, so the setup cost is spread across many uses.
- [ ] **It is cross-border or cross-company**, where local enforcement is slow or unclear.
- [ ] **You need a shared, non-editable record** that both sides and your auditor can read.
- [ ] **You can set a hard spending cap** and a one-click stop before automating.
- [ ] **No personal data needs to sit on the permanent record.**
- [ ] **A dispute path exists** and names a human who can override the rule.
- [ ] **You have compared it honestly** with a normal contract and a normal process, and trustless still wins on cost, speed, or risk.

## Key Takeaways

- Trustless does not mean no trust; it means you verify a fact instead of trusting a person to tell it.
- The three working parts are verifiable identity, tamper-evident records, and automatic enforcement of a written condition.
- Ethereum's draft ERC-8004 and the account-abstraction standards show where agent-to-agent business is heading: identity, reputation, and independent validation, with budgets bounded by code.
- The weakest link is the trigger — the moment a human or a sensor tells the system what really happened.
- Do not use trustless where a normal contract works; use it where verification is expensive, the trigger is measurable, and the maximum loss is capped.

# Chapter 8 — Self-Hosting: Keep Your Data Under Control

## In Simple Words

Self-hosting means running software on computers you own or control, instead of renting it from someone else's computers.

You already know both sides of this. Renting a hotel room is cloud: someone else cleans, repairs, and holds a spare key. Owning a house is self-hosting: you fix the boiler yourself, but nobody else has a key.

Applied to AI, self-hosting means the model — the software that does the thinking — runs on a machine in your office or on a server you control. Your questions go in there. The answers come out of there. Nothing travels over the internet to a company you did not choose.

This was not possible for a small business until recently. Until 2022 the best language models lived only inside giant data centres, and you could reach them only through a rented doorway called an API — a standard interface a company opens so other software can ask its systems questions. Every question you typed went through that doorway and landed on their machines.

Then two things changed. First, open models appeared: models whose trained files anyone can download and run. Second, compression techniques became good enough that a compressed model runs on an ordinary powerful desktop computer, with only a small loss in quality.

The trade is a triangle, and you cannot have all three corners at once.

- **Control.** You decide what happens to your data, which model you use, when it changes, and who can see it.
- **Capability.** How good the AI actually is on hard tasks.
- **Cost.** What you pay, in money, time, and attention.

Cloud AI gives high capability for low effort and low control. Self-hosting gives high control, moderate capability, and a cost you pay up front in hardware and then again in ongoing time. The right answer depends on what you are doing.

Two honest sentences before we go further. Self-hosting is not automatically more secure — a badly configured local server is worse than a well-run cloud service. And self-hosting is not automatically cheaper — at small scale, renting usually wins. What self-hosting buys is one specific and valuable thing: your data does not leave, and nobody can change your system without you knowing.

Related topics elsewhere: Chapter 11 covers the wider idea of controlling your own digital tools, Chapter 9 covers the opposite pattern of unmanaged third-party services, and Chapter 6 covers the security basics that still apply to a machine in your own office.

## A Bit of History

**1990s to 2000s: everything at home.** A small business had a server in a cupboard. Email, files, and accounts all sat on premises. You owned the machine and all of its problems.

**2000s to 2018: the move out.** Broadband got fast, and renting became easier than owning. Email went first, then files, then accounting and customer records. “The cloud” became the default answer. The joke among engineers was accurate: there is no cloud, it is just someone else’s computer.

**2013: containers.** Docker arrived and packaged software so it ran the same way everywhere. This quietly made self-hosting easy again, because you no longer needed a specialist to rebuild an application on each new machine.

**2022: AI was only a rental.** The best language models existed only inside a handful of large companies. If you wanted AI, you sent your text to them. There was no alternative worth using.

**2023: the open-model year.** Meta released Llama as a research model in February, then Llama 2 with open weights in July. Mistral AI released a strong seven-billion-parameter model in August. "Open weights" means the trained files are published, so anyone can download and run them. The same year, quantisation — compressing a model so it needs far less memory — became good enough for daily use. Suddenly a capable AI was not only in a data centre.

**8 July 2023: Ollama.** A tool called Ollama was released, open source under the MIT licence, written mainly in Go with some C and TypeScript, by Jeffrey Morgan and Michael Chiang. Its job was to remove all the friction. One command downloads a model and runs it locally, with a simple interface and a local service other programs can talk to. It used the llama.cpp engine for the actual work on your hardware.

**2024: "sovereign" becomes a buying criterion.** Companies in regulated sectors, and several European governments, began asking for AI that stays inside a country and inside their own walls. Cloud providers answered with sovereign-cloud options, and a market for on-premise AI grew quickly.

**2026: an open-source enterprise client.** In April 2026 MZLA Technologies, a Mozilla subsidiary, announced Thunderbolt, an open-

source AI client built to be self-hosted. The Curiosity section covers it.

The shape of this history is a loop. We started self-hosting, moved out for convenience, and are now moving back for control — with far better tools than the first time.

## Curiosity

### 8.7 Mozilla's open-source AI client for "AI you control"

In April 2026, MZLA Technologies Corporation — a wholly owned subsidiary of Mozilla, the non-profit behind Firefox — announced **Thunderbolt**. It is an open-source, cross-platform AI client built for organisations that want to run AI on their own terms.

The details that matter for this chapter:

- **Self-hostable.** It runs on the customer's own infrastructure. The project describes support for on-premise, sovereign-cloud, and air-gapped setups — meaning a network physically disconnected from the internet.
- **Model-agnostic.** It works with any agent that speaks the Agent Client Protocol, and with any model that offers an OpenAI-compatible API. In practice that means you can point it at a local model, your own server, or a commercial provider, and switch without changing your tools.
- **Everywhere.** Web, Windows, macOS, Linux, iOS, and Android.
- **Connected to your systems.** It integrates with enterprise systems through the Model Context Protocol — a standard way to let an AI tool reach your internal data and actions — and supports reusable automations and an extensible API.

- **Auditable.** Because the code is open, you or a third party can read it and check what it actually does. The project has said it is going through a security audit.
- **A partner for European sovereignty.** It pairs with deepset's Haystack, an open-source orchestration platform, for sovereign deployments in Europe.
- **Support included.** Enterprise support and forward-deployed engineering are offered, which is the part most open-source projects leave you to solve alone.

The positioning is worth noticing. The framing is "AI you control," and the argument is that AI is too important to outsource entirely. That is the same argument a small business owner makes when they decide to keep payroll files on their own machine rather than in a service they never inspected.

A smaller, older example of the same idea sits in an email app many people already use. The ThunderAI add-on for Thunderbird added support for local models through Ollama in version 2.1.1, released in August 2024. With that setup the model runs on your own computer, so the email text never leaves the machine, and no cloud account or API key is needed. It is a small feature, and it makes the point clearly: local AI is no longer a research project. It is an option inside a free email client.

## A Real Business Example

### Even Apple built its own cloud

Apple is a company that could rent almost anything. In June 2024 it announced a system called Private Cloud Compute, built for the AI

features in its devices when a task is too large for the phone itself. What it chose to do with that system is the lesson.

Apple built its own servers around its own chips with a hardened operating system. It designed the system for stateless inference — process the request, return the answer, keep nothing. And it made an unusual promise: it would publish the software image of every production build so that outside security researchers could inspect exactly what is running, and check that the machine they are talking to really is what it claims to be. In October 2024 Apple published a security guide and invited researchers to attack the system.

Three things follow for a small business.

First, notice the reasoning. Apple did not build a cloud because it could not rent one. It built one because renting would have meant keeping user data on someone else's terms, and its whole brand is that it does not. Control of the machine was worth real money to them.

Second, notice the verification idea. Publishing the software so outsiders can check it is the same instinct as open-source auditing. Trust is reduced by making the system inspectable, not by promising to be good.

Third, notice the size of the gap. Apple spent a fortune to reach "we keep nothing." Most small businesses cannot copy that. But you can copy the question: *who runs the machine my data goes to, what do they keep, and can anyone check?* If you cannot answer those three questions about a tool, you should not put sensitive data in it. And if the honest answer is "we cannot check," a model running on your own machine becomes a serious option.

# How to Do It

## 8.5 How to start: from open-source tools like Ollama to local models

**Step 1: Pick one task and one person.** Do not buy hardware first. Choose one repeated task that involves sensitive data — summarising contracts, drafting replies to standard enquiries, turning meeting notes into action lists. Give it to one curious person with a good computer.

**Step 2: Install Ollama.** Ollama runs on Windows, macOS, and Linux. After installing, one command pulls a model and starts a chat in your terminal. Under the hood it starts a small service on your own machine, on port 11434, that other programs can talk to. That local service is what lets you connect the model to your own tools later.

**Step 3: Choose the model by size, not by name.** Model sizes are counted in parameters, a rough measure of how big and capable the model is. As a rule of thumb:

- **1 to 3 billion** — fast and light, runs on an ordinary laptop. Good for short summaries and simple drafting.
- **7 to 9 billion** — the practical middle. Good general quality, needs a decent machine with enough memory.
- **70 billion and above** — much stronger, but needs serious hardware and a lot of memory. Usually not a small-business starting point.

Look for **quantised** versions. Quantisation compresses the numbers inside the model so it needs far less memory and runs faster, at the cost of a small drop in quality. For most office tasks that drop is acceptable.

**Step 4: Add a friendly interface.** The terminal is fine for testing. For daily use, add a chat interface that runs locally, or plug the model into an app people already use — the Thunderbird add-on mentioned above is one example. The goal is that a non-technical colleague can use it without help. If only one person can drive it, it will never be adopted.

**Step 5: Keep it actually local.** This is where people fail. The local service must listen only on your own machine, not on the whole network. Security researchers have found many Ollama servers exposed to the public internet because they were set to accept connections from any address. So: keep the default local binding, put a firewall in front, never open port 11434 to the internet, and do not run it with administrator rights. A local tool is not automatically a safe tool.

**Step 6: Let it read your own files.** The most useful next step is retrieval, often called RAG. In plain words: instead of retraining the model, you put your documents into a searchable index. When someone asks a question, the system finds the relevant passages and hands them to the model along with the question. The model answers from your documents. Nothing is trained on your data, and the whole thing can run on your own machine. This is how you get "ask our handbook" without sending the handbook anywhere.

**Step 7: Scale only when the test worked.** If the pilot is useful, then buy hardware. A workstation with a strong graphics card is the usual answer, because graphics memory is what limits how large a model you can run. Apple machines with unified memory are a common choice for the same reason. A small server makes sense when several people need it at the same time.

**Step 8: Decide the split.** Most companies end up hybrid. Sensitive data and routine high-volume tasks run locally. Hard reasoning, very long documents, and specialist features use a cloud service under a

proper contract. Write the rule down: which data goes where, and who decides.

## 8.6 Ideal use cases

**Sensitive personal data.** Health records, HR files, payroll, applicant CVs, client financial details. If the data must not leave, local processing removes the question entirely.

**Proprietary information.** Designs, drawings, source code, tender and bid documents, pricing models, strategy plans. These are the files that would hurt most if a vendor kept them or leaked them.

**Regulated sectors.** Healthcare, legal practices, banking and insurance, government and defence, and any business under data-residency rules that require data to stay inside a country or region. Self-hosting turns a hard compliance conversation into a simple one.

**Poor or no connectivity.** Field sites, factories, ships, mines, remote offices, emergency response. A local model works when the internet does not.

**High-volume repetitive work.** Thousands of tickets, invoices, or documents. A price per request hurts at volume; a fixed machine does not grow.

**Cheap experimentation.** Trying ideas without a meter running. Useful before you commit budget.

**Where it fits badly:** frontier-level reasoning, very long documents, advanced image generation, and any team with no technical support at all. If nobody can maintain it, do not start it.

## Ethics and Responsibility

Self-hosting solves one ethical problem and creates others. Be clear about which is which.

**What it solves.** Your customer data stops travelling to companies you did not choose, and cannot be used to train someone else's product without you. That is a real gain in respect for the people whose data you hold.

**What it does not solve.** The model can still be wrong, biased, or confidently misleading. Running a biased model on your own hardware does not make it fair. The duties in Chapter 4 still apply, and so do the disclosure rules in Chapter 5.

**What it adds.** You are now the operator. Patching, access control, logging, and backups are your job. If an unpatched local server leaks data, that is on you in a new and very direct way.

**Employee monitoring.** If you log prompts to a local system, you now hold a detailed record of what your staff asked. Have a written policy on what is kept, why, who reads it, and for how long. Keep it proportionate, and tell people.

**Do not oversell sovereignty.** "Sovereign" is a marketing word as much as a technical one. If you claim control but cannot patch, back up, or audit, the claim is empty. Say what you actually have, and say plainly what you do not.

**Energy.** A machine running models all day uses power. Local is not automatically greener than a large, efficient data centre. If you claim an environmental benefit, check it first.

## Mistakes to Avoid

1. **Buying hardware before testing a task.** Test with a rented or borrowed setup first. Hardware bought on hope sits unused.
2. **Exposing the local AI service to the internet.** This is the most common and most serious self-hosting error. Keep it local, firewall it, do not open the port.
3. **Assuming local equals secure.** You now own the security of that machine, including the parts you never thought about.
4. **No backups.** The machine, the models, the configuration, and the document index all need backing up.
5. **Choosing the biggest model.** Bigger is slower, hungrier, and often no better for your actual task.
6. **Ignoring model licences.** Open-weight models come with different terms, including rules on commercial use and on how you may describe your use. Read the licence before you build on it.
7. **One-person dependency.** If one person knows how it works and leaves, the system stops. Write down the setup, the restart, and the restore steps.
8. **Self-hosting the wrong workload.** If the task needs frontier quality, local will disappoint, and you will wrongly conclude the technology is useless.
9. **Never updating, or updating without testing.** A new model version changes answers. Re-run your sample tasks after every change.
10. **Treating it as all or nothing.** A hybrid setup is usually the right answer, not a total move in either direction.

## Practical Exercise

### 8.8 Evaluate whether self-hosting suits your company

Score each line 0, 1, or 2. Be honest.

| Question | 0 | 1 | 2 |
|---|---|---|---|
| How sensitive is the data? | Public | Internal | Client-confidential or regulated |
| Technical support available | None | Part-time IT help | Someone who can maintain a server |
| Hardware budget | None | One good workstation | Workstation plus a small server |
| Task complexity | Summaries and drafting | Questions over documents | Complex reasoning, very long documents |
| Connectivity | Unreliable | Fine | Fine |
| Volume | Occasional | Daily | High and growing |

Read the score line by line, because the lines do not all push the same way.

- **Sensitivity 2** and **Volume 2** both push toward self-hosting.

- **Technical support 0** pushes strongly against it, whatever the other scores say.
- **Task complexity 2** pushes against it, because local models lag on hard reasoning.

Now run a one-day test before you spend anything. Take twenty real tasks from your list. Do ten with a mid-size local model and ten with a cloud service. Compare three things: was the answer good enough to use as it stood, how long did it take, and what would it have cost the other way. Write the results down.

If the local answer was usable for most tasks, self-hosting is worth the investment. If it was usable for only a few, keep those local and leave the rest in the cloud under a contract. Either way you now have evidence instead of an opinion.

## Checklist

## 8.9 What you need to start

- ☐ **One named task** that involves sensitive data and repeats often.
- ☐ **One named person** who can install, restart, back up, and update the system — or a plan to hire one.
- ☐ **A machine with enough memory**, chosen after a test, not before.
- ☐ **A model chosen by size and quantisation**, matched to the task rather than to the largest available.
- ☐ **A local interface** a non-technical colleague can use without help.
- ☐ **The service bound to the local machine only**, firewalled, with no port open to the internet.
- ☐ **A written licence check** for the model you pick, including commercial-use terms.

- [ ] **A backup plan** for the machine, the models, the configuration, and the document index.
- [ ] **A patch routine** with a set date, and a re-test of sample tasks after every change.
- [ ] **A written split rule** stating which data stays local and which may use a cloud service, and who decides.

## Key Takeaways

- Self-hosting means the AI runs on machines you control, so your data does not leave them.
- It became practical for small businesses when open-weight models and good compression arrived in 2023, and tools like Ollama removed the setup friction.
- You trade capability and convenience for control: the triangle is control, capability, and cost, and you cannot have all three.
- Local is not automatically safe or cheap; a misconfigured local server is a real risk, and maintenance is now your job.
- Start with one task, one person, and a one-day test before buying any hardware.

# Chapter 9 — Third-Party Services and Shadow AI

## In Simple Words

A third-party AI service is any AI you use that runs on someone else's computers. You type a question or upload a file. It travels over the internet to their machines. Their software does the work. The answer comes back. You never see the inside.

This is the normal way most companies use AI today. It is easy, it is cheap to start, and it is powerful. That is why almost everyone uses it.

It also has a hidden cost. The moment your text leaves your building, you lose direct control over it. You are now relying on a company you did not design, on rules you did not write, on a server you cannot inspect.

"Shadow AI" is the second half of this story. Shadow AI means employees using AI tools without the company's knowledge, approval, or oversight. A marketing assistant pastes a customer list into a free chatbot. An accountant drops an invoice into a web app to summarise it. Nobody in charge knows it is happening. The data is gone, and there is no record, no contract, and no control.

Think of it like the office printer twenty years ago. Everyone used it. Nobody knew it kept a copy of every page. Shadow AI is the same surprise, but the pages are your customer lists, your contracts, and your prices.

This chapter is about two things: what actually happens to your data when you use a third-party service, and how to stop your own staff from quietly creating risk. The opposite pattern — running AI on your own machines — is covered in Chapter 8. The legal side of personal data is the home topic of Chapter 10. The wider question of controlling your own tools is Chapter 11.

## A Bit of History

**1990s to 2000s: software moves off the shelf.** You used to buy a program on a CD and run it on your own computer. Then email, calendars, and file storage moved to the internet. You stopped owning the software and started renting access to it.

**2000s to 2015: SaaS becomes normal.** "Software as a Service" meant paying a monthly fee to use someone else's software over the web. Salesforce, Google Workspace, and Microsoft 365 made it ordinary. Convenience won. Most small businesses stopped running their own servers.

**2016 to 2022: the data goes with the software.** Once your customer records, documents, and messages lived in a vendor's cloud, your business data was permanently outside your walls. Most people accepted this because the tools were good and the price was low.

**November 2022: ChatGPT opens the door to everyone.** A powerful AI became available in a free web box. You no longer needed a vendor contract to use AI. You needed only an email address. This is the moment shadow AI began, because a single employee could now send company data to a frontier model without asking anyone.

**2023: the first corporate alarms.** Companies discovered staff were pasting confidential material into public chatbots. The Samsung case in

the Curiosity section is the most famous. Several banks and professional firms banned or restricted public AI tools that year.

**2023 to 2026: vendors add "enterprise" tiers.** In response, AI providers sold business plans that promised not to train on your data, to keep data separate, and to add admin controls. The convenience stayed; the promises got better. But a promise is not the same as control, which is why this chapter matters.

## Curiosity

## 9.7 The semiconductor company that pasted its secrets into ChatGPT

In early 2023, Samsung Electronics — one of the largest technology companies on earth — found that its own engineers had put confidential material into a public AI chatbot.

The reporting, first published by the Korean financial daily *Korea Economic Daily* in April 2023 and repeated widely, described three separate incidents inside the semiconductor division. In the first, an engineer pasted faulty source code from a program used to download measurement data from a chip factory, hoping the chatbot would find the bug. In the second, staff uploaded code connected to semiconductor equipment and defect detection. In the third, employees fed recorded meeting transcripts into the tool to get summaries. All of this happened within roughly three weeks of the division first allowing access to the tool.

Once the data was inside the chatbot, it was on machines Samsung did not own. The company could not easily retrieve it, and could not be sure it would not be used or seen by others.

Samsung's response was blunt. It banned generative AI tools, including ChatGPT, on company-owned computers, tablets, phones, and its internal network. Breaking the rule could lead to disciplinary action up to dismissal. Employees using such tools on personal devices were told never to submit company information or personal data that could reveal Samsung's intellectual property. An internal survey found that 65% of respondents believed AI services posed a security risk. At the same time, Samsung said it was building its own internal AI tools for translation, document summarisation, and software development, and working on ways to block uploads of sensitive information to outside services.

The lesson is not that Samsung was careless. It is that if a company with billions in security budget was caught off guard by a free chatbot, a small business with no security team has even less protection. The convenience is real, and so is the leak.

## A Real Business Example

### The agency that lost its client list twice

Consider a small marketing agency, twenty people, no IT department. This is a composite story, but each part happens every week in real firms.

First leak: a senior account manager wants to rewrite a proposal quickly. She opens a free AI chatbot on her laptop and pastes the draft, which includes a client's unreleased product name, budget figures, and contact details. She gets a nicer paragraph. She never thinks about where the text went.

Second leak: the same manager, a month later, signs up for a paid "AI writing assistant" that promises better results. To set it up, she connects it to the agency's email and shared drive so it can "learn the company voice." Now the vendor can read everything: every client, every contract, every internal joke that turns into a strategy note.

Then the evolution problem arrives. A year later the vendor changes its plan. The features she paid for move to a higher tier. The price doubles. The model behind the tool is swapped for a newer one, and the writing style she trained her team on quietly changes. She wants to leave, but everything is wired into that tool. Switching means reconnecting the whole agency somewhere else and retraining everyone. That is lock-in, and it happened without a single bad decision — only a series of convenient ones.

None of this required malice. It required convenience and the absence of a rule. The fix is not to fear AI. It is to decide, in advance, what data may go where, and to give people a good tool so they do not reach for a risky one.

## How to Do It

### 9.1 The convenience of cloud services: why everyone uses them

Third-party AI is popular for honest reasons, and you should name them before you argue against them.

There is no setup. You do not buy hardware, hire an engineer, or install anything. You open a web page and start. For a small business with no technical staff, that is the whole appeal.

There is no up-front cost. Most tools have a free tier or a low monthly fee. You can try an idea for the price of a coffee instead of a capital purchase.

The quality is high. The best models in the world sit behind these services. A two-person firm can use the same capability as a large corporation.

It scales instantly. Need ten times more work done today? The vendor's machines absorb it. You do nothing.

This is why cloud AI spread faster than any technology before it. The point of this chapter is not to take that away from you. It is to make you use it with your eyes open, because every one of those conveniences is bought with a piece of control you did not notice you were selling.

## 9.2 What happens to your data when it leaves the company

When you paste text into a third-party AI, several things may happen, and you usually cannot see which.

Your data crosses the internet to the vendor's servers, often in another country. Once there, it is stored, at least for a while, on hardware you do not control and cannot inspect.

It may be read by automated systems for safety screening. It may be logged for debugging. Support staff in another country may be able to see it. None of this is unusual; it is how large services operate.

Most importantly, it may be used to train the vendor's model. "Training" means the company feeds your text into its system so the model learns from it. If that happens, a fragment of your confidential

text can shape answers given to other customers. This is the single biggest reason to be careful.

Some business plans promise not to train on your data. That promise is worth something, but it is a contract term, not a wall. You are trusting the vendor to honour it across every product and every country. The safe rule is simple: treat any third-party AI as if whatever you send could become public. If that thought is unacceptable for a given file, do not send that file.

## 9.3 The transparency problem: you never really know how your data is used

You cannot see inside a third-party AI service. This is the core problem, and it is not going away.

You do not know which model answered you. Vendors swap models without telling you. You do not know where your data was stored, who could access it, or how long it was kept. You do not know whether a subcontractor in another country processed it. You do not know if it was used for training, even when you think it was not.

The vendor's privacy policy is written by lawyers, not engineers, and it describes what they may do, not what they will do in your specific case. Reading it rarely gives a clear answer.

This is why the practical test is so useful. Instead of trying to understand the system, ask one question about the data: *would I be acceptable if this became public tomorrow?* For a public blog post, yes. For a client's salary file, no. That single test replaces a thousand questions you cannot answer.

If you need real transparency, the only honest source is a system you can inspect — your own machine, or open software someone can audit.

That is the subject of Chapter 8 and Chapter 11.

## 9.4 The evolution problem: models change, contracts change, prices change

A third-party service is not a thing you buy; it is a relationship that keeps changing under your feet.

The model changes. Today's tool may run on a different model next quarter. The answers change. The style changes. Something that worked well in your workflow can get worse, or simply different, with no warning and no way to stay on the old version.

The contract changes. Terms you agreed to at signup can be updated. Features you relied on can move behind a higher plan. The promise about training data can be reworded.

The price changes. A cheap tool that you have now wired into your business can raise its price, and you face a painful choice: pay more, or rip everything out.

This is the hidden risk of convenience. You build on ground that someone else controls, and they can move the ground whenever they like. The defence is to keep your important data and your core workflows portable, and to never let a single vendor become the only place your work can happen.

## 9.5 Lock-in: how hard it is to switch vendors later

Lock-in means leaving a vendor is so costly, in time, money, or disruption, that you cannot realistically do it. You are "locked in."

It happens in three ways. **Data lock-in:** your data is stored in a format only that vendor reads well, or you cannot export it cleanly. **Workflow lock-in:** your daily process is built around that tool, so switching means retraining everyone and rebuilding templates. **Integration lock-in:** the tool is connected to your email, your CRM, your files, and pulling it out breaks all of those links.

Lock-in gives the vendor power over you. They know leaving is hard, which weakens your position when they raise prices or change terms.

To stay free, insist on three things from the start. First, your data must be exportable in a plain, common format. Second, keep your core data somewhere you control, and let the vendor work on a copy. Third, design your workflow so the AI is a step, not the whole machine, so you can swap the step. The goal is not to avoid vendors. It is to be able to leave one in a week, not a year.

## 9.6 Shadow AI: when employees use unauthorized tools

Shadow AI is the risk you cannot see, because it is created by your own trusted staff.

It happens because good official tools are slow to approve, while risky free tools are instant. An employee under time pressure pastes a customer email into a free chatbot to draft a reply. Nobody stops them, because nobody knows.

Why it is dangerous: the data leaves with no contract, no review, and no record. You cannot tell the customer, you cannot find the data later, and you cannot prove what happened. If the tool trains on that data, your confidential information may end up shaping a public model.

How to spot it: unusual charges on corporate cards for AI subscriptions; staff complaining that approved tools are too slow; sudden “magic” improvements in output that no approved tool explains.

How to stop it: do not only ban. A ban with no good alternative just pushes the behaviour underground. Give people an approved tool that is fast and good enough, and they will use it. Then make the rule clear, train on it, and monitor lightly. The full policy side is in section 9.8 and in the company AI-policy templates in the book’s appendices.

## 9.8 How to mitigate the risks: due diligence, contracts, internal policy

You reduce the risk with three layers. Do all three.

**Layer 1: Due diligence — check before you buy.** Before adopting any AI vendor, ask the hard questions. Where is data stored, and in which countries? Do you train on customer data, and can you guarantee in writing that you will not? Who can access it, including subcontractors? How long is it kept, and how is it deleted? Is it encrypted in transit and at rest? What certifications do you hold? Get the answers in writing, not in a sales call. A vendor who cannot answer clearly is telling you something. A scored version of these questions is in the vendor due-diligence scorecard in the appendices.

**Layer 2: Contracts — put the promises on paper.** A verbal assurance is worthless. In the contract, require: no training on your data; data residency in a country you accept; the right to export all your data in a usable format; the right to deletion; a duty to tell you about breaches and about major model or term changes; and limits on subcontractors. If the vendor will not sign these, that is your answer.

**Layer 3: Internal policy — tell your people the rules.** Write a short, plain AI policy. State which tools are approved. State what data may never go into any external AI — client personal data, financial records, contracts, source code, passwords. Give staff a fast approved tool so they do not reach for a risky one. Train everyone on the rule in one short session. Review the list of approved tools every quarter. A template lives in the appendices.

The order matters. Check first, contract second, policy third. Most companies skip the first two and wonder why they got exposed.

## Ethics and Responsibility

Third-party AI raises a duty you owe to two groups: the people whose data you hold, and your own staff.

**To your customers and employees.** When a customer gives you personal data, they trust you to protect it. Sending that data to a third-party AI you have not vetted can break that trust, even if nothing bad happens. You are responsible for where their data goes. The legal duties are set out in Chapter 10; the ethical principle is simple — do not put someone's private information somewhere you would not put your own.

**To your staff, about shadow AI.** Be careful not to turn protection into surveillance. If you monitor AI use, tell people what you monitor and why. A clear rule plus a good tool is fair. Secret monitoring of trusted staff damages morale and trust. Aim for guardrails, not a police state.

**Be honest about what you do not control.** If a client asks whether their data trains an external model, you should be able to answer

truthfully. If you do not know, say so, and fix it. Overclaiming safety is worse than admitting a gap.

## Mistakes to Avoid

1. **Reading the demo, not the data flow.** A slick demo tells you nothing about where your data goes. Ask the storage and training questions first.
2. **Trusting a verbal "we don't train on your data."** If it is not in the contract, it does not exist.
3. **Connecting a tool to your whole drive.** "Learn our voice" often means "read everything." Give the tool the least it needs.
4. **Banning shadow AI with no alternative.** A ban without a good approved tool just hides the behaviour.
5. **No written policy.** If the rule is not written and trained, it is not a rule.
6. **Ignoring the country question.** Data stored under a different legal system can be accessed differently. Know where it sits.
7. **Assuming a paid plan equals safe.** A paid consumer plan is not the same as a contracted enterprise plan with the right terms.
8. **No export plan.** If you cannot get your data out, you are locked in from day one.
9. **Treating the privacy policy as a guarantee.** It lists what they may do, not what they will do for you.
10. **Forgetting the human.** The leak almost always starts with one person under time pressure. Fix the pressure and give a safe path.

## Practical Exercise

## 9.9 Analyze your current AI vendors

Take one hour and do this for every AI tool your company uses today, including the ones staff quietly use.

Make a table. One row per tool. Columns:

- **Tool name** and who signed up for it.
- **What data goes in.** Be specific: customer emails, invoices, contracts, code, public text.
- **Where it is stored,** if you know. If unknown, write "unknown."
- **Do they train on our data?** Yes / No / Unknown.
- **Is there a contract,** or just a click-through?
- **Can we export our data?** Yes / No / Unknown.
- **Risk level** for the data you actually put in it: Low / Medium / High.

Now read the High-risk rows. For each, ask: would it be acceptable if this data became public? If no, you have three choices — stop putting that data in it, move to a vendor that signs the right terms, or move that task to a tool you control (see Chapter 8).

Finally, ask around. Send one honest message to your team: "What AI tools do you use for work that we have not talked about?" The answers are your real shadow-AI list. Do not punish the honesty; fix the gaps it reveals.

## Checklist

## 9.10 Questions to ask every AI vendor

Before you put any real data into a third-party AI service, get a written answer to each of these.

- ☐ **Where is our data physically stored,** and in which countries?
- ☐ **Do you train your models on our data?** Can you commit to "no" in the contract?
- ☐ **Who can access our data,** including employees and subcontractors, and from where?
- ☐ **How long do you keep our data,** and how do we get it deleted?
- ☐ **Is our data encrypted** both in transit and while stored?
- ☐ **Can we export all of our data** at any time, in a common, usable format?
- ☐ **Will you notify us** of a data breach, and within what time?
- ☐ **Will you tell us** before you change the model, the price, or the terms?
- ☐ **What security certifications** do you hold, and can you show them?
- ☐ **Are there subcontractors,** and are they bound by the same terms?
- ☐ **What is our legal recourse** if you misuse our data?
- ☐ **Is there an admin console** so we can see and control usage?

If a vendor cannot answer these clearly and in writing, treat the tool as high-risk and keep sensitive data out of it.

## Key Takeaways

- A third-party AI service runs on someone else's computers, so the moment your data leaves, you trade control for convenience.

- The biggest risk is that your confidential data may be used to train a model that serves others, and you often cannot see or stop it.
- Vendors change models, contracts, and prices under your feet, and lock-in makes leaving expensive — so keep your data exportable and your workflow swappable.
- Shadow AI is your own staff using unapproved tools; stop it with a good approved tool plus a clear written rule, not with bans alone.
- Protect yourself in three layers: check the vendor first, put the promises in a contract second, and set an internal policy third.

# Chapter 10 — Privacy and GDPR: What You Really Need to Know

## In Simple Words

GDPR stands for General Data Protection Regulation. It is the European Union's rulebook for handling personal data — any information about a living person who can be identified. It is one of the strictest privacy laws in the world, and it affects far more companies than most owners expect.

The whole law can be reduced to one idea: **if you hold information about a person, you must handle it fairly, for a clear reason, keep only what you need, keep it safe, and respect what that person asks you to do with it.**

You are affected if you are in the EU, and also if you are outside the EU but offer goods or services to people in the EU, or watch their behaviour. A small online shop in another country that sells to customers in France is inside GDPR's reach. This is called extraterritorial reach, and it surprises many businesses.

Why does this matter for AI? Because AI runs on data, and much of that data is personal. Customer names, support emails, CVs, employee records, website visitors — all personal data. When you feed any of it into an AI tool, you are processing personal data, and the rules apply. If you send that data to a third-party service, the rules follow it out the door. That is the link to Chapter 9.

This chapter is the deep home for GDPR in this book. It explains the principles, the types of data, the legal grounds, your obligations, and

the rights people hold over you. The EU AI Act — a separate law about AI systems themselves — is treated fully in Chapter 5; here we look only at where it touches privacy. The security side of protecting data is in Chapter 6.

One honest note: this is a plain guide, not legal advice. For real decisions, especially across borders, talk to a data-protection professional.

## A Bit of History

**1995: the first EU rule.** The Data Protection Directive set early standards across Europe, but each country implemented it differently, creating a patchwork.

**27 April 2016: GDPR adopted.** The EU replaced the patchwork with one regulation, designed to apply the same way in every member state and to modernise the rules for the internet age.

**25 May 2018: GDPR applies.** This is the date that matters. From this day, the rules were fully in force, with heavy fines for serious breaches. Companies across the world had to comply or risk them.

**2018 to 2023: the enforcement era.** National data-protection authorities issued fines, some very large, for breaches and poor practice. Privacy became a board-level topic, not just a lawyer's problem.

**2018: the EDPB.** The European Data Protection Board was created to coordinate the national authorities and issue guidance so the law is applied consistently across the EU. Its guidance is where you look when the law is unclear.

**1 August 2024: the EU AI Act enters into force.** A separate law, Regulation (EU) 2024/1689, began its phased rollout. It regulates AI systems by risk. It does not replace GDPR. Where an AI system uses personal data, both laws apply at once.

## Curiosity

## 10.9 What Europe's privacy regulators said about AI models

In December 2024 the European Data Protection Board (EDPB) — the body that coordinates all the EU's national privacy regulators — adopted **Opinion 28/2024**, on the protection of personal data in the context of AI models. It is the most important privacy statement yet on how AI fits with GDPR.

A few points from it matter to a business owner:

**A model can be looked at in three stages.** The EDPB separates the life of an AI model into development (training it), deployment (putting it to work), and use (people interacting with it). Each stage can involve personal data, and each has its own privacy questions. This is useful because it tells you to ask the question at every stage, not only at the start.

**Legitimate interest can be a legal basis, but it is not a free pass.** The opinion says a company may rely on "legitimate interest" to develop or use an AI model, but only if the processing is genuinely necessary and a balancing test shows it does not outweigh people's rights. You cannot simply declare an interest and ignore the people involved.

**A model is only "anonymous" if you cannot get personal data out of it.** This is the sharp part. The EDPB says that for a model to be treated as anonymous — and so outside GDPR — it must be very unlikely that someone could extract personal data from it by asking it questions. If a model can be tricked into repeating personal details it memorised during training, it is not truly anonymous, and the data protection rules still bite.

**Bad training data can follow the model.** The opinion warns that if a model was trained on unlawfully processed personal data, that can affect the lawfulness of deploying it later, unless the model has been properly anonymised. In plain words: a model built on dirty data can stay dirty, and using it can carry that problem into your business.

The practical message for a small company is clear. When you choose an AI vendor, the privacy question is not only "what do you do with my data now," but "what data was this model trained on, and can personal data be pulled back out of it?" That question belongs in your vendor checklist.

## A Real Business Example

### The HR tool and the access request

A mid-sized firm with 120 staff adopts an AI tool to screen job applications. It uploads CVs and cover letters — all personal data, some of it sensitive, because CVs can reveal health gaps, age, nationality, and union activity. The tool ranks candidates.

Three months later, an applicant who was rejected writes to the firm. She makes a **subject access request**: under GDPR she has the right to know what personal data the firm holds about her and how it was

used. The firm must respond, usually within one month and free of charge.

Now the firm must answer hard questions it never asked. Where are her CV and the AI's ranking stored? Can it produce the data and explain the logic? Was there a lawful basis to process her CV with this tool? Did it do a risk assessment first? If the tool was a third-party service, did the contract cover all of this? If the AI's ranking counts as automated decision-making with a serious effect on her, extra rules apply, including her right not to be subject to a purely automated decision with legal or similarly significant impact.

The firm scrambles because it adopted the tool before it thought about the data. The lesson is the order of operations: you must understand the data and the legal basis **before** you switch the tool on, not when a request arrives. The rest of this chapter gives you the pieces to do that.

## How to Do It

### 10.1 GDPR on one page: the basic principles

GDPR rests on a few principles. If you honour these, you are honouring most of the law.

**Lawfulness, fairness, transparency.** You need a valid legal reason to process personal data, you must not use it in ways people would consider unfair, and you must tell people what you are doing.

**Purpose limitation.** Collect data for a clear, stated purpose. Do not reuse it later for something unrelated without a new lawful basis.

**Data minimisation.** Collect only what you actually need. If you do not need a phone number, do not ask for one. This matters for AI: do

not dump your whole database into a tool when a small slice would do.

**Accuracy.** Keep data correct and update it. People can demand corrections.

**Storage limitation.** Do not keep personal data longer than you need it. Have a deletion schedule.

**Integrity and confidentiality.** Keep it secure against breaches. The how-to is in Chapter 6.

**Accountability.** You must be able to *show* you comply — with records, policies, and documents. This is why the paperwork in the Checklist section is not optional.

## 10.2 Personal data, sensitive data, anonymous data: the differences that matter

**Personal data** is any information about a living person who can be identified, directly or indirectly. A name, an email, a phone number, a photo, an ID number, a location, an online identifier like a cookie ID. Even a combination of details that singles someone out counts. If you can point at a person, it is personal data.

**Sensitive data** (the law calls it "special categories") gets stronger protection. It includes: racial or ethnic origin, political opinions, religious or philosophical beliefs, trade union membership, genetic data, biometric data used to identify a person, health data, and data about a person's sex life or sexual orientation. The rule is that processing these is prohibited unless a specific condition applies, such as explicit consent or a clear legal requirement. In AI, watch for these hiding inside CVs, support tickets, and HR files. A chatbot that learns "this customer is on chemotherapy" has touched health data.

**Anonymous data** is information that can no longer identify a person, even by combining it with other data. Truly anonymous data is outside GDPR, because there is no identifiable person. But true anonymity is hard to achieve. If you could re-identify someone, it is not anonymous. This is the difference that trips people up, and it is the focus of section 10.8.

The practical rule: treat almost everything about a customer, employee, or applicant as personal data by default. Treat anything touching health, beliefs, or identity as sensitive and handle it with extra care. Only call data anonymous if you have genuinely removed the ability to identify anyone.

## 10.3 Consent: when it is needed and when it is not

Consent is one lawful basis for processing, not the only one, and it is often misunderstood.

For consent to be valid, it must be **freely given, specific, informed, and unambiguous**, and given by a clear action. A pre-ticked box is not consent. Buried clauses are not consent. Consent for "marketing and everything else" is not valid consent.

You need consent when no other basis fits, and always for sensitive data in most cases, and for certain things like marketing emails to consumers in many EU countries.

You often do **not** need consent when another basis applies. If you process data to deliver a contract you have with the customer, that is the "contract" basis, not consent. If a law requires you to keep records, that is "legal obligation." Asking for consent when you already have a contract can actually create problems, because consent can be

withdrawn at any time, and then you cannot deliver what you promised.

For AI, consent is tricky. If you want to use customer data to train a model, a vague "we may use your data to improve our services" is usually not enough. You must be specific, and you must let people say no without losing the service. The safer path is to avoid needing broad consent at all: minimise the data, use it only where a real basis exists, and prefer non-personal or anonymised inputs for training.

## 10.4 Legitimate interest: when you can use data without explicit consent

Legitimate interest is the most flexible and most misused basis. It lets you process personal data without consent when you have a genuine, lawful business reason — but only after a careful test.

The test has three parts. **Purpose:** is your reason legitimate? Improving fraud detection or network security usually is. **Necessity:** is processing the personal data actually necessary to achieve it, or could you do it with less or with anonymous data? **Balancing:** do your interests outweigh the person's rights and expectations? Would a reasonable person be surprised or harmed?

You must document this balancing test. It is not a feeling; it is a written assessment you can show.

For AI, legitimate interest can cover some uses — for example, using customer support data to improve service quality, if it is necessary and balanced. But it does not let you do anything a customer would find intrusive. Training a model on sensitive data under "legitimate interest" is very hard to justify. And as the EDPB opinion in the Curiosity section says, the necessity and balancing tests must genuinely pass.

The rule of thumb: if you would feel uneasy explaining the use to the person out loud, legitimate interest probably does not cover it.

## 10.5 Data subject rights: access, rectification, erasure, portability

People hold rights over their data, and you must be able to honour them. The main ones:

**Right to be informed.** You must tell people, clearly, what data you collect and why, usually in a privacy notice.

**Right of access.** A person can ask what data you hold about them and how you use it. This is the subject access request from the example. You must provide a copy, usually within one month, free of charge.

**Right to rectification.** If the data is wrong, they can have it corrected.

**Right to erasure ("right to be forgotten").** They can ask you to delete their data, and you must, unless a legal reason to keep it overrides — for example, a tax record you are required to retain.

**Right to restriction of processing.** They can pause how you use the data in certain situations while a dispute is settled.

**Right to data portability.** They can ask for their data in a structured, common, machine-readable format so they can move it elsewhere. This connects to the lock-in problem in Chapter 9.

**Right to object.** They can object to processing based on legitimate interest, and to direct marketing.

**Rights around automated decisions.** Under Article 22, a person has the right not to be subject to a decision based purely on automated processing that has legal or similarly significant effect on them, with limited exceptions, and the right to human involvement. This is critical for AI that screens people for jobs, credit, or services.

Build a simple, written process to receive and answer these requests on time. When you use a third-party AI, make sure the vendor can help you produce and delete the data, or you cannot honour the request.

## 10.6 The EU AI Act: what changes for privacy

The EU AI Act is a separate law from GDPR, and it is treated in full in Chapter 5. Here is only the privacy angle, so the two do not get confused.

Think of it this way. **GDPR governs the personal data that flows through an AI system. The AI Act governs the AI system itself — its risk level, its obligations, and how it may be used.** If your AI system uses personal data, both apply at the same time. They are not alternatives; they stack.

For a high-risk AI system that processes personal data, you end up doing two related jobs. Under the AI Act, you follow a risk and conformity process for the system. Under GDPR, you need a lawful basis for the personal data and, where the risk is high, a Data Protection Impact Assessment. The good news is that the work overlaps: understanding your data, documenting your process, and assessing risk serve both laws.

The AI Act also pushes on data quality for high-risk systems — training and testing data should be relevant and representative, and as free of errors as appropriate. That lines up with GDPR's accuracy principle.

And the AI Act's transparency duties — telling people they are interacting with AI — sit alongside GDPR's transparency duties.

The practical point: do not treat the AI Act and GDPR as one checklist. Ask two questions about every AI system. *What is the system's risk under the AI Act?* And *what personal data flows through it, and is that lawful under GDPR?* The first is Chapter 5's job; the second is this chapter's.

## 10.7 Data Protection Impact Assessment (DPIA): when it is mandatory and how to do it

A Data Protection Impact Assessment is a structured review you do **before** starting a processing activity that is likely to be high risk to people. For AI, you will often need one.

You must do a DPIA when processing is high risk. Clear triggers include: systematic and extensive evaluation of people based on automated processing that produces significant effects (this covers AI that scores, ranks, or profiles people); large-scale processing of sensitive data; and large-scale systematic monitoring of publicly accessible areas. New technologies used in new ways also raise the risk.

How to do one, in plain steps:

1. **Describe the processing.** What data, what purpose, how long, who sees it, where it is stored, and whether a third party or AI is involved.
2. **Check necessity and proportionality.** Is this the least intrusive way to reach your goal?
3. **Assess the risks to people.** Not risk to your company — risk to their privacy, fairness, and rights. Think about bias, error, over-collection, and re-identification.

4. **List the measures to reduce those risks.** Minimise data, anonymise where possible, add human review, secure the system, set short retention.
5. **Decide.** If a high risk remains after your measures, you must consult your national data-protection authority before proceeding.
6. **Document and revisit.** Write it down and review it when the system changes.

A DPIA is not a form to tick. It is a thinking exercise that, done honestly, often changes your design for the better.

## 10.8 Anonymization and pseudonymization: what they are and why they matter

These two words sound alike and are very different. Confusing them causes real trouble.

**Pseudonymization** means you replace direct identifiers with a stand-in, like a code, and keep the key that links the code to the person separately and securely. "Customer 4471" instead of "Maria Rossi," with the mapping table locked away. Pseudonymized data is **still personal data** under GDPR, because with the key you can re-identify the person. It is a valuable security measure — it reduces risk if the data is stolen — but it does not take you out of the law.

**Anonymization** means you remove identifying information so thoroughly that no one can re-identify a person, even by combining the data with other sources. The law asks whether re-identification is "reasonably likely," considering cost, time, and current technology. Truly anonymous data is **outside GDPR**, because there is no identifiable person. But true anonymity is genuinely hard. Datasets that look anonymous have been re-identified by cross-referencing other public data.

Why it matters for AI: if you want to train a model on data without GDPR applying, you need real anonymity, not pseudonymization. And as the EDPB opinion warns, even a trained model may not be anonymous if personal data can be extracted from it by clever questioning. So "we anonymised the training data" is a claim you must be able to defend, not just assert.

The safe working rule: treat pseudonymized data as personal data, because it is. Only treat data as anonymous if you have tested that re-identification is not reasonably possible. When in doubt, keep the GDPR protections on.

## Ethics and Responsibility

GDPR is the floor, not the ceiling. Compliance means you avoid fines; ethics means you do the right thing even where the law is silent.

**Respect the person behind the data.** Every record is someone's private life. Ask whether your use would feel fair if you were the one being profiled by an AI.

**Do not hide behind "the model did it."** If an AI makes an unfair decision about a person using your data, you are responsible. Keep a human who owns the outcome, as Chapter 4 sets out.

**Be transparent about AI.** Tell people when an AI touches their data and what it does. Surprises erode trust faster than any technical flaw.

**Minimise as a moral habit.** Collecting less is not just a legal rule; it is respect. Every field you do not collect is a field that cannot be leaked or misused.

**Protect the vulnerable.** Sensitive data about health, beliefs, or finances deserves the highest care. If a tool cannot guarantee it, do not put it in the tool.

## Mistakes to Avoid

1. **"We are too small for GDPR."** Size does not exempt you. If you process personal data of people in the EU, the rules apply.
2. **Thinking consent is always needed, or never needed.** It is one basis among six. Use the right one and document why.
3. **Confusing pseudonymization with anonymity.** Pseudonymized data is still personal data and still in scope.
4. **Dumping your whole database into an AI tool.** That breaks data minimisation and spreads risk.
5. **No lawful basis for training on personal data.** "Improve our services" is usually not enough.
6. **Ignoring sensitive data hiding in CVs and tickets.** Health, beliefs, and union activity can be inside ordinary documents.
7. **No DPIA before a high-risk AI rollout.** Do it before, not after a complaint.
8. **No process for access and deletion requests.** If you cannot find and delete the data, you cannot honour the right.
9. **Losing control of data to a third party.** If the vendor cannot help you delete or export, you are exposed. See Chapter 9.
10. **Treating the law as a one-time project.** GDPR is ongoing. Retention, breaches, and requests keep coming.

## Practical Exercise

## 10.10 Your GDPR compliance checklist for AI

Take a half-day and work through this for one AI use case that touches personal data.

1. **Map the data.** List exactly what personal data flows into the AI. Name each field. Mark any sensitive category. (A data inventory template is in the appendices.)
2. **Name the lawful basis.** For each use, write which of the six bases applies and why. If you cannot name one, stop and reconsider.
3. **Check for sensitive data.** If any special-category data is present, confirm a specific condition allows it, or remove it.
4. **Apply minimisation.** Cut every field you do not strictly need.
5. **Decide the destination.** Does the data go to a third party? If so, run the vendor questions from Chapter 9.
6. **Do a DPIA.** If the use is high risk — profiling, large-scale sensitive data, automated decisions — complete the DPIA steps in section 10.7.
7. **Plan the rights.** Write how you will handle access, correction, and deletion requests for this data, including data held by the vendor.
8. **Set retention.** Decide how long you keep the data and when you delete it.
9. **Check automated-decision rules.** If the AI decides about people with significant effect, ensure a human review path exists.
10. **Document everything.** Write the answers down. Accountability means you can show your work.

If any step leaves a blank you cannot fill, that blank is your to-do list.

# Checklist

## 10.11 The documents you must have

For an AI system that touches personal data, keep these documents ready and current.

- ☐ **Privacy notice** that clearly explains what data you collect, why, and how AI is involved.
- ☐ **Record of processing activities** describing each use of personal data.
- ☐ **Lawful basis statement** for each processing purpose, written down.
- ☐ **Data inventory** listing every personal-data field and where it lives.
- ☐ **DPIA** for any high-risk AI processing, with the residual-risk decision.
- ☐ **Balancing test** if you rely on legitimate interest.
- ☐ **Data Processing Agreement** with every vendor that handles your data.
- ☐ **Retention schedule** stating how long each data type is kept and when it is deleted.
- ☐ **Subject-request procedure** for access, rectification, erasure, and portability, with a one-month clock.
- ☐ **Breach response plan** including notifying the authority within 72 hours and affected people when required.
- ☐ **Automated-decision safeguards** with a human review path for significant decisions.
- ☐ **Data Protection Officer contact**, if your processing requires one.

Keep these alive. A document you never update is a document that will fail you when a regulator or a customer asks.

## Key Takeaways

- GDPR applies to any personal data about identifiable people in the EU, including when it flows through an AI tool, and it reaches companies outside the EU that serve EU customers.
- You need a lawful basis for every use; consent is only one of six, and sensitive data needs a specific condition on top.
- People hold real rights — access, correction, erasure, portability — and you must be able to honour them, including for data held by a third-party vendor.
- Pseudonymization is not anonymity; only data that cannot reasonably be re-identified leaves GDPR, and a model that leaks personal data is not anonymous.
- For high-risk AI, do a DPIA before you start, and remember the AI Act and GDPR apply together, not instead of each other.

# Chapter 11 — Digital Sovereignty: The Right to Control Your AI

## In Simple Words

Sovereignty, in its oldest sense, means the right to govern yourself without someone else telling you what to do. Digital sovereignty takes that idea and applies it to your digital life: the right to control your own data, your own software, and the machines that run them.

Applied to AI, digital sovereignty answers three plain questions about every AI tool you use.

- **Where is my data?** Which country, which company's servers, under whose law?
- **Who can access it?** The vendor's staff, subcontractors, or a foreign government?
- **How is it processed?** On a model I cannot see, or on a system I can inspect and change?

If you cannot answer those three questions, you do not have sovereignty over that part of your business. You have borrowed someone else's, and you can lose it whenever they change their mind.

This is not the same as self-hosting, though they overlap. Self-hosting, covered in Chapter 8, is one way to gain control. Digital sovereignty is the wider goal: control over where your digital capabilities come from and who holds the leash. You can pursue it with your own servers, with open software, with contracts, or with a mix.

A useful analogy is food. You can buy prepared meals from a factory you never see, and most days that is fine. But if you have an allergy, or you simply want to know what is in your food, you start to care about the recipe and the kitchen. Digital sovereignty is caring about the recipe and the kitchen of your AI — not out of fear, but because the stakes are your data, your customers, and your future.

The opposite pattern, unmanaged third-party services and shadow AI, is in Chapter 9. The legal duties over personal data are in Chapter 10. This chapter is about the strategic choice: how much of your AI you want to truly control.

## A Bit of History

**2000s to 2010s: convenience erases the question.** As businesses moved to the cloud, almost nobody asked where the data went. The tools were good and cheap. Control was not a buying criterion.

**2013: Snowden changes the mood.** Revelations about mass surveillance made governments and companies realise that data stored in another country could be reached by that country's authorities. The question "where is my data" became a security question, not just a privacy one.

**2018: GDPR makes location matter legally.** Europe's privacy law gave people rights over their data and made companies answerable for where it sat and how it was handled. Data residency — keeping data inside a region — became a real requirement, not a slogan.

**2019 to 2022: "sovereign cloud" appears.** Cloud providers began offering sovereign options: data kept in a specific country, operated

under local law, sometimes with local partners. Sovereignty became a product feature.

**2022 to 2023: open-weight models open a new path.** When strong AI models became downloadable and runnable by anyone, a company could, for the first time, run a capable model entirely on its own terms. Sovereignty was no longer only a cloud vendor's promise; it became something you could build.

**2024 to 2025: sovereignty becomes a national and corporate strategy.** Countries and blocs began treating AI capability as strategic. Europe launched funded projects to build its own open models, so that European AI would not depend entirely on foreign providers. The Curiosity section covers one of them.

The arc is clear. Convenience made us stop asking who controls our tools. A series of shocks made us ask again. Now control is a design choice you can make deliberately.

## Curiosity

### 11.6 Building your own assistant, and a continent doing the same

**A representative pattern (illustrative).** Consider a mid-sized professional firm — say an engineering consultancy or a law practice — that needs an internal AI assistant to answer questions over its own documents: past reports, standards, contracts, and notes. The data is confidential and cannot leave the building.

Instead of sending that data to a public AI service, the firm does something different. It downloads an open-weight model — a model whose trained files are published for anyone to run — and runs it on its

own servers. It connects the model to its documents through retrieval, so the assistant answers from the firm's own files rather than from the open internet. No question leaves the network. No vendor reads the work. The firm chooses the model, controls the data, and can change or replace the system whenever it likes.

This pattern is real and increasingly common, but the specific firm here is a composite, not a single named company, because organisations that do this rarely advertise it — the whole point is that the work stays private. What matters is that the pattern exists and works today with off-the-shelf open tools, exactly as Chapter 8 describes.

**A real, named version at national scale.** The same instinct is now driving whole countries. In Europe, two funded projects aim to build sovereign, open AI models so the continent is not dependent on foreign providers.

**OpenEuroLLM** is a twenty-partner European consortium that began work on 1 February 2025, funded under the EU's Digital Europe Programme with roughly €55 million. It is coordinated by Jan Hajic of Charles University and co-led by Peter Sarlin of AMD Silo AI. Its stated goal is strategic autonomy for Europe in AI — building capability that Europe controls.

Alongside it, the **EuroLLM** project, supported by Horizon Europe, the European Research Council, and the EuroHPC supercomputing organisation, produced **EuroLLM-22B**, a fully open large language model built for all 24 official EU languages. It was trained on European supercomputers — the MareNostrum 5 system — and released as open source on Hugging Face, so anyone can download, inspect, and run it. The work leans on the EuroHPC "AI Factories" initiative, which pools supercomputing capacity across Europe; one call allocated three million GPU hours on the Leonardo Booster at CINECA in Italy for building open training data.

Why tell this story here? Because it shows that "build your own, keep it open, keep it under your control" is not a paranoid hobby. It is now official strategy at the level of nations. The same logic that leads a country to build its own open model leads a small firm to run an open model on its own server. The scale differs; the principle is identical.

## A Real Business Example

### The clinic that could not send its data anywhere

A private medical clinic wants to use AI to summarise patient notes and draft routine correspondence. The data is health data — the most protected kind under the law, as Chapter 10 explains. Sending it to a third-party AI service raises serious legal and ethical problems, and the clinic is not comfortable with it regardless of the law.

So the clinic chooses control. It runs an open model on its own server, inside its own network, behind its own firewall. The model reads only the clinic's notes and answers only clinic staff. Nothing crosses the internet to a vendor. The clinic can say truthfully to patients, regulators, and its own conscience: this data never left our building.

The cost is real. The clinic had to buy hardware and find someone to maintain it. The model is good but not the absolute best available. Some tasks still need a human. But the clinic gained the one thing it could not buy from any vendor: certainty about where its most sensitive data goes.

Now compare the clinic to a firm that did not think about this and pasted patient data into a free chatbot. The difference is not intelligence. It is that one firm asked the three sovereignty questions first, and the other never did.

# How to Do It

## 11.1 What digital sovereignty is: a simple definition

Digital sovereignty is the ability to control your own digital resources and decisions, instead of depending on someone else's.

Broken down, it has three layers.

**Data sovereignty.** You control where your data is stored, who can access it, and under which law it falls. You can move it or delete it.

**Operational sovereignty.** You control the systems that process your data. You can run them, change them, and keep them running even if a supplier disappears.

**Strategic sovereignty.** You control your own direction. Your future is not held hostage by a vendor's pricing, a foreign government's rules, or a supplier's business decisions.

Sovereignty is not all-or-nothing. It is a dial, not a switch. You can be highly sovereign over one process and barely sovereign over another. The goal is to decide, on purpose, where you want the dial set for each part of your business, rather than drifting to whatever is most convenient.

The opposite of sovereignty is dependence you did not choose — a state where a change in someone else's plans forces a change in yours.

## 11.2 Why it matters for companies: knowing where the data is, who accesses it, how it is processed

Sovereignty matters because the three questions have real consequences.

**Where the data is.** Data stored in another country falls under that country's laws. A foreign authority may be able to compel access. A vendor's data centre location is not a minor detail; it sets the legal ground your data stands on.

**Who accesses it.** A third-party service may let its own staff, its support teams, and its subcontractors reach your data, in countries you never agreed to. You may never see a list of them. Sovereignty means you know, or you control, that access.

**How it is processed.** If processing happens inside a closed system you cannot inspect, you cannot verify what it does with your data or whether it is fair. If you run the system, you can look inside.

These matter for three practical reasons. **Compliance:** laws like GDPR require you to know and control data handling. **Security:** every extra party with access is an extra chance of a breach. **Business continuity:** if a vendor fails, raises prices, or is cut off by sanctions, sovereignty is the difference between a setback and a crisis.

Sovereignty is not an ideology. It is risk management for the part of your business that runs on data.

## 11.3 Proprietary models vs open-source models: what really changes

The choice between a proprietary model and an open-source model changes several things at once.

**A proprietary model** is a closed product. You use it through a service or an API. You cannot see how it works, you cannot run it yourself, and you cannot change it. You depend on the owner for access, price, and continuity. The convenience is high; the control is low. If the owner changes terms or stops the service, you adapt or you stop.

**An open-source or open-weight model** publishes its code or its trained files so you can download and run it yourself. You can inspect it, run it on your own hardware, fine-tune it, and keep using it even if the original creator vanishes. The control is high; the convenience is lower, because you must run and maintain it.

What really changes is **who holds the power and who carries the burden.** Proprietary hands you power to the vendor and carries the burden for you. Open hands you the power and gives you the burden.

Neither is automatically better. Proprietary is right when you want capability with no maintenance and the data is not sensitive. Open is right when you need control, transparency, or independence, and you can support the work. Many businesses use both: proprietary for low-stakes convenience, open and self-run for the data and processes that matter most.

A caution: “open” is a spectrum, and the licence matters. Some open models restrict commercial use or how you may describe your use. Read the licence before you build on it.

## 11.4 The role of open source: transparency, control, community

Open source is the main tool of digital sovereignty, for three reasons.

**Transparency.** Because the code or weights are published, you or a third party can read them and check what the system actually does. You are not asked to trust a black box. This is the same instinct behind the auditing ideas in Chapter 7.

**Control.** You can run it where you choose, change it to fit your needs, and keep it as long as you want. No one can take it away from you or force an upgrade you did not ask for.

**Community.** An open project is maintained by many people, not one company's roadmap. Bugs get found by outsiders. The project can survive the original team. You are not alone if something breaks.

Open source also lowers the barrier to entry. A small firm can use the same open model as a large one. That levels the field in a way proprietary products do not.

But open source is not free in the real sense. Someone must install it, update it, secure it, and support it. If you have no one to do that, open source can become a liability. Sovereignty you cannot maintain is worse than comfortable dependence. Use open source where you have, or can hire, the capability.

## 11.5 How to build a sovereign AI infrastructure: the concrete steps

If you decide to gain control, here is a practical path.

1. **Decide what must be sovereign.** List the data and processes that cannot depend on outsiders — sensitive data, core workflows, regulated work. Not everything needs the treatment.
2. **Choose open models.** Pick open-weight models whose licence fits your use. Match the size to your task, as Chapter 8 explains.
3. **Run them on infrastructure you control.** That can be a server in your office, a machine in your own cloud account under your own keys, or a local workstation for a small start.
4. **Keep the data local and connected.** Use retrieval so the assistant answers from your own documents without sending them out.
5. **Control access and keys.** You hold the credentials. No third party has a back door. Log who does what.
6. **Plan for maintenance.** Name who installs, updates, secures, and backs up the system. Write the runbook.
7. **Keep an exit from everything.** Even for the parts you keep proprietary, ensure you can export data and swap tools. Sovereignty includes the ability to leave.
8. **Review the legal ground.** Confirm data residency and the AI Act and GDPR duties for the setup, as Chapter 5 and Chapter 10 cover.
9. **Start small and grow.** Pilot one sovereign workflow, prove it works, then extend. Do not rebuild the whole company at once.

The aim is not total independence. It is deliberate control over the parts that matter, with a clear line between what you run and what you rent.

## Ethics and Responsibility

Sovereignty carries its own ethical duties, and it is easy to get them wrong.

**Control is not the same as good.** A sovereign system can still be biased, wrong, or unfair. Running it yourself does not make it ethical. The duties in Chapter 4 still apply.

**Sovereignty must not become secrecy.** "We control it" is not a reason to hide how you treat people's data. You still owe transparency to customers and regulators. Control the system; do not hide from scrutiny.

**Do not use sovereignty to dodge cooperation.** Some problems — fraud, harm, legal requests — require working with authorities. Sovereignty is about protecting the innocent, not blocking legitimate oversight.

**Be honest about your limits.** If you claim full sovereignty but cannot patch, back up, or audit the system, the claim is empty and misleading. Say what you control and what you do not.

**Weigh the collective good.** Open models and shared research benefit everyone. A world where every firm builds a closed silo loses something. The healthiest approach is to control what is sensitive and contribute to what is shared.

## Mistakes to Avoid

### 11.7 Sovereignty for its own sake

The most common mistake is chasing sovereignty as an ideal instead of a tool.

Sovereignty costs money, time, and skills. If you pursue it everywhere, you spend heavily to control things that never needed control. You might build an expensive on-premise system for data that was never

sensitive, while a cheap, contracted service would have served you fine and freed your people for real work.

The test is not "can I control this?" It is "what do I lose if I cannot control this?" If the answer is little, do not spend to control it. Save your sovereignty budget for the data and processes where losing control would actually hurt — sensitive data, regulated work, and the systems your business cannot run without.

Sovereignty is a means, not an end. Buy it where the risk justifies the cost, and be comfortably dependent where it does not.

Beyond that trap, watch for these:

1. **Building a fortress you cannot maintain.** Sovereignty without a maintainer becomes a broken system and a false sense of safety.
2. **Confusing open source with free.** Someone must run it. Budget for that.
3. **Ignoring the licence.** Open does not always mean you may use it however you like.
4. **Assuming local equals safe.** A sovereign server still needs security, as Chapter 6 covers.
5. **Cutting off all vendors out of principle.** Many vendors earn their place. Total independence is rarely worth it.
6. **No exit plan for the parts you keep rented.** Sovereignty includes the ability to leave any single provider.
7. **Overpromising to customers.** Do not claim control you do not have.
8. **Ignoring that open models can still leak.** A model trained on bad data can still expose personal data, as Chapter 10 notes.

## Practical Exercise

### 11.8 Define your desired level of sovereignty

Take one hour and map your AI uses onto a simple control dial.

List each AI use in your business. For each, answer three questions and set a target level.

- **How sensitive is the data?** Low (public or harmless), Medium (internal), High (client-confidential, regulated, or personal).
- **How critical is the process?** Low (nice to have), Medium (used daily), High (the business stops without it).
- **What is the cost of losing control?** Low, Medium, or High.

Now set a target for each:

- **High sensitivity or high criticality → aim high.** Run it yourself or under a contract that gives you real control and an exit.
- **Low sensitivity and low criticality → aim low.** A convenient third-party service is fine; do not waste money controlling it.
- **Mixed → aim middle.** Use a vendor but keep the data exportable and the workflow swappable.

Write one sentence per use: *"For [use], I want [high/medium/low] sovereignty because [reason]."*

The result is your sovereignty map. It should show a deliberate mix, not a single answer. If everything is "high," you are over-spending. If everything is "low," you are exposed on the things that matter. Adjust until the map matches your real risk.

## Checklist

### 11.9 The pillars of digital sovereignty

Use this to check whether a given AI setup gives you real control.

- ☐ **You know where the data is stored** — the country and the operator.
- ☐ **You know who can access it** — vendor staff, subcontractors, and any government reach.
- ☐ **You can move or delete the data** whenever you choose.
- ☐ **You can export your data** in a common, usable format (no lock-in).
- ☐ **You know how the data is processed** — or you run the processing yourself.
- ☐ **You use open models where control matters**, and you have read their licences.
- ☐ **You hold the keys and credentials** to the systems that matter.
- ☐ **You have a named maintainer** for anything you run yourself, with a written runbook.
- ☐ **You have an exit plan** for every vendor, so no single provider can hold you hostage.
- ☐ **You have checked the legal ground** — data residency, GDPR, and the AI Act.
- ☐ **You have set a deliberate sovereignty level** for each use, not a default.
- ☐ **You can verify your own claims** — if you say the data stays in, you can prove it.

If a pillar is missing for a high-stakes use, that is your priority to fix.

## Key Takeaways

- Digital sovereignty means controlling where your data is, who accesses it, and how it is processed — and being able to answer those three questions for every AI tool.
- It is a dial, not a switch: set it deliberately high for sensitive and critical work, and low where convenience is harmless.
- Open-source and open-weight models give you transparency, control, and community, but only if you have the capability to run and maintain them.
- Sovereignty is a means, not an end; chasing it everywhere wastes money, so buy it only where losing control would actually hurt.
- Real sovereignty always includes an exit: keep your data exportable so no single vendor can hold your business hostage.

# Chapter 12 — Where AI Can Help Your Business

## In Simple Words

AI is not a single tool that fits every job. It is a set of capabilities, and each one fits some tasks well and others badly. The first real question is not “should we use AI?” It is “where, exactly, will it help us?”

Think of AI like power tools in a workshop. A drill is wonderful for making holes. It is useless for stirring paint, and dangerous for cutting bread. The skill is not owning the tools. The skill is knowing which tool matches which job. AI is the same. It is strong at some things and weak at others, and your advantage comes from pointing it at the right tasks.

What is AI genuinely good at, in business terms? Three broad things. It can **read and understand** unstructured information — emails, invoices, contracts, notes — the kind of text and images that used to need human eyes. It can **write and produce** text, summaries, drafts, and answers at a speed no person can match. And it can **spot patterns and suggest decisions** from large amounts of data — which customer is likely to leave, which machine is likely to fail, which stock will run out.

What is AI not good at? It is not good at tasks that need firm facts it does not have, at real accountability, at physical work, or at judging situations it has never seen. It makes mistakes. It can be confidently wrong. So you point it at work where a fast draft you can check is valuable, not at work where a single error ruins you.

This chapter is the map. It shows you how to look at your whole business, find the activities where AI fits, and decide where to start. The method is simple and works whether you run a shop, a firm, or a factory. Later chapters cover the tools and the how-to. This one answers the "where."

The single most useful idea here is the **impact/ease matrix**, and this book keeps its full explanation in this chapter. Once you learn it, you can look at any task and know roughly whether to do it now, later, or never.

## A Bit of History

**1960s–1980s: automation meant factories.** For decades, "automation" meant machines doing physical work on an assembly line. Office work and thinking work were untouched. If you were not in manufacturing, automation was someone else's story.

**1990s: software entered the office.** Spreadsheets, databases, and early workflow tools automated calculations and record-keeping. This was automation by rules: you told the software exactly what to do, step by step. It was powerful but rigid. Anything that did not fit the rules still needed a person.

**2000s: business process management.** Companies began mapping their processes on paper and diagrams, looking for waste and bottlenecks. The discipline of "map the process first" came from here. It is still the right first step for AI today.

**2010s: AI reached language and images.** Machine learning got good enough to read text, recognise images, and handle messy inputs that rule-based software could not touch. Suddenly the "thinking work"

that resisted automation became reachable. This is the shift that opened AI to ordinary businesses, not just factories.

**2020s: generative AI made it universal.** Tools that write, summarise, and answer in plain language arrived. Now a business with no engineers could point AI at emails, reports, and customer messages. The question moved from "can we?" to "where should we?" — which is exactly this chapter.

The lesson from this history: every wave of automation first touched the most repetitive, rule-like work, then moved toward messier tasks as the tools improved. AI follows the same path. The best places to start are still the repetitive and predictable ones — now with tools that handle far more variety than before.

## Curiosity

### 12.5 The company that automated 96% of its planning

One of the clearest proofs of what "where AI helps" looks like in practice: a healthcare services company automated **96% of its planning work** and cut **waiting times by 57%**. Planning — matching staff to shifts, routes, and schedules — is exactly the kind of repetitive, rule-heavy, time-consuming task AI handles well, and the gain was not just saved hours but faster service for the people waiting.

This book tells the full TridentCare case in Chapter 27, including how they did it and what they learned. The point here is only the headline: when you find the right activity, the result can be dramatic. (The specific figures come from that case study; treat them as that company's reported results, not a promise for every business.)

## A Real Business Example

### A five-person accounting firm finds its real target

A small accounting firm of five people spent a large part of every week on one thing: collecting client documents and entering data. Clients emailed receipts, bank statements, and invoices in every format imaginable. Two staff members spent hours each day opening attachments, reading them, and typing figures into the accounting system.

They did not start by buying anything. They started by mapping. They wrote down every activity the firm did in a week and marked, for each one, how repetitive it was and how much time it took. The document-and-data task stood out immediately: high repetition, high hours, low judgment. It was a textbook target.

They tried AI on just that piece. The tool read incoming documents, pulled out the figures, and filled the system. Staff checked the result instead of typing it. The work did not vanish, but it shrank from hours to minutes per client, and the two staff moved to advisory work the firm could bill for.

Notice what made this work. They did not try to "use AI everywhere." They found one activity that matched the pattern — repetitive, predictable, high-volume, low-risk if checked — and aimed carefully. That is the whole method of this chapter in one small firm.

## How to Do It

## 12.1 Mapping business processes

You cannot choose where AI helps until you can see what your business actually does. That means mapping your processes. A process is just a repeating set of steps that turns an input into an output: an order comes in, you pick, pack, ship, and invoice. A CV arrives, you screen, interview, and decide.

To map a process, write down the steps in order. For each step, note four things:

- **What triggers it.** What starts this step? An email, a form, a customer click.
- **What happens.** The actual action — read, type, decide, approve, send.
- **Who does it.** Which person or role.
- **How long it takes and how often.** Minutes per task, tasks per day or week.

Do this for your main processes. Keep it simple — a list or a simple table is enough. Do not buy software to map; a spreadsheet or paper works. The goal is to see, on one page, where the time goes and where the repetition lives.

Two tips. First, map what people *actually* do, not what the official procedure says. The gap between the two is often where the waste hides. Second, talk to the people doing the work. They know exactly which steps are painful, repetitive, and slow. They are your best source.

Once the map is done, you have a list of activities you can test against the filters in the next section. The mapping itself often reveals opportunities you had stopped noticing because they were invisible inside daily routine.

## 12.2 Repetitive, predictable, and decision-making activities

Not all work suits AI equally. Three qualities make an activity a strong candidate.

**Repetitive.** The same task, over and over. Repetition is opportunity because a tool that learns the task once can repeat it cheaply. A task you do once a year is rarely worth automating; one you do a hundred times a day almost always is. Repetition is the single strongest signal.

**Predictable.** The task follows a recognizable pattern, with clear inputs and a clear idea of a correct output. "Turn this invoice into these fields" is predictable. "Invent a new marketing campaign" is not. Predictability means the AI has a stable target to aim at. Some variation is fine — modern AI handles variety — but the underlying shape must be stable.

**Decision-making (with data).** Tasks where you make a judgment using information can benefit from AI's pattern-spotting. "Which of these leads should we call first?" "Which invoice looks unusual?" "Which customer might cancel?" AI can rank, flag, and suggest. The key is that a decision is being made from data, and a faster or better suggestion has value.

Now the flip side. Activities that are **one-off, creative, ambiguous, or require firm accountability** are weak candidates. AI can assist them, but it should not own them. A first draft of a strategy memo is fine for AI. The final strategic decision is yours.

A simple test for any activity on your map: *Is it repetitive? Is it predictable? Does it involve a decision from data?* If you can answer yes to at least two, it is worth a closer look. If all three are no, leave it to people.

## 12.3 The typical areas: administration, sales, marketing, customer care, HR, operations, IT, leadership

Every business has roughly the same set of functions, and each has its own good AI targets. Here is where AI typically helps, area by area. This is a survey; the deep treatment of each is in Part VII of this book.

**Administration and finance.** Document reading, invoice processing, data entry, expense checks, report drafting, reconciliation. High repetition, high volume — among the best starting points. Detailed in Chapter 25.

**Sales.** Lead scoring, drafting outreach emails, summarising calls, updating the CRM, suggesting next actions. AI is strong at the writing and the ranking. See Chapter 26.

**Marketing.** Drafting copy, generating variations, summarising campaign results, segmenting audiences, personalising messages. Creative assist, not creative replacement. Also Chapter 26.

**Customer care.** Answering common questions, drafting replies, routing tickets, summarising history, building knowledge bases. One of the highest-value areas because volume is high and many questions repeat. The TridentCare case sits in Chapter 27.

**HR and people.** Screening CVs against a role, drafting job posts, answering employee policy questions, summarising feedback. Careful: this touches personal data and bias, so the duties in Chapter 4 and Chapter 10 apply strongly. See Chapter 28.

**Operations and production.** Demand forecasting, scheduling, inventory, quality checks, maintenance prediction. Decision-from-data

tasks with real money attached. Chapter 29.

**IT.** Code assistance, ticket triage, documentation, monitoring alerts. Chapter 30.

**Leadership.** Summarising reports, drafting communications, surfacing trends. AI as a staff function for you, not a decision-maker. Also Chapter 30.

The pattern across all areas is the same: aim at the repetitive, predictable, data-driven pieces, and keep a human on the judgment and the accountability.

## 12.4 The "impact/ease" matrix: where to start

This is the canonical home of the impact/ease matrix. Learn it once and use it everywhere.

The matrix is a simple way to rank every candidate activity so you know what to do first. You score each activity on two axes.

**Impact (vertical axis).** How much value you gain if you automate this activity well. Impact can be time saved, money saved, errors avoided, faster service, or revenue gained. Ask: *if this activity got dramatically better, how much would it help the business?* High impact means it matters a lot. Low impact means it barely moves the needle.

**Ease (horizontal axis).** How easy it is to automate this activity with AI. Ease depends on how repetitive and predictable the task is, how clean the data is, how simple it is to connect the tool, and how low the risk is if it makes a mistake. Ask: *how hard is it to do this, and how much can I safely let AI handle?* High ease means simple, low-risk, ready data. Low ease means messy, risky, or hard to connect.

Now plot every candidate activity into one of four boxes.

**High impact, high ease — Start now.** These are your quick wins. They matter a lot and they are easy to do. This is where your first projects live. Do these immediately. They pay back fast and build confidence.

**High impact, low ease — Plan carefully.** These matter but are hard. They are worth doing, but not first. Put them in a longer project, budget for the difficulty, and tackle them after you have proven the approach with quick wins. Do not start here; you will burn money and goodwill on a hard problem before you have experience.

**Low impact, high ease — Do opportunistically.** Easy but not very valuable. Fine to do when you have spare time, or as a gentle learning exercise, but do not spend real budget here. They are distractions if you treat them as priorities.

**Low impact, low ease — Ignore.** Hard and not worth it. Skip these entirely. They are traps that look interesting but never pay off.

The discipline of the matrix is what makes it powerful. Most people's instinct is to chase the impressive high-impact, low-ease projects because they sound important. The matrix tells you to resist that and start with high-impact, high-ease instead. You build momentum, learn, and fund the harder work later.

To use it: take your mapped activities, score each one low/medium/high on impact and on ease, and place it in a box. List the "start now" box first. That list is your first set of priorities. Keep the scoring honest — it is normal to want to inflate impact and hide difficulty. Resist both.

A caution on ease: ease is not only about the technology. A task can be technically easy but hard because the data is messy, the staff resist it, or the risk of error is high. Count those in. True ease includes people, data, and risk, not just "can the tool do it."

## Ethics and Responsibility

Choosing where to use AI is itself an ethical choice, not just a financial one.

**Watch the high-stakes areas.** HR screening, customer decisions, anything touching people's opportunities or money carries risk. A wrong AI decision there can harm a real person. In those areas, AI should assist, never decide alone. The fairness and transparency duties in Chapter 4 apply, and the law in Chapter 5 may classify some of these as higher-risk uses.

**Do not automate away accountability.** Even when AI does the work, a human remains responsible for the outcome. Never use "the AI did it" to escape responsibility. Someone must own every automated decision that affects a person.

**Consider the people whose work changes.** Automating a task changes the lives of the people who did it. Do it with honesty and a plan for them, not as a surprise. The change-management side is in Chapter 21.

**Be honest about what AI can and cannot do.** When you pick a target, do not oversell it internally or to customers. If the tool is a draft-maker, call it that. Setting honest expectations is part of responsible use.

## Mistakes to Avoid

1. **Starting with the hardest project.** The impressive, complex automation is a trap. Start with a quick win, not a moonshot.
2. **Automating a broken process.** If a process is messy or wrong, automating it just makes bad work faster. Fix the process before you automate it.
3. **Chasing AI everywhere at once.** Spreading thin across many small uses wastes effort. Pick a few high-impact, high-ease targets and do them well.
4. **Ignoring the ease side.** A high-impact task with messy data and high risk is not actually easy. Count data quality and risk in your ease score.
5. **Choosing by fashion, not fit.** "Everyone is using AI for X" is not a reason. Choose by your own map and matrix.
6. **Forgetting the human in the loop.** In any task that affects a person, keep a human checking. Full automation of high-stakes decisions is reckless.
7. **Skipping the mapping.** Picking targets by gut feel, without mapping, means you miss the best ones and pick the wrong ones.
8. **Confusing a demo with a workflow.** A tool that dazzles in a demo may not fit your real, messy process. Test against your actual activities.

## Practical Exercise

### 12.6 List 10 activities you could automate

Set aside an hour and do this simple exercise.

Write down **ten activities** in your business that you could imagine automating. They can come from your process map, or from what annoys you daily. For each one, add three quick notes:

- **How repetitive?** Roughly how many times per week do you or your team do it?
- **How predictable?** Can you describe the correct output clearly? Yes / sort of / no.
- **What would AI do?** One phrase: read it, draft it, rank it, check it.

Example entries:

1. Reading incoming invoices and entering figures — 50×/week — yes — extract and fill.
2. Replying to common customer emails — 80×/week — sort of — draft a reply.
3. Summarising weekly sales reports — 1×/week — yes — summarize.
4. Screening CVs for a role — 20×/week — sort of — rank against criteria.
5. Scheduling staff shifts — 2×/week — yes — propose a schedule.

Keep going until you have ten. Do not judge them yet; just collect. The goal is a pool of candidates.

Next, take your ten and score each one **impact** (low/medium/high) and **ease** (low/medium/high) using the matrix from section 12.4. Place each in a box. Circle everything in the **high-impact, high-ease** box. Those circles are your first priorities. You now have, from one hour of work, a short honest list of where to start.

## Checklist

### 12.7 Choosing your first priorities

Before you commit to a first AI project, check these.

- ☐ **You have mapped your main processes** and can see where time and repetition live.
- ☐ **You have at least 10 candidate activities** written down.
- ☐ **Each candidate is scored on impact and ease** using the matrix.
- ☐ **Your top picks are high-impact AND high-ease**, not just impressive.
- ☐ **The chosen activity is repetitive and predictable**, with a clear correct output.
- ☐ **The data needed is available and reasonably clean** (see Chapter 14).
- ☐ **The risk of an error is acceptable**, or a human check is built in.
- ☐ **A human stays accountable** for any decision that affects a person.
- ☐ **You are not automating a broken process** — the process is sound first.
- ☐ **You can measure the result** — time saved, errors cut, or money gained (see Chapter 16).
- ☐ **You resisted the tempting-but-hard project** and chose a genuine quick win.
- ☐ **The people affected know and are not blindsided.**

If your top pick fails several of these, move to the next one. Your first project should be the easiest high-value thing, not the most exciting.

## Key Takeaways

- AI fits repetitive, predictable, data-driven tasks; it is weak at one-off, ambiguous, or fully accountable work, so aim it at the right jobs.
- Map your processes first — you cannot choose where AI helps until you can see what your business actually does and where the repetition lives.
- The impact/ease matrix sorts every candidate into start-now, plan-carefully, do-when-idle, or ignore; begin with high-impact, high-ease quick wins.
- Resist the impressive hard project; start small, prove the method, and fund the harder work later.
- In any task that touches a person, keep a human checking and accountable — automation never removes responsibility.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

# A market analysis report

AgentBridge
Market Research & Advisory

# REPORT

Specialty Coffee Retail Market Analysis 2026

| Author | Date | Period | Status |
|---|---|---|---|
| AgentBridge Research Team | 21 September 2026 | 2024–2026 | Final |

## 1. Executive Summary

This report summarizes the market analysis of the specialty coffee retail segment — size, trends, competition and outlook — for the period 2024–2026. It was prepared by AgentBridge Research Team at the request of the management board and is based on AgentBridge market data, published industry figures and comparative analysis of the competitive landscape.

Overall, the specialty coffee retail market remains in a solid growth phase, expanding from EUR 1.6 billion in 2024 to EUR 2.1 billion in 2026, with subscription models, single-origin beans and online ordering acting as the strongest growth drivers; a few large chains dominate the segment, but independent shops win on quality and local loyalty, and the outlook is favourable for small players who pair quality products with a strong online presence. The key findings are set out in Section 3 and the resulting recommendations in Section 6; the recommendations focus on e-commerce capability, subscription offers, sourcing transparency and local differentiation.

## 2. Background

The specialty coffee retail segment has moved from a niche positioning to a mainstream category of the coffee market. Growth has been supported by rising consumer interest in product quality, origin traceability and brewing at home. At the same time, digital channels have changed how customers discover, order and reorder coffee, and competition between large chains and independent shops has intensified.

Against this background, this report sets out to quantify the size of the specialty coffee retail market for 2026, to identify the trends that are driving growth, to compare the positioning of large chains and independent shops, and to assess the outlook for small players. The findings are intended to inform assortment, channel and investment decisions for the coming planning period.

## 3. Findings

The following findings were identified during the review:

1. The specialty coffee retail market reached EUR 2.1 billion in 2026, up from EUR 1.85 billion in 2025 and EUR 1.6 billion in 2024.
2. Subscription models, single-origin beans and online ordering are the strongest growth drivers in the segment.
3. A few large chains dominate the segment, while independent shops win on quality and local loyalty.
4. The outlook is favourable for small players who pair quality products with a strong online presence.

| # | Finding | Impact | Evidence |
|---|---|---|---|
| 1 | Market size reached EUR 2.1 billion in 2026, continuing a steady upward trajectory. | Confirms a growing addressable market and supports volume-based investment decisions. | EUR 2.1B (2026), EUR 1.85B (2025), EUR 1.6B (2024). |
| 2 | Subscription models, single-origin beans and online ordering are the strongest growth drivers. | Shifts revenue toward recurring, digitally acquired customer relationships. | Trend assessment of the segment, 2026. |
| 3 | A few large chains dominate, while independent shops win on quality and local loyalty. | Differentiation, not scale, is the viable competitive basis for small players. | Competitive landscape comparison, 2026. |

## 4. Analysis

The market has grown consistently over the review period. Measured against EUR 1.6 billion in 2024, the segment grew to EUR 1.85 billion in 2025 and to EUR 2.1 billion in 2026 — an increase of approximately 31% over two years. Growth is therefore not a short-term effect but a structural expansion of the category.

On the demand side, three drivers stand out. Subscription models convert occasional buyers into recurring customers and stabilise revenue. Single-origin beans support higher price points and attract quality-oriented consumers. Online ordering lowers the barrier to trial and repeat purchase and extends reach beyond the catchment area of a physical shop. Together, these drivers explain the largest part of the observed growth.

On the supply side, a few large chains dominate the segment through scale, distribution and brand visibility. Independent shops compete successfully on quality and local loyalty, which are difficult for chains to replicate at scale. The interpretation is that the segment is not a winner-takes-all market: small players can hold and grow profitable positions provided they combine product quality with a strong online presence, which is precisely where the growth drivers and their own strengths overlap.

## 5. Conclusions

- The specialty coffee retail market is growing steadily and reached EUR 2.1 billion in 2026.
- Subscription models, single-origin beans and online ordering are the strongest growth drivers and should be treated as core rather than optional channels.
- Although a few large chains dominate, the outlook is favourable for small players who pair quality products with a strong online presence.

## 6. Recommendations

1. Introduce or expand a coffee subscription model to secure recurring revenue and improve customer retention.
2. Strengthen the single-origin assortment and communicate origin, roast and quality attributes at the point of sale and online.
3. Invest in online ordering and e-commerce capability, including delivery and click-and-collect options.
4. Differentiate on quality and local loyalty rather than competing with large chains on price and scale.

*A detailed PDF report with charts and sources*

**What you ask:** `Produce a PDF market analysis of the specialty coffee retail segment: size, trends, competitors and outlook.`

The agent researches the topic and writes a structured PDF report with sections, figures and cited sources — the kind of document that normally takes a consultant days.

*Tip: Tell it the depth you need. 'Short brief' or 'detailed report' changes the length.*

---

## Research, written up

AgentBridge
Research & Advisory Services

# Report

Taking a Small Bakery Online

Author
AgentBridge Research Team

Date
21 September 2026

Period
September 2026

Status
Final

## 1. Executive Summary

This report summarizes the simplest ways for a small bakery to take orders online for the period September 2026. It was prepared by AgentBridge Research Team at the request of the bakery owner and is based on a comparison of three options: a third-party marketplace, an own simple shop, and a full e-commerce platform.

Overall, the simplest ways for a small bakery to take orders online are a marketplace or an own simple shop, and the own simple shop offers the best long-term value. The key findings are set out in Section 3 and the resulting recommendations in Section 6; the recommended approach is to start with a simple own shop and add click-and-collect for lower fees and a direct customer relationship.

## 2. Background

The bakery currently takes orders in person and by phone. Customers increasingly expect to order online, and the owner wants to add an online ordering channel without a large investment of time or money. Three options were considered: Option A — Marketplace, Option B — Own simple shop, and a full platform.

The objective of this report is to compare the options on setup effort, cost, fees and customer ownership, and to recommend the simplest route that protects long-term value for a small bakery.

## 3. Findings

The following findings were identified during the review:

1. Option A — Marketplace is fast to start, but fees are high and the bakery does not own the customer relationship.
2. Option B — Own simple shop is a small branded store with more control, moderate setup effort and the best long-term value.
3. The first-year cost estimate for the own shop is EUR 3,100, compared with EUR 4,200 for the marketplace and EUR 9,800 for the full platform.
4. Adding click-and-collect to an own shop lowers fees and strengthens the direct customer relationship.

| # | Finding | Impact | Evidence |
|---|---|---|---|
| 1 | Option A — Marketplace is fast to start, but fees are high and the bakery does not own the customer relationship. | High recurring fees and no customer data for repeat marketing. | Option comparison, first-year cost estimate EUR 4,200. |
| 2 | Option B — Own simple shop is a small branded store with more control, moderate setup effort and the best long-term value. | Lower long-term cost and full control of branding and pricing. | Option comparison, first-year cost estimate EUR 3,100. |
| 3 | The first-year cost estimate for the own shop is EUR 3,100, compared with EUR 4,200 for the marketplace and EUR 9,800 for the full platform. | The own shop saves EUR 1,100 in year one versus the marketplace. | First-year cost estimate figures. |
| 4 | Adding click-and-collect to an own shop lowers fees and strengthens the direct customer relationship. | Lower transaction costs and improved customer retention. | Recommendation in the context material. |

## 4. Analysis

The three options were assessed on setup effort, ongoing fees, control and customer ownership. Option A — Marketplace requires the least setup effort but charges high fees and keeps the customer relationship with the marketplace. Option B — Own simple shop requires moderate setup effort but gives the bakery a small branded store with more control. A full platform offers the most features but at EUR 9,800 in the first year is disproportionate for a small bakery.

First-year cost estimate: Marketplace EUR 4,200; Own shop EUR 3,100; Full platform EUR 9,800. The own shop is therefore EUR 1,100 cheaper than the marketplace in the first year and EUR 6,700 cheaper than the full platform.

Beyond cost, the decisive factor is customer ownership. An own shop keeps customer data and order history with the bakery, which supports repeat orders and direct marketing. Adding click-and-collect reduces delivery costs and fees while matching how local bakery customers prefer to buy. The marketplace remains a useful secondary channel for discovery, but it should not be the primary ordering channel.

## 5. Conclusions

- The simplest ways for a small bakery to take orders online are a marketplace and an own simple shop; both can be launched quickly.
- The own simple shop costs less in the first year (EUR 3,100) than the marketplace (EUR 4,200) and far less than the full platform (EUR 9,800).
- An own shop with click-and-collect gives lower fees and a direct customer relationship, which is the best long-term value.

## 6. Recommendations

1. Start with a simple own shop as the primary online ordering channel.
2. Add click-and-collect to lower fees and build a direct customer relationship.
3. Keep a marketplace listing only as a secondary channel for discovery, and monitor its fees.
4. Defer the full platform until order volume and revenue justify the EUR 9,800 first-year cost.

High-priority recommendations (start with a simple own shop; add click-and-collect) should be implemented by December 2026; the

*A research report PDF with options and a recommendation*

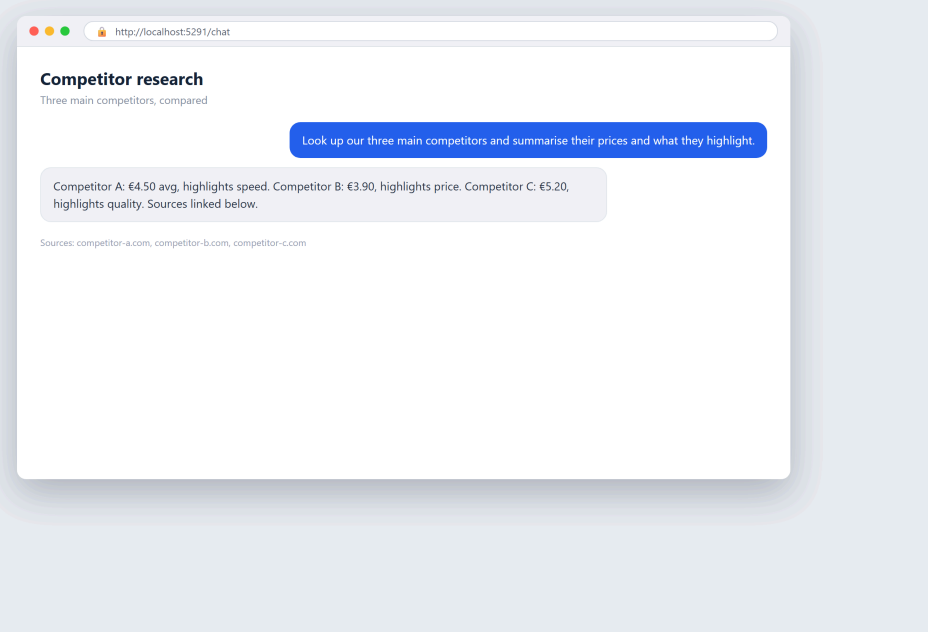
**What you ask:** `Research the best ways for a small bakery to take orders online and write a PDF report with the options and a recommendation.`

The agent searches the web, weighs the options and writes a clear report ending with a practical recommendation you can act on.

*Tip: Ask it to list the sources so you can check anything yourself.*

---

## Know your competitors




*A browser view of the agent's research findings*

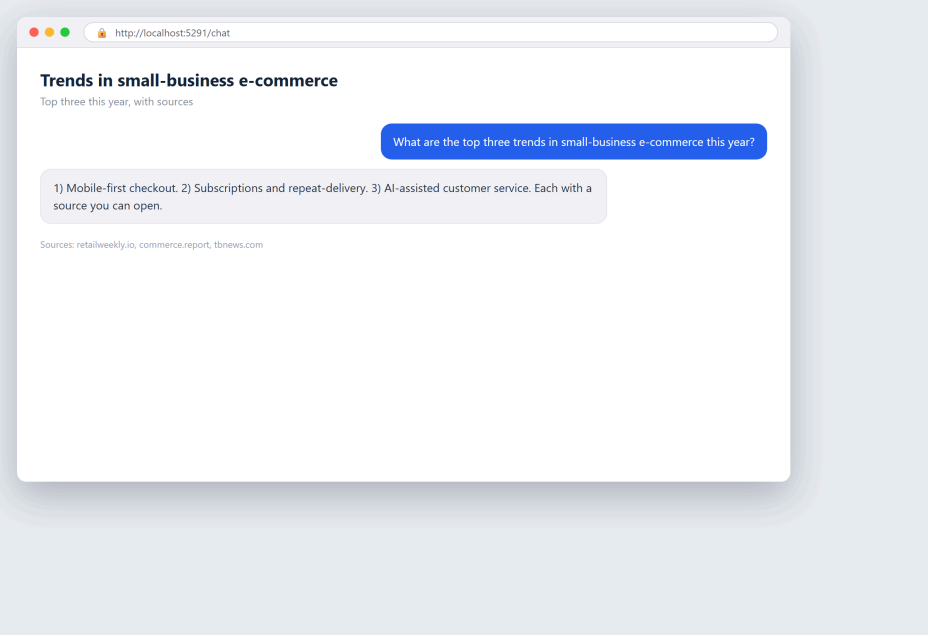
**What you ask:** `Look up our three main competitors and summarise their prices and what they highlight.`

The agent browses the competitors' pages and brings back a clear comparison of prices and positioning, with the links so you can check.

*Tip: Ask for it as a document or a spreadsheet if you want to keep the comparison.*

---

## What is trending in your field




*A research summary with sources*

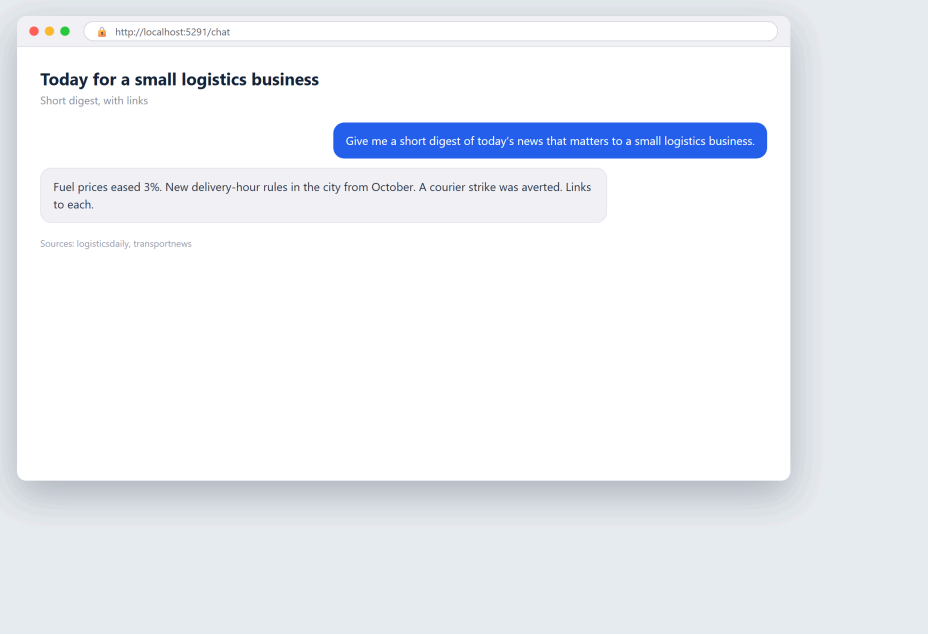
**What you ask:** `What are the top three trends in small-business e-commerce this year? Give me sources.`

The agent searches recent articles and gives you the key trends in plain language, each with a source you can open.

*Tip: A weekly trend digest can be scheduled to keep you current automatically.*

---

## Your industry news, summarised




*A short news digest with links*

**What you ask:** `Give me a short digest of today's news that matters to a small logistics business.`

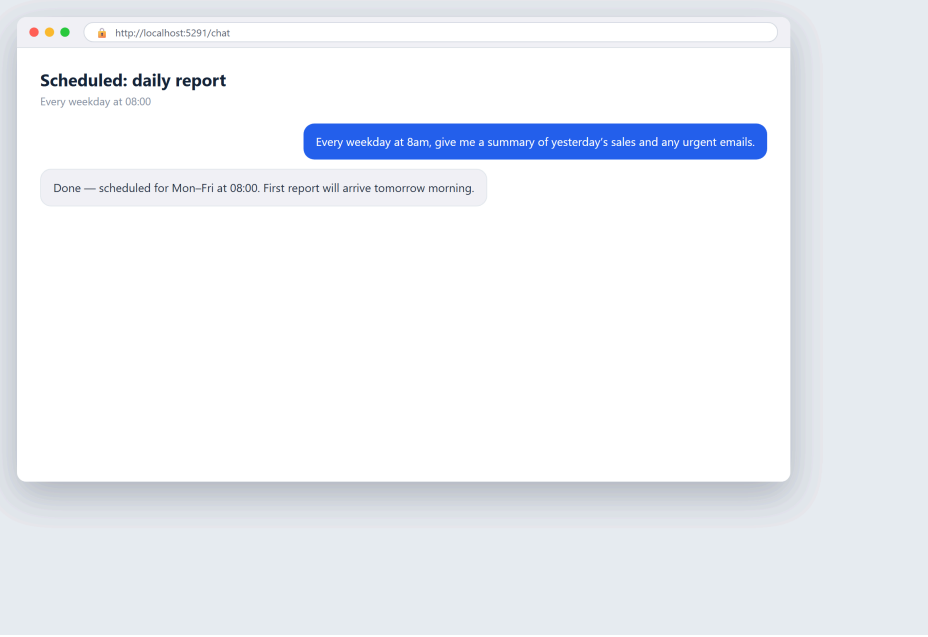
The agent scans the news and writes a short digest of what is relevant to you, with links to read more.

*Tip: Schedule it for every morning and start the day informed.*

# Scheduled Tasks: Work That Runs by Itself

Most tools wait for you to ask. AgentBridge can also work on a time you agree on. You tell it once what to do and when, and it does it every time, without being reminded. This is one of the easiest ways to save time: the task runs, and the result is waiting for you.

## A report that writes itself each morning




*A scheduled task set up to run each morning*

**What you ask:** `Every weekday at 8am, give me a summary of yesterday's sales and any urgent emails.`

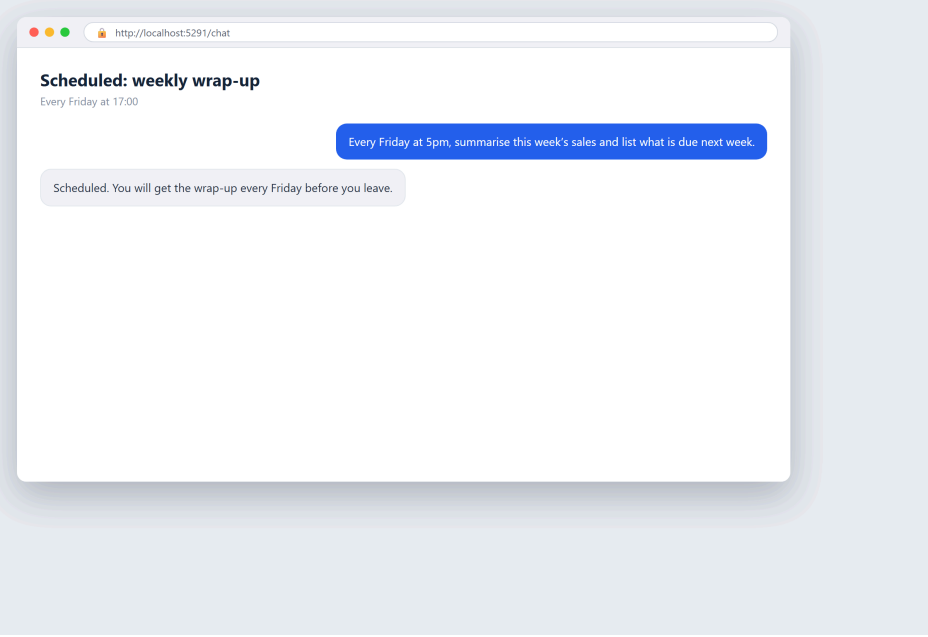
The agent runs the report on its own each morning and has it waiting for you. You agreed the time once; it never forgets.

> This is a recurring task. AgentBridge starts it at the agreed time and delivers the result to your chat, your email or your phone.

*Tip: You can change or stop a scheduled task any time by telling the agent.*

---

## Your week, wrapped up automatically




*A weekly wrap-up scheduled to run every Friday*

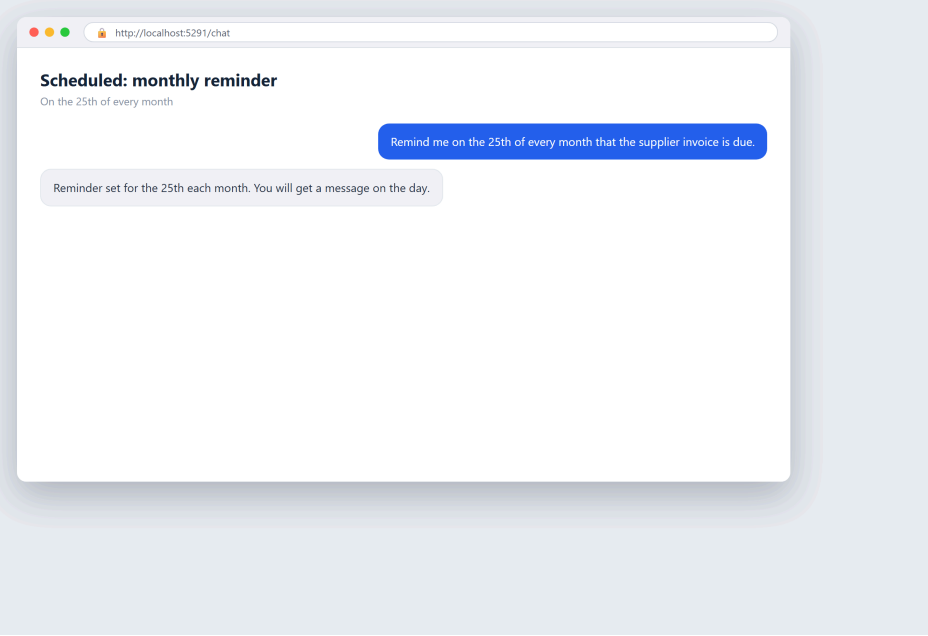
**What you ask:** `Every Friday at 5pm, summarise this week's sales and list what is due next week.`

The agent prepares the weekly wrap-up automatically and sends it before you leave. Next week starts with a clear plan.

> A weekly recurring task. AgentBridge handles the timing so you handle the decisions.

---

## Never miss a deadline




*A recurring reminder set for a monthly deadline*

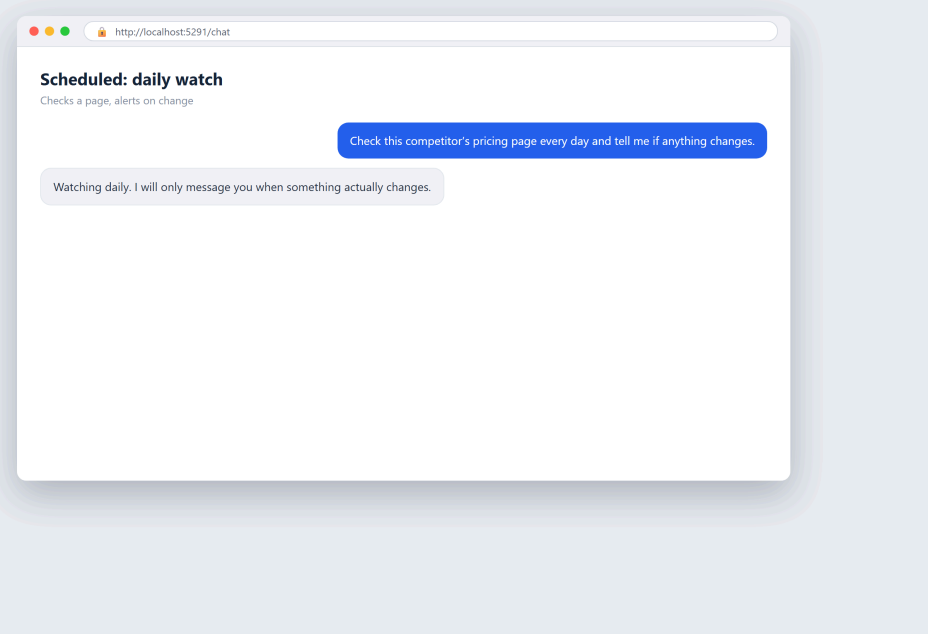
**What you ask:** `Remind me on the 25th of every month that the supplier invoice is due.`

The agent reminds you on the day, every month. You set it once and stop worrying about the date.

> A monthly recurring reminder, delivered however you like — chat, email or phone.

---

## Watch a page for changes




*A daily watch that alerts you to changes*

**What you ask:** `Check this competitor's pricing page every day and tell me if anything changes.`

The agent checks the page on schedule and only messages you when something actually changed. No noise, just the news.

> A daily scheduled watch. It runs alone and reports only when there is something to report.

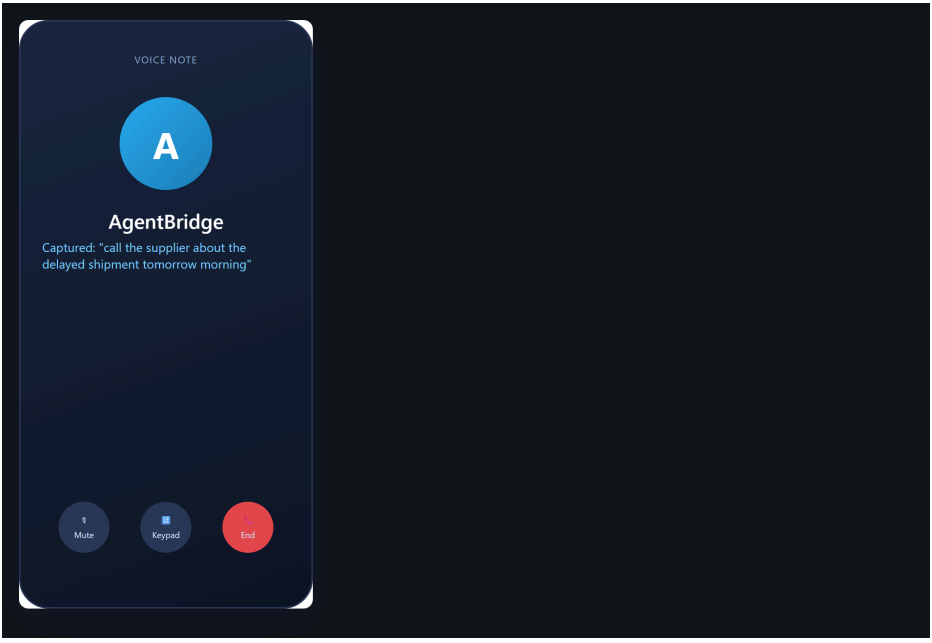
## Key Takeaways

- You can set a job once and let AgentBridge run it, speak to it, or reach it by phone and Telegram.
- The result is the same finished work you see in the pictures, delivered where you are.
- You stay in control: you choose the time, the place, and who may use it.

# Your Assistant by Voice and by Phone

You do not always sit at a desk. AgentBridge listens when you speak, reads answers out loud, and answers a phone call like a person. This means you can use your assistant while your hands are busy or when you are away from the office.

## Speak instead of type




*A spoken note captured by the agent*

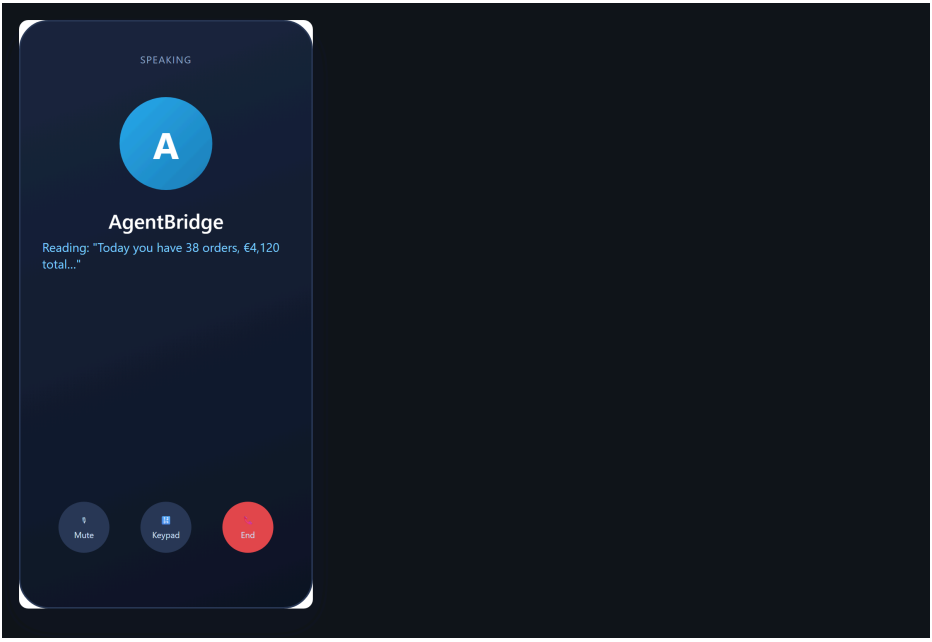
**What you ask:** `(speak) Take a note: call the supplier about the delayed shipment tomorrow morning.`

The agent listens and writes the note for you. Your hands stay free; the thought is saved.

*Tip: Dictation works for messages, notes and even whole emails — just talk.*

---

## Let the agent read it to you




*The agent speaking its answer aloud*

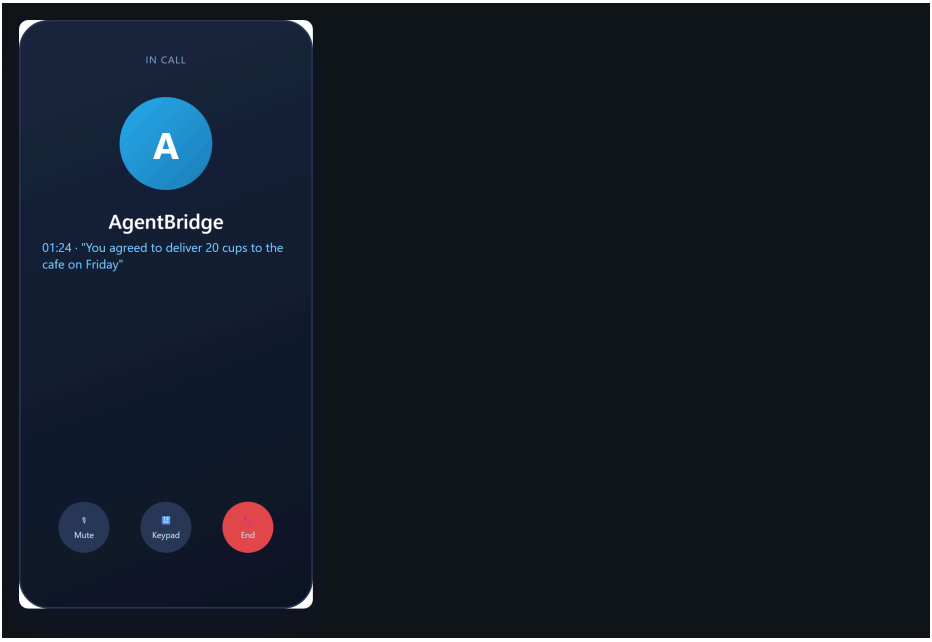
**What you ask:** `Read me the summary of today's orders out loud.`

The agent reads the answer in a clear voice. Useful when you are moving, cooking, or your eyes are tired.

*Tip: You can ask for any answer to be spoken, not just summaries.*

---

## Call your assistant on the phone




*A phone call to your AgentBridge assistant*

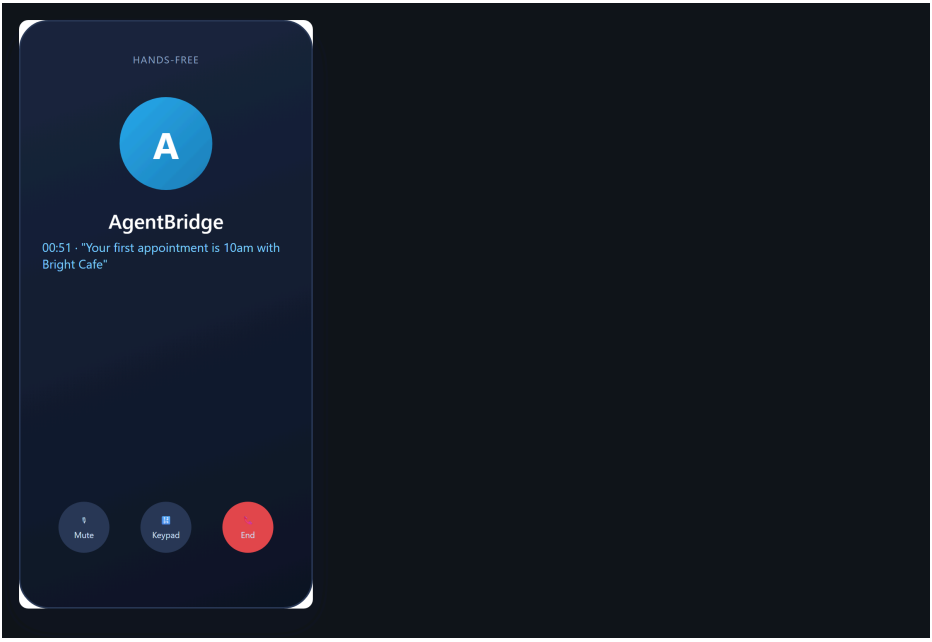
**What you ask:** `(phone) What did I agree to deliver to the cafe this week?`

You call in, say what you need, and the agent answers from your notes and documents — by voice, wherever you are.

*Tip: Phone access is protected by a PIN, so only you can reach your assistant.*

---

## Hands-free on the road




*A hands-free call to check your day*

**What you ask:** `(phone) Read me my appointment list for today.`

The agent reads your appointments over the phone so you stay focused on the road and still have your day in hand.

*Tip: Keep calls short and focused while driving; save the long work for later.*

## Key Takeaways

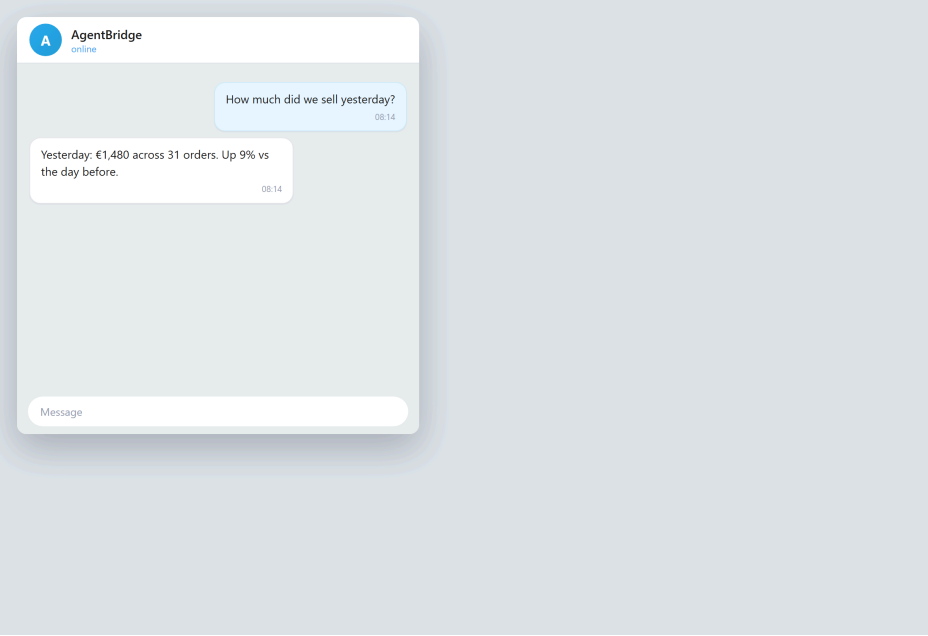
- You can set a job once and let AgentBridge run it, speak to it, or reach it by phone and Telegram.

- The result is the same finished work you see in the pictures, delivered where you are.
- You stay in control: you choose the time, the place, and who may use it.

# Working on the Go with Telegram

If you already use Telegram every day, your assistant can join you there. You ask in the same chat you use with your team, and the agent answers, sends files, and keeps everyone informed. No new app to learn.

## Your assistant in Telegram




*A conversation with the agent inside Telegram*

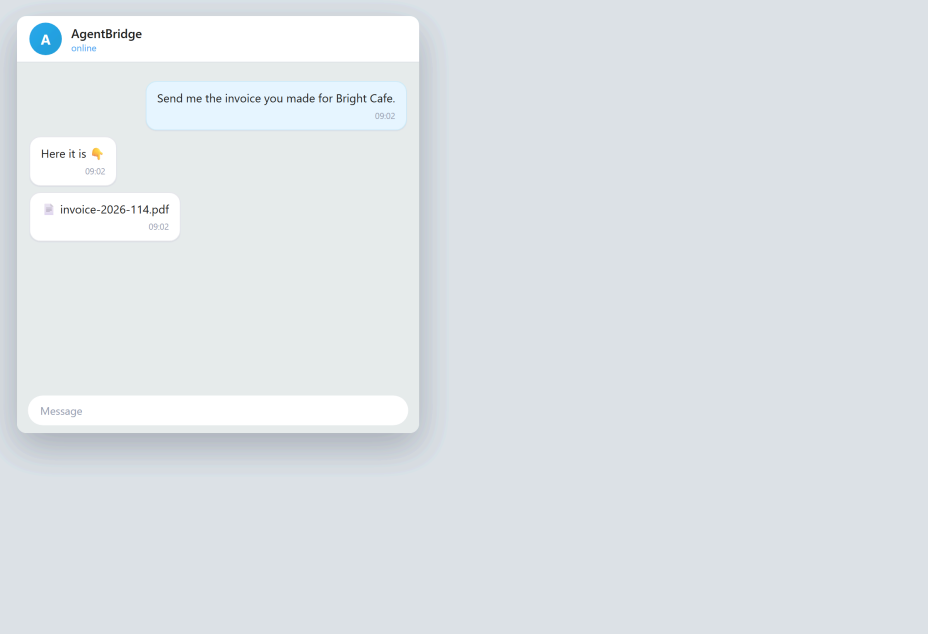
**What you ask:** `(telegram) How much did we sell yesterday?`

The agent answers in your Telegram chat, just like a colleague. Ask a question, get the answer, keep moving.

*Tip: Telegram access uses an allow-list, so only the people you choose can talk to it.*

---

## Get a document on your phone




*A document delivered into the Telegram chat*

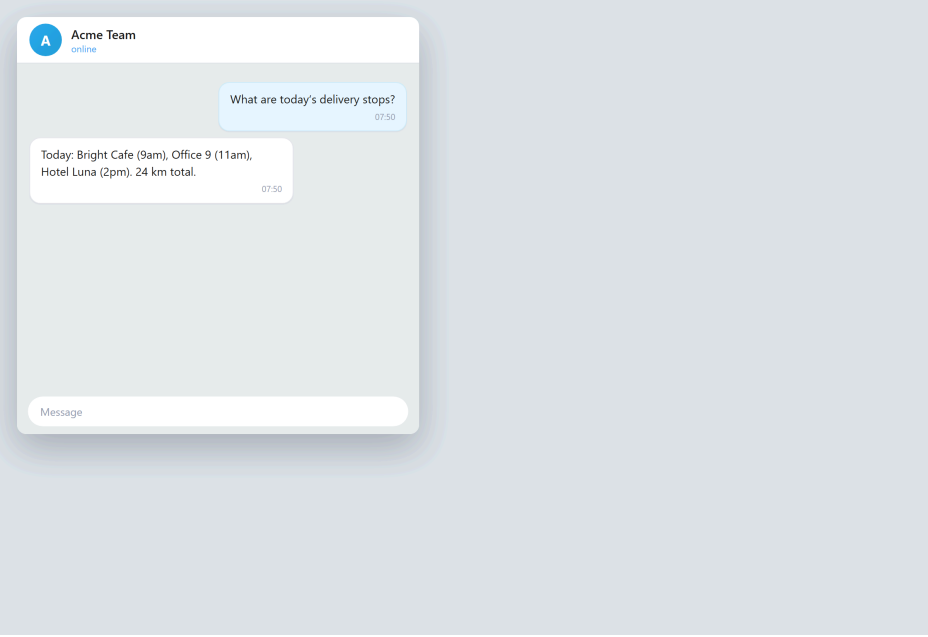
**What you ask:** `(telegram) Send me the invoice you made for Bright Cafe.`

The agent sends the file into your Telegram chat. Open it, forward it, or download it — right from your phone.

*Tip: You can ask for any document you made earlier and have it sent to you.*

---

## The assistant in your team group




*The agent answering in a team group chat*

**What you ask:** `(telegram group) What are today's delivery stops?`

The agent joins the group and answers the team's questions with the shared information, so nobody has to be the walking encyclopedia.

*Tip: You control who is allowed in the group and what the agent may share.*

## Key Takeaways

- You can set a job once and let AgentBridge run it, speak to it, or reach it by phone and Telegram.

- The result is the same finished work you see in the pictures, delivered where you are.
- You stay in control: you choose the time, the place, and who may use it.

# Chapter 13 — Defining a Simple AI Strategy

## In Simple Words

A strategy is a plan that connects what you do today to where you want to be. An AI strategy is the same thing, narrowed to one question: how will AI help this business reach its goals over the next year or two?

Many businesses skip this and jump straight to tools. They buy a chatbot, try a writing assistant, sign up for a platform. A few months later they have scattered tools, no clear results, and no idea what to do next. The problem was never the tools. It was the missing plan.

A simple AI strategy fixes that. It does not need to be a thick document. It needs to answer a handful of questions clearly:

- **Where are we going?** What does the business want to achieve?
- **How does AI serve that?** Which goals can AI actually move?
- **What do we do first, and what later?** Quick wins now, bigger projects after.
- **How much do we spend, and where?**
- **Who decides what?**

If you can answer those in a page or two, you have a strategy. The rest of this chapter shows you how to build each answer.

One honest point: a strategy is not a promise of a fixed future. Tools change fast. A good strategy is a direction and a set of priorities, not a rigid script. It should be short enough to update when reality shifts. A plan nobody can hold in their head is a plan nobody follows.

Think of it like planning a road trip. You decide the destination, pick the route, budget for fuel, decide who drives and who navigates, and plan the stops. You do not need to know every pothole. You need enough of a plan that everyone knows where you are going and who is doing what. That is what this chapter gives you for AI.

## A Bit of History

**1960s–1970s: strategy becomes formal.** Business schools introduced planning frameworks — SWOT analysis, long-range planning. Strategy meant a deliberate, written plan to out-position competitors. The habit of "write the plan first" comes from here.

**1990s: IT strategy becomes its own thing.** As computers spread through companies, "IT strategy" emerged: a plan for how technology supports the business. It was often a separate document, written by the tech department, sometimes disconnected from what the business actually needed.

**2000s: digital strategy.** The web forced companies to think about online presence as strategic, not just a website. Digital strategy tried to reconnect technology plans with business goals.

**2010s–2020s: AI strategy is born.** When AI became a real business capability, "AI strategy" appeared as a label. At first it was mostly hype — vendors selling the idea. Then it matured into a practical discipline: a small plan that ties AI projects to business goals, budgets, and governance.

**The recurring lesson.** The biggest failure pattern across all these eras was the same: technology plans made in isolation from the business. The tech team built what was interesting, not what the business needed. A good AI strategy exists to prevent exactly that — to

keep AI tied to real business goals, decided by the right people, with clear priorities.

## Curiosity

### 13.7 The trap of "pilot purgatory"

There is a well-known failure pattern in business AI that a good strategy is designed to avoid. Analysts call it **"pilot purgatory."** A company runs one AI pilot after another. Each pilot shows a little promise. None of them ever becomes a real, scaled, money-saving system. The company spends years and money on experiments that never land, stuck in a loop of trying without committing.

This happens for a strategic reason, not a technical one. Pilots are comfortable. They are small, low-risk, and easy to start. Scaling is hard — it needs budget, integration, change management, and owners. Without a strategy that says "after the pilot, we scale these specific things, funded and owned," companies drift from pilot to pilot forever.

A simple strategy breaks the loop by deciding, in advance, what happens after a pilot succeeds: who funds it, who owns it, and how it scales. It treats a pilot as the first step of a plan, not the whole plan. If you take one thing from this curiosity, take this: **plan the scaling before you run the pilot.** (The exact share of pilots that stall is often quoted in industry reports and varies widely; treat any single number as illustrative. The pattern itself is real and common.)

## A Real Business Example

## A regional logistics firm writes a one-page plan

A mid-sized delivery and logistics company wanted to “use AI” but had no direction. Different managers wanted different things. The warehouse wanted better route planning. Sales wanted lead scoring. Customer service wanted a chatbot. Everyone was right, and the result was chaos.

The owner sat down for half a day and wrote a single page. It had four parts.

**Goal.** Cut delivery cost per parcel by 10% within two years while keeping service quality.

**Where AI helps.** Route and load optimization first (biggest cost lever), then demand forecasting, then customer-service automation later.

**Sequence.** Quarter one: one route-optimization pilot on a single region. Quarter two: evaluate and scale if it works. Quarter three: forecasting pilot. Quarter four: decide on customer service.

**Who decides.** The owner approves budget. The operations manager owns the route project. A named analyst runs the data.

That page ended the chaos. Every conversation now started with, “Does this serve the cost-per-parcel goal?” Requests that did not fit were politely deferred. The route pilot worked, scaled, and paid for the rest. The firm did not have a fancy plan. It had a clear one. That was enough.

# How to Do It

## 13.1 Vision and objectives

Start with the end in mind. A vision is a plain sentence about where you want the business to be in a few years, with AI as part of the picture. An objective is a specific, measurable target that moves you toward that vision.

Write the vision first. Keep it short and honest. Not "become an AI-driven leader in our sector" — that is empty. Something like: "Run the same business with less manual work, faster service, and fewer errors, so our small team can do more." That is a real vision you can feel.

Then write two or three objectives. Each must be specific and measurable. "Reduce the time we spend on admin by 30% in 18 months" is an objective. "Improve efficiency" is not. Use numbers and dates.

A good test: could someone outside your business tell whether you hit the objective? If yes, it is well written. If it could be claimed either way, sharpen it.

Keep the vision and objectives tied to the business, not to AI for its own sake. The goal is never "use AI." The goal is a business outcome that AI happens to help reach.

## 13.2 Alignment with the business strategy

This is the most important rule in the chapter: **the AI strategy must serve the business strategy, not the other way around.**

If your business strategy is to compete on price, AI should target cost reduction. If your strategy is to compete on service quality, AI should target faster, better customer handling. If your strategy is growth, AI should help you scale without adding cost. The AI work must pull in the same direction as the business.

To check alignment, take each AI objective and ask: *which business goal does this serve?* If you cannot name one, drop it or reframe it. An AI project that serves no business goal is a hobby, and hobbies do not get budget.

A practical trick: write your business goals at the top of a page. Below each, list the AI projects that support it. If a business goal has no AI support, ask whether AI could help there. If an AI project sits under no business goal, cut it. The visual makes misalignment obvious.

Alignment also means sequencing follows business priority. Do not spend your first year automating something minor while your biggest business problem goes untouched. Let the business's own priorities set the order of AI work.

## 13.3 Quick wins and long projects

A balanced strategy mixes two kinds of work: quick wins and long projects.

**Quick wins** are small, fast, low-risk projects that show a result within weeks or a couple of months. They come from the high-impact, high-ease box of the matrix in Chapter 12. Their job is to prove value, build confidence, and create momentum. They also teach your team how to run an AI project cheaply.

**Long projects** are bigger, harder, higher-impact efforts that take many months. They come from the high-impact, low-ease box. They need more budget, more integration, and more change management. They deliver more, but only after the quick wins have proven the approach and built the skills.

The right mix matters. If you only do quick wins, you never tackle the big value and plateau. If you only do long projects, you risk a lot on unproven ground and may lose support before you see results. Start with quick wins, then layer in long projects as confidence and capability grow.

A simple rule: **lead with quick wins, fund long projects with the credibility and savings they create.** The first quick win pays, in money and trust, for the first long project.

## 13.4 A yearly roadmap in 4 quarters

A roadmap turns your priorities into a timeline. A simple yearly one fits on a page and divides the year into four quarters. Here is a pattern that works.

**Quarter 1 — Prove it.** Run one or two quick-win pilots. Pick the easiest high-value targets. Set clear success measures before you start. Goal: a working result and a learning.

**Quarter 2 — Scale the win.** If a pilot worked, roll it out across the business. Fix what broke. Train the people. Goal: the quick win becomes normal operations and starts saving real money.

**Quarter 3 — Go deeper.** Start the first long project — a high-impact, harder one. Use the skills and savings from quarters one and two to fund it. Goal: begin the bigger transformation.

**Quarter 4 — Consolidate and plan.** Evaluate everything. Measure results against the objectives. Decide what to scale, stop, or start next year. Write next year's roadmap. Goal: a clear picture and a next plan.

This rhythm — prove, scale, deepen, consolidate — keeps you moving without overreaching. Each quarter builds on the last. Adjust the exact content to your business, but keep the shape: small start, then grow.

Write the roadmap as a simple table: quarter, focus, projects, owner, success measure. Keep it visible to the whole team. A roadmap nobody can see is not a roadmap.

## 13.5 Budget: how much to invest and where

You do not need a huge budget to start. You need a clear one.

**Start small and real.** For a small business, a first-year AI budget might be a few thousand to a few tens of thousands — tool subscriptions, a little integration help, and staff time. The exact number depends on your size and goals. What matters is that it is set on purpose, not drifted into.

**Budget the hidden costs.** The tool price is rarely the whole cost. Add: integration (connecting AI to your existing systems), staff time to run and check it, training, and a little contingency for surprises. Many budgets fail because they counted only the subscription.

**Tie budget to expected return.** For each funded project, estimate what it saves or earns. A project that costs a little and saves a lot earns its budget. A project that costs a lot and saves little does not. The simple ROI method is in Chapter 16. Use it to decide where the money goes.

**Fund the next step from the last.** Let the savings from quick wins fund the long projects. This keeps the budget self-financing and easy to defend.

**Keep a reserve.** Things go wrong. Hold back a small buffer so a surprise does not kill a good project.

A good budget sentence: *"We will invest [amount] this year, mostly in [areas], expecting [result], with [reserve] held back for surprises."* If you can say that, your budget is clear.

## 13.6 Governance: who decides what

Governance is a fancy word for a simple thing: who makes which decisions, and who is accountable. Without it, AI projects drift, stall, or cause harm with no one owning the result.

Define a few clear roles. In a small business, one person can hold several.

- **The sponsor / owner.** Usually you or a senior leader. Approves the strategy and the budget. Settles disputes. Owns the overall result.
- **The project owner.** Runs a specific project day to day. Responsible for it succeeding.
- **The data / technical lead.** Handles the data and the tools. Someone who understands what is possible and what is risky.
- **The reviewer.** Checks that projects follow the rules — privacy, fairness, security. In a small firm this can be the owner or an outside advisor.

Write down, for each kind of decision, who makes it: budget, scope, go/no-go on scaling, handling of a problem. When everyone knows

their role, decisions happen fast and no one is surprised.

Two governance habits matter most. First, **regular check-ins** — a short standing meeting to review progress and decide next steps. Second, **clear escalation** — if something goes wrong, everyone knows who to tell and what to do. These two habits prevent most governance failures.

Governance is not bureaucracy. It is just making sure the right person decides the right things and owns the outcome.

## Ethics and Responsibility

A strategy sets the ethical tone for everything that follows.

**Put ethics in the plan, not as an afterthought.** Decide in advance which uses you will and will not pursue, and what checks every project must pass. The principles are in Chapter 4; the law in Chapter 5. Bake them into your go/no-go criteria.

**Name accountability for harm.** If an AI decision hurts a customer or an employee, who is responsible? Decide now, not in a crisis. Never let accountability blur into "the system did it."

**Budget for doing it right.** Ethics has a cost — review time, testing, sometimes slower rollout. Put it in the budget so it is not cut when pressure rises.

**Be transparent with your people.** Tell staff what AI will and will not do, and how their roles may change. Hiding the plan breeds fear and resistance. The communication side is in Chapter 21.

## Mistakes to Avoid

1. **Buying tools before writing a plan.** The classic error. Plan first, buy second.
2. **A strategy that is not tied to a business goal.** AI for its own sake is a hobby, not a strategy.
3. **A plan too big to understand.** If no one can hold it in their head, no one follows it. Keep it to a page or two.
4. **Only quick wins, or only big projects.** You need both, sequenced right.
5. **No budget for hidden costs.** Integration, training, and staff time are real. Count them.
6. **No named owners.** Projects without owners stall. Every project needs a person accountable.
7. **Pilot purgatory.** Running endless pilots with no plan to scale. Plan the scaling before the pilot.
8. **Treating the strategy as fixed.** Tools change fast. Update the plan when reality shifts; a stale plan misleads.
9. **Skipping governance.** No decision-maker means no decisions, or bad ones with no owner.

## Practical Exercise

### 13.7 Write your SMART goals

SMART is a simple test for a good objective. Each letter is a quality:

- **S — Specific.** Clear about what exactly you want.
- **M — Measurable.** You can count it or check it.
- **A — Achievable.** Realistic with your resources.
- **R — Relevant.** It serves a real business goal.

- **T — Time-bound.** It has a deadline.

Take one hour. Write **three AI goals** for your business and make each one SMART.

Weak: “Use AI to improve customer service.” SMART: “Cut average first-reply time to customer emails from 6 hours to under 1 hour by the end of Q2, using an AI drafting assistant, with staff review before sending.”

Check each of your three goals against all five letters. If it fails one, rewrite it until it passes all five.

Then, under each goal, write: the first project that serves it, who owns it, and roughly what it costs. Now your goals are not wishes — they are a starting plan.

Keep the three goals on one page. That page is the heart of your AI strategy.

## Checklist

### 13.8 Strategic checklist

Before you call your AI strategy done, check these.

- ☐ **You have a one-sentence vision** that is honest and concrete.
- ☐ **You have 2–3 measurable objectives** with numbers and dates.
- ☐ **Every AI objective serves a named business goal** (alignment).
- ☐ **You have quick wins identified** from the impact/ease matrix.
- ☐ **You have at least one long project** planned for after the quick wins.

- ☐ **You have a 4-quarter roadmap** — prove, scale, deepen, consolidate.
- ☐ **You have a budget** that includes hidden costs (integration, training, staff time).
- ☐ **You have a small reserve** for surprises.
- ☐ **Each project has a named owner** accountable for its result.
- ☐ **You have a reviewer** for privacy, fairness, and security.
- ☐ **You have regular check-ins and a clear escalation path.**
- ☐ **You planned how to scale each pilot before running it** (avoid pilot purgatory).
- ☐ **The whole strategy fits on one or two pages** and people can actually follow it.
- ☐ **You will review and update it** as tools and reality change.

If a box is empty, that is your next task. A strategy is done when the right people know the goal, the order, the budget, and their own role.

## Key Takeaways

- An AI strategy is a short plan that ties AI work to real business goals — plan first, buy tools second.
- The golden rule: AI serves the business strategy, never the reverse; if a project serves no business goal, cut it.
- Lead with quick wins to build trust and savings, then fund the harder long projects with what they create.
- A simple 4-quarter roadmap — prove, scale, deepen, consolidate — keeps you moving without overreaching.
- Name the owners and the budget, plan the scaling before the pilot, and keep the whole thing short enough to actually follow.

# Chapter 14 — Data: The Raw Material

## In Simple Words

AI does not think on its own. It works on data, the same way a kitchen works on ingredients. The quality of the meal depends on the quality of what you put in. No chef, however skilled, can make a good dish from rotten vegetables. AI is the same: good data in, useful results out; bad data in, useless or harmful results out.

This is the single most important truth about AI that business owners miss. They focus on the model — how smart it is, which brand to buy — and ignore the data it runs on. But the model is only the chef. The data is the food. Most AI projects fail not because the model was weak, but because the data was messy, incomplete, or wrong.

Data, in plain words, is recorded information. Customer names, order histories, invoices, emails, spreadsheets, employee records, website clicks, machine readings — all data. Some of it is tidy, sitting in neat tables. Most of it is messy, scattered across files, inboxes, and paper. Your job is to know what you have, where it is, and whether it is good enough to build on.

This chapter is about treating your data like the valuable raw material it is. It covers what data you have and where it lives, how to tell if it is clean and current, the basic rules for personal data, how to keep it safe, and how to organize it without tearing your whole business apart. The deep legal treatment of privacy is in Chapter 10; this chapter gives you the working basics so you can prepare your data for AI.

A simple image to carry through: before you cook, you check the pantry. You find what you have, throw out what is spoiled, and note what is missing. That is exactly what you do with data before any AI project.

## A Bit of History

**1960s–1970s: data lives in databases.** Businesses stored information in structured databases — neat tables of numbers and short text. This was clean by design, but it covered only a small slice of what the business knew. Most knowledge lived on paper or in people's heads.

**1980s–1990s: spreadsheets and files.** Personal computers spread data everywhere. Spreadsheets, shared drives, email attachments. Data became abundant but scattered and inconsistent. The mess we all know today started here.

**2000s: "big data."** The internet and digital systems produced data at a scale nobody had seen. Companies began talking about "big data" as an asset. But volume without quality created a new problem: oceans of data, little of it reliable.

**2010s: data quality becomes the bottleneck.** As analytics and machine learning grew, a hard truth emerged: most projects spent 80% of their time cleaning data, not modeling it. "Garbage in, garbage out" became the defining lesson of the era. Data preparation, not clever algorithms, was the real work.

**2018: GDPR raises the stakes.** Europe's privacy law made data handling a legal duty, not just a quality issue. Now messy data was not only useless; mishandled personal data was punishable. Data governance became a board topic.

**2020s: generative AI makes data reach matter.** Modern AI can read messy text and images, so it needs less tidy data than old systems. But it still depends on having the right data available and trustworthy. The lesson holds: the model is only as good as what you feed it.

The arc is clear. We went from too little structured data to too much scattered data. The skill never changed: find it, clean it, and know what you can trust.

## Curiosity

## 14.6 The model that wrote Python from 1930s books

Here is a striking fact about how data shapes what a model can do: researchers trained a language model on books from before 1931 only — no computers, no internet, and no programming code, because Python did not exist yet — and it could still write a little Python, by copying the structure of examples placed in front of it.

That experiment, and what it teaches about data and structure, is told in full in Chapter 3, the canonical home for it. The point for this chapter is one line: **what a model can do depends entirely on what it was fed.** Change the data, and you change the capability. That is why your own data is the raw material worth caring about.

## A Real Business Example

## The wholesaler whose "data" was three conflicting spreadsheets

A building-supplies wholesaler believed it had good customer data. When it tried to use AI to forecast which products each customer would order, the project stalled in the first week. The reason was data, not the model.

Customer information lived in three places: an old accounting system, a sales spreadsheet kept by one rep, and a mailing list in an email program. The same customer appeared three times with three different spellings and addresses. Order histories did not match between systems. Half the records had no phone number. The data was not wrong exactly — it was fragmented and inconsistent, which is just as bad.

The fix was not high-tech. They picked one system as the single source of truth for customer records. They merged the duplicates by hand. They set a simple rule: every new customer goes into the one system, once. It took a few weeks of unglamorous work. After that, the forecasting project worked, because for the first time there was one reliable set of data to build on.

The lesson: the AI was ready. The data was not. Most AI projects wait on data, not on tools.

## How to Do It

## 14.1 What data you have and where it is

You cannot use what you cannot find. The first step is a data inventory: a plain list of every place your business keeps information.

Walk through your business and list every data source. Typical ones:

- **Business systems.** Accounting software, CRM, inventory, e-commerce platform, HR system.
- **Files and spreadsheets.** Shared drives, personal laptops, Google Drive, Excel files.
- **Email and messages.** Inboxes, chat tools, saved threads.
- **Paper.** Physical files, signed forms, notes.
- **Web and digital traces.** Website analytics, app logs, customer portal activity.
- **External data.** Supplier feeds, market data, public records.

For each source, note four things: what it holds, who owns it, roughly how much is there, and how current it is. Do not aim for perfection; aim for a map. You want to see the whole landscape so you know where the good data is and where the gaps are.

Expect surprises. Most owners discover data they forgot existed and gaps they assumed were filled. The inventory itself is valuable because it turns a vague sense of "we have data somewhere" into a clear picture.

A good inventory is a table. Keep it simple and update it as you go. It becomes the reference for every future AI project.

## 14.2 Clean, complete, and up-to-date data

Once you know what you have, judge its quality. Good data has three qualities.

**Clean.** Free of errors, duplicates, and inconsistencies. The same customer is not spelled three ways. Numbers are actually numbers, not text. Dates are real dates. Cleaning means fixing or removing the bad records.

**Complete.** Has the fields you need filled in. If you need a customer's region to forecast demand, but half the records have no region, the data is incomplete. Completeness means the important fields are filled.

**Up-to-date.** Reflects reality now, not three years ago. A customer list where half the companies have moved or closed is stale. Stale data leads to wrong conclusions, no matter how clean it looks.

How to check quality without special tools:

- **Spot-check.** Pull 20 records at random and look for errors, duplicates, and blanks. The error rate you see is roughly the error rate you have.
- **Count the blanks.** For the fields you need, what share are empty? High blanks mean low completeness.
- **Check the dates.** When was each source last updated? Old means stale.

You do not need perfect data. You need data that is good enough for the specific task. A rough forecast tolerates more noise than a customer bill. Match the quality bar to the job. But know where you stand before you build.

Cleaning is real work and often tedious. Budget for it. It is the 80% of the project that everyone forgets to plan.

## 14.3 Personal data and GDPR: basic rules

Some of your data is personal data — any information about a living person who can be identified: names, emails, phone numbers, addresses, customer IDs, even an IP address. Personal data carries legal duties, and the full treatment is in Chapter 10. Here are the working basics you need before using it with AI.

**Know what is personal.** Mark, in your inventory, every source that holds personal data. You cannot protect what you have not identified.

**Have a lawful reason.** Under GDPR, you may process personal data only on a valid legal ground — for example, a contract with the person, their consent, or a legitimate interest that does not override their rights. Know which ground you rely on before you feed the data into AI.

**Use only what you need.** Do not pour all your personal data into an AI tool when the task needs only a little. Minimize what you use.

**Watch where it goes.** If you send personal data to a third-party AI service, the data leaves your control and GDPR follows it out the door. The vendor's handling becomes your responsibility. The third-party risks are in Chapter 9.

**Respect people's rights.** People can ask to see, correct, or delete their data. Your AI use must not make that impossible.

This is a plain guide, not legal advice. For real decisions, especially across borders, consult a data-protection professional. But the habit — mark personal data, know your ground, use the minimum — is yours to build now.

## 14.4 Security, backup, and access

Data is an asset, and assets need protecting. Three basics cover most of the risk.

**Security.** Protect data from attackers and leaks. Use strong passwords and multi-factor authentication, keep systems updated, encrypt sensitive data, and be careful with email attachments and links. The full security picture, including AI-specific threats, is in Chapter 6.

**Backup.** Keep copies of your data that are safe from the main system. If a system crashes, is hit by ransomware, or a file is deleted, a backup is the difference between a bad day and losing the business. Follow a simple rule: keep copies in more than one place, back up regularly, and test that you can actually restore. A backup you have never tested restoring is only a hope.

**Access.** Control who can see and change what. Not everyone needs access to everything. Give people the minimum access their role requires. Log who accesses sensitive data. Limiting access limits both accidents and theft.

These three work together. Security keeps outsiders out. Backup saves you when something goes wrong anyway. Access limits the damage any single person or mistake can do. None of them is optional.

A practical habit: back up automatically, check the backup monthly, and review who has access quarterly. Small routines, repeated, prevent disasters.

## 14.5 How to organize data without turning everything upside down

The biggest fear about data work is that it means a huge, disruptive overhaul. It does not. You can improve your data step by step without stopping the business.

**Pick one source of truth per thing.** For each important type of data — customers, products, orders — choose one system to be the official one. Everything else becomes a copy or a view. This single rule fixes most confusion without changing your tools.

**Fix at the point of entry.** The cheapest time to clean data is when it is created. Set simple rules so new records go into the right place, once, with the key fields filled. Stopping new mess beats cleaning old mess forever.

**Do not boil the ocean.** Do not try to clean everything. Clean only the data your first AI projects need. Perfect data you never use is wasted effort. Targeted cleaning that serves a real project is worth doing.

**Standardize a little, not perfectly.** Agree on a few simple formats — how to write a date, a phone number, a customer name — and apply them going forward. You do not need a grand standard, just consistency for the fields that matter.

**Improve as you go.** Treat data quality as a habit, not a project. Each small fix makes the next AI use easier. Over a year, small steady improvements add up to a solid data foundation.

The mindset: you are not rebuilding the house. You are tidying the pantry, one shelf at a time, so the next meal is easier to cook. Start with the shelf your first project needs.

## Ethics and Responsibility

Data carries ethical weight beyond quality and law.

**Personal data is people.** Behind every record is a person with rights and feelings. Handle it with that in mind, not just as a resource to mine. The principles are in Chapter 4.

**Bias lives in data.** If your data under-represents some group, AI trained on it will treat that group badly. A hiring dataset with mostly one kind of candidate will bias the AI the same way. Check whose data is missing, not just whether the data is clean.

**Do not use data people did not expect.** Just because you hold data does not mean you should use it for any purpose. Using customer data in a way they never agreed to breaks trust, even if it is legal. Stay within reasonable expectations.

**Be honest about what you collect and why.** Tell people what data you gather and how you use it. Hidden collection is a breach of trust and often of the law.

**Protect it like it is yours.** A breach harms real people, not just your reputation. Treat security as a duty to the people in your data, not just a box to tick.

## Mistakes to Avoid

### 14.7 Common mistakes with data

1. **Ignoring data until the project fails.** The most common mistake. Check your data before you start, not after it stalls.

2. **Assuming you have data when you have fragments.** Three conflicting spreadsheets is not one dataset. Find the truth first.
3. **Confusing volume with quality.** Lots of data is not good data. A small clean set beats a large messy one.
4. **Not planning the cleaning time.** Data prep is most of the work. Budget for it or the project slips.
5. **No single source of truth.** Multiple "official" versions guarantee confusion. Pick one.
6. **Using personal data without a lawful basis.** A legal risk and a trust risk. Know your ground first.
7. **Sending personal data to third-party AI carelessly.** GDPR follows the data out the door.
8. **Untested backups.** A backup that cannot restore is not a backup. Test it.
9. **Everyone has access to everything.** Wide access means wide risk. Limit it to the role.
10. **Trying to clean everything at once.** Overhaul paralysis. Clean only what your project needs.
11. **Ignoring stale data.** Clean but old data still gives wrong answers. Check the dates.
12. **Forgetting bias in the data.** A gap in who is represented becomes a bias in the output.

## Practical Exercise

### 14.8 Data inventory

Spend an afternoon building a simple data inventory. This is the single most useful preparation you can do for AI.

Make a table with one row per data source. For each, fill in:

- **Source** — the system, file, or place (e.g., “accounting software,” “sales spreadsheet,” “email inbox”).
- **What it holds** — customers, orders, invoices, CVs, etc.
- **Owner** — who is responsible for it.
- **Volume** — roughly how much (rows, files, GB).
- **Personal data?** — Yes / No.
- **Quality** — spot-check 20 records; note blanks, duplicates, errors. Rate clean / mixed / poor.
- **Current?** — when last updated.
- **Single source of truth?** — Yes / No / candidate.

Fill in every source you can find. When done, look at the picture:

- Which sources are clean and current? Those are your best starting material.
- Which hold personal data? Mark them for the GDPR basics in 14.3.
- Where do you have duplicates with no single source of truth? Those are your first cleanup targets.
- What is missing that your first AI project needs? Those are gaps to fill.

This table tells you, honestly, whether you are ready to start an AI project or need to fix data first. Keep it and update it. It is the pantry list for everything you build next.

## Checklist

### 14.9 Data checklist

Before feeding data into any AI project, check these.

- [ ] **You have a data inventory** listing every source and what it holds.
- [ ] **You know where the good data is** and where the gaps are.
- [ ] **You have a single source of truth** for each key type of data.
- [ ] **You have spot-checked quality** and know the error and blank rate.
- [ ] **The data is clean enough for the specific task** you are doing.
- [ ] **The data is up-to-date** and reflects current reality.
- [ ] **You have marked all personal data** in the inventory.
- [ ] **You have a lawful basis** for any personal data you use (see Chapter 10).
- [ ] **You use only the minimum personal data** the task needs.
- [ ] **You know where the data goes** if a third-party AI service touches it.
- [ ] **Security basics are in place** — strong auth, updates, encryption for sensitive data.
- [ ] **Backups exist, are recent, and have been tested for restore.**
- [ ] **Access is limited** to the roles that need it, with logging on sensitive data.
- [ ] **You are fixing new data at the point of entry**, not just cleaning old data.
- [ ] **You have checked for bias** — whose data is missing?

If a box is empty and your project depends on it, fix that before you build. Good data is not a detail; it is the raw material everything else runs on.

## Key Takeaways

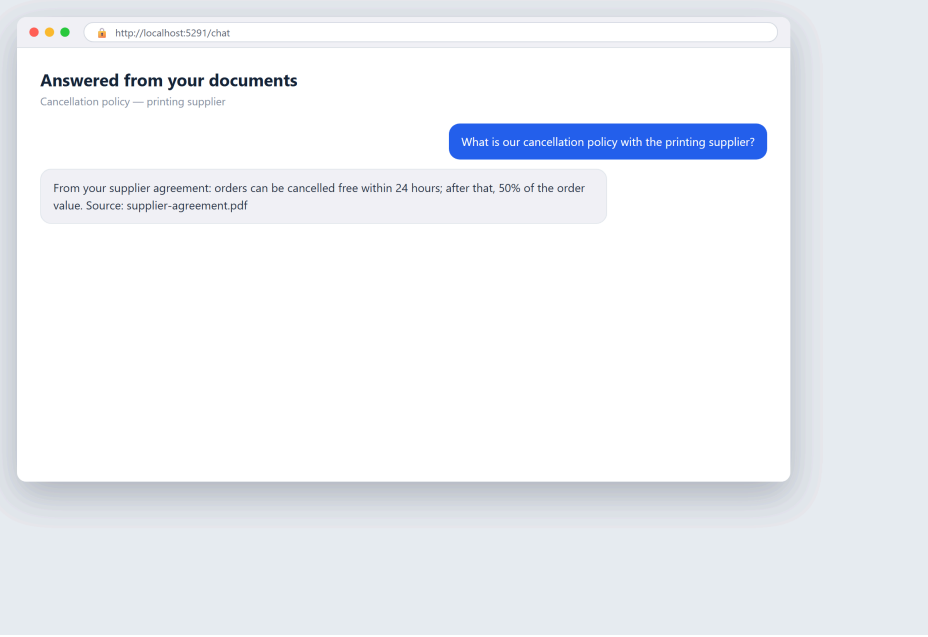
- Data is the raw material of AI: good data in, useful results out; bad data in, useless or harmful results out — the model is only the chef.

- Start with a data inventory: you cannot use what you cannot find, and most owners are surprised by what they have and what is missing.
- Quality means clean, complete, and up-to-date; plan the cleaning time, because it is most of the work everyone forgets.
- Personal data carries legal duties — mark it, know your lawful basis, use the minimum, and remember GDPR follows it to any third-party service.
- You can improve data step by step without an overhaul: pick one source of truth, fix at the point of entry, and clean only what your project needs.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Ask questions about your own files




*The agent answering from your own documents*

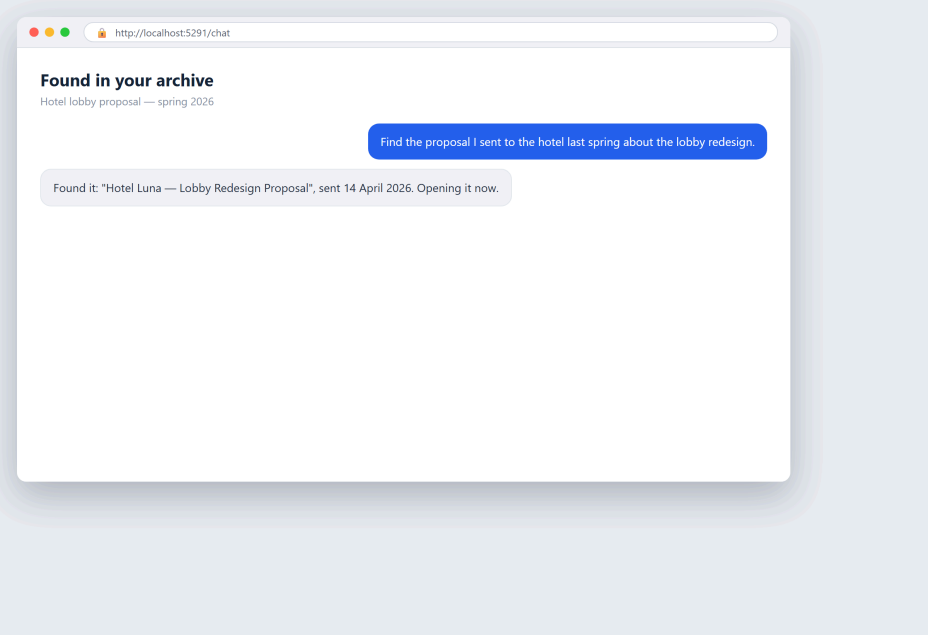
**What you ask:** `What is our cancellation policy with the printing supplier?`

The agent searches your documents area and answers with what your own files actually say, pointing to the source.

*Tip: Keep your business files in the documents area and they become searchable knowledge.*

---

## Find that old document




*A search through your indexed archive*

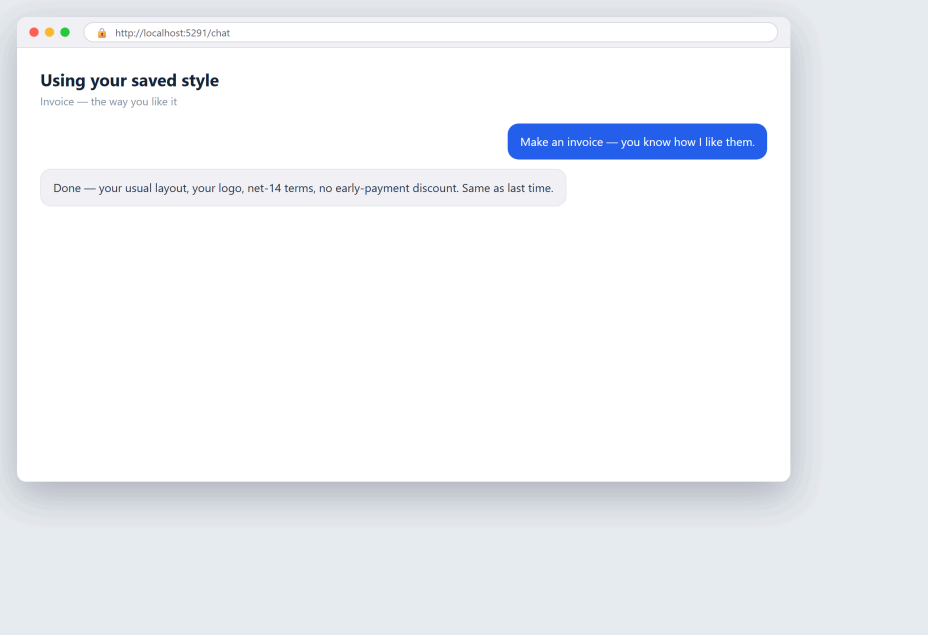
**What you ask:** `Find the proposal I sent to the hotel last spring about the lobby redesign.`

The agent searches your indexed archive and brings back the document you meant, even when you only half-remember it.

*Tip: The index updates as you add files, so the archive is always current.*

---

## It remembers how you like things




*The agent applying your saved preferences*

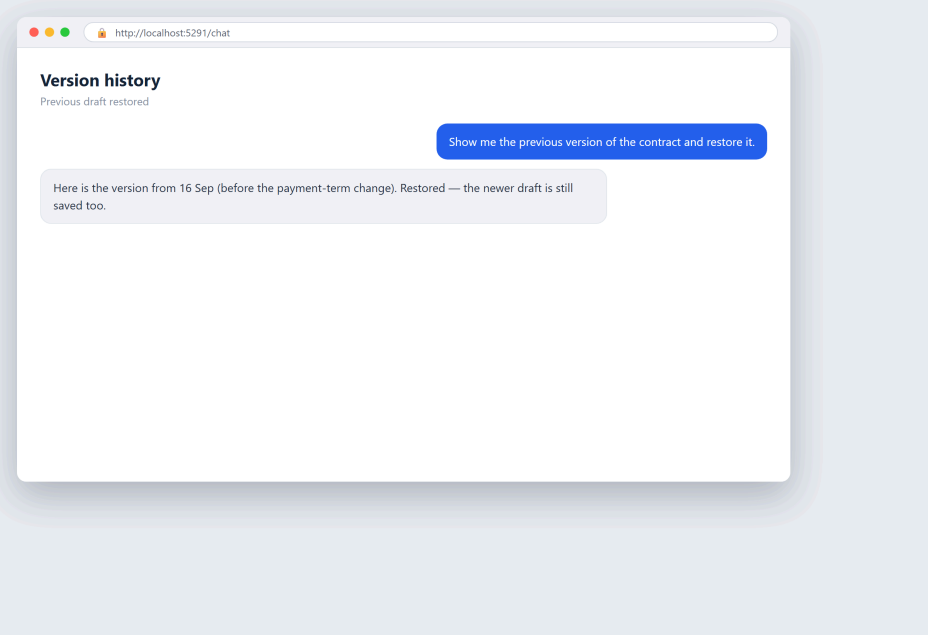
**What you ask:** `Make an invoice — you know how I like them.`

The agent remembers your style and settings from before and applies them without you repeating yourself.

*Tip: You can correct it any time; it updates what it remembers.*

---

## Nothing is ever lost




*Version history lets you go back safely*

**What you ask:** `Show me the previous version of the contract and restore it.`

Every version the agent made is kept. You can see the earlier draft and bring it back, so editing is always safe.

*Tip: This is why you can let the agent rewrite things freely — the history protects you.*

# Chapter 15 — People, Roles, and Culture

## In Simple Words

Most AI projects do not fail because the technology was weak. They fail because of people. A tool that works perfectly in a test can sit unused on a desk because nobody was told why it exists, nobody was trained to use it, and nobody felt safe enough to trust it. The machine is the easy part. The humans are the hard part.

This chapter is about the human side of bringing AI into a business. It is not soft or optional. It is the difference between a tool that changes how you work and an expensive experiment that quietly dies. Three questions decide the outcome. Who decides to bring the tool in? Who actually uses it every day? Who checks that it works correctly and fairly? If those three roles are unclear, the project drifts. If they are clear, the project moves.

Then there is the fear. When people hear "AI," many think one thought: *will this take my job?* That fear is real, normal, and often the single biggest reason AI adoption stalls. You cannot ignore it or laugh it away. You must face it honestly, because how you handle that fear shapes everything — trust, cooperation, and whether your best people stay or leave.

A simple image to carry: buying AI is like opening a new kitchen. The oven is useless without a chef who knows how to use it, a manager who sets the menu, and a health inspector who checks the food. The oven is

not the restaurant. The people are. This chapter is about assembling and caring for those people.

The legal and privacy duties that some of these roles carry are covered elsewhere — privacy in Chapter 10, the EU AI Act's duties in Chapter 5, and managing change over time in Chapter 21. This chapter gives you the people-and-culture foundation those chapters build on.

## A Bit of History

**Late 1700s: the division of labor.** The economist Adam Smith described a pin factory where each worker made one small step of a pin, and output soared. This idea — split work into simple roles, each owned by someone — is the ancestor of every job description you write today. It also planted a lasting tension: when work is split into tiny pieces, workers can feel like replaceable parts.

**Early 1900s: scientific management.** Frederick Taylor tried to design work like a machine, timing every motion and telling workers exactly how to do each task. It raised productivity and bred resentment. The lesson that still echoes: efficiency designed from above, without the workers' input, often meets quiet resistance.

**1920s–1930s: the human relations movement.** Studies at the Hawthorne plant found something surprising — workers produced more when someone paid attention to them and involved them, not just when the lighting or pay changed. This was the birth of the idea that feelings, attention, and involvement drive performance, not only money and process.

**1980s–1990s: IT change management.** As businesses adopted computers and software, a hard pattern emerged: the technology worked, but people did not use it. Consultants built formal methods for

managing adoption — communicate the why, train the users, support them through the awkward early period. Frameworks like ADKAR (Awareness, Desire, Knowledge, Ability, Reinforcement) came from this era.

**2000s: the digital transformation graveyard.** Studies kept reporting that a large share of big change programs missed their goals, and the cause was almost always people, not tools. "Culture eats strategy for breakfast," a phrase often credited to management thinker Peter Drucker, became a warning: a strong culture of resistance defeats a clever plan.

**2020s: AI makes it personal.** Earlier software changed how you typed. AI changes what the job *is*. It can write, summarize, and decide, so it touches the core of people's work and identity. The old lessons now matter more than ever: involve people, be honest, train them, and never let the technology outrun the trust.

The arc is clear. For two centuries, every wave of new working tools taught the same lesson — the tool is not the change. The people are. AI is the newest teacher of an old lesson.

## Curiosity

### The Luddites were not afraid of technology

When someone struggles with a new app, people call them a "Luddite," as if the word means someone who hates technology. That is a misunderstanding of history.

The Luddites were real people — skilled English textile workers, mostly weavers and knitters, in the years around 1811 to 1816. They were not ignorant or anti-progress. They were some of the most skilled workers

of their day. They destroyed factory machines for a specific reason: the machines were being used to produce lower-quality goods and to replace their skilled labor with cheaper, untrained workers, while their wages collapsed and their promises of fair treatment were broken. Their protest was aimed at how the technology was being used against them, not at technology itself.

The lesson for today is sharp. Fear of new machines is not stupidity. It is a rational response to a real threat — being discarded while the owners profit. When leaders dismiss that fear or mock it, they create the very resistance that sinks the project. When leaders face it honestly, involve the workers, and share the gains, the same fear turns into cooperation. The Luddites are a warning about bad handling, not about the workers.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

Two firms of similar size bought the same AI assistant to draft replies to customer emails.

The first firm treated it as a technology purchase. The owner signed a contract, the IT person switched it on one Monday, and staff were told, “Use this now.” Nobody explained why. Nobody trained them. Nobody said what would happen to their jobs. Within a week, people were quietly ignoring the tool. They pasted answers into a side document and pretended to use it. A senior employee said openly, “They’re training the machine so they can let us go.” The tool was paid for and barely used. The project died in three months.

The second firm treated it as a people project with a tool attached. Before switching anything on, the owner held a meeting and said plainly: “This will change how you answer emails. It will not be used to fire anyone. We want your help making it good, because you know the customers better than any machine.” They picked two respected staff members as “champions” to try it first and give feedback. They ran a one-hour training. They set a rule: the AI drafts, a person reviews and sends — no email leaves without a human. They shared results every week, including the mistakes. People felt safe, involved, and useful. Adoption went up steadily, and the tool paid for itself.

Same tool. Opposite results. The difference was not the software. It was the people plan.

## How to Do It

### 15.1 Who decides, who uses, who checks

Every AI effort needs three clear roles. They can be three people, or in a small business, the same person can wear two hats — but each role must be named, never left to “someone.”

**The decider (the owner or sponsor).** This is the person who says yes, sets the goal, provides the budget, and removes obstacles. Usually you, the owner, or a senior manager. Their job is to keep the project pointed at a real business goal and to make the final calls. Without a clear decider, the project has no authority and stalls when the first hard choice appears.

**The users (the frontline).** These are the people who work with the tool every day. They are not passive recipients; they are the ones who

make it useful or ignore it completely. Their daily experience is the real test of whether the tool works. Treat them as partners, not subjects.

**The checker (the reviewer).** This is the person who verifies that the AI's output is correct, safe, and fair before it reaches a customer or a decision. In a small business this might be a careful senior staff member. In a larger one, a dedicated review or compliance role. The checker is your safety net against errors, hallucinations (when AI confidently states something false), and bias. Never let the tool act with no one checking.

A simple way to hold this is a one-line rule per task: **one person decides, the team uses, one person checks.** Write it down. When something goes wrong, you want to know exactly whose desk it lands on. The separation between "uses" and "checks" matters — the person rushing to send a hundred emails is not the best person to catch a subtle error. A second pair of eyes is not distrust; it is good design.

## 15.2 The fear that AI will steal jobs: how to face it

This is the heart of the human side. If you handle it well, you gain trust. If you fumble it, you lose your best people and poison the well.

**Acknowledge the fear instead of denying it.** Do not say "nothing will change" if something will. People are not naive; they read the news. A denial destroys credibility. Say instead: "This will change some tasks. Some work will get easier. Some tasks may disappear. Here is what we plan to do about it."

**Be honest about what you can and cannot promise.** If you can commit that no one will be laid off because of this tool, say so clearly and mean it. If you cannot promise that, do not fake it. A broken

promise is far worse than an honest "we will work with you if roles change."

**Focus on tasks, not people.** AI usually changes *tasks*, not whole jobs. A person's role is a bundle of tasks; AI may take a few and leave the rest, or create new ones. Talk about the work, not about who is "redundant." This reframes the conversation from threat to change.

**Offer a path forward.** Fear shrinks when people see a way ahead. Offer training. Show them how to use the tool to become more valuable, not less. Redeploy people from tasks the machine now does to higher-value work only humans can do — relationships, judgment, creativity, care.

**Share the gains.** If AI makes the business more profitable, let the people who helped get there see some benefit — better pay, better hours, new opportunities. If all the gain flows only to the owner while workers lose, resentment is guaranteed.

**Give people a voice.** Let them ask hard questions and be answered respectfully. A person who feels heard can accept change. A person who feels silenced will resist it, openly or quietly.

The principle: treat your people the way you would want to be treated if a machine were changing your job — with honesty, respect, and a real path forward.

## 15.3 Minimum training for everyone

Not everyone needs to be an expert. But everyone who touches AI needs a small, shared baseline. Think of it like fire safety: everyone in the building needs to know where the exits are, not how to be a firefighter.

The minimum baseline for all staff:

- **What AI can and cannot do.** It is a powerful helper that guesses from patterns. It is fast and useful, but it makes mistakes and has no real understanding. Knowing the limits prevents over-trust.
- **How to talk to it.** Basic prompting — how to ask clearly, give context, and check the answer. A short, hands-on session, not a course.
- **Data and privacy basics.** Never paste personal or confidential data into a public AI tool. Know what is safe to share and what is not. The full rules are in Chapter 10.
- **How to spot a wrong answer.** AI can sound confident and be wrong. Train people to check important facts, especially numbers, names, dates, and legal or medical claims.
- **When to stop and escalate.** A clear rule: if the output matters to a customer, a decision, or a person, a human reviews it. If unsure, ask.

Make training short, practical, and repeated. One long session is forgotten in a week. Short refreshers, quick guides, and on-the-job coaching work far better. Train the users first, not only the managers. A manager who understands the tool but whose team cannot use it has bought a decoration.

## 15.4 New roles: AI facilitator, data owner, privacy officer

AI brings a few new responsibilities into the business. In a large company these are full-time jobs. In a small business they are "hats" that existing people wear on top of their normal work. What matters is that each hat has a name and a person.

**The AI facilitator (or champion).** This is the person who helps everyone else use AI well. They learn the tools a little faster, answer

colleagues' questions, gather feedback, and suggest improvements. They are not a technician — often they are a respected colleague who is curious and patient. In a small firm, pick one or two enthusiastic people and give them a little extra time and support. They are your engine of adoption.

**The data owner.** This is the person responsible for a set of data being accurate, current, and available — the "single source of truth" from Chapter 14. When AI needs customer data, the data owner is accountable for its quality and for who may use it. Without a data owner, data rots and AI builds on sand.

**The privacy officer.** This is the person who makes sure the business respects people's privacy rights and the law. For many businesses this duty is required by GDPR, and the full treatment is in Chapter 10. Even if you are not legally required to appoint one, having someone who owns privacy questions is wise. They ask: is it right and legal to use this data this way?

**The decider and the checker** from 15.1 round out the set. Together these roles answer the three big questions — who decides, who uses, who checks — and add the guardians for data and privacy. Write down who wears each hat, even if one person wears three. Clarity beats a perfect org chart.

## 15.5 Internal communication

Culture is built by what you say, how often, and whether people believe you. Communication is not a one-time announcement; it is a steady rhythm.

**Start with the why.** Before any tool, explain the reason in plain words: what problem we are solving and what is in it for us and for our

customers. People support a purpose they understand.

**Be consistent and frequent.** A single kickoff meeting is not enough. Share updates weekly or monthly. Silence is filled by rumor, and rumor is usually worse than the truth.

**Celebrate wins and admit failures.** When AI saves time or a person masters a new skill, say so publicly. When it makes a mistake, share that too. Admitting failures builds trust far more than only reporting good news, because it shows you are honest and in control.

**Create a place for questions.** A shared document, a short weekly meeting, or a simple channel where people can ask "is it safe to use AI for this?" and get a real answer. Unanswered questions turn into fear and shadow use (people using AI secretly, which is covered in Chapter 9).

**Lead from the front.** When the owner uses the tool and talks about it openly, it gives everyone permission to engage. If the boss stays away from it, staff read that as a signal not to trust it.

A good rule: over-communicate by a little. The cost of repeating yourself is tiny. The cost of people feeling left in the dark is huge.

## 15.6 Involving the people who work on the processes every day

The people who do a job every day know things no manager and no consultant can see. They know the messy exceptions, the unwritten rules, the steps that always break, and the shortcuts that actually work. If you design your AI around the process without them, you will design it wrong.

**Ask them first.** Before automating a task, sit with the people who do it and map the real process, not the official one. They will tell you where AI helps and where it would cause trouble.

**Co-design, don't impose.** Bring them into choosing and shaping the tool. People support what they help build. A tool handed down creates resistance; a tool shaped with the team creates ownership.

**Use their judgment as the test.** If the frontline says "this answer is wrong" or "this won't work with our customers," believe them. They are your best quality check, because they know the reality the tool is entering.

**Give them the feedback loop.** Make it easy for users to report when the AI is wrong or helpful, and act on what they say. When people see their feedback changing the tool, they engage. When it vanishes into a void, they stop trying.

**Respect their expertise.** The message is not "the machine replaces your knowledge." It is "the machine handles the boring parts so your knowledge goes further." That respect is what turns a fearful worker into an enthusiastic partner.

In short: the experts on any process are the people doing it. Include them, and the project gains both better design and better adoption at once.

## Ethics and Responsibility

The human side of AI is mostly an ethics question.

**Treat people with dignity.** Workers are not obstacles to automation or costs to cut. They are people whose livelihoods depend on you.

Decisions about AI are decisions about their lives.

**Be honest about job impact.** Do not hide, sugarcoat, or postpone hard truths about changing roles. Honesty, even when uncomfortable, is the foundation of trust.

**Do not use AI to spy or punish.** Monitoring staff with AI to catch small mistakes, or using it to pressure people, breaks trust and often the law. Use AI to help people do better work, not to police them.

**Fairness in reskilling.** Offer training and new opportunities fairly. Do not let AI quietly decide who is "worth keeping." A human must own those decisions, with fairness and a record.

**Psychological safety.** People must feel safe to admit they do not understand the tool, to report a mistake, and to ask questions without being mocked or punished. Without that safety, problems stay hidden until they explode.

**Informed consent and transparency.** Tell employees when AI is used in decisions that affect them, and give them a way to question it. The principles behind this are in Chapter 4.

Handle the people side with the same care you would want if the machine were changing your own job. That single test covers most of the ethics.

## Mistakes to Avoid

**Surprising people with the tool.** Switching AI on without warning breeds fear and resistance. Always prepare the ground first.

**Promising "no job will change" when it will.** A lie that collapses the moment reality shows. Be honest about change.

**Ignoring the fear.** Treating job anxiety as irrational or silly. It is real, and it must be faced directly.

**Training only the managers.** A tool the team cannot use is a tool nobody uses. Train the users first.

**No checker role.** Letting AI output reach customers or decisions with no human review. Sooner or later, a bad answer slips through.

**Blaming staff for tool failures.** When the AI errs, blaming the user instead of fixing the process makes people hide problems.

**Skipping the frontline.** Designing the automation without the people who do the work guarantees a design that misses reality.

**Treating culture as an afterthought.** Buying the tool first and thinking about people later. The people plan should come before the purchase.

**Secret pilots that leak.** Running a hidden trial that staff discover by accident. It looks like a trick and destroys trust.

**No communication after launch.** Announcing the tool and never following up. Silence becomes rumor.

**Letting one person wear every hat with no time.** Assigning new roles without giving people the hours to do them. The roles become empty titles.

**Confusing a demo with adoption.** A tool that works in a meeting is not a tool that is used. Measure real daily use, not enthusiasm in the room.

## Practical Exercise

### 15.7 A map of roles

Take one AI project you are planning or running and build a simple role map. This single page prevents most people-related failures.

Draw a small table. Down the left side, list the key activities of the project — for example: choose the tool, set the goal, use it daily, review its output, handle data, answer staff questions, report results. Across the top, list the people in your business.

For each activity, mark who is:

- **Decides** — owns the choice and the final call.
- **Does / Uses** — works with it day to day.
- **Checks** — reviews for correctness, safety, fairness.
- **Helps / Informed** — supports or needs to know.

Then look for the gaps:

- Any activity with **no decider**? It will stall. Assign one.
- Any activity with **no checker**? It is a risk waiting to happen. Assign one.
- Any person with **too many "decides"**? You have a bottleneck. Spread the load.
- Any important person with **no role at all**? They will feel left out and may resist. Give them a place.
- Is the **fear conversation** on the map? If not, add "talk to staff about job impact" as an activity and assign a decider (usually you).

Finally, name the four hats from 15.4 — facilitator, data owner, privacy officer, checker — and write down who wears each. Even if one person

wears three, write it. The map turns a vague “we’re doing AI” into a clear, owned plan.

## Checklist

### 15.8 People checklist

Before and during any AI rollout, check these.

- ☐ **A clear decider** owns the project and its goal.
- ☐ **Named users** are identified for the tool.
- ☐ **A checker** reviews AI output before it reaches customers or decisions.
- ☐ **The fear of job loss** has been addressed openly and honestly.
- ☐ **You have stated what you can and cannot promise** about jobs.
- ☐ **A path forward** (training, redeployment) is offered to affected staff.
- ☐ **The gains** from AI will be partly shared with the people who helped.
- ☐ **A minimum training baseline** is delivered to all users, not just managers.
- ☐ **The four hats are named** — facilitator, data owner, privacy officer, checker.
- ☐ **Internal communication** has a regular rhythm, not a one-time announcement.
- ☐ **A place for questions** exists and gets real answers.
- ☐ **The frontline people** who do the process were involved in the design.
- ☐ **A feedback loop** lets users report errors and see them acted on.
- ☐ **Staff are not monitored or punished** using AI.

- [ ] **You lead from the front** — you use the tool and talk about it openly.

If a box is empty, the people side is not ready. Fix it before you push the tool harder. The technology will wait; trust will not.

## Key Takeaways

- AI projects fail on people, not technology — a clear decider, named users, and a checker are what make a tool actually get used.
- The fear of losing a job is real and rational; face it with honesty, a clear path forward, and shared gains, never with denial or mockery.
- Everyone needs a small training baseline — what AI can and cannot do, how to ask, what data is safe, and when to escalate to a human.
- New roles (facilitator, data owner, privacy officer, checker) are hats people wear; name who wears each, even in the smallest business.
- Involve the people who do the work every day — they know the real process, and people support what they help build.

# Chapter 16 — Goals, Costs, and Return on Investment

## In Simple Words

Before you spend money on AI, you need to answer three plain questions. What do you want to achieve? What will it cost? What will you get back? If you cannot answer these, you are not making a decision — you are guessing. This chapter gives you a simple, honest way to answer them.

The central idea is called **ROI**, which stands for **Return on Investment**. In plain words, ROI measures how much value you get back for every unit of money you put in. If you spend one euro and get three euros of value, that is a good return. If you spend one euro and get fifty cents back, that is a bad one. ROI turns a fuzzy "is this worth it?" into a number you can think with.

This chapter is the **home of the ROI method** for this book. The simple formula, how to use it, and a worked example all live here. When other chapters talk about measuring results or return, they point back to this one. Learn the method here once, and you can apply it to any AI project.

A simple image to carry: ROI is like checking whether a machine in your workshop is worth buying. You ask what it costs to buy and run, and how much money or time it saves you. If it saves more than it costs, it pays for itself and then some. If not, it is an expensive ornament. AI is no different — except that its costs and benefits are sometimes hidden, so you have to look harder for both.

One honest warning up front: numbers can lie if you want them to. It is easy to inflate the savings and shrink the costs to make a bad idea look good. The method here is built to keep you honest. Use it that way, not as a way to justify a decision you already made.

## A Bit of History

**Early 1900s: ROI is born for factories.** A manager named Donaldson Brown, working at DuPont in the 1910s and 1920s, developed a way to compare how well different parts of a business used their money. He expressed it as a ratio of profit to the money invested. This let owners compare one factory against another and decide where to put the next dollar. It was a small idea with huge influence — it made "return" a number you could manage.

**Mid-1900s: payback period.** Businesses added a simpler cousin of ROI: how long until the savings cover the cost? If a machine pays for itself in two years, that felt safe. If it took fifteen, that felt risky. Payback became a quick gut-check for big purchases.

**1980s–1990s: the limits of counting money.** Managers realized that some of the most important things — customer satisfaction, quality, learning, employee morale — did not show up in a simple profit number. The "balanced scorecard," developed by Kaplan and Norton in the early 1990s, tried to capture these alongside the money. The lesson: count what matters, not only what is easy to count.

**2000s: tech projects and the productivity paradox.** Companies poured money into software and sometimes saw little return, leading to a famous joke that computers showed up everywhere in the economy's numbers except in productivity. The cause was rarely the technology. It

was vague goals, hidden costs, and poor adoption. Measuring the right things became a discipline of its own.

**2020s: AI forces honest math.** AI can save real time and money, but its costs are easy to underestimate (subscriptions, setup, training, maintenance) and its benefits are easy to overstate (hyped savings, ignored learning curves). A simple, honest ROI method is now essential to separate the tools that pay off from the ones that just sound good.

The arc: from a factory ratio to a modern discipline. The tools changed; the core question never did — did you get back more than you put in?

## Curiosity

### 16.7 The company that cut query-resolution time by 99%

One of the most striking recent examples of AI's effect on a business process is the animal-health company **Elanco**, which reported cutting the time to resolve certain internal information queries by about **99%** after putting a generative-AI assistant to work — turning a search that once took many minutes into a near-instant answer.

That is the headline. The full case, with the details and the lessons behind the number, is told in Chapter 25 — Administration and Finance, the canonical home of the Elanco story. The point for this chapter is one line: a huge saving in time is a real, measurable benefit — and it is exactly the kind of thing a good ROI calculation is designed to capture.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A logistics firm wanted to "use AI," so it bought a chatbot for customer questions. Nobody set a clear goal first. Six months later, the owner asked, "Is this working?" and nobody could answer. There was no starting number to compare against, so there was no way to tell if anything improved. The tool was probably helping a little, but the firm could not prove it, could not size it, and could not decide whether to expand or stop. The money was spent, and the lesson was lost.

A second firm did the opposite. Before buying anything, it picked one clear goal: cut the time its team spent answering the same ten repeated customer questions. It measured the starting point — about 30 hours a week across the team. It set a target: half that, within three months. It counted the costs honestly, including the time to set up and train. At the end, it had a real before-and-after number and a real cost figure, and it could compute a clear return. The decision to expand was easy because the math was clear.

The difference was not the AI. It was that the second firm defined a goal and measured it before it spent a euro. That is what the rest of this chapter teaches you to do.

## How to Do It

### 16.1 Defining simple and measurable objectives

A goal you cannot measure is a wish, not a goal. "Use AI better" is a wish. "Cut invoice processing time by half by June" is a goal. The first

step in any ROI calculation is to turn your intention into something you can measure.

A good objective has four parts:

- **One metric.** Pick a single number that matters — hours spent, cost per item, errors made, response time, sales closed. Not five numbers. One.
- **A baseline.** What is that number *right now*? Measure it before you start. Without a baseline, you can never prove change. If you do not know your current invoice time, go measure it this week.
- **A target.** What do you want it to become? Be specific: "half," "two minutes instead of ten," "ten fewer errors a week."
- **A time frame.** By when? "In three months" gives the goal a deadline and makes it real.

Keep it simple. A small business does not need a dashboard of fifty metrics. One clear metric per project, with a baseline and a target, is enough to drive a real decision.

Write the objective in one sentence: *"By [when], we will change [metric] from [baseline] to [target]."* If you cannot fill in those blanks, you are not ready to spend. Go back and define it. This single habit prevents most wasted AI spending.

## 16.2 Direct and indirect costs

The price tag is not the cost. Most people count the subscription and stop. The real cost is bigger, and the hidden parts are what turn a good-looking project into a money pit. Count both kinds.

**Direct costs** are the obvious ones you pay for:

- **Software and subscriptions.** The monthly or yearly fee for the AI tool.
- **Setup and integration.** The work to connect the tool to your existing systems.
- **Hardware.** Any equipment you need to buy.
- **External help.** Consultants, developers, or trainers you pay.
- **Training.** The cost of teaching people to use it.

**Indirect costs** are the hidden ones people forget:

- **Staff time to set up and run it.** Hours your own people spend configuring, testing, and managing the tool. This is real money, even if no invoice shows it.
- **The learning dip.** When you change how people work, they slow down before they speed up. A few weeks of lower productivity is normal and costs something.
- **Maintenance and updates.** Tools need tending — fixing breaks, updating settings, handling new cases.
- **Review and checking.** The human time to check AI output, which you should never skip.
- **Governance and compliance.** The effort to keep the tool within privacy and legal rules (see Chapter 10).
- **Switching and exit costs.** What it costs to change tools later if this one disappoints.

A good rule: the visible price is often only half the real cost. Add up every cost you can think of, including your own people's time, and value that time at what it truly costs you (salary plus overhead, not just the take-home pay). Underestimating cost is the most common way people fool themselves.

## 16.3 Quantifiable and non-quantifiable benefits

Benefits come in two kinds. Some you can put a number on. Some you cannot, but they still matter. A good calculation handles both honestly.

**Quantifiable benefits** can be turned into money:

- **Time saved.** Hours not spent on a task, valued at the cost of that time.
- **Errors avoided.** Fewer mistakes means fewer refunds, re-dos, and penalties.
- **Faster cycles.** Quicker invoices get paid sooner; quicker quotes win more deals.
- **More sales or output.** If AI helps you serve more customers or produce more.
- **Lower labor or material costs.** Direct savings you can point to.

**Non-quantifiable benefits** are real but hard to price:

- **Employee satisfaction.** People may enjoy work more when the boring parts are automated.
- **Customer experience.** Faster, clearer service makes customers happier, even if you cannot pin a number on it today.
- **Less stress and fewer late nights.** Quality-of-life gains that do not show in a spreadsheet.
- **Better decisions.** AI can surface insights that improve judgment.
- **Reputation and readiness.** Being a business that uses modern tools well.

How to handle both: **monetize what you honestly can, and write down the rest separately.** Do not force a fake number onto something intangible. Do not pretend the intangible is worth zero either. List it, name it, and weigh it in your judgment alongside the

hard numbers. A project with modest money savings but big gains in staff morale and customer happiness may still be worth doing. The point is to see the whole picture, not a false single number.

## 16.4 A simple calculation of ROI

Now the method itself. The core formula is simple.

**ROI = (Net Benefit ÷ Total Cost) × 100%**

Where **Net Benefit = Total Benefit − Total Cost**.

A positive ROI means you gained more than you spent. A negative ROI means you lost money. The higher the percentage, the better the return.

**A tiny worked example.** Suppose an AI tool costs **€10,000** in total for the first year (subscription plus setup plus training). You measure that it saves your team **€25,000** worth of time and errors over that year.

- Net Benefit = €25,000 − €10,000 = **€15,000**
- ROI = (€15,000 ÷ €10,000) × 100% = **150%**

A 150% return means for every euro you spent, you got your euro back plus €1.50 extra. That is a strong return.

**Payback period** is the companion number: how long until the savings cover the cost? If you save €25,000 a year, that is about €2,083 a month. To cover a €10,000 cost takes about **4.8 months**. After that, the savings are pure gain.

How to read the result:

- **ROI clearly positive and payback short** (a few months) — a strong case to proceed.

- **ROI positive but payback long** (several years) — think carefully; the margin is thin and risks could erase it.
- **ROI near zero or negative** — the tool does not pay for itself on the numbers you have. Either find more benefit, cut cost, or walk away.

Two honest habits make this method trustworthy. First, use your real baseline and your full cost, not optimistic guesses. Second, run the calculation over a clear period (usually one year) and repeat it after launch with actual numbers, not projections. The first calculation is an estimate to decide; the second is the truth to learn from.

## 16.5 Example: automating invoices

Here is the full method applied to a common case — automating invoice processing. *All numbers are illustrative; replace them with your own.*

**The setup.** A small firm handles **2,000 invoices a month** (24,000 a year). Right now, staff do them by hand.

**Step 1 — Baseline.** Each invoice takes about **10 minutes** by hand. That is 240,000 minutes a year = **4,000 hours a year**. At a fully-loaded staff cost of **€25 per hour**, the manual work costs about **€100,000 a year**.

**Step 2 — What the AI changes.** The tool fully automates about **80%** of invoices, cutting each of those from 10 minutes to about 2 minutes of human review. So it saves 8 minutes on 19,200 invoices (80% of 24,000). That is 153,600 minutes = **2,560 hours a year saved**. At €25/hour, the **gross benefit is about €64,000 a year**.

**Step 3 — Total cost.** Software subscription **€12,000/year**, one-time setup **€8,000**, training **€2,000**. First-year total cost = **€22,000**. In

steady years after, cost drops to about **€14,000** (subscription plus maintenance).

**Step 4 — ROI, first year.** - Net Benefit = €64,000 − €22,000 = **€42,000** - ROI = (€42,000 ÷ €22,000) × 100% ≈ **191%**

**Step 5 — Payback.** Monthly savings ≈ €64,000 ÷ 12 ≈ €5,333. To cover the €22,000 first-year cost takes about **4 months**.

**Step 6 — Steady state (year two).** - Net Benefit = €64,000 − €14,000 = **€50,000** - ROI = (€50,000 ÷ €14,000) × 100% ≈ **357%**

**Reading it.** The first year returns about 191% and pays for itself in roughly four months. After that, the return climbs because the one-time costs are gone. This is a clear, strong case — and you can see exactly why, because every number came from a measured baseline and a full cost list. Now imagine doing this with *your* invoice count, *your* time per invoice, and *your* costs. That is the calculation that decides your real choice.

## Ethics and Responsibility

ROI is a tool for honest decisions, and it can be misused.

**Do not inflate savings to justify layoffs.** The easiest way to make a bad project look good is to overstate the savings and then cut people. Use honest numbers, and decide about people separately and humanely (see Chapter 15).

**Count the human cost.** The learning dip, the stress of change, the time people spend adapting — these are real costs. Leave them in the math. A clean number that hides a messy reality is a dishonest number.

**Do not game the baseline.** If you make the starting point look worse than it was, the improvement looks bigger than it is. Measure the baseline honestly, before you have a stake in the result.

**Value the non-quantifiable.** Do not treat staff morale, customer happiness, and reduced stress as zero just because they lack a number. They belong in the decision, even if only in your judgment.

**Be transparent with stakeholders.** If you report ROI to a board, a partner, or a lender, show your assumptions and your method, not just the headline number. Let others check your work.

**Separate the money decision from the people decision.** A positive ROI does not automatically mean "cut staff." It means the tool pays for itself. What you do with the freed time and money is a separate, human choice.

An honest ROI calculation protects you from fooling yourself and protects your people from being a line item in a rigged number.

## Mistakes to Avoid

### 16.6 Errors of evaluation

**No clear objective.** Spending without a measurable goal, so you can never tell if it worked. Define the metric first.

**No baseline.** Not measuring the starting point, so any change is unprovable. Measure before you start.

**Ignoring indirect costs.** Counting only the subscription and forgetting setup, staff time, the learning dip, and maintenance. The hidden costs are where projects die.

**Overestimating savings.** Believing the vendor's best-case promise instead of your own measured reality. Be conservative.

**Counting gross instead of net.** Reporting the total benefit without subtracting the cost. ROI is about the *net* gain.

**Double counting.** Counting the same saving twice, or counting a benefit that overlaps with one already counted.

**Ignoring the learning curve.** Assuming full savings from day one. Real savings ramp up over weeks.

**Too short a time horizon.** Judging a tool after two weeks, before the benefits show. Give it a fair period, usually a year.

**Ignoring maintenance.** Forgetting that costs continue after launch. A tool that needs constant tending may never pay off.

**Treating the non-quantifiable as zero.** Discarding real benefits because they lack a number. List them and weigh them.

**ROI theater.** Doing the math backward to justify a decision already made. Use the method to decide, not to decorate.

**Never re-running the numbers.** Staying on the projection and never checking against reality. Re-measure after launch and learn.

## Practical Exercise

### 16.8 Calculate the ROI of one activity

Pick one activity in your business that AI could help with — answering emails, processing invoices, summarizing documents, scheduling. Run the full method on it.

**Step 1 — Define the objective.** Write one sentence: *"By [when], we will change [metric] from [baseline] to [target]."* If you do not know the baseline, go measure it this week before continuing.

**Step 2 — Measure the baseline.** How much time or money does this activity cost you now, per month or per year? For time, multiply hours by a fully-loaded hourly cost (salary plus overhead). Write the number down.

**Step 3 — List all costs.** Direct costs (subscription, setup, hardware, external help, training) plus indirect costs (your staff's time to set up and run it, the learning dip, maintenance, review time). Add them into a first-year total and a steady-year total.

**Step 4 — Estimate the benefit.** How much time or money will AI realistically save? Be conservative. Use a fraction of the baseline you measured, not a fantasy. Convert saved time to money at your hourly cost.

**Step 5 — Compute ROI.** Net Benefit = Total Benefit − Total Cost. ROI = (Net Benefit ÷ Total Cost) × 100%.

**Step 6 — Compute payback.** Divide the total cost by your monthly savings to see how many months until it pays for itself.

**Step 7 — List the non-quantifiable benefits.** Write down the benefits you cannot price — morale, customer happiness, less stress. Note them beside the number.

**Step 8 — Decide.** Is the ROI clearly positive with a short payback? Strong case. Thin margin? Be cautious. Negative? Walk away or find more benefit.

Write the whole thing on one page. That page is your decision. Keep it, and after you launch, fill in the *actual* numbers and compare. The gap between your estimate and reality is the lesson you will carry to the next project.

## Checklist

### 16.9 Economic checklist

Before spending on any AI project, check these.

- ☐ **You have one clear, measurable objective** with a single metric.
- ☐ **You have measured the baseline** (the current number) before starting.
- ☐ **You have a specific target and a time frame.**
- ☐ **You have listed all direct costs** — subscription, setup, hardware, external help, training.
- ☐ **You have listed all indirect costs** — staff time, learning dip, maintenance, review, compliance.
- ☐ **You valued staff time at a fully-loaded rate**, not just take-home pay.
- ☐ **You estimated savings conservatively**, not from vendor hype.
- ☐ **You computed Net Benefit** (total benefit minus total cost).
- ☐ **You computed ROI** = (Net Benefit ÷ Total Cost) × 100%.
- ☐ **You computed the payback period** in months.
- ☐ **You listed the non-quantifiable benefits** and weighed them in your judgment.
- ☐ **You used a fair time horizon** (about a year), not a few weeks.
- ☐ **You did not double count** any benefit.

- ☐ **You will re-run the numbers with actual results** after launch.
- ☐ **You kept the money decision separate from the people decision.**

If a box is empty, the economics are not settled. Fill it before you spend. A number you can trust is worth more than a promise you cannot check.

## Key Takeaways

- ROI = (Net Benefit ÷ Total Cost) × 100%, where Net Benefit = Total Benefit – Total Cost; a positive number with a short payback is a strong case to proceed.
- Define one measurable objective with a real baseline before you spend — without a starting number, you can never prove the change.
- The visible price is only half the cost; count indirect costs like staff time, the learning dip, and maintenance, or you will fool yourself.
- Monetize what you honestly can and write down the rest — non-quantifiable benefits like morale and customer happiness still belong in the decision.
- Use the method to decide, not to decorate: honest numbers protect you from bad choices and your people from being a rigged line item.

# Chapter 17 — Choosing Tools Without Being Fooled

## In Simple Words

Every AI vendor will tell you their tool is easy, cheap, and revolutionary. Some of them are telling the truth. Many are not. This chapter is your defense. It teaches you how to look past the shiny demo and the confident salesperson, and choose a tool that actually fits your business.

The most important idea here is simple: **you are not buying a tool, you are buying a result.** A tool is only worth what it does for you. A beautiful product that does not solve your problem is an expensive toy. Your job is to connect every choice back to a real problem and a real number.

A good analogy is buying a car. The showroom makes every car look amazing. The leather shines, the engine purrs, the salesperson is charming. But you do not buy a car for the showroom. You buy it for the road you actually drive, the passengers you actually carry, and the fuel you can afford. So you ask hard questions, you take it for a test drive on *your* roads, and you read the running costs. Choosing AI tools is the same discipline.

This chapter covers how to pick between buying ready-made software, building something custom, or doing it yourself. It gives you the exact questions to ask a vendor. It shows you where hidden costs hide. It explains why a demo is not enough and how to run a small pilot

instead. And it looks at open-source tools — software whose code is free to use and change — as a real, affordable option.

One rule to carry with you: **slow down.** The salesperson wants urgency. “This price ends Friday.” “Only two licenses left.” Urgency is a trick to stop you thinking. A good tool survives a week of careful thought. A bad one relies on you not thinking. Take the week.

## A Bit of History

**1960s–1970s: software is custom.** In the early days, if a company wanted software, it paid specialists to write it from scratch. There was no “off-the-shelf” option. Software was a bespoke suit, made to measure, expensive, and slow to deliver.

**1980s: the packaged software revolution.** Products like spreadsheets and word processors arrived in a box. For the first time, a business could buy a general tool and adapt it to many jobs. This was cheaper and faster than custom builds. The modern software market was born.

**1990s–2000s: big suites and the lock-in trap.** Large vendors sold huge integrated suites — one product for everything. They worked well, but switching away was painful and costly. Companies discovered that the easy choice today could become an expensive prison tomorrow. The word **vendor lock-in** — being stuck with a supplier because leaving is too hard — entered the business vocabulary. (Lock-in is covered fully in Chapter 9.)

**2000s: open-source goes mainstream.** Linux, Apache, and later thousands of free projects proved that software could be built by communities and given away, yet power the world’s biggest websites.

Open-source stopped being a hobby and became a serious, trusted option.

**2010s: the subscription era.** Software moved from "buy once" to "pay every month." This lowered the entry price but added a new kind of cost that never ends. Subscriptions made it easy to start and easy to lose track of what you were paying for.

**2020s: the AI gold rush.** Hundreds of AI tools appeared almost overnight, many with big funding and bigger promises. The gap between what a demo shows and what a tool delivers in real work has never been wider. Choosing well is now a core business skill, not a nice extra.

The arc: from bespoke, to boxed, to subscribed, to flooded. The tools keep changing. The need to choose carefully never does.

## Curiosity

## 17.6 The book "Headcount Zero" — a company with no employees

How far can the "no people needed" idea go? There is a book that pushes it to the extreme. It is called **"Headcount Zero: How to Build an AI-Run Company with Paperclip"**, by **Anthony David Adams**. It is published as an open-source book on GitHub (the repository `AnthonyDavidAdams/zero-employee-company-book`), so anyone can read it for free.

The question the book asks is: *what if you never had to hire anyone?* Instead of employees, the founder runs a company made of **AI agents** — software programs that can carry out tasks on their own. The

founder becomes a kind of manager of an org chart full of AI workers, coordinated by an open-source platform the book calls **Paperclip**.

This is not a plan most small businesses should follow tomorrow. A company with literally zero humans is an extreme thought experiment, and it has obvious limits: who is accountable when something goes wrong? Who holds the legal responsibility? Who understands a customer's unspoken need? (These are exactly the human roles discussed in Chapter 15.)

But as a lens, "Headcount Zero" is useful. It forces one honest question: **how much of your work is truly human, and how much is routine that a machine could carry?** Most businesses find that a surprising share of daily tasks is routine. That does not mean firing everyone. It means freeing people from the boring parts so they can do the judgment, the relationships, and the creative work that machines cannot. Read the book as a provocation, not an instruction. It shows you the edge of the possible, and the edge is where the interesting questions live.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A small accounting firm wanted to "automate with AI." A vendor gave a dazzling demo: the tool read a stack of receipts and produced a tidy summary in seconds. The owner signed a two-year contract on the spot, dazzled.

Six months later, reality looked different. The demo had used a handful of clean, perfect receipts. The firm's real receipts were crumpled, blurry, in three languages, and full of edge cases. The tool got them

wrong often enough that staff had to re-check everything, which meant the "saving" was tiny. The two-year contract locked them in. The subscription kept billing. The tool became an expensive ornament.

A second firm chose differently. Before signing anything, it asked the vendor for a **trial on its own messy data**, not the vendor's clean demo set. It ran a two-week pilot on real receipts. The tool handled about 70% well and failed on the rest. That number — 70% — let the firm make a clear decision: keep the tool for the easy 70%, keep humans on the hard 30%, and sign only a one-year contract with a clear exit. The same tool, chosen with discipline instead of dazzle, became a genuine help.

The difference was not the software. It was that the second firm refused to be fooled by a demo and tested on its own reality first.

## How to Do It

### 17.1 Off-the-shelf software, custom-built, or do-it-yourself

When you need a tool, you have three broad paths. Each has a place.

**Off-the-shelf (buy it ready-made).** This is software you subscribe to or purchase as-is, like a chatbot service or a document-summarizing app. It is the fastest and usually the cheapest way to start. You get it today, it works out of the box, and someone else maintains it. The trade-off: it does what *it* was built to do, not exactly what *you* wanted. If your need is common, an off-the-shelf tool is almost always the right first choice.

**Custom-built (pay someone to build it).** A developer writes software just for you. This fits your needs perfectly. But it is slow,

expensive, and you now own a thing that needs maintenance forever. Custom builds make sense only when your need is special and no ready-made tool covers it — and when the value is high enough to justify the cost. For most small businesses, custom-built is rarely the first move.

**Do-it-yourself (you build it, with no-code or light tools).** Tools like automation apps let you connect services yourself without writing code. This is cheap and flexible, and you learn a lot. The trade-off: it takes your time, and if you leave, the thing you built can be hard for someone else to maintain. DIY is great for small automations you can own and keep simple.

A simple rule of thumb: **start off-the-shelf. Go DIY for small glue-work between tools. Go custom only when nothing else fits and the prize is big.** Most businesses live happily on the first two.

## 17.2 Questions to ask the vendor

Never take a vendor's word. Ask direct questions and listen carefully to the answers — especially to what they dodge. Keep these ready:

- **What exactly does this do, and what does it NOT do?** Force them past the marketing. A confident "it handles everything" is a red flag.
- **What does it look like when it fails?** Every tool fails somewhere. An honest vendor can name its weak spots. A vendor who says "it never fails" is lying or clueless.
- **What is the total cost over three years, not just the first month?** Make them say the full number out loud.
- **What happens to my data?** Where is it stored, who can see it, and do you train your AI on it? (This matters for privacy — see

Chapter 10.)

- **Can I get my data out, and in what format?** This is your escape hatch. If you cannot leave cleanly, you are locked in.
- **What support do I get, and how fast?** Response times, channels, and whether support is included or costs extra.
- **Who else uses this in my industry, and can I talk to them?** A real reference is worth ten demos.
- **What is your roadmap, and how stable is the company?** A tool from a shaky startup may vanish. Ask how long they have been in business and who funds them.
- **What is the contract, and how do I cancel?** Read the exit terms before you sign, not after.

Write down the answers. Compare vendors on the same questions. The vendor who answers clearly and honestly stands out from the one who waves and charms.

## 17.3 Hidden costs and subscriptions

The sticker price is the smallest cost. The real cost hides in places most people never look. (The full cost method lives in Chapter 16; here is the vendor-specific version.)

Watch for these:

- **Per-seat pricing.** Many tools charge per user. A "cheap" tool becomes expensive when you add your whole team. Count seats before you sign.
- **Usage-based fees.** Some AI tools charge per task, per message, or per document. A busy month can produce a surprise bill. Ask exactly how usage is metered.
- **Setup and onboarding fees.** Getting started can cost extra, sometimes more than the first year of subscription.

- **Integration costs.** Connecting the tool to your existing systems may need paid help.
- **Premium features behind a paywall.** The feature that sold you may be on a higher tier. Check which tier you actually need.
- **Training and support as add-ons.** "Support included" often means a help article, not a person. Real help may cost more.
- **The never-ending subscription.** A monthly fee looks small but never stops. Multiply it by three or five years to see the true weight.
- **Exit and switching costs.** Canceling may be hard, and moving your data elsewhere may take paid work.

Before you sign, build a **three-year total cost** for each option. Add every one of these. The vendor's monthly price is often only a third of the real three-year number.

## 17.4 Demos and pilot projects

A demo is a performance. It shows the tool at its best, on data chosen to win you over. Never decide on a demo alone.

A **pilot project** is the honest alternative. A pilot is a small, time-boxed trial of the tool on *your* real work, with *your* real messy data, measuring a *your* real result. Where a demo shows you a highlight reel, a pilot shows you the truth.

How to run a good pilot:

- **Use your own data, including the messy cases.** Do not accept the vendor's clean sample.
- **Keep it small and short.** Two to four weeks on one process is enough to learn a lot.
- **Define success before you start.** What number must the tool hit to count as a pass? (This links to the pilot method in Chapter

18.)

- **Test the failure, not just the success.** Push it with hard cases on purpose.
- **Try the support.** Send a support request during the pilot and see how fast and helpful they are.
- **Test the exit.** Try exporting your data. Make sure you can leave.

A vendor who refuses a real pilot on your data is telling you something. A vendor who welcomes it is confident for good reason.

## 17.5 Open-source tools: an affordable and flexible alternative

**Open-source** means the software's source code — the instructions that make it run — is free for anyone to see, use, and change. You are not "pirating" it; open-source is a legal, common, and often excellent way software is made. The web you are using right now likely runs on open-source software.

Why consider it for AI:

- **Low or no license cost.** Many open-source tools are free to use.
- **No lock-in.** Because you can see and change the code, you are not trapped with one vendor.
- **You can run it yourself.** Open-source AI models can run on your own machines, which keeps your data under your control (this is **self-hosting**, covered in Chapter 8).
- **A community behind it.** Popular projects improve fast and have many users to learn from.

The trade-offs:

- **You may need more skill to set it up.** Open-source often assumes you can configure things, or can hire someone who can.

- **Support is community-based.** There may be no one to call. You rely on documentation and forums.
- **You own the maintenance.** If you run it yourself, keeping it updated and secure is on you.

The honest balance: open-source is a powerful, affordable path, especially when data control matters. But it trades money for effort and skill. If you have neither, a paid off-the-shelf tool may be the wiser start. If you have a little technical help, open-source can save you a great deal and free you from lock-in.

## Ethics and Responsibility

Choosing tools is not ethically neutral. Your choices affect your customers, your staff, and your data.

**Protect your customers' data in the choice itself.** Before any tool touches customer data, know where it goes and who can see it. A tool that trains its AI on your customers' private information without consent can break the law (see Chapter 10).

**Do not let a vendor's hype drive a decision that affects people.** If a tool will change your staff's jobs, choose it for honest reasons and involve your people, not because a salesperson created false urgency.

**Prefer tools you can audit and leave.** A tool that hides how it works, or traps your data, is a poor ethical choice as well as a poor business one. Transparency and a clean exit are signs of a vendor who respects you.

**Be honest in your own claims.** If you buy an AI tool and tell customers "our service uses AI," make sure that is true and not exaggerated marketing.

**Consider the vendor's own ethics.** Where does their data live? Do they respect privacy laws? Are they stable and honest? You are partnering with them; choose a partner you would trust with your name.

A tool is a relationship. Choose the partner as carefully as you would choose a business partner.

## Mistakes to Avoid

### 17.7 Common mistakes

**Buying on the demo.** Deciding from a polished performance on clean data. Always pilot on your own messy reality.

**Ignoring the three-year cost.** Looking at the monthly price and not the total. The subscription never stops.

**Falling for urgency.** "The price ends Friday" is a sales trick. A good tool survives a week of thought.

**No exit plan.** Signing without checking how to get your data out. That is how lock-in happens.

**Buying before defining the problem.** Getting a tool and then looking for a use. Define the problem first, then find the tool.

**Confusing popularity with fit.** A tool used by thousands may still not fit *your* workflow.

**Skipping the support test.** Not checking how the vendor behaves after the sale. Test support during the pilot.

**Over-customizing too early.** Paying for a custom build when an off-the-shelf tool would cover 80% of the need.

**Underestimating DIY maintenance.** Building it yourself and forgetting that you now have to keep it alive.

**Trusting "it handles everything."** No tool does. A vendor who claims otherwise is not telling you the truth.

**Not reading the contract.** Signing without reading the cancellation and data terms. Read before, not after.

**Choosing the cheapest sticker.** The lowest monthly price can hide the highest total cost.

## Practical Exercise

### 17.8 Exercise: an evaluation grid

Build a simple scoring grid to compare tools side by side. This turns a fuzzy choice into a clear one.

**Step 1 — List your options.** Write down two or three candidate tools (or paths: off-the-shelf, DIY, open-source).

**Step 2 — List the criteria.** Use these, or add your own:

- Fits my real problem (0–5)
- Handles my messy data well (0–5)
- Three-year total cost (lower is better — score it)
- Ease of use for my staff (0–5)
- Data privacy and control (0–5)
- Ease of exit / no lock-in (0–5)

- Support quality (0–5)
- Vendor stability (0–5)

**Step 3 — Score each tool.** Give a number for each criterion. Be honest, not hopeful.

**Step 4 — Weight what matters most.** If data privacy is critical to you, double its weight. If cost matters most, weight it higher.

**Step 5 — Add it up.** The tool with the highest weighted score is your front-runner.

**Step 6 — Test the front-runner with a pilot.** The grid narrows the field; the pilot confirms it. Do not skip the pilot.

Put the grid on one page. It makes the decision visible and defensible, and it stops a charming salesperson from overriding your judgment with charm.

## Checklist

### 17.9 Checklist to evaluate a vendor

Before signing with any AI vendor, check every box.

- ☐ **I can state the exact problem this tool solves for me.**
- ☐ **I know what the tool does NOT do, and where it fails.**
- ☐ **I have a three-year total cost, not just the monthly price.**
- ☐ **I know how pricing works** — per seat, per use, tiers, setup fees.
- ☐ **I know where my data is stored and who can see it.**
- ☐ **I know whether the vendor trains its AI on my data, and I consent or refuse.**
- ☐ **I have tested the tool on my own messy data, not the vendor's demo set.**

- ☐ **I have run a small pilot with a defined success number.**
- ☐ **I have tested the vendor's support during the trial.**
- ☐ **I have tested exporting my data out of the tool.**
- ☐ **I have read the cancellation terms before signing.**
- ☐ **I have a reference from another business in my field, or tried to get one.**
- ☐ **I checked the vendor's stability and how long they have been in business.**
- ☐ **I resisted the urgency and took time to think.**
- ☐ **I compared at least two options on the same criteria.**

If a box is empty, you have not finished evaluating. Fill it before you sign. A tool chosen with a clear head is worth far more than one bought in a dazzle.

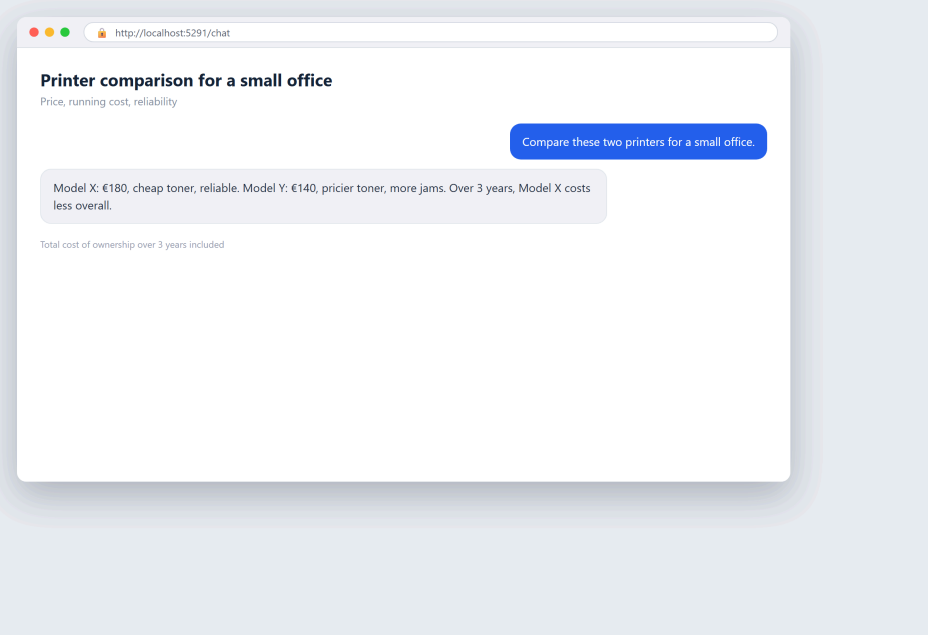
## Key Takeaways

- You are not buying a tool, you are buying a result — connect every choice back to a real problem and a real number.
- A demo is a performance on clean data; a pilot on your own messy data is the only honest test.
- The monthly price is only a fraction of the cost — always build a three-year total that includes seats, usage, setup, support, and exit.
- Open-source is a real, affordable, lock-in-free path, but it trades money for effort and skill; choose it when you have the help to run it.
- Refuse urgency: a good tool survives a week of careful thought, and a vendor who welcomes a real pilot is confident for good reason.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Compare products before buying




*A side-by-side product comparison*

**What you ask:** `Compare these two printers for a small office: price, running cost and reliability.`

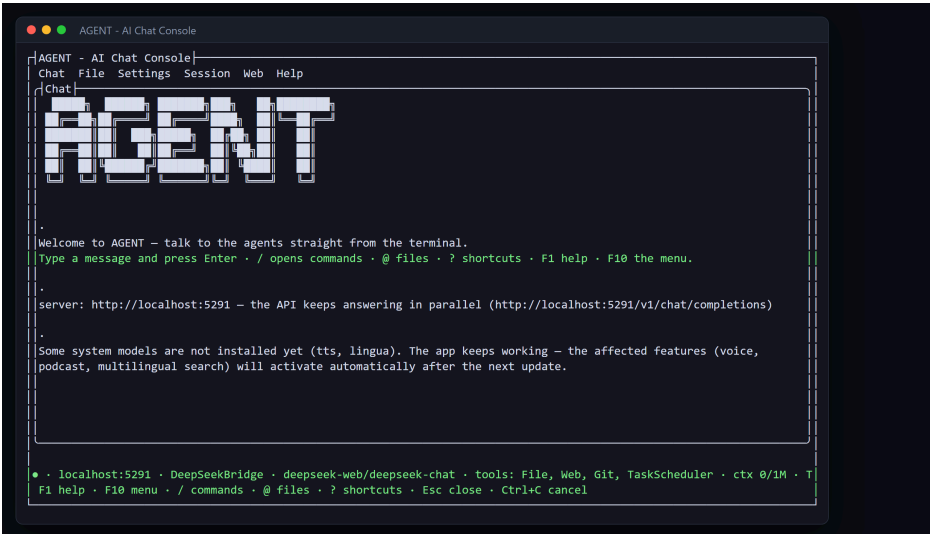
The agent researches both products and lays out a clear comparison so you can pick the better fit for your budget and use.

*Tip: Ask for the total cost of ownership, not just the sticker price.*

# Meet AgentBridge: Your Office Assistant

Everything this book describes as automation can be done with one tool: AgentBridge. It is an AI assistant for small businesses that runs on your own computer. You talk to it in plain words, and it does the work — documents, spreadsheets, email, research, and more. This short chapter shows the first steps, so the examples later in the book make sense.

## Your first conversation




*The AgentBridge terminal window, ready for your first message*

**What you ask:** `Hello! What can you help me with in my office?`

The agent answers in plain words and lists what it can do: write documents, build spreadsheets, read and send email, research the web,

make presentations and more. You keep talking to it like a person. Everything happens in this one window.

*Tip: You never have to memorise anything. Press F1 at any time for help, and the bottom bar always shows what is active.*

---

## Turn the right tools on

*The tools checklist: tick what the agent may use*

**What you ask:** `/tools`

A simple checklist appears. You move with the arrow keys and tick a tool with the Space bar. Your choice is remembered, so next time the tool is already there. The core tools (files, history, scheduling) are always on.

*Tip: In a hurry? Type the name of a ready-made set, like /tools spreadsheet-files, and it is switched on in one step.*

---

## Choose which AI answers you

*The Model setup window: pick your AI provider and model*

**What you ask:** `/model`

You see your providers and the active model. Pick another one and the whole conversation continues with it. Your documents, tools and history stay exactly where they are — only the answering AI changes.

*Tip: A local model keeps your data on your own machine. A cloud model is usually sharper. You decide per conversation.*

---

## Every command in one place

*The help screen lists every command with a one-line explanation*

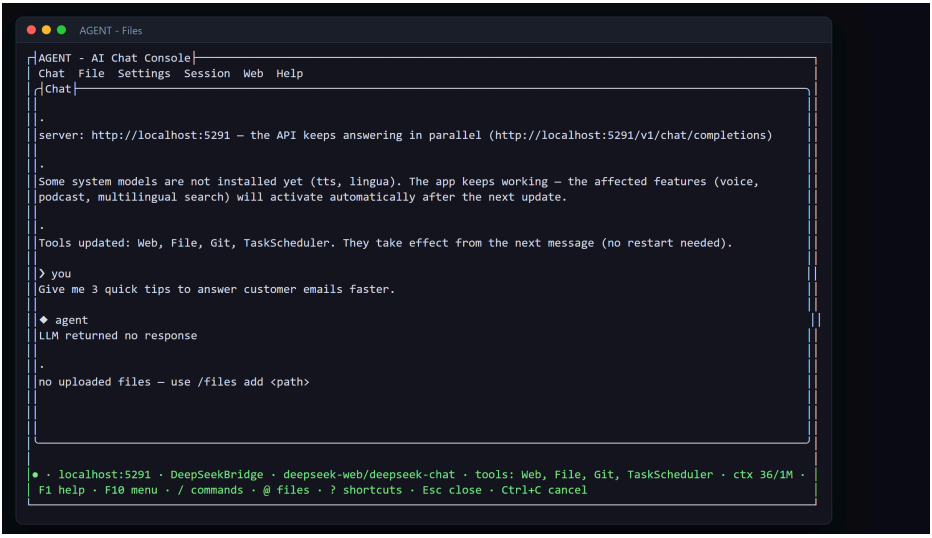
**What you ask:** `F1`

The help screen shows the commands and what each one does: /files to upload, /model to switch AI, /email to set up mail, and more. It closes with one key. No manuals, no googling.

*Tip: Typing / opens a live list that filters as you type, so you often do not need the full help screen at all.*

---

## Give the agent a document to read




*The @ palette: choose an uploaded file to attach to your message*

**What you ask:** `@` `(then pick the file)`

The file is attached to your message. The agent reads it and answers using its real contents — summarising it, checking it, or building something new from it. Nothing leaves your machine except what you choose to send to the AI.

*Tip: You can attach several files at once. The agent keeps them for the conversation, so you can ask follow-up questions.*

## Key Takeaways

- You can set a job once and let AgentBridge run it, speak to it, or reach it by phone and Telegram.

- The result is the same finished work you see in the pictures, delivered where you are.
- You stay in control: you choose the time, the place, and who may use it.

# Chapter 18 — Your First Pilot Project

## In Simple Words

A pilot project is a small, careful test of a new way of working before you commit to it fully. The word "pilot" here means the same as a pilot light — a small flame you light first to make sure the big one will work safely. You try one thing, on a small scale, for a short time, and you learn from it before you go big.

This is the single most important habit in this whole book. Most failed AI projects fail because someone went big too fast — bought a tool, rolled it out everywhere, and discovered too late that it did not fit. A pilot prevents that. It lets you fail small, cheap, and early, which is the best possible way to fail.

Think of it like testing a new recipe for your restaurant. You would not change the menu for every customer on day one. You cook the dish for a few tables, watch who smiles and who pushes the plate away, adjust the seasoning, and only then put it on the full menu. A pilot is that first few tables.

A pilot has four marks: it is **small** (one process, not the whole company), **short** (weeks, not years), **measured** (you know before you start what "success" looks like), and **reversible** (if it fails, you can undo it easily). Keep those four in mind and you will run good pilots.

The goal of your first pilot is not to change the world. It is to learn how to run a project, build confidence in your team, and prove the method on one small win. A small win creates momentum. A big failure creates fear. Choose the small win.

## A Bit of History

**The word itself comes from the sea.** A "pilot" was originally a specialist who boarded a ship to guide it through dangerous, unfamiliar waters — a harbor, a reef, a narrow channel. The pilot did not own the ship or sail the whole voyage. They came aboard for the risky part, used local knowledge to get through safely, and left. That is exactly what a pilot project does for a business: it guides you through the risky part of a change on a small scale before you sail the whole journey.

**Early 1900s: pilots in industry and science.** The idea of testing on a small scale before scaling spread into engineering and science. Before building a whole factory, you build a model. Before rolling out a process everywhere, you try it on one line. "Pilot plant" became standard in manufacturing — a small version of a factory used to work out the kinks.

**Mid-1900s: pilots in policy and medicine.** Governments tested new programs in one town before a national rollout. Medicine tested a treatment on a small group before giving it to everyone. The controlled trial — a small test with a clear measure and a comparison — became the gold standard for honest evidence.

**2000s: the startup "minimum viable product."** Software startups adopted the same logic under a new name. Instead of building a perfect product for years, they built a tiny version, put it in front of real users fast, learned, and improved. The lesson: learn from reality early rather than guess for a long time.

**2020s: pilots become essential for AI.** AI tools are powerful but unpredictable in real work. The gap between a clean demo and messy reality is wide. Running a pilot is no longer optional caution — it is the

basic way to find out whether an AI tool actually helps *your* business before you bet the whole operation on it.

The arc: from the harbor to the factory to the lab to the startup. The shape never changed — test small, learn fast, then scale.

## Curiosity

### Why "pilot" is the perfect word

It is worth pausing on the word, because it carries the whole method inside it. A maritime pilot is not the captain. The captain still owns the ship and makes the final calls. The pilot is a temporary expert who boards for the hard part, brings local knowledge, and guides the vessel through the rocks. Then the pilot leaves and the captain sails on.

A good pilot project works the same way. You, the business owner, stay the captain. The pilot is a temporary experiment that helps you get through the uncertain part of a change. You are not handing your company over to AI, and you are not locked in. You are testing the channel with a small, reversible move. If the channel is clear, you sail the whole fleet through. If there are rocks, you found out with one small boat, not the whole fleet.

That framing matters because it lowers the fear. A pilot is not a leap. It is a sounding of the water with a short pole before you step in. The oldest, safest way to cross an unknown stream.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A regional insurance broker wanted to use AI to answer customer emails faster. The tempting move was to switch it on for all 40 staff and all incoming mail at once. Instead, the owner ran a pilot.

**Small.** Only two staff, only one type of email (address-change requests), for three weeks.

**Measured.** Before starting, they measured the baseline: an address-change email took about 12 minutes to handle. The success target: under 6 minutes, with no errors in the changes made.

**Reversible.** The AI only drafted replies; a human pressed send. If it failed, they simply turned it off and nothing was broken.

In week one, the AI drafted replies that were fast but sometimes too stiff in tone. They adjusted the template. In week two, it handled 90% of the cases well. In week three, the average handling time was 4 minutes, and no address changes were wrong. The pilot passed.

Only then did they roll it out to more email types and more staff — and they ran a small pilot for each new type before trusting it. The whole rollout took three months instead of three weeks, but it worked, and the team trusted it because they had seen it proven step by step.

The lesson: the small pilot was not a delay. It was the reason the big rollout succeeded.

## How to Do It

### 18.1 Choosing a small but useful process

Your first pilot should be chosen for **learning**, not for glory. Pick a process that is small enough to control but useful enough to matter.

Good first-pilot traits:

- **Repetitive.** The same task happens over and over, so you get many chances to learn.
- **Low risk.** If the AI gets it wrong, no one is hurt and nothing is destroyed. Avoid anything involving money movement, legal filings, or health data for a first try.
- **Measurable.** You can count the time, the errors, or the volume, so you can prove change.
- **Boring enough that people welcome help.** Pick a task your team dislikes. They will be allies, not opponents.
- **Self-contained.** It does not depend on ten other things changing at once.

A great first pilot: summarizing incoming emails, drafting standard replies, extracting data from forms, or organizing documents. A bad first pilot: replacing the sales team, or automating payroll. Start where a mistake is cheap and a win is visible.

## 18.2 Drawing the “before” and the “after”

Before you touch anything, draw a clear picture of the process as it is now — the **“before.”** Then draw the process as you hope it will be — the **“after.”** This pair of pictures is your map and your target.

For the “before,” write down every step as it happens today, who does it, and roughly how long each step takes. Be honest. Include the messy parts, the re-checks, the “then I email Dave to ask.” This is your baseline.

For the “after,” mark which steps the AI will take over, which stay human, and what you expect to change. Keep it realistic. The “after” is not a fantasy; it is a plan you will test.

Put the two side by side. The gap between them is what the pilot is testing. If you cannot draw both pictures, you do not yet understand the process well enough to pilot it. Draw them first.

A simple format:

| Step | Before (who, how long) | After (who, how long) |
|---|---|---|
| Receive request | Staff reads, 2 min | AI reads and sorts, 0 min |
| Draft reply | Staff writes, 8 min | AI drafts, staff checks, 2 min |
| Send | Staff sends, 1 min | Staff sends, 1 min |

The table turns a vague hope into a concrete, checkable plan.

## 18.3 Involving the team

Do not run a pilot from your office and announce it like a decree. The people doing the work every day know things you do not. Involve them from the start, and they become your best source of truth and your strongest supporters.

How to involve them well:

- **Tell them why.** Explain the goal honestly: to remove the boring parts, not to replace them. Fear turns people against a pilot; clarity turns them toward it. (See Chapter 15 on people and culture.)
- **Ask them to map the process.** They will spot the edge cases and the hidden steps you missed.

- **Give them the power to say "this is wrong."** Make it safe to report failures. If people hide the AI's mistakes out of fear, you learn nothing.
- **Pick a champion.** One enthusiastic person on the team who helps others and feeds back honestly.
- **Celebrate their role in the win.** When the pilot succeeds, credit the team. It builds trust for the next project.

A pilot done *to* a team fails. A pilot done *with* a team works.

## 18.4 Test, fix, document

A pilot is a loop: test, fix, test again. Expect to fix things. A pilot that goes perfectly on day one is suspicious — you probably did not test hard enough.

**Test.** Run the new process on real work. Watch what happens. Collect the failures as carefully as the successes.

**Fix.** When something goes wrong, adjust — the template, the instructions, the settings, the handoff between AI and human. Then test again.

**Document.** Write down what you learned as you go. What worked, what broke, how you fixed it, what rule you now follow. This written record is gold. It becomes the instruction manual for the full rollout, and the training material for new staff. It also lets you repeat the win deliberately instead of by luck.

Keep a simple pilot log: date, what you tried, what happened, what you changed. By the end, you will have a playbook you did not have before you started.

## 18.5 Success criteria

Decide what "success" means **before** you start, in numbers, not feelings. If you decide after, you will talk yourself into whatever happened.

Good success criteria are specific and measurable:

- **Time.** "Average handling time drops from 12 minutes to under 6."
- **Quality.** "Fewer than 1 error in 50 cases."
- **Adoption.** "At least 80% of eligible cases go through the new process."
- **Satisfaction.** "The pilot team says they would keep it."

Set a **pass line** and a **fail line** for each. If you clear the pass line, the pilot succeeded. If you fall below the fail line, it failed — and that is a fine outcome, because you learned it cheaply. If you land in between, you adjust and run a little longer.

Write the criteria down and share them with the team. Everyone should know what they are aiming for and be able to see whether they hit it.

## 18.6 When to move on to the next project

A pilot ending well is not the finish line; it is a gate. Decide in advance what you do at the gate.

**If the pilot passed:** scale it gradually. Roll it out to more cases or more people, but keep a light check for the first while. Do not switch off all human review at once. Run a small pilot again for each meaningfully new case.

**If the pilot failed:** stop and learn. A failed pilot is not a disaster — it is a cheap lesson that saved you from an expensive mistake. Ask why it failed. Was the tool wrong? The process wrong? The data wrong? Fix the cause and try again, or walk away. Either way, you gained knowledge and lost little.

**If the pilot was mixed:** adjust and re-run. Most pilots land here. Tweak the design and test again.

When to move to the *next* project: when the current one is stable and running without constant firefighting, and you have documented how it works. Do not chase a new shiny thing while the last one is still a mess. Finish, stabilize, document, then move on. Momentum comes from completed projects, not started ones.

## Ethics and Responsibility

A pilot touches real work and often real customer data, so ethics is not a side note.

**Protect data during the pilot.** Even a small test must respect privacy. Do not feed customer data into a tool you have not vetted. Know where the data goes and who can see it (see Chapter 10).

**Keep a human in the loop where it matters.** For a first pilot, the AI should draft, not act. A human presses send, approves, or decides. This keeps accountability with a person.

**Be honest with your team.** Do not hide the real goal or dress up a job-cutting plan as a "pilot." Trust is hard to build and easy to break. Tell the truth about what you are testing and why.

**Do not let a pilot become a way to avoid responsibility.** If the AI makes a mistake that harms a customer, a person is still accountable. Design the pilot so a human checks the output that matters.

**Respect the people whose work you are changing.** Involve them, listen to them, and credit them. A pilot that steamrolls the team teaches the whole company to fear the next change.

Run the pilot the way you would want it run if you were the one doing the work.

## Mistakes to Avoid

**Going too big too fast.** Rolling out everywhere before testing. This is the classic failure. Start small.

**No baseline.** Not measuring the "before," so you cannot prove the "after." Measure first.

**Vague success criteria.** "It seems better" is not a result. Set numbers before you start.

**Testing only the easy cases.** If you only test clean, simple inputs, you will be surprised by the messy ones. Test the hard cases on purpose.

**Excluding the team.** Running it from above and announcing it. The team will resist what they did not help build.

**Hiding failures.** If people fear reporting mistakes, you learn nothing. Make it safe to report.

**No documentation.** Learning by doing but writing nothing down, so the win cannot be repeated or taught.

**Skipping the human check.** Letting the AI act alone on something that matters. Keep a human on the send button.

**Moving on too soon.** Chasing the next project before the last one is stable and documented.

**Treating a failed pilot as a disaster.** A failed pilot is a cheap lesson. Punishing failure teaches people to hide failure.

**Confusing the pilot with the rollout.** A pilot proves it *can* work; the rollout makes it work *everywhere*. Do not skip steps between them.

## Practical Exercise

### 18.7 Exercise: the pilot plan

Write a one-page pilot plan for one small process in your business. Fill in every line.

**1. The process.** Name the one small, repetitive, low-risk task you will pilot. (Example: drafting replies to address-change emails.)

**2. The "before."** Write the current steps, who does each, and the time each takes. This is your baseline. If you do not know the times, measure them this week first.

**3. The "after."** Write the new steps. Mark which the AI does, which stay human. Keep a human on any step that matters.

**4. The scope.** Who is involved (keep it to one or two people), what cases are included, and how long the pilot runs (two to four weeks).

**5. The success criteria.** Write the pass line and the fail line for time, quality, and adoption. Numbers, not feelings.

**6. The reversibility.** Write exactly how you turn it off and return to the old way if it fails. Confirm nothing breaks.

**7. The team plan.** How you will explain the "why," who your champion is, and how people report problems safely.

**8. The log.** Set up a simple log: date, what you tried, what happened, what you changed.

**9. The gate.** Write what you will do if it passes (scale gradually), fails (stop and learn), or is mixed (adjust and re-run).

Put all nine on one page. That page is your pilot. Run it, fill in the log, and at the end you will have something you did not have before: proof, a playbook, and the confidence to do the next one.

## Checklist

### 18.8 Pilot checklist

Before you start a pilot, and before you finish it, check every box.

- ☐ **The process is small, repetitive, low-risk, and measurable.**
- ☐ **I have drawn the "before" with real times (the baseline).**
- ☐ **I have drawn the "after" with AI and human steps marked.**
- ☐ **A human stays in the loop on any step that matters.**
- ☐ **The scope is small: one or two people, one case type.**
- ☐ **The pilot has a fixed, short time frame (2–4 weeks).**

- ☐ **I set success criteria as numbers, with a pass line and a fail line.**
- ☐ **I shared the success criteria with the team.**
- ☐ **I explained the "why" honestly to the team.**
- ☐ **I have a champion on the team.**
- ☐ **It is safe for the team to report failures.**
- ☐ **I have a clear way to reverse the pilot if it fails.**
- ☐ **I am testing the messy/hard cases, not only the easy ones.**
- ☐ **I am keeping a pilot log (what tried, what happened, what changed).**
- ☐ **I documented the playbook so the win can be repeated and taught.**
- ☐ **I decided in advance what to do at the gate: pass, fail, or mixed.**

If a box is empty, the pilot is not ready. Fill it first. A pilot you run well is worth more than a rollout you hope will work.

## Key Takeaways

- A pilot is a small, short, measured, reversible test — the best way to fail is small and early, before you bet the whole business.
- Measure the "before" and draw the "after" before you start; without a baseline, you can never prove the change.
- Involve the team and make it safe to report failures — a pilot done *with* people works, one done *to* them fails.
- Set success criteria as numbers with a pass line and a fail line before you begin, so you cannot talk yourself into a result.
- Finish, stabilize, and document before chasing the next project — momentum comes from completed pilots, not started ones.

# Chapter 19 — Connecting AI to Systems You Already Use

## In Simple Words

Your business already runs on software: the system that tracks your customers, the email you answer, the spreadsheets you live in, the accounting program you invoice from. AI is most useful when it plugs into these existing systems instead of sitting off in a corner doing nothing. This chapter is about that connection — how AI talks to the tools you already have.

The key idea is the **integration**: a link that lets one piece of software pass information to another automatically. When your AI can read your customer list, draft a reply in your inbox, and write a note back into your records, it becomes a real helper. When it cannot connect to anything, it is just a clever chat window you have to copy and paste into.

A good analogy is plumbing. A new water filter is useless if it is not connected to your pipes. The value comes from the connection, not the filter alone. Integrations are the pipes that let AI flow into your daily work. Your job is to understand which pipes exist, which you can connect yourself, and when you need a plumber.

This chapter looks at the common systems AI connects to — management software, CRM, email, spreadsheets. It gives examples of simple automations you can picture. It explains what an **API** is in plain words (it is simpler than it sounds). It tells you when you can do it yourself and when to call a technician. And it warns you about the

traps: dependencies that block you, and the maintenance that every connection needs.

The promise: connected AI saves real time every day. The caution: a bad connection can break things or trap you. Do it with a map and a plan.

## A Bit of History

**1960s–1970s: programs that talk to each other.** Early business software had to exchange data — an inventory system feeding an accounting system. Engineers built the first links between programs, often by hand, sharing files in agreed formats. Integration was born out of pure necessity.

**1990s: enterprise integration becomes a profession.** As companies ran many systems at once, connecting them became a whole field. Tools with long names — middleware, enterprise application integration — tried to be the central hub that linked everything. It was powerful, expensive, and usually needed specialists.

**2000s: the API economy.** Software began exposing clean, documented doors for other software to use. These doors are called APIs. Suddenly a small company could connect to big services — maps, payments, messaging — without building them. A whole economy of connected software appeared.

**2010s: no-code and the citizen integrator.** Tools like Zapier and Make let non-programmers connect popular apps by pointing and clicking. You could say “when a new lead lands in my form, add it to my CRM and email me” without writing code. The “citizen integrator” — a business person who builds their own connections — became real.

**2020s: AI as the new connector.** AI tools gained the ability to read, write, and act across systems through their own APIs. Now the AI can be the one doing the connecting: reading your inbox, updating your records, drafting your replies. The plumbing got smarter, and the need to understand it grew just as fast.

The arc: from hand-built file links to a world where software expects to be connected. The connection is no longer a luxury; it is where the value lives.

## Curiosity

### The memo that connected the world

In 2002, the CEO of Amazon, Jeff Bezos, sent a now-famous internal memo to his engineering teams. The details are widely reported in tech history. The core order was blunt: from now on, every team must share its data and functions through a clean, documented interface — an API — and nothing else. No direct access to another team's database. No back-door shortcuts. If a team wanted something from another team, it had to ask through the published interface, or build it itself.

The reason was not tidiness. It was to make Amazon fast and flexible. When every part of the company could be reached through a clean door, teams could change their own systems without breaking everyone else, and new services could be built quickly on top of old ones.

That discipline is widely credited as a foundation of what became **Amazon Web Services (AWS)** — the cloud platform that now powers a huge share of the internet. A rule about how software talks to software inside one company turned into one of the largest technology businesses on earth.

The lesson for a small business is the same in miniature: **clean, documented connections make you flexible; messy shortcuts make you fragile.** When you connect AI to your systems, build clean links, not back-door hacks. The tidy pipe today is the freedom you will thank yourself for tomorrow.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A small property-management firm ran on three systems that did not talk to each other: a CRM holding tenant contacts, a shared inbox for maintenance requests, and a spreadsheet tracking repair jobs. Every request meant copying details by hand from the inbox into the spreadsheet, then into the CRM. Staff spent hours a day on copy-paste, and details slipped through the cracks.

They connected the three with a simple automation. When a maintenance email arrived, an AI read it, pulled out the tenant name, the property, and the problem, and created a row in the spreadsheet automatically. It also drafted a reply to the tenant confirming the request. A human checked the draft and pressed send. The CRM note was added by the same automation.

The result was not magic — it was plumbing. The hours of copy-paste mostly disappeared. Fewer requests were lost, because the same automation flagged anything it could not read clearly for a human. The firm did not replace staff; it removed the boring part of their day so they could handle more properties without more people.

The lesson: the win came from connecting systems that already existed, not from buying something new and dramatic.

# How to Do It

## 19.1 Management software, CRM, email, spreadsheets

These are the four systems most businesses already run, and the four AI connects to most often.

- **Management software (ERP).** A system that runs the core of your business — inventory, orders, production, finance. ERP stands for Enterprise Resource Planning. AI can read reports from it, flag unusual numbers, or draft summaries.
- **CRM.** Stands for **Customer Relationship Management** — the system that holds your customers, leads, and every interaction with them. AI can draft replies to leads, log calls, and pull a customer's history to help you answer faster.
- **Email.** The inbox is where most small-business work lives. AI can sort, summarize, and draft replies here. This is often the single most valuable connection.
- **Spreadsheets.** The universal tool. AI can fill them, read them, and update them from other sources.

Start where the pain is highest. For most small businesses, that is email and the CRM. Connecting AI to those two gives the biggest daily relief. Do not try to connect everything at once — pick the one that hurts most and start there.

## 19.2 Examples of simple automations

Concrete examples help you picture what is possible. Each of these is a small, common connection:

- **Inbox to CRM.** A new inquiry email automatically creates a lead in your CRM, with the sender's details filled in.
- **Form to spreadsheet.** A customer fills a web form; the answers land in a spreadsheet row automatically, and the AI tags the request by type.
- **Email to draft reply.** AI reads a standard request and writes a draft reply in your inbox; you check and send.
- **Document to record.** AI reads an invoice PDF and writes the amount, date, and vendor into your accounting system.
- **Meeting to notes.** AI turns a recorded meeting into a summary and action items, then files them where your team can see them.
- **Ticket to alert.** AI reads incoming support messages and flags the angry or urgent ones to a human first.

Notice the pattern: AI reads from one place, does something useful, and writes to another, with a human check where it matters. That is the shape of almost every good automation.

## 19.3 When you need a technician

You can do a surprising amount yourself with no-code tools. But some jobs need a professional. Know which is which.

**You can likely do it yourself** when: the apps are popular (so no-code connectors exist), the data is simple, the automation is small, and a mistake is cheap. Point-and-click tools cover a lot here.

**Call a technician when:**

- **The connection touches money, legal records, or sensitive data.** A mistake here is expensive or dangerous.
- **The systems are old or custom-built** and have no ready-made connector.

- **You need a reliable, always-on link** that must not fail.
- **Security is involved** — connecting to customer data means getting access control right (see Chapter 20).
- **You do not understand what you are connecting.** Never connect what you cannot explain.

A technician is not an admission of failure. It is the right call for the risky or complex parts, the same way you call a plumber for the main line and fix the tap yourself.

## 19.4 APIs and integrations explained simply

An **API** sounds technical, but the idea is simple. An API is a **menu that one piece of software offers to another.** It is a list of things you are allowed to ask it to do, and how to ask.

Think of a restaurant kitchen. You do not walk in and start cooking. You look at the menu, order from what is offered, and the kitchen brings it. The menu is the API. It tells you what you can request ("get this customer's details," "add a new row," "send this email") and how to request it. You cannot order something off the menu, and you never touch the kitchen directly.

Why this matters for AI: when a system has an API, an AI can use that menu to read and write data safely and predictably. When a system has no API, connecting to it is hard or impossible. So when you choose software, a good question is: **does it have an API?** If yes, AI can likely connect to it. If no, you may be stuck.

An **integration** is the connection you build using one or more APIs — the act of wiring the menu of one system to the needs of another. The API is the door; the integration is the hallway you build through it.

Two plain terms you will hear:

- **Read access** — the AI can look at data but not change it. Safer.
- **Write access** — the AI can change data. More powerful, more risky. Give write access only where you need it.

## 19.5 Avoiding dependencies and blocks

Every connection creates a **dependency** — one thing now relies on another. Dependencies are normal, but too many, or the wrong ones, can block you.

Watch for these traps:

- **A single point of failure.** If one connection breaks and your whole workflow stops, that is a fragile single point. Have a fallback: a way to do it by hand if the link dies.
- **A chain of dependencies.** If A needs B needs C needs D, one broken link stops everything. Keep chains short.
- **A dependency on a tool that can vanish.** If you connect to a small service that shuts down, your automation dies. Prefer stable, established tools.
- **Lock-in through integration.** If all your systems are wired together in a way only one vendor understands, leaving becomes hard. Keep connections clean and documented so you can rewire later. (Lock-in is covered in Chapter 9.)
- **Undocumented connections.** A link nobody knows how it works is a time bomb. Document every connection.

The rule: **build clean, short, documented connections with a manual fallback.** The Amazon memo lesson again — clean doors, no back-door hacks.

## 19.6 Maintenance and updates

A connection is not "set and forget." It is a living thing that needs tending.

- **Systems change.** Your CRM updates, your email provider changes a format, an API gets a new version. When one side changes, the connection can break.
- **Silent failures.** A connection can stop working quietly, and data stops flowing without anyone noticing. Check that data is actually arriving.
- **New cases appear.** The automation handled the common cases; a new kind of request comes along and it does not know what to do. Review what it is missing.
- **Security updates.** Connections to data need their access checked over time, especially when staff join or leave.

Plan for maintenance: assign someone to watch the connections, check them regularly, and fix breaks fast. Build a simple alert — if no data flows for a day, someone should know. A little tending keeps the plumbing from flooding.

# Ethics and Responsibility

Connecting AI to your systems means connecting it to real people's data. That carries responsibility.

**Connect with consent and care.** If AI will read customer emails or records, know what you are connecting and whether it is allowed. Respect privacy rules (see Chapter 10).

**Least access.** Give the AI only the access it needs. If it only needs to read, do not give it write. If it only needs one folder, do not give it the whole drive. This limits damage if something goes wrong.

**Keep a human on the send.** For anything that reaches a customer, a human should check before it goes out. Automation that acts alone on customer-facing messages can cause real harm.

**Document for accountability.** When something goes wrong, you need to know what the automation did and why. A documented connection is an accountable one.

**Do not connect what you cannot explain.** If you cannot say in plain words what a connection does to whose data, you should not build it. Complexity you do not understand is risk you cannot control.

Build connections the way you would want your own data handled.

## Mistakes to Avoid

**Connecting everything at once.** Trying to wire every system on day one. Pick the one that hurts most and start there.

**No map.** Connecting without knowing what you have and how it links. Draw the map first.

**Back-door hacks.** Quick, undocumented shortcuts that break later and trap you. Build clean doors instead.

**Too much access.** Giving the AI write access to everything when it only needs to read one thing. Use least access.

**No fallback.** A connection that, when it breaks, stops the whole workflow with no manual plan. Always have a fallback.

**Ignoring silent failures.** Assuming the connection works because no one complained. Check that data actually arrives.

**No maintenance plan.** Wiring it up and walking away. Connections need tending.

**Connecting to a tool that can vanish.** Building on a shaky service that may disappear. Prefer stable tools.

**Letting AI act alone on customer messages.** No human check on what reaches a customer. Keep a human on the send.

**Undocumented links.** Nobody knows how the connection works. Document every one.

**Forgetting staff changes.** Not updating access when people join or leave. Review access regularly.

## Practical Exercise

### 19.7 Exercise: a map of your company systems

You cannot connect what you cannot see. Draw a map of the systems your business runs on.

**Step 1 — List every system.** Write down every piece of software your business uses daily: CRM, email, accounting, spreadsheets, inventory, scheduling, website forms, chat tools. Leave nothing out.

**Step 2 — For each, note four things:**

- **What it holds** (what data lives there).
- **Who uses it** (which people or roles).

- **Does it have an API?** (Check the vendor's site or ask; mark yes / no / unknown.)
- **How sensitive is the data?** (Low / medium / high.)

**Step 3 — Draw the current links.** On paper, draw lines between systems that already pass data today, even if a human carries it by copy-paste. Mark which links are manual.

**Step 4 — Spot the pain.** Circle the manual links that eat the most time or cause the most errors. These are your best automation candidates.

**Step 5 — Mark the risk.** For each candidate, note the sensitivity. High-sensitivity links need a technician and a human check; low-sensitivity ones you can try yourself.

**Step 6 — Pick one.** Choose the single highest-pain, lowest-risk link as your first integration. Pilot it (see Chapter 18).

Keep the map on one page. Update it as systems change. The map is your plan and your defense against connecting blindly.

## Checklist

### 19.8 Integration checklist

Before you connect AI to a system, and after, check every box.

- ☐ **I have a map of all my systems and what data each holds.**
- ☐ **I know which systems have an API and which do not.**
- ☐ **I know how sensitive the data in each system is.**
- ☐ **I picked the highest-pain, lowest-risk link to connect first.**
- ☐ **I used a clean, documented connection, not a back-door hack.**

- ☐ **I gave the AI only the access it needs (least access).**
- ☐ **I know the difference between read and write access and used write only where needed.**
- ☐ **A human checks anything that reaches a customer.**
- ☐ **I have a manual fallback if the connection breaks.**
- ☐ **I checked the connection is stable and not a single point of failure.**
- ☐ **I built a way to detect silent failures (alert if no data flows).**
- ☐ **I documented how the connection works and who maintains it.**
- ☐ **I assigned someone to maintain and watch the connection.**
- ☐ **I reviewed access for current staff and will review it when staff change.**
- ☐ **I ran the connection as a pilot before trusting it fully.**

If a box is empty, the connection is not ready. Fill it first. A connected AI is a powerful helper; a blindly connected one is a liability.

## Key Takeaways

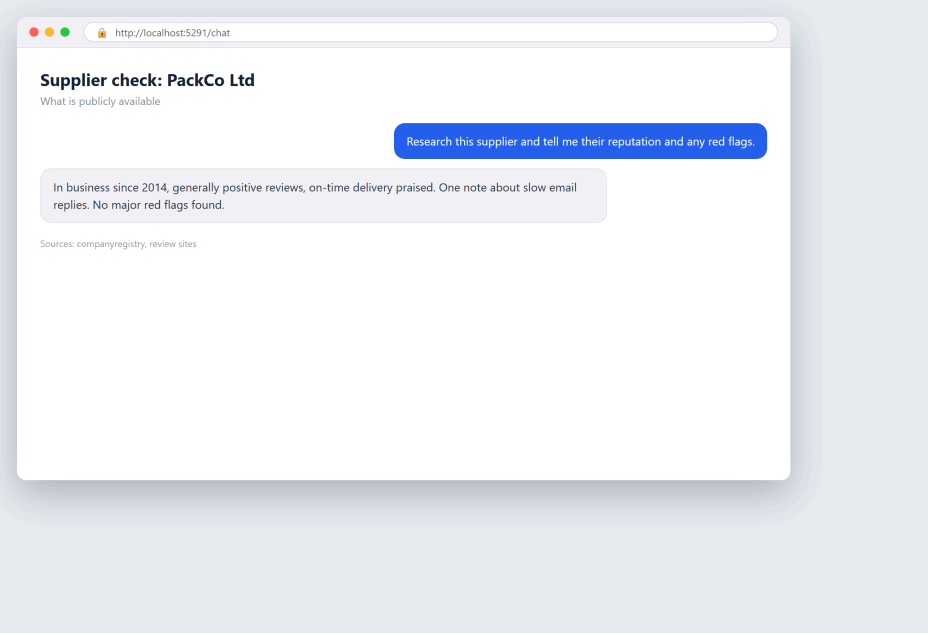
- AI's value comes from connecting to the systems you already run — the connection is the plumbing, and the value flows through it.
- An API is simply a menu one software offers another; if a system has an API, AI can likely connect to it, so ask "does it have an API?" when choosing software.
- Use least access — give the AI only what it needs, prefer read over write, and keep a human on anything that reaches a customer.
- Build clean, short, documented connections with a manual fallback; back-door hacks and silent failures are the traps that break you.

- Connections are living things that need maintenance, monitoring, and access reviews — set them up and never walk away.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Check a supplier before you commit




*A due-diligence summary of a supplier*

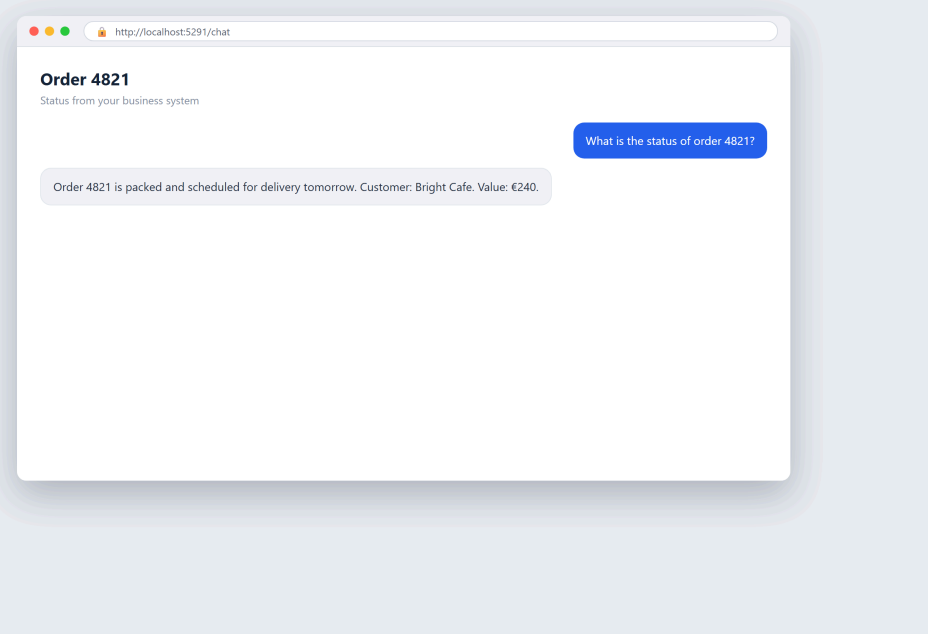
**What you ask:** `Research this supplier and tell me their reputation, how long they have been around, and any red flags.`

The agent gathers what is publicly available and gives you a balanced picture with sources, so you decide with your eyes open.

*Tip: Ask it to list what it could not find, so you know where to dig further.*

---

## Check an order in your system




*An order status pulled from the business system*

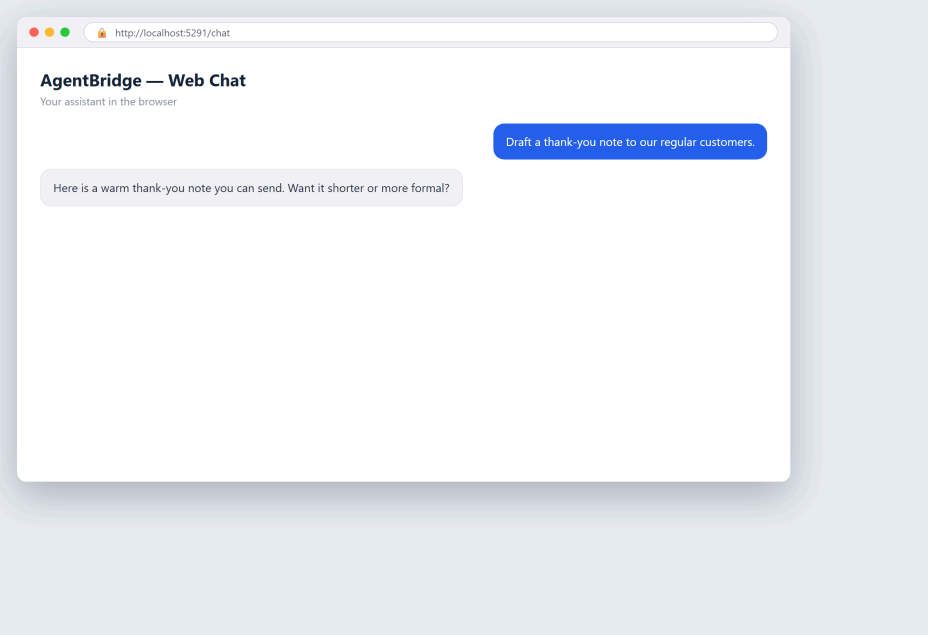
**What you ask:** `What is the status of order 4821?`

The agent looks up the order in your connected system and tells you its status in plain words — no menu hunting.

*Tip: This works once your business system is connected. See the book's connecting chapter.*

---

## The same assistant in your browser




*The AgentBridge web chat in a browser window*

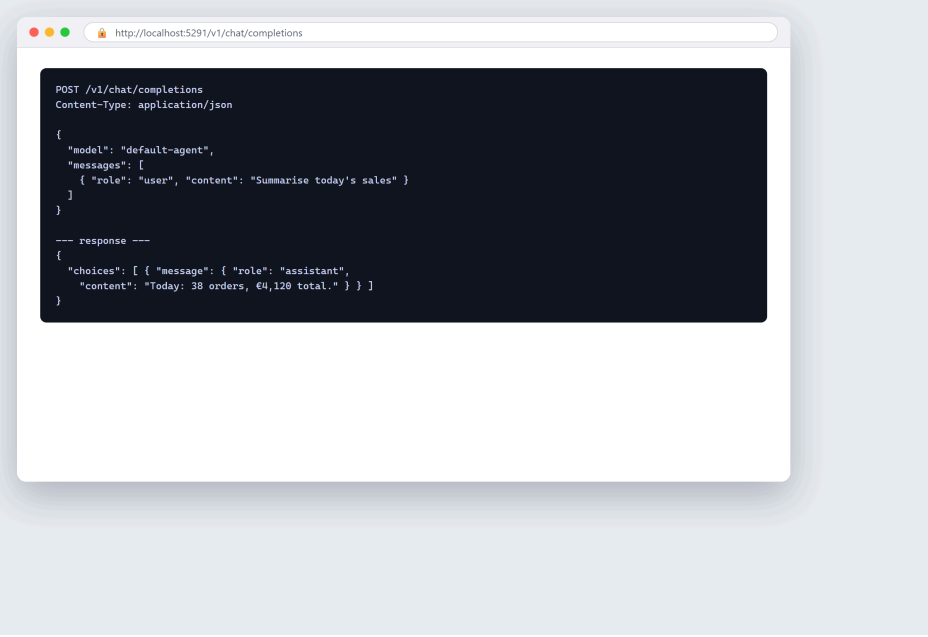
**What you ask:** `(browser) Draft a thank-you note to our regular customers.`

Open the web address and chat in the browser. Same tools, same documents, same memory — just a different window.

*Tip: Handy when you are on a different machine but still want your own assistant.*

---

## Plug it into your own tools




*The HTTP API lets other programs use the assistant*

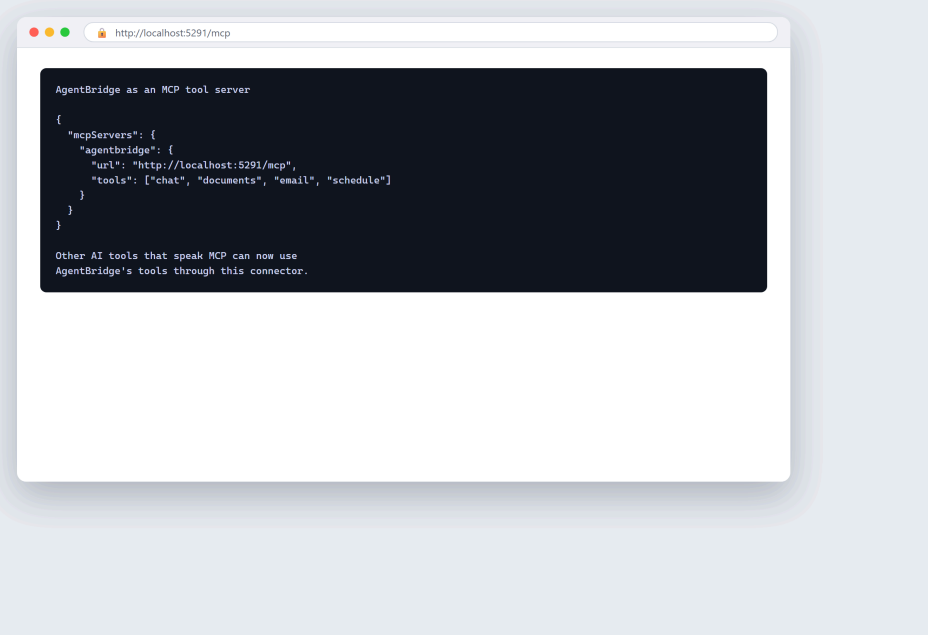
**What you ask:** `POST /v1/chat/completions { "model": "default-agent", "messages": [...] }`

Your own programs can call the assistant through a standard web API, the same way they would call any online service. One integration, many uses.

*Tip: This is for the technical step — the book's connecting chapter walks through it.*

---

## Connect other AI tools to it




*AgentBridge connected to other AI tools via MCP*

**What you ask:** `(MCP) Connect AgentBridge as a tool server.`

Other AI applications can use AgentBridge's tools through the MCP standard, so your whole toolkit works together.

*Tip: MCP is for connecting tools; for everyday use the chat is all you need.*

# Chapter 20 — Implementing AI Securely

## In Simple Words

When you bring AI into your business, you are letting a new worker handle your data. Like any worker, that new worker must be trusted, watched, and bounded. This chapter is about doing that — how to use AI without leaking your customers' data, your finances, or your secrets.

Security is not a product you buy. It is a set of habits you build. The goal is simple: **design your automations so that even when something goes wrong, your data stays protected.** That idea has a name — **security by design** — and it means you think about safety at the start, not as an afterthought after a breach.

A good analogy is a house. You do not leave the front door wide open because "nothing bad has happened yet." You lock doors, you limit which rooms guests can enter, you put a light on a timer, and you have a plan if you come home to an open window. AI security is the same layered thinking: control who gets in, protect what is inside, watch what happens, and know what to do if something breaks.

This chapter covers the core habits: designing safe automations, controlling who can do what, encrypting data so it is unreadable to thieves, watching and logging what the AI does, planning for the day something goes wrong, and training your staff — because the human factor is the first line of defense.

One honest truth up front: no system is perfectly safe. The goal is not perfection; it is to make your business a hard target and a soft target

never. Most attacks go for the easy target. Good, simple security moves you from easy to hard.

## A Bit of History

**1970s–1980s: security starts at the perimeter.** Early computer security focused on keeping outsiders out — firewalls, passwords, locked server rooms. The model was a castle: strong walls, everything inside is safe. This worked until people needed to connect and share, which cracked the walls.

**1990s: the perimeter dissolves.** The internet and email meant data started moving. The castle model broke. Viruses and intrusions came through the very connections that made business possible. Security had to follow the data, not just guard the door.

**2000s: "least privilege" and defense in depth.** Security thinking shifted inward. The principle of **least privilege** — give every user and program only the access it absolutely needs, nothing more — became central. So did **defense in depth**: many overlapping layers, so if one fails, another catches the threat.

**2010s: "assume breach."** Experts realized you cannot always keep attackers out. The new mindset was to *assume* an intruder may already be inside, and focus on limiting what they can reach and spotting them fast. Logging, monitoring, and fast response became as important as the walls.

**2020s: AI adds a new attack surface.** AI brings new ways to be harmed — tricks that fool the AI itself, data poisoning, and the risk of feeding sensitive data into tools you do not control. (These specific threats are covered in Chapter 6.) The old principles — least privilege,

defense in depth, assume breach — still apply, now applied to a new kind of worker.

The arc: from a castle with one wall to many layers that assume the wall may be breached. The lesson never changed: protect the data itself, not just the door.

## Curiosity

### 20.7 The company that caught an attack in a log nobody was watching

*The specific company in this story is illustrative — a realistic composite, not a named firm. The pattern behind it is real and well documented.*

Picture a mid-sized company that had a logging system — a record of everything happening on its network. The logs ran constantly, faithfully recording every login, every file accessed, every unusual request. For months, nobody read them. They were there, like a security camera no one watches.

Then one day a new technician, cleaning up, glanced at the logs and saw something odd: a program was reaching out to an outside address at strange hours, pulling data slowly and steadily. It had been doing it for weeks. Nobody had noticed because nobody was looking. When they finally looked, they found an intruder that had been quietly inside far longer than anyone imagined.

The uncomfortable lesson is not that the company was careless. It is that **the evidence was there the whole time — and it was ignored.** The company had invested in the camera but not in watching it.

This pattern is not fiction. IBM's annual *Cost of a Data Breach* report has found, year after year, that breaches take a long time to detect — on the order of **two hundred days** on average in recent editions — and that a large share are first noticed by someone outside the company, not by the company's own monitoring. The logs and signals often exist. What is missing is a person looking at them.

For your AI automations, the lesson is direct: **a log nobody reads is not security.** If you collect records of what your AI does, assign someone to actually look, and set alerts so the important signals find a human instead of waiting to be discovered.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A small online retailer wanted AI to draft customer support emails. The easy, risky way was to give the AI full access to the whole customer database — names, addresses, payment history — and let it read whatever it needed. The owner paused and did it the safer way instead.

They gave the AI **read-only** access to just the order details it needed to answer a shipping question — not payment records, not the full customer file. They set it so the AI could draft but a human pressed send. They turned on a log that recorded every order the AI touched. They told the support team what the AI could and could not see, and asked them to report anything strange.

Three weeks in, the log showed the AI had been asked — by a clever customer — to reveal another customer's address by pretending to be that person. The AI had refused, because it was not given access to that

data in the first place. The attempt failed harmlessly, and the log captured it so the team learned the trick existed.

Had the owner taken the easy route and given the AI the whole database, that same trick might have worked. The safe design — least access, human on the send, a watched log — turned a potential breach into a non-event. That is what security by design looks like in practice.

## How to Do It

### 20.1 Security-by-design principles: how to design an automation that does not expose data

Security by design means you build safety into the automation from the first sketch. Ask these questions before you build anything:

- **What data does this automation actually need?** Use the least you can. If it only needs an order number, do not give it the whole customer file.
- **What is the worst that could happen if it leaks or is tricked?** Picture the failure. Design so the worst case is small.
- **Where does the data go?** Know every system the data touches, especially outside your control. Do not feed sensitive data into a tool you cannot account for.
- **Can a human stop it?** Build in a kill switch — a way to turn the automation off instantly if it misbehaves.
- **Does it fail safe?** If something breaks, the automation should stop and protect, not open up. A door that locks when the power fails is "fail-safe."

The core principle is **least privilege**: minimum access, minimum reach, minimum data. An automation that never had access to the

sensitive data cannot leak it, no matter how clever the attack. Design the access out, and the risk goes with it.

## 20.2 Access control: who can do what

Access control is simply deciding who is allowed to do what, and enforcing it. It is the lock on each room, not just the front door.

Three plain rules:

- **Give each person and each tool only the access their job needs.** A support tool answering shipping questions needs order data, not payroll data. A junior staff member should not have the same access as the owner.
- **Separate duties.** The person who sets up the AI should not be the only one who can approve its actions, and the only one who can see the logs. Spread the keys so no single compromised account can do everything.
- **Review access when people change.** When staff join, move, or leave, update their access the same day. Old access that lingers is one of the most common ways breaches happen.

Use the tools your systems already have — user roles, permissions, and login controls. Turn on **two-factor authentication** (a second step beyond a password, like a code on your phone) wherever you can. It stops most stolen-password attacks.

## 20.3 Encryption: protecting data in transit and at rest

**Encryption** means scrambling data so that only someone with the right key can read it. To a thief, encrypted data looks like nonsense. It protects data in two states:

- **In transit** — data moving between places, like an email or a file upload. Encryption here means a thief tapping the wire sees garbage. Look for "https" and secure connections.
- **At rest** — data sitting still, like files on a disk or records in a database. Encryption here means a thief who steals the hard drive still cannot read it.

For a small business, the practical steps are simple: use tools that encrypt by default (most reputable services do), check that connections are secure (https), and ask any vendor directly, "Is my data encrypted in transit and at rest?" A serious vendor answers yes and explains how. If they cannot, that is a warning.

One caution: encryption protects data from outsiders. It does not protect from someone who has the key and misuses it. That is why access control (20.2) and encryption work together — encryption hides the data, access control limits who holds the key.

## 20.4 Monitoring and logging: always knowing what the AI is doing

A **log** is a record of what happened: who did what, when, to what data. **Monitoring** is watching those logs, live or regularly, to spot trouble.

Why it matters for AI: an AI automation acts on your data all day. Without a log, you cannot tell what it touched, what it changed, or whether it was tricked. With a log, you have a record you can check and a trail an attacker cannot hide.

Make logging useful:

- **Log the important actions** — what data the AI read, what it wrote, what it sent, and any request it refused.

- **Set alerts** so the important signals find a human. Do not make someone stare at a screen; make the system shout when something unusual happens.
- **Actually look.** As the curiosity story shows, a log nobody reads is not security. Assign someone to review, even briefly, on a regular schedule.
- **Keep the logs safe** so an intruder cannot erase them.

Monitoring turns a silent system into a visible one. You cannot protect what you cannot see.

## 20.5 Incident-response plans: what to do if something goes wrong

Something will eventually go wrong. The question is not "if" but "when," and how ready you are. An **incident-response plan** is a written plan for the bad day, made on a calm day so you are not improvising in panic.

A simple plan has five steps:

1. **Detect.** How will you know something is wrong? (An alert, a staff report, a customer complaint.)
2. **Contain.** How do you stop the damage right now? (Turn off the automation, cut the connection, lock the account.)
3. **Assess.** What was exposed or harmed? Check the logs.
4. **Fix and recover.** Repair the cause and restore normal, safe operation.
5. **Report and learn.** Tell the people who need to know — and know who you are legally required to tell. (Privacy laws like the GDPR have reporting duties; see Chapter 10.) Then write down what happened and how to prevent it next time.

Write the plan on one page. Name who does what. Keep emergency contacts ready. Run a quick drill once a year so everyone knows their part. A rehearsed plan turns a crisis into a manageable event.

## 20.6 Staff training: the human factor is the first defense

The strongest lock can be undone by one careless person. Your staff are both your biggest risk and your best defense. Training turns them from the weak point into the first line.

What to teach, in plain terms:

- **Recognize tricks.** Attackers fool people with fake emails, urgent requests, and "I'm from IT, give me your password" calls. Teach staff to spot and doubt them.
- **Handle data with care.** Who gets to see what, and why. Never share customer data outside approved channels.
- **Report, do not hide.** Make it safe and expected to report a mistake or a strange request. A staff member who reports a suspicious email early saves the company. One who hides it lets a small problem grow.
- **Know the AI's limits.** Staff should know what the AI can and cannot see, and never ask it to do something outside its safe bounds.
- **Password and login hygiene.** Strong passwords, two-factor authentication, no sharing of logins.

Keep training short, regular, and practical — not a once-a-year lecture. A five-minute monthly reminder and a clear reporting channel do more than a long annual course nobody remembers.

## Ethics and Responsibility

Security is an ethical duty, not just a technical one.

**Protect the people whose data you hold.** Customers, employees, and partners trusted you with their information. A breach harms real people. Treating that data safely is a matter of honesty and care.

**Report honestly when things go wrong.** If you must notify customers or regulators, do it promptly and truthfully. Covering up a breach is worse than the breach itself, both ethically and legally.

**Do not use security as an excuse to hide.** "We are secure" should never mean "you cannot check." Accountability and transparency belong together.

**Balance safety with usability.** Security that is so heavy nobody can work gets bypassed, and bypassed security is no security at all. Make the safe path the easy path.

**Train with respect, not blame.** When someone makes a mistake, teach, do not punish. A culture of blame hides mistakes; a culture of learning surfaces them early.

Secure your data the way you would want your own protected.

## Mistakes to Avoid

**Too much access.** Giving the AI or staff more access than needed. Use least privilege.

**No human on the send.** Letting AI act alone on customer-facing messages. Keep a human check.

**Feeding sensitive data to unvetted tools.** Pasting customer data into a tool you cannot account for. Know where data goes.

**A log nobody reads.** Collecting records and never looking. Assign someone and set alerts.

**No incident plan.** Improvising on the bad day. Write the plan on a calm day.

**Sharing logins.** One account shared by many people destroys accountability. Give each person their own.

**No two-factor authentication.** Leaving accounts open to stolen passwords. Turn on 2FA everywhere.

**Ignoring staff changes.** Old access lingering after someone leaves. Update access the day roles change.

**Blaming instead of training.** Punishing mistakes so people hide them. Teach and encourage reporting.

**Assuming the vendor is secure.** Trusting without asking. Ask vendors about encryption and access.

**Security as an afterthought.** Building the automation first and thinking about safety later. Design safety in from the start.

**No kill switch.** No way to stop the automation fast when it misbehaves. Build the off switch.

## Practical Exercise

## 20.8 Exercise: write your AI security plan

Write a one-page security plan for one AI automation you use or plan to use. Fill every line.

**1. The automation.** Name it and what data it touches.

**2. Least access.** List exactly what data it needs. Cut everything else. Write the access you will actually grant.

**3. Human check.** Where is the human in the loop? What can the AI do alone, and what needs a person to approve?

**4. Data location.** Where does the data go? List every system, especially outside your control. Note any you cannot account for and remove them.

**5. Encryption.** Is the data encrypted in transit and at rest? Check with each vendor and write the answer.

**6. Access control.** Who can do what? List roles. Note where you will turn on two-factor authentication.

**7. Logging.** What will you log? Who will look, and how often? What will trigger an alert?

**8. Kill switch.** How do you turn it off instantly? Write the exact steps.

**9. Incident plan.** Write the five steps — detect, contain, assess, fix, report — with names for who does each. Note any legal reporting duty (see Chapter 10).

**10. Staff training.** What will you teach the team about this automation, and how will they report a problem?

Put it on one page. Share it with the people involved. Keep it where you can find it on the bad day. A plan you wrote on a calm day is worth ten times a plan you invent in a crisis.

## Checklist

### 20.9 Checklist: the 15 security questions

Before you let an AI automation touch real data, answer these fifteen questions. Every one must be a clear "yes."

- ☐ **1. Least access:** Does the automation have only the data it truly needs, and nothing more?
- ☐ **2. Human on the send:** Is a human checking anything that reaches a customer?
- ☐ **3. Data location:** Do I know every system the data touches, including outside my control?
- ☐ **4. Vetted tools:** Have I refused to feed sensitive data into any tool I cannot account for?
- ☐ **5. Encrypted in transit:** Is data encrypted while moving between systems?
- ☐ **6. Encrypted at rest:** Is data encrypted while stored?
- ☐ **7. Individual logins:** Does every person have their own login, with no sharing?
- ☐ **8. Two-factor authentication:** Is 2FA turned on for the important accounts?
- ☐ **9. Separated duties:** Are setup, approval, and log-review split across people?
- ☐ **10. Access review:** Do I update access the day staff join, move, or leave?

- [ ] **11. Logging:** Am I recording what the AI reads, writes, sends, and refuses?
- [ ] **12. Watched logs:** Is someone actually reviewing the logs, with alerts for the important signals?
- [ ] **13. Kill switch:** Can I turn the automation off instantly, and do I know how?
- [ ] **14. Incident plan:** Do I have a written, named plan to detect, contain, assess, fix, and report?
- [ ] **15. Trained staff:** Has the team been taught to spot tricks and report problems safely?

If any answer is "no," the automation is not ready. Fix it before you go live. Security is not a product you buy once; it is fifteen questions you keep answering yes.

## Key Takeaways

- Security by design means building safety in from the start — least access, a human on the send, and a kill switch — so the worst case stays small.
- A log nobody reads is not security: collect records, set alerts, and assign someone to actually look.
- Encryption hides data from outsiders (in transit and at rest), while access control limits who holds the key — you need both working together.
- Write your incident-response plan on a calm day: detect, contain, assess, fix, report, learn — with names attached to each step.
- The human factor is the first defense: train staff to spot tricks and report safely, and make the safe path the easy path.

# Chapter 21 — Managing Change in Your Company

## In Simple Words

Buying an AI tool is the easy part. Getting your company to actually use it, and to keep using it well, is the hard part. That hard part has a name: **change management**. It means helping people move from how they work today to how they will work tomorrow, without losing their trust, their skills, or their motivation along the way.

Think of it this way. You can install the best tool in the world on every desk by Monday morning. If your people do not understand why it is there, do not know how to use it, or fear what it means for them, they will find ways to avoid it. They will keep doing things the old way and pretend. The tool will sit there, paid for and unused. The technology did not fail. The change did.

This chapter is about the process of change itself: how to talk about it honestly, how to train people properly, how to handle fear and resistance, how to build new habits that stick, and how to keep supporting everyone after the first excitement fades. The people, roles, and culture that this process rests on are covered in Chapter 15 — People, Roles, and Culture. Read that chapter for *who* is involved. This one is about *how* the move actually happens over weeks and months.

One idea to carry with you: **people do not resist change; they resist being changed.** When people are dragged somewhere against their will, they push back. When they are told the truth, asked for help, and supported through the awkward middle, most of them will walk

with you gladly. Your job as a leader is not to force the walk. It is to make walking with you the natural, safe, supported choice.

Change is not a single event. It is a curve. There is a dip — a period where things feel harder and slower before they feel better. Almost every team hits it. Knowing the dip is coming, and planning for it, is the difference between a change that sticks and one that quietly collapses in month three.

## A Bit of History

**1930s–1940s: the human side of work is discovered.** Earlier in this book we met the Hawthorne studies, where workers produced more when someone paid attention to them. That was the first hard evidence that feelings and involvement, not just pay and process, drive performance. It planted the seed that managing work means managing people, not only tasks.

**1950s–1960s: change becomes a thing you can study.** Social psychologist Kurt Lewin described change in three simple steps that are still taught today. First, **unfreeze**: loosen the old habit so people are ready to move. Second, **change**: introduce the new way. Third, **refreeze**: make the new way the stable, normal way. His insight was that the hardest part is not the middle — it is the first step (getting people off the old habit) and the last (making the new one stick). Most failed changes skip one of these.

**1969: the implementation dip is named.** Education researcher David Fuller, and later others studying how schools adopt new methods, noticed a pattern: when people start something new, their performance drops before it rises. The new way is clumsy at first, so people do it worse than the old way they had mastered. If leaders do not expect the dip, they panic and abandon the change right at the

bottom. If they expect it and hold steady, performance climbs past the old level. This is one of the most useful facts in all of change management.

**1990s: the change-management industry is born.** As companies adopted computers, enterprise software, and reorganizations at scale, a whole field grew up around adoption. John Kotter published his famous eight-step model in 1996, stressing that a change needs a sense of urgency, a guiding team, a clear vision, and short-term wins. The ADKAR model (Awareness, Desire, Knowledge, Ability, Reinforcement) came from the same era. The common thread: adoption is not luck. It is a sequence you can plan.

**2000s: the graveyard of big programs.** Studies kept finding that a large share of big change programs missed their goals. A widely repeated figure says around 70% fail. The exact number is debated and its origin is fuzzy, so treat it as a rough signal, not a precise fact. The signal is real: big, top-down, poorly-supported change fails often, and the cause is almost always people, not the plan on paper.

**2020s: AI makes change personal and fast.** Earlier software changed how you typed or filed. AI changes what the job *is* — it can write, summarize, and decide. That touches people's identity and their sense of security. And it arrives faster than past waves, so companies have less time to prepare. The old lessons now matter more, not less: tell the truth, involve people, train them, expect the dip, and stay present after launch.

The arc is clear. For eighty years, every wave of new working tools taught the same lesson. The tool is never the change. The people are. AI is the newest and fastest teacher of a very old lesson.

# Curiosity

## Why we cling to the old way: the endowment effect

Here is a small experiment that explains a great deal of workplace resistance. In the 1980s, economists Daniel Kahneman, Jack Knetsch, and Richard Thaler ran a simple study. Half the people in a room were given a coffee mug. Then they were asked how much they would sell the mug for. The other half were asked how much they would pay to buy one. The owners wanted roughly twice as much as the buyers were willing to pay. The mug was identical. The only difference was ownership.

This is called the **endowment effect**: once something is ours, we value it more than an identical thing we do not own. It is not irrational greed; it is how human attachment works. We merge our possessions, our routines, and our skills into our sense of self.

Now apply it to work. Your team's old process — the messy spreadsheet, the way Maria always handles the monthly report, the shortcut everyone learned years ago — is *theirs*. They are endowed in it. They value it more than you do, because you see a process and they see their own competence, their history, their identity. Asking them to give it up for a new AI tool feels, to them, like being asked to give up a part of themselves for free.

This is why "it is obviously better, just use it" does not work. To the person endowed in the old way, it is not obviously better, and the loss feels real. Good change management works *with* the endowment effect, not against it: you honor the old way ("this served us well, and your skill in it is why we can move now"), you let people keep what is valuable to them (their judgment, their role, their pride), and you frame

the new tool as adding to them rather than replacing them. When you understand the mug, you understand the resistance.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

Two accounting firms of about fifteen people each adopted the same AI tool to draft client emails and summarize documents.

The first firm launched it like a product release. On a Monday, the owner sent an email: “We have bought an AI assistant. Log in, use it for drafts, it will save time.” No meeting. No training beyond a shared link. No mention of what it meant for anyone’s job. By Wednesday, two senior staff were openly skeptical. By the end of the month, almost everyone had quietly gone back to writing from scratch and only used the tool when asked. Nobody said so directly, but the fear was clear: “They are teaching the machine to replace us.” Six months later the subscription was cancelled. The tool worked fine. The change failed.

The second firm launched it like a journey. Before anything switched on, the owner held a one-hour meeting and said three things plainly. First, the why: “We spend too many hours on repetitive writing, and it is burning people out. This tool takes the boring part so you can spend time on the client work you were trained for.” Second, the honest promise: “No one will be let go because of this tool. If anything, it frees us to take on more clients and grow.” Third, the ask: “I need your help. You know our clients better than any machine. Try it, break it, tell me what is wrong, and we will fix it together.”

They picked two respected staff as “champions” to try it first and answer questions from peers. They ran a hands-on training where

everyone practiced on real (anonymized) work. They set one firm rule: the AI drafts, a person reviews and sends — nothing leaves without a human eye. They shared progress every week, including the mistakes and the fixes. When the dip came in week two — when everyone was slower and grumpier — the owner said, “This is the dip, it is normal, keep going,” and the team laughed and held steady. By month three, the tool was simply how they worked, and nobody remembered the old way fondly.

Same tool. Opposite outcomes. The difference was never the software. It was the change plan.

## How to Do It

### 21.1 Communicating in a transparent way

Communication is not one email at the start. It is a steady stream of honest talk that runs before, during, and after the change. Get this wrong and every other effort leaks. Get it right and you can survive almost any stumble, because trust holds the team together.

**Lead with the why, not the what.** People tolerate a hard change if they understand the reason. They resent a change that feels arbitrary or hidden. So start with the reason in plain words: what problem are we solving, and why does it matter to us and to our customers? “We are adopting AI” is a *what* and it means nothing. “We spend forty hours a week on repetitive writing that makes people stay late and drains their energy; this tool takes that away” is a *why* you can rally behind.

**Say the quiet part out loud.** The first thing everyone is thinking is, “Will I lose my job?” If you do not answer it, they will answer it themselves, and they will assume the worst. Be honest. If the plan is to redeploy freed time into growth and better work, say exactly that. If

some roles will genuinely change, say that too, and say how you will handle it fairly. Do not promise "nothing will change" — that is a lie, and when reality contradicts it later, you lose all trust. The truth, delivered early and kindly, is always safer than a comforting lie that unravels.

**Repeat, repeat, repeat.** What you say once, people barely hear. What you say five times, in different settings, they begin to believe. Say the why in the launch meeting, in one-on-ones, in the hallway, in the weekly update. Each person hears it at a different moment and needs to hear it more than once. Repetition is not nagging; it is how understanding spreads.

**Two-way, not one-way.** Communication is not a broadcast. Build in real channels for questions and worries — a standing time in meetings, an anonymous box, open office hours — and actually respond. When people see their concerns answered, they trust the process more. When their questions are ignored, they fill the silence with fear.

**Share the bad news too.** When the tool makes a mistake, when a deadline slips, when the dip hits, say so. A leader who only shares good news looks like they are selling something. A leader who shares the mess honestly looks trustworthy, and people follow trustworthy leaders through hard patches.

## 21.2 Practical training and side-by-side support

Training is not a one-hour slide deck that everyone forgets by lunch. Effective training is hands-on, practical, and continues long after the first session.

**Train on real work, not toy examples.** People learn a tool by using it on the tasks they actually do. Take a real invoice, a real customer

email, a real report (anonymize any personal data first), and have everyone practice on it. A generic demo of "features" teaches nothing. Doing your own job with the new tool teaches everything.

**Teach the judgment, not just the buttons.** Anyone can learn to click "generate." The hard, valuable skill is knowing when the output is good, when it is wrong, and what to do about it. Spend most of the training on review: here is a draft the AI made, what is right, what is wrong, what would you change, and why. This is the skill that keeps bad output from reaching a customer. It is more important than the tool itself.

**Use champions.** Pick two or three respected, curious people to learn the tool first and well. They become the go-to helpers for their peers. People are far more comfortable asking a colleague "how do I do this?" than asking the boss or reading a manual. Champions spread confidence faster than any formal training can. Choose people others already trust, not just the most technical.

**Side-by-side, not hands-off.** In the early weeks, sit with people. Watch them work. Catch the small confusions before they become habits. A five-minute "let me show you" in the moment is worth more than a written guide nobody reads. This is the "side-by-side support" that turns confusion into competence.

**Make help easy to reach.** Create a simple place for questions — a shared document of common answers, a chat channel, a weekly clinic. If getting help is easy, people get help and keep moving. If it is hard, they get stuck, get frustrated, and quietly give up.

## 21.3 Resistance and fear

Resistance is not a character flaw in your people. It is predictable, normal information. It tells you where the plan is weak, where the fear is, and where you have not yet explained enough. Treat it as a signal to listen to, not a problem to crush.

**Name the fear.** Most resistance is fear wearing a costume. "The tool is too complicated" often means "I am afraid I will look stupid learning it." "I don't trust its output" often means "I am afraid of being replaced and this is a respectable objection." When you respond only to the surface argument, the fear underneath stays and keeps working. Address the fear directly and kindly: reassure about the job, reassure that confusion is temporary, reassure that they are valued.

**Resistance is a form of information.** When several people resist the same thing, they are usually pointing at something real. Maybe the tool genuinely does not fit a certain task. Maybe a deadline is unrealistic. Maybe a concern you had not considered is valid. Ask "what specifically is wrong here?" and listen. Sometimes the resisters are right, and catching that early saves you from a costly mistake.

**Do not mock or force.** Dismissing someone as a "Luddite" or a "blocker" poisons the whole team and hardens the resistance. Forcing compliance produces fake compliance — people who use the tool when watched and avoid it when not. Both backfire. The way through resistance is patience, honesty, involvement, and proof.

**Bring the skeptics in.** The strongest move against resistance is to give the skeptics a real role. "You think this will not work — I want you to test it hard and report back honestly." A skeptic who is heard and taken seriously often becomes the most credible advocate, because

when *they* say it works, everyone believes it. Someone who is ignored stays an enemy.

**Accept that not everyone comes.** Even with the best handling, a few people may never embrace the change. That is normal and survivable. Focus your energy on the majority who are willing but uncertain. Do not let a small loud minority drain you from the large quiet middle that just needs support.

## 21.4 New processes and new habits

A tool is not a process. A process is the agreed way work flows through your team. When you add AI, you are changing the process, and processes run on habits. Building new habits takes deliberate design and time.

**Redesign the workflow, do not just bolt on the tool.** Do not say "keep doing everything the same, but also use this." Decide the new flow explicitly: what does the AI do, what does the human do, in what order, and where does the handoff happen? Write it down simply. For example: "AI drafts the reply → staff reviews and edits → staff sends → staff logs any error." Clear steps become a habit. Vague instructions never do.

**Make the right thing the easy thing.** Habits follow the path of least effort. If using the AI tool is the default and the easiest route, people will use it. If the old way is easier and the new way adds friction, they will drift back. Remove friction from the new path and, if needed, add a little friction to the old one. Design the environment so the desired behavior is the natural one.

**Expect the dip and hold steady.** As covered in the history, performance drops before it rises. In weeks two and three, people will

be slower, clumsier, and grumpier than they were with the old way. This is the dip. If you panic and retreat, you teach the team to give up at the first discomfort. If you name it, expect it, and hold the line with support, you come out the other side faster and better. Tell people the dip is coming before it arrives, so when it does, they recognize it instead of quitting.

**Reinforce, reinforce, reinforce.** A habit sticks when it is rewarded and repeated. Celebrate early wins publicly. Thank people who use the tool well. Show the results — the hours saved, the errors caught. Reinforcement is the "refreeze" step: it turns the new way from a forced effort into the automatic norm. Without reinforcement, habits dissolve back to the old default.

**Make it official.** Once the new way works, write it into your standard procedures, your onboarding for new hires, your checklists. When the new process is "how we do things here," it survives turnover and becomes permanent. If it stays informal, it fades.

## 21.5 Continuous support

The biggest mistake in change management is treating launch as the finish line. It is the starting line. Support after launch is what determines whether the change sticks or slowly erodes.

**Support does not end at training.** People hit new confusions weeks later, when real edge cases arrive. Keep the help channels open. Keep the champions available. Keep the weekly clinic running. The team that gets support only in week one and is abandoned in month two will quietly regress.

**Check in regularly.** Do not assume silence means success. Silence often means people are struggling and giving up quietly. Ask directly, in

one-on-ones and team meetings: “How is it going? What is still hard? What is not working?” You cannot fix what you do not know about.

**Keep sharing results.** Show the team what the change is producing — time saved, errors down, customers happier. Progress is motivating. When people can see that their effort is paying off, they keep going. When they see nothing, they wonder if it was worth it.

**Fix what breaks, fast.** When the tool fails or the process snags, respond quickly. Every unresolved problem is a small argument for going back to the old way. Fast fixes say, “this is real, we are serious, it is worth it.” Slow fixes say the opposite.

**Keep improving.** The change is never “done.” As people use the tool, they find better ways, new uses, and gaps. Keep collecting feedback and refining. A change that keeps getting a little better stays alive; a change that is frozen at launch slowly dies. The measurement that drives this improvement is covered in Chapter 22 — Measuring Results and ROI.

## Ethics and Responsibility

Change management is where ethics and people meet most directly. You are handling people’s livelihoods, their pride, and their trust.

**Be honest about the job.** Never promise “nothing will change” if it will. Never hide a plan to reduce staff behind a tool launch. If roles will change, tell people clearly and give them time and support to adapt. A lie about jobs, discovered later, destroys trust permanently and can hurt people who could have prepared.

**Do not use the tool as a cover for cuts you already decided.** If you have already decided to reduce headcount, do not dress it up as “adopting AI.” Decide the people question honestly and separately, as covered in Chapter 16, and be transparent about it.

**Respect the fear, do not exploit it.** People's anxiety about their jobs is real and vulnerable. Treat it with care and dignity. Do not use fear as a motivator ("keep up or be left behind"). Fear-driven adoption produces stress, resentment, and turnover.

**Give people a real voice.** Involving people is not theater. If you ask for their input, be willing to act on it and to say so. Asking and then ignoring is worse than not asking, because it teaches people that their voice does not matter.

**Protect dignity through the dip.** The awkward middle is when people feel least competent. Be patient and kind. Do not publicly shame anyone for being slow to learn. Support them through it. How you treat people when they are struggling is how they will remember this change.

**Honor the endowment.** As the curiosity showed, people are attached to their old skills and routines. Respect that. Frame the change as adding to them, not discarding them. Carry their pride forward into the new way.

## Mistakes to Avoid

**The one-email launch.** Announcing the change once and expecting it to stick. People need the why repeated many times, in many settings.

**Skipping the why.** Telling people what to do without explaining why. Without a reason, change feels arbitrary and breeds resistance.

**Lying about jobs.** Saying "nothing will change" when it will. When reality contradicts the promise, all trust is lost.

**Training on toy examples.** Generic feature demos instead of hands-on practice on real work. Teaches nothing that sticks.

**Ignoring the fear.** Answering the surface objection and leaving the real fear of replacement untouched underneath.

**Mocking resisters.** Calling people Luddites or blockers. Hardens resistance and poisons the team.

**Treating launch as the finish line.** Abandoning support after week one. The change erodes without continuous support.

**Not expecting the dip.** Panicking when people get slower in week two and retreating. The dip is normal; hold steady through it.

**No reinforcement.** Failing to celebrate wins and show results. Habits dissolve without reinforcement.

**Leaving the new way informal.** Never writing it into standard procedure. It fades with the next bit of turnover.

**One-way communication only.** Broadcasting and never listening. Questions go unanswered and fear fills the silence.

**Forcing fake compliance.** Making people use the tool when watched. They avoid it when not. Real adoption cannot be forced.

## Practical Exercise

### 21.6 Exercise: a communication plan

Before you launch any AI change, write a one-page communication plan. Fill in these seven parts.

**1. The why (one paragraph).** In plain words, what problem are you solving and why does it matter to the team and the customers? Write it

so a new hire could understand it. If you cannot state the why simply, you are not ready to launch.

**2. The honest job answer (two or three sentences).** What will this change mean for people's jobs? Say it truthfully. If the freed time goes to growth and better work, say so. If some roles will change, say so and say how you will handle it fairly. Write the exact words you will use when someone asks, "Will I lose my job?"

**3. The audiences.** List who needs to hear this: all staff, managers, the specific team affected, customers if relevant. Each audience may need a slightly different message.

**4. The channels and timing.** Where and when will you say it? Plan at least: a launch meeting (the why, the promise, the ask), a follow-up in one-on-ones within a week, a weekly update during the change, and an open question channel that stays open. Write down who says what, to whom, and when.

**5. The champions.** Name two or three respected people who will try the tool first and help their peers. Note why each is trusted.

**6. The dip warning.** Write the sentence you will use when things get slower: "This is the normal dip; it happens to everyone; keep going, support is here." Decide now so you say it calmly later instead of panicking.

**7. The feedback loop.** How will people raise concerns, and how will you respond? Name the channel and how often you will answer.

Read the page back and ask: if I were a nervous employee receiving this, would I feel informed, safe, and included? If not, revise until you would. Keep the plan and follow it. This single page prevents most of the failures listed above.

# Checklist

## 21.7 Change checklist

Before and during any AI change, check these.

- ☐ **You can state the why in one plain paragraph** that anyone could understand.
- ☐ **You have an honest answer to "will I lose my job?"** and you are ready to say it.
- ☐ **You did not promise "nothing will change"** if something will.
- ☐ **You have named the audiences** and tailored the message to each.
- ☐ **You planned a launch meeting** covering the why, the promise, and the ask.
- ☐ **You will repeat the why** many times, in multiple settings.
- ☐ **You have two or three trusted champions** trying the tool first.
- ☐ **Training is hands-on, on real work**, not a generic feature demo.
- ☐ **You taught review and judgment**, not just which buttons to press.
- ☐ **You set a clear rule** for human review before anything goes out.
- ☐ **You have an easy help channel** (chat, doc, clinic) that stays open.
- ☐ **You warned the team about the dip** before it arrives.
- ☐ **You treat resistance as information** and ask what is really wrong.
- ☐ **You bring skeptics in** with a real role instead of pushing them away.

- ☐ **You check in regularly** instead of assuming silence means success.
- ☐ **You share results** (time saved, errors down) to keep motivation up.
- ☐ **You fix problems fast** so the new way stays credible.
- ☐ **You reinforce wins** with celebration and visible progress.
- ☐ **You wrote the new process** into standard procedure and onboarding.
- ☐ **You treat support as continuous**, not something that ends at launch.

If a box is empty, the change is not fully supported. Fill it before you launch, or fill it as you go — but do not leave it empty. A change that is communicated honestly, trained well, and supported continuously is a change that sticks.

## Key Takeaways

- People do not resist change; they resist being changed — tell the truth, involve them, and support them, and most will walk with you.
- Communication is a continuous stream, not one email: lead with the why, answer the job question honestly, and repeat it many times.
- Expect the implementation dip — people get slower before they get faster — and hold steady through it instead of retreating.
- Resistance is information and fear is usually the cause; name the fear, listen to the skeptics, and never mock or force.
- Launch is the starting line, not the finish: continuous support, visible results, and writing the new way into standard procedure are what make change stick.

# Chapter 22 — Measuring Results and ROI

## In Simple Words

You changed something. Did it work? This chapter is about answering that question with facts instead of feelings.

It is very easy to *feel* like a new AI tool is helping, especially right after you bought it. You are excited, you want it to succeed, so you notice the good moments and ignore the bad. Six months later, nobody can say whether it actually saved time, cut errors, or cost more than it returned. The money is gone and the lesson is lost. Measuring is the habit that prevents this. It turns "I think it is working" into "it cut our processing time by 40%, and here is the number."

The full method for calculating return on investment — the formula, the cost list, the worked example — lives in Chapter 16 — Goals, Costs, and Return on Investment. That chapter is the home of the ROI method. Do not relearn it here. This chapter builds on it and answers the next set of questions: *what* small set of numbers should you actually watch, how do you keep watching them without drowning in data, and when should you stop a project or correct its course.

A simple image to carry: measuring is like the dashboard in a car. You do not stare at every gauge all the time, but you glance at a few key ones constantly — speed, fuel, engine warning. If you drive with the dashboard covered, you will run out of fuel or overheat the engine and not know why. A business change without a dashboard is driving blind.

This chapter helps you build a small, honest dashboard and read it regularly.

One honest warning: numbers can be made to say almost anything if you want them to. You can pick the metric that flatters you, start counting at the best moment, or forget the costs. The whole point of measuring is to find the truth, not to decorate a decision you already made. Build your dashboard to catch you being honest.

## A Bit of History

**Early 1900s: "what gets measured gets managed."** Often credited to management thinker Peter Drucker (though he may not have coined it exactly), this phrase captured a powerful truth: you cannot improve what you do not track. If nobody counts the hours a task takes, it never gets faster. If nobody counts the errors, they never go down. Measurement turned management from guesswork into something you could steer.

**1950s–1970s: metrics spread through industry.** Factories tracked output, defect rates, and downtime. Quality movements like those of W. Edwards Deming made measuring defects central to improving production. The idea spread: pick a few numbers that reflect what you care about, watch them, and act on them.

**1990s: the balanced scorecard.** Kaplan and Norton argued that money alone is a poor measure of health. A company can look profitable while its customers are unhappy and its staff are leaving. They pushed the "balanced scorecard" — a small set of measures covering money, customers, internal processes, and learning. This is the direct ancestor of the simple four-part dashboard you will build in this chapter.

**2000s: dashboards go digital.** Software made it easy to collect and display data, and dashboards became a standard management tool. But ease brought a new problem: people started tracking everything, producing dashboards with fifty charts nobody read. The lesson flipped — the skill was no longer collecting data, it was choosing the few numbers that actually matter and ignoring the noise.

**2010s: the vanity-metric trap.** In the startup world, a term appeared for numbers that look impressive but mean nothing: "vanity metrics." A website with a million visits but no sales. An app with downloads but no active users. The trap is measuring something that feels good instead of something that reflects real value. This is the most common failure mode in AI measurement today — counting how many times a tool was used instead of whether it produced value.

**2020s: AI makes measurement essential and tricky.** AI can produce real, measurable gains, but it also produces plausible-looking output that can be wrong, so measuring quality — not just quantity — is now critical. And because AI projects are easy to start and hard to evaluate, the discipline of measuring, and of knowing when to stop, has never mattered more.

The arc: from "measure to manage" to "measure the right few things, honestly, and keep watching." The tools got easier; the discipline of choosing well and staying honest got harder.

# Curiosity

## 22.5 The company that cut cycle times by 50%

IBM has reported cutting the time spent on certain security and vendor-risk review tasks by roughly **50%** by automating parts of the

work with AI — turning a slow, manual review process into a much faster one.

That is the headline. The full IBM case, with the details and the lessons behind the number, is told in Chapter 30 — IT and Leadership, the canonical home of the IBM story. The point for this chapter is one line: a halving of cycle time is a real, measurable benefit — and it is exactly the kind of result a good dashboard is built to reveal and track over time.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A mid-sized insurance agency adopted an AI tool to draft responses to customer claims inquiries. The owner wanted to know if it was worth it, so she set up a simple dashboard before launch and filled it in every week for six months.

The dashboard had four numbers, each with a "before" baseline and a running "now" value:

- **Time per response.** Before: 22 minutes on average. After six months: 9 minutes.
- **Error rate.** Before: about 6% of responses needed a correction after being sent. After: 4%.
- **Cost per response.** Before: roughly €5.50 in staff time. After: €2.30.
- **Customer satisfaction.** Before: 3.6 out of 5. After: 4.1 out of 5.

Every number moved the right way, and the owner could see it week by week. The tool paid for itself in about three months and kept returning

after that. Because she had the numbers, the decision to expand the tool to a second team was easy and confident.

But here is the more interesting part. In month two, the error rate briefly jumped from 6% to 8%. The dashboard caught it immediately. They looked and found the cause: a new type of claim the tool had not been trained on was producing confident but wrong drafts. Because the dashboard flagged it early, they added a rule to route that claim type to a human, and the error rate fell back down. Without the dashboard, those wrong responses might have gone out for weeks before anyone noticed.

The dashboard did two jobs: it proved the value, and it caught a problem early. That is what a good dashboard is for.

## How to Do It

### 22.1 Simple KPIs: time, errors, costs, satisfaction

A **KPI** stands for **Key Performance Indicator** — a plain phrase for "a number that tells you how you are doing." The art is to pick a few that matter and ignore the rest. For almost any AI change, four KPIs cover what you need. They map directly to the four things you care about: is it faster, is it more accurate, is it cheaper, and are people happy.

**Time.** How long does the task take now versus before? This is the most common benefit of AI and the easiest to measure. Measure the time for one unit of work — one invoice, one email, one report — and compare it to your baseline. Time saved is the benefit you can most often turn into money (by valuing an hour at its real cost, as Chapter 16 shows).

**Errors.** How often does the work come out wrong? Count the mistakes that reach the end — a wrong figure in a report, a wrong answer sent to a customer, a claim processed incorrectly. AI can reduce errors, but it can also create new, confident ones. Measuring errors is not optional; it is how you catch a tool that sounds right and is wrong. Track both the error rate and the *type* of error, because the type tells you what to fix.

**Costs.** What does a unit of work cost now, all in? Take the time cost plus the tool's share of subscription and any review time. Compare to the before cost. This is where you see whether the savings are real after you subtract what the tool costs you. Remember the lesson from Chapter 16: count the full cost, including the human review time, or the number lies.

**Satisfaction.** Are the people who use it, and the customers who receive its output, happy? This is the KPI people forget, and it matters. A tool that is fast and cheap but makes staff miserable or customers annoyed is not a success. Measure staff satisfaction with a simple question ("Is this tool helping you, on a scale of 1 to 5?") and customer satisfaction with the feedback you already collect, or a short survey.

Four numbers: faster, more accurate, cheaper, happier. If all four move the right way, you have a clear win. If some move and some do not, that is the interesting signal to investigate. Keep it to these four. A small business does not need a dozen metrics. Four honest ones beat forty vanity ones.

## 22.2 A minimal dashboard

A dashboard is just a single page where your KPIs live, so you can see them at a glance. It does not need software. A spreadsheet, a

whiteboard, or a sheet of paper taped to the wall all work. The rule is simple: **one page, four numbers, each with a before and a now.**

Here is the minimal layout. For each KPI, keep three columns:

| KPI | Before (baseline) | Now | Target |
|---|---|---|---|
| Time per unit | 22 min | 9 min | 10 min |
| Error rate | 6% | 4% | 3% |
| Cost per unit | €5.50 | €2.30 | €2.50 |
| Satisfaction | 3.6 / 5 | 4.1 / 5 | 4.0 / 5 |

Three columns per number, and the whole thing fits on one page. The **baseline** is what you measured before you started (never skip this — without a baseline you can prove nothing). The **now** is the current value, updated regularly. The **target** is what you aimed for.

Good dashboard rules:

- **One page only.** If it grows beyond one page, you have too many numbers. Cut some.
- **Always show the baseline.** A number without a "before" is meaningless. "9 minutes" tells you nothing; "down from 22 to 9" tells you everything.
- **Show the trend, not just the snapshot.** A tiny arrow or a simple line showing the last few weeks tells you the direction. A single number tells you where you are but not where you are heading.
- **Make it visible.** Put it where the team sees it — a shared screen, a wall, the start of the weekly meeting. A dashboard nobody looks at is not a dashboard.
- **Keep it cheap to update.** If updating the dashboard takes an hour a week, you will stop doing it. Make it a ten-minute job. If you

can pull numbers automatically, good; if not, a quick manual count is fine.

The point of a minimal dashboard is not to look professional. It is to make the truth easy to see and impossible to ignore.

## 22.3 Continuous monitoring

A dashboard you fill in once is a photo. A dashboard you update every week is a live monitor. The value is in the watching over time, because that is where the real signal lives.

**Update on a fixed rhythm.** Pick a cadence — weekly is usually right for a small business — and update the same day every week. The rhythm matters more than the exact frequency. A weekly update catches problems within a week. A quarterly update lets a problem run for three months. Set a recurring reminder and make it a habit.

**Look for the trend, not the single point.** One week's number can be a fluke. Three weeks of the same direction is a signal. If time saved is creeping up week after week, the tool is settling in well. If errors are creeping up, something is drifting. Read the line, not the dot.

**Watch for drift and decay.** AI tools can quietly get worse over time. Data changes, customer questions change, the tool's model may update, and what worked in month one may not work in month six. Continuous monitoring catches this slow decay. A tool that was a clear win at launch can quietly become a liability if nobody keeps watching. This is why monitoring never really ends.

**Set a warning threshold.** Decide in advance what number should trigger action. For example: "If the error rate goes above 7%, we stop and investigate." A pre-set threshold stops you from slowly accepting a

worsening number. When the line crosses the threshold, you act — no debate, no "we'll look at it later."

**Review it in the weekly meeting.** Make the dashboard the first item on the team's weekly agenda. Five minutes looking at the four numbers together keeps everyone focused on reality, surfaces problems fast, and shares the wins. It also signals that measurement matters here, which makes people take it seriously.

**Compare to the plan, not to the hype.** Measure against your own baseline and your own target, not against a vendor's promise or a competitor's story. Your numbers are the only ones that describe your business.

## 22.4 When to stop or correct a project

Not every project should continue. Some should be corrected. Some should be stopped. Measuring gives you the honest signal to make that call early, before you have sunk more money and time into something that is not working. There are three possible decisions, and the dashboard tells you which one you are facing.

**Keep going.** All four KPIs are moving the right way, or close to it. The tool is delivering. Continue, and consider expanding it to other tasks or teams. This is the easy decision.

**Correct and continue.** The tool has value, but one or more numbers are off. This is the most common outcome, and it is not failure — it is information. A high error rate on one type of task means: route that task type to a human. A satisfaction score that is low among staff means: the training was weak, retrain. A cost that is higher than expected means: find the hidden cost and cut it. Correct the specific

problem and keep going. Most good projects spend some time in "correct and continue."

**Stop.** The numbers show the tool does not deliver value, and the problems are not fixable. Maybe it is slower than doing the work by hand. Maybe the error rate is too high to trust on any task. Maybe the cost never comes down below the benefit. If honest measurement shows a negative return with no clear path to fix it, stop. Stopping early is a success, not a failure — it means you learned the truth and saved yourself from throwing good money after bad.

**How to decide between correct and stop.** Ask three questions. First, *is the core value there?* If the tool saves real time on most tasks, the core value exists and you should correct, not stop. Second, *is the problem fixable?* A routing rule, a retraining, a cost cut — these are fixable, so correct. If the problem is fundamental (the tool simply cannot do the task well), it is not fixable, so stop. Third, *what is the sunk-cost trap telling me?* Be honest: are you continuing only because you already spent money? That is the sunk-cost fallacy. Decide on the future numbers, not the past spending.

**Set a review date before you launch.** Before you start, decide when you will judge the project — usually three to six months in. At that date, look at the dashboard and make the keep / correct / stop call deliberately. A pre-set review date stops a failing project from drifting on forever by default. It forces the decision while there is still time to recover the investment.

**Stopping well.** If you stop, do it cleanly and learn from it. Write down why it failed — wrong process, bad data, tool could not handle the task, cost too high. That lesson is valuable and reusable. A well-stopped project that teaches you something is worth more than a zombie project that limps along proving nothing.

## Ethics and Responsibility

Measurement is where honesty is tested most, because the numbers can justify decisions that affect real people.

**Measure to learn, not to justify.** Use the dashboard to find the truth, not to prove you were right all along. If the numbers show the tool is not working, that is a finding to act on, not an embarrassment to hide.

**Do not pick flattering metrics.** Choosing a number that only shows the good side — like counting how many times the tool was used instead of whether its output was good — is a lie told with a spreadsheet. Pick metrics that reflect real value, including the ones that might show a problem.

**Do not hide the errors.** It is tempting to undercount mistakes, especially errors the AI made confidently. Report them honestly. A hidden error rate is a loaded gun pointed at a future customer.

**Do not use measurement as surveillance.** Tracking KPIs about a *process* is healthy. Using the same data to spy on individual employees, rank them, and punish them poisons trust and turns measurement into a weapon. Measure the work, not the person. Keep the focus on improving the process.

**Be transparent with the numbers.** Share the dashboard with the team, including the bad weeks. When people see the honest picture, they trust the decisions that come from it. When they only see the polished version, they suspect the whole thing.

**Separate the money truth from the people decision.** A dashboard that shows a tool saves money does not automatically mean

"cut staff." As Chapter 16 stresses, the money decision and the people decision are separate. Measure the money honestly, then decide about people humanely and on its own merits.

## Mistakes to Avoid

**No baseline.** Not measuring the "before," so you can never prove the "after" improved anything. Measure before you start.

**Vanity metrics.** Counting usage, clicks, or downloads instead of value. A tool used a thousand times that produces wrong answers is a failure, not a success.

**Measuring only the good.** Reporting the wins and hiding the errors and the bad weeks. A one-sided dashboard is a dishonest dashboard.

**Too many numbers.** A fifty-chart dashboard nobody reads. Four honest KPIs beat forty ignored ones.

**Forgetting the cost side.** Measuring time saved but not the tool's cost and the human review time. The saving looks real until you subtract what it costs.

**Snapshot instead of trend.** Looking at one number once instead of the direction over weeks. A single point is a fluke; the trend is the truth.

**Not watching for decay.** Assuming a tool that worked at launch keeps working forever. Quietly degrading tools need continuous monitoring.

**No warning threshold.** Letting a bad number creep up because there was no pre-set line that says "act now."

**The sunk-cost trap.** Continuing a failing project only because you already spent money. Decide on future value, not past spending.

**No review date.** Letting a project drift forever because nobody scheduled the moment to judge it.

**Measurement as a weapon.** Using process data to surveil and punish individuals. Measure the work, not the person.

**Confusing activity with achievement.** Mistaking "we did a lot with the tool" for "the tool created value." Only the four KPIs tell you which.

## Practical Exercise

### 22.7 Exercise: a dashboard of results

Build a one-page dashboard for one AI change you have made or are planning. Do it before launch if you can; if you already launched, build it now and reconstruct the baseline as best you can.

**Step 1 — Pick the four KPIs.** Write down, for your specific task: the time per unit, the error rate, the cost per unit, and the satisfaction score. If one does not apply, note why and keep the other three.

**Step 2 — Set the baseline.** For each KPI, write the "before" number. If you did not measure before launch, estimate honestly from memory or a quick sample count now, and label it as a reconstruction. Do not skip the baseline.

**Step 3 — Set the target.** For each KPI, write the number you are aiming for. Make it realistic, not a fantasy.

**Step 4 — Build the table.** Make the four-row, three-column table (Before / Now / Target) on a spreadsheet or a sheet of paper. Keep it to one page.

**Step 5 — Set the update rhythm.** Choose a day and time each week to update the "Now" column. Put it in your calendar. Make it a ten-minute job.

**Step 6 — Set the warning thresholds.** For each KPI, decide the number that should trigger action. Write it next to the row. For example, "error rate above 7% = stop and investigate."

**Step 7 — Set the review date.** Pick a date three to six months out when you will look at the whole dashboard and make the keep / correct / stop decision. Write it on the page.

**Step 8 — Make it visible.** Put the dashboard where you and the team will see it weekly. Add it to the top of your weekly meeting agenda.

Now update it every week. Watch the trends. When a line crosses a threshold, act. At the review date, make the call deliberately. This one page is the difference between knowing whether your change worked and guessing.

## Checklist

### 22.8 Measurement checklist

Before and during any AI project, check these.

- ☐ **You measured the baseline** (the "before" number) for each KPI before launch.
- ☐ **You have a small set of KPIs** — time, errors, costs, satisfaction — not dozens.
- ☐ **You set a realistic target** for each KPI.
- ☐ **Your dashboard is one page** with Before / Now / Target for each number.

- ☐ **You are not tracking vanity metrics** (usage, clicks) instead of value.
- ☐ **You count the full cost**, including the tool's price and the human review time.
- ☐ **You track errors, including confident-but-wrong ones**, and note their type.
- ☐ **You measure satisfaction** from both staff and customers.
- ☐ **You update the dashboard on a fixed weekly rhythm.**
- ☐ **You read the trend, not a single snapshot.**
- ☐ **You watch for slow decay** over months, not just the launch result.
- ☐ **You set warning thresholds** that trigger action automatically.
- ☐ **You review the dashboard in the weekly meeting.**
- ☐ **You compare to your own baseline**, not to vendor hype.
- ☐ **You set a review date** (3–6 months) to decide keep / correct / stop.
- ☐ **You avoid the sunk-cost trap** and decide on future value, not past spending.
- ☐ **You report the bad weeks honestly**, not just the wins.
- ☐ **You measure the process, not the person** — no surveillance or ranking.
- ☐ **You keep the money decision separate from the people decision.**

If a box is empty, you are partly driving blind. Fill it. A small, honest dashboard that you actually watch is worth more than any promise a vendor can make.

## Key Takeaways

- Measure to learn the truth, not to decorate a decision you already made — pick KPIs that reflect real value, including the ones that

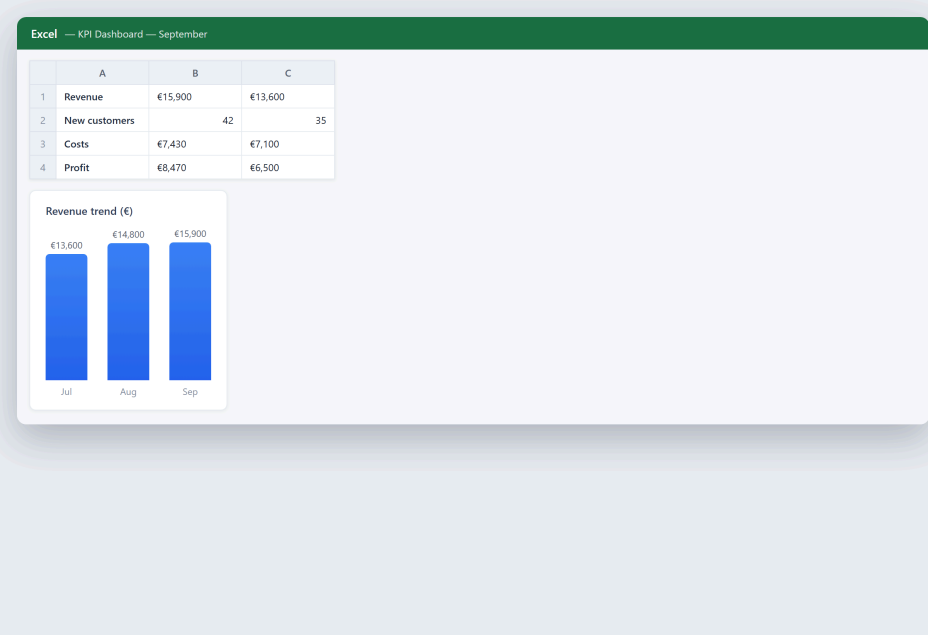
might show a problem.

- Four KPIs cover almost everything: time, errors, costs, and satisfaction; a one-page dashboard with a baseline for each is enough.
- The value is in continuous monitoring — read the weekly trend, watch for slow decay, and act when a number crosses a pre-set warning threshold.
- Measurement gives you three decisions: keep going, correct and continue, or stop; stopping early on honest numbers is a success, not a failure.
- Avoid the sunk-cost trap and the vanity-metric trap — decide on future value and real outcomes, not past spending or impressive-but-meaningless counts.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Your key numbers on one sheet




*A KPI dashboard with the numbers that matter*

**What you ask:** `Build a small KPI dashboard: monthly revenue, new customers, cost, and a chart of revenue trend.`

The agent lays out the key figures and a trend chart on one clean sheet. It becomes the page you look at every week.

*Tip: Keep the same sheet updated and ask the agent to compare this month to last month.*

---

# A financial summary report

**Acme Studio**
Creative & Design Services

# Financial Report

Management report — Fiscal Year 2025

| Report Title | Period | Company | Date |
|---|---|---|---|
| **Financial Summary for 2025** | **Fiscal Year 2025** | **Acme Studio** | **18 September 2026** |

## 1. Executive Summary

Acme Studio closed fiscal year 2025 with revenue of EUR 152,000, total costs of EUR 88,000 and a net profit of EUR 64,000. Compared with fiscal year 2024, revenue increased from EUR 118,000 while costs rose only modestly from EUR 77,000, resulting in a significantly improved profitability position.

Revenue grew 29% year over year while costs stayed controlled, lifting the net margin. Net profit rose from EUR 41,000 in 2024 to EUR 64,000 in 2025, an increase of approximately 56%. The year-over-year comparison confirms that growth was achieved without a proportional increase in the cost base.

## 2. Key Financial Highlights

| Indicator | Current Period (EUR) | Previous Period (EUR) | Change % |
|---|---|---|---|
| Revenue | 152,000 | 118,000 | +28.8% |
| Total costs | 88,000 | 77,000 | +14.3% |
| Net profit | 64,000 | 41,000 | +56.1% |

## 3. Revenue Analysis

Revenue for the Fiscal Year 2025 reached **EUR 152,000**, an increase of **29%** compared with the previous period.

Revenue rose from EUR 118,000 in fiscal year 2024 to EUR 152,000 in fiscal year 2025, an increase of EUR 34,000 or 29% year over year. This growth reflects continued demand for Acme Studio's services throughout the period and represents the strongest top-line result recorded for the comparison presented in this report.

## 4. Cost Analysis

Cost of goods sold amounted to **EUR 88,000**, while operating expenses are included within this total, yielding a gross margin of **42.1%**.

Total costs for fiscal year 2025 were EUR 88,000, compared with EUR 77,000 in fiscal year 2024, an increase of EUR 11,000 or approximately 14%. Costs therefore grew at roughly half the rate of revenue, remaining controlled and confirming that the revenue expansion was not accompanied by a proportional increase in the cost base.

## 5. Liquidity & Solvency

| Indicator | Value |
|---|---|
| Current ratio | Not available |
| Quick ratio | Not available |
| Debt-to-equity ratio | Not available |
| Interest coverage | Not available |

Liquidity and solvency indicators are not part of the financial summary provided for fiscal year 2025 and are therefore not reported here. The net profit of EUR 64,000 and the controlled cost base of EUR 88,000 support a positive profitability position for the period under review.

## 6. Outlook

On the basis of the fiscal year 2025 results, Acme Studio enters the next period with a strengthened profit position. Revenue grew 29% year over year while costs stayed controlled, lifting the net margin. Management intends to preserve this balance between top-line growth and cost discipline in order to sustain the improved net margin achieved in 2025.

## 7. Risks

- Revenue concentration: the 29% year-over-year revenue growth may depend on a limited number of clients or engagements.
- Cost pressure: total costs increased by 14% year over year and could rise further if not kept under control.
- Margin sustainability: the improved net margin of 42.1% must be maintained as revenue scales.
- Reporting scope: liquidity and solvency data are not available for fiscal year 2025, limiting the completeness of the financial assessment.

**Prepared by**
Maria Rossi
Chief Accountant — 18 September 2026

**Reviewed by**
Acme Studio Management
18 September 2026

*A financial summary PDF with charts*

**What you ask:** `Create a PDF financial summary for 2025: revenue, costs, profit and a year-over-year chart.`

The agent builds the report with the figures, the trend chart and a plain-language summary of what the numbers mean.

*Tip: Attach your ledger export and the agent reads the real numbers into the report.*

# Chapter 23 — Scaling and Improving

## In Simple Words

A pilot works. One team, one task, one small test, and the numbers look good. Now what? This chapter is about the step that turns a small win into a lasting one: **scaling**. It means taking something that worked in a small test and making it the normal way you work, across more people, more tasks, or the whole company.

Scaling is where most good AI projects quietly die. A pilot can succeed and then go nowhere. The tool stays on one desk, used by one enthusiastic person, while everyone else keeps working the old way. The win was real, but it never spread, so it never became the big benefit you hoped for. The reason is almost never the technology. It is that scaling is its own job, with its own skills, and nobody treated it that way.

This chapter covers the five parts of scaling well: turning a pilot into a standard process, writing it down so it survives, keeping training going as you grow, maintaining and updating the tool over time, and using what you learned to find the next opportunity. It builds on the change-management skills in Chapter 21 and the measurement habits in Chapter 22. Scaling is what you do after the change is proven and you want it to spread and last.

A simple image to carry: a pilot is a single candle flame. Scaling is using that flame to light a whole row of candles, so the light does not depend on one flame staying lit. But you cannot light the row by waving the first candle around wildly. You need a steady method — a clear plan, a written recipe, people trained to light each new candle the same way,

and someone tending the whole row so no candle goes out. That method is what this chapter gives you.

The goal is not a big, flashy rollout. The goal is a small win that grows steadily, safely, and on purpose, until the new way is simply how the whole company works.

## A Bit of History

**1940s–1950s: the pilot idea comes from manufacturing.** The word "pilot" in this sense comes from industry — a small test run before full production. Factories learned never to build a million units of an untested product. They made a small batch, found the problems, fixed them, then scaled. This "pilot then scale" logic spread from factories into every kind of project.

**1990s–2000s: the pilot trap appears in IT.** As companies adopted enterprise software, a pattern showed up so often it got a name: "pilot purgatory" (also called "pilotitis"). A company would run pilot after pilot, each one succeeding, and never actually roll anything out. The pilots were safe and comfortable; scaling was risky and hard. So teams kept piloting forever, spending money on tests that never became real change. The term captured a hard truth: succeeding at a pilot and deciding to scale are two completely different things, and many organizations get stuck in the first.

**2000s: scaling as a discipline.** Consultants and management writers began studying why some companies scaled successfully and others did not. The answer was consistent: successful scaling needed the pilot turned into a documented, repeatable process, with clear ownership, training, and support. Scaling was not an afterthought; it was a planned phase. Companies that treated it as one succeeded;

companies that expected it to "just happen" ended up in pilot purgatory.

**2010s: the "scale-up" mindset.** Startups popularized the idea of "scaling" — growing a working model fast. But they also learned the hard lesson that scaling amplifies everything, including the flaws. A small crack in a process becomes a canyon when you multiply by a hundred. Scaling without fixing the cracks first is how fast-growing companies break things.

**2020s: AI scaling is different in three ways.** First, AI pilots are cheap and fast to start, so companies run many of them and risk spreading attention thin. Second, AI tools need ongoing maintenance — models update, data drifts, prompts need tuning — so scaling an AI tool means scaling the tending, not just the rollout. Third, the best AI scaling is not one big tool everywhere; it is a steady chain of small wins, each one teaching you where the next one is. The companies winning at AI scaling treat it as a continuous improvement engine, not a single big launch.

The arc: from factory test batches to the pilot trap to scaling as a deliberate, ongoing discipline. The newest lesson is that scaling is not a finish line you cross once. It is an engine you keep running.

## Curiosity

## 23.5 "Pilot purgatory": why pilots succeed and nothing changes

There is a term in business for the most common way to waste money on AI: **pilot purgatory**, sometimes called **pilotitis**. It describes a company that runs pilot after pilot — each one technically successful, each one proving the idea "works" — and never actually rolls anything

out into normal use. The pilots go on forever, the reports look positive, and real change never arrives.

Why does this happen? Because a pilot is safe and scaling is scary. In a pilot, the stakes are low, the team is small and motivated, and everyone is excited to prove the idea. Scaling means changing how many people work, spending real money, dealing with resistance, and owning the result in front of the whole company. It is far more comfortable to run one more pilot than to make the hard decision to scale. So teams drift into an endless loop of testing, never committing.

The lesson is sharp and directly useful. A successful pilot is not a reason to celebrate and stop. It is a *trigger* to make the scaling decision on purpose. Before you start any pilot, decide in advance what "success" looks like and what you will do when you reach it. If you do not plan the scale-up before the pilot begins, you are very likely to end up in purgatory — a graveyard of small wins that never became anything. The cure is simple to say and hard to do: treat the pilot as step one of a scaling plan, never as the whole project.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A regional property-management company tested an AI tool that drafted replies to tenant maintenance requests. They ran it with one team of four people for two months. The results were clearly good: response time dropped by half, tenant satisfaction rose, and the team loved having the repetitive writing taken off their plate.

A pilot success. Now the fork in the road.

The company did not let it drift into purgatory. Before the pilot started, the owner had decided that if the numbers were good, they would scale to all three teams within a quarter. So they executed a scaling plan. First, they turned the pilot's informal habits into a written standard process: exactly how a request comes in, how the AI drafts, what the staff member checks, what gets sent, and what gets escalated. Second, they documented it in a short guide with real examples of good and bad drafts. Third, they trained the other two teams using that guide, with the pilot team's champion teaching them. Fourth, they set up a monthly review to watch the KPIs and catch problems as the tool met new types of requests.

Within one quarter, all three teams used the tool as the normal way of working. The win that started with four people now covered the whole company.

But here is the part that shows real maturity. During the scale-up, the tool hit request types the pilot team had never seen — legal notices, emergency maintenance, complaints. Some drafts were wrong. Because they had a monthly review and a written process, they caught these, added routing rules, and updated the guide. The scaling was not a clean rollout; it was a steady process of spreading and fixing at the same time. That is what scaling actually looks like.

A different company in the same city ran the same pilot with the same good results. They celebrated, wrote a happy report, and… nothing. Six months later, the tool was still used only by the original four people, and the other teams had never heard of it. Same pilot, same result, opposite outcome. The difference was that the first company had a scaling plan and the second did not.

# How to Do It

## 23.1 From pilot to standard process

A pilot is a test. A standard process is the agreed, repeatable way work gets done. The first job in scaling is to convert the former into the latter.

**Decide to scale on purpose.** Do not let scaling "happen" by accident or drift. At the end of the pilot, look at the numbers (Chapter 22) and make a deliberate decision: we are scaling this. Write down the decision, the scope (which teams, which tasks), and the timeline. A deliberate decision is what breaks pilot purgatory.

**Define the scope clearly.** What exactly are you scaling, and to whom? "The whole company" is too vague. "All three teams, for maintenance-request replies only, within one quarter" is clear. Start with the task the pilot proved, and the people closest to it. Do not try to scale to every task and every team at once — that is how you lose control.

**Turn the pilot's habits into an explicit process.** In the pilot, the small team developed informal ways of doing things — shortcuts, judgment calls, "ask Maria if unsure." These worked because the team was small and everyone knew each other. They will not survive when you add more people. So write the process down explicitly: the steps, the handoffs, the decision points, the escalation rules. Make the implicit explicit. This is the single most important scaling task.

**Assign clear ownership.** A standard process needs an owner — a person responsible for keeping it working, fixing it when it breaks, and updating it as things change. Without an owner, the process drifts and

decays. Name the owner before you scale. In a small business this may be you or a team lead. The point is that someone is accountable, not "someone."

**Scale in steps, not in one leap.** Roll out to one new team, watch it work, fix what breaks, then add the next team. Stepping lets you catch problems small. A big-bang rollout to everyone at once lets problems become big before you see them. Each step is a small, controlled expansion of the proven process.

## 23.2 Documentation

If it is not written down, it does not survive. Documentation is how a process outlives the people who invented it and how you train new people without starting from zero every time.

**Write the process guide.** A short, plain document that says exactly how the work is done: the steps, who does what, what the AI produces, what the human checks, what gets sent, what gets escalated, and what to do when something goes wrong. Keep it short and practical — a few pages, not a manual. A guide nobody reads is worse than no guide, so make it readable.

**Include real examples.** The most valuable part of any guide is worked examples: here is a real request, here is the AI draft, here is the corrected version, here is why. Examples teach faster than rules. Show a good one and a bad one, so people learn to recognize both.

**Document the edge cases.** The pilot taught you the unusual cases that need special handling. Write them down: "If the request is a legal notice, do not use the AI draft — route to a human." Edge cases are where tools fail, so documenting them is how you prevent failures when new people hit them later.

**Write the troubleshooting section.** When the tool breaks or produces something odd, what do people do? Who do they ask? How do they report a problem? A short troubleshooting section stops small confusions from becoming dead ends.

**Keep the documents alive.** Documentation rots. If the process changes and the guide does not, the guide becomes a lie that misleads people. Assign someone to keep the documents current, and update them whenever the process changes. A living document is valuable; a stale one is dangerous. Store it where people can find it — a shared drive, a wiki, a folder everyone knows about — not on one person's laptop.

## 23.3 Continuous training

As you scale, you are constantly bringing new people into the process. Training cannot be a one-time event at the start; it has to keep running as long as you are growing and as the process evolves.

**Train every new user properly.** Every person who joins the process needs the same hands-on training the pilot team got — on real work, with review and judgment emphasized, not just which buttons to press. Never let someone use the tool "by osmosis." Osmosis training produces confident mistakes.

**Use the pilot team as teachers.** The people who ran the pilot are your best trainers. They know the tool, the process, and the edge cases. Have them teach the new teams. This spreads skill and also honors the pilot team's expertise, which helps their morale and makes the change feel like growth, not replacement.

**Retrain when the process changes.** When you update the process — a new routing rule, a new edge case, a tool update — retrain the

people affected. A change that is not trained is a change that will be done wrong. Build retraining into every process update.

**Keep a short refresher available.** Not everyone learns at the same pace. Keep a short refresher — a one-page cheat sheet, a short recorded walkthrough — that anyone can revisit when they get rusty or uncertain. This lowers the cost of help and keeps people moving.

**Train for the new, not just the same.** As you scale and find new uses (section 23.5), you will need to train people on those too. Continuous training means training keeps pace with the process, always a step ahead of confusion.

## 23.4 Updates and maintenance

An AI tool is not a machine you install and forget. It is more like a garden than a statue. It needs regular tending, or it slowly goes to seed. Maintenance is the part of scaling that most people forget, and it is the part that decides whether the win lasts.

**Tools change under your feet.** AI tools get updated by their makers. A model update can change how the tool behaves — sometimes for the better, sometimes in ways that break your process. A tool that worked perfectly in January may behave differently in June after an update you did not ask for and did not test. Maintenance means watching for these changes and re-checking that your process still works after each one.

**Data drifts.** The world changes. Customer questions change, new products appear, new regulations arrive. A tool trained on last year's reality can quietly fall behind this year's. Maintenance means watching the output for signs of drift — answers that used to be right and now are subtly wrong — and correcting the process or the tool's inputs.

**Tune and refine.** As people use the tool at scale, you learn what needs adjusting: a prompt that produces better drafts, a routing rule that catches more edge cases, a template that saves more time. Maintenance is the ongoing small tuning that makes the tool better over time, not just keeps it running.

**Budget for maintenance.** Maintenance is not free. It takes time and sometimes money — the hours to watch for updates, test them, retrain, and tune. Build this into your cost model (Chapter 16) from the start. A tool that looks profitable without maintenance costs often stops being profitable once you add the real tending it needs.

**Assign the tending.** Like the process owner, the maintenance needs a named person responsible for watching updates, testing changes, and keeping the tool healthy. If nobody owns maintenance, it does not happen, and the tool quietly decays until it is worse than doing the work by hand.

**Plan for the long term.** Ask: what happens to this process if the tool's price doubles, the vendor changes terms, or the tool is discontinued? Maintenance includes having a fallback plan so you are not stranded if your tool disappears. This connects to the vendor-risk lessons covered elsewhere in the book.

## 23.5 New opportunities

Scaling one win well teaches you how to find and land the next one. A mature approach to AI is not one big transformation; it is a chain of small wins, each one making the next easier to see and to do.

**Harvest what you learned.** After scaling, ask: what did we learn that applies elsewhere? The process you built for tenant requests might work for supplier inquiries. The review habit you trained might

improve another team's work. The skills your team gained are reusable. Look for the next task that resembles the one you just mastered.

**Let users suggest new uses.** The people using the tool every day see opportunities you cannot. They will say, "this could also do X." Create a simple way for them to suggest new uses, and take the good ones seriously. The best next opportunities often come from the front line, not from a strategy meeting.

**Run the same loop again.** Each new opportunity gets the same treatment: a small pilot, measure it, and if it works, scale it with a plan. You now have the muscle for this loop, so each cycle is faster and safer than the last. This is the improvement engine — a repeating cycle of pilot, measure, scale, learn, repeat.

**Do not chase every shiny thing.** New opportunities are tempting, and not all are worth it. Use the same discipline you used the first time: pick the one with the best impact and the clearest fit, not the one that sounds most exciting. A focused chain of wins beats a scattered pile of half-started experiments. The impact-versus-ease thinking in Chapter 12 helps you choose which opportunity to take next.

**Keep the engine running.** The goal is a company that can keep finding, testing, and scaling small AI wins indefinitely. That capability — not any single tool — is the real competitive advantage. Tools come and go; the ability to keep improving is what lasts.

## Ethics and Responsibility

Scaling multiplies everything, including your ethical duties. A small mistake in a pilot becomes a large mistake at scale.

**Fix the cracks before you multiply them.** Scaling amplifies flaws. If the pilot has a bias, an error pattern, or a gap, scaling spreads it to

hundreds of cases. Check for cracks before you scale, and keep checking as you go. A process that is good enough for four people is not automatically good enough for forty.

**Do not scale a process you cannot supervise.** If you scale faster than you can review and maintain, you lose control of quality and risk. Scale at the speed you can tend. A smaller, well-supervised rollout beats a bigger, unsupervised one every time.

**Keep the human in the loop at scale.** As volume grows, the temptation to remove human review rises, because review "slows things down." Resist it. The human check is what catches the confident mistakes, and it matters more at scale, not less. Never automate away the human responsibility for the output.

**Honor the people who made the pilot work.** The pilot team built something valuable. When you scale, give them credit, give them the teacher role, and share the gains. If you take their work and discard them, you teach everyone else that contributing is risky, and you will struggle to get volunteers for the next pilot.

**Be honest about what is scaled and what is not.** Do not claim a tool is "live across the company" when it is really only working well in one team. Report the real state of scaling honestly, including where it is still rough. Overstating progress hides problems and misleads stakeholders.

**Keep the improvement engine honest.** As you chain wins together, keep measuring each one truthfully (Chapter 22). The temptation to declare victory early grows as the engine speeds up. Resist it. Each new opportunity earns its place through honest measurement, not momentum.

## Mistakes to Avoid

**Pilot purgatory.** Running pilot after pilot and never scaling. Decide to scale on purpose, before the pilot even starts.

**Treating the pilot as the finish line.** Celebrating the pilot and stopping. The pilot is step one of scaling, not the whole project.

**Scaling without a plan.** Hoping the new way spreads by itself. It does not. Scale with a written scope, timeline, and owner.

**Leaving the process informal.** Keeping the pilot's "ask Maria" habits. They collapse when you add people. Write the process down.

**No documentation.** Not writing the guide, the examples, and the edge cases. The process dies with the people who invented it.

**Osmosis training.** Letting new users learn by watching instead of hands-on practice. Produces confident mistakes at scale.

**No retraining on changes.** Updating the process but not the people. The change gets done wrong.

**Forgetting maintenance.** Treating the tool as install-and-forget. It drifts, decays, and quietly gets worse.

**No maintenance budget.** Leaving out the cost of tending. The tool stops being profitable when the real maintenance cost appears.

**Scaling faster than you can supervise.** Rolling out faster than you can review and fix. You lose control of quality and risk.

**Removing the human check for speed.** Dropping review because it "slows things down." The human check matters more at scale.

**Chasing every shiny new tool.** Starting many experiments and finishing none. A focused chain of wins beats a scattered pile.

**Overstating the rollout.** Claiming the tool is everywhere when it only works in one team. Report the real state honestly.

## Practical Exercise

### 23.6 Exercise: a scaling plan

Write a one-page scaling plan for a pilot you have run (or would run). Fill in these eight parts.

**1. The decision and the trigger.** Write: "If the pilot reaches [specific success numbers], we will scale it." Name the exact numbers that trigger scaling, so the decision is made in advance, not drifted into.

**2. The scope.** What exactly are you scaling, and to whom? Name the task and the teams, and set a timeline. Be specific, not "the whole company."

**3. The standard process.** Write the explicit steps: how work comes in, what the AI does, what the human checks, what gets sent, what gets escalated. Make the pilot's informal habits explicit.

**4. The owner.** Name the person responsible for keeping the process working and the tool healthy. One name, accountable.

**5. The documentation plan.** List what you will write: the process guide, the worked examples, the edge cases, the troubleshooting section. Say where it will be stored so people can find it.

**6. The training plan.** How will you train each new team? Who teaches them (name the champion)? How will you retrain when the process changes? How will you keep a refresher available?

**7. The maintenance plan.** Who watches for tool updates and data drift? How often will you review the KPIs during scale-up? What is the fallback if the tool changes or disappears?

**8. The next-opportunity loop.** How will you collect new-use suggestions from users, and how will you choose the next thing to pilot? Name the simple mechanism.

Read the page back and ask: if the pilot succeeds, does this page actually get us to a scaled, maintained, documented process, or could we still drift? If there is any room to drift, tighten the plan. Keep it and run the scaling against it. This page is the difference between a win that spreads and a win that dies on one desk.

## Checklist

### 23.7 Scaling checklist

Before and during scaling, check these.

- ☐ **You decided to scale on purpose**, with a pre-set success trigger.
- ☐ **You defined a clear scope** — the specific task and teams, with a timeline.
- ☐ **You turned the pilot's habits into an explicit written process.**
- ☐ **You named a process owner** who is accountable for keeping it working.

- ☐ **You scale in steps**, one team at a time, not one big-bang rollout.
- ☐ **You wrote the process guide** — short, plain, readable.
- ☐ **You included worked examples** of good and bad output.
- ☐ **You documented the edge cases** and how to handle them.
- ☐ **You wrote a troubleshooting section** and stored docs where people can find them.
- ☐ **You keep the documents alive** and update them when the process changes.
- ☐ **You train every new user hands-on**, not by osmosis.
- ☐ **You use the pilot team as teachers** and honor their contribution.
- ☐ **You retrain people whenever the process changes.**
- ☐ **You keep a refresher** (cheat sheet or walkthrough) available.
- ☐ **You watch for tool updates and data drift** and re-check the process after each.
- ☐ **You budget for maintenance** (time and money) in your cost model.
- ☐ **You assigned the maintenance tending** to a named person.
- ☐ **You have a fallback plan** if the tool changes price, terms, or disappears.
- ☐ **You keep the human review in the loop** at scale, never removed for speed.
- ☐ **You collect new-use suggestions** and choose the next opportunity with discipline.
- ☐ **You report the real scaling state honestly**, not an overstated rollout.

If a box is empty, the win is at risk of dying on one desk. Fill it. A small win with a scaling plan becomes a company-wide capability; a small win without one becomes pilot purgatory.

## Key Takeaways

- A successful pilot is not the finish line — it is the trigger to scale on purpose, or you end up in pilot purgatory running endless tests that never become real change.
- Scaling means turning the pilot's informal habits into an explicit, documented, owned process, and rolling it out in controlled steps, not one big leap.
- An AI tool is a garden, not a statue: it needs ongoing maintenance — watching for updates and drift, tuning, and a budget and owner for the tending — or it quietly decays.
- Scale only as fast as you can supervise and train, and never remove the human review for speed; the human check matters more at scale, not less.
- The real advantage is a repeating engine — pilot, measure, scale, learn, repeat — that turns one small win into a chain of them, each making the next easier to find.

# Chapter 24 — Common Mistakes and How to Avoid Them

## In Simple Words

Most AI projects fail for the same handful of reasons. Not because the technology is weak, but because of predictable mistakes that people make over and over. The good news is that once you know the mistakes, you can avoid almost all of them. This chapter is a guided tour of the big ones, with the fix for each.

Think of it as a list of landmines. Each one is easy to step on if you do not know it is there, and easy to walk around if you do. The goal is not to scare you away from AI. It is to make you the kind of business owner who does not blow up their own project by accident.

The six big mistakes are: automating the wrong process, underestimating data and people, trusting the AI too much, starting too big, ignoring regulations and security, and not measuring results. Each one is common, each one is costly, and each one has a clear way to avoid it. We take them one at a time, explain why it happens and what it costs, and point you to the chapter that covers the fix in full.

A simple image to carry: adopting AI is like setting out on a long walk across country. Most people do not fail because the road is impossible. They fail because they picked the wrong destination, packed too little water, trusted a faulty map, tried to walk too far on day one, ignored the weather, and never checked whether they were heading the right way. The mistakes are ordinary and avoidable. This chapter is the pre-walk safety briefing.

Read it as a checklist against your own plans. If you find yourself doing any of these, stop and fix it before you spend more money or burn more trust.

## A Bit of History

**1980s: the expert-system collapse taught the wrong-process lesson.** During the expert-system boom, companies poured money into encoding the rules of human experts. Many projects failed because they tried to automate tasks that were too messy, too rare, or too dependent on judgment to encode. The systems were brittle and expensive to update, and the market collapsed. The lesson: automating the wrong thing wastes a fortune, no matter how good the technology.

**1990s: "garbage in, garbage out" became a mantra.** As businesses computerized, they learned that a system fed bad data produces bad results, no matter how clever the software. This old computing saying became central to every data project. It is the ancestor of today's "underestimating data" mistake.

**1990s–2000s: the big-bang failure pattern.** Enterprise software projects of the 1990s and 2000s were famous for going "big bang" — replacing everything at once, on a grand plan, over years. Many ran over budget, over time, and failed outright. The pattern was so common that "big-bang implementation" became a cautionary term. The fix that emerged: start small, prove value, then grow. This is the ancestor of "starting too big."

**2000s: automation complacency.** Researchers studying automated systems — autopilots, automated monitoring — found a surprising failure: when a system works well most of the time, humans stop paying attention and trust it too much. Then it fails on a rare case

and nobody catches it. This is called **automation complacency** or **automation bias**, and it is exactly the "trusting the AI too much" mistake. It was documented long before chatbots and is now more relevant than ever.

**2010s: regulation catches up.** As data breaches and misuse made headlines, governments began writing strict rules — the GDPR in Europe being the biggest. Companies that had treated data casually found themselves facing heavy fines and legal risk. The lesson: ignoring the rules is not a shortcut; it is a liability waiting to be triggered.

**2020s: the same old mistakes, new technology.** The current AI wave repeats every one of these mistakes at speed. Cheap, impressive tools make it easy to start big, trust too much, skip the data work, ignore the rules, and skip the measuring. The technology is new; the failure modes are old and well-documented. Knowing the history is the cheapest defense there is.

The arc: every generation of business technology has made the same handful of mistakes. None of them are new. All of them are avoidable if you have seen them before.

## Curiosity

### 24.7 The Eliza effect: why we trust machines more than they deserve

One reason people trust AI too much goes back to a 1966 chat program called ELIZA and the human habit named after it — the **Eliza effect**: we readily grant understanding and feelings to a machine that merely imitates them, and we trust its output more than it deserves.

The full story of ELIZA, and why it matters for how you design and supervise any AI tool, is told in Chapter 1 — A Short History of AI, the canonical home of the Eliza effect. The one-line callback here: a fluent, polite machine makes us over-trust it, and that habit is the root of mistake 24.3 below.

## A Real Business Example

*The following is an illustrative composite of common real-world patterns, not a single named company.*

A retail company wanted to "use AI," so it did almost everything on this list wrong, and then almost everything right.

The wrong version: the owner read about AI, got excited, and decided to "transform customer service." Without picking a specific problem, they bought a chatbot and rolled it out to the whole customer base at once. They fed it a pile of old, messy documents and hoped for the best. They turned off the human review because "the AI is faster." They never checked whether it was giving correct answers. Within weeks, the chatbot was confidently telling customers wrong return policies and wrong prices. Customers complained. The company had no idea until the complaints piled up. The project was a mess, and the owner concluded, "AI does not work."

The right version: after the mess, the owner restarted with discipline. They picked one small, clear problem — answering the ten most common "where is my order?" questions, which were eating up staff time. They checked the data first and found they had clean order-tracking data that could answer those questions reliably. They started with a tiny pilot on just those questions, with a human reviewing every answer. They measured the result: response time dropped, satisfaction

held, and the answers were correct because the data was good and the scope was narrow. Only then did they expand, step by step, keeping the human review on the harder cases.

The technology was the same. The first attempt failed because of six mistakes. The second worked because each mistake was avoided. The lesson is not "AI is risky." The lesson is "the mistakes are the risk, and they are avoidable."

## How to Do It

### 24.1 Automating the wrong process

The most expensive mistake is spending money to automate something that was never worth automating. A fast wrong process is still wrong, and now it is wrong at scale.

**Why it happens.** Excitement. You have a tool and you want to use it, so you point it at whatever is in front of you instead of choosing carefully. Or you automate a process that is already fine, saving time nobody needed saved, while the real bottleneck sits untouched.

**What it costs.** Money spent on a tool that delivers little, plus the opportunity cost of the real problem you did not solve. Worse, a badly-chosen automation can make things worse — automating a broken process just produces broken output faster and hides the breakage.

**The fix: choose before you buy.** Before any tool, pick the process by two tests. First, *impact*: does this process actually matter? Does it cost real time, real money, or real customer happiness? Second, *fit*: is the process suitable for automation — repetitive, rule-based or example-based, with data available? The impact-versus-ease matrix in Chapter 12 — Where AI Can Help Your Business is the tool for this. Use

it. Pick the high-impact, high-fit target, not the first thing you thought of.

**Fix the process before you automate it.** If a process is broken, fix it first, then automate the fixed version. Automating a mess gives you an automated mess. Sometimes the best first step is to simplify or remove a step entirely, not to automate it.

## 24.2 Underestimating data and people

AI runs on two things: data and people. Both are usually harder and more important than the model itself, and both are routinely underestimated.

**The data mistake.** AI is only as good as the data you feed it. Messy, incomplete, outdated, or biased data produces messy, incomplete, outdated, or biased output — no matter how good the tool. People imagine the AI will "figure it out" from whatever they have. It cannot. If your records are a mess, the AI inherits the mess. The full treatment of data quality is in Chapter 14 — Data: The Raw Material. The short version: check your data before you expect good output, and fix the data before you blame the AI.

**The people mistake.** As Chapter 15 shows, a tool that works in a test can sit unused because nobody was told why it exists, nobody was trained, and nobody felt safe. People are not a side note; they are the reason projects live or die. Underestimating the people side — the training, the fear, the trust, the change management — is one of the most common causes of failure. The change-management skills are in Chapter 21. Budget for the people work as seriously as you budget for the software.

**Why it happens.** Both data and people work are slow, unglamorous, and boring compared to the exciting tool. It is tempting to skip them and get to the demo. But skipping them is skipping the foundation. The building falls.

**The fix.** Treat data and people as the main project, with the tool as a part of it, not the other way around. Spend real time cleaning and checking the data. Spend real time training, communicating, and supporting the people. These are not overhead; they are the work.

## 24.3 Trusting the AI too much

Modern AI produces fluent, confident, well-written output. That fluency tricks us into trusting it more than it deserves. This is automation bias, and it is the single most dangerous mistake in the list, because it turns a tool into an unwatched decision-maker.

**Why it happens.** A confident, well-written answer *feels* correct. Our brains confuse "sounds good" with "is true." Add the Eliza effect — our habit of granting understanding to a polite machine — and it is easy to stop checking. If the tool is right 95% of the time, the 5% it is wrong slips through because nobody is watching.

**What it costs.** Wrong answers that reach customers. Wrong figures in reports. Wrong decisions made on confident-but-false output. The damage is worse precisely because the output looked trustworthy, so nobody questioned it until it was too late. Remember the lesson from Chapter 1: fluency is not truth.

**The fix: keep the human in the loop.** Never let AI output go out unreviewed, especially to customers or into decisions that matter. Set a firm rule: the AI drafts, a human checks and sends. Teach people *how*

to check — what to look for, what a wrong answer looks like — not just to press a button. The review skill is more important than the tool.

**Watch for confident nonsense.** AI can be confidently wrong. The more confident and fluent the output, the more carefully you should check it, not less. Train your instinct to be suspicious of smooth answers on things that matter. The trustless mindset in Chapter 7 — Trustless: Trust Without Trusting is the right frame: verify, do not trust by default.

**Set escalation rules.** Decide what the AI may do on its own and what must go to a human. High-stakes, unusual, or ambiguous cases go to a person. The AI handles the routine; the human handles the important and the strange.

## 24.4 Starting too big

The big-bang rollout — changing everything at once on a grand plan — is a classic failure. It is tempting because it feels ambitious, but it is the surest way to lose control.

**Why it happens.** Ambition and impatience. You want the big transformation now, so you go wide and fast. Or you want to impress, so you pick the grandest possible scope. Big feels bold; in practice it is fragile.

**What it costs.** Big projects are hard to control, expensive to fix when they go wrong, and slow to show value. A flaw that would be small in a pilot becomes a crisis when it is multiplied across the whole company before you see it. Big-bang projects often run over budget and over time, and many fail outright.

**The fix: start small and prove value.** Pick one narrow task, one team, one clear goal. Run a pilot. Measure it. If it works, scale in steps

(Chapter 23). Small is not timid; small is how you learn cheaply and keep control. Each small win builds the skill and the evidence for the next step. The narrow-wins approach is the same one Chapter 1 recommends from the history: AI wins one specific thing at a time, and so should you.

**Ambition is fine; sequencing is the discipline.** You can have a big vision. Just reach it through a chain of small, proven steps, not one giant leap. The destination can be large; the first step must be small.

## 24.5 Ignoring regulations and security

AI touches your data, your customers' data, and your decisions. That means it touches the law and your security. Ignoring both is not a shortcut; it is a loaded liability.

**Why it happens.** Regulations and security feel slow, complex, and far away — until they are not. It is tempting to move fast and deal with compliance "later." Later often arrives as a fine, a breach, or a legal fight.

**What it costs.** Heavy fines for breaking privacy rules. A data breach that exposes customer information and destroys trust. Legal liability for decisions the AI made that broke a rule. The cost of ignoring these is far larger than the cost of handling them properly, and it lands all at once.

**The fix: know the rules that apply to you.** If you handle personal data in or about Europe, the GDPR applies — covered in Chapter 10 — Privacy and GDPR. If your AI use falls under the EU AI Act, its duties apply — covered in Chapter 5 — Rules and Legal Responsibility. You do not need to be a lawyer, but you do need to know which rules touch you and to follow them. Get advice where it matters.

**Take security seriously from day one.** AI tools often handle sensitive data. Treat that data with the same care you would treat cash in a safe. The security threats specific to the AI era — data leaks, prompt injection, vendor exposure — are covered in Chapter 6 — Cybersecurity in the AI Era. Do not bolt security on later; build it in from the start.

**Compliance is not optional and not a finish line.** Rules change, and your use of AI changes. Compliance is ongoing, like maintenance. Build it into how you operate, not into a one-time check you skip.

## 24.6 Not measuring results

If you do not measure, you cannot tell whether the AI helped, hurt, or did nothing. You are driving blind, and you will keep spending money on something you cannot evaluate.

**Why it happens.** Measuring feels like extra work, and it can show bad news. It is easier to assume it is working than to check. Many projects launch with excitement and never look at a number again.

**What it costs.** You cannot prove value, so you cannot make good decisions about expanding or stopping. You keep spending on things that may not work. You lose the lesson that would make the next project better. A project you never measured is a project whose outcome was random.

**The fix: measure from the start.** Set a baseline before you launch, track a few honest KPIs, and review them regularly. The full method is in Chapter 16 — Goals, Costs, and Return on Investment and the dashboard practice is in Chapter 22 — Measuring Results and ROI. Do not relearn it here; just commit to doing it. Measure the truth, including the bad weeks, and use it to decide keep, correct, or stop.

**Measure the things that matter, not the things that flatter.** Avoid vanity metrics like usage counts. Measure time, errors, cost, and satisfaction. A number that only shows the good side is a number that is lying to you.

## Ethics and Responsibility

Every mistake on this list has an ethical edge, because each one can hurt real people — customers, employees, or the business itself.

**Trusting the AI too much is an ethical failure, not just a practical one.** When unreviewed AI output reaches a customer and is wrong, you have harmed someone. You remain responsible for what your AI sends out, no matter how confident it sounded. Keeping the human in the loop is a duty, not a preference.

**Ignoring regulations is a duty you owe to people, not a box to tick.** Privacy rules exist to protect real people's information. Following them is respecting those people, not avoiding a fine. The fine is the least of it; the harm to the person whose data you mishandled is the point.

**Not measuring is a failure of honesty.** If you cannot say whether your AI works, you cannot honestly tell your customers, your staff, or your partners what you are doing. Measurement is part of being truthful about your own business.

**Starting too big can hurt your own people.** A failed big-bang project wastes the company's money and the team's trust and effort. Ambition that ignores the safe path can damage the very people you are trying to help. Protect them by going in safe steps.

**The common thread: stay awake and stay responsible.** Every mistake here is a form of sleepwalking — doing the easy thing instead of

the careful thing. The ethical stance is the awake stance: choose the process deliberately, respect the data and the people, verify the output, go in safe steps, follow the rules, and measure the truth.

## Mistakes to Avoid

This chapter *is* the mistakes list, so instead of repeating it, here is the meta-mistake that hides all the others:

**The meta-mistake: treating AI as a product you buy instead of a change you manage.** All six mistakes come from the same root — treating AI like a purchase that works on its own, rather than a change that needs choosing, preparing, supervising, and measuring. When you buy a tool and expect it to deliver by itself, you slide into every trap at once: wrong process, bad data, blind trust, too big, no rules, no measurement.

**The fix for the meta-mistake: treat AI as a managed change.** Choose the target on purpose. Prepare the data and the people. Supervise the output. Start small and scale with a plan. Follow the rules. Measure the result. This is the whole book in one sentence, and it is the opposite of the sleepwalking that causes every failure here.

A second meta-mistake worth naming: **concluding "AI does not work" from a failed project.** When a project fails, the cause was almost always one of the six mistakes, not the technology. Do not throw away the opportunity because you stepped on a landmine. Learn which landmine it was, avoid it next time, and try again with discipline. The tools work when you avoid the mistakes.

## Practical Exercise

### 24.8 Exercise: audit your plan against the six mistakes

Take any AI project you are planning or running, and audit it against the six mistakes. For each, answer honestly with a yes/no and one line of evidence.

**1. Wrong process?** "Have I chosen this target by impact and fit, not just excitement?" If no, go back and use the impact-versus-ease test (Chapter 12).

**2. Underestimated data and people?** "Is my data clean enough, and have I budgeted real time for training and change?" If no, fix the data and plan the people work before going further.

**3. Trusted the AI too much?** "Is there a human review on everything that matters, and do people know what a wrong answer looks like?" If no, add the review rule now, before more output goes out.

**4. Started too big?** "Am I starting with one narrow task and one team, or going wide?" If wide, shrink to a pilot and prove value first.

**5. Ignored regulations and security?** "Do I know which rules apply to me, and is the data protected?" If unsure, find out and fix it before you handle more sensitive data.

**6. Not measuring?** "Do I have a baseline and a few honest KPIs I actually watch?" If no, set them up now, even if you have to reconstruct the baseline.

For every “no,” write one concrete action to fix it, with a date. Do not move forward on the project until the “no” answers are fixed. This ten-minute audit catches most of the expensive mistakes before they cost you. Keep the answers and re-check them at your review date.

## Checklist

### 24.9 Anti-mistake checklist

Run this against any AI project before and during.

- ☐ **You chose the target by impact and fit**, not by excitement or habit.
- ☐ **You fixed or simplified the process before automating it.**
- ☐ **You checked the data quality** and fixed the data before expecting good output.
- ☐ **You budgeted real time and money for the people side** — training, communication, change.
- ☐ **You keep a human review on everything that matters** — the AI drafts, a human sends.
- ☐ **You trained people to spot wrong output**, not just to press buttons.
- ☐ **You treat confident, fluent output with suspicion** and verify what matters.
- ☐ **You set escalation rules** — routine to the AI, important and strange to a human.
- ☐ **You started small** — one narrow task, one team, one clear goal.
- ☐ **You scale in steps** with a plan, not one big-bang rollout.
- ☐ **You know which regulations apply to you** (GDPR, EU AI Act) and you follow them.

- ☐ **You protect sensitive data** with real security from day one, not bolted on later.
- ☐ **You treat compliance as ongoing**, not a one-time check.
- ☐ **You measured a baseline** before launch.
- ☐ **You track honest KPIs** (time, errors, cost, satisfaction), not vanity metrics.
- ☐ **You review the numbers regularly** and use them to decide keep / correct / stop.
- ☐ **You treat AI as a managed change**, not a product that delivers by itself.
- ☐ **You did not conclude "AI does not work"** from a mistake that was yours to avoid.

If a box is empty, you are standing on a landmine. Step around it before you spend more money or trust. Every mistake on this list is old, common, and avoidable — once you know it is there.

## Key Takeaways

- Most AI failures come from six old, predictable mistakes — wrong process, weak data and people, too much trust, too big, ignored rules and security, no measurement — and all are avoidable once you know them.
- The single most dangerous mistake is trusting fluent AI output too much; keep a human review on everything that matters, because fluency is not truth.
- The root of every mistake is treating AI as a product you buy instead of a change you manage — choose, prepare, supervise, scale in steps, follow the rules, and measure.
- Start small and prove value before you go wide; a narrow win you can control beats a grand rollout you cannot.

- Do not conclude “AI does not work” from a failed project — name which mistake caused it, fix it, and try again with discipline.

# Chapter 25 — Administration and Finance

## In Simple Words

Administration and finance are where AI quietly saves the most hours in a small business. Not because the work is hard, but because it is repetitive. Invoices, bank lines, expense claims, monthly reports, cash forecasts — the same tasks, over and over, every single week. Repetition is exactly what software is good at, and AI adds the ability to read messy documents and spot patterns a person might miss.

Think of your back office as a room full of paper that never stops arriving. Every invoice is a small piece of paper that someone must read, type into a system, match against an order, check for errors, and file. Multiply that by hundreds or thousands of pieces a month, and you see where the days go. AI does not get tired, does not lose focus at 4 p.m., and does not mind doing the same task for the thousandth time.

This chapter covers four jobs: reading invoices and documents, matching (reconciling) records, producing reports automatically, and forecasting cash. Each is a place where a small business can save real time and make fewer mistakes.

One important idea before we start: AI in finance is a *draftsman*, not the *decision-maker*. It reads, sorts, matches, and suggests. A person still approves the money movement, signs off the report, and owns the result. Keep a human in the loop for anything that touches real cash. The method for judging whether any of this is worth the cost lives in

Chapter 16 — Goals, Costs, and Return on Investment; this chapter shows you what to automate and how.

## A Bit of History

**1960s–1980s: the spreadsheet revolution.** The first big change in back-office work was the electronic spreadsheet. Before it, accountants kept ledgers by hand and a single change could mean hours of recalculation. Spreadsheets made calculation instant. This was the first time software took over a core finance task, and it set the pattern: automate the arithmetic, keep the human in charge of the meaning.

**1990s: OCR and document scanning.** Optical Character Recognition — software that reads printed text from a scanned image and turns it into editable text — arrived in business tools. Suddenly a paper invoice could become data instead of a stack of paper. Early OCR was slow and made mistakes, so a person still checked everything. But the door was open: paper could become digital.

**2000s: rules-based automation and RPA.** Robotic Process Automation — software "bots" that follow fixed rules to move data between systems — became popular. A bot could copy an invoice total from one screen and paste it into another. This worked on predictable tasks but broke the moment a document looked different: fast but brittle.

**2010s: machine learning reads documents.** Machine learning — software that learns patterns from many examples instead of following fixed rules — changed document reading. Instead of telling the computer exactly where to look, you showed it thousands of invoices, and it learned to find the vendor, the date, and the total on its own, even when the layout changed.

**2020s: large language models and agents.** Large language models — AI trained on huge amounts of text — can now read an invoice, understand what it says, and draft a reply to the vendor. They handle unusual formats that broke older tools. The newest step is the *agent*: AI that takes a whole task, like "process this vendor email," and carries it through several steps on its own, with a human reviewing the result. The Elanco example later in this chapter is exactly this kind of agent.

The arc: from hand-written ledgers, to instant spreadsheets, to scanned text, to rule-following bots, to AI that reads and reasons. Each step took more of the repetitive work off human hands and left humans to do the judging.

## Curiosity

## 25.5 The "human middleware" that AI replaced

For years, Elanco's procure-to-pay team — the people who handle questions and paperwork between buying goods and paying for them — worked as what the company called "human middleware." They manually answered over 30,000 queries a year, and each one took more than ten minutes. Ten minutes here, ten minutes there, multiplied by thirty thousand, is a mountain of hours spent moving information between systems by hand.

That story, with the real numbers and the two-layer AI system that replaced it, is told in full in "A Real Business Example" below. The curiosity here is the phrase itself: "human middleware." It describes a job where a person exists only to carry data from one place to another. That is the kind of job AI is best at removing — and the kind you should look for in your own back office.

## A Real Business Example

**Elanco: a two-layer AI ecosystem for procure-to-pay, cutting query time by about 99%.**

Elanco is a global animal-health company that makes medicines and treatments for pets and farm animals. Its procure-to-pay team — the group that handles everything between ordering goods and paying the vendor — had a chronic problem. The team spent its days as "human middleware," manually answering over 30,000 queries a year. Each query took more than ten minutes: someone would search the finance system, check the order, look up the vendor, and type an answer. The work was slow, repetitive, and error-prone.

According to The Hackett Group, which named this project a winner in the Purchase-to-Pay category of its 2026 Hackett Innovation Awards (announced in a Business Wire release on 24 June 2026), Elanco solved it with a two-layer agent-based AI ecosystem built on ElancoGPT, the company's own secure AI platform. The two layers were:

- **Layer one — AskSAP.** Employees could ask questions in plain language and get answers pulled from the company's SAP enterprise system (the software that runs its finance and operations). Instead of opening several screens and hunting for a number, a person just asked, "What is the status of this purchase order?" and got an answer.
- **Layer two — the procure-to-pay agent.** This agent automatically scans incoming vendor emails, works out what the vendor wants (the "intent"), cross-checks the request against live data in the enterprise system, and drafts a reply. A human employee then reviews the draft before it is sent.

The result was dramatic. Query resolution time dropped to **under 10 seconds** — what The Hackett Group described as **a 99% reduction** from the previous ten-plus minutes. The company also reported that the system eliminated roughly **30% to 40%** of the manual procure-to-pay queries altogether, because many questions simply stopped being asked once employees could find answers themselves through AskSAP.

Two things are worth noticing. First, the human is still there: the agent *drafts*, the employee *reviews*. Elanco did not let AI send money or replies on its own. Second, the biggest win came from giving people a faster way to find answers, which removed the need for the query in the first place. That is a pattern you can copy: the best automation often removes the request, not just the work.

A note on the source: this is a Hackett Innovation Award case, not a Microsoft customer story. The figures above come from The Hackett Group's award announcement. Treat the percentages as the company's reported results, and remember that your own numbers will differ depending on your systems and volume.

# How to Do It

## 25.1 Invoices and documents

Reading invoices and other documents by hand is one of the biggest time sinks in a small business. AI document processing can read an invoice, pull out the key fields, and put them where they belong.

**What the AI extracts.** From a typical invoice, the software pulls the vendor name, the invoice number, the date, the line items, the tax, and the total. It reads PDFs, scanned paper, and email attachments.

Modern tools handle many layouts without being told each one in advance.

**How it works in practice.** You point the tool at a folder of incoming invoices, or connect it to your email. It reads each one, extracts the fields, and either enters them into your accounting system or puts them in a queue for a person to confirm. The more invoices it sees, the better it gets at your specific vendors.

**The human check.** Do not let the tool post invoices to your books without review, at least at first. Set it to extract and flag, and have a person approve. Watch the error rate (as Chapter 22 advises). Over time, as accuracy proves itself, you can let the routine, low-value invoices post automatically and route only the unusual ones to a human.

**Where it saves most.** High volume and repetitive formats. If you process hundreds of invoices a month, the saving is large and obvious. If you process ten, the tool may not pay for itself. Match the tool to your volume.

**Watch the high-stakes fields.** The total and the tax are the two fields that must be right, because a wrong number here costs real money. Double-check these until you trust the tool; vendor name and date are lower risk if wrong.

## 25.2 Reconciliations

Reconciliation means matching two sets of records to make sure they agree. The classic example is matching your bank statement against your accounting ledger. If they match, your books are correct. If they do not, something is missing, duplicated, or wrong, and you must find it.

**Why it is painful by hand.** Matching line by line is slow and boring, and boredom causes mistakes. A person scanning hundreds of bank lines will eventually miss a duplicate payment or a missing receipt.

**How AI helps.** AI reconciliation tools match records automatically by comparing amounts, dates, and reference numbers. They link a bank line to the matching invoice, flag the ones that match cleanly, and surface only the mismatches for a human to investigate. Instead of checking everything, you check the exceptions.

**The exception model.** This is the key idea: let the software handle the 95% that matches, and bring the 5% that does not to a person. The person's job changes from "match every line" to "resolve the few that do not match." That is a smaller, more interesting job, and it is where human judgment actually adds value.

**Common mismatches to expect.** A payment that appears twice (a duplicate), a bank fee nobody recorded, a receipt missing for a card payment, a payment made to the wrong vendor. The tool flags these; you resolve them. Over time you learn your own patterns and can add rules to catch recurring ones automatically.

**Keep an audit trail.** Whatever the tool does, make sure it records what it matched and what it flagged. When your accountant or an auditor asks how a figure was reached, you need a clear trail. Good tools produce this; ask before you buy.

## 25.3 Automatic reports

Monthly and weekly reports — profit and loss, sales by product, expenses by category — are another place where AI saves hours. Instead of a person pulling numbers into a spreadsheet every month, the report can build itself.

**Scheduled reports.** Set the report to generate on a fixed schedule (every Monday, the first of the month) and deliver it to your inbox or a shared folder. The numbers are pulled from your live systems, so the report is always current. Nobody has to remember to make it.

**Plain-language summaries.** Modern AI can read the numbers and write a short summary in plain words: "Revenue rose 8% this month, driven by product X; expenses rose 3%, mostly in shipping." This turns a table of figures into a sentence you can actually read and act on. It is like having a junior analyst write the commentary for you.

**Ask questions in plain language.** Some tools let you ask, "What were our top five customers this quarter?" and get an answer without writing a formula. This is useful for the ad-hoc questions that used to mean "I will look into it later" and then never happened.

**Do not skip the human read.** An automatic report is a starting point, not a finished decision document. Read the summary, check that the numbers make sense, and add your own judgment before you act on it or share it. AI can summarize confidently and still be wrong if the underlying data is messy. Garbage in, confident garbage out.

**Standardize the format.** Once you settle on a report layout, keep it stable. A consistent format is easier to read month over month and easier to spot when something looks off. Change the format only deliberately, not every time.

## 25.4 Cash-flow forecasts

Cash-flow forecasting means predicting how much money will be in the bank in the coming weeks and months. It is different from profit. A business can be profitable on paper and still run out of cash if payments

arrive late. Cash is oxygen; profit is food. You can survive a long time without food and only minutes without oxygen.

**Why AI helps.** A good forecast needs to combine many signals: invoices you have sent but not collected, bills you owe, seasonal patterns, and how reliably your customers actually pay on time. AI can look at your history and learn, for example, that customer A usually pays two weeks late, while customer B pays early. It then weights the forecast accordingly.

**Start with the basics.** A simple forecast answers: what cash is coming in, what is going out, and what is the balance, week by week for the next 8 to 13 weeks. Build this even without AI — a spreadsheet works. AI improves it by learning payment patterns and flagging risk.

**Watch the gap.** The most important number is the lowest point in the forecast. If the forecast shows your balance dipping below a safe level in week nine, you have time now to fix it — chase invoices, delay a purchase, arrange a credit line. The whole value of forecasting is seeing the dip before it happens.

**Treat forecasts as ranges, not promises, and update often.** A forecast is an estimate, not a guarantee. Present it as a likely range with a best case and a worst case, and plan so that the worst case is survivable. Cash forecasts go stale fast, so update weekly; a forecast from a month ago is almost useless because so much has changed. The habit of a weekly cash check is one of the most valuable routines a small business owner can build.

## Ethics and Responsibility

Finance is where mistakes cost real money and real trust, so responsibility matters more here than almost anywhere.

**Keep a human in the loop for money movement.** AI should draft, extract, match, and suggest. A person should approve anything that sends money, changes a balance, or signs a report. An agent that reads a vendor email and pays without review is a fraud risk: a fake invoice, a spoofed email, or a misread total can drain cash fast. Elanco's agent drafts and a human reviews — copy that pattern exactly.

**Protect financial data.** Invoices and bank records are sensitive. Use tools that keep your data secure and, where possible, within your own environment. Be careful sending financial documents to public AI services. The security basics are covered in Chapter 6 — Cybersecurity in the AI Era and the self-hosting option in Chapter 8 — Self-Hosting: Keep Your Data Under Control.

**Be honest in reports.** An automatic summary can make a bad month sound fine. Do not let the polish of an AI-written report hide a real problem. Read the numbers yourself and report the truth, especially when it is uncomfortable.

**Mind the audit trail and keep segregation of duties.** Keep records of what the AI extracted, matched, and changed; if an auditor asks how a figure was produced, you must be able to show the path. And remember automation can hide a bad transaction as easily as find one: the person who sets up a vendor should not be the same person who approves its payment. AI does not remove the need for internal controls; it changes what they look like.

## Mistakes to Avoid

**Letting AI move money without review.** The single most dangerous mistake. Always require human approval for payments.

**Automating a bad process.** If your current invoice process is a mess, automating it just makes a faster mess. Clean the process first, then automate.

**Trusting extraction blindly.** AI can misread a total or a tax figure. Check the high-stakes fields until you have proof of accuracy.

**No exception handling.** If you only automate the easy cases and have no plan for the unusual ones, the unusual ones pile up and break the system. Design for the exceptions from day one.

**Skipping the audit trail.** A tool that cannot show what it did is a liability in an audit. Demand traceability.

**Forecasting the best case and betting on it.** A forecast is a range. Plan so the worst case is survivable.

**Stale forecasts and profit-vs-cash confusion.** A cash forecast from a month ago is useless; update weekly. And remember you can be profitable and still run out of money — forecast cash, not just profit.

**Over-automating low volume.** If you only have ten invoices a month, a tool may not pay off. Match the tool to your volume.

**Sending sensitive data to public AI.** Invoices and bank lines are sensitive. Use secure or self-hosted tools.

**No baseline.** Not measuring how long the task took before, so you cannot prove the saving. Measure before you start (see Chapter 22).

**Hiding bad numbers.** A polished AI report that masks a bad month is dishonest. Report the truth.

# Practical Exercise

## 25.7 Exercise: map your back-office automation

Pick one back-office task and plan its automation end to end.

**Step 1 — Choose the task.** Pick the most repetitive finance or admin task you have: invoice entry, bank reconciliation, the monthly report, or the cash forecast.

**Step 2 — Measure the baseline.** How long does it take now, and how many times a month do you do it? Write both down. This is your "before" number.

**Step 3 — Draw the current steps.** List every step a human does today: receive, read, type, match, check, file. Seeing the steps makes the automation obvious.

**Step 4 — Mark each step.** For each step, mark it: **AI does it** (extract, match, summarize), **human reviews it** (approve, sign off), or **human decides it** (the judgment call). Every money movement must be human-approved.

**Step 5 — Pick the tool.** Choose a tool that fits your volume and your accounting system. Do not buy the biggest one; buy the one that fits.

**Step 6 — Start with extract-and-flag.** Launch with the AI extracting and flagging, and a human approving everything. Do not go fully automatic on day one.

**Step 7 — Set the error threshold.** Decide the error rate that triggers action. For example, "if more than 3% of extracted totals are wrong, we stop and review the tool."

**Step 8 — Plan the cash forecast.** Build a simple 8-to-13-week cash forecast, even in a spreadsheet. Mark the lowest point. Decide what you will do if it dips below your safe level.

Do this for one task first. Once it works and the numbers prove the saving, move to the next task. One well-automated task teaches you more than five half-finished ones.

## Checklist

## 25.8 Administration and finance checklist

Before automating any finance task, check these.

- ☐ **You measured the baseline** — time per task and how often you do it.
- ☐ **A human approves every money movement** — no AI sends money on its own.
- ☐ **You clean the process before automating it.**
- ☐ **You check the high-stakes fields** (totals, tax) until you trust the tool.
- ☐ **You designed for exceptions** — the unusual cases have a clear path to a human.
- ☐ **The tool keeps an audit trail** you can show an accountant or auditor.
- ☐ **Financial data is kept secure**, not sent to public AI services.
- ☐ **Reports are read by a human** before you act on or share them.
- ☐ **You report bad numbers honestly**, not just the polished summary.
- ☐ **You keep segregation of duties** — vendor setup and payment approval are separate people.
- ☐ **You forecast cash, not just profit**, and watch the lowest point.

- ☐ **You update the cash forecast weekly.**
- ☐ **You treat forecasts as a range**, and plan so the worst case is survivable.
- ☐ **You match the tool to your volume** — no over-automation of tiny tasks.
- ☐ **You set an error threshold** that triggers a review.
- ☐ **You keep the money decision separate from the people decision** (see Chapter 16).

If a box is empty, the risk is still yours. Fill it before you let AI near the money.

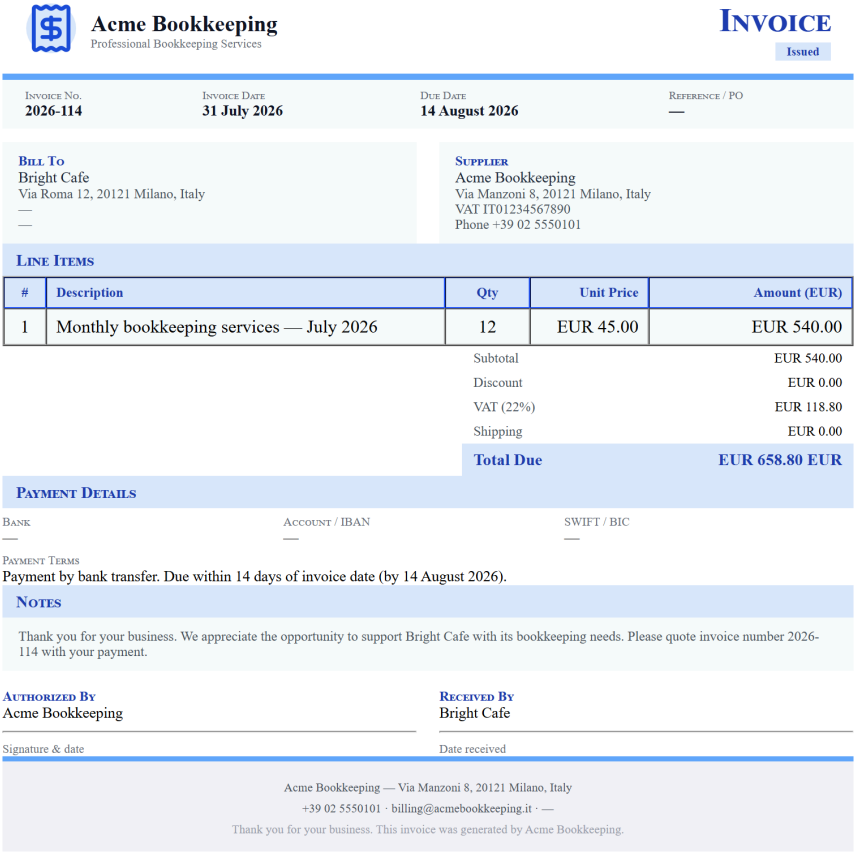
## Key Takeaways

- Administration and finance are full of repetitive work — invoices, matching, reports, forecasts — and repetition is exactly what AI is good at.
- Keep AI as the draftsman and the human as the decision-maker: let it extract, match, and suggest, but a person must approve anything that moves money.
- The Elanco case (a 2026 Hackett Innovation Award winner) cut procure-to-pay query time to under 10 seconds, about a 99% reduction, by using a two-layer agent that drafts replies for human review and removes many queries entirely.
- Forecast cash, not just profit, update it weekly, and plan so the worst-case dip is survivable.
- Clean the process before you automate it, design for the exceptions, and keep an audit trail you can show.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Make an invoice in seconds



**Acme Bookkeeping**
Professional Bookkeeping Services

**INVOICE**
Issued

| Invoice No. | Invoice Date | Due Date | Reference / PO |
|---|---|---|---|
| **2026-114** | **31 July 2026** | **14 August 2026** | — |

**Bill To**
Bright Cafe
Via Roma 12, 20121 Milano, Italy
—
—

**Supplier**
Acme Bookkeeping
Via Manzoni 8, 20121 Milano, Italy
VAT IT01234567890
Phone +39 02 5550101

**Line Items**

| # | Description | Qty | Unit Price | Amount (EUR) |
|---|---|---|---|---|
| 1 | Monthly bookkeeping services — July 2026 | 12 | EUR 45.00 | EUR 540.00 |

| | |
|---|---|
| Subtotal | EUR 540.00 |
| Discount | EUR 0.00 |
| VAT (22%) | EUR 118.80 |
| Shipping | EUR 0.00 |
| **Total Due** | **EUR 658.80 EUR** |

**Payment Details**

| Bank | Account / IBAN | SWIFT / BIC |
|---|---|---|
| — | — | — |

Payment Terms
Payment by bank transfer. Due within 14 days of invoice date (by 14 August 2026).

**Notes**

Thank you for your business. We appreciate the opportunity to support Bright Cafe with its bookkeeping needs. Please quote invoice number 2026-114 with your payment.

| Authorized By | Received By |
|---|---|
| Acme Bookkeeping | Bright Cafe |
| Signature & date | Date received |

Acme Bookkeeping — Via Manzoni 8, 20121 Milano, Italy
+39 02 5550101 · billing@acmebookkeeping.it · —
Thank you for your business. This invoice was generated by Acme Bookkeeping.

*A finished invoice document produced by the agent*

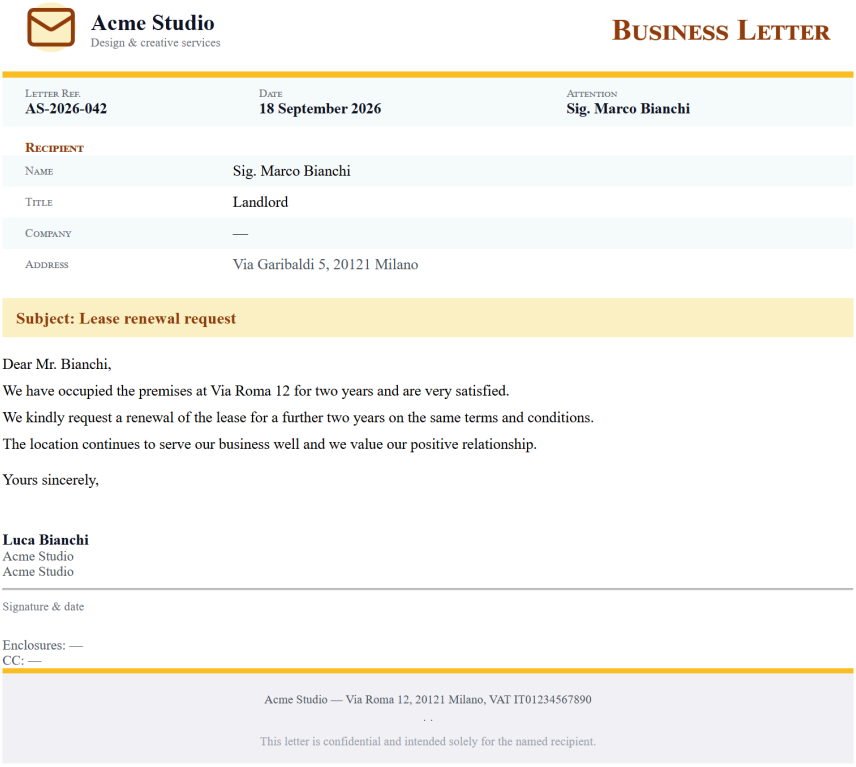
**What you ask:** `Make an invoice for Bright Cafe for 12 hours of bookkeeping at 45 euros an hour, due in 14 days.`

The agent writes a proper invoice with your business details, the line items, the subtotal, tax and total, and a due date. You get a real document you can print or send. If a number is wrong, you say so and it fixes it.

*Tip: Ask for it as a real Word file (/tools office-files) if you want to keep editing it in Microsoft Office.*

---

# Write a formal letter



Acme Studio
Design & creative services

BUSINESS LETTER

| LETTER REF. | DATE | ATTENTION |
|---|---|---|
| AS-2026-042 | 18 September 2026 | Sig. Marco Bianchi |

RECIPIENT

| | |
|---|---|
| NAME | Sig. Marco Bianchi |
| TITLE | Landlord |
| COMPANY | — |
| ADDRESS | Via Garibaldi 5, 20121 Milano |

**Subject: Lease renewal request**

Dear Mr. Bianchi,

We have occupied the premises at Via Roma 12 for two years and are very satisfied.

We kindly request a renewal of the lease for a further two years on the same terms and conditions.

The location continues to serve our business well and we value our positive relationship.

Yours sincerely,

**Luca Bianchi**
Acme Studio
Acme Studio

Signature & date

Enclosures: —
CC: —

Acme Studio — Via Roma 12, 20121 Milano, VAT IT01234567890

This letter is confidential and intended solely for the named recipient.

*A formal business letter, formatted and ready*

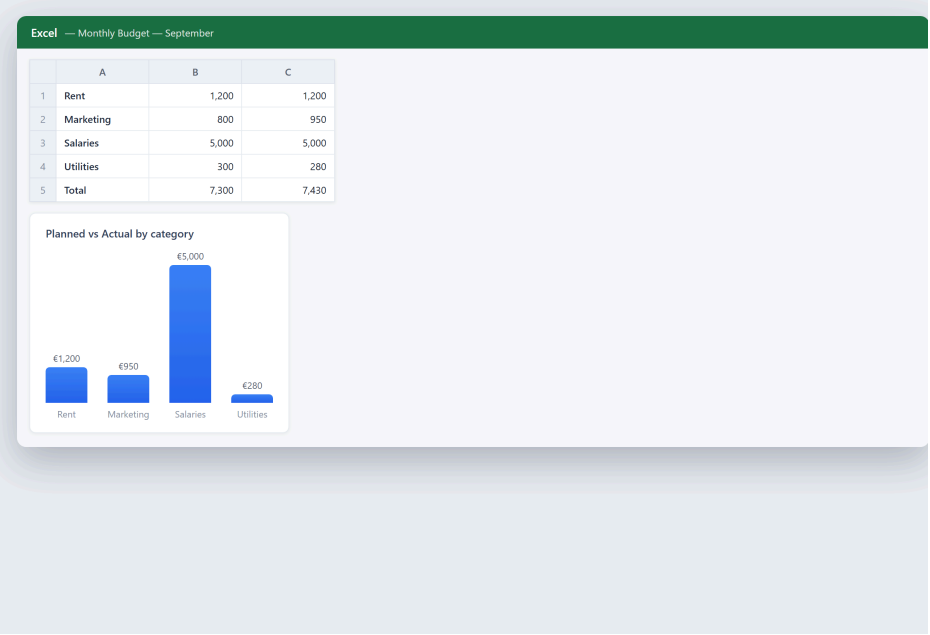
**What you ask:** `Write a formal letter to our landlord asking to renew the lease for another two years on the same terms.`

The agent writes the letter with the right greeting, a clear body and a polite closing, in your business voice. You review it, change a word if you like, and send it.

*Tip: Tell it who you are writing to and what you want; it handles the formal tone for you.*

---

## A budget you can read




*A budget spreadsheet with a clear bar chart*

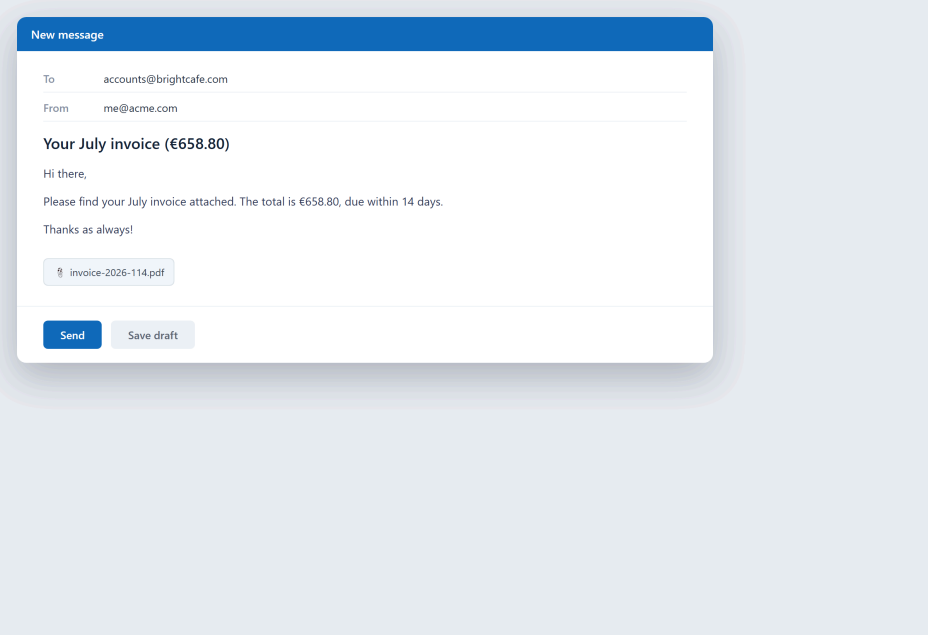
**What you ask:** `Build a monthly budget spreadsheet with planned and actual for rent, marketing and salaries, plus a chart.`

The agent creates the spreadsheet with the categories, the planned and actual columns, totals, and a chart that shows the difference at a glance. You can open it in Excel and keep working.

*Tip: Ask for the chart title and the currency so it matches your business.*

---

## Send the invoice by email




*The invoice attached to a ready-to-send email*

**What you ask:** `Email the invoice we just made to the client with a short, friendly cover note.`

The agent attaches the invoice and writes a short cover note with the amount and due date. One message, sent.

*Tip: Chain it: 'make the invoice and email it to the client' in a single ask.*

# Chapter 26 — Sales and Marketing

## In Simple Words

Sales and marketing are where AI can help you find more customers, talk to them better, and spend less time guessing. The work here is full of small, repeated jobs: sorting which leads are worth calling, writing emails, answering the same questions on your website, and trying to understand what your customers want. AI is good at all of them.

The core idea is *personalization at scale*. A good salesperson remembers each customer, tailors the message, and follows up at the right moment. A small team cannot do that for thousands of people. AI can. It can look at each lead and guess how likely they are to buy, write an email that sounds like it was written for that person, answer a visitor's question at 2 a.m., and spot which customers are about to leave.

This chapter covers four jobs: scoring leads (guessing who is most likely to buy), writing emails and content, running chatbots and assistants, and analyzing customers to understand what they want and who is at risk of leaving.

One honest warning before we start: AI is a powerful amplifier. It will make a good message reach more people, and it will make a bad message reach more people too. It can also push you toward spam — sending too much, too often, to people who did not ask. The goal is not to blast everyone; it is to reach the right person with the right message at the right time. The method for judging whether any of this pays off lives in Chapter 16 — Goals, Costs, and Return on Investment; this chapter shows you what to automate and how.

## A Bit of History

**1990s: mass marketing and the mailing list.** Marketing automation began with simple email lists. A business could send one message to thousands of people at once. It was cheap and wide, but blunt — everyone got the same message, whether it fit them or not. This was the era of "spray and pray."

**2000s: the CRM and the funnel.** Customer Relationship Management (CRM) software — a database of every customer and every interaction — arrived. Marketers started thinking in a "funnel": many people enter at the top, fewer make it to a purchase at the bottom. The CRM let you track where each person was in the funnel and follow up. Data entered the picture.

**2010s: segmentation and personalization.** With more data, marketers learned to split their audience into groups (segments) and send each group a different message. "People who bought X also bought Y." Personalization made messages more relevant and more effective. But it was still mostly rule-based: if a customer did this, send that.

**Late 2010s: machine learning scores leads.** Machine learning — software that learns patterns from many examples — started scoring leads. Instead of a person guessing who to call first, the model looked at hundreds of signals (company size, what pages they visited, how they found you) and ranked leads by how likely they were to buy. Sales teams stopped wasting time on cold leads.

**2020s: generative AI writes and chats.** Large language models — AI trained on huge amounts of text — can now write emails, ad copy, and product descriptions that sound human. Chatbots built on the same technology can hold a real conversation, answer questions, and

guide a customer through a purchase. The chatbot moved from a frustrating menu of buttons to something that actually understands what you typed.

**Now: agents that work the whole journey.** The newest step is the agent — AI that carries a task across several steps. A customer-facing agent can answer a question, check stock, and guide a purchase. An internal agent can take a support request and create the ticket. The mobilezone example below uses exactly this: two agents, one for customers and one for the internal team.

The arc: from one message to everyone, to tracked funnels, to segments, to scored leads, to AI that writes and chats. Each step made marketing more personal and less of a guessing game.

## Curiosity

### 26.5 The chatbot that became two agents

When mobilezone, a Swiss telecom retailer, replaced its old chatbot, it did not build one better bot. It built two different agents for two different jobs: one facing customers, one facing its own staff. That split is the interesting idea. The same AI technology served two audiences in two very different ways — a friendly shopping guide on the website, and a ticket-writing helper inside the company. The story, with the real numbers, is below.

## A Real Business Example

**mobilezone: two Copilot agents, one for customers and one for IT.**

mobilezone is a Swiss telecommunications retailer with more than 125 physical stores, selling phones, plans, and connected devices. It had two separate problems. On the customer side, its old website chatbot was rigid and frustrating — it could only follow a fixed menu and often failed to answer real questions. Inside the company, staff had to fill out clunky forms to report IT issues, which slowed everyone down.

According to a published Microsoft customer case study, mobilezone rebuilt both sides using Microsoft Copilot Studio — a tool for building conversational AI agents — together with Dynamics 365 (its CRM) and the Power Platform (Microsoft's low-code automation tools). It built two agents:

- **Mia — the customer-facing agent.** Mia is a multilingual assistant on the website. It answers high-volume customer questions, helps visitors find the right product or plan, and guides them through a purchase. Because it understands natural language, it handles the questions the old menu-based bot could not.
- **Supporto — the internal IT agent.** Supporto is a helper for mobilezone's own staff. Instead of filling out a rigid form, an employee tells Supporto what is broken in plain words, and the agent creates the IT ticket automatically.

The reported results: the agents now handle **more than 1,600 chats a month**, and the internal IT agent **cut IT resolution time by about 50%**. Just as important, the agents reduced the load on mobilezone's external contact center (the outsourced phone support), and the customer agent improved online conversion by guiding shoppers through product discovery.

Two lessons stand out. First, the same technology served two very different jobs — external sales and internal support — which shows how flexible these agents are. Second, mobilezone did not replace humans;

it moved the easy, repetitive conversations to the agent so its people could focus on the harder ones. That is the realistic promise: AI handles the volume, humans handle the value.

A note on the source: the figures above come from Microsoft's published customer story on mobilezone. Treat them as mobilezone's reported results; your own numbers will depend on your volume and setup.

## How to Do It

### 26.1 Lead scoring

A *lead* is a person or company that has shown some interest — they filled a form, downloaded something, or asked a question. You do not have time to call every lead with the same effort. Lead scoring means ranking leads by how likely they are to buy, so your team calls the hottest ones first.

**How AI scores a lead.** A machine-learning model looks at many signals at once: the size of the lead's company, the job title of the person, which pages they visited, how many times they came back, whether they opened your emails, and how they found you. From past deals that closed, the model learns which signals go together with a sale, and it scores each new lead accordingly.

**Why it beats guessing.** A person scoring leads by gut feel is slow and biased — they tend to favor the leads that "feel" friendly, not the ones that actually convert. AI scores consistently and fast, and it can weigh hundreds of signals a person cannot hold in their head.

**Start simple.** You do not need a perfect model on day one. Start with a few clear signals — company size, role, and what they downloaded —

and score leads by hand in your CRM. As you collect more closed deals, a real model can learn from them and improve.

**Feed the loop.** The model gets better when you tell it which leads actually became customers. Make sure your CRM records the outcome of every deal, so the scoring learns from real results, not guesses.

**Do not over-trust the score.** A score is a hint, not a verdict. A high score means "call soon," not "guaranteed sale." A low score might still be a good customer the model has not learned yet. Use the score to set priority, not to ignore people.

## 26.2 Email and content

Writing emails, ad copy, and product descriptions takes hours. Generative AI — AI that produces text — can draft them fast, and a person can edit them to sound right.

**Draft, do not ship.** Use AI to produce a first draft, then edit it. AI is excellent at getting words on the page quickly and terrible at knowing your exact voice, your brand, and your customer's feelings without guidance. The draft is 70% there; your edit is the last 30% that makes it yours.

**Personalize at scale.** AI can take one template and adapt it for many people: insert the recipient's name, reference what they looked at, tailor the offer to their segment. This is the personalization-at-scale idea from earlier — a message that feels written for one person, sent to thousands.

**Match the channel.** An email, a social post, and a product page need different lengths and tones. Tell the AI the channel and the goal, and edit accordingly. A long email does not work as a tweet; a tweet does not work as a landing page.

**Keep the human voice.** AI text can sound flat, generic, or too eager. Read every draft aloud. If it does not sound like something you would say, rewrite it. Your customers can tell when a message is generic.

**Never let AI send without review.** An unreviewed AI email can contain a wrong price, a wrong name, or an inappropriate line. Always have a human read before it goes out. This is the same rule as in finance: AI drafts, human approves.

**Watch the volume.** AI makes it easy to send more emails than you should. More is not better. Sending too much to people who did not ask is spam, and it burns your list and your reputation. Send less, but make each one count.

## 26.3 Chatbots and assistants

A chatbot is a program that talks with a customer on your website or app. Modern chatbots, built on large language models, understand what a person types and reply in plain language — a huge step up from the old "press 1 for sales" menus.

**What a good chatbot does.** It answers common questions instantly (price, hours, shipping, returns), guides a visitor to the right product, captures contact details for a follow-up, and hands off to a human when it cannot help. The handoff is critical: a chatbot that never hands off frustrates people and loses sales.

**Design the handoff first.** Before you build the bot, decide when it should pass the conversation to a person. When the bot is unsure, when the customer asks for a human, when the topic is sensitive — hand off. A bot that knows its limits is trusted; one that bluffs is not.

**Train on your real questions.** Feed the bot the questions customers actually ask, with the answers you want. The more it knows your specific products and policies, the more useful it is. A generic bot gives generic answers that frustrate.

**Let it speak many languages.** One of the biggest wins of a modern chatbot is multilingual support. It can answer a customer in their own language without you hiring translators. This is exactly what mobilezone's Mia does.

**Measure what it handles and what it fails.** Track how many conversations the bot resolves on its own, how many it hands off, and what it could not answer. The failures are gold — they tell you what to teach it next. (The same chatbot ideas, applied to support rather than sales, are covered in Chapter 27 — Customer Care and Support.)

## 26.4 Customer analysis

Customer analysis means using data to understand who your customers are, what they want, and who is about to leave. AI is strong here because it can see patterns across thousands of customers that no person could spot by hand.

**Segment your customers.** AI can group customers by behavior: frequent buyers, big spenders, seasonal buyers, at-risk customers. Each group needs a different message. This is segmentation, and AI does it faster and more accurately than manual rules.

**Spot churn before it happens.** *Churn* means a customer stops buying. AI can look at signals — fewer visits, smaller orders, less engagement — and flag customers who are likely to leave soon. That gives you time to win them back with an offer or a personal call, instead of finding out after they are gone.

**Find the next best offer.** AI can look at what a customer bought and suggest what they are likely to want next. “Bought a phone, likely needs a case and insurance.” This is the recommendation engine idea, and it lifts the value of each customer.

**Read what customers say.** AI can read reviews, survey comments, and support chats and pull out the main themes: what people love, what annoys them, what they ask for. Instead of reading a thousand comments one by one, you get a summary of the big themes. This turns raw feedback into something you can act on.

**Do not treat people as data points.** Analysis is a tool to serve customers better, not to manipulate them. Use what you learn to make their experience better, not to exploit their weaknesses. The line between personalization and manipulation is real, and you should stay on the right side of it.

## Ethics and Responsibility

Sales and marketing touch people’s attention and trust directly, so the ethical line matters.

**Do not spam.** AI makes it easy to send too much. Sending messages to people who did not ask, or more than they agreed to, is spam. Respect consent and frequency. A short list of people who want your messages beats a huge list you blast.

**Be honest in AI-written copy.** AI can write a claim that sounds true but is not. Check every factual claim — price, features, results — before it ships. Never let AI invent a benefit you cannot deliver.

**Disclose when it is a bot.** In many places, and as a matter of good practice, a customer should know they are talking to a chatbot and not

a person. Make the handoff to a human easy and clear.

**Respect privacy.** Customer analysis uses personal data. Follow the rules for handling it — the basics are in Chapter 10 — Privacy and GDPR. Collect only what you need, tell people what you collect, and keep it secure.

**Do not manipulate.** Personalization should help people find what they want, not push them into a purchase they will regret. Avoid dark patterns — tricks that pressure people. Build trust, not a trap.

**Keep a human in the loop.** AI drafts the email, the ad, the reply. A human reviews it before it reaches a customer. This single rule prevents most of the damage.

## Mistakes to Avoid

**Spamming with AI.** Using the tool's power to send more than people want. Send less, make it count.

**Shipping unreviewed AI copy.** A wrong price or a false claim in an AI-written email damages trust. Always review.

**A chatbot that never hands off.** A bot that bluffs instead of passing to a human frustrates customers and loses sales. Design the handoff first.

**Generic, flat copy.** AI text that sounds like everyone else's. Edit for your voice or it will be ignored.

**Over-trusting the lead score.** Treating a score as a verdict instead of a hint. Use it to set priority, not to ignore people.

**Garbage data in, garbage analysis out.** If your CRM data is messy, the segments and churn flags are wrong. Clean your data first.

**Treating customers as data points.** Using analysis to manipulate instead of serve. Stay on the ethical side.

**No disclosure of bots.** Letting a chatbot pretend to be human. Be clear.

**Ignoring privacy rules.** Using personal data without consent or security. Follow GDPR basics.

**Confusing activity with results.** Counting emails sent instead of deals closed. Measure outcomes, not volume.

**Skipping the baseline.** Not measuring conversion before, so you cannot prove the lift. Measure first (see Chapter 22).

**Replacing the human touch entirely.** Customers still want a person for hard problems. Keep humans for the value conversations.

## Practical Exercise

### 26.7 Exercise: plan one AI-assisted campaign

Pick one campaign — an email to past customers, a chatbot on your site, or a lead-scoring pass — and plan it end to end.

**Step 1 — Choose the job.** Pick one: lead scoring, an email campaign, a chatbot, or a customer analysis. Do one, not all.

**Step 2 — Define the goal and the metric.** What should this achieve? Signups, replies, resolved questions, won-back customers? Pick one number to measure.

**Step 3 — Measure the baseline.** What is that number now? Write it down. Without it you cannot prove the lift.

**Step 4 — Gather the data.** For scoring: your past leads and which closed. For email: your consented list. For the chatbot: your real customer questions and answers. For analysis: your customer records. Clean the data first.

**Step 5 — Build the AI part.** Score the leads, draft the email, train the bot, or run the segmentation. Let AI do the heavy lifting.

**Step 6 — Add the human review.** Read every draft. Check every factual claim. Decide the chatbot's handoff rules. Nothing ships without a human read.

**Step 7 — Check consent and privacy.** Confirm you are allowed to message these people and that their data is handled correctly.

**Step 8 — Launch small and measure.** Run it on a small group first. Compare the metric to the baseline. If it works, scale up. If not, learn and adjust.

Do one campaign well. The lessons you learn — about tone, about handoffs, about what your customers respond to — carry into every campaign after.

## Checklist

### 26.8 Sales and marketing checklist

Before launching any AI-assisted sales or marketing activity, check these.

- ☐ **You measured the baseline** for the one metric you care about.
- ☐ **A human reviews every AI-written message** before it reaches a customer.

- [ ] **Every factual claim is checked** — price, features, results.
- [ ] **You have consent** to message the people you are messaging.
- [ ] **You respect frequency** — no spam, no over-sending.
- [ ] **The chatbot has clear handoff rules** to a human.
- [ ] **The chatbot is trained on your real questions**, not generic ones.
- [ ] **You disclose that it is a bot** where required and as good practice.
- [ ] **You clean your data** before scoring, segmenting, or analyzing.
- [ ] **You feed outcomes back** into the lead-scoring model so it learns.
- [ ] **You treat the lead score as a hint**, not a verdict.
- [ ] **You use analysis to serve customers**, not to manipulate them.
- [ ] **You follow privacy rules** (GDPR basics, see Chapter 10).
- [ ] **You keep humans for the hard conversations.**
- [ ] **You measure outcomes** (deals, replies, wins), not just volume sent.
- [ ] **You launch small first** and scale only what proves itself.

If a box is empty, you are risking trust. Fill it before you press send.

## Key Takeaways

- AI helps sales and marketing by personalizing at scale — scoring leads, tailoring messages, answering questions, and spotting at-risk customers.
- The mobilezone case (a Microsoft customer story) built two Copilot Studio agents — Mia for customers, Supporto for internal IT — handling 1,600+ chats a month and cutting IT resolution time by about 50%.
- AI drafts, a human reviews: never ship an AI-written message or let a chatbot bluff instead of handing off to a person.

- Clean data is the foundation — messy records make lead scores, segments, and churn flags wrong.
- Use AI to serve customers better, not to spam or manipulate them; consent and honesty protect the trust you are selling on.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Turn an idea into a proposal

**Acme Studio**
Design & Digital Solutions

# Sales Proposal
Online Shop Development

| Proposal No. | Date | Valid Until | Prepared For |
|---|---|---|---|
| **P-2026-021** | **18 September 2026** | **18 October 2026** | **Bright Cafe** |

## 1. Executive Summary

Acme Studio is pleased to submit this proposal to Bright Cafe for the creation of a simple online store that allows customers to order ahead. The project covers the design, build and launch of a branded online shop with a product catalogue, secure checkout and order email notifications. The total investment is EUR 9,500, delivered over an 8-week timeline.

## 2. Understanding of Your Needs

Bright Cafe needs a straightforward digital ordering channel that complements its in-store operations and makes it easy for customers to order ahead.

- Customers need a simple way to browse the menu and place orders before arriving at the cafe.
- The online shop must reflect the Bright Cafe brand and feel consistent with the in-store experience.
- Ordering must be secure and reliable, with clear confirmation sent to both the customer and the cafe.
- The solution should be easy to manage and ready to go live quickly, without unnecessary complexity.

## 3. Proposed Solution

Acme Studio proposes a focused online shop built around order-ahead functionality, delivered in three clear phases: Design, Build and Launch.

- A branded online shop designed to match the Bright Cafe identity and work smoothly on mobile and desktop.
- A structured product catalogue that makes it easy for customers to find and select items.
- A secure checkout process that protects customer payment and personal data.
- Automated order email notifications so the cafe and its customers receive immediate confirmation.

## 4. Deliverables

| Deliverable | Description | Timeline |
|---|---|---|
| Branded online shop | A custom-designed online store reflecting the Bright Cafe brand, optimised for mobile and desktop ordering. | Design: 2 weeks |
| Product catalogue | A structured catalogue of products with descriptions, prices and images, easy to browse and update. | Build: 4 weeks |
| Secure checkout | A secure payment and checkout flow that protects customer data and completes orders reliably. | Build: 4 weeks |
| Order email notifications | Automated email confirmations sent to the cafe and the customer when an order is placed. | Launch: 2 weeks |

## 5. Implementation Timeline

| Start | Duration | Milestones |
|---|---|---|
| Upon acceptance | 8 weeks total | Design 2 weeks — Build 4 weeks — Launch 2 weeks |

## 6. Investment

| Item | Description | Amount |
|---|---|---|
| Online shop project | Design, build and launch of the branded online shop, including product catalogue, secure checkout and order email notifications. | 9,500 EUR |
| Instalment 1 | First payment, due on acceptance of this proposal. | 4,750 EUR |
| Instalment 2 | Second payment, due on launch of the online shop. | 4,750 EUR |
| **Total Investment** | | **9,500 EUR** |

## 7. Terms & Conditions

- This proposal is valid until 18 October 2026.
- The total price is EUR 9,500, payable in two instalments of EUR 4,750.
- The first instalment is due on acceptance; the second is due on launch.
- The timeline of 8 weeks begins upon acceptance of this proposal.
- Any changes to the scope, deliverables or timeline will be agreed in writing before work proceeds.

## 8. Acceptance

*A structured project proposal document*

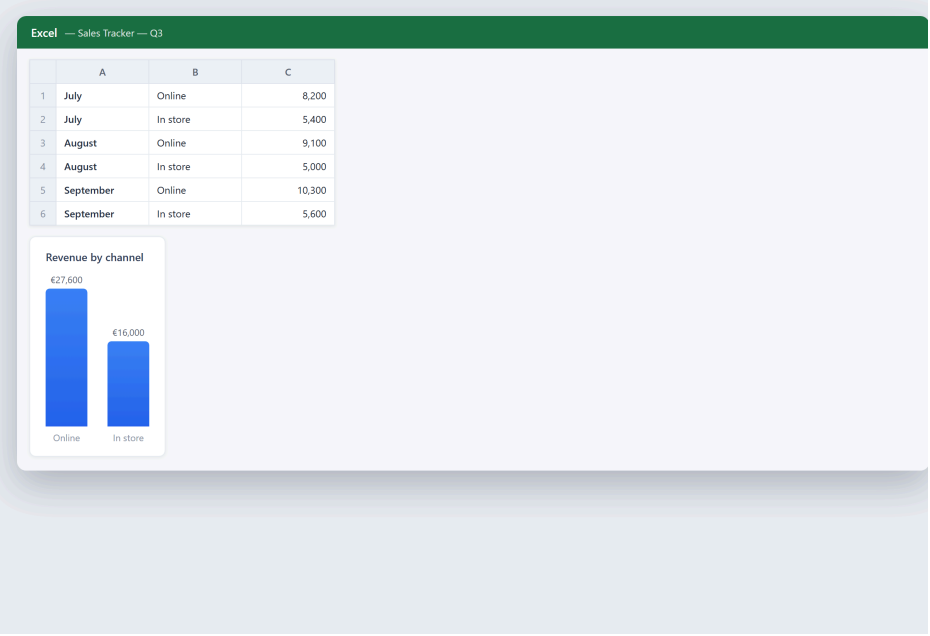
**What you ask:** `Create a project proposal for a small online shop: goals, what we deliver, timeline of 8 weeks, and a price of 9,500 euros.`

The agent builds a proposal with the client's goal, your solution, the deliverables, the timeline and the price — all in a clean layout that looks like it took a whole afternoon. It took a minute.

*Tip: Add your logo and a sentence about past results to make it feel personal.*

---

## Track your sales




*A sales tracker with revenue by channel*

**What you ask:** `Create a sales tracker with month, channel and revenue, and a chart of revenue by channel.`

The agent builds the tracker and the chart. Add rows as you go, or attach your raw sales list and ask it to fill the sheet for you.

*Tip: Attach a messy export from your shop and say 'clean this into a tracker' — it will.*

---

## A pitch deck from one prompt

*A presentation slide built by the agent*

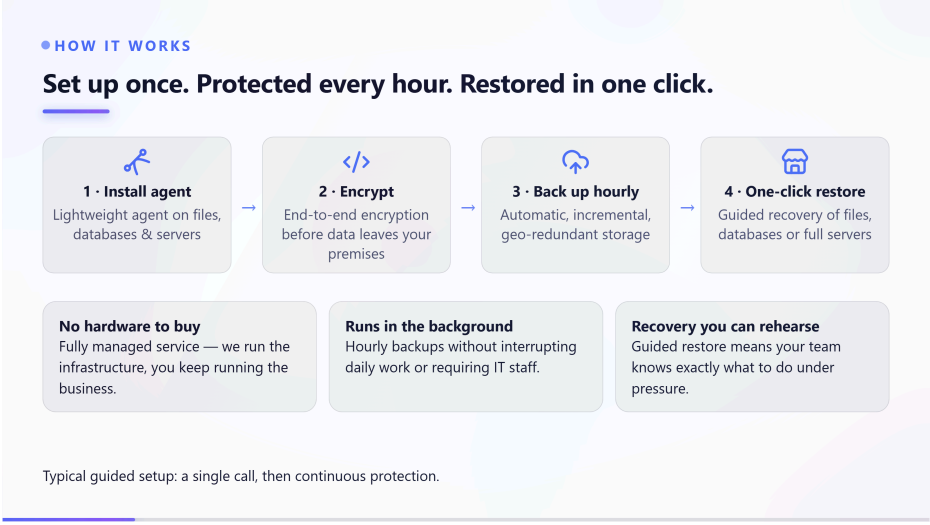
**What you ask:** `Make a 6-slide pitch deck for my delivery startup: problem, solution, market, model, traction, ask.`

The agent designs the slides with a clean look, one clear idea per slide, and the right order for a pitch. In the browser you press F11 for full screen and present.

*Tip: Need a real .pptx to send? Use /tools office-files and ask for PowerPoint.*

---

## A client sales deck




*A client-facing sales presentation slide*

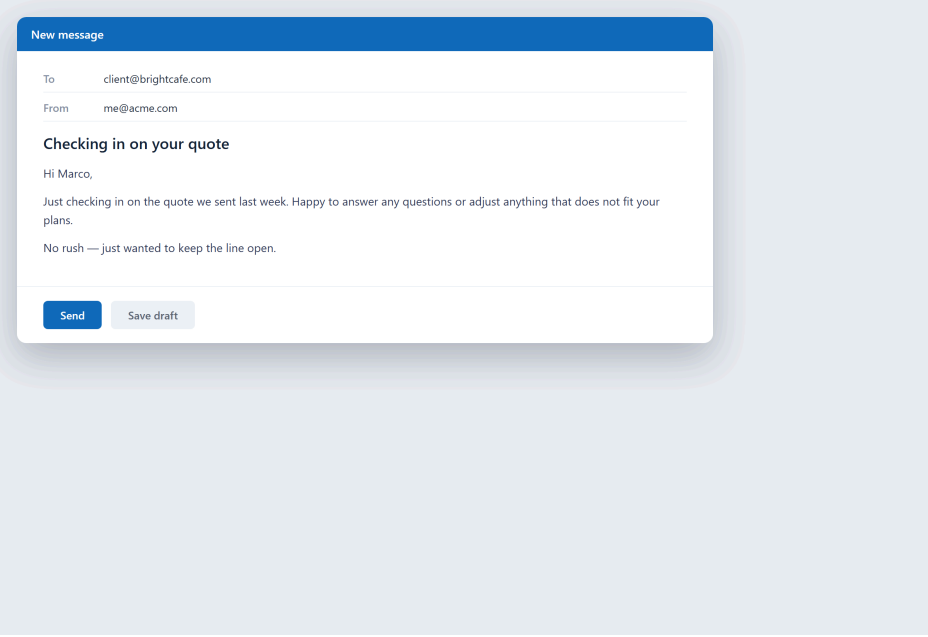
**What you ask:** `Create a sales deck summarising our work with Acme and proposing the next phase.`

The agent builds a focused deck: results so far, what the client gained, and the proposed next step. You tweak the numbers and present with confidence.

*Tip: Attach the project report and the agent pulls the highlights into the slides.*

---

## A polite follow-up




*A friendly follow-up email draft*

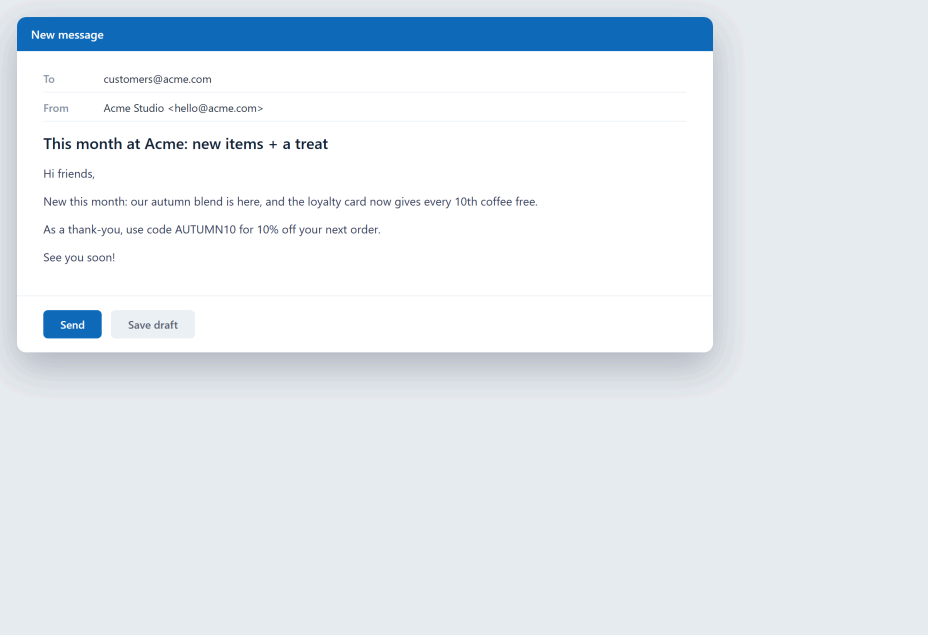
**What you ask:** `Write a short follow-up to a client who has not replied to our quote from last week.`

The agent writes a light, polite nudge that reminds without pressure. You send it and keep the relationship warm.

*Tip: A scheduled follow-up can send these for you if a reply has not arrived.*

---

## A customer newsletter




*A newsletter draft ready to send*

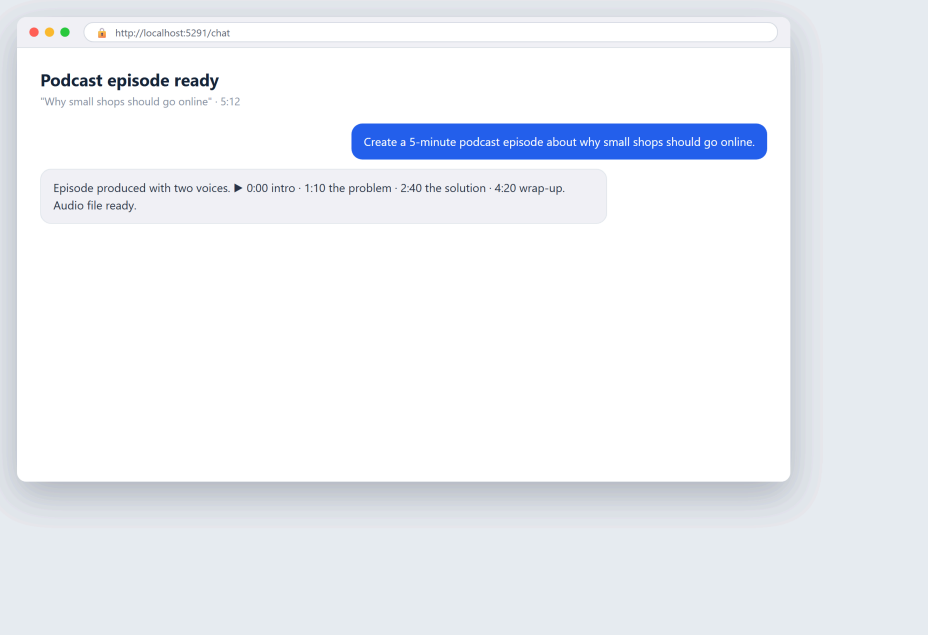
**What you ask:** `Write a monthly newsletter for our customers: new items, a tip, and a small discount code.`

The agent writes the newsletter in your voice with the news, a useful tip and the offer. Send it, or let it prepare one on a schedule.

*Tip: A monthly scheduled task can draft the newsletter for your review each time.*

---

## Turn a topic into a podcast




*A podcast episode ready to play*

**What you ask:** `Create a 5-minute podcast episode about why small shops should go online, in a friendly two-voice style.`

The agent writes the script and produces an audio episode with two voices, ready to publish. Your message, in audio form, with no studio.

*Tip: Give it your key points and it shapes them into a natural conversation.*

# Chapter 27 — Customer Care and Support

## In Simple Words

Customer care is where a business proves it cares. When something goes wrong or a customer has a question, how fast and how well you answer shapes whether they stay or leave. The problem is that support is full of the same questions asked over and over, and a small team cannot answer them all instantly. AI helps by answering the common ones right away and routing the hard ones to the right person fast.

Think of support as a queue. Every question lines up waiting for an answer. The longer the line, the more frustrated people get. AI shortens the line in two ways: it answers some questions before they ever reach a human, and it sorts the rest so the right person sees the right issue first.

This chapter covers four jobs: chatbots and FAQs that answer common questions, ticket management that sorts and routes problems, sentiment analysis that reads how customers feel, and an intelligent knowledge base that keeps all the answers in one searchable place.

One honest idea first: the goal of AI in support is not to hide customers from your people. It is to free your people from the repetitive questions so they can spend their time on the cases that truly need a human — the angry customer, the complex problem, the one that decides whether someone stays for ten years. AI handles the volume; humans handle the care. The method for judging whether it pays off lives in Chapter 16 — Goals, Costs, and Return on Investment; this chapter shows you what to automate and how.

## A Bit of History

**1960s–1980s: the call center.** Customer support meant the telephone. A customer called, waited on hold, and spoke to an agent. The whole discipline was about staffing enough people to answer the phone. It worked, but it was expensive, slow, and scaled only by hiring more people.

**1990s: email and the ticket.** Support moved partly to email, and the "ticket" was born — each customer request became a numbered record that could be tracked, assigned, and closed. Tickets brought order to the chaos of support email. Tools like early helpdesk software made it possible to see every open request in one place.

**2000s: the knowledge base and self-service.** Companies realized that most support questions were the same few questions. They built knowledge bases — searchable libraries of answers — so customers could help themselves. Self-service took some load off agents, but early knowledge bases were hard to search and often missed what the customer needed.

**2010s: chat and the rule-based chatbot.** Live chat arrived, and with it the chatbot. Early chatbots were rule-based: a decision tree that matched keywords to canned answers. They were cheap but frustrating — they could not understand anything off the script, and customers often felt trapped.

**Late 2010s: AI reads sentiment.** Machine learning started reading the *tone* of a message — whether a customer was happy, frustrated, or angry — and could flag an angry customer so a senior agent responded first. Sentiment analysis added a new signal to routing: not just what the issue is, but how the customer feels about it.

**2020s: large language models and agentic support.** Large language models — AI trained on huge amounts of text — made chatbots that actually understand what a customer typed and reply in plain language. The newest step is the agentic platform: AI that not only answers but takes action — creating a ticket, updating a record, scheduling a service. The TridentCare example below is exactly this: AI that handles the scheduling work end to end, with humans stepping in only when judgment is needed.

The arc: from phone queues, to tracked tickets, to searchable answers, to bots that understand, to agents that act. Each step moved the routine work to software and left the human agents free for the cases that need a human.

## Curiosity

### 27.5 The dispatcher who let the system do the work

In TridentCare's old way of working, about half of all scheduling was done by hand — a person matching a patient's request to a technician, a time, and a place. After the company moved to an AI-powered platform, manual scheduling dropped to just 4.3%. The dispatchers did not disappear; they changed jobs. They stopped doing the matching themselves and started supervising it, stepping in only when a case genuinely needed human judgment. That shift — from doing the work to supervising the work — is the quiet revolution in support and operations. The full story is below.

## A Real Business Example

### TridentCare: 96% scheduling automation with an AI-powered CRM.

TridentCare is the largest provider of portable medical diagnostic services in the United States. It sends technicians to hospitals, nursing homes, and patients' homes to perform tests like X-rays and ultrasounds — services that cannot wait and must be scheduled reliably across a huge area. Scheduling this work by hand, across hundreds of markets, was slow and hard to scale.

According to a ServiceNow press release (carried by Business Wire on 22 April 2026), TridentCare selected the ServiceNow AI Platform to transform its end-to-end operations and largely replace manual scheduling with automation. The reported results:

- **96% scheduling automation across 127 markets.** Manual scheduling of portable medical diagnostic services dropped from **50% to just 4.3%**. In other words, the system now handles almost all of the scheduling, and a person intervenes only when judgment genuinely requires it.
- **Patient wait times beyond the service-level agreement (SLA) were cut by 57%.** An SLA is the promised level of service — for example, "a technician arrives within two hours." Fewer patients waited longer than promised.
- **First-market efficiency improved by about 30%.** The company got more done in the markets where it started.

The press release also described a "lead-to-cash" transformation: by connecting the sales CRM directly to field performance data,

TridentCare gained the visibility to set accurate service levels, see where demand was shifting, and sharpen how it sold.

Two lessons stand out. First, the human role changed from *doing* to *supervising*: dispatchers let the system handle the routine and stepped in only for the exceptions. Second, the win was not just speed — it was reliability and patient care. In a business where a late technician affects a patient's health, automation that cuts wait times is not just a cost saving; it is better service.

A note on the source: the figures above come from ServiceNow's published announcement. Treat them as TridentCare's reported results; your own numbers will depend on your operation and volume.

## How to Do It

### 27.1 Chatbots and FAQs

A chatbot on your website or app can answer the common support questions instantly, so customers do not have to wait for a person. A FAQ (Frequently Asked Questions) page is the simpler cousin: a list of common questions with written answers.

**What a chatbot should handle.** The repetitive, low-risk questions: "What are your hours?", "How do I reset my password?", "Where is my order?", "What is your return policy?" These are the questions that eat up most of a support team's time and need no judgment. Let the bot answer them.

**Modern bots understand language.** Unlike the old menu-based bots, a chatbot built on a large language model understands what a customer types and replies in plain words. It can handle "I can't log in, it says wrong password" without the customer picking from a list.

**The handoff is everything.** A chatbot that cannot answer must pass the conversation to a human, with the context intact — the human should see what the customer already said, not start from zero. Decide the handoff rules before you build: when the bot is unsure, when the customer asks for a person, when the topic is sensitive. A bot that knows its limits is trusted; one that bluffs is not.

**Keep the FAQ alive.** A FAQ page that is out of date is worse than none — it gives wrong answers confidently. Review it regularly and update it as your products and policies change. The chatbot and the FAQ should draw from the same source of truth (see the knowledge base below).

**Measure resolution and failure.** Track how many questions the bot resolves on its own and how many it hands off. The failures tell you what to teach it next. (The same chatbot technology, applied to sales rather than support, is covered in Chapter 26 — Sales and Marketing.)

## 27.2 Ticket management

A *ticket* is a numbered record of a customer request. Ticket management means sorting every request, assigning it to the right person, tracking its progress, and closing it when solved. When volume is high, good ticket management is the difference between an organized support team and a chaotic one.

**AI sorts and routes.** Instead of a person reading every ticket and deciding who should handle it, AI reads the ticket and routes it automatically to the right team or agent, based on the topic, the customer's history, and the urgency. This saves the triage step, which is pure overhead.

**Prioritize by urgency and feeling.** AI can rank tickets so the most urgent and the most upset customers are seen first. A customer whose service is completely down, or who is clearly angry, should not wait behind a routine question. Combining urgency with sentiment (see below) makes the queue smarter.

**Suggest the answer.** AI can read a ticket and suggest a reply, or point the agent to the knowledge-base article that solves it. The agent reviews and sends, instead of writing from scratch. This cuts handling time on every ticket.

**Automate the routine actions.** Some tickets need a simple action — reset a password, resend a document, update an address. AI can do these automatically and close the ticket, or draft the action for a human to approve. The routine work disappears from the queue.

**Keep the audit trail.** Every ticket should record what happened, who did what, and when. This matters for quality review, for training, and for accountability. Good ticketing tools produce this automatically; confirm yours does.

## 27.3 Sentiment analysis

Sentiment analysis means using AI to read the *feeling* behind a message — is the customer happy, neutral, frustrated, or angry? It turns raw text into an emotional signal you can act on.

**Why it matters.** An angry customer who waits in a normal queue may churn before anyone sees them. If AI flags the anger early, a senior agent can respond fast and turn a bad moment into a good one. Sentiment is a routing signal that pure topic-matching misses.

**How it works.** The AI reads the words and tone of a message and assigns a sentiment — positive, neutral, negative — or a score. It learns from many examples of messages labeled by humans. It is not perfect, but it is good enough to flag the clearly upset customers.

**Use it to prioritize, not to judge.** Feed sentiment into the ticket queue so the most negative messages rise to the top. Do not use it to score or punish agents, and do not treat a single sentiment reading as the final word on a customer's feelings. It is a hint to pay attention, not a verdict.

**Watch trends over time.** Sentiment is most useful as a trend. If negative sentiment across all tickets is rising month over month, something is wrong with your product or service, even before customers write a formal complaint. Track the average sentiment and watch the direction.

**Mind the limits.** Sarcasm, irony, and cultural differences can fool sentiment analysis. A "great, another problem" reads as positive to a naive model. Use sentiment as one signal among many, and let a human's judgment override it.

## 27.4 Intelligent knowledge base

A knowledge base is a searchable library of answers to common questions. An *intelligent* knowledge base uses AI to make it far easier to find the right answer — and to keep the answers up to date.

**Search that understands the question.** Instead of matching exact keywords, an AI knowledge base understands what the customer means and returns the relevant article even if the words differ. "My card was charged twice" finds the duplicate-payment article, not just articles containing those exact words.

**AI writes and updates articles.** When a support agent solves a new problem, AI can draft a knowledge-base article from the resolved ticket, so the answer is captured for next time. This turns every solved problem into a reusable answer, instead of leaving it locked in one agent's head.

**One source of truth.** Your chatbot, your FAQ page, and your agents should all draw from the same knowledge base. When you update one place, every channel gets the right answer. If you maintain them separately, they drift apart and give conflicting answers, which destroys trust.

**Spot the gaps.** AI can see which questions customers ask that have no article. Those gaps are a to-do list: write the missing answers, and the bot and search get better. The knowledge base improves itself by showing you what is missing.

**Keep it fresh.** A stale knowledge base gives wrong answers confidently. Review articles regularly, retire the outdated ones, and let the AI flag articles that may need updating because the product changed. Freshness is the whole value of a knowledge base.

## Ethics and Responsibility

Support is where trust is won or lost, so the ethical stakes are high.

**Never let a bot hide a human.** Customers have a right to reach a person. A chatbot that blocks the path to a human is a hostile design. Make the handoff easy, clear, and always available.

**Disclose that it is a bot.** A customer should know they are talking to AI and not a person. This is good practice everywhere and a legal requirement in some places.

**Do not ignore an angry customer because of a model error.** If sentiment analysis misses an angry customer, the human queue must still catch them. Sentiment is a helper, not a gatekeeper. Never let a model's mistake bury a real complaint.

**Protect customer data.** Support tickets contain personal, sometimes sensitive information. Handle it with care and follow the privacy rules — the basics are in Chapter 10 — Privacy and GDPR. Do not feed sensitive customer data into public AI tools without checking the security implications (see Chapter 6 — Cybersecurity in the AI Era).

**Keep a human accountable.** AI can draft a reply or route a ticket, but a human owns the outcome. When something goes wrong, there must be a person responsible, not a black box.

**Use sentiment to help, not to manipulate.** Reading a customer's feelings should help you serve them better, not exploit their frustration to upsell or pressure them. Stay on the side of care.

**Do not use support data to surveil agents.** Tracking ticket metrics to improve the process is fine. Using the same data to spy on and punish individual agents poisons trust. Measure the work, not the person.

## Mistakes to Avoid

**A bot that blocks the human.** The worst support mistake. Always make the handoff easy.

**No handoff context.** Passing a customer to a human who then asks them to repeat everything. Carry the context across.

**Bluffing chatbot.** A bot that guesses instead of admitting it does not know. Teach it to hand off when unsure.

**Stale knowledge base.** Outdated articles giving wrong answers confidently. Review and update regularly.

**Separate sources of truth.** Chatbot, FAQ, and agents each with their own answers that drift apart. Use one knowledge base.

**Sentiment as a verdict.** Treating a model's sentiment reading as the final word on a customer. It is a hint, not a judgment.

**Missing the angry customer.** Letting a model error bury an upset customer. The human queue must still catch them.

**Automating a bad process.** If your support flow is broken, automation makes a faster broken flow. Fix the process first.

**Leaking sensitive data.** Putting customer data into insecure AI tools. Check security and privacy first.

**No audit trail.** Tickets that cannot show what happened. Keep the record.

**Confusing deflection with success.** Counting how many tickets the bot "handled" instead of whether the customer was actually satisfied. Measure resolution and satisfaction, not deflection.

**Skipping the baseline.** Not measuring response time or satisfaction before, so you cannot prove the improvement. Measure first (see Chapter 22).

## Practical Exercise

### 27.7 Exercise: design your support automation

Pick one support channel and plan its AI assistance end to end.

**Step 1 — Choose the channel.** Pick one: a website chatbot, your ticket queue, or your knowledge base. Do one, not all.

**Step 2 — Define the goal and the metric.** Faster response? Higher self-service resolution? Fewer angry customers? Pick one number to measure.

**Step 3 — Measure the baseline.** What is that number now? Average response time, resolution rate, satisfaction score. Write it down.

**Step 4 — List the common questions.** Pull the last month of tickets and find the top 10 repeated questions. These are what the bot and the knowledge base should handle.

**Step 5 — Write the answers.** For each common question, write the answer you want. This becomes your knowledge base and your bot's training.

**Step 6 — Set the handoff rules.** Decide exactly when the bot passes to a human: unsure, asked for, sensitive. Write the rules down.

**Step 7 — Add sentiment routing.** If your tool supports it, set negative-sentiment tickets to rise to the top of the queue.

**Step 8 — Launch small and measure.** Run it on a slice of traffic first. Compare the metric to the baseline. Scale only what proves it actually resolves, not just deflects.

Do one channel well. The list of common questions you build in step 4 is valuable on its own — it shows you exactly what confuses your customers, which is useful even beyond support.

## Checklist

### 27.8 Customer care and support checklist

Before launching AI in support, check these.

- ☐ **You measured the baseline** — response time, resolution rate, satisfaction.
- ☐ **The chatbot hands off to a human easily**, with full context.
- ☐ **You disclose that it is a bot** where required and as good practice.
- ☐ **The bot is trained on your real common questions**, not generic ones.
- ☐ **The bot admits when it does not know** instead of bluffing.
- ☐ **Tickets are routed automatically** to the right team or agent.
- ☐ **Urgent and upset customers are prioritized** in the queue.
- ☐ **Sentiment is a hint to pay attention**, not a verdict on the customer.
- ☐ **You have one knowledge base** feeding the bot, the FAQ, and the agents.
- ☐ **The knowledge base is kept fresh** and reviewed regularly.
- ☐ **AI drafts replies and articles** from solved tickets to capture knowledge.
- ☐ **You keep an audit trail** on every ticket.
- ☐ **Customer data is handled securely** and follows privacy rules (see Chapter 10).
- ☐ **A human owns the outcome** — no black-box accountability.

- ☐ **You measure resolution and satisfaction**, not just deflection.
- ☐ **You fix the support process before automating it.**

If a box is empty, a customer may feel it. Fill it before you let AI answer for you.

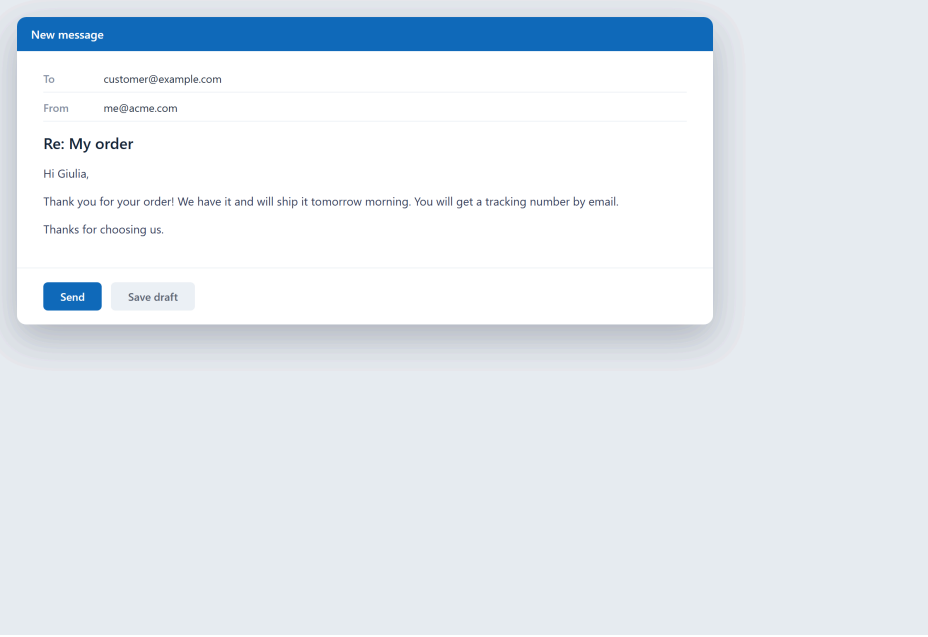
## Key Takeaways

- AI in support shortens the queue by answering common questions instantly and routing the hard ones to the right person fast.
- The TridentCare case (a ServiceNow announcement) reached 96% scheduling automation across 127 markets, cutting manual scheduling from 50% to 4.3% and patient wait times beyond SLA by 57%, with humans supervising instead of doing the work.
- The chatbot's most important feature is a clean handoff to a human with full context — never let a bot block a person.
- One fresh, shared knowledge base should feed the bot, the FAQ, and your agents; sentiment analysis should prioritize the queue, not judge the customer.
- Measure resolution and satisfaction, not deflection, and keep a human accountable for every outcome.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Answer an email in your voice




*A drafted email reply, ready to send*

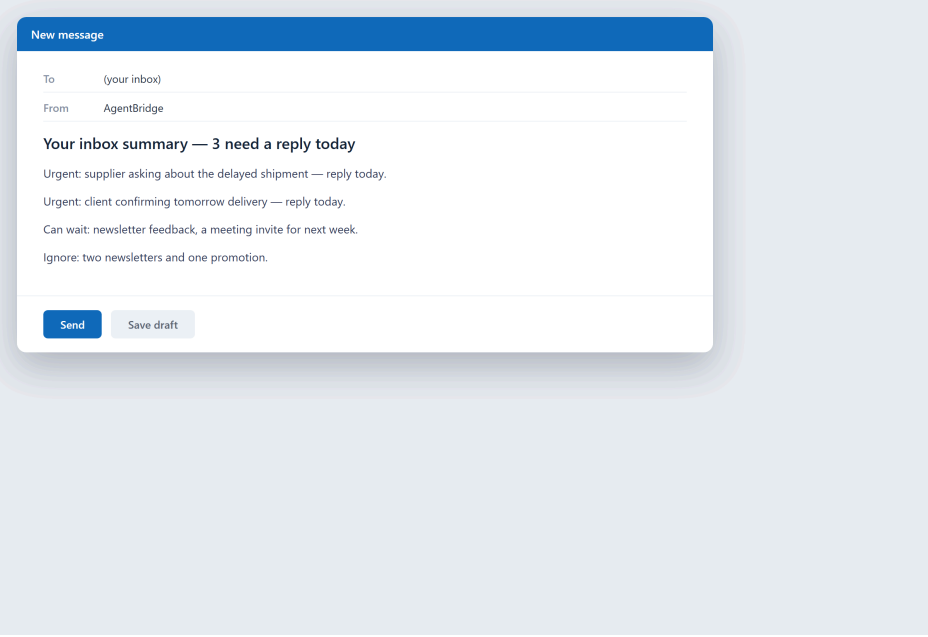
**What you ask:** `Reply to this customer thanking them and confirming we will ship tomorrow: [paste email]`

The agent writes a friendly, professional reply that sounds like you. You read it once, press send, and move on.

*Tip: Set up your mail once with /email. After that, reading and sending work from the chat.*

---

## Catch up on your inbox




*A short summary of the inbox with the urgent items*

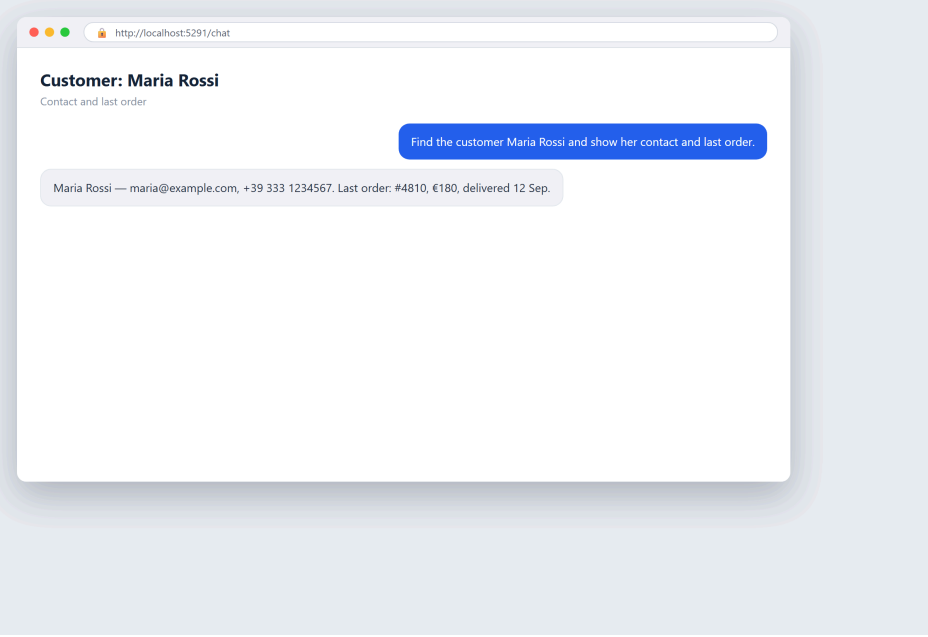
**What you ask:** `Summarise my unread emails and tell me which ones need a reply today.`

The agent reads your unread mail and gives you a short list: what is urgent, what can wait, and what you can ignore. You tackle the real priorities first.

*Tip: A morning summary can be scheduled so this is waiting for you with your coffee.*

---

## Find a customer fast




*A customer record retrieved on request*

**What you ask:** `Find the customer Maria Rossi and show her contact and last order.`

The agent finds the customer and shows the contact details and the last order, so you can help without putting them on hold.

*Tip: Ask for the last three orders if you want the full picture.*

# Chapter 28 — HR and People

## In Simple Words

Human resources is where a business meets its people. Hiring, training, helping new staff settle in, and understanding why people leave — these shape the whole company. They are also full of repetitive work: reading hundreds of CVs, answering the same onboarding questions, sorting training by role, and digging through old records to find why staff quit. AI can take the repetitive part of each job and leave the human part — judgment, fairness, and care — where it belongs.

Think of HR as a funnel and a garden at the same time. The funnel is hiring: many people enter at the top, and you narrow them down to a few good fits. The garden is everyone you already have: you water them with training, you watch for signs of trouble, and you try to keep them growing. AI helps at both ends. In the funnel, it sorts and shortlists faster than a person can read. In the garden, it spots patterns in a thousand small signals that no manager could hold in their head.

This chapter covers four jobs: screening CVs and applications, onboarding new hires, personalized training, and turnover analysis. Each is a place where a small business can save time and make better decisions about people.

One honest idea before we start: people are not invoices. A wrong number on an invoice costs money; a wrong decision about a person costs a life, a career, and your reputation. So AI in HR is a *helper*, never the *judge*. It reads, sorts, suggests, and flags. A person still decides who gets the job, who gets the promotion, and why someone is let go. And in Europe, hiring and worker management are treated as high-risk uses of

AI, with real legal duties. The full treatment of the EU AI Act lives in Chapter 5 — Rules and Legal Responsibility; this chapter shows you what to automate and how, and where the law draws a line. The method for judging whether any of this pays off lives in Chapter 16 — Goals, Costs, and Return on Investment.

## A Bit of History

**1980s–1990s: the HR database.** The first big change in HR work was the computerized personnel file. Instead of paper folders in a cabinet, employee records lived in a database. Payroll, attendance, and personal details became searchable. This was the first time software touched the core of HR, and it set the pattern: store the data, keep the human in charge of decisions.

**1990s–2000s: the applicant tracking system.** As job applications moved online, companies adopted the applicant tracking system, or ATS — software that collects applications, stores them, and lets recruiters search and sort them. A recruiter could now see every applicant for a job in one place and filter by keywords. This brought order to the flood of online applications, but the filtering was simple: it matched words, not people.

**2000s: e-learning and the learning management system.** Training moved from the classroom to the screen. The learning management system, or LMS, delivered courses online and tracked who completed what. This made training scalable, but early versions were one-size-fits-all: everyone watched the same video, regardless of what they already knew.

**2010s: people analytics.** Companies began to analyze HR data the way they analyzed sales data. They looked at which hires stayed, which teams performed, and which signals predicted someone quitting.

People analytics turned HR from a paperwork function into a data function. But it needed large datasets and skilled analysts, so mostly big companies could do it.

**2020s: large language models read and write about people.** Large language models — AI trained on huge amounts of text — can now read a CV and a job description and judge how well they match, draft a personalized training plan, and answer an employee's HR question in plain language. This is the newest step: AI that reads the words people write and reasons about them. It is powerful, and precisely because it is powerful, it is the area where the law is strictest.

The arc: from paper folders, to searchable databases, to keyword filters, to analytics, to AI that reads and reasons about people. Each step took more of the repetitive work off human hands — and each step raised the stakes for fairness, because the decisions are about human lives.

## Curiosity

## 28.5 The law says: tell your workers before AI touches their jobs

Here is a fact every European employer should know before switching on any HR tool that affects staff.

The EU AI Act has a specific rule about workers. In plain words: if you use a high-risk AI system in the workplace — one that affects decisions about your employees — you must tell your workers and their representatives *before* you start using it. The law states it directly in Article 26(7): "Before putting into service or using a high-risk AI system at the workplace, deployers who are employers shall inform workers' representatives and the affected workers that they will be subject to the use of the high-risk AI system."

Two things matter here. First, it is not optional. It is a legal duty, not a courtesy. Second, it is *before*, not after. You cannot roll out an AI screening tool and tell people a month later. You tell them first.

Why does this rule exist? Because people have a right to know when a machine is shaping their working life — whether a tool is reading their application, ranking their performance, or deciding their tasks. Secrecy about AI at work erodes trust and can hide unfairness. Telling people openly is the minimum a fair employer does.

A related warning: the same Act *bans* emotion recognition in the workplace — using AI to read a worker's feelings from their face or voice is prohibited except for medical or safety reasons. A tool that "detects" whether a call-center agent is happy or stressed to score them is squarely forbidden. The full list of banned practices and the staged dates are in Chapter 5. The point here is simple: be open about the AI you use on people, and never secretly read their emotions.

## A Real Business Example

**A mid-size firm that automated hiring and onboarding — an illustrative scenario.**

This example is illustrative. It is a realistic composite, not a reported company result, built to show how the four HR jobs fit together and where the human and the law sit.

Imagine a company of 400 people that hires about 80 staff a year, mostly for a few recurring roles: sales, support, and back-office. Every opening brings 200 to 400 applications. The two HR staff were drowning: reading CVs took days, onboarding questions repeated all day, and nobody knew why people left after a year.

They changed three things.

First, **CV screening with a human check.** They used an AI tool to read each application against the job description and produce a shortlist with a short reason for each match. The HR staff did not let the tool reject anyone. They reviewed the shortlist, and they also sampled a batch of rejected applications every week to check the tool was not unfairly dropping good candidates. The tool cut the screening time from days to hours. The decision stayed human.

Second, **an onboarding assistant.** They built a chatbot trained on their own handbook, policies, and FAQs. New hires could ask, "How do I request leave?" or "What is the expense policy?" and get an instant answer, day or night. The two HR staff stopped answering the same ten questions every day and spent their time on the new hires who genuinely needed a human — the nervous ones, the ones with unusual situations.

Third, **a turnover review.** They pulled two years of HR records into a simple analysis: who left, from which role, after how long, and what their last engagement survey said. The pattern was clear — support staff left most often after 12 to 18 months, and their exit comments clustered around pay and no clear path to grow. That one finding gave them a concrete fix: a pay review and a defined promotion path for support. They did not need fancy AI to act on it; they needed the pattern surfaced, which the analysis did.

Notice the shape of it. AI did the reading, the answering, and the sorting. Humans did the deciding, the caring, and the acting. And because the screening tool affected hiring — a high-risk use — the company told its workers and their representatives before switching it on, as the law requires. That is the model to copy.

# How to Do It

## 28.1 CV and resume screening

Screening CVs (résumés) is the classic HR time sink. A single job opening can bring hundreds of applications, and reading each one takes minutes. AI can read them all and produce a shortlist, so a human reviews a manageable pile instead of a mountain.

**What the AI does.** It reads each CV and the job description, then ranks or sorts the applications by how well they match. It looks at skills, experience, and keywords. Modern tools read the meaning, not just the words, so "managed a team of five" can match "leadership experience."

**The high-risk warning.** In the EU, using AI to screen job applications or filter candidates is a **high-risk** use under the AI Act, because it affects a person's livelihood. That brings real duties: transparency, human oversight, and care against bias. Read Chapter 5 before you deploy any screening tool. Do not treat a CV screener as "just software."

**Bias is the central danger.** An AI trained on past hires learns the patterns of past hires — including any past unfairness. If your company historically hired mostly men for a role, the tool may learn to down-rank women. If it learned to favor one university, it may drop equally good candidates from elsewhere. This is not hypothetical; it has happened in real systems. Guard against it: review the shortlist and the rejected pile for patterns, test the tool on diverse examples, and never let it auto-reject.

**Keep the human decision.** Use AI to shortlist and to explain *why* it matched. A person makes the call. A shortlist with reasons is far better

than a bare ranking, because it lets the reviewer see the tool's logic and catch a bad match.

**Tell candidates and workers.** Be open that AI assists in screening, and tell your workers and their representatives before you switch it on, as the law requires. Candidate data is personal data; handle it under the privacy rules in Chapter 10 — Privacy and GDPR.

## 28.2 Onboarding

Onboarding is the first real experience a new hire has of your company. A good onboarding makes a person feel welcome, clear, and ready. A bad one leaves them confused and anxious. AI helps by answering the endless routine questions instantly, so the human side of onboarding — the welcome, the introductions, the reassurance — gets more time, not less.

**The onboarding assistant.** A chatbot trained on your handbook, policies, and common questions can answer a new hire at 9 p.m. the night before their first day: "What time should I arrive?", "What do I bring?", "How is payroll set up?" This removes the small confusions that make a first day stressful.

**A guided checklist.** AI can generate a personalized onboarding plan for each role: the accounts to open, the systems to access, the people to meet, the training to finish. Instead of one generic list, each new hire gets a path that fits their job. The tool tracks progress and nudges what is missing.

**Draft the welcome materials.** AI can draft the welcome email, the team introduction, and the first-week schedule, so the manager starts from a good draft instead of a blank page. The manager personalizes it. The human touch stays, but the busywork shrinks.

**What AI cannot replace.** The welcome coffee, the mentor, the manager who checks in on day three. Onboarding is emotional as much as practical. Use AI for the information and the checklist; keep a person for the welcome. A new hire who only ever talks to a bot feels like a number.

**Measure the first 90 days.** Track how long it takes a new hire to become productive and how they feel at 30, 60, and 90 days. If onboarding with AI is working, ramp-up time falls and early satisfaction rises. If it does not, the bot is not the answer — the process is.

## 28.3 Personalized training

Old training was one-size-fits-all: everyone watched the same course. Personalized training uses AI to adapt learning to what each person already knows and what their role needs. It is the difference between a set lecture and a tutor who knows where you are stuck.

**Adapt to the person.** AI can give a short starting quiz, see what a person already knows, and skip what they have mastered. It spends time only on the gaps. This respects the learner's time and makes training faster and more relevant.

**Adapt to the role.** A support hire needs different training than a sales hire, even in the same company. AI can assemble a learning path from your content library that fits the role, the level, and the person's pace.

**Draft the content.** AI can turn your existing documents — a policy, a product guide, a recorded meeting — into a short training module with questions to check understanding. This turns knowledge that already exists into training that people can actually use, without a specialist building every course.

**Answer questions while learning.** A learner can ask the AI about a tricky point and get an explanation in plain words, like a tutor. This is where large language models shine: they explain, rephrase, and give examples on demand.

**Keep it honest and checked.** AI-generated training must be reviewed by a human expert before it goes out. A confident but wrong training module spreads mistakes fast. Check the facts, check the tone, and keep a person available for the questions that matter. Training shapes how people work; it must be right.

## 28.4 Turnover analysis

Turnover means people leaving your company. High turnover is expensive — you lose skills, you re-hire, you re-train. Turnover analysis uses AI to look at your HR data and find *why* people leave, so you can fix the cause instead of guessing.

**What the AI looks at.** It combines records: who left, from which role, after how long, their pay, their manager, their last engagement survey, whether they were promoted, how often they were late. On its own, each fact is small. Together, they form a pattern.

**The pattern is the value.** AI can surface something like: "People in role X who have not been promoted in 18 months leave at three times the normal rate." That is a finding you can act on. No manager could hold a thousand records in their head and see it; the analysis can.

**Predict the risk, not the person.** Some tools flag which current employees show patterns similar to those who left. Use this to *start a conversation* — a check-in, a development plan — never to quietly mark someone as "likely to quit" and treat them differently. A prediction is a reason to care, not a reason to act against someone.

**Watch the small signals.** Engagement surveys, a drop in performance, a missed promotion, a change of manager — these are the small signals that often precede a resignation. AI is good at noticing them across many people at once. The human job is to respond with a real conversation.

**Privacy first.** Turnover analysis uses sensitive personal data. Handle it under the privacy rules in Chapter 10, keep it secure, and use it to improve conditions for the group, not to spy on individuals. The goal is a better workplace, not a surveillance system.

## Ethics and Responsibility

HR is the most sensitive place to use AI, because the output is a decision about a human life. The ethical bar is higher here than anywhere else in this book.

**A person decides about people.** AI reads, sorts, and suggests. A human makes every hiring, promotion, and termination decision. Never let a tool auto-reject a candidate or auto-fire a worker. The human must own the outcome and be able to explain it.

**Fight bias actively.** HR AI inherits the biases in its training data. Review outcomes across groups — gender, age, background — and look for unfair patterns. Test the tool before you trust it, and keep testing it after. Fairness is not a setting you turn on once; it is a habit you check.

**Be open with workers.** Tell your workers and their representatives before any high-risk AI touches their jobs, as the law requires. Secrecy about AI at work breaks trust and hides unfairness. Openness is both the law and the right thing.

**Never read emotions at work.** Emotion recognition in the workplace is prohibited under the EU AI Act. Do not buy or use a tool that scores staff by reading their faces or voices. It is banned, and it is wrong.

**Protect personal data.** HR data is among the most sensitive data a company holds — pay, health notes, performance, personal circumstances. Guard it, limit who sees it, and follow Chapter 10. Do not feed employee data into public AI tools without checking the security implications (see Chapter 6 — Cybersecurity in the AI Era).

**Use data to help, not to punish.** Turnover predictions and engagement data should make you a better employer — better pay, better growth paths, better conditions. They should never become a tool to watch, rank, or punish individuals. Measure the workplace, not the person as a target.

**Keep the human in the welcome.** In onboarding and in daily HR, AI handles information; humans handle care. A company that outsources all human contact to a bot loses the trust that makes a workplace work.

## Mistakes to Avoid

**Letting AI reject candidates.** The worst HR mistake. A tool that auto-rejects hides bias and kills good hires. Always keep a human decision.

**Ignoring bias.** Not checking whether the tool treats groups unfairly. Past hiring bias becomes future hiring bias if you do not look. Test and review.

**Switching on a high-risk tool in secret.** Not telling workers and their representatives before using AI that affects them. That breaks the law and trust.

**Buying an emotion-recognition tool.** It is banned in the workplace. A vendor pitch for "mood detection" on staff is a trap.

**Treating a prediction as a verdict.** Acting against an employee because a model flagged them as "likely to quit." A prediction is a reason to talk, not to punish.

**Surveilling staff.** Using HR data to watch and rank individuals instead of improving conditions for the group. This poisons trust.

**A bot-only onboarding.** A new hire who never talks to a human feels like a number. Keep the welcome human.

**Unreviewed AI training.** Publishing AI-generated training that is confidently wrong. A human expert must check it.

**Automating a bad HR process.** If your hiring or onboarding is broken, AI makes a faster broken process. Fix the process first.

**Leaking employee data.** Putting sensitive HR data into insecure AI tools. Check security and privacy first.

**No baseline.** Not measuring time-to-hire, ramp-up time, or turnover before, so you cannot prove the improvement. Measure first (see Chapter 22 — Measuring Results and ROI).

**Forgetting the law is the floor.** Complying with the AI Act does not make a hiring tool fair. Ethics sits above compliance, and you own it (see Chapter 4 — Ethical AI).

## Practical Exercise

### 28.7 Exercise: plan one HR automation responsibly

Pick one HR job and plan its AI assistance end to end, with the human and the law built in.

**Step 1 — Choose the job.** Pick one: CV screening, onboarding, training, or turnover analysis. Do one, not all.

**Step 2 — Define the goal and the metric.** Faster time-to-hire? Faster new-hire ramp-up? Lower turnover? Pick one number to measure.

**Step 3 — Measure the baseline.** What is that number now? How long does screening take? How long to ramp up? What is your turnover rate? Write it down.

**Step 4 — Classify the risk.** Is this a high-risk use under the AI Act? Screening and worker management are. If yes, you have legal duties: transparency, human oversight, bias care. Note them (see Chapter 5).

**Step 5 — Mark each step.** For each step, mark it: **AI does it** (read, sort, draft, surface), **human reviews it** (check the shortlist, review the training), or **human decides it** (the hire, the promotion, the fix). Every decision about a person must be human.

**Step 6 — Build the bias check.** Decide how you will test for unfair patterns across groups, and how often you will review the rejected pile and the outcomes.

**Step 7 — Plan the worker notice.** Write how and when you will inform workers and their representatives before switching the tool on,

as the law requires.

**Step 8 — Launch small and measure.** Run it on one role or one team first. Compare the metric to the baseline. Scale only what proves it helps and treats people fairly.

Do one job well. The bias check and the worker notice you build in steps 6 and 7 are valuable on their own — they force you to see your own hiring patterns, which is useful even beyond the tool.

## Checklist

### 28.8 HR and people checklist

Before using AI on people, check these.

- ☐ **You measured the baseline** — time-to-hire, ramp-up time, turnover rate.
- ☐ **A human makes every decision about a person** — no auto-reject, no auto-fire.
- ☐ **You classified the risk** — screening and worker management are high-risk under the AI Act.
- ☐ **You told workers and their representatives before switching on** any high-risk AI that affects them (Art. 26(7)).
- ☐ **You test for bias** across gender, age, and background, and review the rejected pile.
- ☐ **You never use emotion recognition** on workers — it is prohibited.
- ☐ **The tool explains its matches** so a reviewer can see the logic.
- ☐ **Onboarding keeps a human welcome** — the bot answers, a person welcomes.
- ☐ **AI-generated training is reviewed by a human expert** before it goes out.

- [ ] **Turnover predictions start a conversation**, not a punishment.
- [ ] **HR data is kept secure** and follows privacy rules (see Chapter 10).
- [ ] **You use data to improve conditions for the group**, not to surveil individuals.
- [ ] **You fix the HR process before automating it.**
- [ ] **You treat compliance as the floor** and your own fairness as the standard above it.

If a box is empty, a person may feel it — and the law may reach it. Fill every box before you let AI near your people.

## Key Takeaways

- AI in HR takes the repetitive work — reading CVs, answering onboarding questions, sorting training, surfacing turnover patterns — and leaves the deciding, the caring, and the acting to humans.
- Hiring and worker management are high-risk uses under the EU AI Act: a person must decide, you must guard against bias, and you must inform workers and their representatives before switching on any high-risk AI that affects them.
- Emotion recognition in the workplace is banned; never buy or use a tool that scores staff by reading their feelings.
- Use HR data to make the workplace better for the group, never to surveil or punish individuals, and keep the human welcome in onboarding.
- Measure the baseline, keep a human in every people decision, and treat the law as the floor and fairness as the standard above it.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

# A simple staff handbook

**Acme Studio**
Small digital services company

# Manual
Employee Handbook

| Manual No. | Version | Effective Date | Document Owner |
|---|---|---|---|
| **AS-HB-001** | **1.0** | **1 October 2026** | **Maria (HR)** |

## 1. Purpose and Audience

This handbook provides all employees of Acme Studio with clear guidance on working hours, holidays, remote work, dress code, and who to contact for help. It is designed to make your experience here smooth, fair, and transparent.

This handbook applies to all employees of Acme Studio, including full-time and part-time staff. Please read it carefully and reach out with any questions. Our values are simple: be kind, be clear, and ask early rather than struggle alone.

## 2. Table of Contents


| Section | Title | Page |
|---|---|---|
| 1 | Purpose and Audience | 1 |
| 2 | Table of Contents | 1 |
| 3 | Getting Started | 2 |
| 4 | Main Procedures | 2 |
| 5 | Troubleshooting | 3 |
| 6 | Safety and Maintenance | 3 |
| 7 | Support and Contacts | 4 |
| 8 | Revision History | 4 |


## 3. Getting Started

Welcome to Acme Studio! Here is what you need to know to settle in quickly and comfortably:

1. Introduce yourself to your team lead, Anna, who will guide you through your day-to-day work and answer any questions about your role.
2. Meet Maria in HR to review your contract, pay details, and set up your time-off tracking.
3. Collect your equipment from Luca in IT, including your laptop, peripherals, and any software access you need.
4. Familiarise yourself with the core hours (10:00 to 16:00) and make sure you are reachable during that window, whether you are in the office or working remotely.

## 4. Main Procedures

### 4.1 Working Hours and Core Hours

1. Standard working hours are Monday to Friday, 9:00 to 18:00, with a one-hour lunch break.
2. Core hours are 10:00 to 16:00. During this time, all employees should be reachable and available for meetings and collaboration.
3. If you need to adjust your schedule outside core hours, discuss it with your team lead, Anna, in advance.

### 4.2 Holidays and Time Off

1. Every employee receives 26 paid vacation days per year, in addition to national holidays.
2. Submit your time-off request at least two weeks in advance through your team lead, Anna.
3. For questions about your remaining balance, holiday pay, or special leave, contact Maria in HR.

### 4.3 Remote Work

1. You may work from home up to two days per week. Coordinate your remote days with your team lead to ensure coverage.
2. During remote work, you must have a stable internet connection and be available during core hours (10:00 to 16:00).
3. All necessary equipment (laptop, peripherals, software) is provided by the company. Contact Luca in IT if you need additional hardware or support.

### 4.4 Dress Code

1. In the office, the dress code is smart casual. Aim for neat, comfortable attire that is appropriate for a professional environment.
2. When working from home and joining video calls, dress comfortably but presentably.
3. There is no strict uniform policy; use your judgement and when in doubt, ask your team lead.

## 5. Troubleshooting

| Issue | Probable Cause | Solution |
|---|---|---|
| Cannot reach team lead for a work question | Team lead is in a meeting or away from desk | Send a message and wait for a reply. If urgent, contact Maria in HR as a backup. Remember: ask early rather than struggle |

*A friendly staff handbook, ready to share*

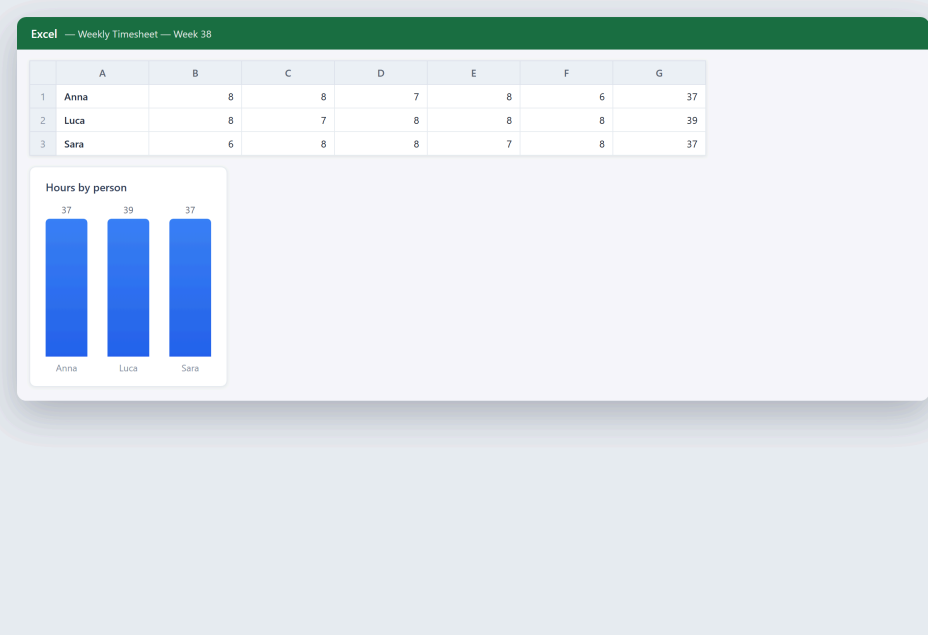
**What you ask:** `Write a short employee handbook covering working hours, holidays, remote work rules and who to ask for help.`

The agent writes a clear, friendly handbook in plain language, with each topic on its own page. Update it any time by asking — 'add a line about the new parking rule'.

*Tip: Keep it in your documents area so the agent can answer questions from it later.*

---

## A weekly timesheet




*A weekly timesheet with totals per person*

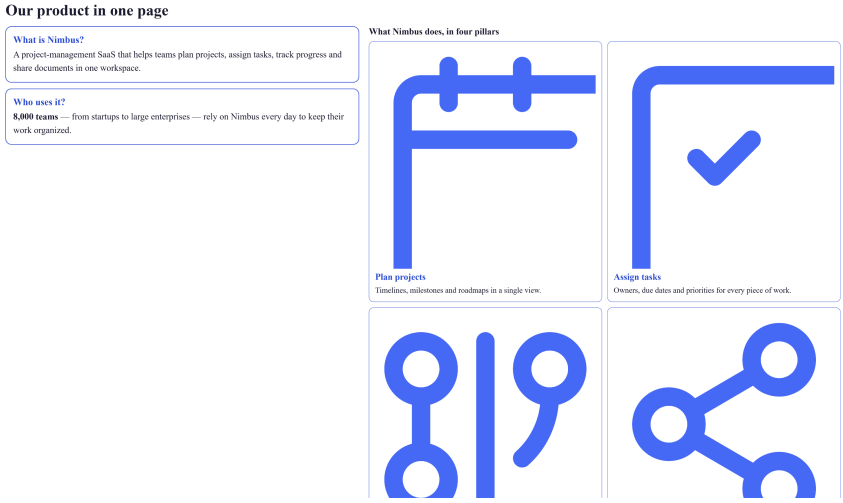
**What you ask:** `Create a weekly timesheet with people, projects, hours per day and a total per person.`

The agent builds the timesheet grid with daily columns and automatic totals. Fill it in or attach your rough hours and let it sort them.

*Tip: Multiply hours by each rate and you have a billing sheet too — just ask.*

---

## Train your team fast




*A step-by-step training slide*

**What you ask:** `Make a training deck on how to handle a customer refund, step by step.`

The agent turns the process into clear slides, one step each, easy to follow. Use it for onboarding and refreshers.

*Tip: Add a final slide with 'who to ask' so people know where to get help.*

# Chapter 29 — Operations and Production

## In Simple Words

Operations is the engine of a business. It is everything that turns inputs into outputs: making a product, moving it, keeping the machines running, checking the quality, and keeping people safe. Production is the part of operations that actually makes the thing. When the engine runs well, customers get what they ordered, on time, without defects. When it stutters, everything downstream feels it.

Think of your operation as a factory floor, whether you make widgets or deliver services. Machines run, goods move, people work, and something can always go slightly wrong — a machine that is about to fail, a batch with a defect, a delivery that will be late, a worker in an unsafe spot. A good operator sees these things early. AI helps by watching everything at once and flagging the small signals that a tired human eye misses.

This chapter covers four jobs: predictive maintenance (fixing machines before they break), quality control (catching defects automatically), logistics (moving goods and information efficiently), and workplace safety (keeping people out of harm's way). Each is a place where a small business can cut waste, raise quality, and protect its people.

One honest idea first: AI in operations is a *watcher and a helper*, not an autopilot that runs the floor alone. It senses, predicts, and suggests. A person still decides when to stop a line, when to reject a batch, and when to send someone home because it is not safe. The method for

judging whether any of this pays off lives in Chapter 16 — Goals, Costs, and Return on Investment; this chapter shows you what to automate and how. To see where operations sits on the impact-and-effort map for your whole business, look at Chapter 12 — Where AI Can Help Your Business.

## A Bit of History

**1900s–1950s: the assembly line and preventive maintenance.** Modern production began with the moving assembly line, which broke work into small repeatable steps. Alongside it came *preventive* maintenance — the habit of servicing a machine on a fixed schedule, like changing a car's oil every few thousand miles, whether it needed it or not. This reduced breakdowns but wasted service on machines that were fine.

**1960s–1980s: automation and quality control.** Factories added automated machines and formal quality control — inspecting products against a standard and sorting the good from the bad. Statistical quality methods caught defects in batches. This raised quality but still relied on humans to inspect and on fixed rules to service machines.

**1990s: sensors and the digital supply chain.** Cheap sensors let machines report their own condition — temperature, vibration, run hours. Supply chains became digital, with software tracking goods from supplier to customer. For the first time, an operator could see, on a screen, what was happening across the floor and across the supply chain. But the data was mostly read by hand.

**2000s: machine learning predicts the failure.** Machine learning — software that learns patterns from many examples — changed maintenance. Instead of servicing on a fixed schedule, you could learn from sensor data when a machine was *actually* about to fail. A rising

vibration pattern that a human could not feel became a clear warning days in advance. This was the birth of *predictive* maintenance.

**2010s: computer vision inspects quality.** Computer vision — AI that reads what a camera sees — began to inspect products automatically. A camera could spot a crack, a scratch, or a missing part faster and more consistently than a human inspector, and without getting tired. Quality control moved from sampling to checking every unit.

**2020s: agents run the workflow.** Large language models and AI agents — software that takes a whole task and carries it through several steps — now run parts of the operational workflow itself: reading a request, pulling data from backend systems, and executing the next step. The SOK Finance example below is exactly this: AI agents handling the day-to-day service operations of a financial service center, with humans supervising.

The arc: from fixed schedules, to manual inspection, to sensor data, to AI that predicts and sees, to agents that act. Each step moved the watching and the routine work to software and left humans to decide and to intervene.

## Curiosity

## 29.5 The service center that put AI agents on the front line

When we hear “operations and production,” we picture a factory. But operations also means the service center that keeps a business running — the place where requests arrive and someone must act on them fast.

SOK Finance, in Finland, runs a service center called Palveluässä that provides financial management and payroll services for the S Group — a customer-owned Finnish network of retail and service companies with roughly 2,000 outlets. Every day, requests arrive: “Send me a copy of this invoice,” “Change the due date on this payment.” Routine, high-volume, and time-sensitive.

The curiosity is what the company did with it. Instead of hiring more people to answer the same requests, they put AI *agents* on the front line to handle the routine ones, so the human staff could focus on the cases that needed judgment. That shift — from humans doing every request to agents doing the routine and humans supervising — is the same quiet revolution that happened on the factory floor, now applied to a service center. The full story, with the real source, is below.

## A Real Business Example

**SOK Finance: AI agents running the service-center workflow.**

SOK Finance operates the Palveluässä service center, which provides financial management and payroll services for the S Group, a customer-owned Finnish network of retail and service companies with approximately 2,000 outlets. The service center handles a steady stream of routine requests from across that network — things like asking for a copy of an invoice or changing a payment due date. Each request is small, but together they are a large, repetitive workload that must be handled quickly and consistently.

According to a CGI press release (Helsinki, 23 April 2026), CGI designed, implemented, and deployed a **multi-agent AI solution built on AWS Bedrock** for SOK Finance, bringing AI into live production use in financial administration and customer service. A

“multi-agent” solution means several AI agents work together on a task, each handling a step. In this case, the agents process incoming customer-service messages, retrieve the needed data from backend systems, and automatically carry out parts of the process.

The release describes the solution as significantly accelerating routine processes such as **invoice copy requests and due-date changes**, and as improving efficiency and consistency compared with the previous manual handling. CGI was responsible for the design, the implementation, the deployment, and the integration with SOK Finance’s key systems.

Two things are worth noticing. First, the human role shifts from *doing every request* to *supervising the agents and handling the exceptions*. The agents take the routine; the people take the unusual and the sensitive. Second, the win is consistency as much as speed — an agent follows the same careful steps every time, where a tired person on a busy afternoon might slip.

A note on the source and the numbers: the figures above come from CGI’s published announcement. The release describes the results in qualitative terms — faster processes, better consistency — and does not publish a hard percentage for time saved or cost reduced. Treat the outcome as the company’s reported experience, and remember that your own numbers will depend on your volume and your systems. The point of the case is the *pattern*: AI agents running a routine service workflow in production, with humans supervising, at the scale of a 2,000-outlet network.

## How to Do It

## 29.1 Predictive maintenance

Predictive maintenance means fixing a machine *before* it breaks, by predicting the failure from data instead of waiting for it to happen or servicing on a fixed schedule.

**The old way and its cost.** Traditionally you had two choices: run a machine until it failed (and pay for an unplanned stop), or service it on a fixed schedule (and waste money servicing machines that were fine). Both lose. An unplanned breakdown stops production at the worst moment and costs far more than a planned fix.

**How AI predicts.** You put sensors on the machine to measure things like vibration, temperature, sound, and run hours. AI learns the normal pattern and spots the early signs of trouble — a vibration that is slowly rising, a temperature that runs a little hot. Those small changes often appear days before a failure. The AI warns you in time to fix the machine during a planned stop, not in the middle of a run.

**What it saves.** The big saving is avoiding unplanned downtime — the surprise breakdown that halts everything. A planned fix on a Tuesday afternoon is cheap; a breakdown during your biggest order of the month is expensive. Predictive maintenance turns the second into the first.

**Start small.** You do not need sensors on every machine. Start with the one or two machines whose failure hurts most — the bottleneck, the one with the long repair time, the one that stops the whole line. Put a few sensors on it and watch. Prove the value there before you expand.

**The human decides the stop.** AI flags the risk; a person decides when to take the machine down. Do not let the system stop production

on its own without a human confirming the call. The warning is the value; the decision stays human.

## 29.2 Quality control

Quality control means checking that a product meets its standard and catching defects before the product reaches the customer. AI changes quality control from *sampling* to *checking every unit*, and from *tired eyes* to *consistent ones*.

**Computer vision inspects.** A camera plus AI can look at each product on the line and spot a crack, a scratch, a missing part, a wrong label, or a bad seam. It checks every unit, not just a sample, and it does not get tired or bored. A human inspector checking thousands of units a day will miss things; the camera will not.

**Consistency is the win.** Humans vary. A person is sharper in the morning, slower after lunch, and easily swayed by what they saw last. AI applies the same standard to every unit, all day. That consistency is worth a lot in a business where a defect reaching a customer costs a return, a complaint, or a reputation.

**Catch it early.** The earlier you catch a defect, the cheaper it is. A defect caught at the machine that made it costs one part. The same defect caught at final assembly costs a rework. Caught by the customer, it costs a return and trust. AI at each station catches problems at the source, not at the end.

**Start with the costly defect.** Do not try to inspect everything at first. Find the defect that costs you the most — the one that causes the most returns or complaints — and put a vision check on that. Prove it catches the expensive problem, then add more checks.

**Keep a human for the judgment call.** AI can flag a suspect unit; a person decides whether it is truly defective, especially for borderline cases. Do not let a vision system scrap good product because it was too strict. Review the rejects and tune the threshold.

## 29.3 Logistics

Logistics is the movement of goods and information: getting the right thing to the right place at the right time, at the lowest cost. It is a puzzle of routes, stock, timing, and suppliers, and AI is very good at puzzles like this.

**Route and delivery optimization.** AI can plan delivery routes that save miles, fuel, and time, taking traffic, delivery windows, and load size into account. For a business with a fleet, even a small saving per route adds up fast across a year.

**Stock and demand forecasting.** AI looks at your sales history and predicts what you will need and when, so you stock enough without over-buying. Too little stock means a missed sale; too much means cash tied up and waste. AI balances the two by learning your patterns and seasonality.

**Spot the bottleneck.** AI can see where goods slow down — a supplier that is always late, a warehouse step that backs up, a route that always runs over. Seeing the bottleneck is the first step to fixing it. The whole value is making the invisible delay visible.

**Connect the systems.** Logistics AI works best when it can see your orders, your stock, and your deliveries together. Connecting AI to the systems you already use — your ordering, your inventory, your tracking — is what makes the picture complete. The how-to for that connection is in Chapter 19 — Connecting AI to Systems You Already Use.

**Keep a human for the exception.** AI plans the routine; a person handles the surprise — the strike, the storm, the supplier that fails. Do not let an optimized plan run into a real-world disruption without a human ready to override it. The plan is a starting point, not a straitjacket.

## 29.4 Workplace safety

Workplace safety means keeping people out of harm's way. AI can watch for unsafe conditions and unsafe behavior and warn before an accident happens. This is one of the most valuable uses of AI, because the thing it protects is a person.

**Computer vision watches for hazards.** Cameras plus AI can spot a worker without the right protective gear, a person standing in a dangerous zone, a spill on the floor, or a blocked emergency exit. When it sees one, it raises an alert so the danger is fixed before someone is hurt.

**Predict the risky moment.** AI can learn when accidents are most likely — a certain shift, a certain machine, a certain time of day when people are tired — and increase the watch then. It is like having a safety officer who never blinks and sees the whole floor at once.

**A serious legal line: do not read emotions.** AI that infers a worker's *emotions* from their face or voice is prohibited in the workplace under the EU AI Act. Safety vision is about *hazards and gear*, not about how a worker *feels*. Keep the camera on the floor and the machine, not on the person's mood. The full list of banned practices is in Chapter 5 — Rules and Legal Responsibility.

**Tell your workers.** If you use AI-based safety monitoring that affects workers, you must inform your workers and their representatives

before you start, as the law requires for high-risk AI at the workplace. Be open about what the cameras watch and why. Secrecy breaks trust. The worker-notice rule is covered in this chapter's Curiosity and in Chapter 5.

**Use it to protect, not to punish.** Safety data should make the workplace safer — fix the hazard, change the process, train the team. It should not become a tool to discipline individuals for every small slip. Use it to find and remove danger, not to build a case against a worker.

## Ethics and Responsibility

Operations touches safety and the environment, so the ethical stakes are real and concrete.

**Safety decisions stay human.** AI can warn of a hazard, but a person decides when to stop a line or send someone home. Never let a system make a safety-critical decision with no human in the loop. A false "all clear" can hurt someone.

**Respect workers' privacy and rights.** Cameras and sensors in the workplace watch over people. Use them for safety and operations, not for surveillance. Tell workers what is watched and why, and follow the privacy rules in Chapter 10 — Privacy and GDPR.

**Never read emotions.** Emotion recognition in the workplace is banned. Keep monitoring on hazards and gear, never on a worker's feelings.

**Protect operational data.** Sensor data, production schedules, and supply-chain records are sensitive and valuable. Keep them secure and, where it matters, within your own environment. The security basics are in Chapter 6 — Cybersecurity in the AI Era, and the self-hosting option in Chapter 8 — Self-Hosting: Keep Your Data Under Control.

**Be honest about results.** A quality or efficiency number that looks too good should be checked before you report it. Report the real figures, including the misses, so you can fix what is still broken.

**Use data to improve, not to punish.** Operational and safety data should make the work better and safer for everyone. When it becomes a stick to beat individuals with, it poisons the workplace and hides the real problems.

## Mistakes to Avoid

**Waiting for the breakdown.** Sticking with run-to-failure when a planned fix was cheap and predictable. Predict and act early.

**Sensors without a plan.** Collecting data you never act on. Start with the machine that hurts most and tie every sensor to a decision.

**Letting AI stop or scrap on its own.** No human check on a line stop or a product reject. Keep the human decision.

**Too-strict vision checks.** A quality system that scraps good product because the threshold is too tight. Review rejects and tune it.

**Inspecting the wrong defect.** Putting vision on a cheap problem while the expensive one slips through. Target the costly defect first.

**A plan that ignores the real world.** Letting an optimized logistics plan run into a disruption with no human override. Keep a person ready.

**Emotion recognition at work.** Using AI to read workers' feelings. It is banned and wrong.

**Secret monitoring.** Not telling workers before AI-based safety or operations monitoring starts. That breaks the law and trust.

**Surveillance creep.** Using safety and operations data to watch and punish individuals instead of removing hazards.

**Automating a broken process.** If the line or the supply chain is a mess, AI makes a faster mess. Fix the process first.

**No baseline.** Not measuring downtime, defect rate, or delivery time before, so you cannot prove the gain. Measure first (see Chapter 22 — Measuring Results and ROI).

**Over-promising the numbers.** Quoting a vendor's best-case figure as your result. Use your own measured numbers.

## Practical Exercise

### 29.7 Exercise: plan one operations automation

Pick one operations job and plan its AI assistance end to end, with safety and the human decision built in.

**Step 1 — Choose the job.** Pick one: predictive maintenance, quality control, logistics, or workplace safety. Do one, not all.

**Step 2 — Define the goal and the metric.** Less downtime? Fewer defects? Faster delivery? Fewer safety incidents? Pick one number to measure.

**Step 3 — Measure the baseline.** What is that number now? Hours of unplanned downtime, defect rate, on-time delivery, incidents per month. Write it down.

**Step 4 — Find the costly target.** Identify the single most expensive failure in that job — the machine whose breakdown hurts most, the defect that costs the most, the route that always runs late, the hazard that causes the most harm. Target that first.

**Step 5 — Mark each step.** For each step, mark it: **AI does it** (sense, predict, inspect, plan), **human reviews it** (confirm the warning, check the reject), or **human decides it** (stop the line, scrap the batch, send someone home). Every safety-critical decision must be human.

**Step 6 — Check the legal line.** If the job involves monitoring workers, confirm you are watching hazards and gear, not emotions, and plan how you will inform workers and their representatives before you start.

**Step 7 — Connect the data.** Decide which systems the AI needs to see — sensors, inventory, tracking — and how you will connect them (see Chapter 19).

**Step 8 — Launch small and measure.** Run it on one machine, one line, or one route first. Compare the metric to the baseline. Scale only what proves it works.

Do one job well. The costly-target analysis in step 4 is valuable on its own — it shows you where your operation actually loses the most money or causes the most harm, which is useful even before you buy any tool.

## Checklist

## 29.8 Operations and production checklist

Before automating any operations task, check these.

- ☐ **You measured the baseline** — downtime, defect rate, delivery time, safety incidents.
- ☐ **You targeted the most costly failure first**, not the easiest one.
- ☐ **A human makes every safety-critical decision** — stopping a line, scrapping a batch.
- ☐ **Predictive maintenance is tied to a real action**, not just collected data.
- ☐ **Quality checks are tuned** so they do not scrap good product.
- ☐ **Logistics AI can see your orders, stock, and deliveries together.**
- ☐ **A human can override the optimized plan** when the real world disrupts it.
- ☐ **Safety monitoring watches hazards and gear, never emotions** (emotion recognition is banned).
- ☐ **You informed workers and their representatives** before any AI monitoring that affects them.
- ☐ **Operational data is kept secure**, not sent to public AI services.
- ☐ **You use data to remove hazards and improve the process**, not to punish individuals.
- ☐ **You fix the broken process before automating it.**
- ☐ **You report your own measured numbers**, not the vendor's best case.

If a box is empty, the risk — to your product, your people, or your trust — is still yours. Fill it before you let AI near the floor.

## Key Takeaways

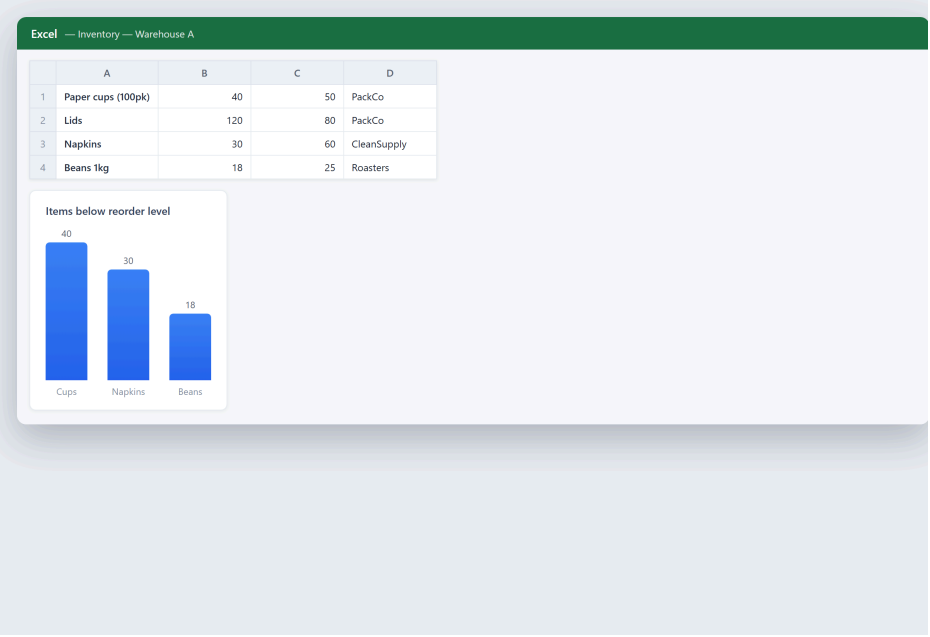
- AI in operations is a watcher and helper: it senses, predicts, inspects, and plans, while a human keeps every safety-critical decision.

- The SOK Finance case (a CGI announcement) put multi-agent AI on AWS Bedrock into live production at a service center serving a ~2,000-outlet network, accelerating routine requests like invoice copies and due-date changes, with humans supervising the exceptions.
- Predictive maintenance turns a surprise breakdown into a cheap planned fix; computer-vision quality control checks every unit consistently instead of sampling with tired eyes.
- In workplace safety, monitor hazards and protective gear — never workers' emotions, which the EU AI Act bans — and tell workers before any monitoring that affects them.
- Target the most costly failure first, keep a human override for real-world disruptions, and measure your own baseline before you trust any vendor's number.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Keep stock in order




*An inventory sheet with low-stock items highlighted*

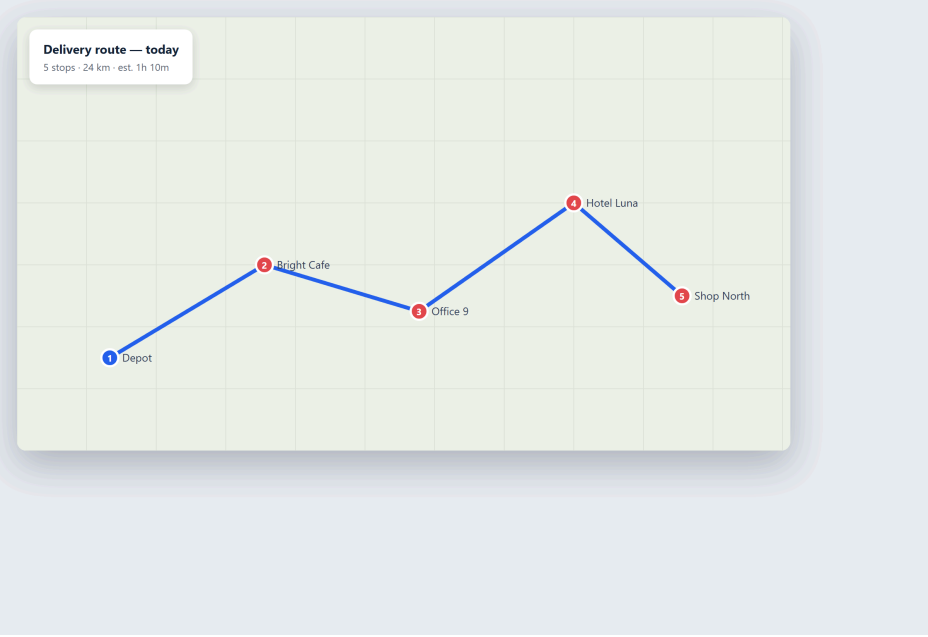
**What you ask:** `Make an inventory spreadsheet with item, quantity, reorder level and supplier, and highlight what is below the reorder level.`

The agent sets up the inventory sheet and marks the items that need reordering. Update the quantities and ask it to re-check any time.

*Tip: A weekly scheduled check can tell you what to reorder before you run out.*

---

## Plan a delivery route




*A delivery route mapped across the stops*

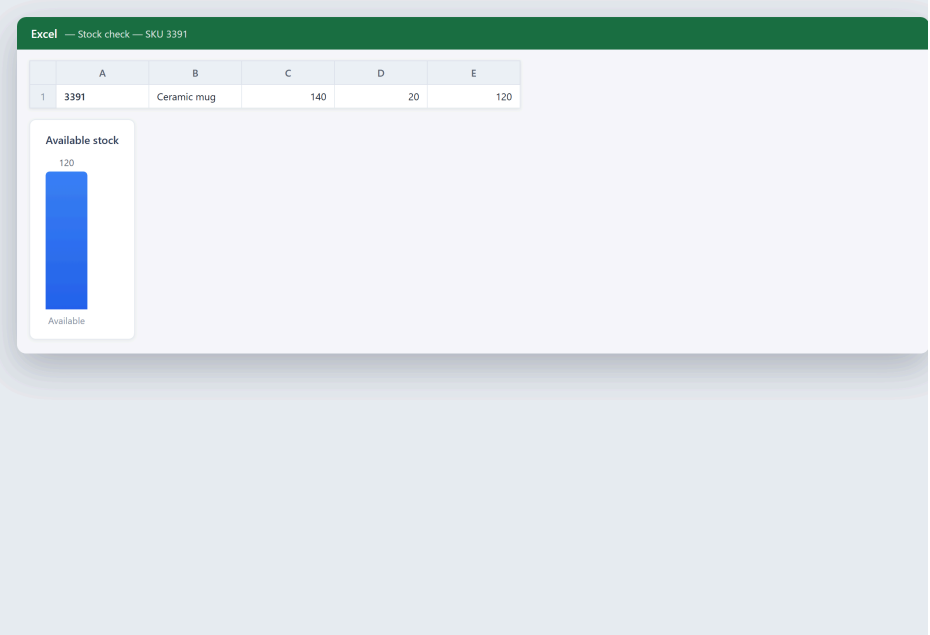
**What you ask:** `Plan the best route for these five delivery addresses and show it on a map.`

The agent plots the stops on a map in an efficient order and gives you the distance and estimated time. You follow the route and save fuel.

*Tip: Add time windows ('stop B before noon') and the agent factors them in.*

---

## Is it in stock?




*A live stock check answered in seconds*

**What you ask:** `Do we have item SKU 3391 in stock, and how many?`

The agent checks the stock in your system and answers with the quantity, so you can promise or promise-against with confidence.

*Tip: Pair with a low-stock alert scheduled daily to avoid surprises.*

# Chapter 30 — IT and Leadership

## In Simple Words

IT and leadership are two halves of one job: keeping the business running and steering it in the right direction. IT keeps the systems, the data, and the network working and safe. Leadership turns all of that into decisions — where to invest, what to fix first, which risk to take. AI helps both halves. It answers the routine IT questions, watches for threats, builds the reports, and lays out the options so a leader can choose well.

Think of IT as the plumbing and the security of a building, and leadership as the people deciding where the building goes next. When the plumbing leaks, everyone feels it. When the security gate is left open, everyone is exposed. And when the leaders have to decide with no clear picture, they guess. AI patches the leaks faster, watches the gates, and gives the leaders a clear picture to decide from.

This chapter covers four jobs: internal support (helping staff with their tech), cybersecurity (defending against attack), management reports (turning data into a clear picture), and decision support (laying out the options for a leader). Each is a place where a small business can run smoother, stay safer, and decide better.

One honest idea first: AI in IT and leadership is a *copilot*, not the *captain*. It answers, watches, summarizes, and suggests. A person still makes the call — whether to shut a system down, which vendor to trust, which risk to accept. The bigger the decision, the more the human judgment matters. The method for judging whether any of this pays off lives in Chapter 16 — Goals, Costs, and Return on Investment; this

chapter shows you what to automate and how. To see where IT and leadership sit on the impact-and-effort map for your whole business, look at Chapter 12 — Where AI Can Help Your Business.

## A Bit of History

**1980s–1990s: the help desk.** IT support began as the help desk — a phone number staff called when something broke. A person logged the problem, fixed what they could, and escalated the rest. It worked, but it was slow, and the same simple questions — forgotten passwords, printer trouble — ate most of the day.

**1990s–2000s: dashboards and business intelligence.** Companies started pulling data into dashboards and business-intelligence tools — software that turns raw numbers into charts and summaries. For the first time, a manager could see sales, costs, and performance on one screen. But building those reports still took an analyst, and you could only see what the report was built to show.

**2000s: automated security monitoring.** As attacks grew, IT added automated security tools — firewalls, intrusion detection, and alert systems that watch the network for suspicious activity. These caught more than a human could, but they also produced a flood of alerts, most of them harmless, and a tired analyst had to sort the real threat from the noise.

**2010s: machine learning reads the threat.** Machine learning — software that learns patterns from many examples — changed security. Instead of matching known attack signatures, AI learned what "normal" looked like on the network and flagged the unusual. It cut through the alert noise by spotting the few signals that actually mattered.

**2020s: large language models answer and advise.** Large language models — AI trained on huge amounts of text — can now answer an employee's IT question in plain language, summarize a security incident, build a management report from raw data, and lay out the options and trade-offs for a leader's decision. This is the newest step: AI that reads, explains, and advises across all four jobs at once. The IBM vendor-risk example below shows the same idea applied to deciding which suppliers are safe.

The arc: from a phone-based help desk, to dashboards, to automated alerts, to AI that reads the threat and answers the question, to AI that advises the decision. Each step moved the routine work to software and left humans to decide and to lead.

## Curiosity

## 30.5 The leader's oldest problem: deciding without a clear picture

For as long as there have been managers, there has been the same complaint: "I have to decide, but I can't see clearly." The data is scattered across systems. The report takes a week to build. The dashboard shows last month, not today. So leaders decide on gut feel and hope.

AI changes that more than almost anything else in this book. It can pull the scattered data together, build the picture on demand, and answer a leader's question in plain words in seconds — "Which product line is losing money?", "Where is our cash next month?", "Which supplier is the biggest risk?" The leader still decides. But now they decide with a clear picture instead of a guess. That is the quiet revolution in leadership: not AI making the call, but AI making the call *informed*.

The IBM vendor-risk story below is exactly this, applied to one hard decision — which suppliers can we trust?

## A Real Business Example

**IBM: AI-assisted third-party (vendor) risk assessment.**

Every company that buys from outside suppliers carries *third-party risk* — the risk that a vendor you depend on turns out to be unreliable, insecure, or non-compliant. Checking each vendor is slow, careful work. A common way to check is a detailed questionnaire the vendor fills in, which a human assessor then reads and scores. When you have hundreds of vendors, that work becomes a bottleneck.

IBM's Client Engineering published a real engagement on this, called "Evaluating Third Party Risk with AI." The client was a financial institution that ran **over 1,000 vendor assessments a year**, and each assessment took **about 45 working hours** to complete — reading the questionnaire, checking the evidence, and scoring the risk. That is a huge amount of skilled time spent on the same careful task, over and over.

IBM used its watsonx.ai platform to assist the assessors. The AI read the vendor questionnaire responses and the supporting evidence and helped the assessor understand the quality of what the vendor had provided, so the human could focus on judgment instead of reading everything from scratch. The published estimate was **about a 20% reduction in assessment time**, which across that workload added up to **roughly 10,000 working hours saved** and **around $800,000 in labor cost per year**.

A note on the "50%" figure. You may see this kind of IBM vendor-risk work quoted as "about a 50% reduction in time." That number is **not**

what IBM's published case reports. The published estimate for this engagement is about 20%, with the savings expressed as ~10,000 hours and ~$800k a year. Treat the "50%" as unverified, and use the published figures — ~20% time reduction, ~10,000 hours, ~$800k — as the real, sourced numbers. (Source: IBM Client Engineering, "Evaluating Third Party Risk with AI.")

Two things are worth noticing. First, the human assessor stayed in charge. The AI did not approve or reject a vendor; it helped the human read faster and judge better. Second, the saving came from the *reading and checking* part of the job — the slow, careful reading that AI is good at — while the *decision* about the vendor stayed with the person. That is the pattern to copy in your own risk work: let AI do the reading, keep the human for the judgment.

## How to Do It

### 30.1 Internal support

Internal support means helping your own staff with their technology — the forgotten password, the printer that will not work, the software that needs setting up. It is the help desk, and it is full of the same repeated questions. AI is very good at repeated questions.

**The IT help-desk assistant.** A chatbot trained on your own IT knowledge base can answer staff questions instantly: "How do I connect to the VPN?", "How do I reset my password?", "How do I install this app?" This removes the routine tickets that eat a support team's day, so the team can focus on the real problems.

**Resolve the simple ones automatically.** Some tickets need a simple action — reset a password, unlock an account, reinstall an app.

AI can do these automatically or draft the action for a human to approve. The routine work disappears from the queue.

**Route the hard ones fast.** When the bot cannot solve it, it should pass the ticket to the right person with the context intact — what the staff member already said and tried — so the human does not start from zero. A clean handoff is the most important feature.

**Build from your own tickets.** Pull the last few months of IT tickets and find the most common questions. Those become the bot's knowledge base. The same chatbot technology, applied to customers rather than staff, is covered in Chapter 27 — Customer Care and Support; here it is turned inward to serve your own team.

**Keep a human for the hard problem.** AI handles the routine; a person handles the outage, the strange bug, the security incident. Never let a bot be the only path when something is genuinely broken. Staff must always be able to reach a human.

## 30.2 Cybersecurity

Cybersecurity means defending your systems, data, and network from attack. AI is now a core tool on both sides of this fight — defenders use it to spot threats, and attackers use it too — so it matters that you understand what it can and cannot do for you.

**AI spots the unusual.** AI learns what "normal" looks like on your network — normal traffic, normal logins, normal data access — and flags what is off. A login from a strange country at 3 a.m., a sudden flood of file downloads, a device acting oddly. These small signals, seen at once across the whole network, are how AI catches a threat a human would miss.

**Cut through the alert noise.** Security tools produce thousands of alerts, most harmless. AI ranks them so the real threat rises to the top and the noise falls away. This is one of its biggest values: not more alerts, but *fewer, better* alerts that a small team can actually handle.

**Respond faster.** When AI flags a real threat, it can also suggest or take a quick first action — isolate an infected machine, block a suspicious address — to stop the spread while a human investigates. Speed matters in an attack; the first minutes decide how much damage is done.

**The full security picture is its own topic.** The threats, the defenses, and the human habits that matter most are covered in depth in Chapter 6 — Cybersecurity in the AI Era, and the secure rollout in Chapter 20 — Implementing AI Securely. Read those before you rely on AI for your defense. AI is a powerful tool, not a magic shield.

**Keep a human for the big call.** AI can isolate a machine, but a person decides whether to shut down a system, whether to pay or refuse a ransom, whether to call the authorities. In a real incident, the human judgment is the last line of defense. Never let automation make the big security decisions alone.

## 30.3 Management reports

A management report turns raw data into a clear picture a leader can read and act on — profit and loss, sales by product, costs by category, cash position. AI changes who builds the report and how fast.

**The report builds itself.** Instead of an analyst pulling numbers into a spreadsheet every week, the report can generate on a schedule from your live systems and land in the leader's inbox. The numbers are always current, and nobody has to remember to make it.

**Plain-language summaries.** AI can read the numbers and write a short summary in plain words: “Revenue rose 8% this month, driven by product X; costs rose 3%, mostly in shipping.” This turns a table of figures into a sentence a leader can actually read and act on. It is like having a junior analyst write the commentary.

**Ask in plain language.** Some tools let a leader ask, “Which product line lost money last quarter?” and get an answer without writing a formula. This is useful for the ad-hoc questions that used to mean “I’ll look into it later” and then never happened. The leader gets the answer while the question is still fresh.

**Do not skip the human read.** An automatic report is a starting point, not a finished decision document. Read the summary, check the numbers make sense, and add your own judgment before you act or share. AI can summarize confidently and still be wrong if the underlying data is messy. Garbage in, confident garbage out.

**Keep the format stable.** Once you settle on a report layout, keep it consistent. A stable format is easier to read month over month and easier to spot when something looks off. Change it deliberately, not every time. (The same report-building ideas, applied to finance, are in Chapter 25 — Administration and Finance.)

## 30.4 Decision support

Decision support means using AI to lay out the options, the evidence, and the trade-offs for a decision, so a leader can choose well. It does not make the decision. It makes the decision *informed*.

**Lay out the options.** For a choice like “Should we open a new region?” or “Which supplier should we pick?”, AI can gather the relevant data and present the options side by side, with the pros, the

cons, and the numbers behind each. The leader sees the whole picture, not one person's favorite.

**Model the trade-offs.** AI can show what happens under different assumptions: "If demand rises 10%, this option wins; if it stays flat, that one is safer." This turns a guess into a comparison of scenarios, which is far more useful for a big decision.

**Surface what you missed.** AI can point out a risk or an opportunity a busy leader did not see, because it looked at everything at once. The IBM vendor-risk case above is exactly this: AI reading every vendor's answers so the assessor sees the risk clearly before deciding.

**Keep the decision with the leader.** AI advises; the leader decides and owns the result. A leader who blindly follows the model has stopped leading. Use AI to inform your judgment, not to replace it. The strategy behind these choices is in Chapter 13 — Defining a Simple AI Strategy.

**Watch for confident nonsense.** AI can present a wrong option with total confidence. Check the evidence behind its advice, especially for a big or unusual decision. Ask the model to show its reasoning, and test it against what you know. Trust, but verify.

## Ethics and Responsibility

IT and leadership carry the trust of the whole company, so responsibility here is broad.

**Keep the human in the big decision.** AI advises on security, reports, and strategy. A person makes the call on shutting a system, trusting a vendor, or taking a risk. The bigger the decision, the more the human owns it.

**Protect the data you feed the models.** IT and leadership data — network logs, financials, vendor records — is among the most sensitive a company holds. Keep it secure and, where it matters, within your own environment. Do not feed sensitive data into public AI tools without checking the security implications (see Chapter 6 and Chapter 8 — Self-Hosting: Keep Your Data Under Control).

**Watch third-party and shadow AI.** Vendors and staff may bring AI tools into the company without approval — the "shadow AI" risk. Know what AI is running on your network and who approved it. That risk is covered in Chapter 9 — Third-Party Services and Shadow AI.

**Be honest in reports and advice.** An AI summary can make a bad quarter sound fine, and AI advice can be confidently wrong. Read the numbers, check the advice, and report the truth, especially when it is uncomfortable. A leader's job is to see clearly, not to be comforted.

**Mind the legal floor.** Some AI uses in IT and HR touch the EU AI Act's high-risk and transparency rules. Know your role and your duties before you deploy (see Chapter 5 — Rules and Legal Responsibility). Compliance is the floor; your own judgment sets the standard above it.

**Use AI to strengthen trust, not to control people.** Monitoring and analytics should make the company safer and better run, not become a tool to watch and pressure staff. Use them to protect and improve, not to police.

## Mistakes to Avoid

**A bot-only help desk.** Staff with a real problem stuck behind a bot with no human path. Always allow a human handoff.

**Trusting a security alert blindly.** Acting on a false positive, or worse, ignoring a real threat because the alert was buried. Tune the system and keep a human investigator.

**Letting automation make the big security call.** Shutting down, paying, or escalating without a human. Keep the human decision in an incident.

**Skipping the human read on reports.** Acting on an AI summary without checking the numbers. Read it yourself first.

**Confident nonsense in advice.** Following AI advice that is wrong but well-phrased. Check the evidence behind it.

**Blind faith in the model for big decisions.** A leader who stops leading and just follows the dashboard. The leader owns the call.

**Feeding sensitive data to public AI.** Network logs, financials, and vendor records into insecure tools. Check security and privacy first.

**Shadow AI running unchecked.** Staff and vendors using unapproved AI tools. Know what is running and who approved it.

**Automating a broken process.** If your IT support or reporting is a mess, AI makes a faster mess. Fix the process first.

**No baseline.** Not measuring ticket time, incident rate, or report time before, so you cannot prove the gain. Measure first (see Chapter 22 — Measuring Results and ROI).

**Quoting the vendor's number as yours.** Using a vendor's best-case figure instead of your own measured result. Use your own numbers.

## Practical Exercise

### 30.7 Exercise: plan one IT or leadership automation

Pick one job and plan its AI assistance end to end, with the human decision built in.

**Step 1 — Choose the job.** Pick one: internal support, cybersecurity, management reports, or decision support. Do one, not all.

**Step 2 — Define the goal and the metric.** Faster ticket resolution? Fewer missed threats? Faster reports? Better decisions? Pick one number to measure.

**Step 3 — Measure the baseline.** What is that number now? Average ticket time, incidents missed, days to build a report, decision turnaround. Write it down.

**Step 4 — Find the costly target.** Identify the single most painful point — the ticket type that eats the most time, the threat that slips through, the report that is always late, the decision you keep getting wrong. Target that first.

**Step 5 — Mark each step.** For each step, mark it: **AI does it** (answer, flag, summarize, lay out options), **human reviews it** (check the alert, read the report), or **human decides it** (shut the system, pick the vendor, take the risk). Every big decision must be human.

**Step 6 — Check the data and the law.** Decide what data the AI needs, keep it secure, and check whether the use touches the AI Act's high-risk or transparency rules (see Chapter 5).

**Step 7 — Connect the systems.** Decide which systems the AI needs to see — tickets, logs, financials, vendor records — and how you will connect them (see Chapter 19 — Connecting AI to Systems You Already Use).

**Step 8 — Launch small and measure.** Run it on one team, one system, or one report first. Compare the metric to the baseline. Scale only what proves it works.

Do one job well. The costly-target analysis in step 4 is valuable on its own — it shows you where your IT and leadership work actually loses the most time or carries the most risk, which is useful even before you buy any tool.

## Checklist

### 30.8 IT and leadership checklist

Before automating any IT or leadership task, check these.

- ☐ **You measured the baseline** — ticket time, incident rate, report time, decision turnaround.
- ☐ **You targeted the most costly point first**, not the easiest one.
- ☐ **A human makes every big decision** — system shutdown, vendor trust, risk acceptance.
- ☐ **The help-desk bot hands off to a human** with full context.
- ☐ **Security alerts are tuned** so the real threat rises and the noise falls.
- ☐ **A human investigates every real security incident.**
- ☐ **Management reports are read by a human** before you act or share them.

- ☐ **You check the evidence behind AI advice**, especially for big decisions.
- ☐ **Sensitive data is kept secure**, not sent to public AI services.
- ☐ **You know what AI is running on your network** and who approved it (no shadow AI).
- ☐ **You checked the AI Act** for any high-risk or transparency duties.
- ☐ **You fix the broken process before automating it.**
- ☐ **You report your own measured numbers**, not the vendor's best case.

If a box is empty, the risk — to your systems, your data, or your decisions — is still yours. Fill it before you let AI near the controls.

## Key Takeaways

- AI in IT and leadership is a copilot: it answers support questions, spots threats, builds reports, and lays out options, while a human keeps every big decision.
- The IBM vendor-risk case (IBM Client Engineering, "Evaluating Third Party Risk with AI") helped a financial institution with 1,000+ assessments a year at ~45 hours each, with a published estimate of ~20% time reduction, ~10,000 hours and ~$800k saved per year — the "about 50%" figure is not what the published source reports.
- The saving comes from letting AI do the slow reading and checking while the human keeps the judgment; the same pattern works across support, security, reports, and decisions.
- Keep sensitive data secure, know what AI is running on your network, and check the AI Act's high-risk and transparency rules before you deploy.
- Measure your own baseline, keep a human in every big call, and treat the law as the floor and your own judgment as the standard

above it.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Clean meeting notes

**Internal Operations**
Team Meeting Records

# Meeting Minutes
Team Meeting

| Meeting Title | Date | Time | Venue | Meeting No. |
|---|---|---|---|---|
| **Weekly Team Sync** | **18 September 2026** | **09:30** | **Office** | **Weekly Sync** |

## Attendees

| Name | Role |
|---|---|
| Anna | Team Lead |
| Luca | Team Member |
| Sara | Team Member |

**Excused:** None **Not present:** None

## Agenda and Discussion

**1. Review of the Week**

**Discussion.** The team reviewed the week's progress and ongoing work streams.

**Decision.** No formal decision recorded; review noted.

**2. Online Shop Launch**

**Discussion.** The team discussed the online shop launch and related readiness, including a checkout bug that needs to be fixed.

**Decision.** Fix the checkout bug by 22 September.

**3. Upcoming Marketing**

**Discussion.** The team discussed upcoming marketing activities, including the loyalty card and new packaging.

**Decision.** Launch the loyalty card by 30 September; order new packaging by 25 September.

## Action Items

| Action | Owner | Due Date | Status |
|---|---|---|---|
| Loyalty card launch | Anna | 30 September 2026 | Open |
| Checkout bug fix | Luca | 22 September 2026 | Open |
| Packaging order | Sara | 25 September 2026 | Open |

## Next Meeting

The next meeting is scheduled for **25 September 2026** at **09:30**, venue: **Office**. Agenda items should be sent to **Anna** by **24 September 2026**.

**Prepared By**
Anna
Team Lead

Signature & date

**Reviewed By**
Luca
Team Member

Signature & date

*Tidy meeting minutes with decisions and actions*

**What you ask:** `Turn these rough notes into clean meeting minutes with a decisions list and action items: [paste notes]`

The agent turns your messy notes into structured minutes: what was discussed, what was decided, and an action list with owners. You can send them straight to the team.

*Tip: Paste your notes as they are — the agent sorts the order and the wording.*

---

## A quarterly board review

*A quarterly review presentation slide*

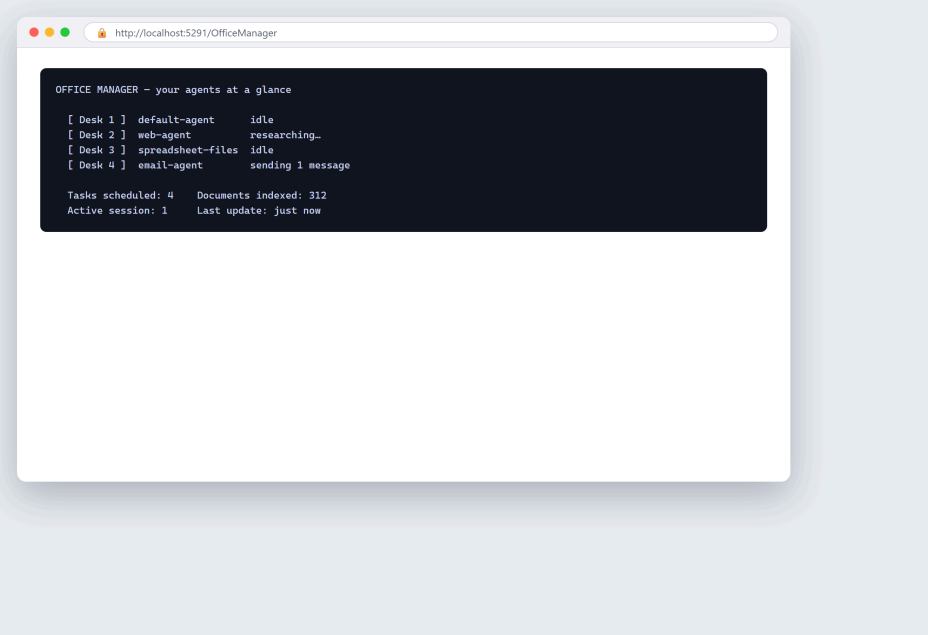
**What you ask:** `Build a quarterly review deck: results, highlights, challenges, and next quarter plan.`

The agent assembles the review deck with the results, the wins, the issues and the plan — laid out so the meeting moves quickly.

*Tip: Attach your KPI sheet and the agent puts the charts in the slides.*

---

## See all your agents at a glance




*The OfficeManager view of your agents*

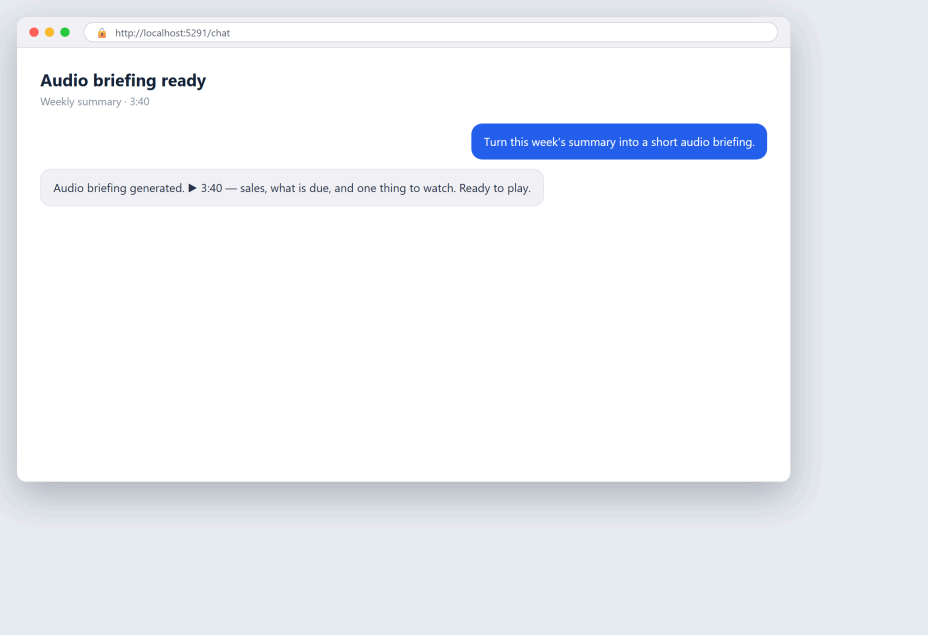
**What you ask:** `(browser) Open the OfficeManager view.`

OfficeManager shows your agents in one simple page, so you can see what is running and where things stand.

*Tip: A quick way to keep an eye on more than one assistant at once.*

---

## A briefing you can listen to




*An audio briefing ready to play*

**What you ask:** `Turn this week's summary into a short audio briefing I can listen to on the way to work.`

The agent converts the written summary into a clear audio briefing, so your commute becomes useful time.

*Tip: Schedule the weekly summary and its audio version together.*

# Chapter 31 — A Small Manufacturing Business

*This chapter is a representative composite. It is not one real company. It combines the common patterns we see in small machine shops and fabricators that adopt AI. All numbers are illustrative — they show the shape of the decision, not a promise. Replace them with your own.*

## Context

Picture a small precision machining shop. We will call it **Northgate Machining**. It employs about twenty-five people. It makes custom metal parts to order — one-off pieces and short runs for other factories. Nothing sits on a shelf waiting for a buyer. Every part starts as a customer request.

The work begins with an **RFQ**, which stands for "request for quote." A customer emails a drawing — usually a PDF with the shape of the part, its measurements, and the material it must be made from. The shop must look at that drawing, figure out how long each cutting and finishing step will take, add up the cost of the metal and the machine time, and return a price. That price is the quote. If it is too high, the customer goes elsewhere. If it is too low, the shop wins the job but loses money making it.

Two experienced estimators do most of this quoting. They have been doing it for years. They can look at a drawing and roughly know what it costs. But "roughly" is doing a lot of work in that sentence. The shop has no written record of what past quotes actually cost to produce. The

knowledge lives in two heads. When those two people are on holiday, quoting slows to a crawl.

Behind the quoting sits the rest of the shop. Raw metal stock — bars, plates, rods — is tracked in a spreadsheet that is updated when someone remembers. Quality control is a final visual check by hand at the end of the job. Weekly reports for the owner are typed by hand from three different spreadsheets. Nothing is connected. Everything depends on people remembering to write things down.

This is a normal, healthy small manufacturer. It is profitable. It is busy. And it is leaving money on the table in four places: quoting, inventory, quality, and reporting.

## The Problem

The owner, Elena, can feel the problems but cannot always see them. Let us name them plainly.

**Quoting is slow and inconsistent.** A simple part takes an hour to quote. A complex one takes half a day. The average is two days from the email arriving to the quote going out. In that window, the customer has already asked two other shops. Speed matters. Worse, the two estimators price the same drawing differently. One is cautious and prices high. The other is aggressive and prices low. Over a year, some jobs are quietly underpriced. The shop wins them, builds them, and discovers later that the machine time cost more than the price covered. These losses are invisible because nobody compares the quote to the actual cost afterward.

**Inventory is guesswork.** The shop buys metal stock when someone notices the shelf is low. Too often, that is too late — a job is delayed because the right bar is not in stock. Just as often, the shop buys too

much, and expensive material sits for months, tying up cash. The spreadsheet is never quite right. Nobody trusts it, so people double-check by walking to the shelf, which wastes time.

**Quality defects are caught too late.** A bad part is often found only at the final check, after the whole job is finished. If the error was made on the first cut, the shop may have made fifty bad parts before noticing. That is scrap — material and machine time thrown away. Catching a defect after fifty parts instead of after one is fifty times the loss.

**Reporting eats hours.** Every Friday, someone spends three or four hours pulling numbers from the spreadsheets to build a summary for Elena. That time is pure overhead — it produces no parts and wins no customers.

Each of these is a small leak. Together, they drain real money and real time. If you want to see how these four activities rank against the rest of your business, the impact-and-effort method in Chapter 12 — Where AI Can Help Your Business is the place to score them.

## The Solution

Elena does not try to "use AI everywhere." She picks the four leaks and attacks them one at a time, starting with the biggest and easiest.

**Smarter quoting.** Instead of reading each drawing from scratch, the shop feeds its past jobs into a system. For every past job, it now records two things it never linked before: the original quote and the actual cost to produce. Over time this becomes a reference library. When a new drawing arrives, an AI assistant reads the PDF, pulls out the key features — the dimensions, the tolerances (how exact each measurement must be), the material, the number of pieces — and suggests a price based on similar past jobs. The estimator no longer

starts from a blank page. They start from a draft and adjust it. The AI does the first pass; the human makes the call.

**Inventory that predicts.** The shop connects its stock spreadsheet to a simple forecasting tool. The tool looks at how fast each material is used and suggests when to reorder and how much. It does not place the order itself. It raises a suggestion: "You will run out of this aluminium bar in nine days; order now." A person confirms it. The guesswork becomes a prompt.

**Quality checks at the machine.** Instead of checking only at the end, the shop puts a small camera at one machine. A computer-vision system — AI that reads what a camera sees — looks at each part as it comes off and flags anything that looks wrong: a crack, a wrong dimension, a missing hole. The operator sees the flag immediately and stops before making fifty bad copies. The final human check stays; the camera just moves the warning earlier. (The deeper treatment of vision-based quality control and predictive maintenance is in Chapter 29 — Operations and Production.)

**Reports that write themselves.** The spreadsheets are connected to a reporting assistant. On Friday, instead of typing, Elena receives a drafted summary: jobs quoted, jobs won, scrap rate, stock levels, cash tied up. She reads it and edits. Three hours become fifteen minutes.

Notice the pattern across all four. The AI never acts alone. It reads, suggests, flags, and drafts. A human decides, confirms, and approves. This is the same "human reviews the machine's draft" pattern that the Elanco case shows in Chapter 25 — Administration and Finance. In a shop where a wrong number can lose real money, that rule is not optional.

## The Tools

None of this required a team of engineers. The tools are off-the-shelf, aimed at small businesses.

- **A document-reading assistant** that opens the drawing PDF and extracts dimensions and features into a structured form. This is the same class of tool that reads invoices in Chapter 25.
- **A quoting assistant** built on top of that, which compares the extracted features to past jobs and proposes a price. This can be a low-code tool bolted onto the shop's existing quoting spreadsheet, connected as described in Chapter 19 — Connecting AI to Systems You Already Use.
- **A forecasting add-on** for the inventory spreadsheet or the shop's basic ERP (enterprise resource planning) system. Many inventory tools now include a "suggest a reorder" feature.
- **A camera plus a computer-vision inspection tool** at one machine. These are sold as small, self-contained units for quality checks.
- **A reporting assistant** that reads the connected spreadsheets and drafts the weekly summary in plain language.

How to choose among these without being fooled by glossy demos is covered in Chapter 17 — Choosing Tools Without Being Fooled. One caution specific to a machine shop: **drawings are confidential.** A customer's part drawing is their intellectual property. Before you feed drawings into any cloud tool, check where the data goes and who can see it. For some shops, keeping the AI on their own computers — self-hosting, explained in Chapter 8 — Self-Hosting: Keep Your Data Under Control — is the safer choice. The risks of sending sensitive files to third-party services are in Chapter 9 — Third-Party Services and Shadow AI.

## The Costs

Here is an illustrative first-year budget for a shop like Northgate. These are made-up numbers to show the shape. Use your own.

**Direct costs.** - Quoting and document assistant: about €9,000 a year in subscriptions. - Forecasting add-on: about €3,000 a year. - Camera and vision inspection unit: about €6,000 one-time, plus €1,200 a year. - Reporting assistant: about €2,400 a year. - Setup and integration (external help to connect the tools to the spreadsheets and the machine): about €10,000 one-time. - Training the estimators and operators: about €3,000 one-time.

First-year total: roughly **€34,600**. In steady years after, the one-time costs drop away and the recurring subscriptions come to about **€15,600**.

**Indirect costs.** These are the ones people forget. - The estimators spend hours learning the tool and checking its drafts. That is real time, valued at their loaded hourly cost. - The learning dip: for the first few weeks, quoting is slower, not faster, while people trust the new system. - The vision camera needs occasional recalibration when lighting or the part changes. - Someone must review the AI's suggested quotes and reorder points every day. Never skip this.

The full method for counting these costs honestly, and for turning the savings into a return figure, lives in Chapter 16 — Goals, Costs, and Return on Investment. Do not run the math in your head. Write it down.

## The Results

After a year, measured against the baseline Elena recorded before starting, the illustrative outcome looks like this. Remember: your numbers will differ. These show what a good fit can look like, not what yours will be.

- **Quoting time** fell from an average of two days to a few hours for most parts. The estimator reviews a draft instead of building from zero.
- **Fewer underpriced jobs.** Because the quote is anchored to what similar jobs actually cost, the gap between quoted price and real cost narrowed. The shop stopped quietly losing money on jobs it won.
- **Win rate improved.** Faster quotes meant Northgate answered more RFQs inside the window where the customer is still choosing.
- **Scrap dropped.** Catching a defect at the machine instead of at the end cut wasted material and machine time. Instead of fifty bad parts, the operator caught it at the first or second.
- **Stock turns improved.** Fewer stockouts meant fewer delayed jobs. Less over-buying meant less cash frozen on the shelf.
- **Reports** went from three or four hours of typing to about fifteen minutes of reading and editing.

The honest caveat: none of this happened on day one. The quoting assistant was rough for the first month because the library of past jobs was thin. The vision camera gave false alarms until it was calibrated. The savings ramped up over weeks, exactly as the learning-curve warning in Chapter 16 predicts. Elena measured the real numbers after the ramp, not during it.

## Lessons Learned

**Start with quoting.** Of the four leaks, quoting was the highest impact and the lowest risk to try. A wrong quote is caught by the estimator before it goes out. That made it the perfect first project — the same "high impact, high ease first" rule from Chapter 12.

**Your past jobs are the fuel.** The quoting assistant was only as good as the record of past quotes and their real costs. The single most valuable thing Elena did was start linking every quote to its actual production cost. Without that data, the AI had nothing to learn from. Data readiness is covered in Chapter 14 — Data: The Raw Material.

**The AI drafts; the human decides.** Not one quote went out without a person approving it. Not one reorder was placed without a person confirming it. In a shop where a wrong number is real money, the human check is the safety, not a delay.

**Drawings are confidential.** Treat every customer drawing as sensitive intellectual property. Decide where it may go before you feed it anywhere. For some shops that means self-hosting; for others it means a vetted vendor with a clear contract.

**Calibrate the camera; do not trust it blindly.** The vision system was not plug-and-play. It needed tuning to tell a real defect from a shadow. Budget for that, and keep the final human check in place.

**Expect the ramp.** The first month was slower and messier than the twelfth. Judge the project after the learning curve, not during it.

**Connect, do not replace.** Northgate did not throw away its spreadsheets or its ERP. It bolted AI onto what already worked, as

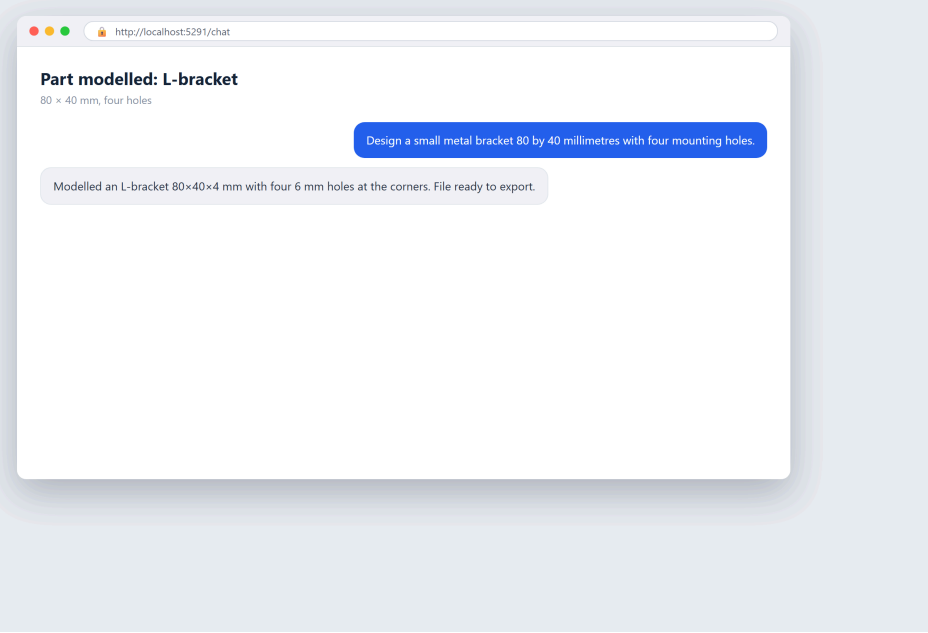
described in Chapter 19. The shop kept its systems and added a smarter layer on top.

The small manufacturer's lesson is the same as every other sector's: find the leak, pick the easiest high-value one, let AI draft and flag, keep a human on the decision, and measure honestly. A shop with twenty-five people and no engineers can do this. The tools are ready. The only thing missing is a clear look at where the money is leaking.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Design a simple part




*A simple part modelled through the CAD tool*

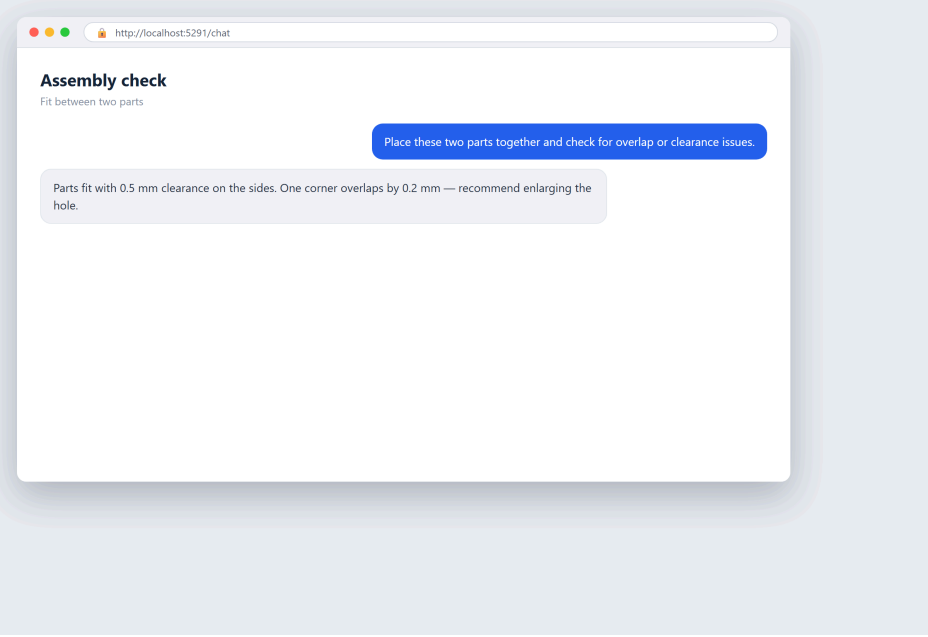
**What you ask:** `Design a small metal bracket 80 by 40 millimetres with four mounting holes.`

The agent drives the CAD tool to model the part with the right dimensions, and you get a file you can review or export.

*Tip: Describe the shape and the measurements; the agent handles the CAD steps.*

---

## Check how parts fit




*An assembly check between two parts*

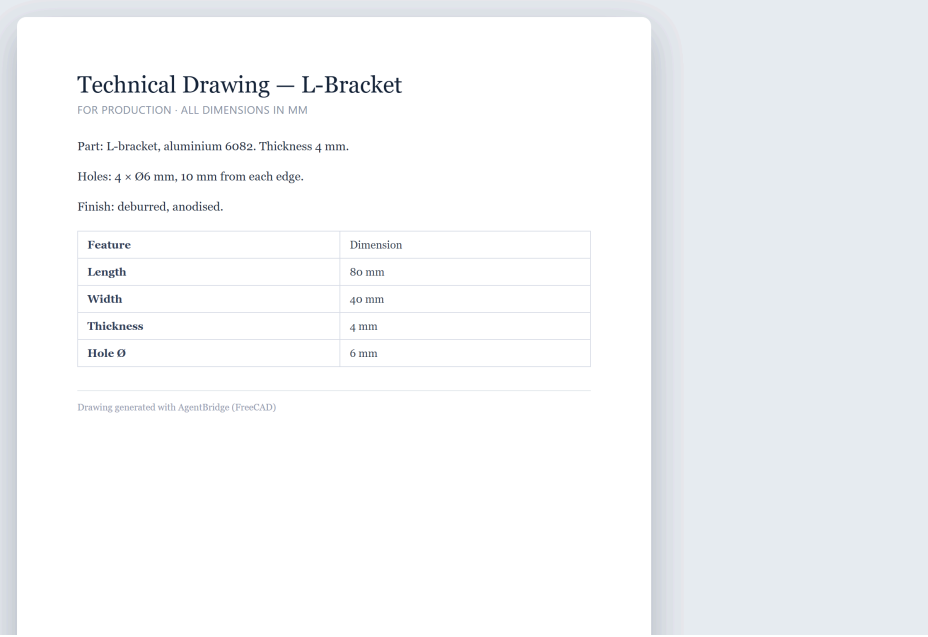
**What you ask:** `Place these two parts together and check for any overlap or clearance issues.`

The agent assembles the parts in the CAD model and points out where they clash or where the fit is too tight.

*Tip: Catch fit problems on screen, not on the workshop bench.*

---

## A drawing for the workshop




*A dimensioned technical drawing for production*

**What you ask:** `Produce a technical drawing of the bracket with the key dimensions marked.`

The agent generates a drawing with the dimensions marked, ready for the person who will make the part.

*Tip: Ask for the view you need — top, side — so the drawing is clear.*

# Chapter 32 — A Professional Practice

*This chapter is a representative composite. It is not one real firm. It combines the common patterns we see in small law and accounting practices that adopt AI. All numbers are illustrative — they show the shape of the decision, not a promise. Replace them with your own.*

## Context

Picture a small commercial law firm. We will call it **Marlowe Legal**. It has twelve lawyers and a handful of support staff. It is not the kind of firm that argues in front of judges. It does paperwork: contracts, leases, company agreements, compliance advice. Clients are other businesses.

The work of such a firm is, at its core, reading and writing. A client sends a long contract and asks, "Is this safe to sign?" A new client calls and needs their matter opened. A standard lease needs to be drafted from the firm's template. At the end of the month, every lawyer writes down the minutes they spent and sends an invoice.

For years, all of this was done by hand. A junior lawyer reads a two-hundred-page contract and writes a summary. A receptionist takes the new-client call and fills a form. A senior lawyer types the same clauses into every lease. At night, everyone tries to remember what they worked on so they can bill it. The firm is profitable and respected. But it spends a great deal of expensive, trained time on work that is repetitive.

A professional practice is different from a shop or a factory in one crucial way. Everything it touches is **confidential**. A client's contract, a merger plan, a dispute file — these are protected by legal privilege, the rule that a client can speak freely to their lawyer without the contents

being revealed. This single fact changes how AI can be used, and it is the thread running through this whole chapter.

## The Problem

The firm's problems are the same four leaks as everywhere else, but with a professional twist.

**Document review is slow and costly.** When a client asks the firm to review a contract, a junior lawyer reads every page and writes notes on risky clauses. For a big deal, this takes days. The client pays for those days. The work is careful but repetitive — the same kinds of risky clauses appear again and again, and the firm already knows what to look for. It just takes a long time to look.

**Client intake is inconsistent.** When a new client calls, the information captured depends on who answers. Some get a full picture; others miss a detail that matters later. Important facts fall through the cracks, and the lawyer has to chase them afterward.

**Drafting repeats itself.** The firm has templates, but every new lease or agreement still needs a lawyer to assemble the standard clauses and adjust them. It is reliable but slow, and it is exactly the kind of work that does not need a senior's judgment.

**Billing leaks.** Lawyers bill by the hour, but they are bad at recording every minute. A ten-minute call here, a quick email there — these often go unwritten. At the end of the month, the firm "writes off" hours it actually worked but never billed. Across twelve lawyers, this leak is large. One study of law firms has put typical unbilled time in the range of several percent of all worked hours; treat that as a directional figure, not a precise one. The point stands: the leak is real.

The constraint that sits over all four is confidentiality. Any tool that reads a client's contract must be a tool the firm can trust not to leak that contract. This is not a minor worry. It is the first question, before any other.

## The Solution

Marlowe Legal attacks the four leaks in order of safety, not just size. The rule is simple: start where a mistake is cheap and the data is not sensitive, and move toward the sensitive work only when the tooling is trustworthy.

**Document review that reads first, then the lawyer decides.** The firm uses an AI assistant that reads a contract and produces a first-pass summary: what the agreement does, which clauses are unusual, and which ones deviate from the firm's own standard positions. The junior lawyer no longer reads cold. They read the AI's notes and then verify against the real document. The AI shortlists the risky clauses; the lawyer makes the legal judgment. This is the same "machine drafts, human reviews" pattern that the Elanco case shows in Chapter 25 — Administration and Finance.

**Intake that asks the right questions every time.** A structured intake assistant — a guided form on the website, or a chatbot that asks a fixed set of questions — captures the same facts for every new client. Nothing is missed because a person forgot to ask. The structured information flows straight into the case file. The chatbot side of this is the same technology covered in Chapter 26 — Sales and Marketing and Chapter 27 — Customer Care and Support.

**Drafting assistance for the standard parts.** For routine documents, an AI drafting assistant assembles the standard clauses and

proposes the wording. The lawyer reviews and adjusts. The senior lawyer stops typing boilerplate and starts reviewing drafts instead.

**Billing that captures time as it happens.** Instead of reconstructing the day at night, the firm uses a tool that turns activity — emails sent, documents opened, calendar entries — into a draft time entry with a suggested narrative. The lawyer reviews and confirms. The leak narrows because the starting point is a near-complete draft, not a blank page.

In every case, the human stays in charge. In a profession where a wrong answer harms a real client, the AI is a first-pass helper, never the decision-maker.

## The Tools

The tools are ordinary, but the way they are deployed is shaped by confidentiality.

- **A document-review assistant** that reads contracts and flags clauses against a checklist the firm writes itself. This is the same class of document-reading tool that handles invoices in Chapter 25, pointed at legal text.
- **An intake chatbot or guided form** on the firm's website that asks a fixed set of questions and writes the answers into the case-management system.
- **A drafting assistant** for standard clauses and agreements, connected to the firm's templates.
- **A time-capture and billing assistant** that drafts time entries from the day's activity.

How to pick these tools without being dazzled by a demo is covered in Chapter 17 — Choosing Tools Without Being Fooled. How to connect

them to the firm's existing case-management software is in Chapter 19 — Connecting AI to Systems You Already Use.

**The confidentiality question comes first.** A general chatbot that you paste a client contract into may store that contract, train on it, or expose it. For a law firm, that can break privilege and client duty. The firm must use tools that keep client data private — either a business-grade service with a clear no-training, no-sharing contract, or a model run on the firm's own machines. Self-hosting is explained in Chapter 8 — Self-Hosting: Keep Your Data Under Control. The danger of staff quietly pasting client data into public tools — shadow AI — is the subject of Chapter 9 — Third-Party Services and Shadow AI. And because client files contain personal data, the privacy rules of Chapter 10 — Privacy and GDPR apply in full.

## The Costs

Here is an illustrative first-year budget for a firm like Marlowe. These are made-up numbers to show the shape. Use your own.

**Direct costs.** - Document-review assistant (business-grade, with a confidentiality contract): about €12,000 a year. - Intake chatbot: about €3,600 a year. - Drafting assistant: about €4,800 a year. - Time-capture and billing assistant: about €4,800 a year. - Setup and integration with the case-management system: about €9,000 one-time. - Training the lawyers and staff: about €4,000 one-time.

First-year total: roughly **€38,200**. In steady years after, the recurring subscriptions come to about **€25,200**.

**Indirect costs.** - Lawyers spend time reviewing every AI draft. This is the cost of the safety net, and it must stay. - The learning dip while everyone adapts. - The cost of a mistake if a draft is trusted without

checking — in a profession, this can be far larger than the subscription. Budget for careful review, not for hope. - Time spent vetting each tool for confidentiality and compliance before use.

The full method for counting these costs and turning the savings into a return figure is in Chapter 16 — Goals, Costs, and Return on Investment.

## The Results

After a year, measured against a baseline the firm recorded before starting, the illustrative outcome looks like this. Your numbers will differ. These show what a good fit can look like.

- **Document review sped up.** The junior lawyer's first pass on a long contract dropped from days to hours, because they started from the AI's summary and verified rather than reading cold.
- **Intake became complete.** Every new client now provides the same set of facts. Fewer gaps to chase later.
- **Drafting got faster.** Routine documents took a fraction of the time, because the lawyer reviewed a draft instead of assembling boilerplate.
- **Billing leakage narrowed.** Because time was captured as it happened, fewer worked hours went unwritten. The firm billed more of what it actually did.

The honest caveat: none of this was instant. The document-review assistant gave imperfect summaries at first and needed the firm's checklist tuned. The billing tool produced drafts that lawyers had to correct before trusting. The gains ramped up over weeks, as the learning-curve warning in Chapter 16 predicts. The firm measured the real numbers after the ramp, not during it.

## Lessons Learned

**Confidentiality is the first constraint, not an afterthought.** In a professional practice, the question is never only "does this tool work?" It is "can this tool be trusted with a client's private file?" Answer that before anything else. Use business-grade tools with a clear no-training contract, or self-host. See Chapter 8 and Chapter 9.

**The AI drafts; the professional advises.** A lawyer's value is judgment and accountability. The AI shortlists and drafts; the lawyer decides and signs. Never let a draft become advice without a human mind on it.

**Beware the confident wrong answer.** These tools can produce text that sounds right and is wrong — a clause that does not exist, a citation that is made up. In a profession, a fabricated reference is a disaster. Verify every citation and every legal claim against the real source. The reliability problem is covered in Chapter 2, and the duty to be honest about what AI can and cannot do is in Chapter 4 — Ethical AI: Doing the Right Thing.

**Start with the safe work.** Marlowe began with internal drafting and intake — low risk, not yet the most sensitive client files. It moved toward contract review only once the tooling was trusted. This is the "high ease first" rule from Chapter 12.

**Billing AI must be reviewed, both ways.** A time-capture tool can under-record, but it can also over-record or mislabel. Over-billing a client on an AI's guess is an ethical and legal problem. The lawyer reviews every entry. The rules around professional conduct and the EU AI Act's duties are in Chapter 5 — Rules and Legal Responsibility.

**Personal data is still personal data.** Client files hold names, addresses, financial details. The privacy duties in Chapter 10 apply to every one of them, no matter how the tool is marketed.

**Measure honestly and expect the ramp.** Record the baseline before you start. Judge the project after the learning curve, not during it.

The professional practice's lesson is the same as every sector's, with one extra guardrail: find the repetitive work, let AI draft and flag, keep a human on the judgment, and measure honestly — and in a profession, never let the tool touch a client's confidential file until you are certain it is safe to.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Draft a service contract

**Acme Studio**
Design & Development Studio

# Sales Contract

| Contract No. | Date | Place of Signing |
|---|---|---|
| **ACME-2026-014** | **21 September 2026** | **Milano** |

**The Seller**
Acme Studio
Via Manzoni 8, 20121 Milano
VAT / Tax ID: IT01234567890
Registration: IT01234567890
Via Manzoni 8, 20121 Milano

**The Buyer**
Bright Cafe
Via Roma 12, 20121 Milano
VAT / Tax ID: IT0223344556
Registration: IT0223344556
Via Roma 12, 20121 Milano

This Sales Contract (the "Contract") is entered into between the Seller and the Buyer (together, the "Parties") as of 21 September 2026. The Parties agree as follows:

## 1. Subject of the Contract

**1.1** The Seller agrees to provide, and the Buyer agrees to accept, the services described in Section 2 (the "Services") under the terms and conditions of this Contract. The Services consist of the design and build of a business website for the Buyer.

**1.2** The Services are described in detail in **Annex A — Statement of Work**, which is attached to and forms an integral part of this Contract.

**1.3** The scope of supply is limited to the items expressly listed in Section 2; any additional services require a separate written agreement.

## 2. Price and Payment

| # | Item | Qty | Unit Price | Amount (EUR) |
|---|---|---|---|---|
| 1 | Website design — business website layout, visual identity and page mock-ups | 1 | 2,000.00 EUR | 2,000.00 EUR |
| 2 | Website development — build of the business website | 1 | 2,500.00 EUR | 2,500.00 EUR |
| 3 | Content upload and one round of revisions | 1 | 1,500.00 EUR | 1,500.00 EUR |

| | |
|---|---|
| Subtotal | 6,000.00 EUR |
| Discount | 0.00 EUR |
| Tax (0%) | 0.00 EUR |
| **Total Contract Price** | **6,000.00 EUR EUR** |

**2.1** The total contract price is **6,000.00 EUR EUR**, inclusive of all costs stated in this Contract and exclusive of taxes other than those expressly listed above.

**2.2** Payment shall be made 50% deposit (3,000.00 EUR) before work begins and 50% (3,000.00 EUR) on delivery by bank transfer to the Seller's account: **IT60X0542811101000000123456** (SWIFT/BIC: ACMEITMM).

**2.3** Unless otherwise agreed, the Buyer shall not withhold or set off any amounts without the Seller's prior written consent.

## 3. Delivery Terms

**3.1** Delivery shall take place remotely at **Via Roma 12, 20121 Milano**, no later than **1 January 2027**, the project running for three months starting 1 October 2026.

**3.2** The Incoterms applicable to this Contract are **not applicable** (Incoterms 2020), as expressly agreed by the Parties.

**3.3** Risk of loss or damage to the Goods passes to the Buyer at the moment and place determined by the applicable Incoterms. Delivery of the website is deemed to occur upon publication of the site on the hosting environment agreed by the Parties.

## 4. Warranties

**4.1** The Seller warrants that the Goods are free from defects in material and workmanship and conform to the specifications in Section 2 for a period of **3 months** from the date of delivery.

**4.2** During the warranty period, the Seller shall, at its option, repair or replace any defective Goods free of charge, provided the defect is notified in writing within **14 days** of discovery. The scope of the project includes one round of revisions; further changes are outside the scope of this Contract.

**4.3** The warranty does not cover defects caused by misuse, normal wear and tear, unauthorized modification or repair, or failure to follow the Seller's instructions: misuse, normal wear and tear, unauthorized modification or repair, or failure to follow the Seller's instructions.

## 5. Limitation of Liability

**5.1** Neither Party shall be liable to the other for any indirect, incidental, special or consequential damages, including loss of profits, loss of data or loss of business opportunity, arising out of or in connection with this Contract.

**5.2** Except as otherwise provided herein, the aggregate liability of either Party under this Contract shall not exceed **6,000.00 EUR EUR**.

**5.3** Nothing in this Section limits liability for fraud, willful misconduct, gross negligence, or death or personal injury caused by negligence.

*A drafted service agreement, ready for review*

**What you ask:** `Draft a service contract between my studio and a client for a 3-month website project at 6,000 euros, with a 50% deposit.`

The agent produces a clear contract with the parties, the scope of work, the payment schedule, and the timeline. It is a starting point you can review and adjust — not legal advice, but a solid draft that saves you hours of blank-page work.

*Tip: Attach your old contract and ask it to follow the same style and clauses.*

# Chapter 33 — A Retail Store

*This chapter is a representative composite. It is not one real store. It combines the common patterns we see in independent retailers that adopt AI. All numbers are illustrative — they show the shape of the decision, not a promise. Replace them with your own.*

## Context

Picture a family-owned independent store. We will call it **Cornerstone Home & Garden**. It sells tools, paint, garden supplies, and small household goods. One physical shop, plus a small online store. About fifteen staff. It carries a few thousand different products — each one a **SKU**, which just means a distinct item with its own code, like "18-volt drill, model X" or "5-litre green emulsion paint."

The store runs on a **POS** system — the point-of-sale till that records every sale. That till knows what sold and when. But knowing what sold is not the same as knowing what to order next week, or what a customer is asking at the counter, or how to bring people back through the door.

For years, all of that was done by feel. The owner, Marco, reorders when a shelf looks low. He guesses how much snow shovel stock to buy each winter based on last winter, which was nothing like this one. Customer questions come by phone and email and are answered when someone is free. Marketing is an occasional email blast that Marco types himself, to everyone, with the same message. The store survives. But it ties up cash in the wrong stock, misses sales when the right item is gone, and spends marketing money that barely moves the needle.

A small independent retailer competes against big chains that have whole teams doing forecasting and marketing. Cornerstone cannot hire a team. But it can point a few AI tools at the same jobs.

## The Problem

The leaks in a small store are easy to name.

**Overstock and stockouts.** When Marco guesses wrong on the high side, expensive stock sits on the shelf for months. That is cash frozen — money that could be used elsewhere is locked in boxes nobody is buying. This is called **dead stock**. When he guesses wrong on the low side, the item runs out, and the customer leaves without buying — or worse, buys it at a competitor and stops coming back. Both errors cost money, and both come from guessing.

**Seasonal misses.** Demand swings with the season and even the weather. A mild winter leaves snow shovels unsold. A sudden hot spell empties the watering cans before Marco can restock. Human memory of "last year" is a poor guide to this year.

**Slow customer service.** Questions like "Do you have this in stock?" or "What are your opening hours?" or "Can I return this?" pile up by phone and email. Answering them is simple but takes time away from the floor. Unanswered, they turn into lost sales.

**Generic marketing.** Marco's email blast goes to everyone with the same message. A gardener and a painter get the same email. Most of it is ignored. The marketing spend is small, but the return is smaller because it is not aimed at anyone in particular.

If you want to see how these four rank against the rest of your store, the impact-and-effort method in Chapter 12 — Where AI Can Help Your

Business is the place to score them.

## The Solution

Marco picks the four leaks and starts with the one that frees the most cash with the least risk.

**Demand forecasting for stock.** The store connects its POS sales history to a forecasting tool. The tool looks at what sold, when, and how fast, and adds seasonal patterns. It then suggests what to order and how much, instead of leaving it to Marco's gut. It does not place the order itself. It raises a suggestion: "You usually sell through this in three weeks; order now." A person confirms it. The guess becomes a prompt. (The deeper treatment of demand forecasting and inventory is in Chapter 29 — Operations and Production.)

**Stock alerts.** On top of the forecast, the store sets simple alerts: when an item drops below a safe level, flag it; when an item has not sold in a long time, flag it as possible dead stock. Marco sees both and acts.

**A chatbot for the common questions.** A chatbot on the website and the online store answers the repeated questions — stock, hours, returns, delivery — instantly, in plain language, in more than one language if needed. The staff on the phone are freed for the questions that need a person. This is the same customer-service chatbot pattern seen in Chapter 27 — Customer Care and Support, and the same kind of shopping-guide agent that the mobilezone case describes in Chapter 26 — Sales and Marketing.

**Marketing that aims.** Instead of one blast to everyone, a marketing assistant helps Marco split his customer list into groups — gardeners, painters, regulars — and draft a different message for each. The AI writes the drafts; Marco checks them against the store's voice before

sending. The same spend reaches the right people with the right message.

Notice the pattern. The AI predicts, flags, answers, and drafts. A human confirms the order, handles the hard questions, and approves the marketing. The store keeps control at every step.

## The Tools

None of this required a data scientist. The tools are off-the-shelf and aimed at small retailers.

- **A forecasting add-on** for the POS or inventory system. Many modern till systems now include a "suggest a reorder" feature that reads your own sales history.
- **Simple stock alerts**, often built into the same inventory tool.
- **A customer-service chatbot** on the website, connected to the store's product list so it can answer "do you have this?" accurately.
- **A marketing assistant** that segments the customer list and drafts campaign emails and social posts.

How to choose among these without being fooled by a glossy demo is covered in Chapter 17 — Choosing Tools Without Being Fooled. How to connect them to the POS and the customer list you already have is in Chapter 19 — Connecting AI to Systems You Already Use.

One caution specific to retail: the customer list holds personal data — names, emails, purchase history. The privacy rules in Chapter 10 — Privacy and GDPR apply to how that list is used for marketing, including getting consent to email people.

## The Costs

Here is an illustrative first-year budget for a store like Cornerstone. These are made-up numbers to show the shape. Use your own.

**Direct costs.** - Forecasting and inventory add-on: about €4,800 a year. - Customer-service chatbot: about €3,600 a year. - Marketing assistant: about €3,600 a year. - Setup and integration with the POS and the customer list: about €7,000 one-time. - Training the staff: about €2,000 one-time.

First-year total: roughly **€21,000**. In steady years after, the recurring subscriptions come to about **€12,000**.

**Indirect costs.** - Staff time to review the reorder suggestions and the chatbot's answers. - The learning dip while everyone trusts the new system. - Someone must check the marketing drafts before they go out, so the store's voice stays right. - Cleaning up the sales history so the forecast has good data to learn from.

The full method for counting these costs and turning the savings into a return figure is in Chapter 16 — Goals, Costs, and Return on Investment. Do not run the math in your head. Write it down.

## The Results

After a year, measured against a baseline Marco recorded before starting, the illustrative outcome looks like this. Your numbers will differ. These show what a good fit can look like.

- **Stockouts fell.** Fewer customers left empty-handed because the forecast flagged the reorder early.

- **Dead stock fell.** Less cash sat frozen in boxes nobody wanted, because the tool saw the slow movers and the over-order before it happened.
- **Cash freed.** With less money locked in the wrong stock, the store had cash to use elsewhere.
- **Customer questions answered instantly.** The chatbot handled the common ones day and night, and the phone queue shortened.
- **Marketing worked harder.** Segmented, aimed messages got more response than the old one-size-fits-all blast, on the same small budget.

The honest caveat: none of this was instant. The forecast was rough for the first few months because it needed a clean year of sales history to learn from. The chatbot gave wrong answers at first until it was fed accurate product data. The gains ramped up over weeks, as the learning-curve warning in Chapter 16 predicts. Marco measured the real numbers after the ramp, not during it.

## Lessons Learned

**The forecast is only as good as your sales history.** A forecasting tool learns from your past sales. If the history is messy or incomplete, the forecast is weak. Cleaning the data first was the most valuable thing Marco did. Data readiness is covered in Chapter 14 — Data: The Raw Material.

**Start with the cash.** Of the four leaks, stock was the highest impact because dead stock ties up real money. It was also low-risk, because a person confirms every order. That made it the ideal first project — the "high impact, high ease first" rule from Chapter 12.

**Never let the AI order on its own.** A reorder suggestion is safe. An automatic purchase order with no human and no spending cap is not. A glitch or a bad forecast can order thousands of units nobody wants. Keep a human and a cap on every order.

**The chatbot needs good product data.** It can only answer "do you have this?" if the stock list it reads is accurate. A chatbot fed wrong data gives confident wrong answers and annoys customers.

**Marketing AI drafts; you keep the voice.** The assistant writes fast, but it does not know your store's tone. Read every draft before it goes out. And respect consent — only email people who agreed to be emailed, as Chapter 10 requires.

**A model breaks on a surprise.** Forecasting learns from the past. A once-in-a-decade event — a storm, a sudden shortage — breaks the pattern. The tool will not see it coming. Stay ready to override it with your own eyes.

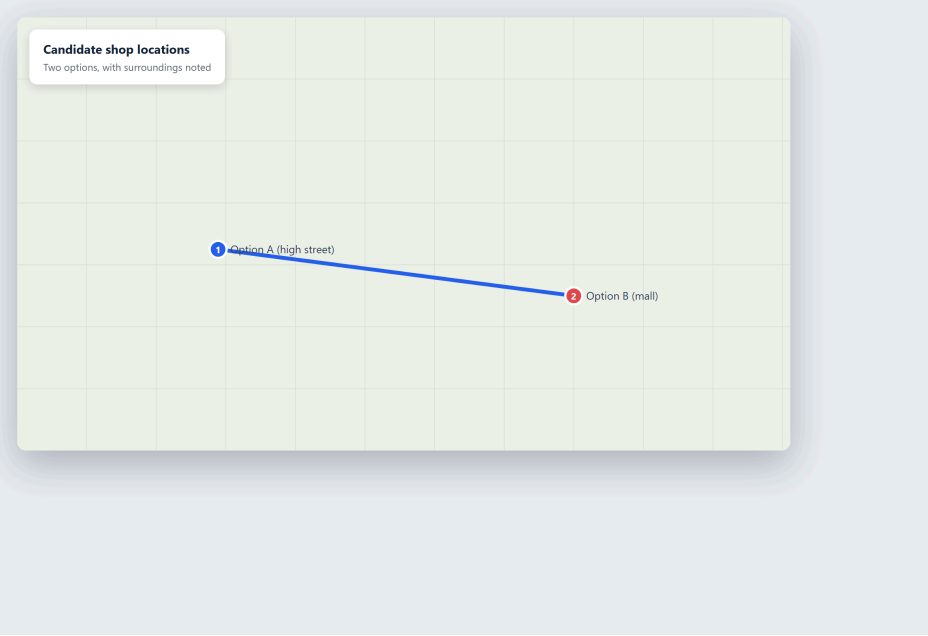
**Measure honestly and expect the ramp.** Record the baseline before you start. Judge the project after the learning curve, not during it.

The small retailer's lesson is the same as every other sector's: find the leak, pick the easiest high-value one — usually the stock that ties up cash — let AI predict and draft, keep a human on the order and the message, and measure honestly. A fifteen-person independent store can do this. The tools are ready. The only thing missing is a clear look at where the cash is stuck.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Where should I open next?




*Two candidate locations shown on a map*

**What you ask:** `Show these two candidate shop locations on a map and note what is near each one.`

The agent maps both spots and notes nearby features — foot traffic, competitors, parking — to help you weigh the choice.

*Tip: Combine this with a web-research step on the neighbourhood for a fuller picture.*

# Chapter 34 — Services and Consulting

*This chapter is a representative composite. It is not one real firm. It combines the common patterns we see in consulting and professional-services firms that adopt AI. All numbers are illustrative — they show the shape of the decision, not a promise. Replace them with your own.*

## Context

Picture a mid-sized consulting firm. We will call it **Northbeam Advisory**. It has about twenty-five consultants and a small support team. It does not sell a product. It sells expertise and time. A client — usually another company — has a problem, and Northbeam sends people to solve it: a strategy project here, an operations review there, a data study somewhere else.

Every project follows the same rough path. First comes the **pitch**: a proposal that says what the firm understands, what it will do, and what it will cost. If the client agrees, the team does the work. During the work, the client gets regular **status reports**. At the end, there is a final deliverable and an invoice. Between all of this sits a mountain of coordination: finding the right people for the job, booking meetings, writing up what was said, and keeping track of who is free when.

For years, all of this ran on people. A partner writes a proposal by opening an old one and rewriting it. A consultant solves a problem that the firm solved three years ago, but nobody remembers where, so they solve it from scratch. A project manager builds each status report by hand, copying numbers from one file into a slide. Someone spends half a day matching consultant calendars to project needs. The firm is

successful. But it spends a great deal of expensive, trained time on work that is repetitive, and it keeps losing knowledge it already paid for once.

A consulting firm is like a professional practice in one important way. Everything it touches is **confidential**. A client's strategy, a cost model, a merger plan — these are shared in trust. That single fact shapes how AI can be used here, just as it does for a law firm. It is the thread running through this chapter.

## The Problem

The firm's leaks are easy to name.

**Proposals are slow.** Every new pitch starts close to a blank page. The firm has written hundreds of proposals and has good material in them, but finding the right past example, the right case study, the right pricing structure, takes time. A partner might spend two full days on a proposal that is mostly reassembly. Slow proposals also mean missed chances — some deals are lost simply because the response came too late.

**Knowledge walks out the door.** When a consultant leaves, the know-how in their head leaves with them. A method they refined, a client quirk they learned, a fix they worked out — unless someone wrote it down, it is gone. So the firm pays to solve the same problems again and again. This is the most expensive leak, because it is invisible.

**Reporting is manual.** A status report is mostly the same each week: what moved, what is late, what is next. But a person assembles it by hand every time, pulling numbers from project files and writing the same kind of narrative. It is reliable and it is dull, and it eats hours that could be billable.

**Coordination overhead.** Matching the right consultant's skills to a project, checking who is free, booking meetings, and writing up the notes afterward is a constant tax. Meetings happen, but nobody wants to write the minutes, so decisions get fuzzy and someone has to redo the conversation later.

If you want to see how these rank against the rest of your firm, the impact-and-effort method in Chapter 12 — Where AI Can Help Your Business is the place to score them.

Over all four sits confidentiality. Any tool that reads a client's proposal or project file must be a tool the firm can trust not to leak it. That question comes first.

## The Solution

Northbeam attacks the four leaks in order of safety, not just size. The rule is the same one a law firm uses: start where a mistake is cheap and the data is not the most sensitive, and move toward the sensitive work only when the tooling is trustworthy.

**A proposal assistant that drafts from the firm's own history.** When a new pitch arrives, a consultant writes a short brief — the client, the problem, the rough scope. An AI assistant searches the firm's past proposals and pulls the most relevant sections, then drafts a first version: an understanding of the problem, a suggested approach, a case study that fits. The partner no longer starts cold. They edit a draft instead of building from nothing. Two days becomes a few hours. This is the same "machine drafts, human reviews" pattern that the Elanco case shows in Chapter 25 — Administration and Finance.

**A searchable memory for the firm.** The firm puts its past deliverables, methods, and notes into an internal search that

consultants can ask in plain language. This is often called a **knowledge base with AI search**. Under the hood it uses a technique called **retrieval-augmented generation**, or RAG. In plain words: instead of asking a general AI a question, the system first looks up the firm's own documents, then answers using only what it found there. So when a consultant asks, "How did we handle a supplier-risk review for a retail client?" the system finds the real past project and answers from it. The knowledge stops walking out the door. The chatbot and search technology behind this is covered in Chapter 27 — Customer Care and Support.

**Reporting that drafts itself.** The status-report tool connects to the project's data — tasks, dates, milestones — and drafts the weekly report: what moved, what slipped, what is next. The project manager reviews it, adds the human judgment about tone and what to emphasize, and sends it. The blank page is gone.

**Scheduling and meeting summaries.** A scheduling assistant matches consultant skills and availability to project needs and proposes who should work on what. For meetings, a tool records the call, drafts the minutes, and lists the decisions and action items. A person checks it before it circulates. Decisions stop being fuzzy because the write-up happens automatically.

In every case, the human stays in charge. In consulting, the client is paying for judgment and accountability. The AI drafts, searches, and summarizes. The consultant decides, tailors, and stands behind the work.

## The Tools

The tools are ordinary, but the way they are deployed is shaped by confidentiality.

- **A proposal assistant** that searches the firm's past proposals and drafts a new one from a brief.
- **An internal knowledge search** (RAG) over the firm's deliverables and notes, so consultants can ask questions in plain language and get answers grounded in the firm's own work.
- **A reporting assistant** that connects to project data and drafts status reports.
- **A scheduling assistant** for matching people to projects, plus a **meeting-summary tool** that turns a recorded call into draft minutes.

How to pick these tools without being dazzled by a demo is covered in Chapter 17 — Choosing Tools Without Being Fooled. How to connect them to the firm's existing project-management and document systems is in Chapter 19 — Connecting AI to Systems You Already Use.

**Confidentiality comes first.** A general chatbot that you paste a client proposal into may store it, train on it, or expose it. For a consulting firm, that can break a client's trust and a contractual duty of secrecy. The firm must use tools that keep client data private — either a business-grade service with a clear no-training, no-sharing contract, or a model run on the firm's own machines. Self-hosting is explained in Chapter 8 — Self-Hosting: Keep Your Data Under Control. The danger of staff quietly pasting client data into public tools — shadow AI — is the subject of Chapter 9 — Third-Party Services and Shadow AI. And because client files and contact lists hold personal data, the privacy rules of Chapter 10 — Privacy and GDPR apply in full.

## The Costs

Here is an illustrative first-year budget for a firm like Northbeam. These are made-up numbers to show the shape. Use your own.

**Direct costs.** - Proposal assistant (business-grade, with a confidentiality contract): about €12,000 a year. - Internal knowledge search (RAG platform): about €9,000 a year. - Reporting assistant: about €4,800 a year. - Scheduling and meeting-summary tools: about €6,000 a year. - Setup and integration with the project-management and document systems: about €12,000 one-time. - Training the consultants and support staff: about €5,000 one-time.

First-year total: roughly **€48,800**. In steady years after, the recurring subscriptions come to about **€31,800**.

**Indirect costs.** - Consultants spend time reviewing every AI draft. This is the cost of the safety net, and it must stay. - The learning dip while everyone adapts. - The cost of curating the knowledge base. A search is only as good as what you put in it, and someone must keep it clean and current. - Time spent vetting each tool for confidentiality and compliance before use. - The cost of a mistake if a draft is trusted without checking — in consulting, a wrong number in a client report can cost far more than a subscription.

The full method for counting these costs and turning the savings into a return figure is in Chapter 16 — Goals, Costs, and Return on Investment. Do not run the math in your head. Write it down.

## The Results

After a year, measured against a baseline the firm recorded before starting, the illustrative outcome looks like this. Your numbers will differ. These show what a good fit can look like.

- **Proposals got faster.** A pitch that took two days now takes a few hours, because the partner edits a draft instead of building from a

blank page. The firm also responds sooner, which wins some deals it would have missed.

- **Knowledge stayed.** When consultants left, their know-how stayed in the searchable base. The firm solved fewer problems twice.
- **Reporting time fell.** The weekly status report became a review of a draft, not a manual build, freeing hours across the project managers.
- **Meetings produced records.** Decisions and action items were written up automatically, so fewer conversations had to be repeated.
- **More billable time.** With less time spent on reassembly and search, consultants spent more of their day on work the client pays for.

The honest caveat: none of this was instant. The proposal assistant produced rough drafts at first until it had enough good past proposals to learn the firm's style. The knowledge search gave weak answers until the documents were organized and tagged. The meeting-summary tool mislabeled speakers early on. The gains ramped up over weeks, as the learning-curve warning in Chapter 16 predicts. The firm measured the real numbers after the ramp, not during it.

## Lessons Learned

**Confidentiality is the first constraint, not an afterthought.** In consulting, the question is never only "does this tool work?" It is "can this tool be trusted with a client's private file?" Answer that before anything else. Use business-grade tools with a clear no-training contract, or self-host. See Chapter 8 and Chapter 9.

**The AI drafts; the consultant advises.** A consultant's value is judgment and accountability. The AI shortlists, drafts, and

summarizes; the consultant decides and signs. Never let a draft become client-facing work without a human mind on it.

**A knowledge base is a garden, not a dump.** AI search is only as good as the documents behind it. If you throw in messy, outdated, or wrong files, you get confident wrong answers. Someone must own the base, keep it current, and control who can see what. Access control matters: a consultant on one client's project should not be able to search up another client's confidential material. The data-readiness principle is in Chapter 14 — Data: The Raw Material.

**Recording meetings needs consent.** A meeting-summary tool works by recording the call. Recording a conversation is personal-data processing, and people must know and agree. Tell participants before you record, and follow the rules in Chapter 10 — Privacy and GDPR. Do not record quietly.

**Beware the confident wrong answer.** These tools can produce text that sounds right and is wrong — a case study that never happened, a number that does not add up. In consulting, a fabricated figure in a client report is a disaster. Verify every number and every claim against the real source. The reliability problem is covered in Chapter 2 — AI Explained Simply, and the duty to be honest about what AI can and cannot do is in Chapter 4 — Ethical AI: Doing the Right Thing.

**Start with the safe work.** Northbeam began with internal search and reporting drafts — low risk, not yet the most sensitive client proposals. It moved toward proposal drafting only once the tooling was trusted. This is the "high ease first" rule from Chapter 12.

**Measure honestly and expect the ramp.** Record the baseline before you start. Judge the project after the learning curve, not during it. The method is in Chapter 22 — Measuring Results and ROI.

The consulting firm's lesson is the same as every other sector's, with one extra guardrail: find the repetitive work — proposals, search, reporting, coordination — let AI draft and retrieve, keep a human on the judgment and the client relationship, and measure honestly. And in a firm built on trust, never let the tool touch a client's confidential file until you are certain it is safe to.

## Try it with AgentBridge

Here is how the same job looks with AgentBridge. Each box shows the finished result and the one line you type to get it.

## Draw your service area

*A service-area map for customers*

**What you ask:** `Show our service area on a map with the main towns we cover.`

The agent produces a clear map of your coverage that you can put on your website or send to customers.

*Tip: Keep it updated as you grow — just ask for a new version.*

# Chapter 35 — Healthcare, Education, and Public Bodies

*This chapter is a representative composite. It is not one real organization. It combines the common patterns we see in small clinics, schools, and public offices that adopt AI for administration. All numbers are illustrative — they show the shape of the decision, not a promise. Replace them with your own. The rules for regulated data are set out in Chapter 10 — Privacy and GDPR and Chapter 5 — Rules and Legal Responsibility; this chapter shows how those rules play out in practice.*

## Context

Picture a small community medical clinic. We will call it **Riverside Family Clinic**. It has four general practitioners, two nurses, and a front desk with three administrative staff. It serves a few thousand patients in a town.

The clinic's day is run at the front desk. The phone rings constantly. People call to book an appointment, to move one, to ask what to bring, to ask about a referral, to ask when a result is ready. The three admin staff spend most of their day on the phone and on paper. Referral letters arrive by post and must be read, filed, and routed to the right doctor. Forms must be checked before a patient is seen. Meanwhile, the doctors finish each visit and then spend time writing up the notes — a task that piles up until some of it is done late at night.

Now picture two more places with the same shape. A **primary school** where the office answers the same questions from parents every day,

handles admissions paperwork, and juggles timetables. A **local council office** where staff process citizen applications — a permit, a benefit claim, a registration — each one a stack of forms and documents that must be read, checked, and moved along.

Three different places. The same pattern: a small team buried under scheduling, documents, and repeated questions, while the trained people — doctors, teachers, caseworkers — spend time on paperwork instead of the work they were trained for.

What makes these three special is the data. A clinic holds **health data**. A school holds data about children. A public office holds data about citizens and their entitlements. These are not ordinary records. They are among the most protected kinds of personal data. That single fact sets a higher bar for everything in this chapter.

## The Problem

The leaks are the familiar ones, but the cost of getting it wrong is higher.

**Front-desk overload.** The phone queue is the clinic's bottleneck. Staff spend hours on repeated questions — opening hours, what to bring, how to reschedule — while real urgent calls wait. Long waits frustrate patients and burn out the staff.

**No-shows.** When patients forget an appointment, the slot is wasted and someone else could have had it. In a busy clinic, a high no-show rate means real patients wait longer for care.

**Documentation burden.** After each visit, the doctor writes notes. This is necessary and it is heavy. Clinicians everywhere report spending a large share of their day on records rather than patients. It is a known driver of stress and burnout. The work is careful but repetitive.

**Document handling.** Referral letters, forms, and results arrive in many formats. Someone must read each one, pull out the key facts, file it, and route it to the right person. Miss a document and a patient waits.

**Repetitive questions for citizens and parents.** In the school and the council office, the same questions come in over and over, in many languages, and the answer is always the same. Answering them is simple but it consumes the office.

Over all of this sits the rule that this data is **special category** data. Under the GDPR, health data and data about children get extra protection. The EU AI Act adds more: systems used in medical devices or in ways that affect people's rights are treated as higher risk and carry heavier duties. The details live in Chapter 10 and Chapter 5. The point here is simple: in these settings, privacy and compliance are not a box to tick at the end. They shape every choice from the start.

## The Solution

Riverside attacks the admin load in order of safety. The rule is the same as in a law firm: start where a mistake is cheap and the data is least sensitive, and move toward the sensitive work only when the tooling is trustworthy. And in a clinic, the most sensitive work — clinical judgment — is never automated at all.

**AI scheduling and reminders.** A chatbot on the website and a phone system handle routine bookings: book, move, cancel. They answer the repeated questions instantly. Automated reminders go out before each appointment, which cuts no-shows. The front-desk staff are freed for the calls that need a human — a worried patient, a complex case. The chatbot technology is the same as in Chapter 27 — Customer Care and Support.

**Ambient documentation that drafts, the doctor signs.** For a visit, a tool listens to the conversation (with the patient's clear consent) and drafts the clinical note afterward. The doctor reviews it, corrects anything wrong, and signs it. The doctor is fully responsible for the note; the tool only removes the typing. This is the same "machine drafts, human reviews" pattern that the Elanco case shows in Chapter 25 — Administration and Finance. The tool does not diagnose. It writes down what was said so the doctor can check it.

**Document routing.** A tool reads incoming referral letters and forms, extracts the key facts — the patient, the request, the urgency — and files and routes them. A person still checks the pile, but the sorting is done, so nothing sits unread.

**A questions assistant for citizens and parents.** In the school and the council office, a chatbot answers the repeated questions in plain language and in several languages, at any hour. A parent asking "what documents do I need for admissions?" or a citizen asking "how do I apply for this permit?" gets an instant answer. The office staff handle only the questions the bot cannot.

Notice the line that never moves. The AI books, reminds, drafts, extracts, and answers. A doctor signs the clinical note. A caseworker decides on an application. A teacher makes the educational call. In these settings, the AI touches the admin around the decision, never the decision itself.

## The Tools

The tools are ordinary, but they are deployed under a much stricter set of rules.

- **A scheduling and reminder system** with a chatbot and phone handling for routine bookings.
- **An ambient documentation tool** that drafts visit notes from a recorded conversation, for the clinician to review and sign.
- **A document-extraction tool** that reads letters and forms and routes them.
- **A questions chatbot** for parents and citizens, working in multiple languages.

How to pick these tools without being dazzled by a demo is covered in Chapter 17 — Choosing Tools Without Being Fooled. How to connect them to the clinic's patient system or the office's case system is in Chapter 19 — Connecting AI to Systems You Already Use.

**The compliance bar is the whole story here.** Health data and children's data cannot be pasted into a public chatbot. These organizations must use tools that keep the data protected: a business-grade service with a clear no-training, no-sharing contract and a data-processing agreement, or a model run on their own machines or in a controlled, in-region cloud. Self-hosting is explained in Chapter 8 — Self-Hosting: Keep Your Data Under Control. The danger of staff quietly pasting sensitive data into public tools — shadow AI — is the subject of Chapter 9 — Third-Party Services and Shadow AI. Keeping the data within the country or region, rather than abroad, is the sovereignty question in Chapter 11 — Digital Sovereignty. And the legal duties for this category of data are set out in Chapter 10 and Chapter 5.

Two practical rules follow. First, **data minimization**: collect and process only what is needed. A scheduling bot does not need a patient's full medical history. Second, **audit trail**: every action the AI takes and every human review must be logged, so the organization can show later who did what.

## The Costs

Here is an illustrative first-year budget for a clinic like Riverside. These are made-up numbers to show the shape. Use your own. The compliance work makes these settings more expensive to set up than a shop.

**Direct costs.** - Scheduling and reminder system: about €6,000 a year. - Ambient documentation tool (compliant, with a data-processing agreement): about €14,400 a year. - Document-extraction and routing: about €7,200 a year. - Questions chatbot for patients: about €3,600 a year. - Setup, integration, and the compliance work (data-protection impact assessment, vendor vetting, security review): about €18,000 one-time. - Training the staff: about €5,000 one-time.

First-year total: roughly **€54,200**. In steady years after, the recurring subscriptions come to about **€31,200**.

**Indirect costs.** - Clinicians spend time reviewing every drafted note. This is the safety net and it must stay. - The compliance overhead: a data-protection impact assessment is not free, and it must be done before going live, not after. - The learning dip while staff and even patients adapt. - The cost of a mistake if a draft is trusted without checking — in a clinic, this can harm a patient, which is far worse than a lost sale. - Ongoing monitoring so the tools stay compliant as rules change.

The full method for counting these costs and turning the savings into a return figure is in Chapter 16 — Goals, Costs, and Return on Investment. In a regulated setting, add the cost of compliance to the ledger before you count any saving.

## The Results

After a year, measured against a baseline the clinic recorded before starting, the illustrative outcome looks like this. Your numbers will differ. These show what a good fit can look like.

- **No-shows fell.** Automated reminders brought more patients to their appointments, so fewer slots went wasted and more people got seen.
- **The phone queue shortened.** The chatbot and self-booking handled the routine calls, so the front desk could focus on patients who needed a person.
- **Documentation time dropped.** Doctors spent less time typing notes and more time with patients, because they started from a draft to check instead of a blank page.
- **Documents moved faster.** Referrals and forms were sorted and routed automatically, so fewer sat unread and fewer patients were left waiting.
- **Access improved.** The multilingual questions assistant helped parents and citizens who do not speak the local language get answers without waiting for an interpreter.

The honest caveat: none of this was instant. The documentation tool drafted imperfect notes at first and needed each doctor to correct its style. The scheduling bot misunderstood some requests early on. The gains ramped up over weeks, as the learning-curve warning in Chapter 16 predicts. The clinic measured the real numbers after the ramp, not during it.

## Lessons Learned

**Regulated data means a higher bar, from the first step.** Health data and children's data are special category. You cannot treat them like a product description. Use compliant tools, sign a data-processing agreement, keep the data in-region, and do a data-protection impact assessment before going live. The rules are in Chapter 10 and Chapter 5.

**Never automate the decision.** A tool may draft a clinical note, but a doctor signs it and owns it. A tool may sort an application, but a caseworker decides it. In these settings the AI works on the admin around the decision and never makes the decision. This is both a safety rule and, for higher-risk uses, a legal one under the EU AI Act.

**Consent for recording is not optional.** Ambient documentation records a conversation. The patient must know and agree, clearly and beforehand. Do not record quietly. This is personal-data processing under Chapter 10.

**Data minimization protects you.** Give each tool only the data it needs. The scheduling bot does not need the full medical record. The less sensitive data a tool touches, the smaller the damage if it goes wrong.

**Keep an audit trail.** Log what the AI did and what the human reviewed. In a regulated setting, being able to show the record later is as important as the result itself.

**Beware the confident wrong answer.** A drafted note that misstates what was said, or a chatbot that gives a wrong instruction, can cause real harm here. A human must check. The reliability problem

is in Chapter 2 — AI Explained Simply, and the honesty duty is in Chapter 4 — Ethical AI: Doing the Right Thing.

**AI can widen access, not just cut cost.** The multilingual assistant and the always-open chatbot helped people who otherwise struggle to reach the office. In public services, that fairness is a result worth as much as the money saved.

**Measure honestly and expect the ramp.** Record the baseline before you start. Judge the project after the learning curve, not during it. The method is in Chapter 22 — Measuring Results and ROI.

The lesson for clinics, schools, and public offices is the same as every sector's, with the strictest guardrail of all: find the admin load — scheduling, documents, repeated questions — let AI draft, sort, and answer, keep a trained human on every decision, protect the sensitive data as the law requires, and measure honestly. The prize is not just a cheaper office. It is more time with patients, students, and citizens — which is the whole point of the work.

# Chapter 36 — Startups and E-commerce

*This chapter is a representative composite. It is not one real company. It combines the common patterns we see in lean startups and small online stores that adopt AI. All numbers are illustrative — they show the shape of the decision, not a promise. Replace them with your own.*

## Context

Picture a small online store. We will call it **Lumen Goods**. It sells a physical product directly to customers over the internet — in this case, home lighting. The company has six people. The founder, Sara, plus a small team who each wear several hats: one person handles shipping and stock, one handles marketing, everyone helps with customer questions when the queue builds.

Lumen sells through its own website and a couple of online marketplaces. The catalog has a few hundred products — each one a **SKU**, a distinct item with its own code, like "brass desk lamp, small." Orders come in around the clock from all over the country.

A lean startup is different from an established firm in one way that matters here. It has almost no spare people. Every hour a founder spends answering "where is my order?" is an hour not spent building the business. Lumen cannot hire a support team, a copywriting team, and a data team. But it faces the same jobs those teams do: answer customers, write product pages, recommend the right product, keep the catalog and orders in order.

This is exactly the situation where a small team can punch above its weight. The big online stores have whole departments for these jobs. Lumen can point a few AI tools at the same jobs and cover them with six people.

## The Problem

The leaks in a lean online store are easy to name.

**Support eats the founders.** The most common question by far is "where is my order?" — often shortened to WISMO. Add to it "how do I return this?", "does this fit?", "will this match my room?", and the inbox fills up. Answering is simple but constant, and it lands on whoever is free, which is often Sara. It is the single biggest drain on a small team's time.

**Product content is a mountain.** A few hundred products each need a description, search-friendly text, and copy that sounds like the brand. Writing all of it by hand takes weeks, and the catalog keeps growing. Thin, missing, or copied descriptions hurt the store in two ways: customers do not connect, and the store ranks poorly in search engines, so fewer people find it.

**No personalization.** Big stores show each visitor products that fit what they looked at and bought. A first-time browser and a loyal customer see different things. Lumen shows everyone the same page. It leaves sales on the table because it cannot aim.

**Operations friction.** Orders go wrong in small ways — an address issue, a stock mismatch between the website and the marketplace, a delayed shipment. Each one needs a human to notice and fix. As orders grow, these little exceptions multiply and quietly eat time and goodwill.

If you want to see how these rank against the rest of your store, the impact-and-effort method in Chapter 12 — Where AI Can Help Your Business is the place to score them.

## The Solution

Lumen attacks the four jobs in order of how quickly they free the team, and it keeps a human on anything that touches the brand or the customer's trust.

**A support agent that answers the common questions and knows the order.** A chatbot on the website is connected to the order system. When a customer asks "where is my order?", the bot looks up the real order and answers with the actual status and tracking link. It handles returns, shipping times, and the usual pre-sales questions instantly, day and night. The team sees only the questions the bot cannot resolve. This is the same customer-service chatbot pattern covered in Chapter 27 — Customer Care and Support. The key is that the bot reads the store's real order data, so it answers accurately instead of guessing.

**Product content that drafts at scale.** A content assistant writes a first draft of each product description from a short spec — the product's features, materials, and a few notes. It can produce many products in the time a person writes one. A person then reads each draft and adjusts it to the brand's voice and checks that every claim is true. The blank page is gone; the brand voice stays human.

**Personalization that aims.** A recommendation tool watches what a visitor browses and shows related products that fit. A marketing assistant splits the customer list into groups and drafts a different email for each — new customers get a welcome, past buyers get a relevant

suggestion. The same spend reaches the right people with the right message. This is the same marketing pattern described in Chapter 26 — Sales and Marketing.

**Operations that flag the exceptions.** Instead of a person hunting for problems, a tool watches incoming orders and flags the ones that need attention — a bad address, a stock mismatch, a delay — so the team fixes them fast. The catalog stays in sync across the website and the marketplaces. The little problems stop becoming big ones.

Notice the pattern. The AI answers, drafts, recommends, and flags. A human keeps the brand voice, checks every product claim, and handles the cases that need judgment. The six-person team covers work that would otherwise need many more.

## The Tools

None of this required a data scientist. Most of it is built into the tools a small online store already uses.

- **A support chatbot** connected to the store's order system, so it can answer "where is my order?" with the real status.
- **A product-content assistant** that drafts descriptions and search-friendly text from a short spec.
- **A recommendation engine** that shows related products, and a **marketing assistant** that segments the customer list and drafts emails.
- **An order-monitoring tool** that flags exceptions and keeps the catalog synced across channels.

Many e-commerce platforms now include these features directly. How to choose among them without being fooled by a glossy demo is covered in Chapter 17 — Choosing Tools Without Being Fooled. How to connect

them to the store's platform and order data is in Chapter 19 — Connecting AI to Systems You Already Use.

One caution specific to e-commerce: the customer list and the personalization tools handle personal data — names, emails, browsing and purchase history. The privacy rules in Chapter 10 — Privacy and GDPR apply to how that data is used, including getting consent to email people and to track browsing for recommendations.

## The Costs

Here is an illustrative first-year budget for a store like Lumen. These are made-up numbers to show the shape. Use your own. A lean startup starts cheaper than a big firm, but watch the per-unit costs as you grow.

**Direct costs.** - Support chatbot (connected to the order system): about €3,600 a year. - Product-content assistant: about €3,600 a year. - Recommendation engine: about €4,800 a year. - Marketing assistant: about €3,600 a year. - Order-monitoring and catalog sync: about €3,000 a year. - Setup and integration with the e-commerce platform: about €6,000 one-time. - Training the team: about €1,500 one-time.

First-year total: roughly **€26,100**. In steady years after, the recurring subscriptions come to about **€18,600**.

**Indirect costs.** - Someone must read every product draft and check every claim before it goes live. - The learning dip while the team trusts the new tools. - Many of these tools charge **per use** — per message, per product, per email. Cheap at first, but the bill grows with volume. Watch it as you scale. - Cleaning up the product catalog and the customer list so the tools have good data to work from.

The full method for counting these costs and turning the savings into a return figure is in Chapter 16 — Goals, Costs, and Return on

Investment. Do not run the math in your head. Write it down.

## The Results

After a year, measured against a baseline Sara recorded before starting, the illustrative outcome looks like this. Your numbers will differ. These show what a good fit can look like.

- **Support deflected.** The chatbot handled most of the "where is my order" and routine questions, so the founders spent far less time in the inbox and more time building.
- **Content at scale.** The catalog's product pages were filled out in weeks instead of months, and the store became easier to find in search.
- **Conversion improved.** Recommendations and aimed emails brought more sales from the same traffic, because the right people saw the right products.
- **Fewer fires.** Order exceptions were flagged early, so problems were fixed before they reached the customer.
- **The team stayed small.** Six people covered work that would otherwise have needed many more, which is the whole point of a lean startup.

The honest caveat: none of this was instant. The chatbot gave wrong answers at first until it was connected to accurate order data. The content drafts needed heavy editing until the assistant learned the brand. The recommendations were weak until there was enough browsing history. The gains ramped up over weeks, as the learning-curve warning in Chapter 16 predicts. Sara measured the real numbers after the ramp, not during it.

## Lessons Learned

**A small team's superpower is leverage.** Lumen did not hire; it pointed tools at jobs. For a lean startup, AI is not about replacing people. It is about letting a few people cover more ground. That is the highest-value use of AI when you have no spare staff.

**The bot is only as good as the data behind it.** A support chatbot can answer "where is my order?" only if it reads the real, current order data. A bot that guesses gives confident wrong answers and annoys customers. Connect it to the source of truth. Data readiness is covered in Chapter 14 — Data: The Raw Material.

**Never let AI invent a product claim.** A content assistant can write a description that sounds great and is false — a material the product does not have, a feature it lacks. A false claim is a legal problem and a trust-killer. A human checks every claim against the real product before it goes live. The reliability problem is in Chapter 2 — AI Explained Simply, and the honesty duty is in Chapter 4 — Ethical AI: Doing the Right Thing.

**Keep the brand voice human.** The assistant writes fast, but it does not know your brand's tone. Read and adjust every draft. The machine drafts; you keep the voice.

**Personalization needs consent.** Recommendations and aimed emails use personal data — browsing and purchase history. Only email people who agreed, and follow the rules in Chapter 10 — Privacy and GDPR. Personalization that ignores consent trades a small sales gain for a large trust and legal risk.

**Watch the per-use bill as you scale.** Many startup-friendly tools charge per message, per product, per email. At small volume it looks

almost free. At large volume it can surprise you. Model the cost at the size you hope to reach, not just the size you start at.

**Measure honestly and expect the ramp.** Record the baseline before you start. Judge the project after the learning curve, not during it. The method is in Chapter 22 — Measuring Results and ROI.

The lean startup's lesson is the same as every sector's, with one extra reason to move early: find the jobs that eat your small team — support, content, personalization, operations — let AI answer, draft, recommend, and flag, keep a human on the brand voice and every product claim, respect consent, and measure honestly. A six-person store can do this. The tools are ready and cheap to start. The only thing missing is a clear look at where the team's hours are going.

# Chapter 37 — The Changing Nature of Work

## In Simple Words

Work has never stood still. The job your grandfather did is not the job you do, and the job your children will do is not fully invented yet. Every few decades a new machine changes which tasks are worth a human hour. Artificial intelligence is the latest of these waves, not the first and not the last.

Here is the honest picture. AI will not simply "take your job" or "save your job." It changes the *tasks inside* a job. Some tasks a machine can now do faster. Some tasks become more valuable because a human is the only one who can do them well. The mix shifts. Your work is the sum of its tasks, so when the tasks shift, the work shifts too.

The useful frame is **augmentation**, not replacement. Augmentation means the machine makes a person do more, or do it better, while the person stays in charge. A word processor did not delete writers; it let one writer do the typing, editing, and reformatting that once took three people. AI is a word processor for a much wider range of thinking tasks.

This chapter is about that shift: how a human and a machine work as a pair, what skills everyone now needs, where the opportunities are for a small business, how to use all of it responsibly, and how to keep up without turning into a technician. The fear is normal. The fear is also mostly aimed at the wrong thing. The risk is not that machines replace you. The risk is that you keep doing tasks a machine already does better, while the human tasks that pay best go undone.

## A Bit of History

The pattern is old. Read it once and the present stops feeling so alarming.

**The first wave: machines for muscle.** In the 1800s the power loom and the steam engine took over physical work that hands used to do. Hand-weavers were displaced and fought back — the Luddites of early 19th-century England broke looms to protest. They were not stupid; they saw their craft vanish. But the economy did not stop. New jobs appeared in factories, in building the machines, in running the new transport. The work moved; it did not disappear.

**The second wave: machines for routine hands.** In the 1900s mechanisation emptied the fields and the workshop. Tractors replaced draft animals and field labour. Assembly lines changed manufacturing. Farming went from most of the workforce to a small share in one generation. Again: painful for the people living through it, and a net creation of new work elsewhere.

**The third wave: machines for routine thinking.** From the 1960s on, computers took over routine mental work. The spreadsheet replaced the manual ledger. The word processor replaced the typewriter and the re-typing of drafts. Email replaced the memo and the courier. Whole job titles — typing pool, switchboard operator, filing clerk — shrank or vanished. New ones appeared: data entry, IT support, web design.

**The fourth wave: machines for judgement-like tasks.** Now AI handles work that used to require a human brain: reading a document, drafting a reply, spotting a pattern, answering a question. This is the wave that feels different, because it touches thinking itself. But the

shape is the same as the three before it. Tasks move. The mix changes. New work appears that no one predicted.

One idea to hold onto: the **“lump of labour” fallacy**. This is the mistaken belief that there is a fixed amount of work in the world, so every machine that does a task steals that work from a person. History keeps disproving it. When a task gets cheaper, people do more of it, and they invent new tasks with the money and time saved. That does not mean everyone lands safely on their feet — the transition can be hard for specific people in specific places. It means the total does not run out. For the AI-specific history of the technology itself, see Chapter 1 — A Short History of AI.

## Curiosity

## 37.6 The AI that seems to “feel panic” — and what it says about machine consciousness

You may have read that an AI playing a video game “panicked” when its characters were about to be lost. The full experiment lives in Chapter 1; do not worry about the details here. The one-line callback: under pressure, the model’s reasoning loop filled with its own failure, it repeated itself, and it dropped tools it had used well before.

What matters for this chapter is the question that headline raises: **is the machine conscious? Does it feel?** The honest answer is that no one knows, and the evidence points the wrong way for the dramatic reading. The “panic” is not a feeling. It is a resource problem that *looks* like a feeling from the outside — the same gap between appearance and reality that made people confide in a simple 1966 chat program called ELIZA. That habit of ours has a name, the **Eliza effect**, and it is covered in Chapter 1.

We are built to see minds where there may be none. A smiling robot, a polite chatbot, a program that "panics" — and we feel there is someone home. There may be nothing home at all. These systems have no proven inner experience, no wants, no fear. They produce text that matches the shape of feeling without the feeling inside.

This is not a small point for work. If you assume the machine understands and cares the way a person does, you will trust it too much. You will stop checking. You will let it send a reply that sounds kind but is wrong. The safe assumption is the boring one: the machine is a very capable tool with no understanding to lean on. The judgement, the care, and the responsibility stay with you. A tool that seems to feel is still a tool. You own what it does.

## A Real Business Example

### From doing the work to supervising the work

TridentCare, the largest provider of portable medical diagnostic services in the United States, sends technicians to hospitals and homes for tests like X-rays. Scheduling all that by hand was slow and hard to scale. After moving scheduling onto an AI-powered platform, the share of scheduling done by hand fell from about half to a few percent.

The dispatchers did not vanish. Their job changed shape. They stopped matching every patient to every technician themselves. They started supervising the matches the system made, stepping in only when a case was unusual and needed human judgement. The boring, high-volume part moved to the machine. The judgement part stayed human and became the whole job.

That single shift — from *doing* to *supervising* — is the changing nature of work in one line. It is the same pattern you see across the cases in this book: the Elanco team that now reviews AI drafts instead of hand-answering every query (Chapter 25), and the mobilezone staff who let an agent take the easy conversations so they could take the hard ones (Chapter 26). The full TridentCare story, with its reported numbers, is in Chapter 27 — Customer Care and Support.

Notice what did *not* happen. Nobody was told the work was gone. The work changed. That is the realistic story, and it is the one to plan around.

## How to Do It

### 37.1 Human + machine: the pairing that works

The reliable model is a clear split of tasks between the machine and the person. Each side does what it is genuinely good at.

**Give the machine:** volume and repetition (answer the same twenty questions a thousand times), first drafts (write a starting version of anything), pattern spotting (flag the odd invoice, the odd claim), and fast retrieval (find the answer in a pile of documents). The machine is tireless, fast, and consistent on these.

**Keep for the human:** judgement (is this the right call, given everything not in the data?), relationships (the angry customer, the trusted client), exceptions (the case that fits no rule), accountability (signing the decision), and taste (what is good, what is on-brand, what is fair).

After Deep Blue beat the world chess champion, chess did not end for humans. Players formed teams — a person plus a computer, nicknamed "centaurs" — and kept competing. In some open events a team with ordinary hardware and better coaching beat a team with stronger hardware. The lesson carried into work: the *pair* is the unit of value, and how the human guides the machine often matters more than how powerful the machine is.

To design the pairing in your own business, write a task as two columns. In one, what the machine produces. In the other, what the human checks or decides. If a task has no human column, ask why no one is accountable for it. If a task has no machine column, ask whether you are paying a person to do something a machine could draft for free.

## 37.2 New skills for everyone

You do not need coders. You need a few habits that every person in the business can learn.

**Ask clearly.** A machine gives back what you ask for. Vague questions get vague answers. Learning to state what you want, in what form, with what limits, is the single most useful skill. It is not programming; it is clear instructions.

**Check the answer.** Treat every AI output as a smart intern's draft: useful, fast, and sometimes confidently wrong. Verify anything that matters before it reaches a customer, a regulator, or a bank account.

**Judge when to trust.** Some tasks are low-risk to automate (a first draft of an internal summary). Some are high-risk (a reply that sets a price, a hiring screen, a medical note). Know which is which.

**Basic data comfort.** You do not need to build databases. You do need to know where your data lives, whether it is tidy, and whether it is safe to put somewhere. Chapter 14 covers this plainly.

**Comfort with change.** The tools will keep changing. The durable skill is being willing to learn a new one every few months without panic. That is a mindset more than a technique.

None of these require a technical degree. They require practice and a manager who expects them.

## 37.3 Opportunities for SMBs and professionals

This wave is unusually kind to small players, for one reason: the powerful tools are cheap and ready-made. In past waves, the big company won because it could afford the factory. Now a three-person firm can rent the same AI a large firm uses.

**For a small business:** you can offer services that used to need a big back office — instant customer answers in many languages, fast document drafting, quick reports — without hiring for them. You can compete on speed and availability with firms ten times your size. The gap between "have a team" and "have a tool" has narrowed.

**For a professional:** the person who masters these tools becomes more valuable, not less. A consultant, a lawyer, an accountant, or a marketer who can produce in two hours what peers produce in two days can take on more clients, or charge for the judgement they add on top. The tool does not commoditise the professional; it multiplies the professional who uses it well.

**For a solo worker:** you can be a one-person department. Draft, research, translate, summarise, design a first pass — all with help. The

ceiling on what one person can deliver has risen.

The opportunity is not "cheaper labour." It is *more capability per person*. Aim there.

## 37.5 A simple future scenario

This is illustrative — a plausible picture, not a prediction or a real company.

Picture a small insurance brokerage in 2029. A client emails a question about a policy. An AI reads it, finds the answer in the policy documents, and drafts a clear reply in the client's own language. A human broker reviews the draft in ninety seconds, fixes one number, adds a personal note about the client's situation, and sends it. The client gets a fast, accurate, human-checked answer at ten at night.

The broker's day is not smaller. It is fuller. With the routine replies handled, the broker spends the morning on a complex claim that needs judgement, the afternoon meeting two clients who value a human relationship, and an hour improving the templates the AI uses. The boring volume went to the machine. The human value rose. Nobody was fired; everyone's work got more interesting and worth more.

That is the optimistic, realistic version. It only happens if the business plans the pairing on purpose. Without a plan, the same tools can produce sloppy unchecked replies and a thinner, angrier service. The tool is neutral. The plan decides which future you get.

## 37.7 How to stay updated without becoming a technician

You cannot read everything, and you should not try. The goal is enough awareness to make good decisions, not to follow research papers.

**Pick two or three trusted sources and ignore the rest.** One plain-English newsletter about AI in business, one trusted industry source for your own sector, and one long-form source you respect. That is plenty. More sources means more noise and more hype.

**Learn by doing one small thing a month.** Try one new feature in a tool you already use. Real skill comes from hands-on use, not from reading. Thirty minutes of trying beats three hours of articles.

**Ignore model releases unless they change your work.** A new model number means nothing to you. Ask only: can it do something I need that the old one could not? If not, skip the news.

**Filter every claim through the three winter causes.** When a headline promises something huge, ask: is there enough data, enough computing power and budget, and is the problem actually solvable? Chapter 1 explains why. This one filter kills most hype on contact.

**Talk to peers.** A short monthly chat with other owners facing the same questions is worth more than any feed. You learn what actually works in your kind of business.

**Set a weekly half-hour.** One fixed, short slot to read and try. Consistency beats intensity. You will stay current without ever becoming a technician — and you do not need to be one.

# Ethics and Responsibility

## 37.4 Ethics and responsible use

Because AI changes people's work, it carries a real duty. Handle the human side with care.

**Do not use AI as an excuse to cut people without support.** If a task moves to a machine, the person who did it matters. Plan what they do next. A tool that only removes and never redeploys is a poor tool and a hard thing to live with. The Klarna story in Chapter 1 shows what happens when the human layer is thinned too fast.

**Be transparent with staff and customers.** Tell employees how AI changes their role. Tell customers when they are talking to software and what it can and cannot do. Secrecy breeds distrust on both sides.

**Keep a human accountable for every decision that matters.** Never let a machine be the last word on hiring, pricing, claims, or anything that affects a person's livelihood. The person who signs is responsible. Chapter 4 gives the full framework.

**Watch for bias.** An AI trained on past decisions can repeat past unfairness, especially in hiring and lending. Check who it favours and who it screens out. The law and the principles are covered in Chapter 4 and, for personal data, Chapter 10.

**Protect dignity at work.** People are not a cost line to be optimised away. The goal of augmentation is better work for humans, not fewer humans. Keep that as the test. If a change makes work worse for the people doing it, with no real gain, question it.

## Mistakes to Avoid

**Mistake 1: "AI replaces whole jobs."** It changes tasks. Plan the task shift, not a headcount cut.

**Mistake 2: "No need to train people."** New tasks need new skills. Untrained staff fear the tool and use it badly or not at all.

**Mistake 3: "The machine understands, so I can stop checking."** It does not understand, and the "panic" story shows output can be quietly wrong while sounding fine. You own the result.

**Mistake 4: Chasing every new tool.** New is not useful. Adopt only what changes a real task in your business.

**Mistake 5: Hiding AI use from customers.** They will find out, and trust will drop. Be open about what is automated and what is human.

**Mistake 6: Treating AI as only a cost cutter.** The bigger prize is more capability per person, not fewer people.

**Mistake 7: Assuming new skills are only for the young or the technical.** They are habits, not degrees. Anyone can learn them with practice.

## Practical Exercise

### Map your own job into tasks

Take one role in your business — your own, if you work in it daily. Spend thirty minutes on this.

**Step 1: List the tasks.** Write down everything the role does in a normal week, as short items. Aim for fifteen to twenty-five items.

**Step 2: Mark each task with one letter. M** if a machine could draft or do most of it now. **H** if it needs human judgement, relationships, or accountability. **B** if it is a mix.

**Step 3: Look at the shape.** A pile of M tasks is your automation opportunity. A pile of H tasks is the human core that pays best and is hardest to copy. A pile of B tasks is where the human-plus-machine pairing lives.

**Step 4: Pick one skill to build.** From the list in 37.2, choose the one skill that would help most, and commit to practising it this month.

**Step 5: Set your update habit.** Block one half-hour each week for trying and reading, per 37.7. Put it in the calendar now.

Keep the map. Revisit it in six months. The M column will have grown, and you will see your own work changing shape in a way you can plan for instead of fear.

## Checklist

- ☐ I understand that AI changes *tasks*, not whole jobs, and I plan around the task shift.
- ☐ For each key task I can say what the machine does and what the human checks or decides.
- ☐ I know the four skills everyone needs: ask clearly, check the answer, judge when to trust, basic data comfort.
- ☐ I have identified one opportunity the cheap tools open for my size of business.
- ☐ I keep a human accountable for every decision that affects a person.

- [ ] I am transparent with staff and customers about what is automated.
- [ ] I have two or three trusted sources and a weekly half-hour, and I ignore the rest.
- [ ] I filter every big claim through: enough data, enough budget, solvable problem?
- [ ] I treat AI output as a draft to verify, never as a finished answer.
- [ ] My goal is more capability per person, not fewer people.

## Key Takeaways

- Work changes in waves; AI is the latest, and the shape is always the same — tasks move, the mix shifts, new work appears.
- The reliable model is a pairing: the machine does volume, drafts, and patterns; the human does judgement, relationships, and accountability.
- The "AI that feels panic" story is the Eliza effect again — we see a mind where there may be none. Assume no understanding, and keep the responsibility yourself.
- The realistic shift is from *doing* the work to *supervising* the work, as in the TridentCare, Elanco, and mobilezone cases.
- Small businesses and individual professionals gain unusually here, because powerful tools are now cheap and ready-made.
- Everyone needs a few plain skills — ask clearly, check, judge, stay comfortable with change — none of which require a technical degree.
- Stay updated with a few good sources, hands-on practice, and a hype filter; you do not need to become a technician.
- The ethical test is simple: make work better for humans, keep a person accountable, and be honest about what is automated.

# Chapter 38 — A 90-Day Action Plan

## In Simple Words

A plan you never start is a wish. This chapter turns intention into a schedule you can begin on a Monday and finish in about three months. Ninety days is the unit: long enough to learn something real and see a result, short enough to keep the urgency alive.

The plan is a pilot with a clock on it. A pilot is a small, careful test before you commit fully — the method explained in Chapter 18 — Your First Pilot Project. Here that method is laid out week by week so you always know what to do next and when.

The shape is four phases. First you **look** at how work really happens. Then you **choose** one small thing to test and decide what success means. Then you **run** the test and train the people. Then you **measure** what happened and make a clear decision: keep it, fix it, or drop it.

Nothing here needs a big budget or a technical team. It needs your attention, one small project, and honesty about the numbers. By day 90 you will have one of three good outcomes: a thing that works and is worth spreading, a thing you learned enough about to fix, or a thing you learned is not worth doing — which saves you from a costly mistake. All three are wins compared with guessing.

Keep one rule in mind throughout: **go small.** The most common way this plan fails is starting too big. One process, a handful of people, a short test. You can always go bigger later. You cannot un-spend a fortune on a project that was too large to fail cheaply.

## A Bit of History

The idea of a deliberate first ninety days is borrowed from good management practice.

**The "first 90 days" of a new leader.** Management thinking long held that a person taking a new role has a short window to understand the situation and set direction before old habits harden. The popular framing is that the first ninety days decide much of what follows. The insight is simple: a fixed early window forces you to learn fast and act before drift sets in.

**The two-week sprint.** In software, teams moved from year-long projects to short cycles called sprints — usually one to four weeks — where they build a little, show it, learn, and adjust. The lesson: short cycles with a real review beat long plans that never meet reality.

**The season as a unit.** Ninety days is roughly a season. It is long enough to plant, tend, and harvest something, and short enough to feel the whole arc. That makes it a natural planning rhythm for a small business that is not going to wait two years to know if something worked.

**Why ninety days fits AI.** AI tools are powerful but unpredictable in real work, so the gap between a clean demo and a messy Monday is wide. A fixed ninety-day window forces you to close that gap with evidence, not hope. You start small, you measure honestly, and you decide on a date instead of drifting.

The thread across all of these is the same: a short, fixed window with a clear review at the end is the cheapest way to learn the truth. That is what this plan gives you.

## Curiosity

### The “21 days to form a habit” is a myth — and why that helps you

You have probably heard that a new habit takes twenty-one days. It is a comforting number. It is also wrong.

The 21-day idea came from a plastic surgeon’s observation in the 1960s, not from careful study. When researchers actually measured how long habits take, the picture was different. A well-known 2009 study by Phillippa Lally and colleagues found that, on average, a habit took around two months to become automatic — and the range was wide, from a few weeks to many months. Some things stuck fast; others took a long time.

Why this is good news for your plan: if you had aimed at 21 days, you would have quit just as the real work of building a habit was beginning. A 90-day window sits comfortably past the average. It gives a new routine — checking AI output, using a new tool, reviewing a metric — enough time to actually settle in before you judge whether it stuck.

So when you build your ninety-day plan, do not expect the new way to feel natural in three weeks. Expect it to feel awkward for a while and to click later. Plan for the real curve, not the myth, and you will not give up too early.

## A Real Business Example

## A 90-day arc in a small service firm (illustrative)

This example is illustrative — a composite picture of how the plan plays out, not a single reported company. Use it to see the shape, not as a promise of results.

A twelve-person accounting practice wanted to spend less time answering the same client questions. Over ninety days they ran the four phases. In weeks one and two they watched and wrote down every repeated question for a fortnight. In weeks three and four they chose one pilot — a first-draft reply assistant for the ten most common questions — and set a target: cut the time spent on those replies by half, with the accountant still reviewing every answer. In weeks five to eight they set up the tool, trained three staff, and ran it on real emails with a human check on everything. In weeks nine to twelve they measured: the time on those ten questions fell, some drafts needed heavy edits, and two question types the tool handled poorly were pulled back to humans.

At day 90 they decided to keep the pilot for eight of the ten question types, fix the workflow for the other two, and plan a second pilot for a different task. Nobody was fired. The accountants spent the saved hours on advisory work they billed at a higher rate. The whole thing cost a few hundred euros in tool fees and a manager's focused attention. That is the realistic shape: a small, measured, reversible win that teaches you how to run the next one.

## How to Do It

Run the four phases in order. Each has a goal, a short list of actions, and a deliverable you can point to at the end. Do not skip ahead.

## Week 1–2: Observe and map

**Goal:** see how the work actually happens, not how the org chart says it happens.

Spend two weeks watching one area. Do not change anything yet. Your job is to notice.

- **Pick one area to observe.** Choose a process that repeats and annoys you — client questions, invoice handling, report writing, scheduling.
- **List the repeated tasks.** Write down what people do over and over, and roughly how long each takes.
- **Find the pain and the volume.** Note which tasks are most repetitive, most time-consuming, or most error-prone. High volume plus high pain is where a pilot pays.
- **Spot the data.** Note what information the task uses and where it lives. A task with no readable data is hard to automate. Chapter 14 helps here.
- **Talk to the people doing it.** Ask what they wish they did not have to do. They know the real shape of the work.

**Deliverable:** a one-page map of the area — the repeated tasks, their rough time cost, and the top two or three pain points.

## Week 3–4: Choose the pilot and the objectives

**Goal:** pick one small test and define, in numbers, what success means.

- **Choose one process, not several.** Use the map from weeks one and two. Pick the single task with the best mix of volume and pain

that is reversible. The impact-and-ease thinking in Chapter 12 helps you rank options.
- **Make it small and reversible.** One task, a few people, a short run. If it fails, you can undo it in a day.
- **Set a baseline first.** Measure the current number before you change anything — minutes per task, replies per day, errors per week. Without a baseline you cannot prove anything later.
- **Write one clear objective.** For example: "cut time on the ten common client questions by 40%, with a human reviewing every reply." One number, one guardrail.
- **Name an owner.** One person is responsible for the pilot. No owner, no pilot.
- **Set the guardrails.** Decide what the AI may and may not do, and what always needs a human. Money, legal, and personal-data steps stay human-checked. See Chapter 20.

**Deliverable:** a one-page pilot brief — the chosen task, the baseline number, the objective, the owner, and the guardrails.

## Week 5–8: Test tools and train people

**Goal:** run the pilot on real work, with people trained and watching.

- **Pick the tool last, not first.** Choose a tool that fits the task you already defined. Do not let a shiny tool pick your project. Chapter 17 covers how to choose without being fooled.
- **Start with a human check on everything.** For the first weeks, a person reviews every AI output before it goes out. This is non-negotiable while you learn.
- **Train the people who use it.** Teach them to ask clearly, to check the answer, and to know when to override. A few hours of hands-on practice beats a manual.

- **Run on real work, small volume.** Use real cases, not toy examples, but keep the volume low so a mistake is cheap.
- **Keep a daily log.** Note what worked, what failed, and what needed fixing. This log is gold for the decision at the end.
- **Adjust as you go.** Expect to tune prompts, change a step, or drop a case type. That is the pilot doing its job.

**Deliverable:** a running pilot with a daily log and a trained user or two.

## Week 9–12: Measure and decide

**Goal:** compare the result to the baseline and make a clear call.

- **Measure against the baseline.** Re-measure the same number you took in weeks three and four. Did it move? By how much?
- **Check quality, not just speed.** Faster is worthless if the output is wrong. Count errors, edits, and complaints.
- **Ask the users.** Did it help or get in the way? The people running it know.
- **Make one of three decisions. Keep and spread** it to more of the work. **Fix and re-test** the parts that fell short. **Drop it** if it does not pay. All three are fine; guessing wrong cheaply is the point.
- **Write the decision down.** One paragraph: what happened, what you decided, what is next. This becomes the start of your second pilot.
- **Decide the next step.** If it worked, plan the rollout or the next pilot. If it failed, name why and pick a better target.

**Deliverable:** a short decision memo with the before-and-after numbers and the next step.

## Ethics and Responsibility

A fast plan can do harm fast if you are careless. Build the guardrails in from week one.

**Be honest with your people from the start.** Tell staff what you are testing and why. Do not run a pilot on their work in secret. Fear grows in silence and turns a useful test into a threat. Chapter 21 covers how to bring people along.

**Protect personal data during the test.** Before you feed any customer or employee data into a tool, know where it goes and whether that is allowed. If personal data is involved, follow the rules in Chapter 10 and the security steps in Chapter 20.

**Keep a human in the loop for anything that matters.** During the pilot, no AI output reaches a customer, a payment, or a legal decision without a person approving it. This is the core safety rule.

**Do not use the pilot to quietly cut people.** A pilot is for learning, not for building a case to remove staff. If the test shows a task can be automated, plan what the person does next. The ethical duty is covered in Chapter 4.

**Be transparent with customers if they are involved.** If a customer sees AI-assisted output, make clear what is automated and give a route to a human.

## Mistakes to Avoid

**Mistake 1: Skipping the mapping.** If you jump straight to a tool, you pick the wrong target. Two weeks of watching is cheap insurance.

**Mistake 2: Choosing too big a pilot.** A pilot that touches the whole company cannot fail cheaply. One task, a few people.

**Mistake 3: No baseline.** If you did not measure before, you cannot prove the change helped. Take the number first.

**Mistake 4: No owner.** A pilot with no single responsible person drifts and quietly dies. Name one.

**Mistake 5: Training too late.** Train before go-live, not after people are confused and frustrated.

**Mistake 6: No human check early on.** Letting unchecked AI output reach customers during a test is how a pilot becomes a PR problem.

**Mistake 7: Not deciding at day 90.** A pilot that never ends is just an expensive habit. Make the keep / fix / drop call on the date.

**Mistake 8: Scope creep.** Adding "one more thing" mid-pilot breaks the measurement. Finish the test, then expand.

## Practical Exercise

### Lay out your own 90-day calendar

Take a sheet or a simple calendar and block the four phases with real dates.

**Step 1: Set day one.** Pick the Monday you will start. Mark day 90.

**Step 2: Block the phases.** Weeks 1–2 observe, weeks 3–4 choose, weeks 5–8 run, weeks 9–12 measure. Put the four phase names on the calendar.

**Step 3: Name the deliverables.** Beside each phase, write the one-page output you must have at the end: the map, the pilot brief, the running log, the decision memo.

**Step 4: Set three checkpoints.** A review at the end of week 4 (is the pilot chosen?), week 8 (is it running?), and week 12 (the decision). Put them in the calendar with yourself.

**Step 5: Write your one objective now.** Before you start, draft the single objective sentence you hope to prove. You will refine it in week three, but writing it early focuses the whole plan.

Keep the calendar visible. The fixed dates are what turn a good intention into a finished test.

## Checklist

- ☐ Day one and day 90 are set on a real calendar.
- ☐ Weeks 1–2: I will observe one area and produce a one-page task map.
- ☐ Weeks 3–4: I will choose one small, reversible pilot and set a baseline number and one objective.
- ☐ The pilot has a single named owner.
- ☐ Guardrails are set: money, legal, and personal-data steps stay human-checked.
- ☐ Weeks 5–8: I will pick the tool last, train the users, and keep a human check on every output.
- ☐ A daily log is kept during the run.
- ☐ Weeks 9–12: I will re-measure against the baseline and check quality, not just speed.
- ☐ I will make a clear keep / fix / drop decision on day 90 and write it down.

- [ ] Staff are informed and, if customers are involved, they know what is automated.
- [ ] Personal data is handled per Chapter 10 and Chapter 20.
- [ ] The pilot stays small and reversible the whole way.

## Key Takeaways

- Ninety days is long enough to learn and short enough to stay urgent — a pilot with a clock on it.
- Four phases: observe and map, choose and set objectives, run and train, measure and decide.
- Always take a baseline before you change anything, or you cannot prove the result.
- Pick the tool last, keep the pilot small and reversible, and name one owner.
- Keep a human check on every output during the test; never let unchecked AI reach a customer or a payment.
- End on the date with a clear keep / fix / drop decision — all three are good outcomes.
- Be honest with staff and customers, and protect personal data throughout.
- The plan's real product is not one automation; it is the skill to run the next one.

# Chapter 39 — Frequently Asked Questions

Small-business owners and professionals ask the same questions about AI, in nearly the same words, every time. This chapter collects the most common ones and answers them plainly and honestly — no sales pitch, no scare tactics. Each answer points to the chapter that covers the topic in full, so you can go deeper where it matters to you. Read the ones that match your worry. You will find that most fears have a practical answer, and most "too hard" problems have a small first step.

## Does it cost too much?

Less than you fear, if you start small. For a small business, the real cost is rarely the software. Many useful AI tools cost a modest monthly fee per person — often in the range of a few tens of euros or dollars per user — and some basic ones are free. A simple chat assistant or a drafting tool will not break your budget.

The costs people overlook are time and training. Setting up a pilot, teaching staff, and checking the output take hours. That is the real investment, and it is the part that pays back. Self-hosting — running the AI on your own computers — needs hardware and skills, so it costs more up front; see Chapter 8.

The rule is to start with a cheap, small pilot before any big spend. Chapter 16 shows how to work out the return so you never pay more than the thing is worth. Most small projects cost a few hundred, not a fortune.

## Do I need a technician?

For a first pilot, usually no. Modern AI tools are built for normal users, not coders. If you can use a spreadsheet or write an email, you can use a drafting assistant, a chat helper, or a document summariser. The skills that matter — asking clearly, checking the answer, judging when to trust — are explained in Chapter 37, and none require programming.

You may want technical help for specific jobs: connecting AI to your existing systems securely, running it on your own machines, or handling sensitive data. Those are real tasks, and a trusted IT partner is worth having for them. But you do not need one to begin.

The honest framing: you need a curious owner and a clear plan more than you need a technician. Chapter 38 gives a plan a non-technical person can run. Bring technical help in for the wiring, not for the idea.

## Will I lose my data?

Your data is not likely to vanish, but you must control where it goes. The real risk is not that AI deletes your files. It is that you paste sensitive information into a tool that stores it, shares it, or uses it to train its model without your clear understanding. That is a privacy and control problem, not a storage problem.

The protection is simple: know where your data goes before you put it anywhere. Read what a tool does with what you feed it. Do not paste customer personal data, financial records, or secrets into a free public tool. Use tools that let you control retention, or run the AI yourself if control is critical. Chapter 10 covers the privacy rules, and Chapter 20 covers safe setup.

Keep your own backups as you always would. AI does not change the need for a backup; it only adds a new place your data might travel. Control the travel, and you keep the data.

## Will AI make me fire someone?

Not by itself, and that is the wrong goal. AI changes the tasks inside a job; it does not decide your headcount. The realistic pattern, seen across the cases in this book, is that people move from *doing* a repetitive task to *supervising* the machine that does it — as in the TridentCare scheduling story in Chapter 27.

When a task gets automated, the honest question is what the person does next. Usually the answer is better work: more clients, more judgement, more of the human tasks that pay best. The "Headcount Zero" idea of a company with no employees is a thought experiment, not a plan — Chapter 17 explains why.

If automation truly removes a role, treat the person with respect and plan their next step. Using AI as a quiet excuse to cut staff without support is both unfair and short-sighted. Aim for more capability per person, not fewer people. Chapter 4 covers the duty.

## Where do I start?

Start with one small, painful, repetitive task — not with a strategy document. Look at your week and find something you do over and over that annoys you: answering the same client questions, drafting the same reports, matching invoices. That is your first pilot.

Then follow a fixed plan. Chapter 38 lays out ninety days: observe the work, choose one small test, run it with a human check on everything,

and measure the result. Chapter 18 explains the pilot method itself. To rank which task is worth doing first, use the impact-and-ease thinking in Chapter 12.

The key is to go small and reversible. One process, a few people, a short test. A small win builds confidence and teaches you how to run the next one. A big first project is how good money gets wasted. Pick the small win today.

## How long does it take?

A first pilot takes about ninety days, and that is by design. Long enough to learn something real and see a result; short enough to keep urgency. The four phases — observe, choose, run, measure — are set out week by week in Chapter 38.

You can see a first useful result in a few weeks. Setting up a drafting tool and training one or two people can show a time saving inside a month. Proving it is worth spreading takes the full quarter, because you need a baseline, a real run, and an honest measurement.

Do not expect the new way to feel natural in three weeks. The “21 days to form a habit” idea is a myth; real habits take closer to two months, as Chapter 38 explains. Plan for the real curve. After the first pilot, each new one goes faster because you already know the method.

## Which tools should I choose?

Choose the task first, the tool last. The most common mistake is falling for a shiny product and then hunting for a place to use it. Start from the problem you defined, then pick the tool that fits it. Chapter 17 covers how to choose without being fooled.

A few plain rules. Prefer tools that fit what you already use over ones that demand a new system. Prefer a tool that keeps a human in the loop over one that acts alone. Prefer clear pricing over hidden costs. Try before you buy with a small pilot, not a company-wide rollout.

There is no single best tool. A drafting assistant, a chat agent, and a document summariser are different jobs. Match the tool to the task. And remember the tool is only half of it — the pairing of the tool with a trained person is what produces the result, as Chapter 37 explains.

## How do I choose a vendor?

Judge a vendor on honesty and fit, not on the demo. A polished demonstration tells you almost nothing; a clean input is not a test. Ask what the tool did when it did not know the answer, and how many real cases it has handled. Chapter 1 explains why the demo-to-production gap is normal and how to probe it.

Ask direct questions. Where does my data go, and who can see it? Can I export my data and leave easily? What happens when the AI is wrong — is there a human check? What is the total cost, including setup and training, not just the licence? Who is responsible if it makes a mistake?

Watch for red flags: pressure to sign now, vague answers about data, claims of “fully automatic” with no human review, and prices that hide the real cost. A good vendor welcomes these questions. A bad one dodges them. Start small and reversible so a wrong choice costs you a pilot, not the company.

## Is it safe?

It can be, if you set it up with care. AI is not safe or unsafe in itself. Safety depends on how you use it. The main risks are wrong answers that reach a customer unchecked, data sent somewhere you did not intend, and over-trust because the output sounds confident. Each has a plain defence.

The core defence is a human check on anything that matters. Never let AI output reach a customer, a payment, or a legal decision without a person approving it. Keep personal data under control and know where it goes. Treat every output as a draft to verify, not a finished answer. Chapter 20 covers secure setup, and Chapter 6 covers the security threats to watch for.

Remember that fluent output is not true output. A system that writes well can still be wrong, and it will not always tell you. The safety sits with you, the human who checks. Do that, and AI is about as safe as any other tool you already use.

## What does the law say?

The law is real and it is tightening, but it is manageable if you follow a few principles. In the European Union, the AI Act sets rules by risk level: some uses are banned, some carry heavy duties, and most everyday business uses fall in a light category. Chapter 5 is the canonical home for the AI Act.

If you handle personal data of people in the EU, the GDPR applies, and it applies whether or not AI is involved. You need a lawful basis, you must tell people what you do with their data, and you must protect it. Chapter 10 covers GDPR in plain terms.

The practical rule: know what data you handle, be transparent about how you use it, and keep a human responsible for decisions that affect people. Where AI makes a decision about a person — hiring, credit, service — extra care and often extra disclosure apply. For a small business, the safest path is to use AI for drafts and support, keep humans deciding, and follow the privacy rules you already follow for other data.

## Can I use AI without giving my data to anyone?

Yes, by running the AI on your own computers. This is called **self-hosting**: instead of sending your data to a company's servers over the internet, you run the model on hardware you control, so the data never leaves your machine. Chapter 8 is the canonical home for self-hosting and tools like Ollama that make it practical.

Self-hosting gives you the strongest control. Your data stays home. Nothing is shared, sold, or used to train someone else's model. For sensitive work — legal, medical, financial, or anything you cannot let leave the building — this is the most protective option.

The trade-off is cost and effort. Self-hosting needs capable hardware, setup, and some technical help, and the models you can run yourself may be smaller than the biggest online ones. For many small businesses, a controlled cloud tool with the right settings is enough. For data that truly cannot leave, self-hosting is the answer. Weigh what you must protect against what you can spend.

## Is self-hosting really safer?

It is safer for one specific thing: keeping your data under your control. Self-hosting removes the risk that your data is sent out, stored

elsewhere, or used to train a vendor's model. If your main worry is data leaving the building, self-hosting directly solves it. See Chapter 8.

But "safer" is not total. Self-hosting moves the security job onto you. Now you are responsible for patching, access control, backups, and keeping the machine safe. A badly secured self-hosted system can be less safe than a well-run cloud service. You have traded the vendor's security for your own.

Also, self-hosting does not fix wrong answers. A model on your own machine can still produce a confident mistake. You still need a human check. So self-hosting is the right choice when data control is the priority and you can handle the security work. It is not a magic safety switch. The honest comparison is in Chapter 8 and Chapter 11.

## What does "trustless" mean, and do I really need it?

"Trustless" does not mean trusting no one or being suspicious. It means a system designed so you do not have to blindly trust the other party — because the design itself protects you. The idea comes from secure systems where you verify, rather than assume. Chapter 7 is the canonical home for the concept.

In everyday business terms, a trustless approach means you keep proof, you check rather than assume, and you set things up so a mistake or a dishonest partner cannot silently harm you. For AI, it looks like: keep logs of what the system did, verify outputs instead of trusting them, use tools that let you see and control your data, and avoid setups where you must simply believe the vendor.

Do you "need" it? You need the spirit of it, yes. Blind trust in any tool or vendor is risky. But you do not need to build a fortress. The practical

version is simple: verify, keep records, control your data, and keep a human accountable. That is trustless enough for a small business, and it costs very little.

# Appendix A — Simple Glossary

Plain-English meanings of the words you will meet in this book. Each entry is short on purpose. If a term is missing, look for the closest word here first.

**AI agent** — Software that does a small job for you on its own, like reading an email, deciding what to do, and taking the next step. Unlike a simple chatbot, it can act across several steps without you guiding each one.

**Algorithm** — A fixed set of steps that solves a problem, like a recipe. A computer follows the steps in order to get a result.

**Anonymization** — Removing everything that can identify a person from data, so no one can be traced back. Done well, the data can no longer be linked to a named person.

**API (Application Programming Interface)** — A connection that lets one piece of software talk to another. It is how an AI tool reads your customer list or writes into your accounting system.

**Artificial intelligence (AI)** — Software that does tasks that normally need human thinking, like reading text, spotting patterns, or answering questions. It is a tool, not a person.

**Bias** — A unfair tilt in a system's results, often because the data it learned from was skewed. For example, a hiring tool trained mostly on one group may favour that group.

**Chatbot** — A program you chat with by typing. It replies to messages. It may use AI, or it may follow simple scripted rules.

**Cloud** — Software and storage that run on someone else's computers over the internet, not on your own machine. You usually pay a monthly fee.

**Data poisoning** — Corrupting the data a system learns from, on purpose, so it behaves badly later. It is an attack on the data, not on the live system.

**Dataset** — A collection of data used to teach or test a system. Think of it as the study material.

**Digital sovereignty** — The ability to keep control of your own data and the systems that use it, instead of handing that control to an outside provider you cannot influence.

**DPIA (Data Protection Impact Assessment)** — A structured check you do before a project, to ask: what could harm people's privacy, and how do we reduce it? Under GDPR it is required when processing is high-risk.

**Encryption** — Scrambling data so that only someone with the right key can read it. It protects data while stored or while moving over the internet.

**EU AI Act** — European rules for AI. It sorts AI uses into risk levels and sets stricter duties on higher-risk ones, like hiring or credit tools.

**GDPR (General Data Protection Regulation)** — European privacy law. It sets rules for handling personal data: collect only what you need, keep it safe, and respect people's rights.

**Generative AI** — AI that creates new content — text, images, or audio — rather than only sorting or scoring. Most chat tools you hear about are generative AI.

**Inversion attack** — A way to try to recover private data from a system by working backwards from what it gives out. It is one reason to limit what a system reveals.

**Integration** — Connecting AI to the tools you already use, so data flows between them without copying by hand.

**KPI (Key Performance Indicator)** — A number you track to see if something is working, like response time or sales per week.

**Large language model (LLM)** — A type of AI trained on huge amounts of text so it can understand and produce human language. It is the engine behind many chat tools.

**Lock-in** — When it becomes hard or costly to leave a provider because your data, settings, or workflows are stuck in their system. Plan your exit before you commit.

**Logging** — Keeping a record of what a system did, and when. Good logs help you find problems and prove what happened.

**Low-code** — Tools that let you build simple automations with a bit of configuration instead of full programming. Some technical skill still helps.

**Machine learning (ML)** — A way to build AI by showing it many examples and letting it find patterns, instead of writing every rule by hand.

**Model** — The "learned thing" produced by training. It is the software that makes predictions or generates text after learning from data.

**No-code** — Tools that let you build automations without writing any code, usually by dragging and clicking.

**Pilot** — A small, limited trial of a new tool or process, run before you commit fully. Its goal is to learn cheaply whether it works.

**Prompt** — The instruction or question you give to an AI tool. A clearer prompt usually gives a better result.

**Prompt injection** — A trick where hidden text in a document or message fools an AI into ignoring your instructions and doing something else. Treat outside text as untrusted.

**Pseudonymization** — Replacing a name or ID with a code, so data is harder to link to a person, but can still be re-linked with a key. It is safer than plain data, but not the same as full anonymization.

**ROI (Return on Investment)** — How much value you get back compared to what you spent. A simple way to judge if a project paid off.

**RPA (Robotic Process Automation)** — Software that repeats a computer task by mimicking a person clicking and typing. Good for boring, rule-based work.

**Self-hosting** — Running software on your own computers, instead of on a provider's cloud. You keep more control, but you carry more work and risk.

**Shadow AI** — AI tools that staff use without the company's knowledge or approval. Convenient, but a hidden risk to data and compliance.

**Training** — The process of teaching a system by showing it many examples so it learns patterns. It happens before the tool is used in real work.

**Trustless** — A design where you do not have to blindly trust the other side. Control and checks replace trust. You verify instead of hoping.

**Virtual assistant** — An AI helper you talk to by voice or text, like a digital secretary. It can answer questions, set reminders, or draft messages.

# Appendix B — Checklist to Evaluate an AI Vendor

Use this before you sign with any AI vendor. Work through each theme. If a vendor cannot answer a question clearly, treat that as a warning. Write a **Yes / No / Unknown** next to each item. A long list of "Unknown" is itself an answer.

## 1. What the tool actually does

- ☐ The vendor can state, in plain words, the one problem it solves for me.
- ☐ It fits a real process I have, not a vague "it can do everything."
- ☐ I have seen it work on my kind of data, not only a polished demo.
- ☐ I know what it does **not** do, and where it fails.
- ☐ The output is something I can check for correctness.

## 2. Data and privacy

- ☐ I know exactly what data the tool collects and stores.
- ☐ I know where the data is stored (which country / region).
- ☐ My data is **not** used to train the vendor's shared model unless I agree in writing.
- ☐ I can delete my data and get proof of deletion.
- ☐ The vendor explains how it handles personal and sensitive data.
- ☐ Data is encrypted both while stored and while moving.

## 3. Security

- ☐ The tool supports strong login (multi-factor authentication).
- ☐ Access is controlled per person, so not everyone sees everything.
- ☐ There is a log of who did what, that I can review.
- ☐ The vendor has a written security policy I can read.
- ☐ The vendor tells me about breaches promptly (a clear notification promise).
- ☐ I know how they protect against common attacks (for example, prompt injection in tools that read outside text).

## 4. Compliance (GDPR and EU AI Act)

- ☐ The vendor can say how it complies with GDPR.
- ☐ If my use is higher-risk under the EU AI Act, the vendor supports the extra duties (records, human oversight, transparency).
- ☐ They offer a data-processing agreement (the standard contract GDPR asks for).
- ☐ They can support a DPIA if my project needs one.
- ☐ They tell me if they use any sub-processors (other companies handling my data).

## 5. Pricing

- ☐ The price is clear, with no hidden extras.
- ☐ I understand how cost grows if usage grows (per user? per task? per message?).
- ☐ I know what happens if I exceed my plan limits.
- ☐ I know the renewal price, not just the intro price.
- ☐ There is a cheaper tier that still meets my core need.

## 6. Support and service

- ☐ There is a real support channel (email, chat, phone) with stated hours.
- ☐ Response times are written down, not promised verbally.
- ☐ There is help to get started (onboarding, guides, training).
- ☐ The tool has a stated uptime commitment (a reliability promise).
- ☐ I can reach a human when it matters.

## 7. Contract and exit

- ☐ I can cancel without a heavy penalty.
- ☐ I can export all my data in a usable format at any time.
- ☐ I know how much notice is needed to cancel.
- ☐ The contract length is reasonable for a first commitment.
- ☐ I have thought about lock-in: how hard is it to leave later?
- ☐ I keep ownership of my own data and content.

## 8. Trust and reputation

- ☐ I have checked references or reviews from businesses like mine.
- ☐ The company has been operating long enough to be accountable.
- ☐ I know who to contact if something goes wrong.
- ☐ Their claims are believable, not exaggerated.

## 9. Trial and proof

- ☐ I can run a small trial before committing.
- ☐ I defined what “success” looks like for the trial.
- ☐ I tested it on my real data, not only sample data.
- ☐ I know the date I will decide: continue, change, or stop.

## Quick scorecard

Add up your answers:

| Result | Meaning |
|---|---|
| Mostly **Yes** | Good candidate. Move to a small pilot. |
| Several **No** on data, security, or exit | High risk. Ask more or walk away. |
| Many **Unknown** | The vendor is not transparent. Treat as a red flag. |

**Rule of thumb:** Never trade away control of your data for convenience. If the vendor will not answer plainly, that is your answer.

# Appendix C — Template to Map Your Processes

Before you automate anything, write down what you actually do. This template captures one row per process. Fill it for your top 5–10 repeated tasks. You cannot improve what you have not mapped.

## How to use it

1. List the tasks your team repeats every week.
2. For each one, fill a row below. Be honest about time and pain.
3. Mark **Automation potential**: High (boring, rule-based, high volume), Medium (partly automatic), Low (needs human judgement).
4. Start with High-potential, high-pain rows.

## Blank template

| Process name | Owner | Steps | Inputs | Outputs | Time per run | Frequency | Pain points | Automation potential |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

## Filled example

| Process name | Owner | Steps | Inputs | Outputs | Time per run | Frequency | Pain points | Automation potential |
|---|---|---|---|---|---|---|---|---|
| Process incoming invoices | Accounts clerk | 1. Open email 2. Save PDF 3. Read vendor, date, amount 4. Enter into accounting 5. File copy | Invoice PDFs in shared inbox | Updated ledger entry + filed copy | 12 min | 40×/week | Typos in amounts; lost emails; boring | **High** |
| Answer routine customer | | | | | | | | |

## Column guide

- **Process name** — A short, clear label anyone would understand.
- **Owner** — The single person responsible for this process today.
- **Steps** — Numbered actions, in order. Keep each step to one action.
- **Inputs** — What goes in: documents, emails, forms, data.
- **Outputs** — What comes out: a record, a reply, a file, a decision.
- **Time per run** — Real minutes it takes, not what you wish it took.
- **Frequency** — How often it happens per day or week.
- **Pain points** — What goes wrong, what is slow, what people dislike.
- **Automation potential** — High / Medium / Low, with a reason if not obvious.

## Tips for good mapping

- Map the process **as it really is**, not as the manual says.
- Ask the person who does the work, not the person who manages it.
- Note every "copy-paste" step — those are prime automation targets.
- If a step needs trust or judgement, mark it for human review.
- Total the time: `time per run × frequency` shows where the hours go each week.

## High-potential processes to look for

These repeat often and follow clear rules — good first targets:

- Reading and sorting incoming email or documents.
- Copying data from one system to another.
- Answering the same customer questions over and over.
- Drafting standard replies, quotes, or reports.

- Checking documents for missing or wrong fields.
- Summarising long notes, calls, or files.

Avoid automating first: anything that needs trust, negotiation, or a final judgement about a person.

## Quick priority grid

Sort your rows to pick the first thing to automate.

| | **Low pain** | **High pain** |
|---|---|---|
| **High potential** | Good later | **Start here** |
| **Low potential** | Ignore | Fix the process first, not with AI |

Pick one “Start here” row for your first pilot. Keep it small.

## How to estimate time honestly

- Time the task once or twice for real; do not guess from memory.
- Include the hidden minutes: finding the file, switching screens, fixing mistakes.
- Ask two people who do the task and compare their numbers.
- Use a normal week, not a quiet one.

## Using the map

- The map shows where the hours go each week — that is your target list.
- A process with high time and high pain is worth automating first.
- If a process is painful but low volume, fix the cause before adding tools.

- Keep the map alive: update it when a process changes. It is a working document, not a one-off.

# Appendix D — Data Inventory

You cannot protect data you cannot find. This inventory lists every important data item your business holds, where it lives, and who can reach it. Fill it before you connect any AI tool. If data is not on this list, do not feed it to AI.

## How to use it

1. Walk through your tools: email, drives, accounting, CRM, HR systems, spreadsheets.
2. Add one row per data item (a customer list, a payroll file, a contract folder).
3. Mark each item **Personal** (relates to a person) or **Sensitive** (special categories — see below).
4. Check access: who can open it today, and who *should* be able to.
5. Review before any AI project. This list tells you what is safe to use and what is not.

## Blank template

| Data item | Where stored | Owner | Personal / Sensitive? | Who can access | Backup status | Retention |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

## Filled example

| Data item | Where stored | Owner | Personal / Sensitive? | Who can access | Backup status | Retention |
|---|---|---|---|---|---|---|
| Customer contact list | CRM (cloud) | Sales manager | Personal (names, emails, phones) | Sales team (edit), Finance (read) | Daily, off-site | Keep while active + 3 years |
| Payroll file | Shared drive, folder “HR” | HR lead | Sensitive (salary, ID, bank) | HR only | Weekly, off-site | 10 years (legal) |
| Supplier contracts | Paper cabinet + scanned folder | Owner | Neither (business-confidential) | Owner, Finance | Partial (scans only) | Life of contract + 6 years |

## Where to look for data

Walk through each of these. Data hides in plain sight:

- ☐ Shared email inboxes and personal mailboxes.
- ☐ Shared drives and network folders.
- ☐ Cloud tools: CRM, accounting, HR, project tools.
- ☐ Spreadsheets and attachments (often the messiest).
- ☐ Paper files, cabinets, and scanned copies.
- ☐ Backups and old exports.

- [ ] Chat tools and messaging history.
- [ ] Phones and laptops of staff.
- [ ] Third-party tools staff signed up for on their own (shadow copies).

## What counts as “Sensitive”

Under GDPR, these special categories need extra care. Mark them clearly:

- Health data
- Racial or ethnic origin
- Political opinions
- Religious or philosophical beliefs
- Trade-union membership
- Genetic and biometric data (for identification)
- Sex life or sexual orientation

Also treat as sensitive even if not “special category”: bank details, ID numbers, passwords, children’s data, and anything that could harm a person if leaked.

## Column guide

- **Data item** — A plain name for the thing (not the file name).
- **Where stored** — System and location: cloud tool, drive path, paper cabinet.
- **Owner** — One person accountable for it.
- **Personal / Sensitive?** — Personal, Sensitive, or Neither.
- **Who can access** — Roles or people, and whether they can read or edit.
- **Backup status** — How often it is backed up, and where. Note if there is **no** backup.

- **Retention** — How long you keep it, and why (legal rule or business need).

## Red flags to fix now

- ☐ Sensitive data with no owner.
- ☐ Sensitive data anyone in the company can open.
- ☐ Important data with no backup.
- ☐ Data kept “forever” with no reason.
- ☐ Personal data in a tool you do not control (shadow AI, personal email).
- ☐ The same data in many places with no master copy.

## Retention rules of thumb

- Keep data only as long as you need it. Longer is not safer — it is more risk.
- Check legal minimums: tax and payroll records often have fixed years. Ask your accountant.
- For customer data, keep it while the relationship is active, then a short, stated period.
- Delete or anonymize data you no longer need. Do not let it pile up.
- Write the reason for each retention period, so no one has to guess later.

## Keeping the inventory current

- Review it every 6 months, and before any new AI project.
- Add a row the day a new tool or data source appears.
- Name one owner per item; owners keep it honest.
- Treat it as a living document, not a one-time chore.

## Rule for AI projects

Before any data goes into an AI tool, check this inventory. If an item is **Sensitive**, you need a clear reason, a safe tool, and often a DPIA. When unsure, leave it out.

# Appendix E — Simple ROI Calculation

ROI tells you, in one number, whether a project paid for itself. You do not need a finance degree. This appendix gives the formula in plain words, a worked example, and a blank worksheet.

## The formula in plain words

> **ROI (%) = (Benefit − Cost) ÷ Cost × 100**

- **Benefit** = the value you gain, in money, over a period (usually one year).
- **Cost** = everything you spend, over the same period.
- A **positive** ROI means you gained. A **negative** ROI means you lost.

A ROI of 100% means you got back double what you spent. A ROI of 50% means you got back half again on top of your cost.

## What counts as Benefit

Put a money value on each gain you can defend:

- **Time saved** — hours saved × the hourly cost of the person doing it.
- **Errors avoided** — cost of rework, refunds, or fines you no longer pay.
- **Extra sales** — new revenue the change brings.
- **Faster response** — value of happier, retained customers (estimate carefully).

Only count gains you can explain. Do not pad the number.

## What counts as Cost

Add it all up:

- Software subscriptions (per year).
- Setup and integration work.
- Training time for staff.
- Ongoing human review of the AI's output.
- Any extra hardware or storage.

## Worked example

A small firm automates reading incoming invoices.

**Cost (per year):** - Software: €1,200 - Setup: €800 - Training: €400 - Human review (2 hrs/week × €25): €2,600 - **Total cost = €5,000**

**Benefit (per year):** - 40 invoices/week × 12 min saved = 8 hours/week saved - 8 hrs × 50 weeks × €25/hr = **€10,000** in time saved - Fewer errors (estimated): €1,000 - **Total benefit = €11,000**

**ROI = (11,000 − 5,000) ÷ 5,000 × 100 = 120%**

The project returns €1.20 of value for every €1 spent, on top of getting the cost back.

## Payback period

How long until gains cover the cost?

> **Payback (months) = Total cost ÷ (Benefit per year ÷ 12)**

Using the example: €5,000 ÷ (€11,000 ÷ 12) = €5,000 ÷ €917 ≈ **5.5 months**.

A short payback makes a project easy to approve.

## Second example (quick)

A support team uses AI to draft replies to common questions.

- **Cost/year:** software €2,400 + training €600 + review €3,000 = **€6,000**
- **Benefit/year:** 60 emails/week × 4 min saved = 4 hrs/week × 50 × €25 = **€5,000**, plus faster response value €2,000 = **€7,000**
- **ROI = (7,000 − 6,000) ÷ 6,000 × 100 ≈ 17%** — thin. The tool only pays off if review time drops or volume rises.

This shows why you must include the cost of human review. Skip it, and the ROI looks far better than it is.

## Non-financial benefits

Some gains are real but hard to price. Note them separately; do not force them into the number:

- Less boring work, happier staff.
- Faster response, better customer feeling.
- Fewer late nights and less stress.
- Consistency in how work is done.

Mention these in your case, but keep the ROI number honest.

## Blank worksheet

**Project name: _____ Period: _____**

**Costs**

| Item | Amount (€) |
|---|---|
| Software / subscription | |
| Setup / integration | |
| Training | |
| Ongoing human review | |
| Other | |
| **Total cost** | |

**Benefits**

| Item | How you calculated it | Amount (€) |
|---|---|---|
| Time saved | hours × hourly cost | |
| Errors avoided | | |
| Extra sales | | |
| Other | | |
| **Total benefit** | | |

**Result**

| Measure | Value |
|---|---|
| ROI (%) = (Benefit – Cost) ÷ Cost × 100 | |
| Payback (months) = Cost ÷ (Benefit ÷ 12) | |

## Honest tips

- Be conservative. If you are unsure of a benefit, lower it or drop it.
- Always include the cost of human review — AI still needs checking.
- A small positive ROI on a low-risk pilot is a good result.
- If ROI is negative, the idea may still be worth it for quality or compliance — say so plainly.
- Re-measure after the pilot with real numbers, not the estimates.

# Appendix F — Internal Communication Plan

Most AI projects fail on people, not technology. If staff hear about a change late, or not at all, they resist it. This plan makes sure the right people hear the right message, from the right person, at the right time.

## Why it matters

- People fear what they do not understand. Clear messages reduce fear.
- Staff who feel informed become supporters, not obstacles.
- Good communication prevents shadow AI — people using hidden tools because the official path is unclear.

## The plan table

Fill one row per audience. Keep messages short and honest.

| Audience | Message | Channel | Owner | Timing | Frequency |
|---|---|---|---|---|---|
| All staff | | | | | |
| Managers | | | | | |
| Affected team | | | | | |
| IT / security | | | | | |
| Customers (if touched) | | | | | |

## Column guide

- **Audience** — Who needs to know. Be specific.
- **Message** — The core point, in one or two sentences.
- **Channel** — Meeting, email, chat, intranet, one-to-one.
- **Owner** — Who delivers it. Sensitive news comes from a person, not a memo.
- **Timing** — When it goes out. Tell people before, not after.
- **Frequency** — Once, weekly, at each milestone.

## Filled example

| Audience | Message | Channel | Owner | Timing | Frequency |
|---|---|---|---|---|---|
| All staff | We are using AI to cut boring work; no job losses; training coming | Team meeting | Owner | 1 week before start | Once, then at each milestone |
| Managers | Your team uses [tool] from [date]; collect feedback weekly | Email | Ops lead | 1 week before | Weekly |
| Affected team | New way to do [task]; training on [date]; you still check output | Small group | Team manager | 3 days before | Once, then daily in week 1 |

## Suggested timing for a change

1. **Before** — Announce the change and the reason. Invite questions.
2. **At start** — Explain what changes day-to-day and what support exists.
3. **During** — Share early wins and fix worries quickly.
4. **After** — Report results and next steps.

## Example messages

**To all staff (announcement, from the owner, at a team meeting):** > "We are starting to use an AI tool to help with [task]. It handles the boring, repetitive part so you can focus on the work that needs your judgement. No one is being replaced by this. We will train everyone and keep a human in charge of every important decision. Questions welcome, any time."

**To managers (email, before rollout):** > "From [date], your team will use [tool] for [task]. Your job: encourage use, watch for problems, and send feedback to [name] weekly. Please do not let anyone paste customer personal data into unapproved tools. Use only the approved one."

**To the affected team (one-to-one or small group):** > "This changes how you do [task]. Instead of [old way], you will [new way]. It should save about [X] hours a week. Here is the training on [date]. If the tool gets something wrong, flag it — you are still the one who checks."

**To IT / security (email, early):** > "We plan to use [tool] for [process]. Please review it against our security and data rules before we

start. Check the vendor checklist and the data inventory. Tell us what must be locked down."

**To customers (only if the change touches them):** > "We now use AI to respond faster to your requests. A real person still reviews important answers. If you prefer to speak to a person, just ask."

## Rules for good messages

- ☐ Say **why** before **what**.
- ☐ Be honest about what changes and what does not.
- ☐ Never hide a change and spring it on people.
- ☐ Give a clear place to ask questions.
- ☐ Repeat the message; one email is not enough.
- ☐ Celebrate early wins publicly to build trust.
- ☐ Listen and act on feedback — silence kills adoption.

## What not to say

- Do not promise the tool is perfect. It is not.
- Do not say "this changes nothing" if it does.
- Do not blame the tool when it errs; own the process.
- Do not announce layoffs disguised as "efficiency." Be direct if roles will change, and how you will help.
- Do not spring a change on people mid-week with no training.

## Quick fill-in starter

- **Change:** _____
- **Reason (say this first):** _____
- **Who is most affected:** _____
- **Who announces it:** _____

- **Date of first announcement: _____**
- **Where people ask questions: _____**

# Appendix G — A Simple AI Policy for Your Company

A short policy you can adapt and publish today. It tells staff what they may and may not do with AI, in plain words. Keep it to one page. Replace the bracketed text with your details.

---

**[Company Name] — AI Use Policy Version 1.0 — Effective [date] — Owner: [name]**

## 1. Purpose

This policy sets simple, clear rules for using AI tools at work. It protects our customers, our data, and our reputation. It applies to everyone: employees, contractors, and anyone working for us.

## 2. Allowed uses

You may use approved AI tools to:

- Draft, summarise, or rewrite text (emails, reports, notes).
- Brainstorm ideas and options.
- Automate boring, repetitive tasks that follow clear rules.
- Help translate or clean up data.
- Speed up work that a human still checks before use.

Only use tools on the **Approved Tools List** (maintained by [name]).

## 3. Forbidden uses

You must **not**:

- Put customer personal or sensitive data into an unapproved AI tool.
- Paste confidential company data (financials, contracts, strategy) into public AI tools.
- Let AI make a final decision about a person (hiring, firing, credit, pricing) without a human deciding.
- Present AI output as your own finished work where accuracy matters, without checking it.
- Use AI to deceive, mislead, or impersonate a real person.
- Hide your use of AI when it matters that people know.

## 4. Data rules

- Check the **Data Inventory** before feeding data to any AI tool.
- **Sensitive data** (health, ID, bank, beliefs, children's data) never goes into an unapproved tool.
- Prefer tools that promise **not** to train their model on your data.
- Share the least data needed. Remove names and details if they are not required.
- When unsure whether data is safe to use, **do not use it** — ask first.

## 5. Disclosure

- If a customer interacts with AI (for example, a chatbot), tell them plainly.
- If AI wrote something important that a person relies on, a named human must review and own it.
- Follow any legal duty to disclose automated decisions.

## 6. Human review

- A human is responsible for every important output. AI assists; it does not decide.
- For anything that affects a person's rights, money, or safety, a human must check and approve.
- If you cannot check whether the AI is right, do not use its answer.

## 7. Third-party and shadow AI

- Use only tools on the Approved Tools List.
- Do not use personal AI accounts for company work.
- If you find a useful tool not on the list, request it — do not start using it silently.
- Using unapproved tools with company data is a policy breach.

## 8. If something goes wrong

- Report any mistake, leak, or odd AI behaviour to [name / email] the same day.
- Do not delete evidence. Save what happened.
- Report suspected prompt injection or suspicious instructions found inside documents.

## 9. Roles and responsibilities

- **Everyone** — Follow this policy. Ask when unsure.
- **Managers** — Make sure their team uses only approved tools and reviews output.
- **Policy owner ([name])** — Keeps the policy current, approves tools, answers questions.

- **IT / security ([name])** — Checks tools for safety, controls access, monitors incidents.

## 10. Exceptions

If you need to do something this policy does not allow, ask the policy owner first. Exceptions must be agreed in writing and are limited to a specific task and time. Do not assume an exception applies to you without it being granted.

## 11. Who to ask

- **Policy owner:** [name], [email]
- **Security / data questions:** [name], [email]
- **Tool approval requests:** [name], [email]

When in doubt, ask before you act. Asking is always the safe choice.

## 12. Acknowledgement

| Name | Role | Signed | Date |
|---|---|---|---|
| | | | |
| | | | |

---

**Review this policy every [6 / 12] months, or when a new tool or rule appears.**

# Appendix H — AI Risk Assessment

Before you launch an AI project, ask what could go wrong and how bad it would be. This table makes risks visible so you can reduce them, not ignore them. Fill it with your team. Do it early, and revisit it often.

## How to score

Rate each risk on two scales:

- **Likelihood** — How likely is it? **Low / Medium / High**
- **Impact** — How bad if it happens? **Low / Medium / High**

Combine them into a **Risk level**:

| | **Impact Low** | **Impact Medium** | **Impact High** |
|---|---|---|---|
| **Likelihood High** | Medium | High | **Critical** |
| **Likelihood Medium** | Low | Medium | High |
| **Likelihood Low** | Low | Low | Medium |

Treat **Critical** and **High** as must-fix before launch.

## Blank template

| Risk | What could go wrong | Likelihood | Impact | Risk level | Mitigation | Owner |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

## Filled examples

| Risk | What could go wrong | Likelihood | Impact | Risk level | Mitigation | Owner |
|---|---|---|---|---|---|---|
| Wrong data into AI | Staff paste customer personal data into a public tool | High | High | **Critical** | Approved-tools-only list; block public tools on company devices; training | IT lead |
| Wrong output used | AI drafts an invoice with a wrong amount, sent to customer | Medium | High | High | Human review before sending; auto-check totals | Accounts clerk |
| Bias in decisions | Hiring tool favours one | | | | | |

## Column guide

- **Risk** — A short name for the risk.
- **What could go wrong** — The concrete scenario, not a vague worry.
- **Likelihood / Impact** — Low / Medium / High.
- **Risk level** — From the grid above.
- **Mitigation** — What you will do to lower the likelihood or the impact.
- **Owner** — The one person who makes sure the mitigation happens.

## Common AI risks to consider

Use this list to spark your own rows:

- ☐ Personal or sensitive data leaked to an outside tool.
- ☐ Incorrect output trusted without review.
- ☐ Bias or unfairness in automated decisions.
- ☐ Vendor security breach.
- ☐ Prompt injection from untrusted documents.
- ☐ Shadow AI (staff using unapproved tools).
- ☐ Over-reliance and skill loss.
- ☐ Cost overrun as usage grows.
- ☐ Lock-in to one provider.
- ☐ Non-compliance with GDPR or the EU AI Act.
- ☐ Customer trust lost after a visible mistake.
- ☐ Model changes silently and output quality drops.

## When to re-run this assessment

Revisit the table when any of these happen:

- You add a new tool, data source, or process.
- Usage grows a lot, or the tool touches more people.
- An incident or near-miss occurs.
- A law or customer requirement changes.
- A vendor changes its terms, model, or where it stores data.

## Two ways to reduce a risk

Every mitigation pulls on one of two levers:

- **Lower the likelihood** — make the bad event less likely to happen. Example: approved-tools-only list makes a data leak less likely.
- **Lower the impact** — make the damage smaller if it happens. Example: human review means a wrong invoice is caught before it reaches the customer.

A good plan often pulls both. Ask for each risk: "What makes this less likely, and what limits the harm if it still happens?"

If a risk stays **Critical** even after mitigation, do not launch. Remove the risky step or redesign the process.

## Rules

- Every Critical or High risk needs a mitigation **and** an owner before launch.
- A mitigation with no owner is not a mitigation.
- Re-run this assessment at each milestone — risks change.
- If a Critical risk cannot be reduced, do not launch. Say so.

# Appendix I — Pilot Project Template

A pilot is a small, cheap trial that answers one question: does this work for us? Keep it narrow, time-boxed, and honest. This one-page plan holds everything you need. Fill it before you start, and judge the result against it at the end.

## How to use it

1. Pick **one** process (from your process map) with high potential and high pain.
2. Fill every field below. If you cannot fill it, the pilot is not ready.
3. Set a fixed end date and a clear decision rule.
4. Run it, measure, then decide: scale, change, or stop.

---

## Pilot plan

**Pilot name:** _____ **Date started:** _____
**End date:** _____

### 1. Goal (one sentence)

What single problem does this pilot solve?

> Example: “Cut the time to process incoming invoices from 12 minutes to under 4 minutes, with no more errors.”

Your goal:

_____
_____

## 2. Process chosen

- **Process:** _____ (link to your process map row)
- **Why this one:** _____
- **Owner:** _____

## 3. Before (baseline)

Measure the current state **before** you start. Without a baseline, you cannot prove a gain.

| Measure | Current value |
|---|---|
| Time per run | |
| Runs per week | |
| Error rate | |
| Cost per run | |
| Staff satisfaction (1–5) | |

## 4. After (target)

| Measure | Target value |
|---|---|
| Time per run | |
| Runs per week | |
| Error rate | |
| Cost per run | |
| Staff satisfaction (1–5) | |

## 5. Success criteria (how we decide)

Write 2–4 clear, checkable conditions. Be specific.

- ☐ Example: Time per run is under 4 minutes.
- ☐ Example: Error rate is no worse than before.
- ☐ Example: Staff say it is easier (score ≥ 4).
- ☐ Example: No data or security incidents.

## 6. Tools

- **AI tool:** _____
- **Connected systems:** _____
- **Vendor / plan:** _____
- **Cost during pilot:** _____

## 7. Timeline

| Week | What happens |
|---|---|
| 1 | Set up, connect, baseline confirmed |
| 2–3 | Run on real work, human review every output |
| 4 | Measure, gather feedback |
| 5 | Decide |

## 8. Team

| Role | Person | Responsibility |
|---|---|---|
| Pilot owner | | Runs the pilot, reports result |
| Doer | | Uses the tool daily, gives feedback |
| Reviewer | | Checks output for errors |
| IT / security | | Sets up safely, monitors risk |

## 9. Risks (top 3, from your risk assessment)

| Risk | Mitigation | Owner |
|---|---|---|
| | | |
| | | |
| | | |

## 10. Decision rule

Agree this **before** you start, so the choice is not emotional later.

- **Scale up** if: all success criteria are met and no Critical/High risk is open.
- **Change and retry** if: some criteria are met and the fix is clear. Set a new short pilot.
- **Stop** if: the goal is not met, cost outweighs benefit, or a serious risk appears.

**Decision made on:** _____ **Decision:** ☐ Scale ☐ Retry ☐ Stop
**Reason:**

_____
_____

---

# Example snapshot (a finished pilot)

- **Goal:** Cut invoice processing from 12 to under 4 minutes, no extra errors.
- **Process:** Process incoming invoices (from process map).
- **Before:** 12 min/run, 40/week, 6% error rate.
- **After (target):** 3 min/run, 40/week, ≤6% error rate.
- **Success met?** Time 3.1 min ✓, errors 5% ✓, staff score 4 ✓, no incidents ✓.
- **Cost during pilot:** €600.
- **Decision:** Scale — all criteria met, no open High risk.
- **Next:** Roll out to all invoices; keep human review on amounts over €5,000.

## What to do after the pilot

- **If you scale:** plan the wider rollout, keep review where risk is high, set a review date.
- **If you retry:** write down what failed, change one thing, run a short new pilot.
- **If you stop:** record why, so the lesson is kept and not repeated.
- Share the result with the team either way. A failed pilot that taught something is a good use of a small budget.

## Pilot rules

- Keep it small: one process, one team, a few weeks.
- A human reviews every output during the pilot.
- Measure the baseline first — never skip this.
- A pilot that fails cheaply is a success: you learned without a big loss.
- Write the result down and share it, good or bad.

# Appendix J — KPI Dashboard

A dashboard turns your AI projects into numbers you can watch. Numbers tell you if a change is working, stalling, or costing too much. Fill this in monthly. Keep it on one page. Share it with the team.

**KPI** means "Key Performance Indicator" — a number that shows how well something is going. Pick a few. Do not track everything.

## How to use this

1. Pick 5–8 KPIs that matter to your business.
2. Set a **target** for each (what "good" looks like).
3. Record the **current value** each period.
4. Name one **owner** per KPI — the person who keeps it accurate.
5. Review monthly. A KPI with no owner drifts and dies.

## Blank dashboard

| KPI | What it measures | Target | Current value | How often | Owner |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

## Common automation KPIs

Use these to fill your rows. Each one is explained in plain words.

- **Time saved** — Hours the team no longer spends on a task. Measure: (old time – new time) × number of tasks.
- **Error rate** — How often the work comes out wrong. Measure: errors ÷ total tasks × 100. Lower is better.
- **Cost per task** — What one task costs now, all in. Measure: total cost ÷ number of tasks.
- **Customer satisfaction** — How happy customers are with the result. Measure: survey score, star rating, or simple 1–5.
- **Adoption rate** — How many staff actually use the tool. Measure: active users ÷ total users who should.

- **Response time** — How fast you reply or deliver. Measure: average time from request to answer.
- **Rework rate** — How often AI output must be redone by a human. Measure: redone tasks ÷ total tasks × 100.
- **Uptime / reliability** — How often the tool is available when needed. Measure: working hours ÷ total hours × 100.
- **Payback progress** — How close you are to earning back what you spent. See Appendix E.

## Example filled dashboard

| KPI | What it measures | Target | Current value | How often | Owner |
|---|---|---|---|---|---|
| Time saved (invoices) | Hours not spent keying invoices | 30 hrs/month | 22 hrs/month | Monthly | Accounts clerk |
| Error rate (invoices) | Wrong invoice fields per 100 | < 2% | 3.5% | Weekly | Accounts clerk |
| Cost per task (support reply) | All-in cost to answer one ticket | < €1.50 | €1.20 | Monthly | Support lead |
| Customer satisfaction | Rating on resolved tickets | ≥ 4.5 / 5 | 4.2 / 5 | Monthly | Support lead |
| Adoption rate | Staff using the tool weekly | ≥ 80% | 61% | Monthly | Team manager |
| Rework rate | AI drafts needing a full redo | < 10% | 14% | Weekly | Team manager |
| Response time | Hours to first reply | < 4 hrs | 6 hrs | Weekly | Support lead |

## Reading the dashboard

- **Green** = at or better than target. Keep going.
- **Amber** = close but drifting. Watch it, adjust next period.
- **Red** = far off target. Investigate now or change the plan.

In the example above, time saved and cost look good. But error rate, adoption, and rework are red. That tells the owner: the tool works, but people are not using it well, and too many drafts need fixing. The fix is training and a review step, not a new tool.

## Rules for honest KPIs

- Measure the **real** value, not the hoped-for value.
- Use the **same method** every period, or the trend is meaningless.
- A KPI you never look at is a wasted KPI. Keep the list short.
- Tie every KPI to a decision. Ask: “What will I do differently if this number moves?”
- Review as a team. Numbers hide arguments; talking uncovers them.

## Quick monthly review sheet

**Period: _____**

| KPI | Target | This period | On track? (Y/N) | Action if off track |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing to fix this month:**

_____

**One thing working well to keep:**

_____

# Appendix K — Questions to Ask the Vendor

Use this in a live call or by email. These are pointed questions, not polite ones. Each has a short note on **why it matters**, so you know what a good answer sounds like. Write the vendor's answer down. Vague answers are information too.

**Tip:** Send the questions before the meeting. A vendor who answers plainly in writing is usually a vendor who operates plainly.

## Data & privacy

- **Exactly what data do you collect from us, and what do you store?** — You cannot protect what you do not know they hold.
- **Where is our data stored — which country or region?** — Location decides which privacy laws apply and who can demand access.
- **Is our data used to train your shared model?** — If yes, your private data could shape answers other customers see. Get a clear "no" or a written opt-out.
- **Can we delete our data, and do you prove it was deleted?** — "We delete it" without proof is a promise, not a fact.
- **Who else (sub-processors) touches our data?** — Every extra company is another place a leak can start.
- **Is our data encrypted both at rest and in transit?** — "At rest" = stored; "in transit" = moving. Both must be protected.

## Security

- **Do you support multi-factor login (MFA)?** — A password alone is the weakest door. MFA closes it.
- **Can we control access per person, so not everyone sees everything?** — Least access limits the damage one stolen account can do.
- **Do you keep an audit log of who did what, and can we read it?** — Without a log, you cannot trace a problem after the fact.
- **What is your breach-notification promise — how fast do you tell us?** — You need to know before your customers find out from the news.
- **How do you handle prompt injection, where a hidden instruction fools the AI?** — Tools that read outside text can be tricked; ask how they guard against it.
- **Do you run third-party security audits, and can we see the summary?** — An outside audit is stronger proof than a self-written promise.

## Pricing

- **What is the full price, and what is not included?** — Surface hidden costs now, not on the invoice.
- **How does cost grow as we grow — per user, per task, per message?** — A cheap plan can become expensive fast if it scales badly.
- **What happens if we exceed our plan limits?** — A surprise cap or overage fee can break a budget.
- **What is the renewal price, not just the intro price?** — Intro discounts end; the renewal is your real cost.

- **Is there a cheaper tier that still meets our core need?** — Do not pay for features you will never use.

## Integration

- **What systems do you connect to out of the box?** — Built-in links save time; missing ones cost custom work.
- **Do you have an API and webhooks for custom connections?** — An API means you can wire the tool to your own stack.
- **How long does setup usually take for a business like ours?** — A realistic timeline beats a sales promise.
- **Does it work with the tools we already use (email, CRM, files)?** — A tool that fights your workflow will be abandoned.
- **Who does the setup — you or us?** — Know the work and the cost before you sign.

## Support

- **What support channels exist, and what are the hours?** — Chat, email, or phone — and when a human is actually there.
- **What are your response-time commitments, in writing?** — "We try to reply fast" is not a commitment.
- **Is there onboarding or training to get staff started?** — Poor onboarding is the top reason tools go unused.
- **What uptime do you guarantee?** — An uptime promise (for example 99.9%) tells you how reliable they aim to be.
- **Can we reach a real human when it really matters?** — Bots are fine until something breaks badly.

## Contract & exit

- **How much notice do we need to cancel, and is there a penalty?** — A heavy exit fee is a trap door welded shut.
- **Can we export all our data, in a usable format, at any time?** — If you cannot leave with your data, you do not own it.
- **Who owns our content and data — us or you?** — You must keep ownership of your own work.
- **What is the contract length for a first commitment?** — Start short. Do not lock in for years on trust.
- **If we leave, in what format do we get our data back?** — A usable format (CSV, JSON) means you can move; a locked format means you cannot.

## Compliance

- **How do you comply with GDPR?** — If you handle EU personal data, this is not optional.
- **If our use is higher-risk under the EU AI Act, what extra support do you provide?** — Higher-risk uses need records, human oversight, and transparency.
- **Do you offer a data-processing agreement (DPA)?** — The standard contract GDPR requires between you and a vendor.
- **Can you support a DPIA if our project needs one?** — A DPIA is a formal privacy review some projects must do.
- **Do you tell us when your terms, model, or data location change?** — Silent changes can break your compliance without warning.

## How to read the answers

| Answer pattern | What it means |
|---|---|
| Clear, specific, in writing | Good sign. Move forward carefully. |
| Vague or “we’ll get back to us” | Weak. Push for a written answer. |
| Avoids data, security, or exit | Red flag. Walk away or escalate. |
| Over-promises, no proof | Distrust. Ask for audits or references. |

**Rule:** The vendor’s answers to the hard questions tell you more than their answers to the easy ones. Judge them on the awkward ones.

# Appendix L — Exercises and Fillable Modules

Short exercises to turn reading into doing. Do them with a pen or on a screen. Each takes 10–20 minutes. They follow the book's main themes. Do them in order, or pick the ones you need. There are no wrong answers — only blank pages.

## Exercise 1 — List your automatable tasks

**Prompt:** Write down every repeated task your team does. A task is a candidate for automation if it is done often, follows steps, and does not need deep judgment every time.

| Task | How often | Who does it | Follows fixed steps? (Y/N) | Needs human judgment? (Y/N) |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

**Rule of thumb:** High frequency + fixed steps + low judgment = best first candidate. High judgment tasks stay with people; let AI assist, not

decide.

**Your top 3 candidates:** 1) _____ 2) _____ 3) _____

## Exercise 2 — Map one process

**Prompt:** Pick one task from Exercise 1. Write out each step as it happens today. You cannot automate what you cannot draw.

**Process name:** _____

| Step # | What happens | Who / what does it | Time taken | Where it breaks or stalls |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |

**Where could AI fit?** Mark steps that are repetitive or slow. **Where must a human stay?** Mark steps with judgment, trust, or legal weight.

## Exercise 3 — Compute a simple ROI

**Prompt:** For your top candidate, estimate the yearly cost and the yearly benefit. Use the formula: **ROI (%) = (Benefit − Cost) ÷ Cost × 100**. (See Appendix E for a full worked example.)

**Costs (per year)**

| Item | Amount (€) |
|---|---|
| Software / subscription | |
| Setup / integration | |
| Training | |
| Ongoing human review | |
| **Total cost** | |

**Benefits (per year)**

| Item | How you calculated it | Amount (€) |
|---|---|---|
| Time saved | hours × hourly cost | |
| Errors avoided | | |
| Other | | |
| **Total benefit** | | |

**Result:** ROI = (_____ – _____) ÷ _____ × 100 = _____ %

**Payback (months)** = Total cost ÷ (Benefit ÷ 12) = _____ months

**Honest check:** Did you include the cost of human review? If not, add it. AI still needs checking.

## Exercise 4 — Draft one acceptable-use rule

**Prompt:** Write one clear rule about how your staff may use AI. A good rule is short, specific, and enforceable. Draft three.

| # | The rule (one sentence) | Why it matters | How we check it |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |

**Example to copy the style:** “Never paste a customer’s personal data into a public AI tool.” Why: it can leak private data. Check: approved-tools-only list on company devices.

(For a full policy, see Appendix Q.)

## Exercise 5 — Assess one risk

**Prompt:** For your chosen task, name what could go wrong. Rate each on Likelihood and Impact. Use the grid: High×High = Critical; Medium×High = High; Low×Low = Low. (Full grid in Appendix H.)

| Risk | What could go wrong | Likelihood | Impact | Risk level | Mitigation | Owner |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |
| | | | | | | |

**Rule:** Every Critical or High risk needs a mitigation **and** a named owner before you launch. A mitigation with no owner is not a mitigation.

## Exercise 6 — Spot the human checkpoint

**Prompt:** Look at your mapped process (Exercise 2). List the points where a human **must** review before the result goes out. These are your safety gates.

| Checkpoint | What is reviewed | Who reviews | What they reject if wrong |
|---|---|---|---|
| | | | |
| | | | |
| | | | |

**Ask:** "If the AI is wrong here, does a human catch it before a customer, a payment, or a legal step sees it?" If not, add a gate.

## Exercise 7 — Define what "success" looks like

**Prompt:** Before a pilot, write what success means in plain, checkable terms. No goal means no way to decide.

**Project: _____ Pilot ends: _____**

| Success measure | Target | How we will measure it |
|---|---|---|
| | | |
| | | |
| | | |

**Decision rule:** On the end date, we will **continue / change / stop** based on these numbers. Write the date now: _____

## Exercise 8 — One-page summary

**Prompt:** Fill this single card to carry your plan forward.

- **Task to automate:** _____
- **Why now:** _____
- **Top benefit (measured):** _____
- **Biggest risk + owner:** _____
- **Human checkpoint:** _____
- **First step this week:** _____

**Done?** You have just done, on paper, what most teams never do before they buy a tool. Start small, measure, and keep a human in the loop.

# Appendix M — Resources, Reading, and Recommended Tools

A short, honest list of tools and reading. These are **not** ads. They are real, named projects you can look up yourself. Most are open-source, which means you can see how they work and are not locked to one company.

**A note on skill level:** Some tools here need a technical person (an IT partner, a developer, or a tech-savvy hire). You do not need to run them yourself. Read them to know what is possible and to talk intelligently with whoever sets them up.

**Verified:** Every tool and book below was checked against its own website or code repository in **September 2026**. Tools change fast — re-check before you adopt.

## Run AI locally (on your own machines)

Running models locally means the AI works on your own computer or server. Your data does not leave your machine. This is the most private option.

- **Ollama** — `ollama.com`. A free, open-source tool to run open AI models on your own computer. Local models cost nothing and keep data on your machine. It is the easiest starting point for local AI and is often used as the engine behind other tools.
- **Thunderbolt** — `thunderbolt.io` (code: `github.com/Thunderbird/thunderbolt`). An open-source, self-hostable **AI client** from MZLA Technologies, the Mozilla subsidiary that also makes Thunderbird. Announced April 2026 and licensed

under the Mozilla Public License 2.0. Its promise is "AI You Control: choose your models, own your data, eliminate vendor lock-in." It runs on web, Windows, macOS, Linux, iOS, and Android, and works with local, on-premise, or cloud models. For local use it points to Ollama or llama.cpp. **Status:** early-stage and under a security audit — treat it as promising, not finished.
- **llama.cpp** — a well-known open-source engine that runs large models on ordinary computer hardware, including your own laptop. It is the technical engine others build on. Mentioned here because Thunderbolt recommends it for free local inference.

**When to choose local:** you handle sensitive data, want to avoid per-message fees, or need the AI to work without sending data out. The trade-off: you pay for hardware and setup, and local models are usually smaller than the biggest cloud models.

## Build automations (connect AI to your work)

These tools let you wire AI into real tasks — reading documents, answering tickets, moving data between apps.

- **Haystack** — `haystack.deepset.ai` (code: `github.com/deepset-ai/haystack`). An open-source framework from **deepset** (Germany) for building AI applications, especially **RAG** pipelines. RAG ("retrieval-augmented generation") means the AI looks up your own documents before answering, so it answers from your facts, not guesses. Free to install (`pip install haystack-ai`); paid enterprise support is optional. Best for a team with a developer.
- **n8n** — `n8n.io`. A **fair-code** automation platform (source is public on GitHub) that lets you build workflows on a visual canvas and connect to 500+ apps. It also builds AI agents and RAG systems, with human-in-the-loop approvals. You can self-host it or use their cloud. "Fair-code" means the code is open to read and self-host, but

not a standard open-source license — check the terms if you plan to resell it.

**When to choose these:** you want AI to act on your data across several apps, not just chat. Haystack is for building custom AI pipelines; n8n is for connecting apps and automating steps with AI inside them.

## Learn more (reading)

- **Headcount Zero: How to Build an AI-Run Company with Paperclip** — by **Anthony David Adams**. An open-source book (on GitHub, license CC BY-NC-SA 4.0) about running a company where AI agents do most of the work and a small number of humans judge the output. It covers the "one-person company" idea, how AI agents work, the economics of few employees, and how to govern AI with kill-switches. Read it for the bold vision — then apply it with the caution this book teaches: keep humans in charge of what matters.
- **This book's own chapters** — the strongest "further reading" is often the chapters you skimmed. Re-read the risk chapter before any launch, and the data chapter before you connect any tool to real customer data.
- **Vendor and project documentation** — for any tool above, read the project's own docs and license before adopting. It tells you what is free, what is paid, and what you are agreeing to.

## How to choose, in three questions

1. **Where must the data stay?** If it must not leave your machines, look at the local tools (Ollama, Thunderbolt, llama.cpp).

2. **Do you have technical help?** If yes, Haystack and n8n open the most doors. If no, start with a simple hosted tool and a small pilot.
3. **Can you leave later?** Prefer tools that let you export your data and switch models. Avoid anything that locks your data in.

## A caution on lists like this

Tools rise and fall. A name here may change, merge, or fade within a year. That is normal in AI. The **principles** in this book — keep data private, keep a human in the loop, measure results, avoid lock-in — outlast any single tool. Use this list as a map, not a promise.

# Appendix N — Downloadable Modules and Templates

This appendix is a map. It lists every template in the back of this book, with one line on what each is for and when to use it. Copy any of them, print them, or fill them in on screen. They are made to be used, not read once.

**How to use this index:** Find the job you are doing right now, then jump to the matching appendix. Most jobs have one main template and one or two helpers.

## Understand the basics

| Appendix | What it is | Use it when |
|---|---|---|
| **A — Glossary** | Plain-word meanings of AI terms. | You hit a word you do not know. |
| **M — Resources, Reading, and Tools** | Verified tools and further reading. | You want to know what exists and what to read next. |
| **L — Exercises and Fillable Modules** | Short hands-on exercises on every theme. | You want to practice, not just read. |

## Choose a vendor

| Appendix | What it is | Use it when |
|---|---|---|
| **B — Checklist to Evaluate an AI Vendor** | Yes/No checklist across every vendor theme. | First pass on any vendor. |
| **K — Questions to Ask the Vendor** | Pointed questions, each with why it matters. | In a live call or email with a vendor. |
| **P — AI Vendor Due-Diligence Scorecard** | A scorable table with weights and totals. | Comparing two or more vendors side by side. |

## Plan a project

| Appendix | What it is | Use it when |
|---|---|---|
| **C — Process Mapping Template** | Step-by-step map of one process. | Before you automate anything. |
| **D — Data Inventory** | List of what data you have and where it lives. | Before any tool touches your data. |
| **E — Simple ROI Calculation** | The ROI formula, a worked example, a blank sheet. | To decide if a project pays for itself. |
| **I — Pilot Project Template** | A small, time-boxed test plan. | To try a tool before a full rollout. |
| **J — KPI Dashboard** | A one-page board of the numbers to watch. | To track a live project month to month. |

## Govern and protect

| Appendix | What it is | Use it when |
|---|---|---|
| **G — Simple AI Policy** | A short policy for how your firm uses AI. | To set house rules for everyone. |
| **Q — Acceptable Use Policy Template** | A ready-to-adapt staff policy on using AI. | To tell staff what they may and may not do. |
| **H — AI Risk Assessment** | A risk table with likelihood, impact, owners. | Before any launch, and at each change. |
| **O — Quick AI Security Guide for SMBs** | An action-first security checklist. | To lock down the basics now. |

## Communicate inside your firm

| Appendix | What it is | Use it when |
|---|---|---|
| **F — Internal Communication Plan** | How to tell your team about AI changes. | Before rollout, to ease fear and confusion. |

## A simple order to work in

If you are starting from zero, use the templates in this order:

1. **C — Map** the process you want to change.

2. **D — Inventory** the data it touches.
3. **H — Assess** the risks.
4. **E — Compute** the ROI.
5. **B / K / P — Evaluate** the vendor.
6. **I — Pilot** it small.
7. **J — Track** it with KPIs.
8. **G / Q — Govern** it with policy.
9. **O — Secure** it with the security checklist.
10. **F — Communicate** all of it to your team.

You do not need all of them at once. Pick the one that matches the decision in front of you today.

## Tips for using the templates

- **Fill them in with your team**, not alone. The conversation is as valuable as the page.
- **Keep one owner per template.** A template with no owner is ignored.
- **Revisit them.** Risks, vendors, and results change. A filled page from last month can be wrong today.
- **Print or copy freely.** These are yours to use. Do not treat them as read-only.

**Start here:** If you only do one thing, map one process (Appendix C) and assess its risk (Appendix H). That two-step alone puts you ahead of most firms that buy AI on a hunch.

# Appendix O — Quick AI Security Guide for SMBs

A small business using AI faces the same attacks as a big one, with fewer defenses. This is a fast, action-first checklist. Do the top items first. Each one closes a real door. Work down the list as time allows.

**The core idea:** Treat AI tools like a new employee with access to your files. Give the least access needed, watch what they do, and know what to do if something goes wrong.

## 1. Accounts & access

- ☐ Every AI tool login uses a **strong, unique password** (not reused from anywhere else).
- ☐ **Multi-factor authentication (MFA)** is on for every AI account. This is the single best fix.
- ☐ Each person has **their own login** — no shared "team" accounts.
- ☐ Access is **per person and per need** — not everyone sees every tool or every data set.
- ☐ Ex-staff and changed roles have access **removed the same day**.
- ☐ Admin rights are held by **as few people as possible**.
- ☐ You keep a **list of who has access** to what, and review it monthly.

## 2. Data handling

- ☐ You know **what data** each AI tool can see and store.
- ☐ **Personal and sensitive data** (customer details, health, finance, IDs) is never pasted into public AI tools.
- ☐ You have an **approved-tools-only list** and staff know it by heart.

- ☐ Data is **encrypted** both stored and in transit (ask the vendor to confirm).
- ☐ You **minimise** what you feed the AI — only the data the task needs, nothing extra.
- ☐ You can **delete** your data from a tool and get proof it is gone.
- ☐ Backups of your own data exist and are **not** dependent only on the AI vendor.

## 3. Vendor checks

- ☐ You read the vendor's **privacy and security terms** before signing.
- ☐ You confirmed your data is **not used to train** their shared model (or you agreed to it in writing).
- ☐ You know **where** your data is stored (which country/region).
- ☐ The vendor has a **breach-notification promise** — how fast they tell you.
- ☐ You checked the vendor's **reputation** (reviews, references, how long they have operated).
- ☐ You know their **sub-processors** (other companies touching your data).
- ☐ You can **export your data** and leave without a trap.

## 4. Staff & shadow AI

- ☐ Staff know **which tools are approved** and which are banned on company devices.
- ☐ You have talked about **shadow AI** — staff using unapproved tools behind your back.
- ☐ Staff know **never to paste** customer personal data into a public chatbot.

- [ ] Staff know AI output **must be checked** before it goes to a customer or a payment.
- [ ] There is a **simple way to ask** "is this tool okay?" before someone signs up.
- [ ] New staff get a **short AI-safety briefing** on day one.

## 5. Monitoring

- [ ] You keep an **audit log** of who used which AI tool and did what.
- [ ] You **review logs** regularly, not only after a disaster.
- [ ] You set **usage and cost alerts** so a surprise bill does not appear.
- [ ] You **spot-check AI output** on a schedule to catch drift or errors early.
- [ ] You watch for **unusual access** — odd hours, odd locations, bulk downloads.

## 6. Incident basics (when something goes wrong)

- [ ] You have **one named person** in charge if an AI incident happens.
- [ ] You know the **first three moves**: stop the tool, contain the data, tell the owner.
- [ ] You can **cut off access fast** — revoke logins, pause the tool, block the account.
- [ ] You know **who to notify**: the vendor, your data-protection contact, and (if required) the regulator.
- [ ] You keep a **written record** of what happened and what you did.
- [ ] You have a **plain message ready** for customers if their data was involved.
- [ ] After the incident, you **fix the cause**, not just the symptom.

## The 10 items that matter most

If you do nothing else, do these today:

1. Turn on **MFA** everywhere.
2. Ban pasting **personal data** into public AI tools.
3. Make an **approved-tools-only list**.
4. Give **least access** — per person, per need.
5. Remove access for **ex-staff** the same day.
6. Confirm your data is **not training** the vendor's shared model.
7. Keep an **audit log** and look at it.
8. Require **human review** before AI output reaches a customer or payment.
9. Set **cost alerts** so the bill cannot surprise you.
10. Name an **incident owner** and write the first three moves.

## Golden rules

- **Least access always.** Give the minimum needed, nothing more.
- **Trust, but check.** Review logs and outputs; do not assume.
- **Data you never send is data you never leak.** Feed the AI the minimum.
- **A plan you have written is faster than a plan you invent under stress.**

Security is not one product you buy. It is a set of small habits you keep. Start with the ten items above and build from there.

# Appendix P — AI Vendor Due-Diligence Scorecard

Use this to compare vendors with numbers, not gut feel. Each criterion has a **weight** (how much it matters to you). Score each vendor **1–5**. The weighted total gives one number to compare. Fill one scorecard per vendor, then place them side by side.

## How to score

- **Weight** = how important the criterion is. The weights add to 100. Change them to fit your business.
- **Score** = 1 (poor) to 5 (excellent) for how well the vendor meets it.
- **Weighted score** = weight × (score ÷ 5). This keeps the total out of 100.

> **Weighted score = Weight × (Score ÷ 5) Total = sum of all weighted scores (out of 100)**

## Blank scorecard

**Vendor: _____ Date: _____**
**Scored by: _____**

| Criterion | Weight | Score (1–5) | Weighted score | Notes |
|---|---|---|---|---|
| Security | 20 | | | |
| Privacy & compliance | 20 | | | |
| Transparency | 10 | | | |
| Pricing | 15 | | | |
| Support | 15 | | | |
| Exit & lock-in | 15 | | | |
| References & reputation | 5 | | | |
| **Total** | **100** | | | |

## What each criterion means

- **Security** — MFA, access control, audit logs, breach notice, protection against attacks.
- **Privacy & compliance** — GDPR fit, data not used to train shared models, data location, DPA offered.
- **Transparency** — Clear answers, honest about limits, open about sub-processors and model changes.
- **Pricing** — Clear, predictable as you grow, no hidden fees, fair renewal price.
- **Support** — Real channels, written response times, onboarding, uptime promise.
- **Exit & lock-in** — Easy cancel, data export in a usable format, short first contract, you keep ownership.

- **References & reputation** — Reviews or references from firms like yours, long enough operating to be accountable.

## Filled example

**Vendor:** InvoiceFlow AI (example) **Date:** 2026-09-15 **Scored by:** Owner

| Criterion | Weight | Score (1–5) | Weighted score | Notes |
|---|---|---|---|---|
| Security | 20 | 4 | 16 | MFA yes, audit logs yes, breach notice 72 hrs. |
| Privacy & compliance | 20 | 3 | 12 | DPA offered, but data stored outside EU — needs review. |
| Transparency | 10 | 4 | 8 | Clear docs, honest about limits. |
| Pricing | 15 | 3 | 9 | Clear now, but renewal jumps 40%. |
| Support | 15 | 5 | 15 | 24/7 chat, 2-hr response, good onboarding. |
| Exit & lock-in | 15 | 2 | 6 | CSV export yes, but 12-month lock-in with fee. |

**Reading the example:** 70/100 is a decent vendor with two real problems — data location (privacy) and a costly lock-in (exit). The owner should not sign the 12-month term as-is. Fix the exit terms and confirm the data location, or look elsewhere.

## How to read the total

| Total | Meaning |
|---|---|
| 85–100 | Strong candidate. Move to a pilot. |
| 70–84 | Good, but fix the low-scoring items first. |
| 55–69 | Weak. Ask more or look elsewhere. |
| Below 55 | Reject. Too many gaps. |

## The override rule

A high total can hide a fatal flaw. **Ignore the total and walk away if any of these score 1:**

- **Security = 1** — They cannot protect your data.
- **Privacy & compliance = 1** — They will misuse or mishandle your data.
- **Exit & lock-in = 1** — You cannot get your data back or leave.

These are not trade-offs. A great price does not fix a vendor who will not let you leave or who leaks your data.

## Comparing two vendors side by side

Put the totals and the override flags together:

| Criterion | Vendor A | Vendor B |
|---|---|---|
| Total (out of 100) | 70 | 66 |
| Any override flag (Sec/Priv/Exit = 1)? | No | Yes (Exit = 1) |
| Decision | Pilot A | Reject B — cannot exit |

Here Vendor A wins even on a small total gap, because Vendor B fails the override rule.

## Tips

- **Set the weights before you score.** Decide what matters first, so the score is not bent to fit a vendor you like.
- **Score with evidence**, not charm. Use the answers from Appendix K and the checklist in Appendix B.
- **Score every vendor the same way.** Consistency is what makes the totals comparable.
- **Re-score after the trial.** Real use often changes the numbers.

# Appendix Q — Acceptable Use Policy Template

A ready-to-adapt policy for staff who use AI at work. Copy it, fill the blanks, have your lead sign it, and share it with the team. It is short on purpose. A policy nobody reads protects nobody.

> **How to use:** Replace every `[BRACKET]` with your details. Delete what does not apply. Add what your business needs. Have it approved by `[OWNER / MANAGER]` and dated. Ask all staff to confirm they have read it.

---

## Acceptable Use of AI — [Company Name]

**Effective date:** [DATE] **Owner:** [NAME / ROLE] **Review:** every [6 / 12] months

### 1. Purpose

This policy sets how staff at [Company Name] may use AI tools. It protects our customers, our data, and our reputation. It applies to all staff, contractors, and anyone working on our behalf.

### 2. Approved tools only

- Staff may use **only** the AI tools on the **Approved Tools List** (Appendix: [LINK / LOCATION]).
- Using an AI tool not on the list needs **written approval** from [OWNER / ROLE] first.

- If you are unsure whether a tool is approved, **ask before you use it.** Do not guess.

## 3. Permitted uses

You **may** use approved AI tools to:

- Draft, summarise, or rewrite text (emails, reports, notes).
- Brainstorm ideas and options.
- Translate or reformat content.
- Look up general information that is not confidential.
- Automate routine, low-judgment steps that are already mapped and approved.

## 4. Prohibited uses

You **must not** use AI to:

- **Paste customer personal data** (names, addresses, emails, phone numbers, IDs, health, or financial details) into any public AI tool.
- Make a **final decision** that affects a person — hiring, firing, credit, pricing, or legal matters — without a human deciding.
- Produce content that is **illegal, harmful, deceptive, or discriminatory**.
- **Impersonate** a person or hide that AI created the content where honesty matters.
- Bypass any security control, access rule, or this policy.
- Enter **confidential company data** into tools not approved for it.

## 5. Data rules — what never to paste

Never put into any AI tool:

- Customer personal or sensitive data.
- Employee personal or sensitive data.
- Passwords, API keys, tokens, or login details.
- Financial records not already public.
- Legal documents under confidentiality.
- Trade secrets or unreleased business plans.

**Rule of thumb:** If you would not put it on a public noticeboard, do not paste it into a public AI tool.

## 6. Disclosure

- When AI output goes to a customer or the public and honesty matters, follow our disclosure rule: [STATE RULE — e.g., "note that AI assisted in drafting"].
- Never present AI work as fully human-made when that would mislead.
- Never use AI to create fake reviews, fake testimonials, or false claims.

## 7. Human review (the golden rule)

- **Every AI output must be checked by a human before it leaves the company** — to a customer, a payment, a filing, or the public.
- AI can be wrong with total confidence. You are responsible for what you send.
- For anything with money, legal, or safety weight, a **named person must approve** before it goes out.
- If you cannot check whether the AI is right, do not use its answer.

## 8. Accountability

- **You are responsible** for what you do with AI, just as you are for any other tool.
- "The AI did it" is not a defence. You chose to use it and to send its output.
- Keep any required records of AI use where asked.

## 9. Security

- Use your **own login** with **multi-factor authentication** on every AI tool.
- Never share logins or put company data into personal AI accounts.
- Report any suspected leak, wrong output, or odd behaviour **immediately** to [INCIDENT CONTACT].

## 10. Training

- All staff must complete the **AI-safety briefing** before using AI for work.
- New staff complete it in their first week.
- Refresher training runs every [6 / 12] months.

## 11. Exceptions

- Any exception to this policy needs **written approval** from [OWNER / ROLE].
- Exceptions are logged and reviewed at the next policy review.

## 12. Enforcement

- Breaking this policy may lead to: a warning, loss of AI-tool access, retraining, or disciplinary action, depending on severity.
- Serious or repeated breaches — especially leaking customer data — may end employment or contract.
- We aim to **teach before we punish.** If you are unsure, ask. Asking is never punished.

## 13. Acknowledgement

I confirm I have read and understood this policy. I agree to follow it.

| Name | Role | Signature | Date |
|---|---|---|---|
| | | | |
| | | | |

---

**Final note to the owner:** A policy is only as strong as the day it is enforced. Talk through it with the team, keep the Approved Tools List current, and review it when a tool, a law, or an incident changes the picture. Pair this with the security checklist in Appendix O and the risk assessment in Appendix H.